

O'REILLY®

Head First

Архитектура ПО

Руководство
для обучающихся
архитектурному мышлению

Раджу Ганди
Марк Ричардс
Нил Форд



Подарок для мозга

SPRINT
book

Head First Software Architecture

Wouldn't it be dreamy if there was
a book about software architecture
that was more fun than getting a root
canal and more revealing than reams
of documentation?
It's probably just a fantasy...



Raju Gandhi

Mark Richards

Neal Ford

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Head First Архитектура ПО

Как было бы здорово иметь книгу
об архитектуре программного обеспече-
ния, более увлекательную, чем поход
к стоматологу, и более информа-
тивную, чем стопка документации!
Жаль, что это всего лишь мечты...

Раджу Ганди

Марк Ричардс

Нил Форд



SPRINT
book

2025

ББК 32.973.2-018-02
УДК 004.415
Г19

Раджу Ганди, Марк Ричардс, Нил Форд

Г19 Head First. Архитектура ПО. — Астана: «Спринт Бук», 2025. — 480 с.: ил.

ISBN 978-601-08-4838-2

Плох тот разработчик, который не мечтает стать архитектором! Вам тоже хочется пройти этот путь как можно быстрее?

Авторы бестселлера «Фундаментальный подход к программной архитектуре» научат вас мыслить «архитектурно» и расскажут про актуальные проблемы архитектуры ПО. Вы узнаете о различиях между архитектурой и дизайном и о взаимосвязи между кодом, компонентами и архитектурными стилями. На ярких, запоминающихся и реальных примерах вы научитесь работать с самыми распространенными архитектурными стилями.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018-02
УДК 004.415

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

ISBN 978-1098134358 англ.

Authorized Russian translation of the English edition of Head First Software Architecture ISBN 978-1098134358 © 2024 Raju Gandhi, Mark Richards, Neal Ford.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-601-08-4838-2

© Перевод на русский язык ТОО «Спринт Бук», 2025

© Издание на русском языке, оформление ТОО «Спринт Бук», 2025

ПОСВЯЩЕНИЯ

От Марка и Нила

Всем, кто с 2004 по 2024 терпеливо читал
множество черновиков этой книги.

От Раджу

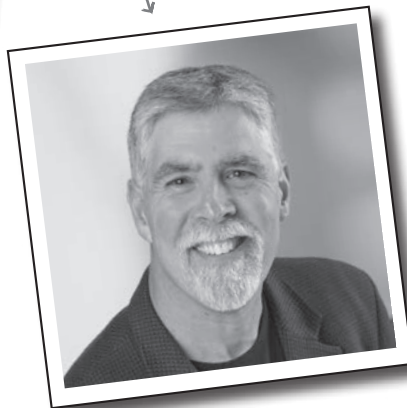
Моему отцу

ॐ नमो भगवते वासुदेवाय

Об авторах



Раджу Ганди



Марк Ричардс



Нил Форд

Раджу Ганди

Раджу — архитектор, консультант, автор книг и преподаватель. Регулярно участвует в конференциях по всему миру. Любит простоту и всегда старается понять и объяснить «почему», а не «как». Живет в Колумбусе, штат Огайо, со своей замечательной женой Мишель, сыновьями Мейсоном и Микой, дочерью Дельфиной и тремя четвероногими членами семьи: Бадди, Скаем и Принцессой Зарой. Его контактную информацию можно найти на сайте RajuGandhi.com.

Марк Ричардс

Марк — опытный архитектор и основатель DeveloperToArchitect.com — сайта, бесплатно помогающего начинающим архитекторам. В индустрии программного обеспечения с 1983 года. Имеет большой опыт и является экспертом в области разработки и интеграции приложений и корпоративной архитектуры. Автор множества технических книг и видеороликов, в том числе *Fundamentals of Software Architecture* и *Software Architecture: The Hard Parts* (O'Reilly), написанных в соавторстве с Нилом Фордом. Кроме того, занимается преподавательской деятельностью, выступил на сотнях конференций и встречах групп пользователей по всему миру.

Нил Форд

Нил Форд — директор, архитектор и идейный вдохновитель ThoughtWorks, консалтинговой компании, специализирующейся на комплексной разработке и доставке программного обеспечения. Участвовал в проектировании и разработке множества приложений, написании большого количества статей и книг, а также в создании видеопрезентаций. В настоящее время занимается проектированием крупномасштабных корпоративных приложений. Широко известен как спикер, выступивший более чем на 300 конференциях по всему миру и представивший более 2000 презентаций.

Содержание (сводка)

	Введение	21
1	Архитектура ПО без мистики: приступим!	33
2	Архитектурные характеристики: <i>знайте свои возможности</i>	73
3	Два закона архитектуры ПО: <i>все основано на компромиссах</i>	113
4	Логические компоненты: <i>строительные блоки</i>	155
5	Архитектурные стили: <i>классификация и философия</i>	199
6	Многоуровневая архитектура: <i>разделение обязанностей</i>	217
7	Модульные монолиты: <i>управляются предметной областью</i>	247
8	Микроядерная архитектура: <i>индивидуальная подгонка</i>	279
9	Сделай сам: <i>приложение для путешественников TrifEZ</i>	309
10	Микросервисная архитектура: <i>шаг за шагом</i>	335
11	Событийно-управляемая архитектура: <i>асинхронные приключения</i>	381
12	Сделай сам: <i>проверка знаний</i>	435
	Приложение: <i>шесть главных незатронутых тем</i>	499

Содержание (настоящее)

Введение

Архитектура ПО — сложный предмет, и ваш мозг будет убеждать вас, что его нельзя освоить. Он будет думать: «Лучше сосредоточиться на чем-то более важном, например, что съесть на обед и есть ли у свиней крылья». Но вы СМОЖЕТЕ убедить его, что архитектура ПО — важный предмет, и в данной главе мы поможем вам сделать это.

Кому адресована эта книга	22
Мы знаем, о чем вы думаете	23
Мы знаем, о чем думает ваш <i>мозг</i>	23
Метапознание: как мы мыслим	25
Вот что сделали МЫ	26
Что можете сделать ВЫ, чтобы заставить ваш мозг повиноваться	27
Важная информация	28
Главы «Сделай сам»	29
Рецензенты	30
Совместные благодарности	31
Индивидуальные благодарности	32

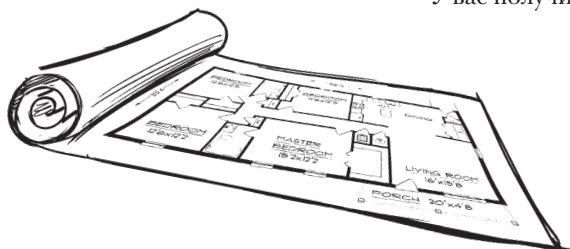
1 Архитектура ПО без мистики

Приступим!

Архитектура программного обеспечения лежит в основе успешной работы системы.

Эта глава срывает покров таинственности с архитектуры ПО и рассказывает об архитектурных измерениях и различиях между архитектурой и дизайном. Об этом важно знать, поскольку понимая архитектурные практики и применяя их, вы сможете создавать эффективные системы, которые не просто лучше функционируют, но и отвечают нуждам и чаяниям бизнеса и продолжают работать, когда техническая и бизнес-среда претерпевают постоянные изменения.

Основы архитектуры программного обеспечения	34
План строительства и архитектура ПО	35
Измерения в архитектуре ПО	36
Мозаика из измерений	37
Измерение первое: архитектурные характеристики	38
Измерение второе: архитектурные решения	40
Измерение третье: логические компоненты	42
Измерение четвертое: архитектурные стили	44
С точки зрения дизайна	48
С точки зрения архитектуры	49
Спектр между архитектурой и дизайном	50
В какую часть спектра попадает ваше решение?	51
Стратегические и тактические решения	52
Большие и малые усилия	54
Значимые и незначимые компромиссы	56
Собираем все воедино	58
У вас получилось!	59

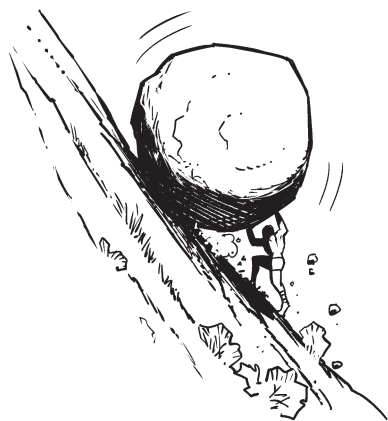


2

Архитектурные характеристики

Найдите свои возможности

Что должна поддерживать архитектура? Архитектурные характеристики (возможности архитектуры) — фундаментальные строительные блоки любой системы. Без них невозможно принимать архитектурные решения, выбирать архитектурные стили и вообще создавать логичные архитектуры. В этой главе вы узнаете, как определить наиболее распространенные характеристики (масштабируемость, надежность, тестируемость), как они влияют на архитектуру ПО, как помогают принимать архитектурные решения и какие из них наиболее важны в конкретной ситуации. Готовы добавить возможности в архитектуру своего ПО?



Причины создания социальной сети Lafter	74
Разговор в кабинете	75
Что такое архитектурные характеристики	76
Определение архитектурных характеристик	77
Характеристики не связаны с предметной областью	78
Характеристики влияют на структуру архитектуры	79
Избегайте чрезмерного усложнения	80
Учитывайте явные и неявные возможности	82
Международный зоопарк характеристик	83
Технологические архитектурные характеристики	84
Структурные архитектурные характеристики	85
Эксплуатационные архитектурные характеристики	86
Сквозные архитектурные характеристики	87
Получение архитектурных характеристик из предметной области	92
Получение архитектурных характеристик из знаний об окружении	93
Получение архитектурных характеристик из предметных знаний	93
Составные архитектурные характеристики	95
Приоритеты определяются контекстом	96
Потери при переводе	98
Архитектурные характеристики и логические компоненты	100
Сбалансированность архитектурных характеристик	101
Сокращение количества архитектурных характеристик	102

3

Два закона архитектуры ПО

Все основано на компромиссах

Что происходит, когда нет «лучших практик»? Лучшие практики — это относительно безопасные способы достижения определенных целей. Они называются «лучшими» (а не «хорошими») не просто так — они работают. Так почему бы просто не использовать их? Но в архитектуре ПО нет лучших практик. Вам придется тщательно анализировать каждую ситуацию, чтобы принять решение, и объяснять не только «что» нужно сделать, но и «почему».

Как преодолеть этот новый рубеж? К счастью, есть законы архитектуры ПО, которые вам помогут. В данной главе вы узнаете, как анализировать компромиссы и создавать записи архитектурных решений, чтобы фиксировать ответы на вопросы «как» и «почему». К концу главы вы освоите инструменты для навигации по изменчивой территории архитектуры ПО.



Приложение для фанатов кроссовок	114
Что мы уже знаем?	116
Взаимодействие с другими сервисами	118
Анализ компромиссов	119
Анализ компромиссов: версия с очередями	120
Анализ компромиссов: версия с темами	121
Первый закон архитектуры ПО	122
Компромиссы неизбежны	124
Принятие архитектурного решения	125
Что еще делает решение архитектурным?	126
Второй закон архитектуры ПО	128
Записи об архитектурных решениях (ADR)	129
Разговор в кабинете	130
Создание ADR: выбор хорошего заголовка	131
Создание ADR: определение статуса	132
Создание ADR: определение контекста	136
Создание ADR: описание решения	137
Создание ADR: оценка последствий	139
Создание ADR: управление выполнением	141
Создание ADR: примечания	141
Преимущества ADR	144
Two Many Sneakers празднует успех	145

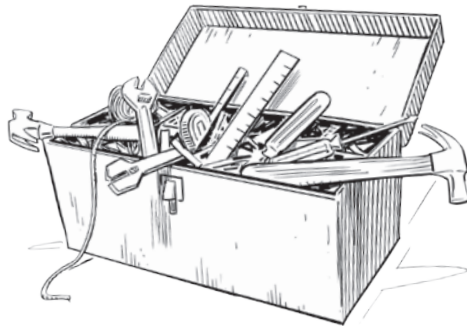
4

Логические компоненты

Строительные блоки

Готовы приступить к созданию архитектуры? Это не так просто, как кажется, и если сделать что-то неправильно, то программная система может обрушиться, как плохо спроектированный небоскреб или мост.

В данной главе показаны несколько приемов определения и создания *логических компонентов* — функциональных строительных блоков системы, которые описывают, как ее части соединяются друг с другом. Эти приемы помогут вам создать надежную архитектуру — фундамент эффективной программной системы.



Еще раз о логических компонентах	156
Запускаем Adventurous Auctions	158
Логическая и физическая архитектура	159
Создание логической архитектуры	161
Шаг 1: выявление основных компонентов	162
Подход на основе рабочих процессов	164
Подход на основе субъектов и действий	166
Ловушка для сущностей	168
Шаг 2: назначение требований компонентам	170
Шаг 3: анализ ролей и обязанностей	172
Стремиться к связности	173
Шаг 4: анализ архитектурных характеристик	174
Компонент «Прием ставок»	176
Связывание компонентов	177
Афферентные связи	178
Эфферентная связь	179
Оценка связанности	180
Сильносвязанная система	182
Применение закона Деметры	183
Балансирование	185
Несколько заключительных слов о компонентах	186

5

Архитектурные стили

Классификация и философия

Существует множество разных архитектурных стилей.

Каждый из них имеет свои причины создания и принципы использования. Понимание философии стиля поможет вам решить, подходит ли он для вашей области. В этой главе мы коротко поговорим о различных типах архитектурных стилей (рассматриваемых далее в книге), что поможет вам ориентироваться в них, а также в других стилях, с которыми вы рано или поздно столкнетесь в своей профессиональной деятельности.

Существует множество архитектурных стилей	200
Мир архитектурных стилей	201
Организация кода: техническая и предметная	202
Модель развертывания: монолитная и распределенная	204
Монолитные модели развертывания: плюсы	206
Монолитные модели: минусы	207
Распределенные модели развертывания: плюсы	208
Распределенные модели: минусы	209
Вот и все!	212



6

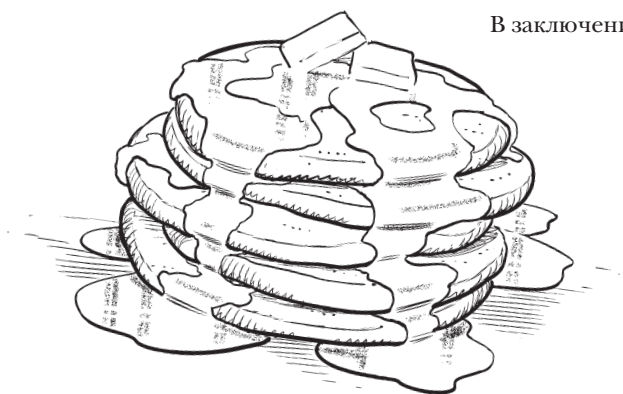
Многоуровневая архитектура

Разделение обязанностей

Как поступить, если задача проста и время поджимает?

Стоит ли беспокоиться об архитектуре? Все зависит от того, как долго должно просуществовать то, что вы создали. Если решение одноразовое, то отбросьте осторожность. Если нет, то выберите самую простую архитектуру, которая обеспечивает некий уровень организации и позволяет получить выгоду, при этом существенно не влияя на скорость разработки. Например, это может быть *многоуровневая архитектура*, поскольку она проста и легко реализуется с помощью известных паттернов проектирования.

Naan & Pop: сбор требований	218
Назад к паттернам проектирования	220
Многоуровневый паттерн MVC	221
Организация уровней	224
Преобразование уровней в код	225
Предметные области, компоненты и уровни	227
Достоинства многоуровневой архитектуры	230
Уровни и реальный мир: физические архитектуры	231
Компромиссы физической архитектуры	232
Предостережение об изменениях в предметной области	235
Сверхвозможности многоуровневой архитектуры	236
Слабые стороны многоуровневой архитектуры	237
Звездные рейтинги многоуровневой архитектуры	238
В заключение	240



7

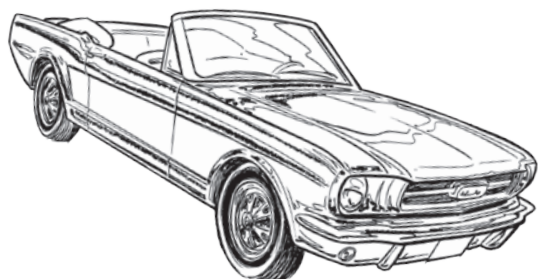
Модульные Монолиты

Управляются предметной областью

Создать монолит можно несколькими способами. Вы уже познакомились с многоуровневой архитектурой, увязывающей компоненты *технически*. Можно долго развивать многоуровневый монолит, но когда возникает необходимость координировать взаимодействие разных команд, то может потребоваться нечто большее, возможно, даже другой архитектурный стиль.

В этой главе описывается архитектурный стиль *модульный монолит*, делящий приложения по *бизнес-задам*, а не по техническим признакам. Вы узнаете, что это значит и на что следует обратить внимание. Кроме того, мы поговорим обо всех компромиссах этого стиля.

Модульный монолит	251
Изменения в предметной области <i>болезненны</i>	253
Что такое модульные монолиты	254
Покажи мне код!	256
Как сохранить модульность модулей	259
Модульность вплоть до базы данных	263
Остерегайтесь соединений	265
Сверхвозможности модульного монолита	266
Слабые стороны модульного монолита	267
Звездные рейтинги модульного монолита	268
Naan & Pop доставляет пиццу!	270



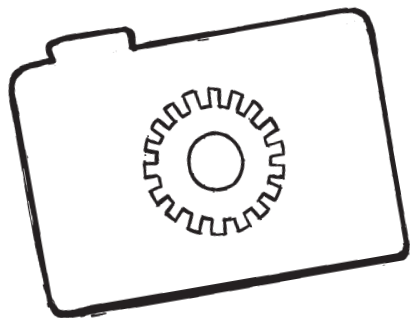
8

Микроядерная архитектура

Индивидуальная подгонка

Новые возможности можно создавать по одной. Есть цели, которые особенно хорошо достигаются с помощью определенных архитектурных стилей. Микроядерная архитектура — чемпион по индивидуальной подгонке. Она хорошо подходит для огромного количества приложений. Поняв идею этого стиля, вы начнете видеть его повсюду!

В данной главе вы познакомитесь с архитектурой, позволяющей пользователям делать все по *своему* усмотрению.



Преимущества Going Green	280
Две части микроядерных архитектур	283
Спектр «микроядерности»	285
Ядро сервиса оценки устройств	287
Инкапсулированные и распределенные плагины	289
Взаимодействие с плагинами	291
Контракты плагинов	296
Going Green включается в работу	297
Сверхвозможности микроядерной архитектуры	298
Слабые стороны микроядерной архитектуры	299
Звездные рейтинги микроядерной архитектуры	300
В заключение	302

9

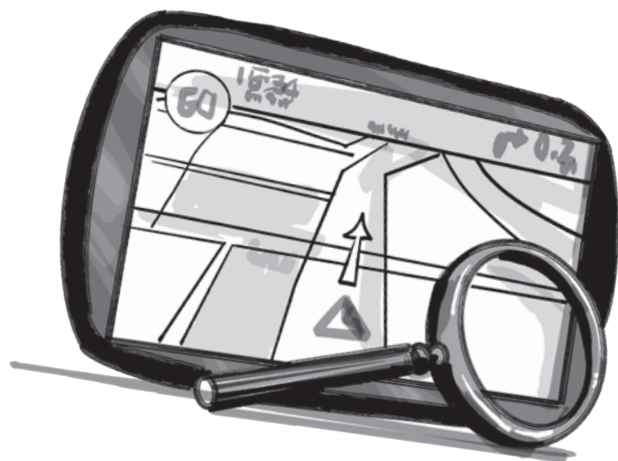
Сделай сам

Приложение для путешественников TripEZ

Готовы продолжить знакомство с архитектурой ПО?

В этой главе вы будете играть роль архитектора программного обеспечения. Вам предстоит определять архитектурные характеристики, создавать архитектуру, принимать архитектурные решения и выбирать между многоуровневой, модульной и микроядерной архитектурами. Упражнения в этой главе помогут вам лучше понять, что делает архитектор ПО, и покажут, как много вы узнали. Ваша задача — создать архитектуру для стартапа, разрабатывающего сайт для путешественников.

Облегчаем возможность путешествий	310
Путь пользователя через TripEZ's	311
Планирование архитектуры	312
План действий архитектора	313
Шаг 1: определение архитектурных характеристик	314
Шаг 2: определение логических компонентов	316
Шаг 3: выбор архитектурного стиля	318
Шаг 4: документирование решения	320
Шаг 5: составление схемы архитектуры	322
Нет правильных (или неправильных) ответов!	324

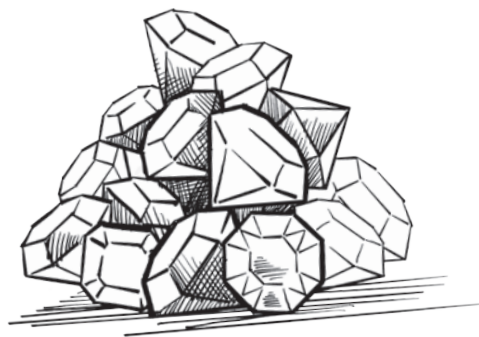


10

Микросервисная архитектура

Шаг за шагом

Как упростить изменение архитектуры? Потребности бизнеса меняются быстро, и архитектуры ПО должны соответствовать им. В этой главе вы узнаете, как создать гибкую архитектуру, которая может меняться вслед за изменениями требований бизнеса, масштабироваться по мере его роста и сохранять работоспособность даже после системных сбоев. Заинтригованы? В данной главе вы познакомитесь с *микросервисами* — архитектурным стилем, решающим все эти и многие другие проблемы.



Хорошо ли вы себя чувствуете?	336
Что такое микросервисы	339
Эти данные только мои	340
Насколько малы эти «микро»	342
Разукрупнение	344
Почему желательно уменьшать микросервисы	345
Укрупнение	346
Почему желательно укрупнять микросервисы	347
Все дело в балансе	348
Общая функциональность	351
Совместное использование кода через общий сервис	352
Совместное использование кода через общую библиотеку	353
Управление рабочими процессами	357
Оркестрация: дирижирование микросервисами	358
Хореография: станцуем	360
Сверхвозможности микросервисной архитектуры	364
Слабые стороны микросервисной архитектуры	365
Звездные рейтинги микросервисной архитектуры	366
В заключение	368

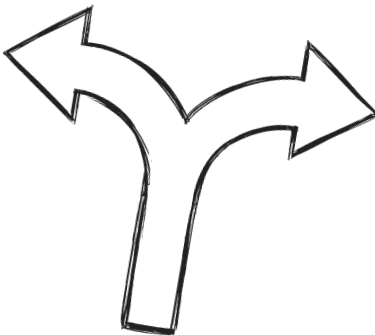
11

Событийно-управляемая архитектура

Асинхронные приключения

Может ли архитектура делать много дел одновременно?

Чтобы развиваться, бизнес должен иметь возможность обслуживать все больше пользователей, без задержек и сбоев. В этой главе вы узнаете, как создавать высокопроизводительные системы на основе *событийно-управляемой архитектуры* — популярной разновидности распределенных архитектур, очень быстрых, легко масштабируемых и расширяемых, но при этом довольно сложных. Вы познакомитесь с такими новыми понятиями, как события, сообщения и асинхронная связь. Это поможет вам создать архитектуру, способную решать множество задач одновременно.



Слишком медленно	382
Ускоряемся	383
Der Nile развивается быстрее, чем прежде	384
Что такое событие	386
События и сообщения	388
Иницирующие и производные события	390
Меня кто-нибудь слышит?	392
Асинхронная связь	393
Запустил и забыл	395
Асинхронность ради победы	397
Синхронность ради победы	399
Топологии баз данных	401
Монолитная база данных	402
Базы данных с разделением на предметные области	404
Отдельная база данных для каждого сервиса	406
Сравнение событийно-управляемой и микросервисной архитектур	410
Гибриды: событийно-управляемые микросервисы	414
Сверхвозможности событийно-управляемой архитектуры	416
Слабые стороны событийно-управляемой архитектуры	417
Звездные рейтинги событийно-управляемой архитектуры	418
Собираем все воедино	420
В заключение	421

12

Сделай сам

Проверка знаний

Готовы проверить свои навыки создания распределенной архитектуры? В этой главе вам предстоит снова побывать в роли архитектора программного обеспечения. Вы будете определять архитектурные характеристики, принимать архитектурные решения, создавать логическую архитектуру и выбирать, какой подход использовать: микросервисный или событийно-управляемый. Упражнения в этой главе помогут вам лучше понять, что делает архитектор ПО, и покажут, как много вы узнали. Вам предстоит создать архитектуру для системы тестирования учащихся Make the Grade. Удачи! Мы надеемся, что вы получите пятерку!

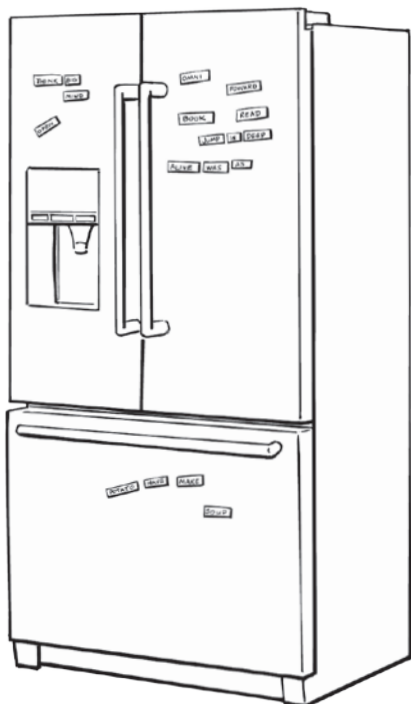


Добро пожаловать в Make the Grade	436
Процесс тестирования учащихся	437
Планирование архитектуры	438
План действий архитектора	439
Шаг 1: определение архитектурных характеристик	440
Шаг 2: определение логических компонентов	442
Шаг 3: выбор архитектурного стиля	444
Шаг 4: документирование решения	446
Шаг 5: составление схемы архитектуры	448
Нет правильных (или неправильных) ответов!	450

Приложение

Шесть главных незатронутых тем

Мы рассказали об архитектуре программного обеспечения далеко не всё. Прочитав эту книгу, вы сделали всего лишь первый шаг к обретению архитектурного мышления, и мы не можем отпустить вас, не обеспечив дополнительными ресурсами. Поэтому в данном приложении мы кратко опишем еще несколько интересных тем, заслуживающих такого же внимания, как и те, которые были представлены в главах.



1. Программирующий архитектор	460
2. Обязанности архитектора	462
3. Нетехнические навыки архитектора	464
4. Приемы построения схем	466
5. Глубина и широта знаний	468
6. Архитектурные каты	470

Как работать с этой книгой

Введение



Здесь мы ответим на важный вопрос: «Почему они включили ТАКОЕ в книгу по архитектуре программного обеспечения».

Кому адресована эта книга

Если ответите «да» на оба вопроса:

- 1 Хотите узнать, что такое архитектура программного обеспечения?
- 2 Предпочитаете **оживленную беседу**, а не **сухие и скучные академические лекции**?

тогда эта книга адресована вам.

Кому стоит воздержаться от чтения этой книги

Если ответите «да» на любой из вопросов:

- 1 Вы совершенно не знакомы с индустрией ПО?

(Мы твердо верим, что разработчики должны понимать основы архитектуры ПО, и, возможно, вам стоит получить некоторый опыт разработки, прежде чем приступить к чтению этой книги.)

- 2 Вы опытный архитектор программного обеспечения и ищете **справочник**?
- 3 **Бойтесь пробовать что-то новое?** Вам спокойнее сидеть на месте, чем продвигать свою карьеру? Вы не верите, что техническая книга может содержать серьезные знания, если архитектурные характеристики, такие как масштабируемость и отказоустойчивость, объясняются в ней на примерах с животными?

Тогда эта книга **не** для вас.

[Замечание от отдела продаж: вообще-то эта книга для любого, у кого есть деньги... И если что, система быстрых платежей тоже подойдет.]



Мы знаем, о чем вы думаете

«Как *такая* книга может быть серьезным источником информации об архитектуре ПО?»

«К чему все эти иллюстрации?»

«Как можно *учиться* при таком подходе?»

Мы знаем, о чем думает ваш мозг

Ваш мозг *жаждет* новизны. Он всегда ищет что-то необычное. Он так устроен, и это помогает вам выживать.

А как ваш мозг поступает с чем-то обычным, повседневным? Он делает *все возможное*, чтобы рутина не мешала его *реальной* работе — запоминать что-то *важное*. Он не утруждает себя запоминанием чего-то скучного, что не может преодолеть фильтр «это явно не важно».

Как мозг *узнает*, что важно? Представьте: вы прогуливаетесь, и из кустов выпрыгивает тигр. Что происходит у вас в голове и теле?

Активизируются нейроны. Эмоции зашкаливают. *Адреналин выбрасывается в кровь*.

Так ваш мозг узнает, что...

Это важно! Это нужно запомнить!

А теперь представьте, что вы дома или в библиотеке. Здесь тепло, безопасно и нет тигров. Вы готовитесь к экзамену. Или пытаетесь освоить сложную тему, на что, по мнению вашего босса, достаточно недели или, в крайнем случае, десяти дней.

Но есть проблема. Ваш мозг пытается оказать вам услугу — убедить вас, что эта тема *явно* не важна и не стоит тратить на нее скудные ресурсы, которые пригодятся для запоминания действительно *важной* информации. Например, о том, как спастись от тигра или пожара или что не стоило выкладывать в ВК фотографии с вечеринки. Вы не можете просто сказать своему мозгу: «Эй, дружище, спасибо тебе, конечно. Но какой бы скучной ни была эта книга и как бы мало эмоций я ни испытывал от нее, я *действительно* хочу, чтобы ты запомнил эти сведения».

Ваш мозг думает,
что **ЭТО** важно.



Ваш мозг думает,
что **ЭТО** не стоит
запоминать.

Отлично.
Еще 450 скучных
сухих страниц.



Эта книга для тех, кто хочет учиться.

Как мы что-то узнаем? Сначала нужно это «что-то» понять, а потом *не забыть*. Затолкать в голову побольше фактов недостаточно. Согласно новейшим исследованиям в области когнитивистики, нейробиологии и психологии обучения, для усвоения материала требуется что-то большее, чем простой текст на странице. Мы знаем, как заставить ваш мозг работать.

Основные принципы серии «Head First»:

Наглядность. Графика запоминается лучше, чем обычный текст, и значительно повышает эффективность восприятия информации (до 89 %, по данным исследований). Кроме того, материал становится более понятным. Текст размещается на рисунках, к которым он относится, а не под ними или на соседней странице — и вероятность успешного решения задач, относящихся к материалу, повышается *вдвое*.

Разговорный стиль изложения. Недавние исследования показали, что при разговорном стиле изложения материала (вместо формальных лекций) улучшение результатов на итоговом тестировании достигает 40 %. Рассказывайте историю, вместо того чтобы читать лекцию. Не относитесь к себе слишком серьезно. Что привлечет ваше внимание: занимательная беседа за столом или лекция?

Активное участие читателя. Пока вы не начнете напрягать извилины, в вашей голове ничего не произойдет. Читатель должен быть заинтересован в результате; он должен решать задачи, формулировать выводы и овладевать новыми знаниями. А для этого необходимы упражнения и каверзные вопросы, в решении которых задействованы оба полушария мозга и разные чувства.

Привлечение (и сохранение) внимания читателя. Ситуация, знакомая каждому: «Я очень хочу изучить это, но засыпаю на первой странице». Мозг обращает внимание на интересное, странное, притягательное, неожиданное. Изучение сложной технической темы не обязано быть скучным. Интересное узнается намного быстрее.

Обращение к эмоциям. Известно, что наша способность запоминать в значительной мере зависит от эмоционального сопереживания. Мы запоминаем то, что нам безразлично. Мы запоминаем, когда что-то *чувствуем*. Нет, сантименты здесь ни при чем: речь идет о таких эмоциях, как удивление, любопытство, интерес и чувство «Да я крут!» при решении задачи, которую окружающие считают сложной, — или когда вы понимаете, что разбираетесь в теме *лучше*, чем всезнайка Боб из технического отдела.

Метапознание: как мы мыслим

Если вы действительно хотите быстрее и глубже усваивать новые знания, задумайтесь над тем, как вы думаете. Учитесь учиться.

Мало кто из нас изучает теорию метапознания во время учебы. Нам *положено* учиться, но нас редко этому *учат*.

Но раз вы читаете эту книгу, то вероятно, хотите узнать об архитектуре ПО, и по возможности быстрее. Вы хотите *запомнить* прочитанное, а для этого абсолютно необходимо сначала *понять* прочитанное.

Чтобы извлечь максимум пользы из учебного процесса, нужно заставить ваш мозг воспринимать новый материал как нечто важное. Критичное для вашего существования. Такое же важное, как тигр. Иначе вам предстоит бесконечная борьба с вашим мозгом, который всеми силами уклоняется от запоминания новой информации.

Как же ЗАСТАВИТЬ мозг относиться к архитектуре ПО как к голодному тигру?

Есть способ медленный и скучный, а есть быстрый и эффективный. Первый основан на тупом повторении. Всем известно, что даже самую скучную информацию *можно* запомнить, если повторять ее снова и снова. При достаточном количестве повторений ваш мозг прикидывает: «Вроде бы несущественно, но раз одно и то же повторяется *столько* раз... Ладно, уговорил».

Быстрый способ основан на *повышении активности мозга* и особенно на сочетании разных ее *видов*. Доказано, что все факторы, перечисленные на предыдущей странице, помогают вашему мозгу работать на вас. Например, исследования показали, что размещение слов *внутри* рисунков (а не в подписях, в основном тексте и т. д.) заставляет мозг анализировать связи между текстом и графикой, а это приводит к активизации большего количества нейронов. Больше нейронов — выше вероятность того, что информация будет *сочтена* важной и достойной запоминания.

Разговорный стиль тоже важен: обычно люди проявляют больше внимания, когда они участвуют в разговоре, так как им приходится следить за ходом беседы и высказывать свое мнение. Причем мозг совершенно *не интересуется*, что вы «разговариваете» с книгой! С другой стороны, если текст сух и формален, то мозг чувствует то же, что чувствуете вы на скучной лекции в роли пассивного участника. Его клонит в сон.

Но рисунки и разговорный стиль — это только начало.



Вот что сделали МЫ

Мы использовали *рисунки*, потому что мозг лучше приспособлен для восприятия графики, чем текста. С точки зрения мозга рисунок стоит тысячи слов. А когда текст комбинируется с графикой, мы внедряем текст *прямо в рисунки*, потому что мозг при этом работает эффективнее.

Мы используем *избыточность*: повторяем одно и то же несколько раз, применяя *разные* средства передачи информации, обращаемся к *разным чувствам* — и все для повышения вероятности того, что материал будет закодирован в нескольких областях вашего мозга.

Мы используем концепции и рисунки несколько *неожиданным* образом, потому что мозг лучше воспринимает новую информацию. Кроме того, рисунки и идеи обычно имеют *эмоциональное содержание*, потому что мозг обращает внимание на биохимию эмоций. То, что заставляет нас *чувствовать*, лучше запоминается, будь то *шутка*, *удивление* или *интерес*.

Мы используем *разговорный стиль*, потому что мозг лучше воспринимает информацию, когда вы участвуете в разговоре, а не пассивно слушаете лекцию. Это происходит и при *чтении*.

В книгу включены многочисленные *упражнения*, потому что мозг лучше запоминает, когда вы что-то *делаете*. Мы постарались сделать их непростыми, но интересными — то, что предпочитает большинство читателей.

Мы совместили *несколько стилей обучения*, потому что одни читатели предпочитают пошаговые описания, другие стремятся сначала представить «общую картину», а третьим хватает фрагмента кода. Независимо от ваших личных предпочтений, полезно видеть несколько вариантов представления одного материала.

Мы постарались задействовать *оба полушария вашего мозга*; это повышает вероятность усвоения материала. Пока одно полушарие мозга работает, другое часто имеет возможность отдохнуть; это повышает эффективность обучения в течение продолжительного времени.

А еще в книгу включены *истории* и упражнения, отражающие другие точки зрения. Мозг глубже усваивает информацию, когда ему приходится оценивать и выносить суждения.

В книге часто встречаются *вопросы*, на которые не всегда можно дать простой ответ, потому что мозг быстрее учится и запоминает, когда ему приходится что-то делать. Невозможно накачать *мышцы*, наблюдая за тем, как занимаются *другие*. Однако мы позаботились о том, чтобы усилия читателей были приложены в *верном* направлении. Вам не придется ломать голову над невразумительными примерами или разбираться в сложном, перенасыщенном техническим жаргоном или слишком лаконичном тексте.

В историях, примерах, на картинках используются образы *людей*, потому что вы тоже *человек* и ваш мозг обращает на людей больше внимания, чем на неодушевленные *предметы*.



Что можете сделать Вы, чтобы заставить ваш мозг повиноваться

Мы свое дело сделали. Остальное за вами. Эти советы станут отправной точкой; прислушайтесь к своему мозгу и определите, что вам подходит, а что не подходит. Пробуйте новое.

Вырежьте и прикрепите на холодильник.

1 Не торопитесь. Чем больше вы поймете, тем меньше придется запоминать.

Просто читать недостаточно. Когда книга задает вам вопрос, не переходите к ответу. Представьте, что кто-то *действительно* задает вам вопрос. Чем глубже ваш мозг будет мыслить, тем скорее вы поймете и запомните материал.

2 Выполняйте упражнения, делайте заметки.

Мы включили упражнения в книгу, но выполнять их за вас не собираемся. И не *разглядывайте* упражнения. **Берите карандаш и пишите.** Физические действия *во время* обучения повышают его эффективность.

3 Читайте врезки.

Это значит: читайте всё. **Врезки — часть основного материала!** Не пропускайте их.

4 Не читайте другие книги после этой перед сном.

Часть обучения (особенно перенос информации в долгосрочную память) происходит после того, как вы откладываете книгу. Ваш мозг не сразу усваивает информацию. Если во время обработки поступит новая информация, часть того, что вы узнали ранее, может быть потеряна.

5 Проговаривайте прочитанное. Вслух.

Речь активизирует другие участки мозга. Если вы пытаетесь что-то понять или лучше запомнить, произнесите вслух. А еще лучше — попробуйте объяснить кому-нибудь другому. Вы будете быстрее усваивать материал и, возможно, откроете для себя что-то новое.

6 Пейте воду. И побольше.

Мозгу нужна вода, так он лучше работает. Дегидратация (которая может наступить еще до того, как вы почувствуете жажду) снижает когнитивные функции.

7 Прислушивайтесь к своему мозгу.

Следите за тем, когда ваш мозг начинает устывать. Если вы поверхностно воспринимаете материал или забываете только что прочитанное, пора сделать перерыв.

8 Чувствуйте!

Ваш мозг должен знать, что материал книги действительно *важен*. Переживайте за героев наших историй. Придумывайте собственные подписи к фотографиям. Поморщиться над неудачной шуткой все равно лучше, чем не почувствовать ничего.

9 Практикуйтесь каждый день!

Есть только один способ научиться *по-настоящему* понимать архитектуру ПО: **иметь с ней дело каждый день.** В данной книге вы будете делать это постоянно, и как с любым другим навыком, единственный способ добиться хороших результатов — практиковаться. Мы дадим вам для этого много возможностей: в каждой главе есть специальные упражнения. Не пропускайте их — работа над упражнениями является важной частью процесса обучения. К каждому упражнению приводится решение — не бойтесь **заглянуть в него**, если окажетесь в тупике! (Иногда можно застрять на элементарном.) Но старайтесь выполнять упражнения самостоятельно. И обязательно добивайтесь результата, прежде чем переходить к следующей части книги.

Важная информация

Это учебник, а не справочник. Мы намеренно убрали из книги все, что могло бы помешать изучению материала, над которым вы работаете. И при первом чтении книги начинать следует с самого начала, потому что книга предполагает наличие у читателя определенных знаний и опыта.

Мы разбираем все на мелкие детали и собираем заново.

Мы обращаем ваше внимание на каждый конкретный аспект архитектуры ПО и подробно разбираем его. Мы используем много наглядных примеров, чтобы объяснить различные тонкости. Мы подробно описываем все аспекты и стараемся объяснить, когда и как их использовать. Только *после* этого мы начинаем собирать все воедино, чтобы представить более сложные идеи.

Мы не стремимся объять необъятное.

Мы используем подход 80/20 и исходим из того, что, решив получить ученую степень в области архитектуры ПО, вы прочтете не только эту книгу. Поэтому мы рассказываем не обо всем, а только о том, что вы действительно будете использовать и что поможет вам сразу приступить к делу. Мы хотим, чтобы вы сделали это не откладывая.

Упражнения ОБЯЗАТЕЛЬНЫ.

Обязательно выполняйте упражнения, они — часть основного материала книги. Одни помогут лучше запомнить прочитанное, другие — понять, а третьи — применить новые знания на практике. *Не пропускайте упражнения.* Они дадут вашему мозгу возможность обдумать слова и термины, которые вы изучаете.

Повторение применяется намеренно.

У книг этой серии есть одна принципиальная особенность: мы хотим, чтобы вы *действительно хорошо* усвоили материал. И чтобы вы запомнили все, что узнали. Большинство справочников не ставят своей целью успешное запоминание, но это не справочник, а *учебник*, поэтому некоторые концепции излагаются в книге по несколько раз.

Приведенные примеры максимально обобщены.

Чтобы помочь вам освоить такой сложный предмет, как проектирование архитектуры ПО, мы использовали реальные примеры, иначе описание идей, представленных в книге, оказалось бы слишком абстрактным и сложным. Мы намеренно сделали примеры максимально обобщенными, но в то же время интересными и забавными. Мы уверены: чем бы вы ни занимались — вы сможете использовать их в своей работе.

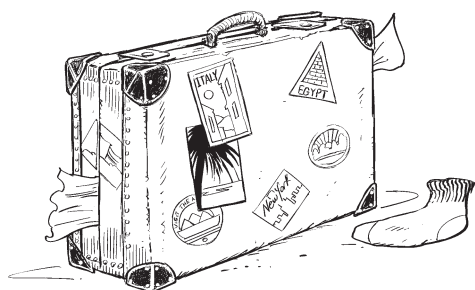
Упражнения «Мозговой штурм» не имеют ответов.

В некоторых из них правильного ответа вообще нет, в других вы должны сами решить, насколько правильны ваши ответы (это является частью процесса обучения). В некоторых упражнениях «Мозговой штурм» приводятся подсказки, которые помогут вам найти нужное направление.

Главы «Сделай сам»

Отличительная особенность этой книги из серии Head First — две главы, которые мы назвали «Сделай сам». Они целиком состоят из упражнений. Вам будет предложено создать архитектуру от начала до конца, применяя идеи, изученные к этому моменту.

В этих главах вы *выступите в роли архитектора программного обеспечения*, определите архитектурные характеристики, создадите логическую архитектуру и примете архитектурные решения, в том числе выберете архитектурный стиль. Упражнения помогут лучше понять, что делает архитектор ПО, и покажут, как много вы узнали.



В главе 9, первой главе «Сделай сам», вы создадите архитектуру системы управления поездками TripEZ, цель которой — помочь всем, кто путешествует. Это новое онлайн-приложение для управления поездками позволит путешественникам управлять своим бронированием, организованным по поездкам, используя браузер или мобильное устройство.

В главе 12, второй главе «Сделай сам», вы создадите архитектуру системы тестирования учащихся Make the Grade. Все учащиеся определенных классов государственных школ в Датавилле сдают тест, помогающий оценить их успеваемость, квалификацию педагогов и в целом качество преподавания в школе. Эта глава станет своеобразным *тестом* и для вас.



Рецензенты

Знакомьтесь: наши рецензенты!

Нам посчастливилось собрать для этой книги мощную команду рецензентов, в которую вошли *опытные разработчики, архитекторы, известные спикеры и авторы множества книг.*

Они внимательно прочитали каждую главу, выполнили упражнения, исправили наши ошибки и дали подробные комментарии к каждой странице книги. Попутно рецензенты выступили в роли консультантов и помогли нам отточить идеи, аналогии и сюжеты, а также продумать структуру книги.

Каждый рецензент внес ценный вклад и помог значительно улучшить качество книги. Мы глубоко признательны им за долгие часы, которые они провели над рукописью. Мы в неоплатном долгу перед ними.

Спасибо вам!

Особое спасибо Моатазу Санаду, нашедшему массу наших опечаток!

Мы очень старались; тем не менее любые ошибки и упущения, оставшиеся незамеченными, — это только наша вина.

Нейт Шутта



Таня Рейли



Кристин Шутта



Клэр Садбери



Венкат Субраманиам



Патрик Виафор



Марк Лой



Джеймс Эрлер



Макс Шуберт



Совместные благодарности

Эта книга была бы невозможна без помощи, руководства и поддержки многих выдающихся людей. Мы хотим поблагодарить их всех, так что начнем!

Нашему редактору

В первую очередь мы выражаем огромную благодарность нашему замечательному редактору **Саре Грей**. Работа над такой книгой, как «Head First. Программная архитектура», поставила перед нами ряд уникальных задач, и Сара все время была рядом, направляя нас. Она помогала не сбиться с пути, когда мы отклонялись от стиля, характерного для Head First (что случалось довольно часто), и вносила предложения по оформлению каждой страницы (действительно *каждой*). Сара взяла на себя роль эксперта по кроссвордам и довольно сильно помогла нам с текстами врезки «Важно знать». Мы часто называли Сару нашим «четвертым автором», и на самом деле то, что эта книга получилась именно такой, — ее большая заслуга.



Блестательная
Сара Грей

Коллективу O'Reilly

Большое спасибо всему коллективу O'Reilly Media, в том числе **Кристин Браун** и **Крису Фаучеру**, за то, что придали книге достойный вид, а также **Рейчел Хед** за ее острый и проницательный взгляд редактора.

Кроме того, мы благодарим **Мелиссу Даффилд** за ее поддержку и терпение на протяжении всего процесса, а также за ее предложение взяться за этот долгий проект.

Спасибо **команде онлайн-обучения в O'Reilly**, особенно **Ясмин Греко**, **Линдси Вентимилья** и всем продюсерам, за то, что предоставили нам платформу для обучения архитектуре ПО разработчиков и архитекторов по всему миру.

Особая благодарность **команде Early Release**, публиковавшей ранние версии глав на платформе O'Reilly, чтобы с ними могла ознакомиться широкая аудитория. Это дало многим нашим читателям возможность внести исправления и оставить отзывы, которые помогли улучшить книгу.

И наконец, мы не можем не упомянуть редакторов серии Head First **Элизабет Робсон** и **Эрика Фримена**, которые нашли время, чтобы просмотреть книгу и убедиться, что она соответствует концепции серии. Кроме того, они дали несколько ценных советов по работе с InDesign. Спасибо!

Индивидуальные благодарности

От Раджу Ганди

Мне трудно выразить, насколько большой честью была работа над этим проектом вместе с Марком и Нилом — двумя умнейшими и замечательными людьми, которые не только любезно согласились взять меня в соавторы, но и провели бесчисленные часы, обучая меня тонкостям архитектуры ПО. Надеюсь, что когда-нибудь я смогу вернуть им этот долг. Пока же я выражаю свою глубочайшую признательность. Кроме того, я благодарен моим друзьям, коллегам, невольным наставникам и учителям, которые были для меня источником вдохновения, — **вы знаете, о ком я говорю**. И наконец, спасибо моей жене **Мишель**. Пока я работал над этим проектом, у нас родилась малышка **Дельфина**, и Мишель, конечно, взяла на себя основное бремя, пока я работал над этой книгой. Спасибо тебе. Я люблю вас обоих.



От Марка Ричардса

Большое спасибо моим друзьям и соавторам Раджу и Нилу. Раджу поделился своим опытом работы с книгами из серии Head First, который он получил при создании замечательной книги *Head First Git*, и помог нам освоить стиль серии, а также тонкости работы с InDesign. Это моя третья книга, написанная совместно с Нилом, и сотрудничество с ним снова было очень полезным и приятным. Кроме того, я хочу поблагодарить мою прекрасную жену **Ребекку** за ее терпение и понимание, которые она проявляла, пока я вечерами сидел в своем кабинете и писал книгу, вместо того чтобы наслаждаться ее обществом.



От Нила Форда

Прежде всего спасибо моим соавторам, Марку и Раджу, с которыми было приятно работать и которые сделали эту книгу возможной. Марк — потрясающий соавтор с хорошим чувством юмора, что очень важно, когда писательство не является вашей повседневной работой. Спасибо нашему редактору, **Саре**, сыгравшей важную роль в этой серии книг и помогавшей нам не отклоняться от намеченной цели. Спасибо моим родным и близким за их поддержку. Спасибо членам нашего еженедельного коктейльного клуба, находящегося по соседству, который во время пандемии переехал на парковку и остался там; благодаря вам я был в курсе происходящего поблизости. И наконец, я благодарю мою замечательную жену **Кэнди**, которая проводит долгие часы вдали от меня и наших кошек, пока я работаю над такими книгами, как та, которую вы сейчас держите в руках.



Хорошо, что авторов всего трое, а то благодарности продолжались бы и продолжались...

И наконец, спасибо вам, читатели. Ваше внимание — ценный ресурс, и мы благодарны вам за то время, которое вы уделите чтению этой книги. Счастливого обучения.

Приступим!



Архитектура программного обеспечения лежит в основе успешной работы системы.

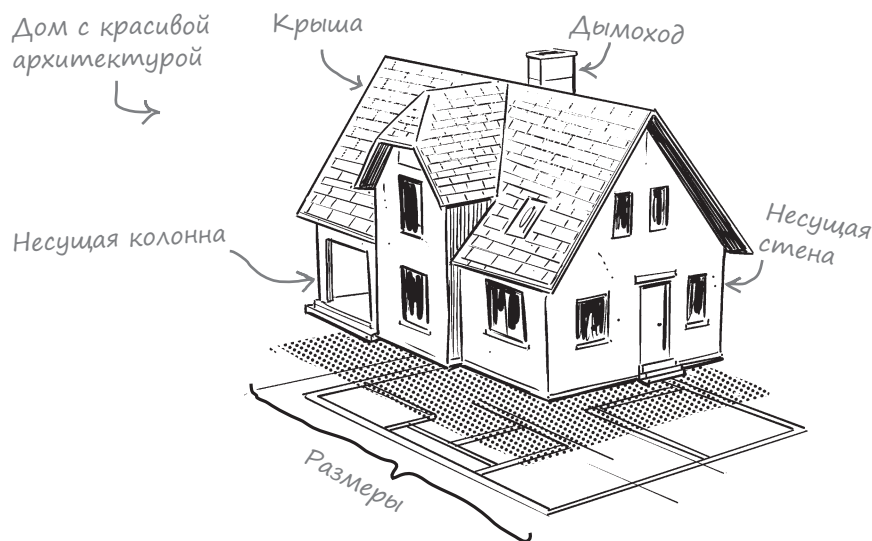
Эта глава срывает покров таинственности с архитектуры ПО и рассказывает об архитектурных измерениях и различиях между архитектурой и дизайном. Об этом важно знать, поскольку понимая архитектурные практики и применяя их, вы сможете создавать эффективные системы, которые не просто лучше функционируют, но и отвечают нуждам и чаяниям бизнеса и продолжают работать, когда техническая и бизнес-среда претерпевают постоянные изменения.

https://t.me/IT_Portal

Основы архитектуры программного обеспечения

Чтобы лучше понять, что такое архитектура ПО, вообразите типичный дом. Структура дома — это его **архитектура**: такие его параметры, как форма, количество комнат и этажей, размеры и т. д. Структура обычно представлена в виде плана здания со всеми линиями и рамками, помогающими понять, как его построить. Структурные элементы, подобные показанным ниже, впоследствии будет сложно и дорого изменить, и все они являются *важными* элементами дома.

Метафора здания очень популярна при знакомстве с архитектурой ПО.



Строительство дома основывается на его архитектуре. Без нее может получиться такой дом, как на рисунке справа.

Архитектура важна и для программных систем. Встречались ли вам системы, которые не масштабируются, ненадежны или сложны в сопровождении? Почти наверняка архитектуре таких систем уделялось недостаточно внимания.



Упражнение

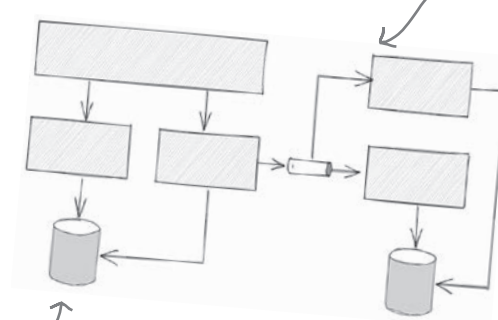
Садоводство — еще одна метафора, полезная для описания архитектуры ПО. Ниже напишите, как планирование сада может быть связано с архитектурой ПО. Наше решение вы сможете увидеть в конце главы.

→ Решение на с. 61

План строительства и архитектура ПО

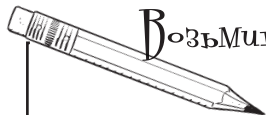
Как же связаны план строительства дома и архитектура ПО? И то и другое представляет строящуюся конструкцию. А как выглядит «план строительства» программной системы? Как множество линий и прямоугольников, конечно.

План здания определяет его структуру: комнаты, стены, лестницы и т. д. Архитектурный план ПО тоже определяет структуру (пользовательские интерфейсы, сервисы, базы данных и протоколы связи). Оба плана отражают рекомендации и ограничения, а также видение конечного результата.



На обеих схемах показаны планы строительства.

Интересный факт: раньше план здания называли чертежом, но сейчас этот термин устарел (по крайней мере, в строительстве).



Возьмите в руку карандаш

Какие особенности дома вы можете назвать *структурными* и связанными с *архитектурой*? Наш ответ вы найдете в конце главы.

Запишите здесь свои мысли.



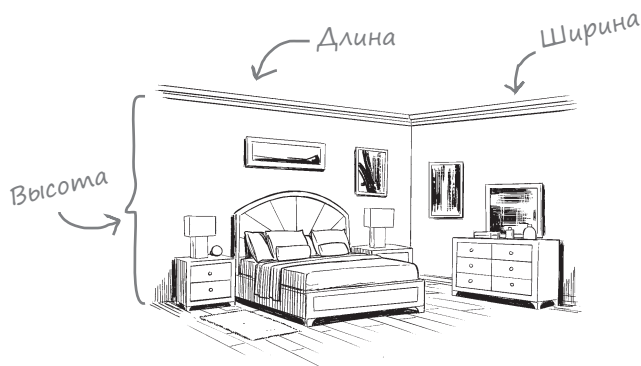
Вы заметили, что на плане дома выше не указаны подробные сведения о комнатах, такие как тип напольного покрытия (ковролин или паркет), цвет стен и место, где в спальне будет стоять кровать? Дело в том, что эти подробности не являются *структурными*. Иначе говоря, они определяют не *архитектуру* дома, а его *дизайн*.



Не волнуйтесь! О различиях между ними вы узнаете далее в этой главе. А пока просто сосредоточьтесь на *структуре*, то есть на *архитектуре*.

→ Решение на с. 61

Измерения в архитектуре ПО



Многие окружающие нас вещи многомерны. Например, конкретную комнату можно описать, сказав, что ее длина — 5 метров, ширина — 4 метра, а высота потолка — 2,5 метра. Обратите внимание: чтобы правильно описать комнату, необходимо указать все три измерения: высоту, длину и ширину.

Таким же образом можно описать архитектуру ПО по ее измерениям. Разница лишь в том, что в ней *четыре измерения*.

1 Архитектурные характеристики

Это измерение описывает, какие аспекты системы: масштабируемость, тестируемость, доступность и т. д. — поддерживает архитектура.

2 Архитектурные решения

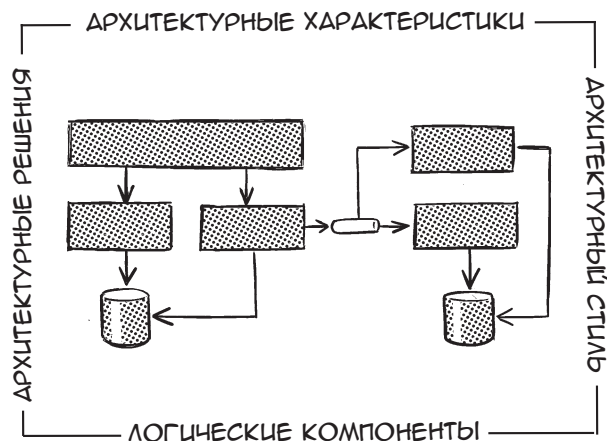
Это измерение содержит важные решения, имеющие долгосрочные или значительные последствия для системы, — например, тип базы данных, количество сервисов и их способ взаимодействия.

3 Логические компоненты

Это измерение описывает функциональные блоки и их взаимодействие. Например, система электронной коммерции может иметь компоненты для управления запасами, обработки платежей и т. д.

4 Архитектурный стиль

Это измерение определяет общую форму и структуру программной системы точно так же, как план здания определяет общую форму и структуру дома.



Далее в книге вы узнаете о пяти наиболее распространенных архитектурных стилях.

Мозаика из измерений

Архитектуру ПО можно представить как мозаику, где каждое измерение определяет отдельный фрагмент. Каждый фрагмент имеет уникальную форму и свойства, но все они должны сочетаться друг с другом и взаимодействовать, формируя полную картину.



Все взаимосвязано

Заметили, как фрагменты этой мозаики соединяются в центре? Именно так работает архитектура ПО: *все измерения должны сочетаться.*

Архитектурный стиль должен соответствовать архитектурным характеристикам и решениям. В свою очередь, определяемые вами логические компоненты должны соответствовать архитектурным характеристикам, стилю и решениям, которые вы принимаете.

Часто задаваемые вопросы

В: Все ли четыре измерения нужны при создании архитектуры или какие-то можно не учитывать, если времени не хватает?

О: К сожалению, ни одно измерение нельзя не учитывать — все они необходимы для создания и описания архитектуры. Начинающие архитекторы часто ошибаются, используя только одно или два измерения для описания архитектуры. Фраза «Наша архитектура — это микросервисы» описывает одно измерение — архитектурный стиль, но оставляет много вопросов без ответов. Например, какие архитектурные характеристики важны для успешной работы системы? Из каких логических компонентов (функциональных строительных блоков) она состоит? Какие важные решения приняты относительно реализации архитектуры?

Измерение первое: архитектурные характеристики



Архитектурные характеристики — основа архитектуры программной системы. Без них невозможно принять архитектурные решения или проанализировать компромиссы.

Представьте, что выбираете один из двух домов. Один большой, но находится рядом с оживленной автомагистралью. Другой намного меньше, зато расположен в красивом и тихом районе.

Что для вас важнее: размер дома или тишина и вид из окна? Не зная этих характеристик, вы не сможете сделать правильный выбор.

То же относится к архитектуре. Допустим, вам нужно решить, какую базу данных использовать в новой системе: реляционную, простую типа «ключ — значение» или сложную графовую. Выбор будет зависеть от архитектурных характеристик, имеющих для вас решающее значение. Например, можно выбрать графовую БД, если нужен быстрый поиск (назовем это *производительностью*), или традиционную реляционную, если нужно сохранить взаимосвязи между данными (назовем это *целостностью данных*).

производительность

время, необходимое системе для обработки запроса

доступность

время безотказной работы системы; обычно измеряется в «девятках» (так, 99,9 % — это три «девятки»).

масштабируемость

способность системы поддерживать постоянное время отклика и количество ошибок по мере увеличения количества пользователей или запросов

Это наиболее распространенные архитектурные характеристики. Вы познакомитесь с ними в главе 2.



Упражнение

Отметьте описания, которые, по вашему мнению, можно считать архитектурными характеристиками, поддерживающими *структуру* программной системы.

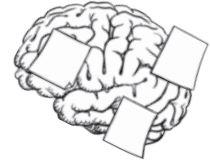
- ☐ Изменение размера шрифта в пользовательском интерфейсе.
- ☐ Простота внесения изменений.
- ☐ Одновременное обслуживание тысяч пользователей.
- ☐ Шифрование паролей перед сохранением в базе данных.
- ☐ Взаимодействие с множеством внешних систем в целях обработки запроса.

→ Решение на с. 62

Термин «*архитектурные характеристики*» может быть вам незнаком, но это не значит, что вы не слышали его раньше. В совокупности такие аспекты, как производительность, масштабируемость, надежность и доступность, известны также как нефункциональные требования, атрибуты качества и просто «возможности». Нам нравится термин «*архитектурные характеристики*», поскольку все эти аспекты помогают определить характер архитектуры и понять, что она должна поддерживать.

Архитектурные характеристики — это возможности, критически важные или значимые для успешной работы системы.

Запомните
это!



Разрабатывая архитектуру,
сначала определите
ключевые возможности
приложения

Что чему соответствует?

Вам выпал шанс оценить, как много вы уже знаете о типичных архитектурных характеристиках. Сопоставьте характеристики (слева) с определениями (справа). Обратите внимание, что определений больше, чем характеристик, поэтому будьте осторожны — характеристика соответствует не каждому определению.

Расширяемость

Пример
от нас.

Гибкость

Интегрируемость

Отказоустойчивость

Осуществимость

При выборе архитектурных решений учитываются сроки, бюджет и навыки разработчиков.

Способность системы сохранять другие части работоспособными при появлении фатальных ошибок.

Простота расширения системы в целях поддержки дополнительных возможностей.

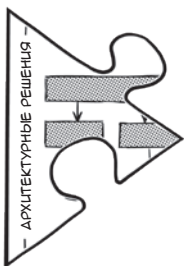
Время, необходимое для отправки ответа пользователю.

Возможность быстро вносить изменения в систему (удобство сопровождения, тестируемость и готовность к развертыванию).

Способность взаимодействовать с другими системами для обработки бизнес-запроса.

Решение на с. 62

Измерение второе: архитектурные решения



Архитектурные решения позволяют выбрать структурные аспекты, имеющие долгосрочные или значительные последствия. Служа своего рода ограничениями, они будут задавать направления работы при планировании и создании системы.

Сколько этажей должно быть в доме — один или два? Какой должна быть крыша — плоской или куполообразной? А может, дом должен быть просторным в стиле ранчо? Это хорошие примеры архитектурных решений, которые затрагивают структурный аспект дома.



?



?



Как должен выглядеть дом? Это архитектурное решение.

Вы можете решить, что пользовательский интерфейс системы не должен напрямую взаимодействовать с базой данных, а получать и изменять данные он может с помощью базовых сервисов. Это архитектурное решение накладывает особые ограничения на разработку пользовательского интерфейса, а также помогает разработчикам узнать, как другие компоненты должны получать данные из базы и обновлять их.

Это пример архитектурного решения.

Архитектурное решение

Читать и изменять данные пользовательский интерфейс должен через специальный сервис; он не должен напрямую взаимодействовать с базой данных.

Это решение накладывает ограничение и указывает направления работы.

Сервис доступа к данным

Так изображаются сервисы. Они часто будут встречаться в книге.



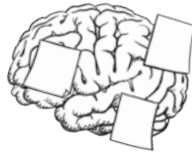
X



Это база данных (БД).

В главе 3 вы узнаете много нового об архитектурных решениях.

Запомните
это!



Решения — это структурные
руководства для разработчиков.
Они фокусируются на важных темах.

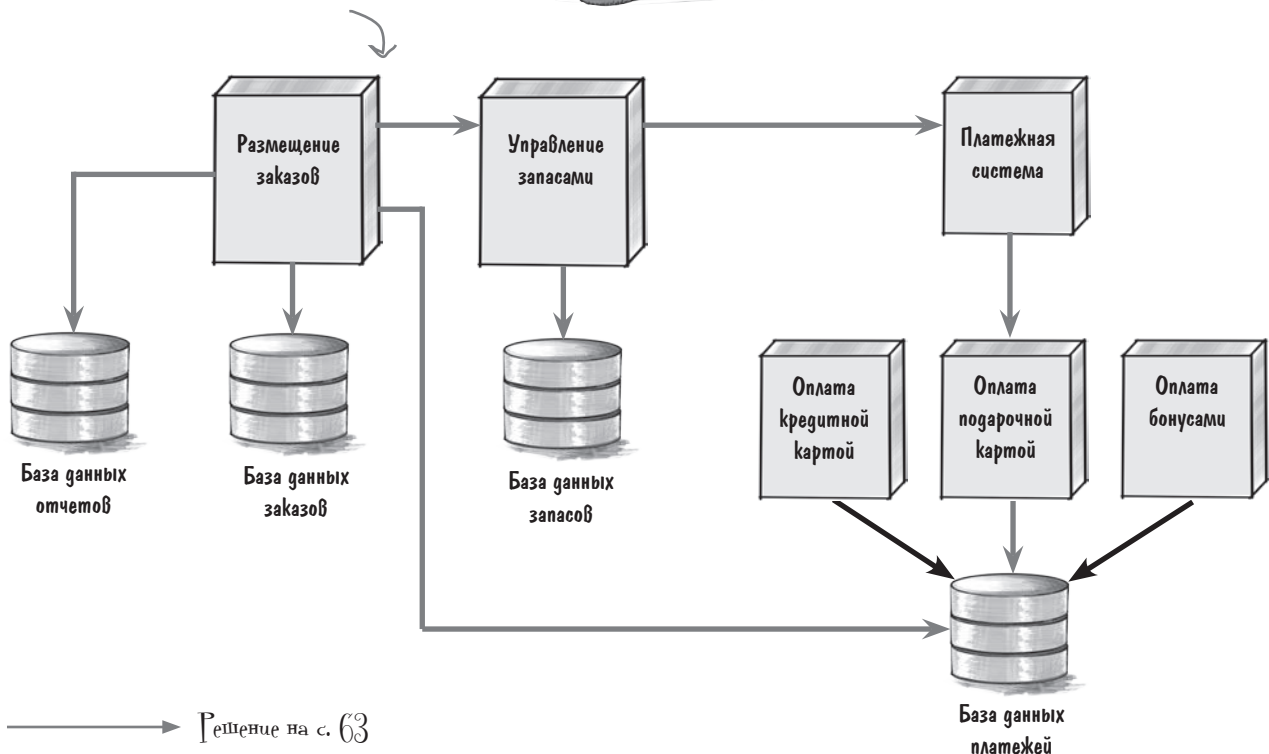
Нередко в системе есть несколько десятков за-
документированных архитектурных решений.
Чем больше и сложнее система, тем больше в ней
архитектурных решений.

СТАНЬТЕ архитектором



Представьте, что Вы архитектор,
и определите как можно больше
архитектурных решений на схеме ниже.
Отметьте все, что, по Вашему мнению,
является архитектурным решением,
и напишите, каким оно может быть.

Вот подсказка: возникают ли у вас
вопросы о том, почему что-то
делается именно так, а не иначе?



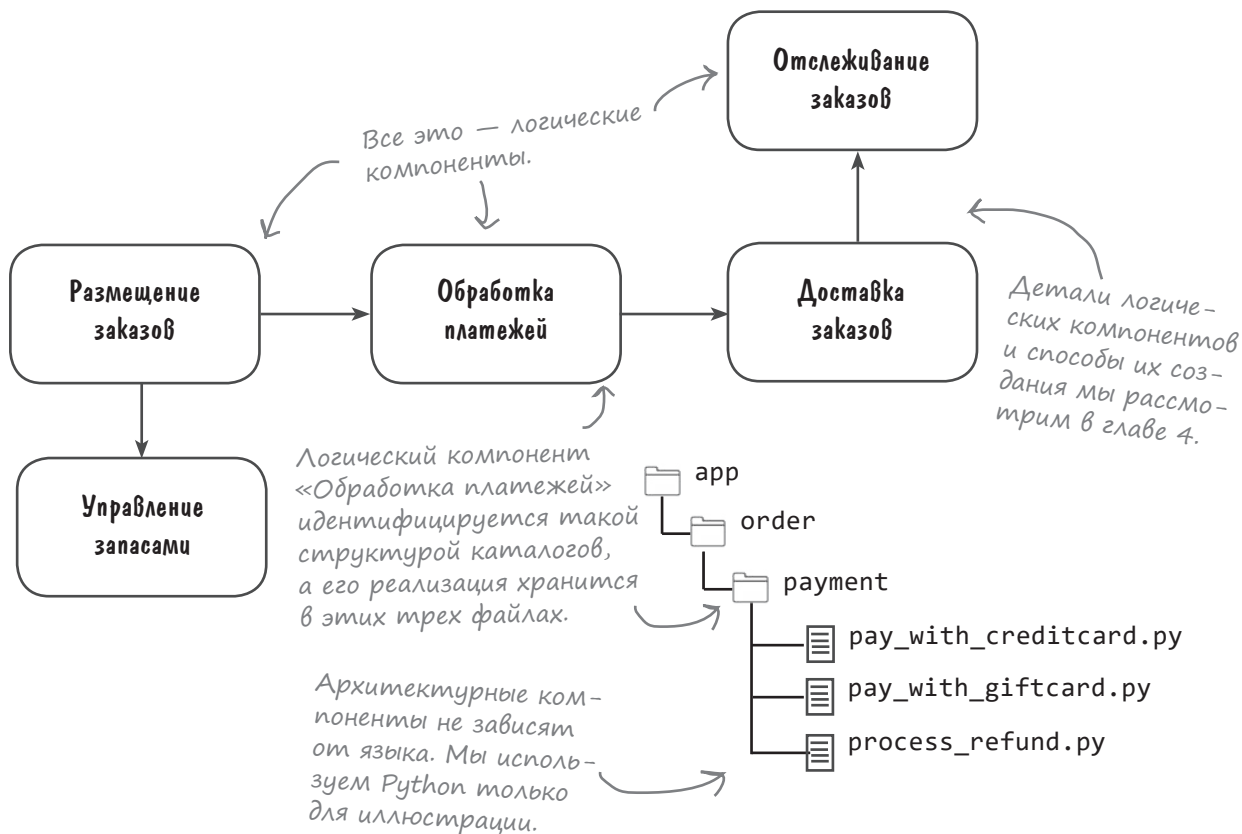
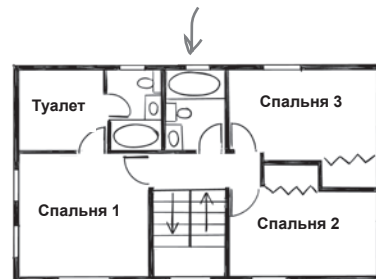
Измерение третье: логические компоненты



Логические компоненты — это строительные блоки системы, так же как комнаты — это строительные блоки дома. Каждый логический компонент решает определенную задачу, например обрабатывает платежи, управляет запасами или отслеживает заказы.

Логические компоненты в системе обычно размещаются в отдельных каталогах или пространствах имен. Например, каталог `app/order/payment` с соответствующим пространством имен `app.order.payment` идентифицирует логический компонент с именем «Обработка платежей». Исходный код, позволяющий пользователям оплачивать заказ, хранится в этом каталоге и использует данное пространство имен.

Комнаты — строительные блоки вашего дома.



Возьмите в руку карандаш

Вы только что определили два компонента для новой системы и дали разработчикам задание написать файлы классов с реализациями. Создайте структуру каталогов, чтобы разработчики могли начать писать код. Наше решение вы увидите в конце главы.

Профиль
клиента

Предпочтения
клиента

Запишите свой
ответ здесь.

Решение на с. 64

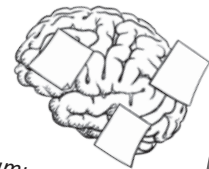
Роль и задачи логического компонента в системе всегда должны быть четко определены.

Этот компонент отвечает за «сбор и упаковку». Он определяет, где находятся товары на складе, а затем выбирает размер упаковки.

Выполнение
заказа

Это описание роли и задачи компонента «Выполнение заказа».

Запомните
это!



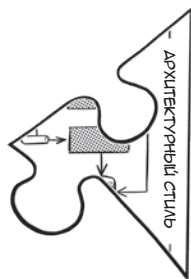
Логические компоненты — это блоки, связанные между собой. Они содержат исходный код для каждой бизнес-функции.

Часто задаваемые вопросы

В: В чем разница между функциональностью и предметной областью системы?

О: **Предметная область** — это задача, которую нужно решить, а **функциональность системы** — это способ решения задачи. Иначе говоря, предметная область — это «что», а функциональность системы — «как».

Измерение четвертое: архитектурные стили



Дома бывают разных форм, размеров и стилей. Конечно, есть дома, выглядящие странно, но большинство построено в определенном стиле, например викторианском, ранчо или тюдоровском. Стиль дома многое говорит о его структуре. Например, дома в стиле ранчо обычно имеют только один этаж, в колониальных домах и домах эпохи Тюдоров обычно есть дымоходы, а современные дома имеют плоские крыши.

В каждом регионе мира распространены свои стили домов — посмотрите их на сайте https://en.wikipedia.org/wiki/List_of_house_styles.

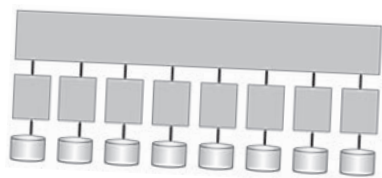
В каком стиле построен ваш дом?



Архитектурные стили определяют форму и структуру программной системы и имеют уникальные наборы характеристик. Например, архитектурный стиль **микросервисов** отличается хорошей масштабируемостью и высокой *гибкостью*, упрощающей внесение изменений. **Многоуровневый** архитектурный стиль менее сложный и затратный. А **событийно-ориентированный** архитектурный стиль имеет хорошую масштабируемость, производительность и отзывчивость.

Со всеми этими стилями вы познакомитесь далее в книге. Каждому из них мы посвятили отдельную главу.

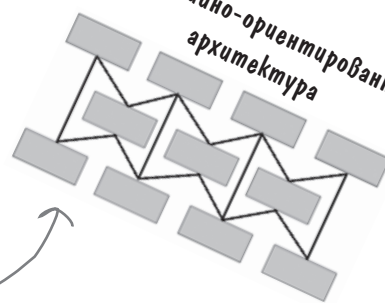
микросервисы



многоуровневая архитектура



событийно-ориентированная архитектура



Существует множество архитектурных стилей ПО. К счастью, их не так много, как в строительстве.

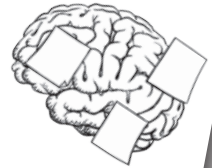
Архитектурный стиль определяет форму и характеристики системы, поэтому важно сделать все правильно с самого начала. Почему? Представьте, что вы начали строить одноэтажный дом в стиле ранчо, а в разгар строительства решили возвести трехэтажный викторианский дом. Это серьезное решение наверняка увеличит бюджет строительства и отодвинет сроки новоселья.

То же можно сказать и об архитектуре ПО. Решить задачу перехода от монолитной многоуровневой архитектуры к микросервисам будет очень нелегко.



Перестроить просторное ранчо в многоэтажный викторианский дом очень сложно, поскольку строения очень разные.

Запомните
это!



Стили формируют систему и помогают достичь ее целей.
Вы можете выбрать монолит или микросервисы.

Далее в книге мы покажем, как правильно выбрать архитектурный стиль, исходя из важных характеристик.

И снова мы возвращаемся к мысли, высказанной выше: все аспекты архитектуры ПО взаимосвязаны. Нельзя выбрать архитектурный стиль, не зная, что для вас важно.



Мозговой
штурм



Тугие сухожилия и мощные мышцы лап льва позволяют ему развивать скорость до 80 километров в час и преодолевать 11 метров в прыжке. Эта особенность помогает львам выживать и догонять добычу, которая быстро бежит.

Оглянитесь вокруг: что еще имеет структуру или форму, определяющую его характеристики и возможности?

Интересный факт: лев не очень вынослив — бежит быстро, но недолго. Продержавшись дольше преследующего вас льва, вы сможете выжить.

Что чему соответствует?

Система должна быть доступна клиентам за рубежом.

Речь идет о доступности.

Клиенты жалуются на цвет фона в новом пользовательском интерфейсе.

Владелец продукта требует предоставлять клиентам новые функции и исправление ошибок максимально быстро.

Система имеет событийно-ориентированную архитектуру.

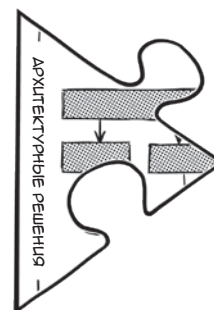
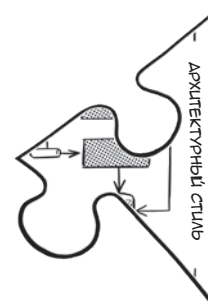
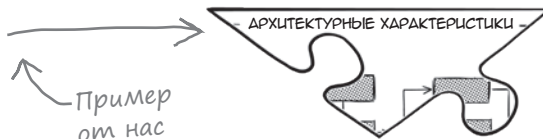
Система должна обслуживать до 300 000 пользователей одновременно.

Единый платежный сервис нужно разбить на отдельные сервисы по разным способам оплаты.

Мы собираемся начать принимать бонусные баллы в качестве оплаты заказа.

Мы собираемся разбить класс `orderPlacement` на три класса меньшего размера.

Пользовательский интерфейс не должен напрямую обращаться к базе данных.



Решение на с. 65

Если я отвечаю за дизайн
программной системы, означает
ли это, что я отвечаю и за ее
архитектуру? Разве это не одно
и то же?



Нет, архитектура и дизайн — не одно и то же.

Архитектура определяет не столько внешний вид, сколько структуру, а дизайн — не столько структуру, сколько внешний вид.

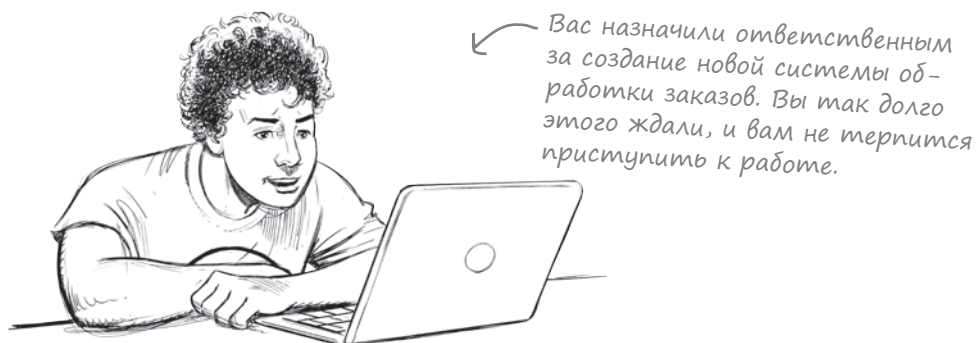
Цвет стен, расстановка мебели и тип напольного покрытия (ковролин или паркет) — все это аспекты дизайна, тогда как размеры комнаты и расположение дверей и окон — это детали архитектуры, то есть *структуры* комнаты.

Представьте типичное бизнес-приложение. Архитектура, или структура, определяет, как веб-страницы взаимодействуют с серверами и базами данных, а под дизайном подразумевается внешний вид страниц: цветовое оформление, расположение полей, используемые паттерны проектирования и т. д. Повторим еще раз: архитектура определяет структуру, а дизайн — внешний вид.

Вы задали хороший вопрос, поскольку иногда трудно определить, что считать архитектурой, а что — дизайном. Далее мы поговорим об их различиях.

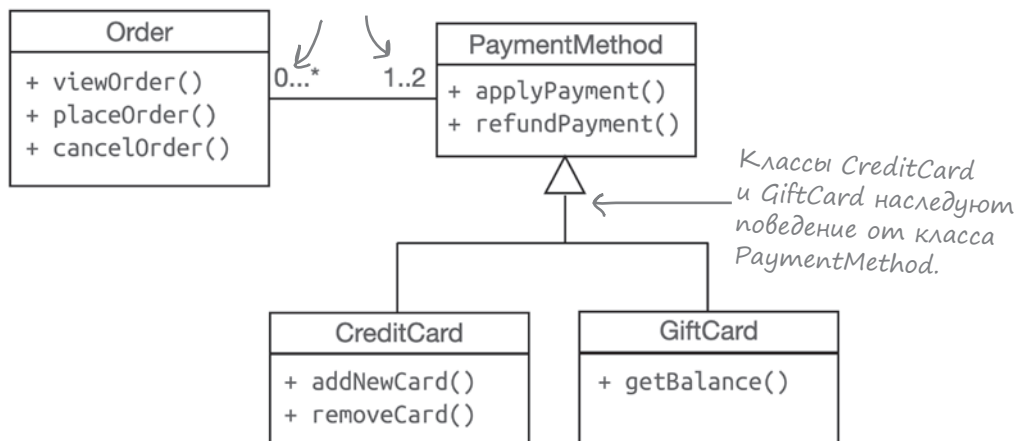
С точки зрения дизайна

Представьте, что в вашей компании решили заменить устаревшую систему обработки заказов новой, которая лучше соответствует конкретным потребностям. С ее помощью клиенты смогут размещать заказы, а также просматривать их или отменять. Они смогут оплачивать заказы кредитными или подарочными картами или обеими сразу.



Решая задачу с точки зрения *дизайна*, можно создать UML-диаграмму классов (Unified Modeling Language, унифицированный язык моделирования), чтобы показать, как классы реализуют функциональность оплаты, взаимодействуя друг с другом (пример такой диаграммы приведен ниже). Можно даже написать исходный код реализации этих файлов классов, но сама диаграмма ничего не говорит о *физической структуре* этого кода — о том, как эти файлы классов будут организованы и развернуты.

Заказы могут оплачиваться одним или двумя способами, а те, в свою очередь, могут быть связаны с нулем или несколькими заказами.

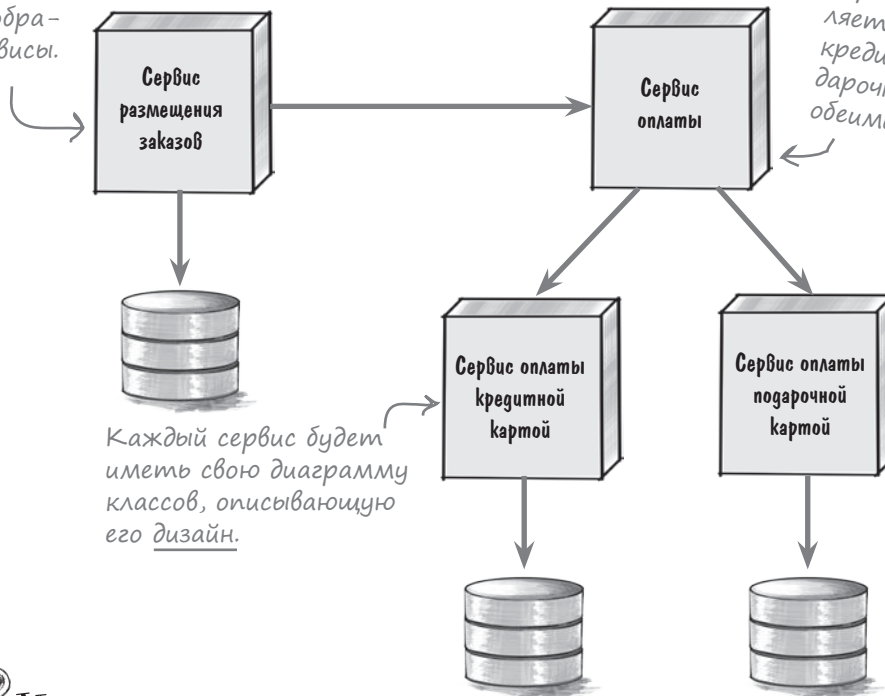


С точки зрения архитектуры

В отличие от дизайна, архитектура определяет *структуру* системы — такие компоненты, как сервисы и базы данных, а также способы взаимодействия сервисов между собой и с пользовательским интерфейсом.

Поразмыслим о новой системе обработки заказов еще немного. Как она будет организована? С точки зрения *архитектуры* для каждого способа оплаты можно создать свой сервис и использовать сервис-оркестратор для управления процессом оплаты, как показано на схеме ниже.

Напоминаем,
что так изображаются сервисы.



Сервис оплаты позволяет оплатить заказ кредитной или подарочной картой или обеими сразу.



Упражнение

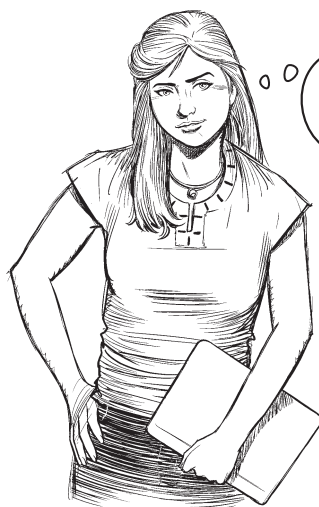
Отметьте все, что следует добавить в схему *с точки зрения архитектуры*.

- ☐ Порядок взаимодействия сервисов друг с другом
- ☐ Платформа и язык реализации сервисов
- ☐ Какие сервисы могут взаимодействовать с базой данных
- ☐ Сколько сервисов и баз данных использовать

→ Решение на с. 66

Спектр между архитектурой и дизайном

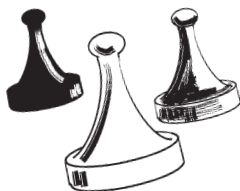
Одни решения, безусловно, являются архитектурными (например, выбор архитектурного стиля), другие явно связаны с дизайном (такие, как положение поля ввода на экране или тип поля внутри класса). Однако большинство решений находятся между этими двумя примерами, образуя *спектр* между архитектурой и дизайном.



Почему меня должно волновать, в какой точке спектра между архитектурой и дизайном находится мое решение? Это действительно важно?

Да, это очень важно. Знание о том, в какой части спектра между архитектурой и дизайном находится ваше решение, позволяет определить, *кто* должен его принимать. Есть решения, которые принимают только разработчики (например, организация классов, реализующих определенную функцию) или только архитектор (например, выбор архитектурного стиля для системы). И есть решения, принимаемые совместно (например, разделение или объединение сервисов).

В какую часть спектра попадает ваше решение?



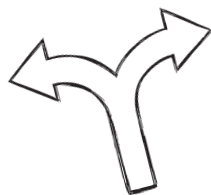
Оно стратегическое или тактическое?

Стратегические решения носят долгосрочный характер и влияют на будущие действия или решения. **Тактические** решения являются краткосрочными и обычно не зависят от других действий или решений (но могут приниматься в контексте конкретной стратегии). Например, решение о размерах вашего нового дома повлияет на количество и размеры комнат, тогда как выбор светильника не повлияет на решение о размере обеденного стола. Чем более стратегическим является решение, тем ближе оно к архитектурной стороне спектра.

Иногда утреннее пробуждение требует больших усилий. Назовем такие утра «архитектурными».

Сколько усилий понадобится для создания или изменения?

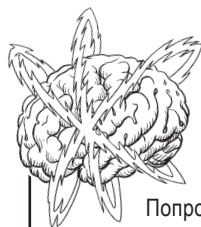
Что касается создания или изменения, то архитектурные решения требуют больших усилий, а решения, касающиеся дизайна, — меньших. Например, строительство пристройки к дому обычно требует многих усилий; следовательно, ее возведение ближе к архитектурному концу спектра. Чтобы постелить ковер в комнате, требуется гораздо меньше усилий, а значит, это решение будет ближе к дизайну.



Имеются ли значимые компромиссы?

Компромиссы — это плюсы и минусы, оцениваемые при выборе решения. Решения, предполагающие большие компромиссы, требуют детального анализа и, как правило, носят более архитектурный характер. Решения с менее значимыми компромиссами принимаются быстрее и проще и потому в большей степени относятся к дизайну.

Далее мы подробно рассмотрим все эти три фактора.

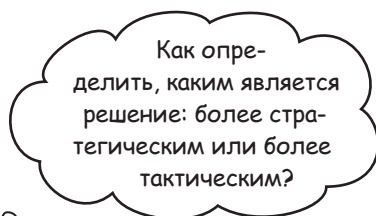


Мозговой шторм

Попробуйте придумать решение, не предполагающее никаких компромиссов, даже самых маленьких. Если вам покажется, что вы нашли такое решение, то поищите еще.

Стратегические и тактические решения

Чем более стратегическим является решение, тем более архитектурным оно становится. Это важное различие: стратегические решения требуют более тщательного обдумывания и планирования и, как правило, носят более долгосрочный характер.



Хороший вопрос. Чтобы ответить на него, можно использовать следующие три вопроса. Помните: чем более стратегическим является решение, тем больше оно связано с архитектурой.

1. Сколько времени уйдет на размышления перед принятием этого решения?

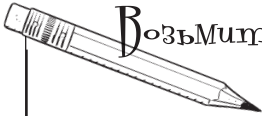
Если не более часа, то скорее всего, решение тактическое. Если несколько дней или недель, то оно, вероятнее всего, более стратегическое (и более архитектурное).

2. Сколько человек участвует в принятии решения?

Чем больше участников, тем более стратегическим является решение. Решение, принимаемое в одиночку или на пару с коллегой, скорее всего, будет тактическим. Решение, требующее многосторонних встреч, вероятно, является более стратегическим.

3. На что нацелено решение: на долгосрочную перспективу или краткосрочное действие?

Быстрые решения, которые вскоре могут измениться, — более тактические и поэтому больше относятся к дизайну. Решения, последствия которых будут долго иметь значение, — более стратегические и больше связаны с архитектурой.

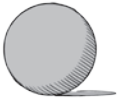


Возьмите в руку карандаш

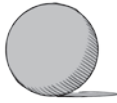
Мы рассыпали шарики и просим вас помочь разложить их по банкам. Используя три вопроса на предыдущей странице, определите, в какую банку следует положить каждый шарик.



Выбор языка программирования для нового проекта



Решение завести первую собаку



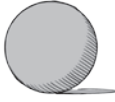
Развертывание в облаке или локально



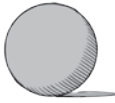
Переделка пользовательского интерфейса



Миграция системы на архитектуру микросервисов



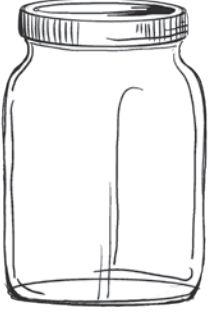
Выбор библиотеки парсинга



Выбор паттерна проектирования



Стратегические



Промежуточные



Тактические

→ Решение на с. 67

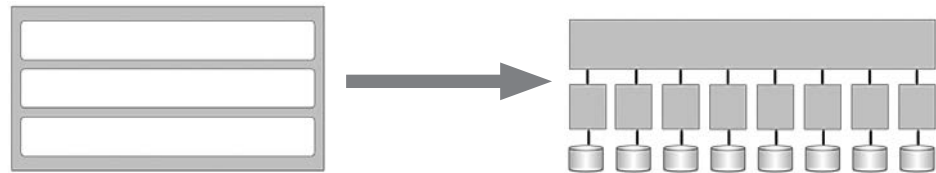
Большие и малые усилия

Мартин Фаулер, известный архитектор ПО и писатель, однажды сказал, что «архитектура ПО — это то, что трудно изменить». Вы можете использовать это определение, чтобы понять, в какой части спектра находится ваше решение. Чем сложнее что-то изменить впоследствии, тем ближе решение к архитектурной стороне спектра. И наоборот, чем легче внести позднейшие изменения, тем вероятнее, что решение связано с дизайном.

На сайте Мартина Фаулера (<https://martinfowler.com/architecture>) вы найдете много полезного об архитектуре.



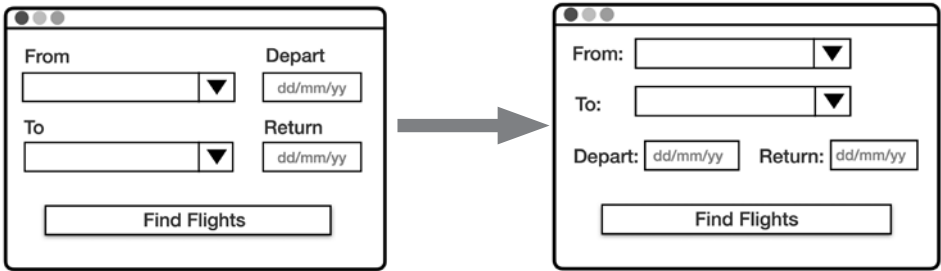
Предположим, вы решили изменить архитектурный стиль, например, вместо традиционной многоуровневой архитектуры использовать микросервисы. Это довольно сложный комплекс действий, на выполнение которых уйдет много времени. Понадобится много усилий, поэтому решение лежит ближе к архитектурному концу спектра.



Это потребует больших усилий. Смена архитектурного стиля — это сложно.

Изменение макета страницы влияет на внешний вид, а не на структуру, вот почему эту задачу можно отнести к области дизайна.

Теперь предположим, что вы меняете порядок элементов на странице. Эта задача требует меньше усилий, поэтому относится скорее к области дизайна.





Развлечения с МаГнитами

На нашем холодильнике висели магнитики с пунктами из списка дел в порядке от сложных к простым, но каким-то образом они все упали на пол и перемешались. Помогите нам расположить их в порядке уменьшения необходимых усилий.

Нарисуйте стрелки так, чтобы задачи, требующие больших усилий, оказались в верхней части страницы, а требующие меньших усилий — в нижней.

Больше усилий



Разрешение конфликта
слияния в Git

Замена фреймворка
пользовательского интерфейса

Миграция системы
в облачную среду

Выбор горчицы
для покупки

Переименование метода или
функции

Разделение одного сервиса
на несколько

Миграция с реляционной
базы данных на графовую

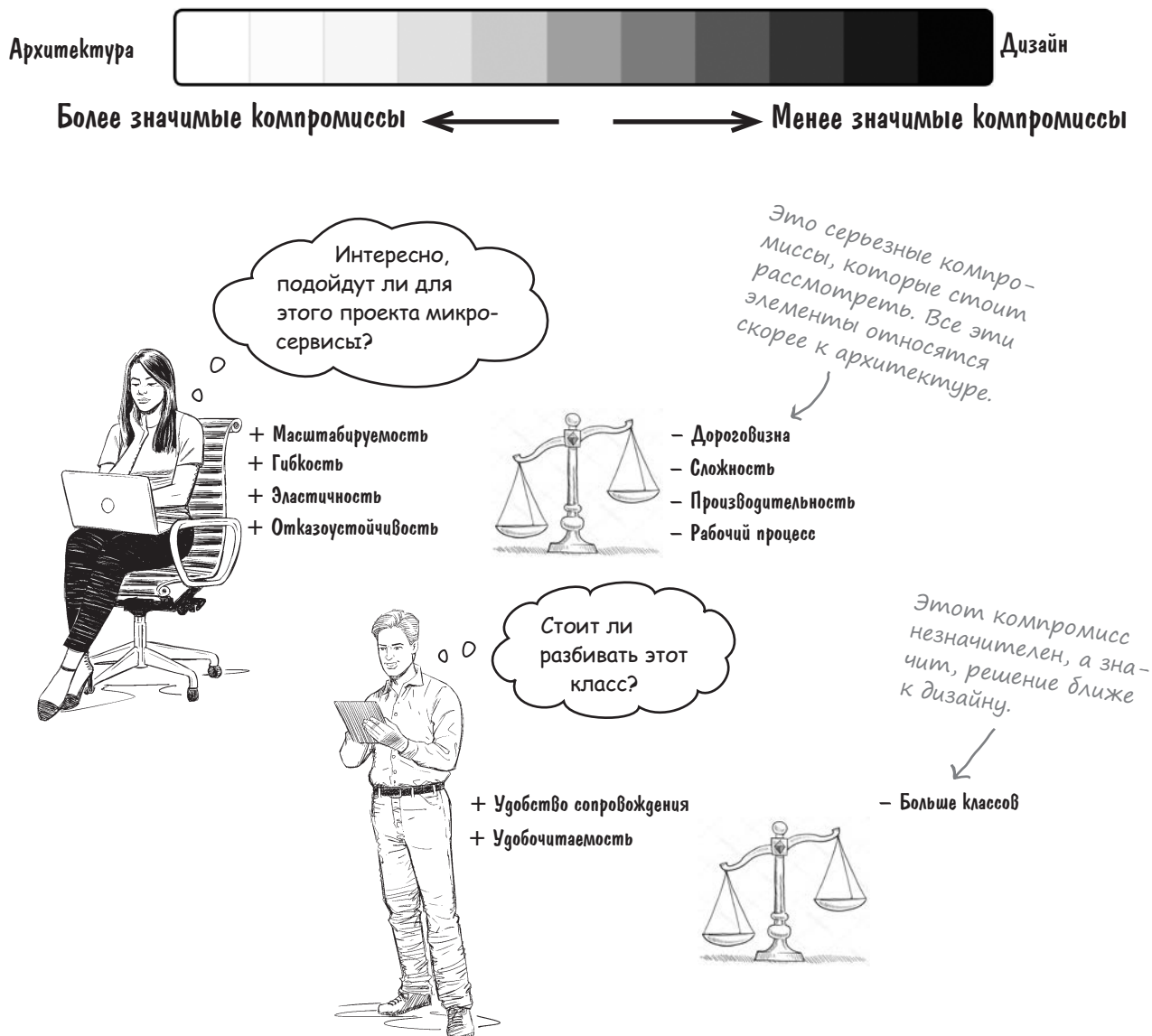
Разделение одного класса
на несколько

Меньше усилий

Решение на с. 68

Значимые и незначимые компромиссы

Одни решения могут предполагать значимые компромиссы, например выбор города для проживания. При принятии других решений в расчет берутся менее важные компромиссы, например, выбор цвета коврика в прихожей. Опираясь на уровень значимости компромиссов, можно определить, к чему ближе это решение: к архитектуре или дизайну. Чем более значимыми являются компромиссы, тем ближе решение к архитектуре. И наоборот, решение тем ближе к дизайну, чем менее значимы компромиссы.





Упражнение

Как же принять все эти решения? Есть способ, позволяющий справиться с данной задачей. Нужно определить решения, предполагающие значимые компромиссы, которые требуют больше времени на обдумывание. Помогите нам определить, какие решения имеют значимые компромиссы, а какие нет.

Это значимые компромиссы?

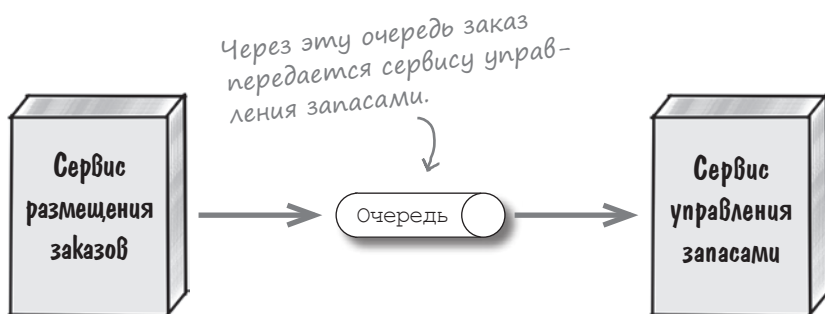
- | | | |
|-----------------------------|------------------------------|---|
| ☐ Да | ☐ Нет | Выбор, какую одежду надеть сегодня на работу |
| ☐ Да | ☐ Нет | Выбор между развертыванием в облаке и локальным |
| ☐ Да | ☐ Нет | Выбор фреймворка пользовательского интерфейса |
| ☐ Да | ☐ Нет | Выбор имени для переменной в файле класса |
| ☐ Да | ☐ Нет | Выбор между сливочным и шоколадным мороженым |
| ☐ Да | ☐ Нет | Выбор архитектурного стиля |
| ☐ Да | ☐ Нет | Выбор между REST и обменом сообщениями |
| ☐ Да | ☐ Нет | Передача в сообщениях полных данных или только ключей |
| ☐ Да | ☐ Нет | Выбор библиотеки анализа XML |
| ☐ Да | ☐ Нет | Решение о разделении сервиса |
| ☐ Да | ☐ Нет | Выбор между атомарными и распределенными транзакциями |
| ☐ Да | ☐ Нет | Решение, идти ли вечером на ужин |

—————→ Решение на с. 69

Собираем все воедино

Пришло время использовать все три фактора и выяснить, к чему больше относится решение: к архитектуре или дизайну. Это поможет разработчикам понять, когда следует сотрудничать с архитектором, а когда принимать решение самостоятельно.

Допустим, вы решили использовать асинхронный обмен сообщениями между сервисами размещения заказов и управления запасами и тем самым повысить скорость отклика системы. В конце концов, клиент не должен ждать, пока компания определит объем запасов. Посмотрим, в какой части спектра находится это решение.



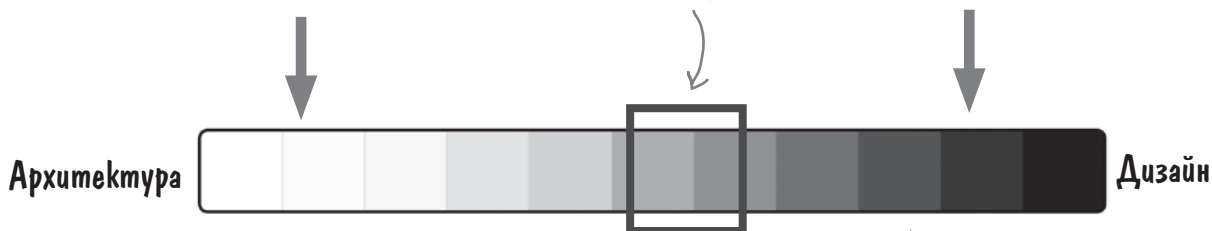
Значимость компромиссов

Использование очереди повысит скорость отклика, но запаздывание обновления запасов может привести к отмене заказа. Это значимый компромисс.

Значимые компромиссы приближают это решение к архитектуре.

Стратегическое или тактическое

В принятии этого решения участвует не так много людей, и оно не предполагает длительного обдумывания, поэтому является скорее тактическим.



С учетом всех трех факторов решение окажется примерно здесь, то есть оно имеет некоторую близость к архитектуре, и вероятно, следует проконсультироваться с архитектором или привлечь его к работе. Нам потребовались все три фактора, чтобы определить, к чему относится это решение: к архитектуре или дизайну.

Количество усилий

Чтобы отправить сообщение другому сервису, не нужны особые усилия. Это реализуется довольно стандартно.

У Вас получилось!

Поздравляем! Вы сделали первый шаг на пути к пониманию архитектуры ПО. Но прежде чем вы продолжите чтение, предлагаем пройти небольшой тест и проверить свои знания. Прочитайте каждое из приведенных ниже утверждений и отметьте, верно оно или нет.

Верно или неверно

Верно	Неверно	Дизайн описывает структуру дома (стены, крыша, планировка и т. д.), а архитектура ПО подобна мебели и отделке.
Верно	Неверно	Большинство решений относится исключительно к архитектуре или дизайну. Очень мало решений, находящихся посередине.
Верно	Неверно	Чем более стратегическим является решение, тем больше оно относится к архитектуре; более тактическое решение ближе к дизайну.
Верно	Неверно	Чем больше усилий требуется для реализации или изменения решения, тем ближе оно к дизайну; решения, требующие небольших усилий, ближе к архитектуре.
Верно	Неверно	<i>Компромиссы</i> — это плюсы и минусы решения или задачи. Чем ближе решение к архитектуре, тем существеннее компромиссы.

→ Решение на с. 70

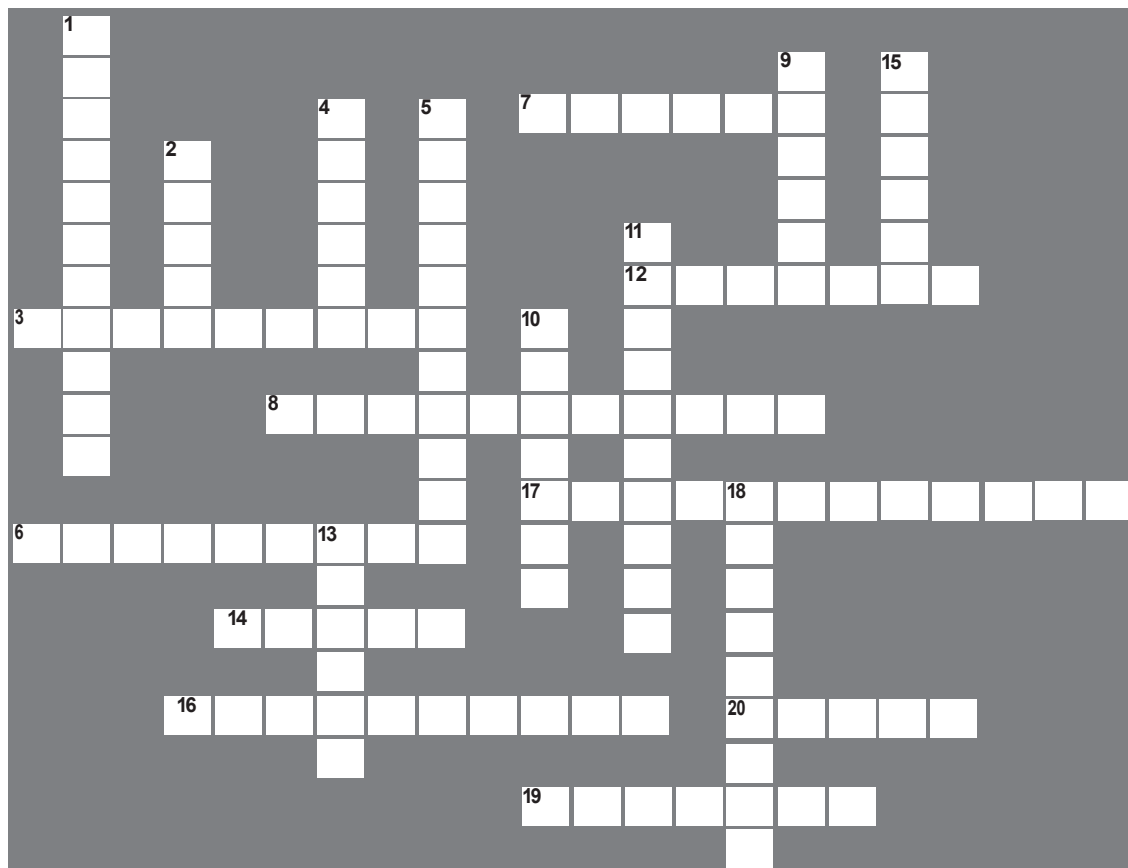
Ключевые моменты

- Архитектура ПО — это не столько внешний вид, сколько структура, а дизайн больше связан с внешним видом и меньше — со структурой.
- Для описания архитектуры ПО используются четыре измерения: архитектурные характеристики, архитектурные решения, логические компоненты и архитектурный стиль.
- Архитектурные характеристики составляют основу архитектуры ПО. Важно знать, какие архитектурные характеристики свойственны вашей системе, — это дает возможность проанализировать компромиссы и принять правильные архитектурные решения.
- Архитектурные решения служат ориентирами, помогающими разработчикам понять условия и ограничения архитектуры.
- Логические компоненты архитектурного решения — это строительные блоки системы. Они определяют, что делает система, и реализуются в файлах классов или исходном коде.
- Как и в строительстве, в разработке ПО существует множество разных архитектурных стилей. Каждый из них поддерживает определенные архитектурные характеристики, поэтому при работе с системой важно выбрать правильный стиль (или их комбинацию).
- Важно знать, к чему относится решение: к архитектуре или дизайну. Это помогает определить, кто должен принимать решение и насколько оно важно.



Архитектурный кроссворд

Поздравляем! Вы прочли первую главу и узнали, что такое архитектура программного обеспечения. А теперь предлагаем разгадать кроссворд.



По горизонтали

3. _____-ориентированный архитектурный стиль.
6. Количество комнат в доме — часть его _____.
7. Строительство этого может служить отличной метафорой.
8. Вы изучаете _____ программного обеспечения.
12. Вы примете много архитектурных _____.
14. Общая форма дома или системы, например викторианской эпохи или микросервисов.
16. _____ компоненты системы являются ее строительными блоками.
17. Решения могут быть стратегическими и _____.
19. Важно знать, к чему ближе решение: к архитектуре или _____.
20. Архитектурный стиль определяет общую _____ системы.

По вертикали

1. Так иногда называют архитектурные характеристики.
2. Компромиссы имеют свои _____ и минусы.
4. Архитектурные решения обычно принимаются на _____ срок.
5. Вы анализируете их, принимая архитектурные решения.
9. Количество измерений архитектуры ПО.
10. Архитектура и дизайн находятся на разных концах _____.
11. Возможно, вы захотите стать им, прочитав эту книгу.
13. Если для реализации решения требуется много _____, то, скорее всего, это архитектурное решение.
15. Стратегические решения обычно принимаются несколькими _____.
18. При создании пользовательского _____ принимается множество дизайнерских решений.



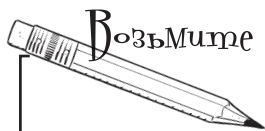
Упражнение Решение

На с. 34

Садоводство — еще одна метафора, полезная для описания архитектуры ПО. Ниже напишите, как планирование сада может быть связано с архитектурой ПО.

Общий план сада можно сравнить с архитектурным стилем, каждую группу похожих (по типу или цвету) растений — с архитектурными компонентами, а отдельные растения в группе — с файлами классов, реализующих эти компоненты.

Сады подвержены влиянию погоды, а архитектура ПО — влиянию изменений в технологиях, средах развертывания, на платформах и т. д. Кроме того, если не уделять должного внимания, то сад зарастет сорняками, а архитектура придет в структурный упадок.



Возьмите в руку карандаш

На с. 35

Решение

Какие особенности дома вы можете назвать *структурными* и связанными с архитектурой?

Размер и форма кухни (кто не жалуется, что его кухня маленькая?)

Количество этажей в доме (с возрастом лестницы могут стать проблемой)

Местонахождение входной двери и доступность входа для инвалидного кресла

Размер шкафа в спальне (если у вас много одежды)

Высота потолков (особенно если вы очень высокий)

Количество ванных комнат (добавить новую ванную очень сложно)

Чердак для вещей, вышедших из употребления

Открытая веранда или внутренний дворик (если, конечно, вы живете не в Арктике)



Упражнение Решение

На с. 38

Отметьте описания, которые, по вашему мнению, можно считать архитектурными характеристиками, поддерживающими *структуру* программной системы.

☐ Изменение размера шрифта в пользовательском интерфейсе

☒ Простота внесения изменений

В архитектуре это называется гибкостью.

☒ Одновременное обслуживание тысяч пользователей

Это называется эластичностью.

☐ Шифрование паролей перед сохранением в базе данных

Это известно как интегрируемость.

☒ Взаимодействие с множеством внешних систем для обработки запроса

Что чему соответствует? Решение

На с. 39

Вам выпал шанс оценить, как много вы уже знаете о типичных архитектурных характеристиках. Сопоставьте характеристики (слева) с определениями (справа). Обратите внимание, что определений больше, чем характеристик, поэтому будьте осторожны — характеристика соответствует не каждому определению.

Расширяемость

← Пример от нас

Гибкость

Интегрируемость

Отказоустойчивость

Осуществимость

При выборе архитектурных решений учитываются сроки, бюджет и навыки разработчиков

Способность системы сохранять другие части работоспособными при появлении фатальных ошибок

Простота расширения системы в целях поддержки дополнительных возможностей

Время, необходимое для отправки ответа пользователю

Возможность быстро вносить изменения в систему (удобство сопровождения, тестируемость и готовность к развертыванию)

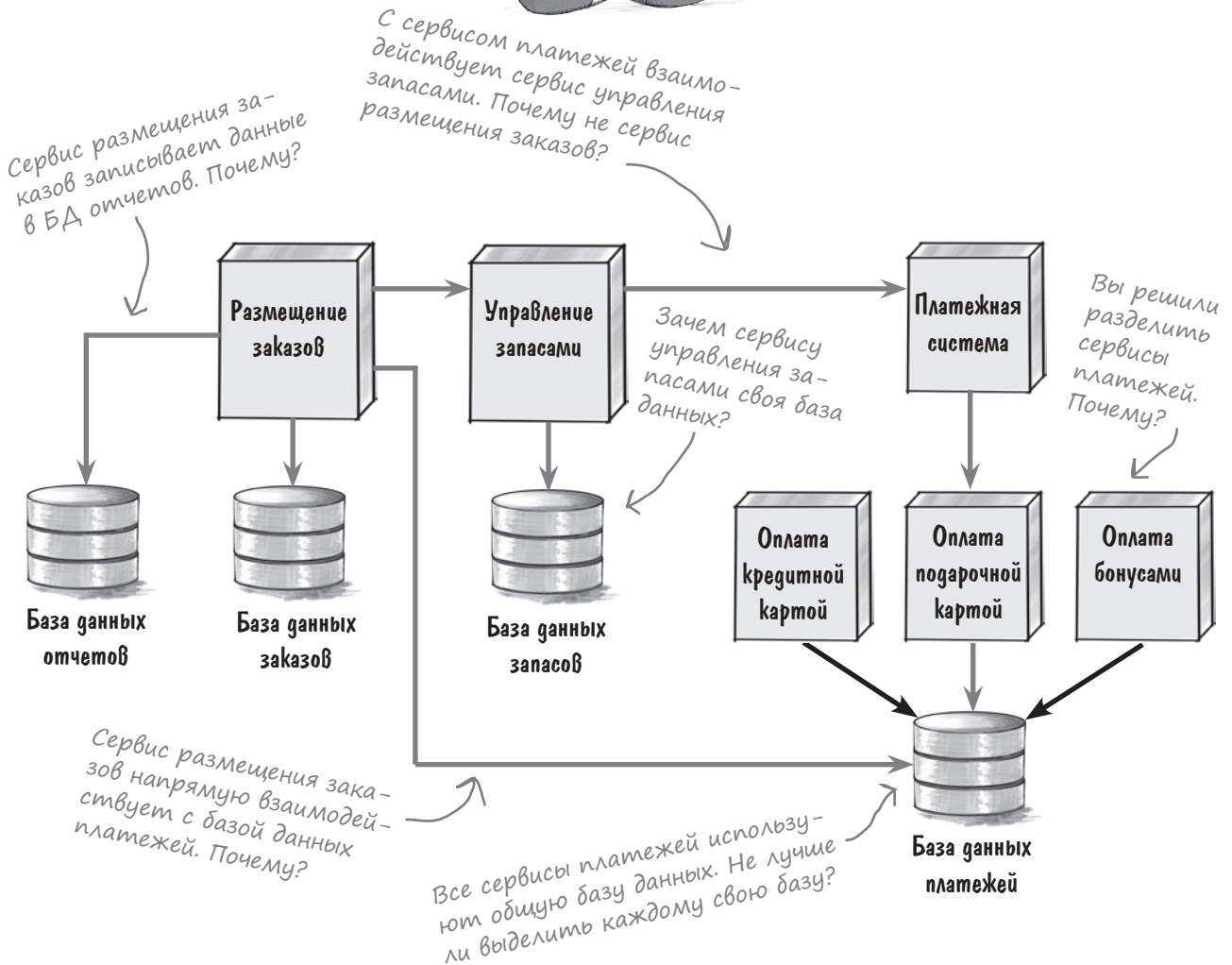
Способность взаимодействовать с другими системами для обработки бизнес-запроса

На с. 41

СТАНЬТЕ архитектором Решение



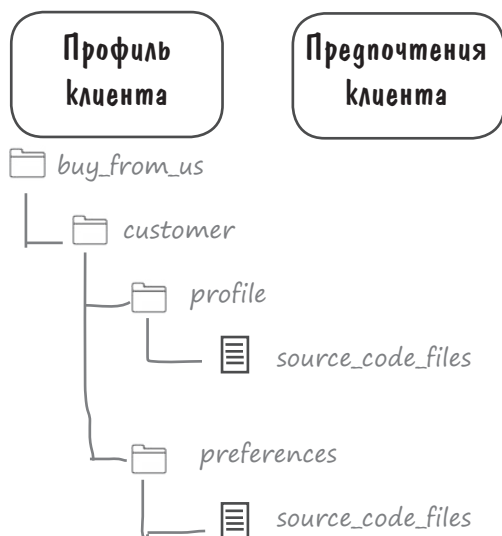
Представьте, что вы архитектор, и определите как можно больше архитектурных решений на схеме ниже. Отметьте все, что, по вашему мнению, является архитектурным решением, и напишите, каким оно может быть.



Возьмите в руку карандаш

Решение

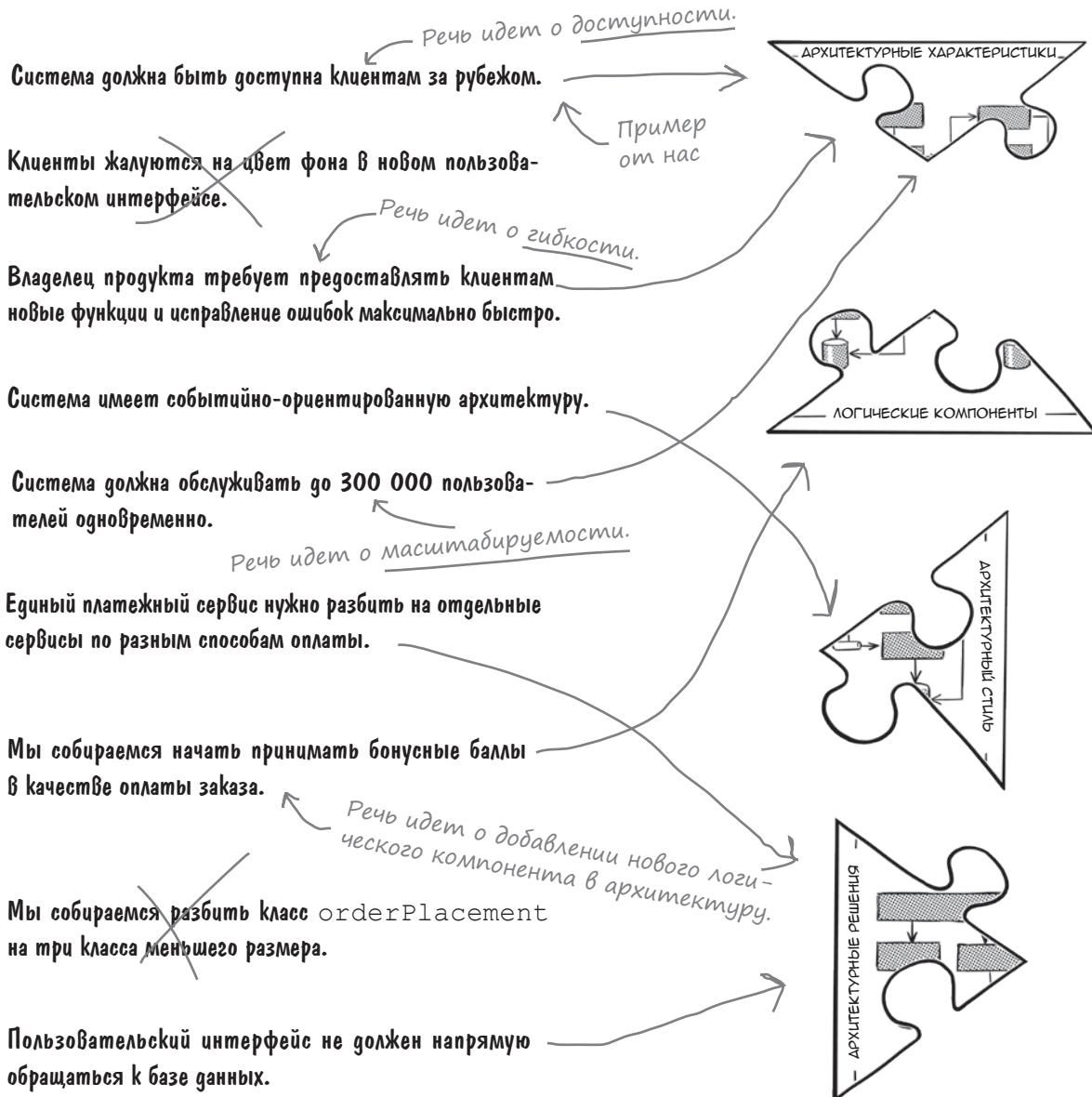
Вы только что определили два компонента для новой системы и дали разработчикам задание написать файлы классов с реализациями. Создайте структуру каталогов, чтобы разработчики могли начать писать код.



Что чему соответствует?

Решение

Мы попытались описать архитектуру, но все фрагменты мозаики перепутались. Помогите нам вспомнить суть каждого измерения, соединив стрелками предложения (слева) с измерениями архитектуры ПО (справа). Будьте осторожны — измерение архитектуры соответствует не каждому предложению.





Упражнение Решение

На с. 49

Отметьте все, что следует добавить в схему *с точки зрения архитектуры*.

- ☒ Порядок взаимодействия сервисов друг с другом
- ☐ Платформа и язык реализации сервисов
- ☒ Какие сервисы могут взаимодействовать с базой данных
- ☒ Сколько сервисов и баз данных использовать

Выбор способа реализации относится к дизайну.



Возьмите в руку карандаш Решение

На с. 50

Обведите решения, которые, по вашему мнению, находятся в середине спектра между архитектурой и дизайном.

Это дизайн.

Разделение класса

Выбор фреймворка хранения данных

Выбор графовой базы данных

Разделение сервиса

Изменение оформления веб-страницы

Выбор фреймворка пользовательского интерфейса

Переход на микросервисы

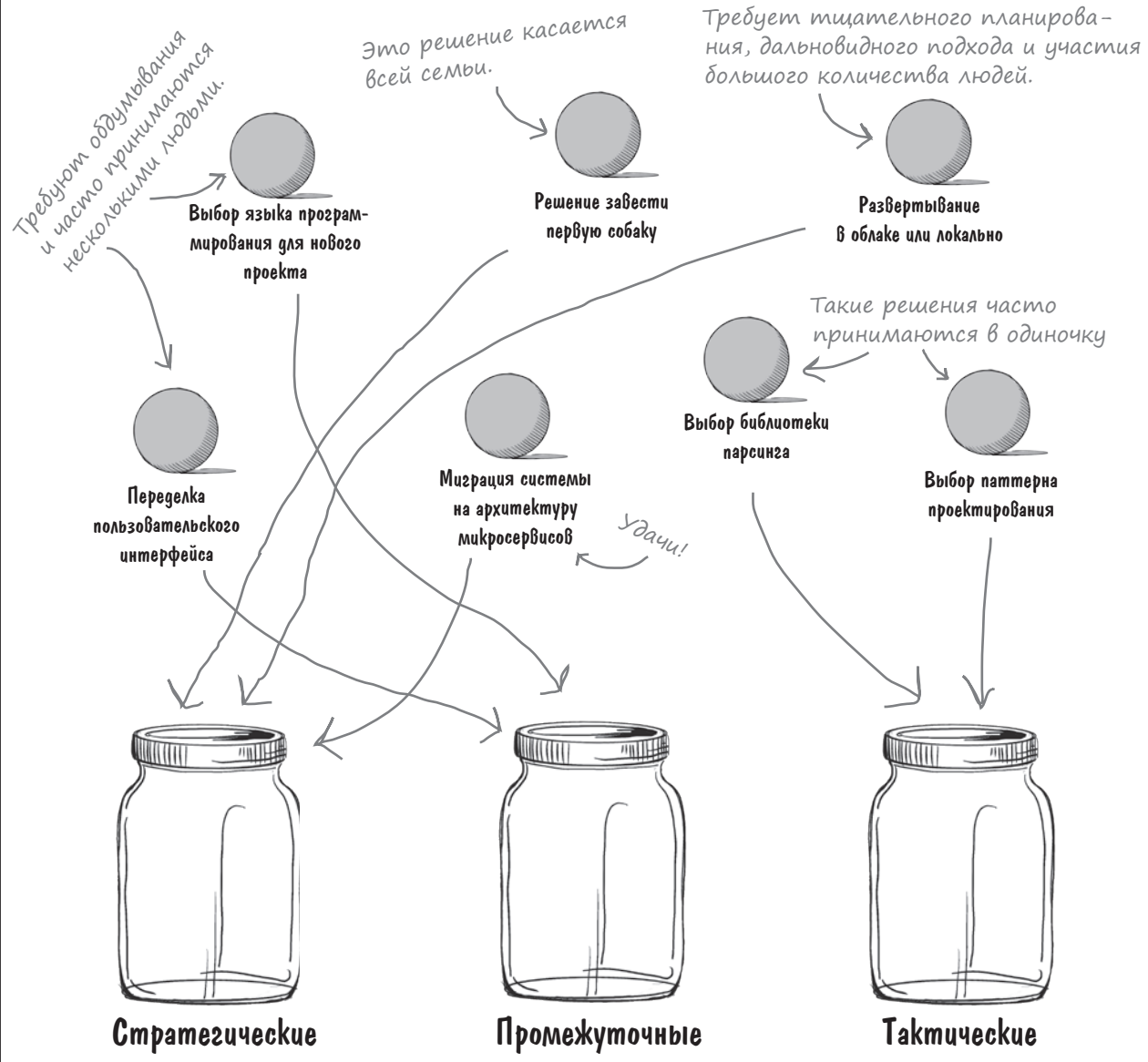
Выбор библиотеки для анализа XML

Это дизайн.

Это архитектура.

Возьмите в руку карандаш Решение

Мы рассыпали шарики и просим вас помочь разложить их по банкам. Используя три вопроса на с. 20, определите, в какую банку следует положить каждый шарик.





Развлечения с магнитами Решение

На с. 55

На нашем холодильнике висели магнитики с пунктами из списка дел в порядке от сложных к простым, но каким-то образом они все упали на пол и перемешались. Помогите нам расположить их в порядке уменьшения требуемых усилий.

Больше усилий

Не верите? Вбейте в поисковике фразу «парадокс выбора».

Требуют больших усилий и поэтому лежат в области спектра ближе к архитектуре.

Находится примерно в середине спектра между архитектурой и дизайном.

Не требуют больших усилий и, следовательно, лежат в области спектра ближе к дизайну.

Выбор горчицы для покупки

Миграция системы в облачную среду

Замена фреймворка пользовательского интерфейса

Миграция с реляционной базы данных на графовую

Разделение одного сервиса на несколько

Разрешение конфликта слияния в Git

Разделение одного класса на несколько

Переименование метода или функции

Меньше усилий



Упражнение Решение

На с. 57

Как же принять все эти решения? Есть способ, позволяющий справиться с данной задачей. Нужно определить решения, предполагающие значимые компромиссы, которые требуют больше времени на обдумывание. Помогите нам определить, какие решения имеют значимые компромиссы, а какие нет.

Это значимые компромиссы?

☐ Да ☒ Нет Выбор, какую одежду надеть сегодня на работу

☒ Да ☐ Нет Выбор между развертыванием в облаке и локальным

☐ Да ☒ Нет Выбор фреймворка пользовательского интерфейса

☐ Да ☒ Нет Выбор имени для переменной в файле класса

☐ Да ☒ Нет Выбор между сливочным и шоколадным мороженым

☒ Да ☐ Нет Выбор архитектурного стиля

☒ Да ☐ Нет Выбор между REST и обменом сообщениями

☒ Да ☐ Нет Передача в сообщениях полных данных или только ключей

☐ Да ☒ Нет Выбор библиотеки анализа XML

☒ Да ☐ Нет Решение о разделении сервиса

☒ Да ☐ Нет Выбор между атомарными и распределенными транзакциями

☐ Да ☒ Нет Решение, идти ли вечером на ужин

Иногда принять это решение действительно трудно.

Здесь есть свои компромиссы, так что любой ответ оправдан.

Может повлиять на масштабируемость, производительность и удобство сопровождения.

Вы уже проголодались?

Может повлиять на целостность данных, а также на масштабируемость и производительность.

Верно или неверно Решение

На с. 59

Верно Неверно Дизайн описывает структуру дома (стены, крыша, планировка и т. д.), а архитектура ПО подобна мебели и отделке.

Все совсем наоборот.

Верно Неверно Большинство решений относится исключительно к архитектуре или дизайну. Очень мало решений, находящихся посередине.

Большинство решений расположены между архитектурой и дизайном.

Верно Неверно Чем более стратегическим является решение, тем больше оно относится к архитектуре; более тактическое решение ближе к дизайну.

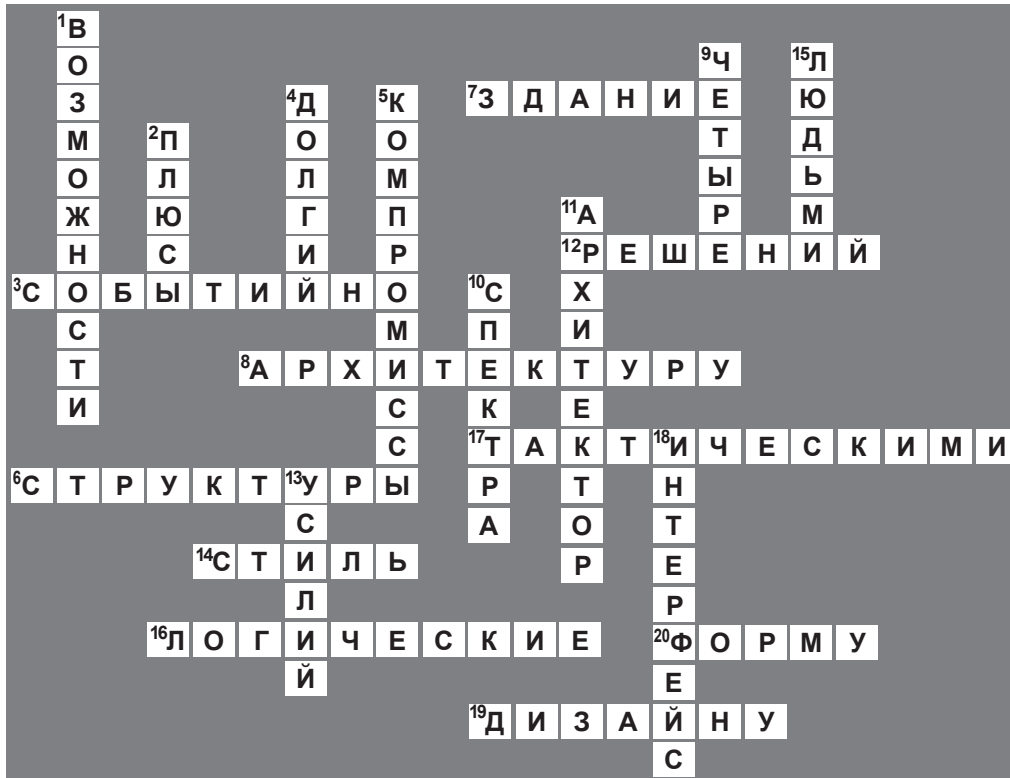
Верно Неверно Чем больше усилий требуется для реализации или изменения решения, тем ближе оно к дизайну; решения, требующие небольших усилий, ближе к архитектуре.

Все совсем наоборот.

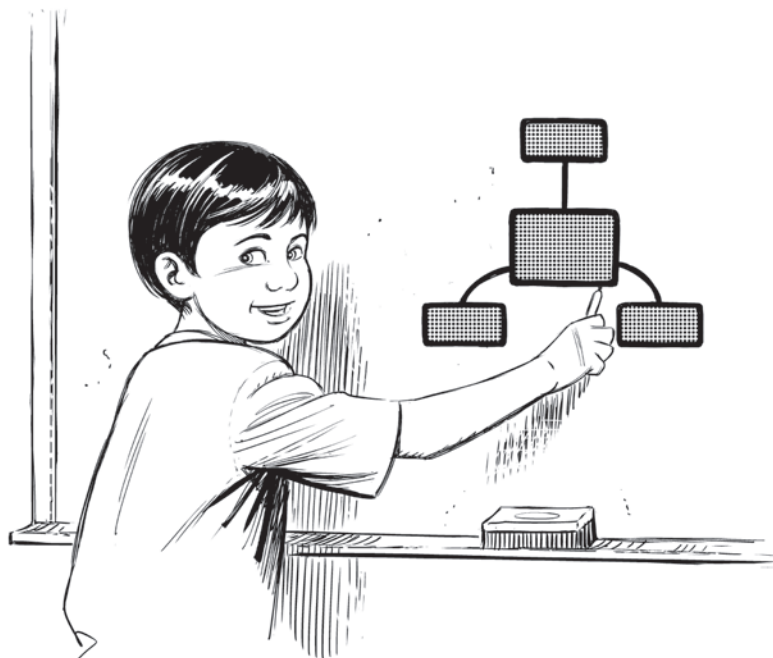
Верно Неверно *Компромиссы* — это плюсы и минусы решения или задачи. Чем ближе решение к архитектуре, тем существеннее компромиссы.

Архитектурный кроссворд. Решение

На с. 60



Знайте свои возможности



Что должна поддерживать архитектура? Архитектурные характеристики (возможности архитектуры) — фундаментальные строительные блоки любой системы. Без них невозможно принимать архитектурные решения, выбирать архитектурные стили и вообще создавать логичные архитектуры. В этой главе вы узнаете, как определить наиболее распространенные характеристики (масштабируемость, надежность, тестируемость), как они влияют на архитектуру ПО, как помогают принимать архитектурные решения и какие из них наиболее важны в конкретной ситуации. Готовы добавить возможности в архитектуру своего ПО?

Причины создания социальной сети Lafter

Sillycon Symposia — это стартап, в бизнес-плане которого конференции на технологические темы сочетаются с комедийными постановками. Собирая единомышленников, Sillycon делает уникальные предложения участникам и поддерживает их вовлеченность, заставляя смеяться.

Сложно ли создать социальную сеть?!

Часть бизнес-плана отведена созданию социальной сети Lafter для участников конференций, которые проводит Sillycon (и не только он). Заинтересованные лица составили **документ с требованиями** к ней:

К счастью, логотип они придумали. А остальное зависит от вас.



Стандартный уровень детализации для документа с требованиями

Sillycon Symposia организует Lafter — социальную сеть единомышленников.

Пользователи: сотни докладчиков, тысячи участников.

Требования:

- Пользователи могут регистрироваться, согласившись с условиями.
- Пользователи могут добавлять свой контент в виде «Шуток» (длинные статьи) или «Каламбуров» (короткие заметки).
- Подписчики могут выражать сильное («Ха-ха») или более сдержанное («Хи-хи») одобрение понравившегося контента.
- Докладчики получают специальный значок Sillycon Symposia.
- Докладчики могут создавать форумы для обсуждения выступления.
- Пользователи могут оставлять сообщения длиной до 281 символа.
- Пользователи могут оставлять ссылки на внешние ресурсы.

Дополнительно:

- Поддержка интернационализации.
- Небольшой штат команды поддержки.
- «Взрывной» трафик во время проведения конференций.

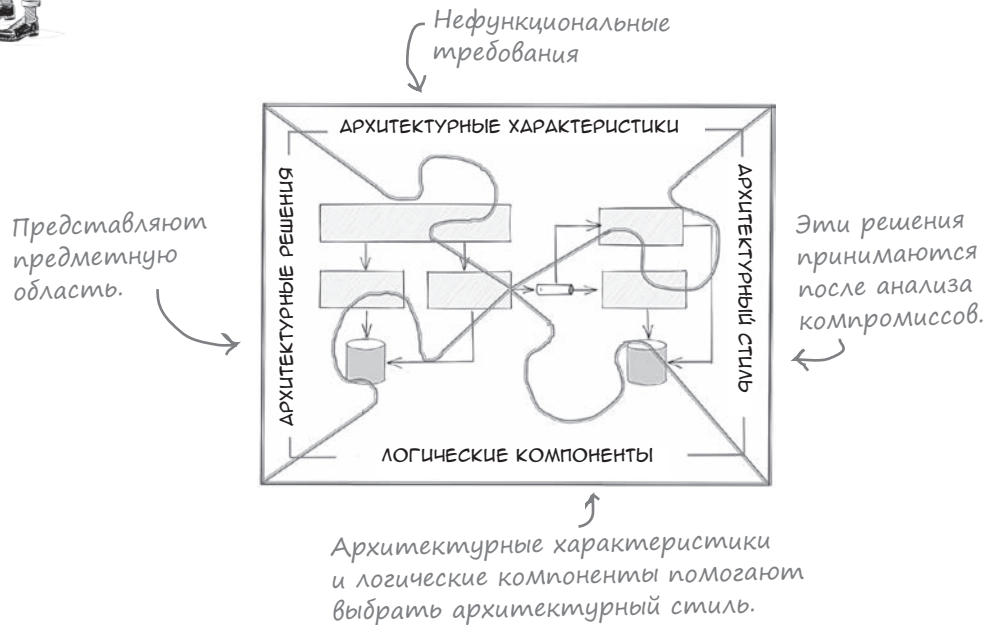
Разговор в кабинете



Алекс. Посмотрите, что пришло на мой имейл: руководство предлагает мне стать архитектором социальной сети Lafter для Sillycon Symposia.

Сэм. Они прислали требования? Ты можешь сразу же приступить к проектированию системы — я не вижу ничего сложного.

Мара. На самом деле такой подход возможен только для очень простых приложений, и это не тот случай. Помните схему, которую я нарисовала пару дней назад?



Перед выбором архитектурного стиля нужно проанализировать архитектурные характеристики и логические компоненты.

Почти любое приложение можно реализовать в любом архитектурном стиле, но некоторые стили подходят лучше. Выбор стиля до анализа — это классический пример того, как телегу ставят впереди лошади.

Сэм. Но можно же начать с чего-то небольшого, а затем постепенно наращивать возможности, пока не получится вся система.

Мара. Нет, такой итеративный подход не подходит для анализа архитектурных характеристик — трудно сделать систему масштабируемой, если она не предусматривает этого изначально.

Алекс. Ты права. Думаю, мне стоит сначала проанализировать архитектурные характеристики. Спасибо!

Что такое архитектурные характеристики

Вам поставили задачу, для решения которой нужно написать программу. Эта задача называется *предметной областью*, и именно ее проектированием вы будете заниматься большую часть времени. Однако архитектор должен обратить внимание не только на ее особенности. Он должен проанализировать еще и *архитектурные характеристики*. Ниже приведены несколько примеров архитектурных характеристик, показывающих, что разные предметные области имеют разные, но часто пересекающиеся множества архитектурных характеристик.

Они составляют одно из измерений, помогающих описать архитектуру.

Обратите внимание, что многие из них заканчиваются на «-ость».

Контролируемость

Банки должны давать возможность проверки транзакций.

Безопасность

Банки должны соблюдать требования безопасности, чтобы защищать финансовые интересы.



Банк

Целостность данных

Финансовые операции должны быть точными и непротиворечивыми.

Масштабируемость

Онлайн-аукционы должны обслуживать большое количество участников одновременно.

Банки должны обслуживать большое количество клиентов одновременно.

Обе предметные области имеют общую архитектурную характеристику, которую нужно поддерживать, — масштабируемость.

Юзабилити

Сайты аукционов должны быть простыми и позволять быстро делать ставки.



Онлайн-аукцион

Согласованность

Чтобы аукционы работали, ставки должны фиксироваться последовательно.

Надежность

Сайт аукциона должен быть надежным — пользователи огорчатся, когда связь обрывается в разгар торгов.

Определение архитектурных характеристик

Часть работы архитектора — проектирование структуры программной системы и определение двух ее составных частей: *логических компонентов* и *архитектурных характеристик*. Логические компоненты (рассматриваются в главе 4) представляют предметную область приложения — причину создания программной системы. Объединив эти части, вы получите структурные черты архитектуры.

Определение архитектурных характеристик — важная часть процесса конструирования программной системы для любой предметной области, поскольку они представляют ее возможности, внутреннюю структуру и другие особенности.

На следующих страницах мы рассмотрим множество примеров архитектурных характеристик, но сначала раскроем само понятие.

Мы определяем **три грани** архитектурных характеристик (см. ниже).



Рассмотрим каждую из граней отдельно.

Характеристики не связаны с предметной областью

Чтобы определить архитектурные характеристики, сначала нужно понять, чем они *не являются*. Требования определяют, что должно делать приложение, а *архитектурные характеристики* — критерии успешной работы, способы реализации требований и причины того или иного выбора. Например, производительность приложения — важная архитектурная характеристика, которая часто отсутствует в документах с требованиями.

Проектирование структуры в архитектуре можно разделить на *аспекты, связанные и не связанные с предметной областью*. Архитектурные характеристики отображают усилия по созданию возможностей, необходимых для успешной реализации проекта.



Мара. Бизнес-аналитики и профильные эксперты усердно трудились над созданием документа с требованиями и начальным проектом предметной области. Но мы должны вместе с ними выяснить, какие архитектурные характеристики следует поддерживать.

Сэм. Разве это не часть предметной области? Зачем привлекать архитектора на этом этапе?

Алекс. Наши бизнес-аналитики никогда не работали над созданием программного обеспечения, поэтому вряд ли поймут, как одно решение влияет на другое.

Мара. Это верно. Часто аспект, для бизнесмена являющийся незначительным, для архитектора имеет большое значение! Желания представителей бизнеса может быть трудно поддерживать в архитектуре. Вот почему так важно привлекать архитекторов на самых ранних этапах проектирования.

Сэм. И что мы ищем?

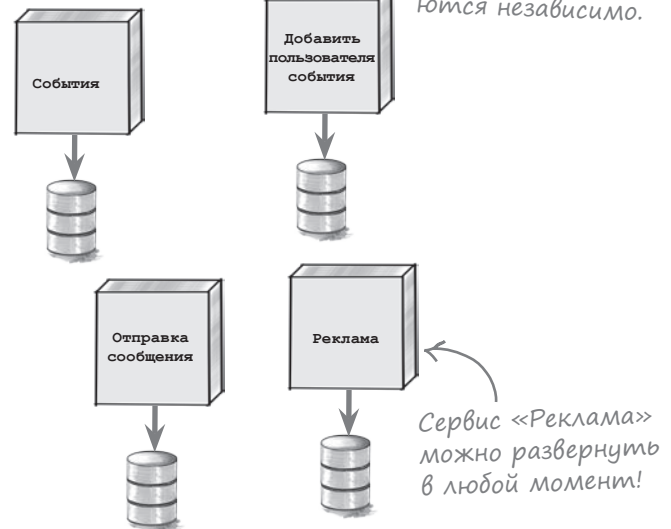
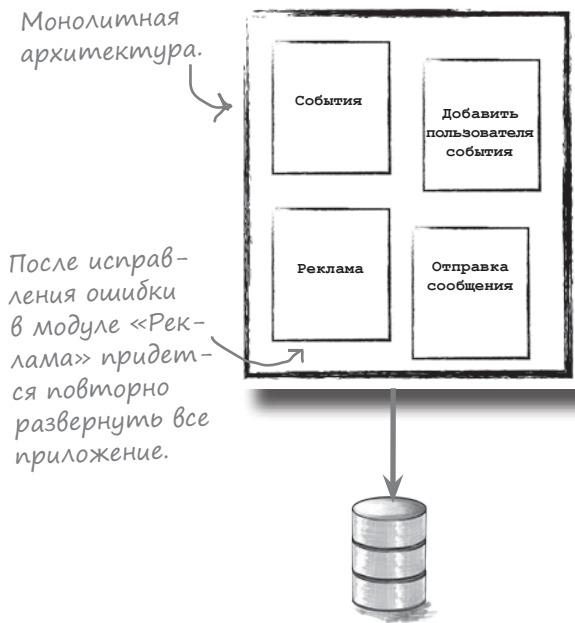
Алекс. В определении *архитектурных характеристик* говорится, что они «не связаны с предметной областью». Посмотрим, что придумали аналитики и эксперты и учли ли они такие аспекты, как производительность и масштабируемость.

Характеристики влияют на структуру архитектуры

Есть главная причина, по которой важно описывать архитектурные характеристики, — они могут требовать специальной структурной поддержки. Например, безопасность — стандартная проблема, и при разработке любой системы должны предприниматься элементарные меры предосторожности. Однако безопасность становится *архитектурной* характеристикой, когда архитектор должен приложить особые усилия для ее воплощения.

Рассмотрим следующие варианты архитектуры Lafter, содержащие возможности и правила проведения рекламных акций. Архитектор может спроектировать все это как *монолитную архитектуру*, развертываемую как единое целое и имеющую соответствующую базу данных, или как группу независимых сервисов.

Если использовать монолитную архитектуру, то при изменении правил проведения акций придется заново развернуть все приложение, поскольку монолиты создаются и развертываются как единое целое. А вот в *распределенной архитектуре* достаточно будет развернуть только сервис «Реклама».



Принимая архитектурные решения, необходимо учитывать множество компромиссов, например выбор между монолитной и распределенной архитектурами.

Избегайте чрезмерного усложнения

Приложения *могут* поддерживать много архитектурных характеристик, но... не должны этого делать. Ведь каждая характеристика, которую нужно поддерживать, усложняет систему.

Большое количество и разнообразие архитектурных характеристик означает наличие множества заманчивых вариантов. Но мы как архитекторы должны стараться *уменьшить* их количество, поскольку архитектурные характеристики:



Невозможно стандартизировать

Разные организации могут по-разному называть одни и те же характеристики. Например, под *производительностью* и *отзывчивостью* может подразумеваться одно и то же.

Мы говорим о третьей грани — и почти у цели!

Желательно создать общий словарь с архитектурными характеристиками для организации — это поможет составить свой стандартный список.

Обладают синергией

Вывод: часто невозможно выбрать архитектурную характеристику, не учитывая ее влияние.

Архитектурные характеристики влияют на другие характеристики и аспекты предметной области. Например, усиление безопасности приложения почти наверняка ухудшит его производительность (увеличение объема шифрования в процессе выполнения и другие подобные изменения повлекут снижение производительности).

Избыточны

Количество архитектурных характеристик велико, и постоянно возникают новые. Например, еще недавно не было понятия эластичности по требованию, введенного облачными провайдерами.

Даже количество категорий увеличилось за последние несколько лет — к существующим добавились ограничения и возможности облачных вычислений.

Одна из подстерегающих нас опасностей — *чрезмерное усложнение*: поддержка слишком большого количества архитектурных характеристик и усложнение проекта до потери почти всех преимуществ. Знание того, какие характеристики *важны для успешной работы приложения*, действует как фильтр, помогая *исключать* функции в целом полезные, но слишком усложняющие систему.

Совсем скоро мы обсудим некоторые категории.

Остерегайтесь выбора технологий «ради строчки в своем резюме»! Осваивать новое интересно, и мы, конечно же, должны продолжать учиться, но поддержка слишком большого количества архитектурных характеристик не соответствует более высоким приоритетам и не поможет приложению успешно работать.

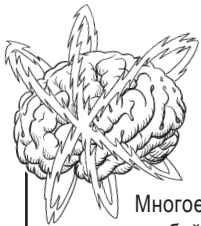


Будьте
осторожны!

Синергия может быть опасна!

Архитекторы предпочли бы проектировать на основе одних только архитектурных характеристик, без учета особенностей предметной области. Увы, это невозможно. Говоря о *синергии* архитектурных характеристик, мы имеем в виду, что их изменение может потребовать изменения других характеристик или особенностей предметной области. Никакой, даже самый умный, архитектор не сможет сделать все архитектуры масштабируемыми — некоторые из них не способны масштабироваться из-за физических ограничений, таких как память и пропускная способность.

Будьте осторожны, изменяя архитектурную характеристику. Подумайте, как изменение повлияет на другие части архитектуры. То же относится к изменениям в предметной области, таким как границы и распределение компонентов: они тоже могут синергетически влиять на архитектурные характеристики. Например, если вы решите хранить в приложении платежную информацию пользователей, то архитектурные характеристики безопасности и целостности данных тоже изменятся.



Мозговой шторм

Многое в реальности обладает синергией, когда объединение дает нечто большее, чем сумма компонентов. Попробуйте привести несколько примеров синергии из реальной жизни. *Подсказка: это могут быть компоненты, которые легко опознать (например, арахисовое масло и шоколад) или неразличимые в смеси (например, эмульсии, образованные смешиванием масла и уксуса).*

Запишите свои
идеи здесь.



Учитывайте явные и неявные возможности

Что-то может быть *явным* — четко сформулированным, а что-то *неявным* — подразумеваемым на основе контекста и других знаний. Представьте, что вы увидели кучу коробок и конвертов перед дверью дома. Что бы вы подумали?

Явное

Коробки сложены снаружи.

Дверь заперта.



Неявное

Никого нет дома.

Жильцы хорошо закупились в онлайн-магазине.

Жильцы уехали в отпуск?

Явные архитектурные характеристики указываются в требованиях к приложению.

Неявные архитектурные характеристики — это факторы, влияющие на решения архитектора, но не упоминающиеся в требованиях. Безопасность часто не указывается явно в требованиях, но архитекторы знают, что система должна быть безопасной.

Выявить эти архитектурные характеристики на этапе анализа помогает знание предметной области. Например, компания высокочастотного трейдинга может не указывать, насколько важно завершение транзакций в течение миллисекунд, но архитекторы в данной области знают, что это очень важно.



Часть Задаваемые Вопросы

В: А что насчет таких важных аспектов, как хорошая внутренняя структура, которые редко указывают в требованиях?

О: Некоторые неявные архитектурные характеристики не бросаются в глаза, но от этого не теряют важности. Например, архитекторы должны следить за внутренней структурой приложения в процессе его разработки, чтобы невнимательность и небрежное отношение разработчиков не ухудшили долговечность приложения. Однако редко в каких требованиях будет указано: «Сохраняйте внутреннюю модульность системы в процессе ее создания!» или «Создавайте систему удобной для сопровождения!».

Международный зоопарк характеристик

Подобно зоопарку, в котором содержится широкий спектр животных от рептилий до приматов, мир архитектурных характеристик тоже содержит много разновидностей: от низкоуровневых, таких как модульность, до высокоуровневых и сложных, таких как масштабируемость и эластичность. К сожалению, нет «универсального списка» архитектурных характеристик, как нет и стандартных определений обозначающих их терминов (хотя многие пытались дать их). Вместо этого каждая организация интерпретирует термины по-своему.

Кроме того, экосистема программного обеспечения постоянно добавляет новые термины и понятия, давая возможность определить еще больше архитектурных характеристик.

Думаете, это большой список? Загляните на <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>.

масштабируемость	доступность	наблюдаемость
доступность	удобство сопровождения	тестируемость
интегрируемость	многообразие	переносимость
защищенность	простота	осуществимость
устойчивость	надежность	юзабилити
гибкость	целостность	производительность
трассируемость	локализуемость	контролируемость

Это не полный список. Полного списка не существует!



Сэм. Мы должны определить архитектурные характеристики для Lafter, но я нигде не могу найти стандартный список.

Алекс. Боже, как их много...

Мара. Вот почему я стараюсь классифицировать их. Помните старые карты зоопарков с изображениями «домиков» и «вольеров» для разных животных? Мы можем применить ту же классификацию по «родам и видам» архитектурных характеристик.

«Дамы и господа, мальчики и девочки всех возрастов, добро пожаловать в международный зоопарк "характеристик"!»



Эксплуатационные

Технологические



Структурные

Сквозные

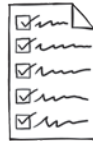
Технологические архитектурные характеристики

Технологические архитектурные характеристики находятся на пересечении процесса разработки с архитектурой программного обеспечения. Они отражают решения, касающиеся механики создания ПО.



модульность

Степень разделения ПО на отдельные компоненты. Модульность влияет на разделение функций и организацию логических строительных блоков.



тестируемость

Доступность и простота модульного, функционального, приемочного и исследовательского тестирования системы.

Имеется в виду тестирование во время разработки (например, модульное), а не формальный контроль качества.



гибкость

Составная характеристика, объединяющая тестируемость, разворачиваемость, модульность и другие архитектурные характеристики, позволяющие использовать методы гибкой разработки.

Гибкость — составная архитектурная характеристика. О ней мы поговорим далее в этой главе.

Упоминается также на следующей странице. Как вы увидите далее, многие архитектурные характеристики попадают в несколько категорий.



разворачиваемость

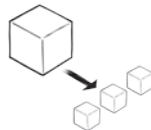
Простота и удобство развертывания программной системы.

Одна из многих архитектурных характеристик, составляющих понятие «гибкость».



расширяемость

Простота расширения программной системы. Может охватывать ее структуру, внутренний дизайн, инженерные практики и управление.



разделимость

Связанность описывает, насколько тесно соединены части системы. Некоторые архитектуры определяют способы разделения, позволяющие достичь некоторых преимуществ. Эта архитектурная характеристика определяет степень возможности такого разделения.

Мы знаем, что это выдуманное слово. Такое часто случается в архитектуре ПО!

Структурные архитектурные характеристики

Структурные архитектурные характеристики влияют на внутреннюю структуру программной системы, в том числе на степень связанности ее компонентов и отношения между разными точками интеграции.



защищенность

Степень защищенности системы. Например, шифрование данных в базе и при передаче между внутренними системами, аутентификация для удаленного доступа пользователей.

Явно или неявно защищенность присутствует в каждом приложении.



удобство сопровождения

Насколько легко вносить изменения в целях улучшения системы и/или исправления ошибок.



Переносимость может относиться к любой части системы, в том числе к пользовательскому интерфейсу и платформе реализации.

Одна из характеристик, которые относятся сразу к нескольким категориям.



расширяемость

Простота расширения программной системы. Может охватывать ее структуру, внутренний дизайн, инженерные практики и управление.

Некоторые архитектурные характеристики затрагивают не только предметную область, но и проблемы разработки.



переносимость

Простота запуска системы на нескольких платформах (например, Windows и macOS).



локализация

Поддержка нескольких языков, единиц измерения, валют и другие факторы, позволяющие использовать систему в глобальном масштабе.

Другой родственный термин — интернационализация (i18n).

Эксплуатационные архитектурные характеристики

Эксплуатационные архитектурные характеристики определяют влияние архитектурных решений на работу группы эксплуатации.



доступность

Какой процент времени система должна быть доступна? Насколько быстро ее можно запустить после сбоя при круглосуточной эксплуатации?

Обычно определяется количеством «девяток» (99,999 % = 5 девяток, равноценно 6 минутам простоя в год).



восстанавливаемость

Скорость восстановления работоспособности в случае аварии. Эта характеристика влияет на стратегию резервного копирования и требования к дублирующему оборудованию.

Хороший пример действия аксиомы, согласно которой можно взять любое прилагательное, добавить «-ость» и получить новую архитектурную характеристику!



устойчивость

Способность обрабатывать ошибки и граничные условия, например, такие как сбой питания, потеря подключения к сети или выход из строя оборудования.

Когда они важны, они очень важны.



производительность

Соответствие системы требованиям к времени решения задач при использовании доступных ресурсов.

Как вы увидите далее, «производительность» имеет множество облиций.



надежность/безопасность

Требования к безотказности и сохранению работоспособности. Если система выйдет из строя, то поставит ли это под угрозу жизни людей или принесет ли компании значительный убыток? Обычно эта характеристика касается медицинских систем, ПО для больниц и самолетов.

Одни «-ости» воплотить проще, чем другие. Эта — одна из самых сложных.



масштабируемость

Сохранение высокой производительности при увеличении количества пользователей или запросов.

Приложению *Lafter* она обязательно понадобится!

Сквозные архитектурные характеристики

Как бы нам ни хотелось иметь красивый, ухоженный зоопарк архитектурных характеристик, среди них все равно встретятся не поддающиеся классификации.



защищенность

Степень защищенности системы. Например, шифрование данных в БД и при передаче между внутренними системами, аутентификация для удаленного доступа пользователей.

Одна из архитектурных характеристик, присутствующих всегда и являющихся сквозной функциональностью.

Аутентификация и авторизация — аспекты защищенности.



законность

Соответствие системы местным законам о защите данных, о том, как должно создаваться или развертываться приложение.



аутентификация/авторизация

Способность системы идентифицировать пользователей и предоставлять доступ только к определенным функциям (с учетом варианта использования, подсистемы, веб-страницы, бизнес-правил, уровня привилегий и т. д.).

Многие госучреждения по всему миру должны обеспечивать базовый уровень доступности.



конфиденциальность

Соккрытие и шифрование транзакций, чтобы внутренние сотрудники, такие как операторы, архитекторы и разработчики, не могли их видеть.

Строгость законов многих стран, касающихся конфиденциальности, затрудняет реализацию приложений для международного рынка.



доступность

Доступность системы для пользователей, в том числе для людей с ограниченными возможностями, такими как дальтонизм или потеря слуха.



юзабилити

Отличный пример неоднозначности некоторых архитектурных характеристик: юзабилити может относиться и к дизайну пользовательского интерфейса.

Насколько легко пользователи решают свои задачи? Нужно ли обучение? К юзабилити нужно относиться так же серьезно, как к любой другой архитектурной характеристике.

Что чему соответствует?

Архитектурные характеристики так многообразны! Мы составили список тех из них, которые наиболее важны для Lafter, вместе с их определениями, но потеряли его. Помогите нам восстановить этот список, нарисовав стрелки, связывающие архитектурные характеристики с соответствующими определениями.

Масштабируемость

Описывает, насколько хорошо компоненты группируются и имеют четко очерченные границы.

Развертываемость

Способность системы восстанавливаться после сбоя питания, потери подключения к сети или выхода из строя оборудования.

Модульность

Доступность системы для пользователей, в том числе для людей с ограниченными возможностями, такими как дальтонизм или потеря слуха.

Устойчивость

Насколько легко вносить изменения в целях улучшения системы и/или исправления ошибок.

Доступность

Описывает, насколько хорошо система справляется с обслуживанием большого количества пользователей, работающих одновременно.

Удобство сопровождения

Описывает периодичность, эффективность и воспроизводимость развертываний.

→ Решение на с. 106

Часто Задаваемые Вопросы

В: Где можно найти стандартный список архитектурных характеристик?

О: Стандартного списка не существует (несмотря на несколько тщетных попыток создать его), поскольку экосистема разработки ПО постоянно меняется.

В: Разве защищенность нужна не во всех приложениях?

О: Зависит от обстоятельств! Если вы создаете систему бесплатного заказа ланчей внутри офиса, то единственной проблемой будет сохранение в тайне от других, что вы каждый день заказываете сэндвич с яичным салатом.

В: Разве не каждое приложение требует доступности?

О: И снова все зависит от обстоятельств! Доступность — общая проблема для большинства приложений. Но если наша мифическая система заказа сэндвичей выйдет из строя, то единственной проблемой станет то, что каждому сотруднику придется покупать себе обед самостоятельно.

В: Могу ли я выбрать любую комбинацию архитектурных характеристик для своего приложения?

О: Некоторые архитектурные характеристики плохо сочетаются. Например, совмещение высокой производительности и масштабируемости архитекторы считают трудной задачей. Отбор наиболее важных архитектурных характеристик — это лишь часть процесса проектирования. Их объединение с логическими компонентами поможет вам выбрать подходящий архитектурный стиль.

В: Что получится, если не добавлять в требования такую архитектурную характеристику, как доступность?

О: Отобранные вами архитектурные характеристики служат ориентиром для выбора архитектурного стиля. Если архитектор опустил доступность, это не значит, что он намеренно проектирует систему с низкой доступностью. Скорее это показатель компромисса: выбор одной архитектурной характеристики вместо другой.

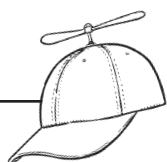


Упражнение

Добро пожаловать в игру «одно из двух». Ее правила просты: слева даны бизнес-требования к приложению Lafter, а справа — архитектурные характеристики. Как известно, в архитектуре все является компромиссом, поэтому, оптимизируя одно, вы ухудшите другое. Ваша задача — сообщить нам, какую характеристику вы считаете более важной для того или иного требования. Наши решения вы найдете в конце главы.

«Мы должны выйти на рынок как можно скорее!»	отказоустойчивость	гибкость
«У нас туго с деньгами!»	масштабируемость	простота
«Эта конференция обещает стать самой масштабной».	высокая доступность	простота сопровождения
«Мы будем хранить информацию о кредитных картах».	защищенность	восстанавливаемость
«Этот сайт будет очень популярен».	гибкость	эластичность

→ Решение на с. 107



Серьезное программирование

В мире архитектуры ПО нет стандартного термина для обозначения *архитектурных характеристик*. Приведем некоторые термины, часто используемые другими, и причины, почему они нас не устраивают.

Многие называют их *нефункциональными требованиями*, что вводит в заблуждение, поскольку на самом деле архитектурные характеристики — функциональны, просто не относятся к предметной области. Слово «нефункциональные» преуменьшает их важность. Другие называют их *атрибутами качества системы*, что подразумевает нечто, определяемое в конце проекта, а не в начале. Еще одно название — *сквозные требования*. В нем нам не нравится слово «*требование*», связывающее его с предметным поведением, исходящим из требований, а не с *возможностями*, которые определяются архитектурными характеристиками.



Архитектурные характеристики не появляются из ниоткуда.

На самом деле есть три источника их возникновения.

1 Предметная область

Анализ задачи помогает выяснить, какие архитектурные характеристики необходимы системе. Вообще говоря, многие структурные решения основываются на предметной области.

2 Знание окружения

Многие требования обусловлены хорошим знанием особенностей окружения, в котором вы работаете, например, в быстро развивающемся стартапе или на крупном предприятии списки требований будут разными.

Некоторые архитектурные характеристики — «неявные» и происходят из этих двух источников.

3 Целостное знание о предметной области

Разрабатывая ПО, вы работаете с конкретной задачей, но уверяем вас, сама предметная область намного шире. Допустим, вы создаете платежную систему. Понимание требований безусловно важно, но вы обнаружите новые архитектурные характеристики, если изучите финансовый мир, правила финансовой отрасли и привычки клиентов.

Рассмотрим каждый источник по очереди.

Получение архитектурных характеристик из предметной области

Многие характеристики извлекаются из предметной области — в конце концов, именно ее потребности стали причиной написания ПО. Для этого требования нужно *преобразовать* в соответствующие архитектурные характеристики. Например, в требованиях к Lafter указано «тысячи пользователей». Как архитектор, вы должны более точно определить, сколько пользователей ожидается (масштабируемость), сколько их будет участвовать в конференциях (параллелизм) и насколько быстро они будут подключаться (эластичность).



Упражнение

Требования часто являются богатым источником архитектурных характеристик. Например, приложение Lafter должно поддерживать большое количество пользователей, поэтому масштабируемость — одна из обязательных характеристик. Попробуйте определить другие характеристики.

Пользователи: сотни докладчиков, тысячи участников.

Требования:

Пользователи могут регистрироваться, согласившись с условиями.

Пользователи могут добавлять свой контент в виде «Шуток» (длинные статьи) или «Каламбуров» (короткие заметки).

Последователи могут выражать сильное («Ха-ха») или более сдержанное («Хи-хи») одобрение понравившегося контента.

Докладчики получают специальный значок Sillycon Symposia.

Докладчики могут создавать форумы для обсуждения выступления.

Пользователи могут оставлять сообщения длиной до 281 символа.

Пользователи могут оставлять ссылки на внешние ресурсы.

Дополнительно:

Поддержка интернационализации.

Небольшой штат команды поддержки.

«Взрывной» трафик во время проведения конференций.

**Архитектурные
характеристики**

Масштабируемость

→ Решение на с. 108

Получение архитектурных характеристик из знаний об окружении

Вы много знаете о своей организации (иногда слишком много!), и это облегчает выбор архитектурных характеристик. Например, архитектор в быстро развивающемся стартапе будет уделять много внимания гибкости, независимо от того, указана ли она в требованиях.

Принятие долгосрочных решений требует понимания **приоритетов организации**. Например, предположим, что нам нужно интегрировать две подсистемы. Для этого можно выбрать нестандартный, но хорошо подходящий протокол, или стандартный, но несколько неудобный. Мы могли бы не учитывать реалии и выбрать первое решение. Однако зная, что целью организации является активное сотрудничество с другими компаниями, мы можем предпочесть более открытое решение.

Архитекторы принимают решения не в вакууме — контекст всегда важен.

Вдобавок это означает, что вам придется внимательно следить за встречами, где определяются приоритеты бизнеса!

Некоторые архитекторы остаются в рамках одной области именно потому, что имеют глубокие знания о ней.

Получение архитектурных характеристик из целостных знаний о предметной области

Кроме того, вы наверняка впитали немало **знаний о предметной области**: информацию, не указанную явно в требованиях, но определяющую важные аспекты этой области.

Допустим, в Lafter решили провести рекламную акцию в местном университете и побудить студентов зарегистрироваться (они посещают конференции, и многие имеют чувство юмора). Мы должны создать приложение для обработки регистрации в день акции. Предположим, в университете учатся 1000 студентов и им дается 10 часов для регистрации. Можем ли мы предположить, что студенты будут регистрироваться равномерно? (Вам приходилось *встречать* студентов университета?)

Учитывайте способность некоторых студентов оттягивать время.

Одно из самых опасных открытий, что можно откладывать до последнего и при этом выполнять работу в срок.

Основываясь на своем опыте, сразу скажем, что такое предположение будет ошибкой. Вспомните, что вы знаете о студентах. Одни очень исполнительные, другие склонны к затягиванию. Поэтому в реальности следует ожидать всплеска в первый час (когда поспешат зарегистрироваться наиболее исполнительные), затем долгого затишья и, наконец, еще одного всплеска перед закрытием окна регистрации, когда поспешат зарегистрироваться отстающие.

Архитекторы должны использовать все доступные источники информации и выявить все компромиссы архитектурных решений.



Будьте
осторожны!

Решения и требования

Заказчики часто приходят к архитекторам с *решениями*, а не с требованиями. Например, в 1970-х ВВС США заказали реактивный истребитель, добавив требование, что он должен развивать скорость до 2,5 Маха. Но удовлетворить данное требование в то время было невозможно из-за отсутствия соответствующих технологий.

Тогда заказчика спросили: «Зачем нужна скорость два с половиной Маха?» Ответ был таков: «Эти штуки дорогие и должны при необходимости иметь возможность уклониться от боя». Конструкторы вернулись к проекту и создали истребитель F-16. Его максимальная скорость составила 2,1 Маха, но это был самый маневренный и быстро разгоняющийся самолет.

Когда пользователи предлагают решения, а не требования, архитектор должен, подобно надоедливому малышу, снова и снова спрашивать: «Зачем?», чтобы выявить *реальные* требования, скрытые в решениях.



Упражнение

Иногда требования трудно отличить от решений. Вот несколько ответов на вопрос «Зачем?», которые могут указывать, что в действительности подразумевается нечто иное. Попробуйте определить, какие из них являются требованиями, а какие — решениями.

«Система должна отслеживать предпочтения пользователей и сохранять настройки между сессиями».

☐

требование

☐

решение

«Нужно ли нам создавать свой сервис опросов? Мы наверняка найдем подходящий сторонний сервис».

☐

требование

☐

решение

«Корпоративная сервисная шина решила бы наши текущие проблемы (пусть и за счет некоторых усилий) и обеспечила исключительную расширяемость».

☐

требование

☐

решение

«По словам дружелюбного торгового агента, этот программный пакет содержит все, что нужно бухгалтерии сейчас и что понадобится в ближайшем будущем».

☐

требование

☐

решение

→ Решение на с. 109

Составные архитектурные характеристики

Алекс. Бизнес-аналитик просил обеспечить «надежность» системы. Что он имел в виду?

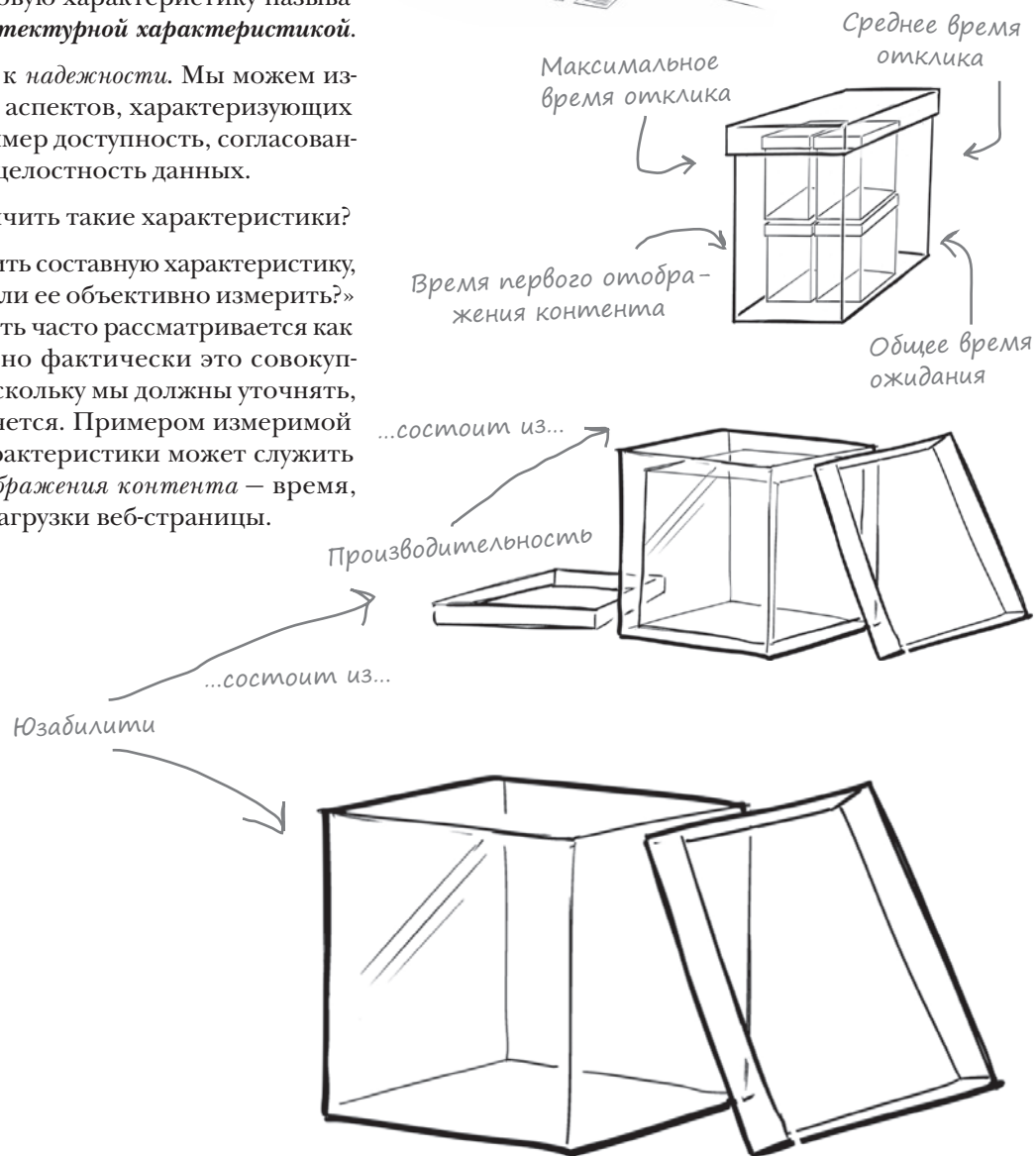
Сэм. Я могу придумать множество определений «надежности».

Мара. Это частое явление. Иногда две и более характеристики объединяются в *комбинацию*, и получившуюся новую характеристику называют *составной архитектурной характеристикой*.

То же относится и к *надежности*. Мы можем измерить множество аспектов, характеризующих надежность, например доступность, согласованность операций и целостность данных.

Алекс. И как различить такие характеристики?

Мара. Чтобы выявить составную характеристику, спросите: «Можно ли ее объективно измерить?» Производительность часто рассматривается как единая величина, но фактически это совокупность величин, поскольку мы должны уточнять, что именно измеряется. Примером измеримой архитектурной характеристики может служить время *первого отображения контента* — время, необходимое для загрузки веб-страницы.



Приоритеты определяются контекстом

Для разных проектов выбираются разные наборы архитектурных характеристик. Характеристики, выбираемые для конкретного приложения, и их важность будут различаться в зависимости от контекста.



Возьмите в руку карандаш

Контекст важен. Далее мы перечислили несколько архитектурных характеристик и три сценария применения. Оцените важность характеристик для каждого сценария. *Подсказка: не всем приложениям понадобятся все эти характеристики.*

Масштабируемость

Производительность

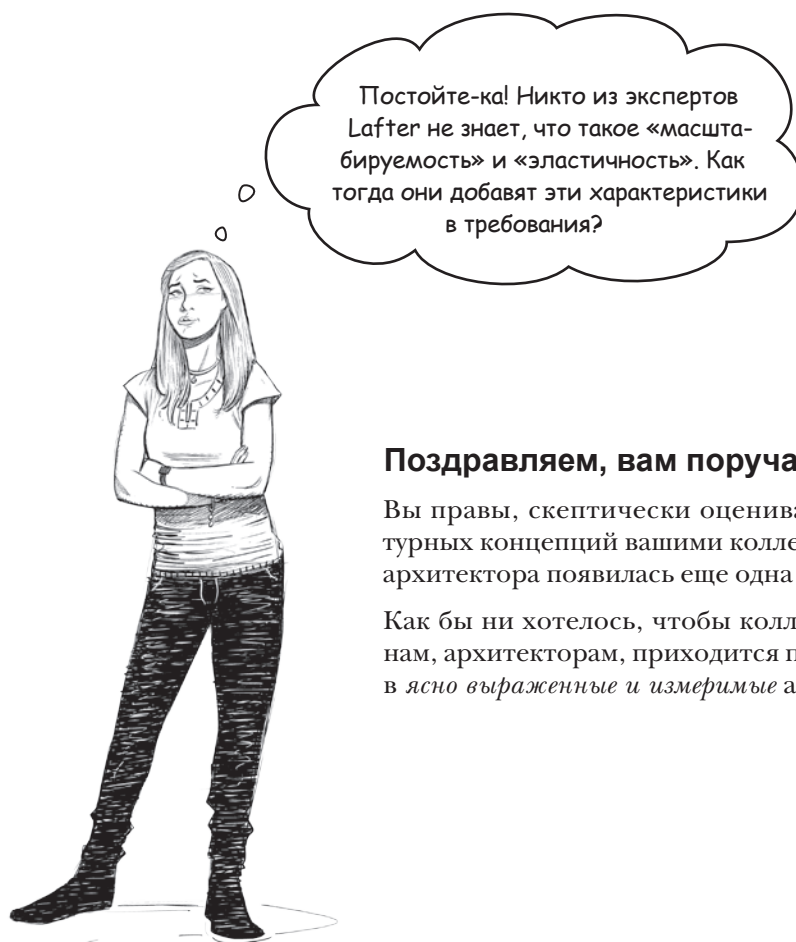
Расширяемость

Защищенность

Целостность данных

Сценарий 1 Интернет-магазин на конкурентном рынке	Сценарий 2 Система для предприятия, цель которого — рост за счет слияний	Сценарий 3 Приложение для автоматизации оценки результатов тестирования при поступлении в университеты
1 _____	1 _____	1 _____
2 _____	2 _____	2 _____
3 _____	3 _____	3 _____
4 _____	4 _____	4 _____
5 _____	5 _____	5 _____

Решение на с. 110



Поздравляем, вам поручается еще одна работа.

Вы правы, скептически оценивая глубину понимания архитектурных концепций вашими коллегами. А это значит, что у вас как архитектора появилась еще одна обязанность: перевод!

Как бы ни хотелось, чтобы коллеги выучили наш язык, именно нам, архитекторам, приходится переводить устремления бизнеса в *ясно выраженные и измеримые* архитектурные характеристики.

Потери при переводе

Нередко бизнес-эксперты и аналитики формулируют требования, сами того не осознавая. Ваша задача как архитектора — выявить эти требования и перевести их в архитектурные характеристики. Рассмотрим несколько примеров.

Когда бизнес-аналитики и эксперты говорят:

«Соцсеть Lafter должна постоянно меняться, чтобы соответствовать требованиям рынка».

«В связи с новыми требованиями заявки должны обрабатываться до конца дня».

«В ближайшие три года мы планируем активно участвовать в слияниях и поглощениях».

«У нашего проекта жесткие временные рамки, фиксированный объем и ограниченный бюджет».

Никто и никогда не выдвинет такую комбинацию требований... хотя...

Осуществимость — оценка возможности что-то сделать, это редко используемая характеристика.

Архитектор должен перевести:

- Гибкость.
- Модульность.
- Расширяемость.

Хорошая модульность позволяет быстро вносить изменения, не плодя побочных эффектов.

- Производительность.
- Восстанавливаемость.
- Масштабируемость.

Система должна быстро работать и быстро восстанавливаться после ошибок.

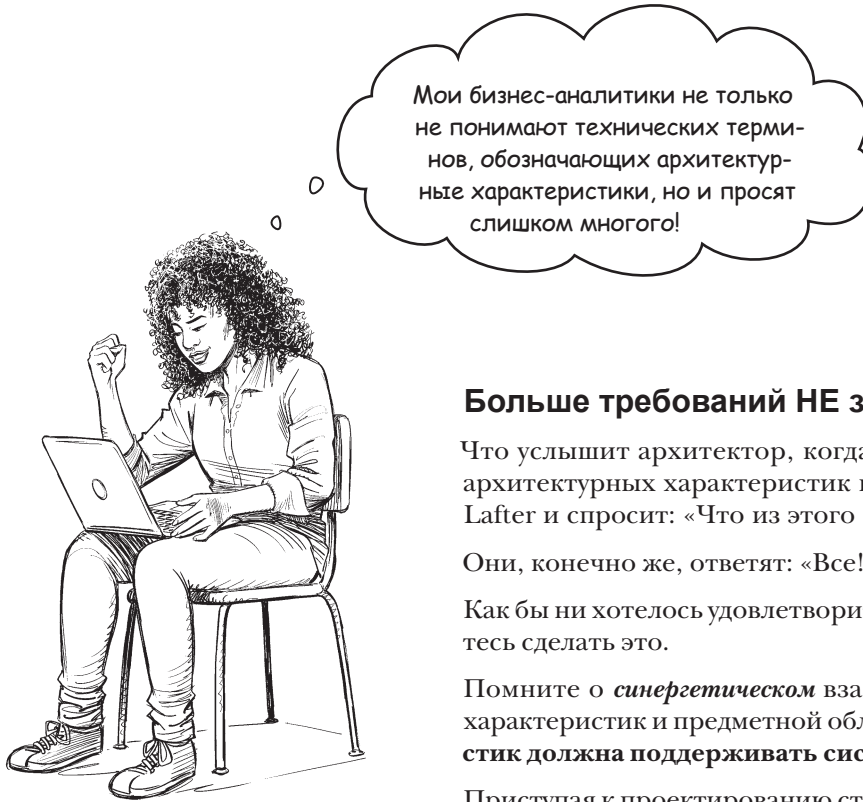
- Ради строчки в резюме.
- Интегрируемость.
- Совместимость.

Многие избегают мест, где постоянно происходят слияния.

- Осуществимость.
- Простота.
- Экономичность.

Скорее это характеристика архитектора как личности.

Архитекторы часто способны быстро оценить, что возможно в заданные сроки.



Больше требований НЕ значит лучше.

Что услышит архитектор, когда огласит список возможных архитектурных характеристик группе бизнес-пользователей Lafter и спросит: «Что из этого вы хотите видеть в системе?»

Они, конечно же, ответят: «Все!»

Как бы ни хотелось удовлетворить их желание, даже не пытайтесь сделать это.

Помните о *синергетическом* взаимовлиянии архитектурных характеристик и предметной области: **чем больше характеристик должна поддерживать система, тем сложнее она будет.**

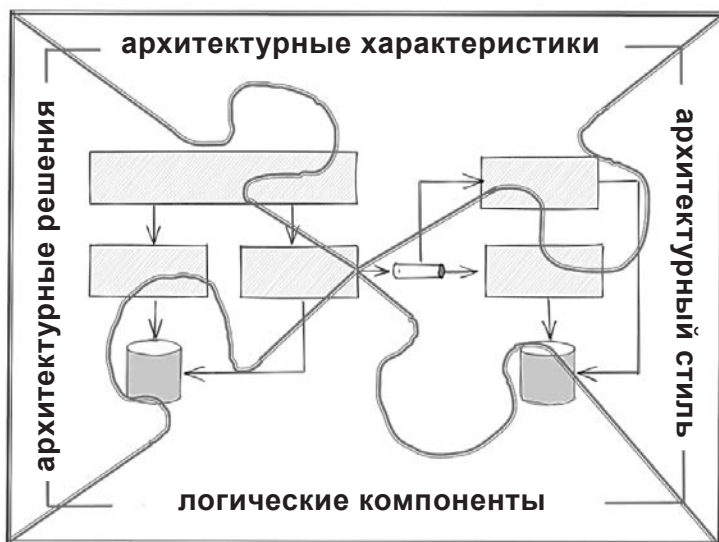
Приступая к проектированию структуры системы, архитектор должен найти баланс между приоритетами предметной области и архитектурными характеристиками, необходимыми для успешной работы системы.

Архитектурные характеристики и логические компоненты

Прежде чем рассказывать, как правильно сбалансировать архитектурные характеристики, мы хотим показать, что архитектурные характеристики и логические компоненты — это две стороны одной медали. И то и другое направлено на поддержку предметной области (также известной как причина разработки ПО).

Архитектурные характеристики $\approx$ возможности

Архитектурные характеристики описывают *возможности*, которые будет поддерживать ваше решение, а не его *поведение*, основанное на требованиях.



Логические компоненты = поведение

Логические компоненты (подробно рассматриваются в главе 4) определяют дизайн системы, реализуемой для решения основной задачи.

Сбалансированность архитектурных характеристик

Архитектурный стиль определяется путем анализа архитектурных характеристик и логических компонентов, и приложение Lafter не исключение. Вы должны найти баланс между ними.

Представьте, что на левой чаше лежат потребности предметной области, а на правой — архитектурные характеристики.



Когда нет архитектурных характеристик

Если не уделить время анализу архитектурных характеристик перед проектированием системы, то позднее, при обнаружении того факта, что система не обладает необходимыми архитектурными характеристиками, придется выполнить дорогостоящую и трудоемкую переделку проекта.

Проекты, не имеющие необходимых архитектурных характеристик, чаще терпят неудачу.

Помните, что архитектурные характеристики подвержены влиянию требований предметной области и других архитектурных характеристик.

Хороший баланс между...

В этом случае мы добились баланса между архитектурными характеристиками и требованиями предметной области.



...архитектурными характеристиками и потребностями предметной области

Это позволило достичь эксплуатационных и структурных целей, обойдясь без чрезмерного усложнения системы.

Слишком много архитектурных характеристик

Иногда, к сожалению, архитекторы отрываются от реальности и тратят слишком много времени на анализ архитектурных характеристик, только чтобы убедиться, что многие из них бесполезны. Это приводит к потере времени, которое можно потратить на реализацию и сопровождение.



В системах, поддерживающих слишком много архитектурных характеристик, остается мало места для поддержки предметных потребностей.

Сокращение количества архитектурных характеристик

Как ограничить аппетиты бизнес-стороны, желающей получить все возможные архитектурные характеристики?

Волшебное число 7

Один из полезных советов архитекторам и бизнес-аналитикам: **выбирайте не более семи архитектурных характеристик**. Согласно исследованиям, люди запоминают элементы группами по семь штук (это одна из причин того, почему первые телефонные номера были семизначными). Кроме того, это число достаточно большое, чтобы обеспечить некоторое разнообразие и не создать парадокс выбора из-за слишком большого количества предложений.

Мы создали контрольный список, помогающий выбрать разумное количество характеристик. У вас будет возможность воспользоваться им на следующей странице.

The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information — знаменитая статья психолога Джорджа Миллера, опубликованная в 1956 году (https://ru.wikipedia.org/wiki/Магическое_число_семь_плюс-минус_два).

В списке с движущими характеристиками выберите три наиболее важные, указав цифры 1, 2 и 3 рядом с ними.

Характеристики, определяющие важные проектные решения, называются движущими.

Неявные характеристики становятся движущими, если влияют на структурные решения.

Топ-3

Движущие характеристики

Неявные характеристики

☐

осуществимость (цена/время)

☐

защищенность

☐

удобство сопровождения

☐

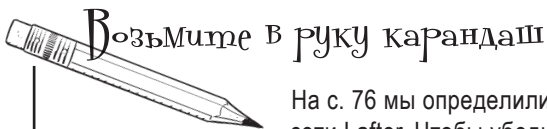
наблюдаемость

☐

☐

☐

По нашему опыту, эти четыре характеристики особенно распространены. Вы можете заменить их любыми другими, подходящими вам.



Возьмите в руку карандаш

На с. 76 мы определили архитектурные характеристики для приложения социальной сети Lafter. Чтобы убедиться в достижении хорошего баланса, ограничьте список семью характеристиками. Затем отметьте три наиболее важные.

Топ-3 Движущие характеристики Неявные характеристики

☐

осуществимость (цена/время)

защищенность

☐

удобство сопровождения

☐

наблюдаемость

☐

☐

☐

☐

↑ Это неявные характеристики. Перенесите их в колонку «Движущие характеристики», если считаете их важными для успешной работы системы.

↑ Выберите три самые важные (в любом порядке).

Возможные кандидаты в архитектурные характеристики

производительность

целостность данных

развертываемость

отзывчивость

согласованность данных

тестируемость

доступность

адаптируемость

конфигурируемость

отказоустойчивость

расширяемость

настраиваемость

масштабируемость

интегрируемость

восстанавливаемость

эластичность

параллелизм

контролируемость

→ Решение на с. III

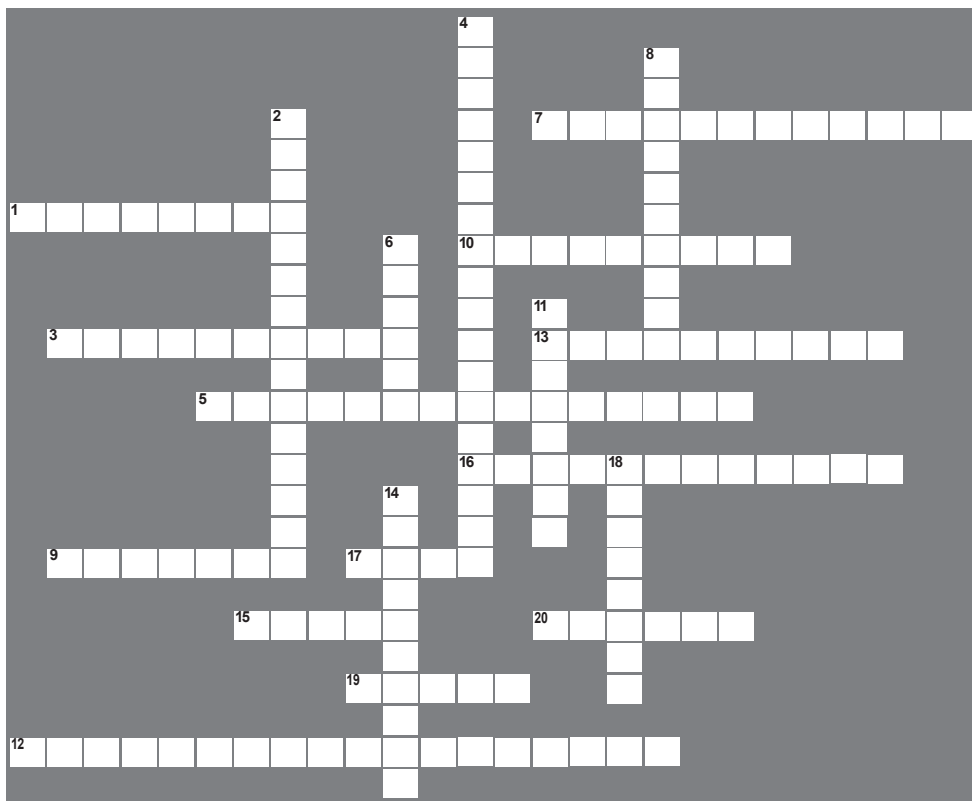
Ключевые моменты

- *Архитектурные характеристики* — один из аспектов структурного анализа при проектировании программных систем. (О другом аспекте, логических компонентах, мы поговорим в главе 4.)
- Архитектурные характеристики описывают возможности системы.
- Некоторые архитектурные характеристики пересекаются с эксплуатационными задачами (например, доступность, масштабируемость и т. д.).
- Существует множество категорий архитектурных характеристик, но пока никому не удалось составить единый список, поскольку экосистема разработки ПО постоянно меняется.
- Определяя архитектурные характеристики, ищите факторы, влияющие на структуру.
- Важно быть осторожными и не указывать слишком много архитектурных характеристик, поскольку из-за *синергии* изменение одной из них потребует изменения других.
- Некоторые архитектурные характеристики — *неявные*: они не указаны в требованиях, но учитываются архитектором.
- Есть архитектурные характеристики, которые относятся к нескольким категориям.
- Многие архитектурные характеристики — *сквозные*: они влияют на другие части организации (и решения, принимаемые в них).
- Архитектурные характеристики извлекаются из требований и других соображений, связанных с проектированием предметной области.
- Некоторые архитектурные характеристики извлекаются из *знания о предметной области и/или окружении*, а не из требований к конкретному приложению.
- Есть *составные* архитектурные характеристики: они получаются при объединении других архитектурных характеристик.
- Архитекторы должны уметь переводить бизнес-жаргон в архитектурные характеристики.
- Количество архитектурных характеристик должно ограничиваться каким-то небольшим числом, например семью.



Кроссворд по характеристикам

Готовы поразвлечься и проверить свои знания о характеристиках? Попробуйте разгадать этот кроссворд.



По горизонтали

1. Время загрузки веб-страницы часто называют временем «первого отображения _____».
3. _____ усложнение — архитектурная проблема.
5. Системе может потребоваться _____, чтобы соответствовать требованиям.
7. Одна из неявных архитектурных характеристик.
9. Характеристика может быть явной или _____.
10. Характеристики влияют на _____ системы.
12. Многие правительства регулируют _____ данных.
13. В Laffer сообщение может быть шуткой или _____.
15. Вопрос, который должен задать архитектор, когда пользователь предлагает решение вместо требования.
16. Сайту с большим количеством _____ работающих пользователей необходима возможность масштабирования.
17. Волшебное число, ограничивающее количество учитываемых архитектурных характеристик.

19. Объединение архитектурных характеристик и логических компонентов дают архитектурный _____.
20. Целостность _____ — архитектурная характеристика.

По вертикали

2. Нефункциональное требование также называют архитектурной _____.
4. _____ — это архитектурное требование.
6. Архитектурные характеристики важны для _____ системы.
8. Выбор характеристик предполагает присваивание _____ каждой из них.
11. Многие характеристики _____.
14. Область, для которой пишется программное обеспечение.
18. Неравномерный трафик иногда называют _____.

→ Решение на с. 112

Что чему соответствует? Решение

Архитектурные характеристики так многообразны! Мы составили список тех из них, которые наиболее важны для Lafter, вместе с их определениями, но потеряли его. Помогите нам восстановить этот список, нарисовав стрелки, связывающие архитектурные характеристики с соответствующими определениями.





Упражнение Решение

На с. 90

Добро пожаловать в игру «одно из двух». Ее правила просты: слева даны бизнес-требования к приложению Lafter, а справа — архитектурные характеристики. Как известно, в архитектуре все является компромиссом, поэтому, оптимизируя одно, вы ухудшите другое. Ваша задача — сообщить нам, какую характеристику вы считаете более важной для того или иного требования.

«Мы должны выйти на рынок как можно скорее!»	отказоустойчивость	гибкость
«У нас туго с деньгами!»	масштабируемость	простота
«Эта конференция обещает стать самой масштабной».	высокая доступность	простота сопровождения
«Мы будем хранить информацию о кредитных картах».	защищенность	восстанавливаемость
«Этот сайт будет очень популярен».	гибкость	эластичность



Упражнение
Решение

Требования часто являются богатым источником архитектурных характеристик. Например, приложение Lafter должно поддерживать большое количество пользователей, поэтому масштабируемость — одна из обязательных характеристик. Попробуйте определить другие характеристики.

Пользователи: сотни докладчиков, тысячи участников.

Требования:

Пользователи могут регистрироваться, согласившись с условиями.

Пользователи могут добавлять свой контент в виде «Шуток» (длинные статьи) или «Каламбуров» (короткие заметки).

Последователи могут выражать сильное («Ха-ха») или более сдержанное («Хи-хи») одобрение понравившегося контента.

Докладчики получают специальный значок Sillycon Symposia.

Докладчики могут создавать форумы для обсуждения выступления.

Пользователи могут оставлять сообщения длиной до 281 символа.

Пользователи могут оставлять ссылки на внешние ресурсы.

Дополнительно:

Поддержка интернационализации.

Небольшой штат команды поддержки.

«Взрывной» трафик во время проведения конференций.

**Архитектурные
характеристики**

Масштабируемость

Эластичность

Авторизация

Аутентификация

Интернационализация

Настраиваемость



Упражнение Решение

Иногда требования трудно отличить от решений. Вот несколько ответов на вопрос «Зачем?», которые могут указывать, что в действительности подразумевается нечто иное. Попробуйте определить, какие из них являются требованиями, а какие — решениями.

«Система должна отслеживать предпочтения пользователей и сохранять настройки между сессиями».



требование



решение

«Нужно ли нам создавать свой сервис опросов? Мы наверняка найдем подходящий сторонний сервис».



требование



решение

«Корпоративная сервисная шина решила бы наши текущие проблемы (пусть и за счет некоторых усилий) и обеспечила исключительную расширяемость».



требование



решение

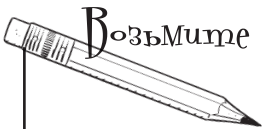
«По словам дружелюбного торгового агента, этот программный пакет содержит все, что нужно бухгалтерии сейчас и что понадобится в ближайшем будущем».



требование



решение



Возьмите в руку карандаш
Решение

Контекст важен. Далее мы перечислили несколько архитектурных характеристик и три сценария применения. Оцените важность характеристик для каждого сценария. Подсказка: не всем приложениям понадобятся все эти характеристики. Вот наше решение.



Сценарий 1
Интернет-магазин на конкурентном рынке

Сценарий 2
Система для предприятия, цель которого — рост за счет слияний

Сценарий 3
Приложение для автоматизации оценки результатов тестирования при поступлении в университеты

- 1 Защищенность
- 2 Производительность
- 3 Масштабируемость
- 4
- 5

- 1 Расширяемость
- 2 Масштабируемость
- 3
- 4
- 5

- 1 Целостность данных
- 2 Защищенность
- 3 Производительность
- 4
- 5



Возьмите в руку карандаш Решение

На с. 76 мы определили архитектурные характеристики для приложения социальной сети Lafter. Чтобы убедиться в достижении хорошего баланса, ограничьте список семью характеристиками. Затем отметьте три наиболее важные.

Топ-3 Движущие характеристики Неявные характеристики

☒	<u>Масштабируемость</u>	<u>осуществимость (цена/время)</u>
☒	<u>Защищенность</u>	<u>защищенность</u>
☐	<u>Эластичность</u>	<u>удобство сопровождения</u>
☒	<u>Отзывчивость</u>	<u>наблюдаемость</u>
☐	<u>Производительность</u>	
☐	<u>Переносимость</u>	
☐	<u>Доступность</u>	

← Помните, что единственно верного ответа не существует. Вопрос лишь в том, сможете ли вы обосновать свой выбор.

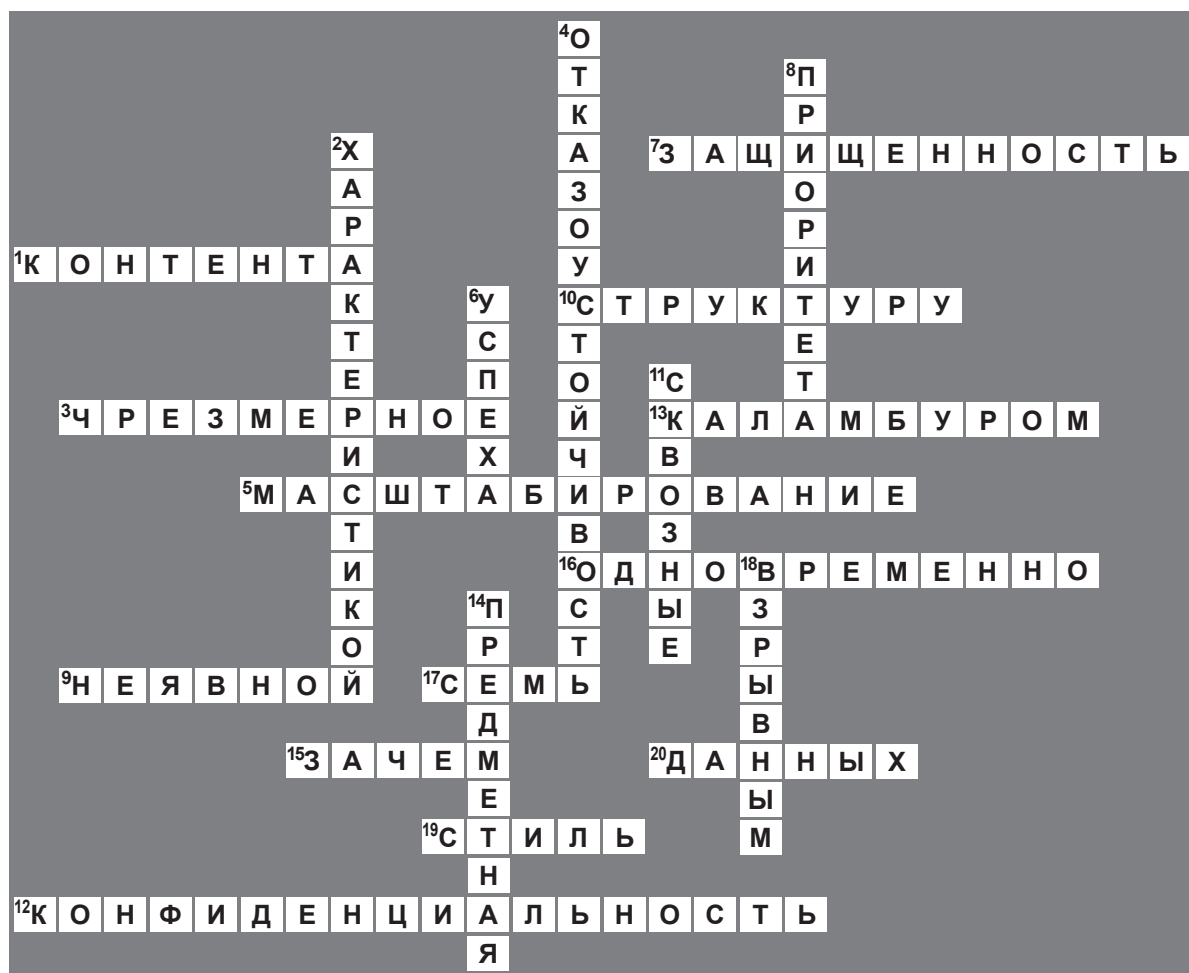
Возможные кандидаты в архитектурные характеристики

производительность	целостность данных	развертываемость
отзывчивость	согласованность данных	тестируемость
доступность	адаптируемость	конфигурируемость
отказоустойчивость	расширяемость	настраиваемость
масштабируемость	интегрируемость	восстанавливаемость
эластичность	параллелизм	контролируемость



Кроссворд по характеристикам. Решение

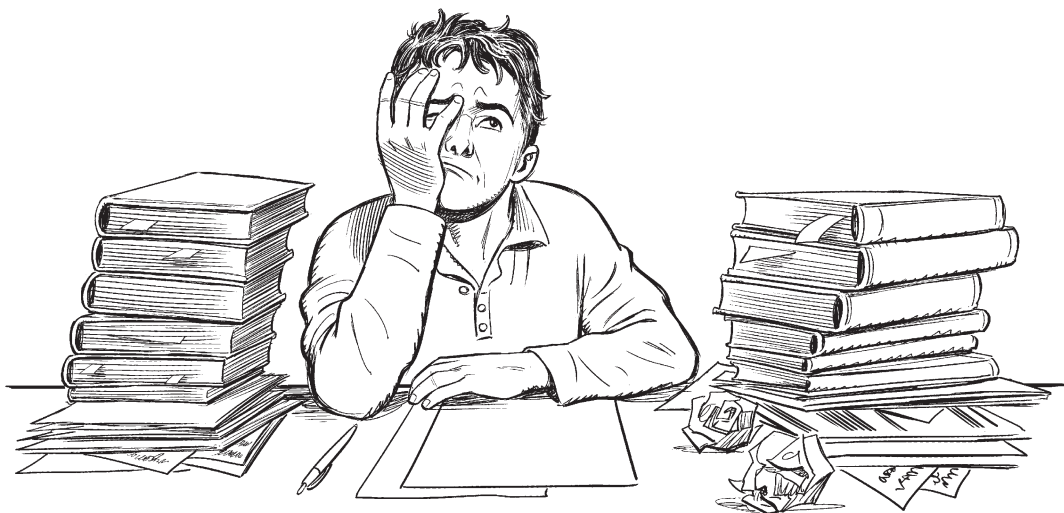
На с. 105



3

Два закона архитектуры ПО

Все основано на компромиссах

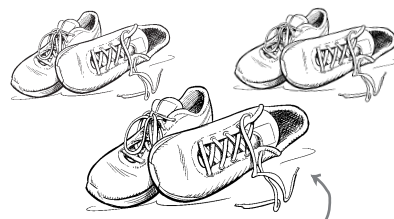


Что происходит, когда нет «лучших практик»? Лучшие практики — это относительно безопасные способы достижения определенных целей. Они называются «лучшими» (а не «хорошими») не просто так — они работают. Так почему бы просто не использовать их? Но в архитектуре ПО нет лучших практик. Вам придется тщательно анализировать каждую ситуацию, чтобы принять решение, и объяснять не только «что» нужно сделать, но и «почему». *Как преодолеть этот новый рубеж?* К счастью, есть законы архитектуры ПО, которые вам помогут. В данной главе вы узнаете, как анализировать компромиссы и создавать записи архитектурных решений, чтобы фиксировать ответы на вопросы «как» и «почему». К концу главы вы освоите инструменты для навигации по изменчивой территории архитектуры ПО.

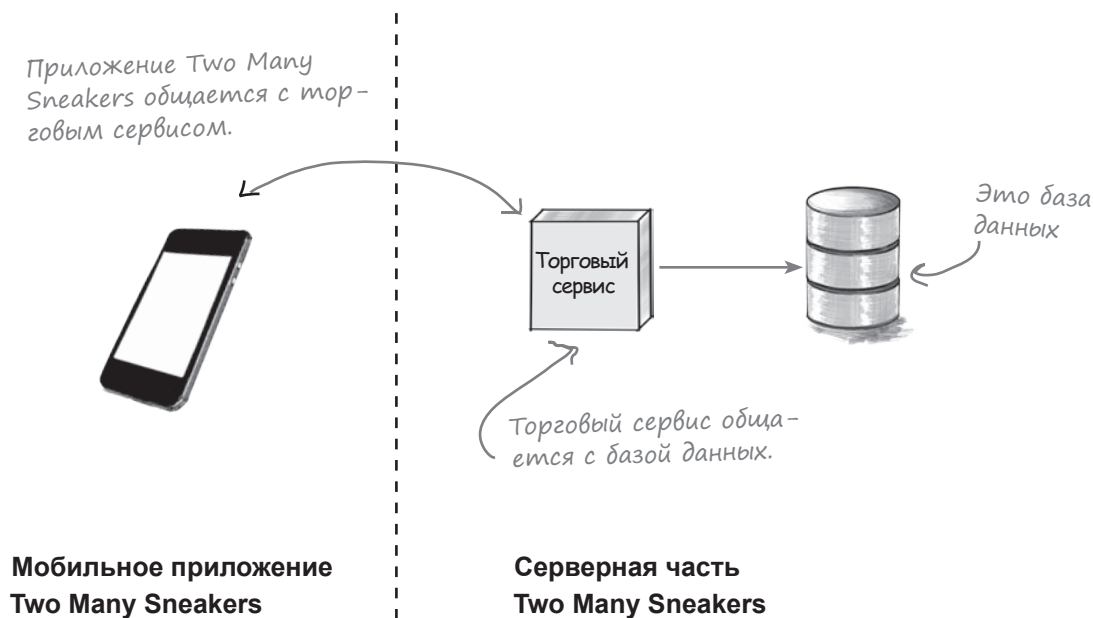
Приложение для фанатов кроссовок

Арчана работает в компании Two Many Sneakers, поддерживающей мобильное приложение, которое помогает фанатам кроссовок продавать, покупать и обменивать коллекционные кроссовки.

Первоначальная архитектура приложения представляла собой единый сервис, как показано ниже:



Слоган Two Many Sneakers:
«У вас всего две ноги, но кроссовок слишком много не бывает!»

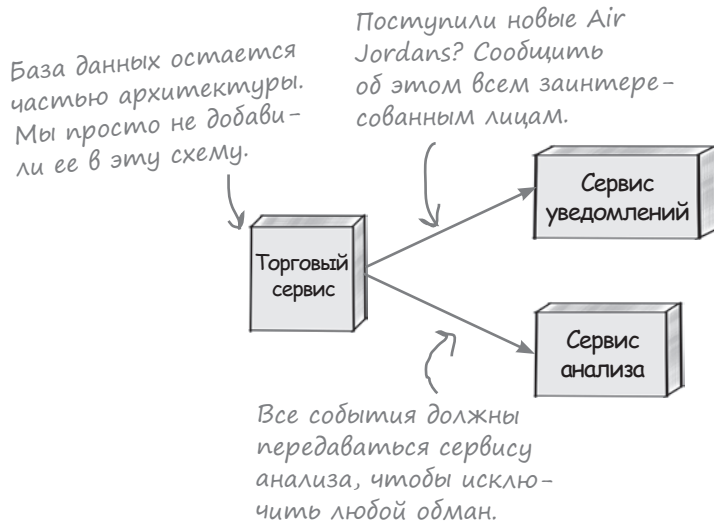
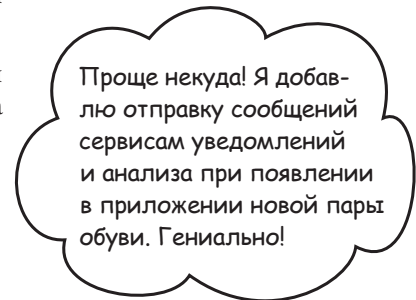


Приложение Two Many Sneakers знает, как связаться с торговым сервисом, чтобы получить или обновить данные (например, фотографию пары Nikes в отличном состоянии). Торговый сервис, в свою очередь, общается с базой данных.

Бизнес процветает. Фанаты радостно пополняют и обновляют свои коллекции, и клиентская база Two Many Sneakers быстро растет. Однако теперь клиенты требуют добавить рассылку уведомлений в режиме реального времени, чтобы знать, например, когда кто-то выставляет на продажу пару Air Jordans в идеальном состоянии.

Защищенность всегда была предметом беспокойства в онлайн-торговле. Подделки никому не нужны, а номера кредитных карт должны надежно защищаться. Чтобы не стать жертвой мошенников, руководство Two Many Sneakers решило во главу угла поставить *борьбу с мошенничеством*. Планируется внедрить анализ данных для обнаружения мошенничества путем выявления аномалий в поведении пользователей и отсеивания ботов.

Работа уже началась. Команде осталось только настроить торговый сервис, чтобы он сообщал новым сервисам уведомлений и анализа о событиях, происходящих в приложении.

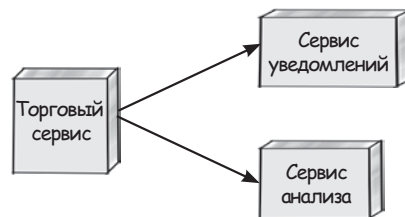


Арчана

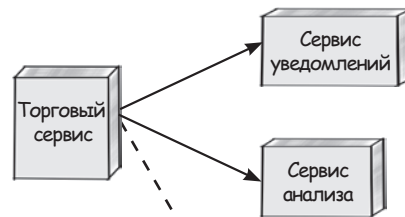
Что мы уже знаем?

Нам нужно определить способ взаимодействия сервисов. Вспомним, что мы знаем (и чего не знаем) на данный момент.

- Текущая архитектура довольно проста — торговый сервис общается только со своей базой данных. Но помимо этого, он должен посылать информацию сервисам уведомлений и анализа.



- В офисе поговаривают, что финансовый отдел (отвечающий за соблюдение нормативных требований) тоже захочет получать уведомления от торгового сервиса, а это означает, что архитектура должна быть *расширяемой*.



*Что понадобится финансистам?
И понадобится ли хоть что-то?*

- Мы не знаем, что отправлять сервисам уведомлений и анализа. Какие данные они должны получать: одинаковые или совершенно разные? Кроме того, мы ничего не знаем о намерениях финансового отдела.

```

{
  "sellerId": 12345,
  "buyerId": 6789,
  "itemId": 1492092517,
  "price": "$125.00"
}
  
```

Как это должно выглядеть?

Итак, кое-что известно, но многого мы еще не знаем. Добро пожаловать в мир архитектуры программного обеспечения.

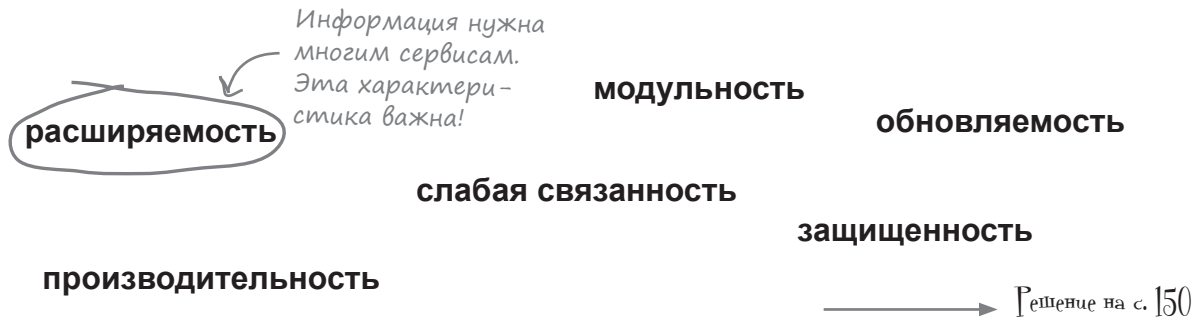
Мы сможем закончить разработку архитектуры, скажем, к следующему четвергу, верно?

Мы как архитекторы должны определить архитектурные характеристики. (Они обсуждались в главе 2.)



Упражнение

Какие из следующих архитектурных характеристик важны для этой конкретной задачи? Подсказка: здесь нет правильных ответов, поскольку многого мы пока не знаем, а в чем-то не уверены. Выбирайте по своему усмотрению — наше решение вы найдете в конце главы. Мы поможем начать:



Мозговой штурм

Набор характеристик выше выглядит неплохо. Например, мало кто откажется от возможности обновления.

Выбирая каждую характеристику, спросите себя: **важна ли она** или является лишь приятной мелочью?

Более того, некоторые характеристики *конфликтуют*. Защита с широким применением шифрования наверняка ухудшит производительность. Вернитесь и проверьте, есть ли конфликтующие варианты. Если есть, то оставьте только один.

Что-то такое было в главе 2? Точно!

Часто задаваемые вопросы

В: Даже в таком простом упражнении оказалось довольно много нюансов. Что-то нам известно, что-то мы только предполагаем, а чего-то определенно не знаем. И как в таких условиях рассуждать об архитектуре?

О: Вы правы. В реальности список архитектурных характеристик почти всегда содержит смесь из «это необходимо» и «это может пригодиться». Даже ваши клиенты не смогут сказать вам, чего они *рано или поздно* захотят. (А ведь это было бы здорово!) А «то, о чем мы даже не подозреваем», часто называют «неизвестные неизвестные».

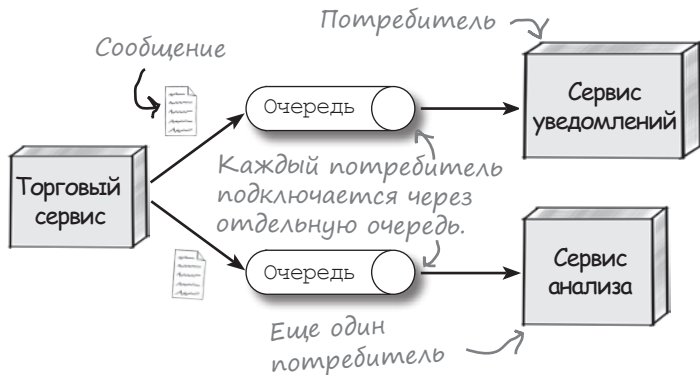
Нередко «неизвестные неизвестные» всплывают в середине проекта и срывают даже тщательно продуманные планы. Есть решение? Научитесь быть гибкими. Осознайте, что ничто, особенно архитектура программного обеспечения, не остается неизменным. То, что работает сегодня, завтра может оказаться серьезным препятствием. Такова природа архитектуры ПО: она постоянно меняется по мере появления новых знаний о задаче и новых требований у ваших клиентов.

Взаимодействие с другими сервисами

Наша цель — организовать автоматическую отправку сообщений сервисам уведомлений и анализа. Допустим, что мы решили задействовать механизм обмена сообщениями. Но тут возникает дилемма: что использовать — очереди или темы?

Прежде чем продолжить, выясним разницу между очередями и темами. Большинство механизмов обмена сообщениями предлагают две модели реализации *издателя* сообщений (в данном случае это торговый сервис) — с одним или с несколькими *потребителями* (другими сервисами).

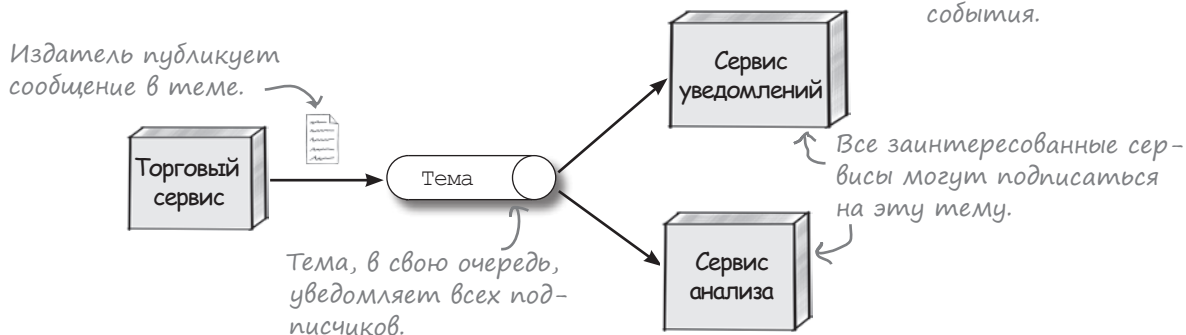
Первый вариант — *очередь*, протокол связи «точка-точка». В этом случае издатель знает, кто получает его сообщения. Чтобы обслужить несколько потребителей, издатель должен отправить сообщение в отдельные очереди, связывающие его с потребителями. В нашем случае конфигурация с очередями будет выглядеть так:



Ничего страшного, если вы не знакомы с очередями или темами. Мы расскажем вам все, что вы должны знать.

Представляйте очереди как групповые сообщения: вы выбираете получателей, вводите текст и нажимаете кнопку «Отправить».

Второй вариант, *темы*, реализует модель вещания. Если другой сервис хочет получать сообщения от издателя, то он должен *подписаться* на тему. Издатель не знает (и ему это неинтересно), сколько сервисов читают его сообщения.



Темы подобны публикации фотографии в социальной сети. Ваши подписчики увидят фотографию, если «подписаны» на ваши события.

Оба варианта хороши, так какой же из них выбрать? Давайте узнаем.

Анализ компромиссов

Нельзя усидеть на двух стульях. Нам часто приходится выбирать, оптимизируя одно за счет другого. Хотите хранить много фотографий в своем телефоне? Тогда приобретите дополнительную карту памяти, потратив деньги, или сжимайте снимки в ущерб их качеству.

Архитектура ПО в этом смысле ничем не отличается. Делая выбор, вы должны чем-то жертвовать или, как мы говорим, *идти на компромиссы*. И что это означает?

Зная, какие характеристики наиболее важны для проекта, вы можете начать думать о решениях, максимизирующих эти характеристики. Но значение одних характеристик часто увеличивается за счет других. Например, решение, обеспечивающее высокую масштабируемость, может усложнить развертывание или ухудшить надежность.

Любое ваше решение *будет* иметь и плюсы, и минусы.

Соответственно, перед вами стоят две задачи: выяснить, какие компромиссы связаны с каждым решением, и выбрать решение, которое наилучшим образом соответствует самым важным архитектурным характеристикам.



Мы уже обсуждали значимые и незначимые компромиссы в главе 1.

Рич Хикки, создатель языка программирования Clojure, однажды сказал: «Программисты знают все о преимуществах и ничего о компромиссах». Мы бы добавили: «Архитекторы должны знать и то и другое».

Невозможно получить все сразу. Вам придется определить наиболее важные архитектурные характеристики и выбрать решение, которое соответствует им наилучшим образом.

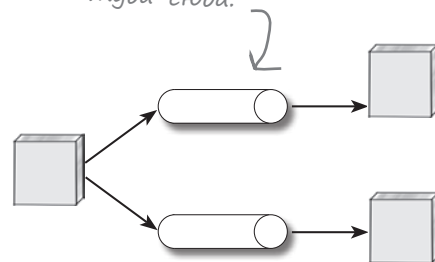
Анализ компромиссов: версия с очередями

Анализ компромиссов — это поиск *достоинств* и *недостатков* конкретного подхода с целью получить максимально полную картину. Рассмотрим оба варианта, начав с очередей.

Выбрав это решение, нам придется создать отдельную очередь для каждого сервиса-получателя. Если получателям нужна разная информация, то мы сможем отправлять разные сообщения в разные очереди. Торговый сервис знает, с какими сервисами взаимодействует, что затрудняет «подслушивание» другим (неавторизованным) сервисам. (Это полезно, если безопасность стоит на первом месте в списке приоритетов.) А поскольку каждая очередь независима, то мы сможем управлять ими и масштабировать их независимо.

Торговый сервис *тесно связан* с получателями — он точно знает, сколько их. Но мы пока не знаем, придется ли отправлять сообщения еще и в финансовый сервис. Если да, то мы должны будем переделать торговый сервис и добавить отправку сообщений в *третью очередь*. Таким образом, выбирая очереди, мы отказываемся от расширяемости.

Просто напомним, как выглядит конфигурация с очередями, чтобы вам не приходилось листать страницы туда-сюда.



Вариант с очередями	
Плюсы	Минусы
– Гетерогенные сообщения для разных потребителей. – Независимость мониторинга и масштабирования. – Лучшая защищенность (и безопасность).	+ Тесная связанность (ухудшает расширяемость). + Торговый сервис подключается к нескольким очередям. + Требуется дополнительная инфраструктура.

Гетерогенные — это синоним слова «разные».

Белые доски хорошо подходят для оценки компромиссов, выполняемой вместе с командой.

Вот что мы подразумеваем под словами «анализ компромиссов».

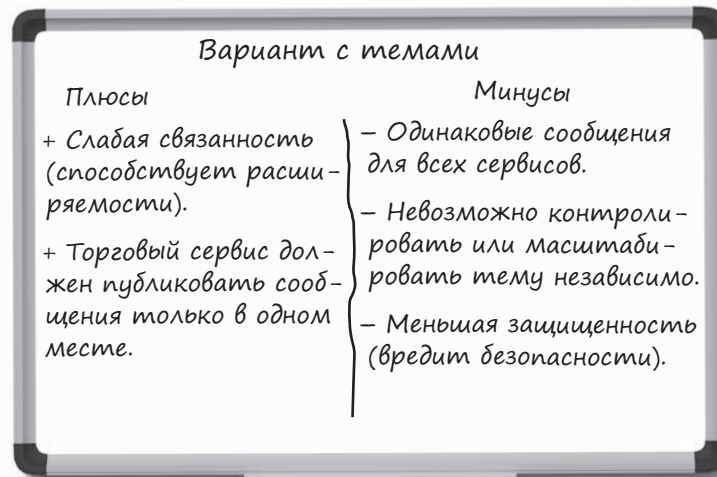
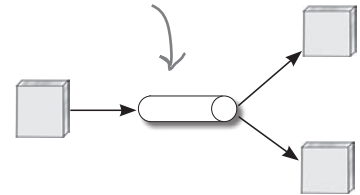
Анализ компромиссов: версия с темами

А что можно сказать о темах? Плюсы очевидны: торговый сервис посылает сообщения в определенную тему, а все, кто заинтересован в их получении, просто подписывается на нее. Финансовый отдел решил включиться в работу? Он может *подписаться* на тему, а нам не придется вносить изменения в торговый сервис. Слабая связанность во всей красе!

Но темы имеют свои минусы. Например, *вы не сможете ограничить круг получателей*, сообщения увидят все подписавшиеся. Масштабирование тоже имеет ограничения, поскольку оно может выполняться только на одной теме. А кроме того, любой может подписаться на тему без ведома торгового сервиса, что создает определенный риск безопасности.

Вернемся к доске!

Так выглядит конфигурация с темами.



Возьмите в руку карандаш

Потратьте несколько минут и сравните результаты анализа. Вы заметили, что каждый вариант поддерживает одни характеристики в ущерб другим? Теперь мы перечислим несколько требований, а вы должны определить вариант, который лучше их поддерживает.

Требования

«Защищенность важна для нас».

«Разные сервисы должны получать разную информацию».

«В будущем мы расширим круг сервисов».

Очереди / Темы

Очереди / Темы

Очереди / Темы

→ Решение на с. 150

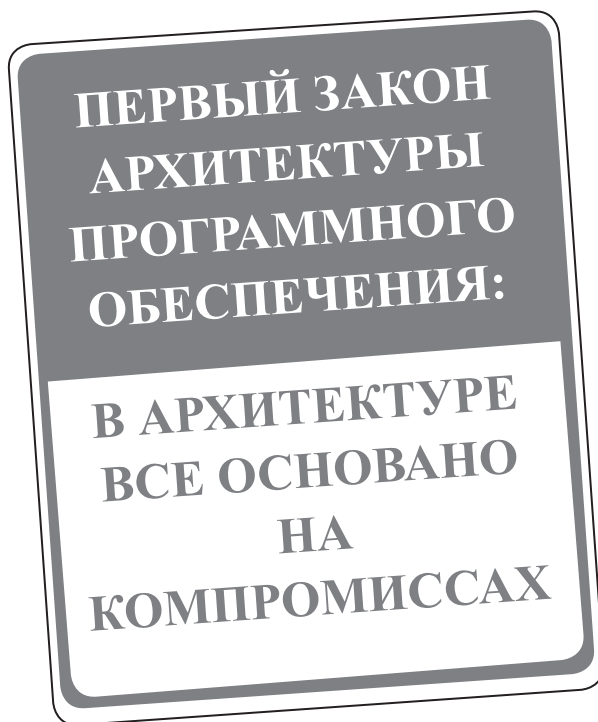
Первый закон архитектуры ПО

Очереди или темы? Хватит ломать голову. Выбор *зависит от ситуации!*

Что важно для бизнеса? Если *защищенность*, то лучше выбрать очереди. Компания Two Many Sneakers быстро растет. Соответственно, увеличивается количество сервисов, заинтересованных в информации о продажах кроссовок. Поэтому главный приоритет — *расширяемость*. То есть мы должны выбрать темы.

Время — еще один фактор: чтобы быстро выйти на рынок, вместо архитектуры с высокой доступностью можно выбрать более простую архитектуру. (Гарантия доступности в три «девятки» имеет значение, только если есть клиенты, верно?)

Главный вывод: в архитектуре ПО вы всегда будете искать компромиссы. И он подводит нас к *Первому закону архитектуры программного обеспечения*.



Один из авторов часто надевает эту футболку. (Если у вас есть возможность сделать футболки с такой надписью, то, пожалуйста, пришлите и нам! Одну размера М и две XXL!)

Если вы нашли решение, не имеющее компромиссов, значит, недостаточно внимательно отнеслись к задаче.

В архитектуре ПО редко встречаются идеальные решения и не существует «лучших практик». Любой выбор зависит от множества факторов, часто противоречащих друг другу. Первый закон — важный урок. Зазубрите его. Запишите на стикере и приклейте к монитору. Сделайте на лбу татуировку в зеркальном отражении, чтобы вы могли читать ее, когда смотрите в зеркало!

Возьмите в руку карандаш

Теперь мы предлагаем вам самим проанализировать возможные варианты. Для связи торгового сервиса с потребителями мы выбрали обмен сообщениями — асинхронный протокол. Однако выбор между асинхронной и синхронной формами связи сопряжен со своими компромиссами! Мы даем вам две доски, по одной для каждой формы связи, и список «возможностей». Подумайте, как каждая архитектурная характеристика будет работать в обоих контекстах, будет ли для нее плюсом или минусом (или ни тем ни другим) синхронная и асинхронная связь? Запишите результаты в соответствующие колонки. *Подсказка: не все из приведенных характеристик применимы к данному решению.* Для примера мы записали первый плюс. С нашими ответами вы можете ознакомиться в конце главы.

отзывчивость

обработка ошибок

транзакции

координация

согласованность

расширяемость

развертываемость

отказоустойчивость

Синхронная связь

Плюсы

Минусы

Асинхронная связь

Плюсы

Минусы

отзывчивость

→ Решение на с. 151

Часто Задаваемые Вопросы

В: Я слышал о методе анализа компромиссов в архитектуре (Architecture Tradeoff Analysis Method, ATAM). Это то, о чем вы говорите?

О: ATAM — популярный метод, в котором вы сначала оцениваете потребности бизнеса, имеющиеся возможности и архитектуру, представляемую заинтересованным сторонам, а затем пробуете множество сценариев, чтобы получить «проверенную архитектуру». ATAM — хороший подход, но у него есть ряд ограничений, и одно из них таково: он предполагает, что архитектура статична и не меняется.

Мы сами предпочитаем фокусироваться не на процессе, а на результатах. Цель любого анализа компромиссов — создание

архитектуры, лучше всего отвечающей потребностям. Возможно, вам придется не раз повторить анализ по мере появления новых сведений о задаче и новых сценариев.

Другой популярный метод — анализ затрат и выгод (Cost Benefit Analysis Method, CBAM). В отличие от ATAM, он фокусируется на стоимости внедрения конкретной характеристики.

Мы советуем изучить оба метода и рассмотреть возможность их совместного применения: ATAM может помочь с анализом компромиссов, а CBAM — получить максимальную отдачу от инвестиций.

Просто помните, что важен не процесс, а цель — создание архитектуры, удовлетворяющей потребности бизнеса.

Компромиссы неизбежны

Некоторые люди всегда выбирают определенный прием, подход или инструмент, независимо от решаемой задачи. Часто они выбирают то, что привело к успеху в прошлом. Иногда у них возникает так называемый синдром блестящего предмета*, когда они думают, что какая-то новая технология или метод решат все их проблемы.

Независимо от прошлых достижений или будущих обещаний помните: *у каждого преимущества есть свои недостатки*. Единственные вопросы, на которые вам нужно ответить: «Помогут ли преимущества реализовать успешное приложение?» и «Можно ли смириться с недостатками?»

Слыша хвалебные отзывы о каком-то подходе, вы должны спрашивать: *«А каковы его недостатки?»*

↗

Мы не говорим, что не нужно использовать новые инструменты и методы. От прогресса не спрячешься. Просто не забывайте учитывать компромиссы, принимая решения.

*Синдром блестящего предмета (Shiny Object Syndrome, SOS) — склонность отвлекаться на свои или чужие новые мысли и идеи, не будучи способным сосредоточиться и завершить что-либо. — *Примеч. ред.*

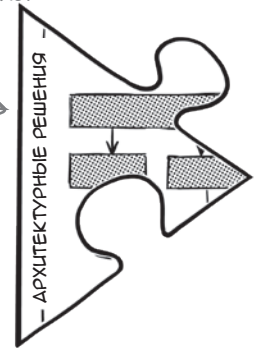
Принятие архитектурного решения

Обсуждать плюсы и минусы вместе с командой — это увлекательно, но рано или поздно вам *придется* принять **архитектурное решение**.

Мы уже упоминали архитектурные решения в главе 1, но можем немного углубиться в эту тему. Проектируя и создавая системы, вы должны будете принимать множество решений: начиная с общей структуры системы и заканчивая выбором используемых инструментов и технологий. Итак, что делает ваше решение *архитектурным*?

В большинстве случаев любой выбор, влияющий на *структуру* системы, является архитектурным решением. Вот несколько примеров.

Мы говорим о втором измерении архитектуры ПО.



К примеру, выбор между одно- и двухэтажным домом — это архитектурное решение.

Используем кэш, чтобы уменьшить нагрузку на базу данных и повысить производительность.

Обратите внимание, что такое решение добавляет новый элемент инфраструктуры. Это один из механизмов, о которых должны помнить разработчики, реализуя доступ к данным.

Полагаю, что задачу можно решить, разделив доставку и отслеживание заказов на два сервиса.

Организуем сервис отчетов как модульный монолит.

Этот пункт очевиден — он буквально описывает структуру сервиса.

Как говорилось в главе 1: «Архитектурные решения служат ориентирами, помогающими разработчикам понять условия и ограничения архитектуры».

Обратите внимание, что эти решения действуют как *ориентиры*, а не правила, и помогают сделать выбор, не будучи слишком конкретными. Большинство архитектурных решений, которые вам предстоит принять (но не все), будут касаться структуры ваших систем.



Что еще делает решение архитектурным?

Обычно архитектурные решения влияют на *структуру*, как, например, выбор между монолитом и микросервисами. Но иногда желательно сохранить определенные архитектурные характеристики. Если безопасность имеет важное значение, то в Two Many Sneakers может быть принято такое решение:

Для асинхронной связи между сервисами мы будем использовать очереди.

Это решение не связано со структурой, а продиктовано требованиями безопасности. Для каждого абонента создается своя очередь, поэтому мы точно знаем своих потребителей.

Как вы помните, темы позволяют любому сервису подписаться и получать сообщения.

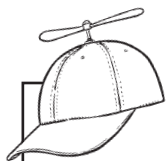
В других случаях решение может касаться выбора конкретного инструмента, технологии или процесса, влияющих на архитектуру или помогающих достичь определенных архитектурных характеристик. Например:

Для разработки MVP мы будем использовать фреймворк Node.js.

MVP означает Minimum Viable Product — минимально жизнеспособный продукт.

Например, если нужно быстро выйти на рынок или у вас большой штат инженеров с опытом работы именно с этим технологическим стеком.

Все, что было написано выше в данной главе, является подготовкой к этому важному моменту — принятию архитектурного решения. Сначала вы анализируете компромиссы. Затем рассматриваете плюсы и минусы вариантов с точки зрения бизнеса и клиентов, архитектурные характеристики, техническую осуществимость, временные и бюджетные ограничения и даже проблемы разработки. И *наконец*, принимаете решение.



Серьезное программирование

Майкл Нейгард (Michael Nygard), автор книги *Release It!*, определяет архитектурное решение как «нечто, влияющее на выполнение остальной части проекта» или способное «влиять на структуру, нефункциональные характеристики, зависимости, интерфейсы или методы реализации» архитектуры. Мы рекомендуем прочитать его статью *Documenting Architecture Decisions* (размещенную в блоге <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>).

Погодите-ка! Доски — это отлично, но результаты анализа, принятые решения и, самое главное, причины, приведшие к этим решениям, желательно зафиксировать и сохранить. Доски для этого совершенно не годятся, не так ли?



Это вы верно подметили. Важно сохранить принятые решения. Кроме того, анализ компромиссов — сложный процесс, и было бы жалко потерять плоды труда только потому, что кто-то поспешил вытереть доску.

Вы сделали еще одно важное замечание: решение важно само по себе, но *причины* его принятия могут быть еще важнее. Что приводит нас к...

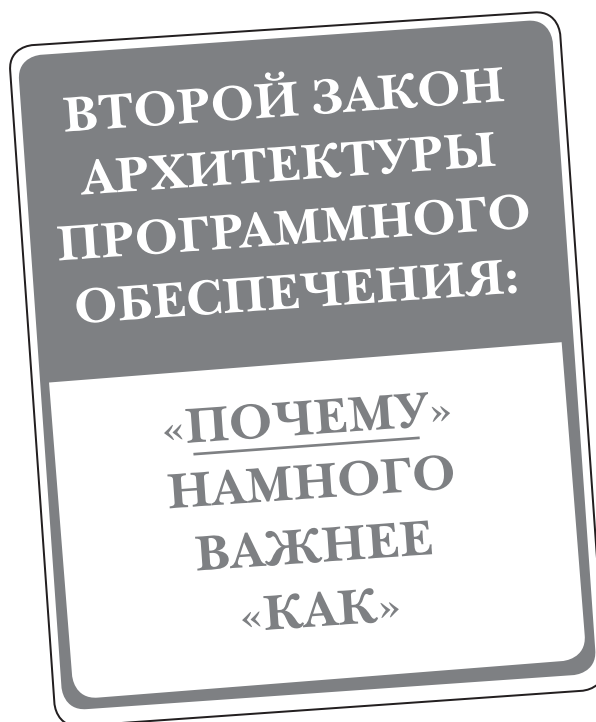
Второй закон архитектуры ПО

Принятие решений — одна из самых важных задач архитекторов программного обеспечения.

Допустим, вы и ваша команда проанализировали компромиссы и решили использовать кэш, чтобы повысить производительность приложения. Результат анализа — применение кэша в системе. Это — *что*, которое легко заметить.

Решение важно само по себе, но не менее значимы обстоятельства его принятия, то, как оно повлияет на команду разработчиков и *почему* из всех доступных вариантов выбрано именно оно.

Это приводит нас ко *Второму закону архитектуры программного обеспечения*.



Спустя время другие архитекторы (или вы сами), возможно, сумеют понять, *что* вы сделали и даже *как*, но им будет трудно уяснить, *почему* вы так сделали. Не имея этого понимания, они могут напрасно потратить время на изучение решений, которые вы уже отвергли, или упустить ключевой фактор, повлиявший на ваш выбор.

Вот для чего существует Второй закон. Вы должны понять и записать все «почему» каждого решения, чтобы они не затерялись в песках времени.

Итак, как *запечатлеть* архитектурные решения? Об этом мы поговорим дальше.

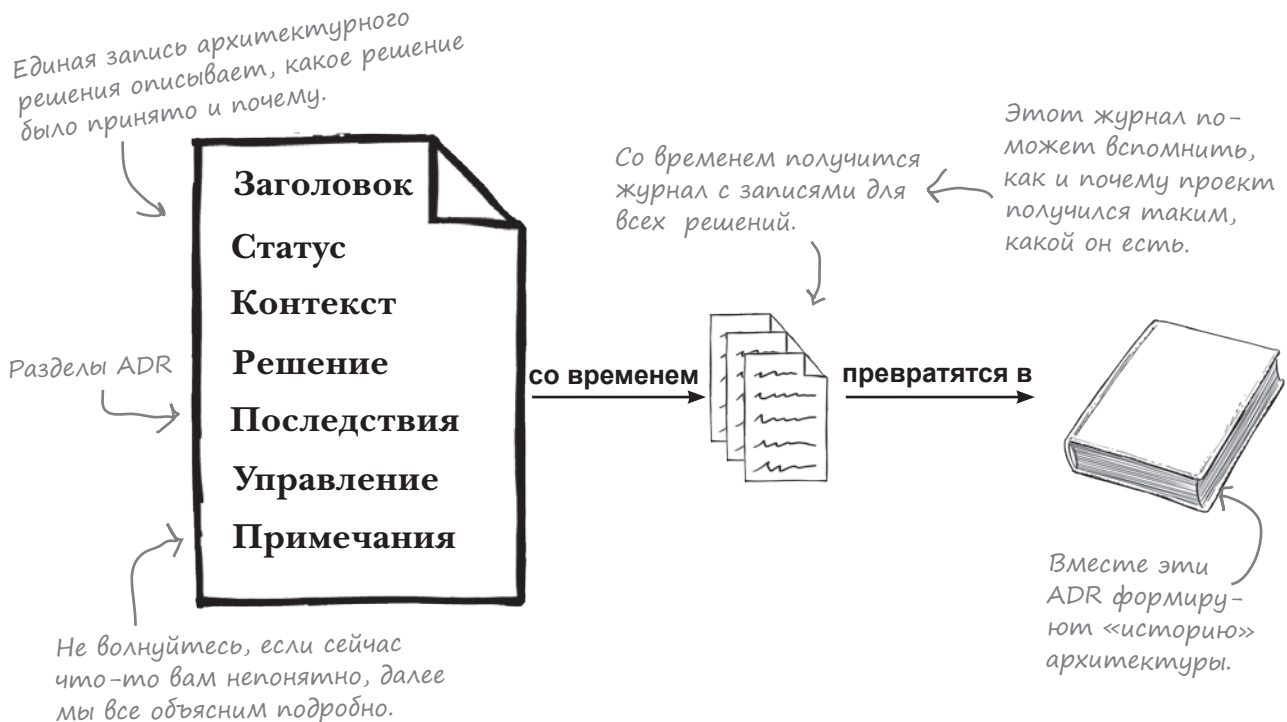
Записи об архитектурных решениях (ADR)

Вы помните все, что делали на прошлой неделе? Нет? Мы тоже. Вот почему нужно документировать важную информацию.

Учитывая Второй закон архитектуры программного обеспечения, мы должны зафиксировать не только само решение, но и причину, по которой его приняли. Для этого архитекторы используют *записи об архитектурных решениях* (architectural decision record, ADR), определяющие конкретный шаблон.

Мы не можем не подчеркнуть, насколько важно вести эти записи.

ADR — это документ, описывающий конкретное архитектурное решение. Он составляется для каждого принимаемого решения. Со временем эти записи превратятся в *журнал архитектурных решений*. Помните, что архитектурные решения образуют второе измерение архитектуры, а ADR — это документация, поддерживающая данное измерение.



ADR состоит из семи разделов: «Заголовок», «Статус», «Контекст», «Решение», «Последствия», «Управление» и «Примечания». В этих разделах отражаются все аспекты архитектурного решения, в том числе само решение. Рассмотрим их подробнее.

Разговор в кабинете



Догадались? Далее вы
поможете команде
писать ADR, выпол-
няя упражнения.

Алекс. Анализ компромиссов между очередями и темами утомил меня.

Мара. Меня тоже. Анализ был трудным, но я рада, что мы его провели. Это важное архитектурное решение, и поэтому мы должны понимать все плюсы и минусы каждого варианта.

Сэм. Итак, мы решили использовать очереди, верно? Теперь мы можем вернуться к программированию?

Мара. Не спеши. Ты прав, мы приняли решение. Но теперь мы должны записать это решение в ADR.

Сэм. Но зачем? Мы уже знаем, что будем делать, и нам предстоит большой объем работы.

Алекс. Послушайте. Мы же знаем, почему выбрали очереди, — этот вариант лучше всего поддерживает нужные нам архитектурные характеристики. К чему лишние хлопоты?

Мара. Да, мы знаем, почему приняли такое решение, но как это узнают, например, будущие сотрудники? Вот почему мы должны записать наши соображения.

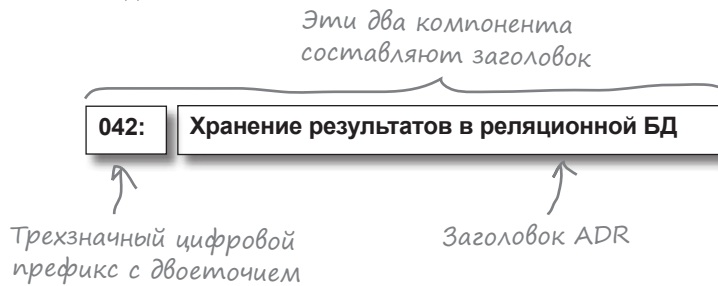
Сэм. Да, в этом есть смысл.

Алекс. Хорошо! Тогда приступим к созданию ADR?

Создание ADR: выбор хорошего заголовка

Каждая запись ADR начинается с **заголовка**, описывающего решение. Тщательно продумайте его. Он должен быть содержательным и в то же время кратким. Хороший заголовок помогает понять, о чем идет речь, что особенно удобно, когда вы лихорадочно ищете ответ!

Как выглядит хороший заголовок ADR? Представьте, что команда создает сервис, проводящий опросы клиентов. Разработчики проанализировали компромиссы и решили хранить результаты в реляционной базе данных. Заголовок их ADR может выглядеть так:



Заголовок должен состоять в основном из существительных. Будьте лаконичны: вам еще представится возможность подробно описать решение. Заголовок должен *отражать суть* ADR подобно заголовку новостной статьи или записи в блоге.

Заголовок должен начинаться с номера — используйте трехзначные числа с ведущими нулями. Это позволит вам *последовательно* нумеровать записи, начиная с 001 и заканчивая 999. Добавляя новое решение, вы должны увеличивать число, которое присваиваете ему. Такой прием поможет другим узнать, какие решения принимались раньше, а какие позже.



Упражнение

В следующих упражнениях вы поможете команде Two Many Sneakers написать ADR. Разработчики решили использовать асинхронный обмен сообщениями, связав торговый сервис с другими сервисами с помощью очередей. Предположим, что команда создает уже **12-ю** запись. Какой заголовок вы бы дали этой ADR? Не забудьте о номере! Запишите свои мысли здесь. Наш вариант вы увидите в конце главы.

→ Решение на с. 150



Часто задаваемые вопросы

В: А если мы добавим больше записей ADR, чем 999?

О: Тогда вам придется исправить множество заголовков (и, возможно, имен файлов). Но нам всегда хватало трехзначного префикса.

В: Можно ли использовать один номер для разных ADR?

О: Каждая запись должна иметь уникальный номер. Это уменьшит путаницу.

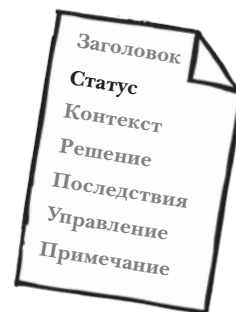
Подробнее об этом мы поговорим в следующем разделе, посвященном статусу.

Создание ADR: определение статуса

Отлично! Вы выбрали описательный заголовок. Теперь нужно определить *статус* ADR. Он сообщает, на какой стадии находится принятие решения.

Постойте-ка! Разве смысл ADR не в том, чтобы *зафиксировать решение*? Так и есть, но принятие решения — это процесс.

ADR не только фиксируют решения, но и выступают в роли *документации*, упрощая сотрудничество. Другим разработчикам может понадобится просматривать или даже утверждать запись. Перечислим статусы, которые может иметь ADR.



Это как планы на вечер. Вы хотите сходить куда-нибудь, знаете, кого из друзей пригласить, и надеетесь, что они согласятся на ресторан.



Запрос на обсуждение (RFC)

Статус «Запрос на обсуждение» (Request For Comments, RFC) присваивается записям ADR, когда требуется дополнительная информация, например, от других команд или экспертов. Часто подобные ADR затрагивают несколько команд или решают сквозные проблемы, такие как безопасность. Они открыты для критики со стороны любого приглашенного. В этом статусе *всегда* должен быть указан срок завершения обсуждения.

Вы просите всех откликнуться до вторника, чтобы успеть забронировать столик. (Крайний срок важен, поскольку Тед всегда был нерешительным.)



Предложено

Когда все желающие выскажут замечания, статус ADR меняется на «Предложено», означающий, что ADR ожидает одобрения. Вы можете исправить или даже пересмотреть решение, обнаружив ограничения, делающие его неприемлемым. Иначе говоря, решение еще не принято, но вы уже на пути к этому.

Вы запланировали встречу, но еще не отправили приглашение — вдруг погода изменится.



Принято

Означает именно то, что написано. Решение принято, и все причастные с ним согласны. Кроме того, статус «Принято» сообщает команде, отвечающей за реализацию, что они могут приступить к работе.

Все проголосовали, и можно выбирать наряд!

Если отзывы других разработчиков не нужны, то статус «Принято» можно установить сразу после принятия решения. Большинство ADR остаются в статусе «Принято», но есть еще один статус, о котором следует знать, — «Отменено».

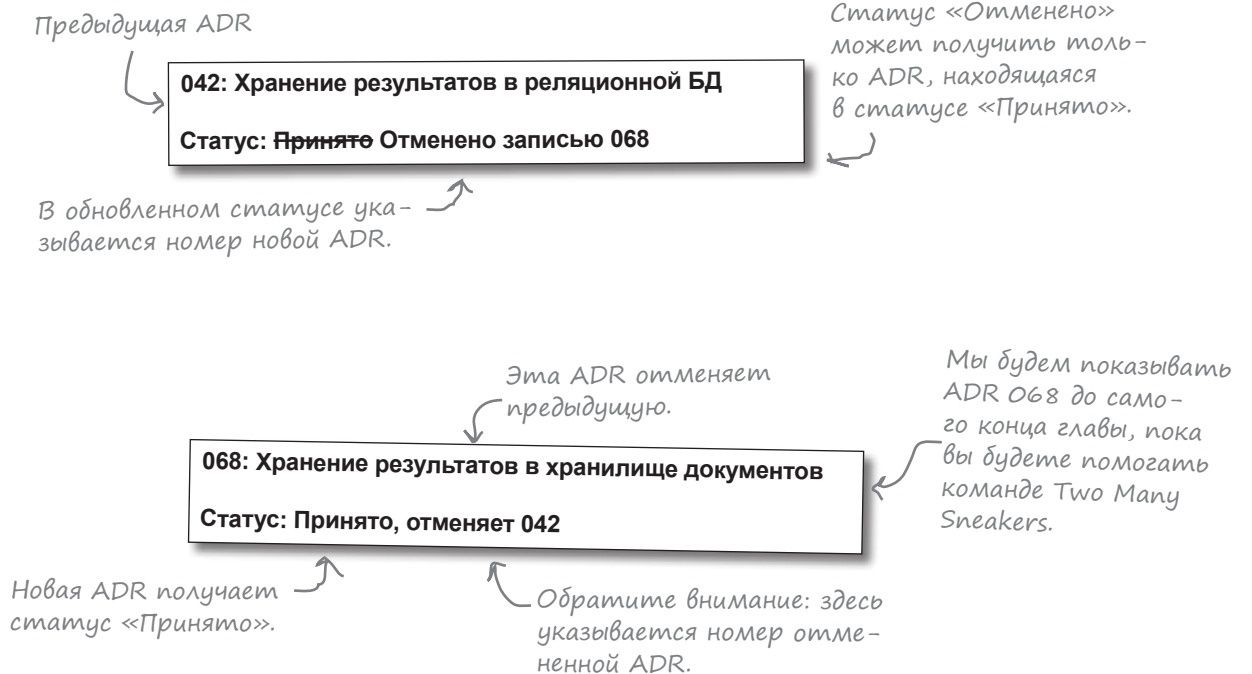
Создание ADR: определение статуса (продолжение)

Вы пришли к решению, которое старательно фиксируете в ADR. Подписали, запечатали и отправили — готово.

Но потом что-то меняется.

Возможно, бизнес растет, и совет директоров решает сосредоточиться на масштабируемости, а не на скорости выхода на рынок. Или компания вышла на международный рынок и должна соблюсти требования ЕС по защите и хранению данных. Независимо от причин, принятое вами решение оказалось неподходящим. Что теперь делать?

Написать новую ADR, которая *отменяет* прежнюю. Представьте, что разработчики сервиса опросов поняли, что реляционная база данных их потребности не удовлетворяет. Они снова проанализировали компромиссы и решили использовать хранилище документов. Вот заголовки и статусы старой и новой ADR:



Запись ADR может перейти в статус «Отменено», если появляется новая запись, изменяющая решение, задокументированное в старой. В обеих ADR важно указать, какая запись была *отменена* и какая ее *отменила*. Такая двусторонняя связь поможет читающему отмененную запись быстро понять, что она больше не актуальна, и определить, где искать новую, а читающему новую запись — быстро отыскать отмененную и выяснить, какие решения уже были опробованы.

Связывание записей ADR является важной частью «истории» проекта и помогает вспомнить, что уже было опробовано.

Часто Задаваемые Вопросы

В: Все эти отмены и нумерация кажутся слишком сложными. Не лучше просто исправить исходную ADR?

О: Мы используем трехзначные номера, так как они помогают понять последовательность действий. Допустим, ADR 007 оказалась неприменима в текущей ситуации, но за это время вы приняли ряд архитектурных решений. Пусть последняя ADR в вашем журнале решений имеет номер 013.

Потом вы решили пересмотреть ADR 007 и исправить ее, а не добавлять ADR 014. Что тогда случится?

Хронологически изменения в ADR 007 вносятся после принятия ADR 013. Но тот, кто попытается проследить процесс принятия решений, увидит ADR в неправильном порядке! Он может

подумать, что более новое решение было принято первым, и не поймет, что сначала вы приняли одно решение, а потом изменили его. Присваивая новый номер, вы даете понять, что старая ADR 007 стала неактуальной после ADR 013. Еще не запутались?

В: То есть принятую ADR нельзя изменить после принятия?

О: Именно! За исключением изменения статуса «Принято» на «Отменено», решение в ADR остается неизменным. Конечно, вы можете отредактировать ADR, добавив дополнительную информацию, но кроме статуса, кардинально ничего не должно меняться.



Упражнение

Выше вы придумали заголовок для записи с решением об использовании очередей. Допустим, что она была принята. Запишите выбранный заголовок в поле ниже и присвойте статус своей ADR:

Заголовок: _____

Статус: _____

Три месяца спустя:

Требования изменились, и последний анализ компромиссов показал, что темы подходят лучше. Все согласились с этим, и теперь вам нужно заменить прежнюю ADR новой, с номером 21. Запишите заголовок и новый статус старой ADR:

Заголовок: _____

Статус: _____

А здесь запишите заголовок и статус новой ADR:

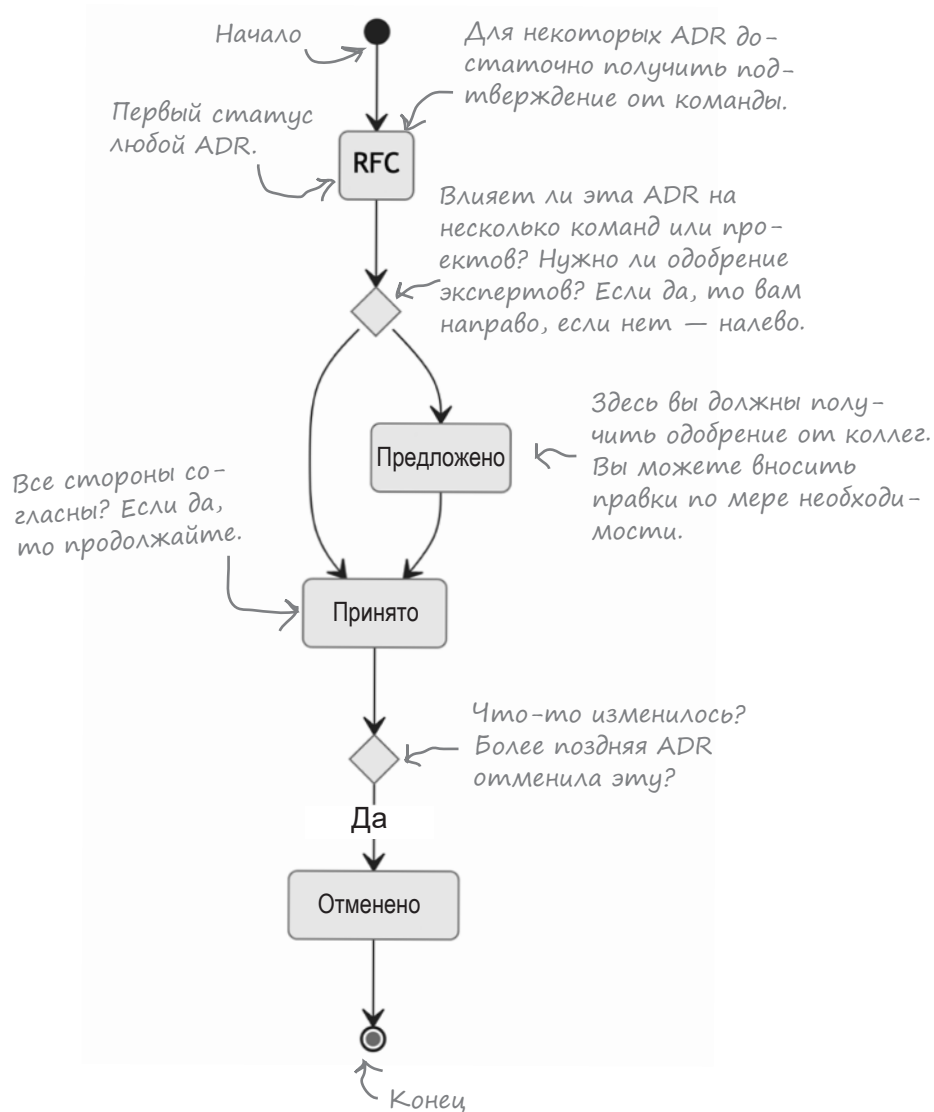
Заголовок: _____

Статус: _____

→ Решение на с. 150

Создание ADR: определение статуса (резюме)

Изменение статуса ADR связано со множеством аспектов, поэтому мы нарисовали схему, которая поможет вам запомнить их.



Создание ADR: определение контекста

Контекст важен. Любое решение принимается в определенном контексте и имеет конкретные ограничения. Выбирая, что съесть на завтрак сегодня утром, вы могли учитывать наличие аппетита, свое самочувствие, планы на обед и предписания диетолога. В архитектуре программного обеспечения все обстоит точно так же.

В разделе «Контекст» вы можете описать обстоятельства и факторы, повлиявшие на принятие решения. Обычно в этом списке перечисляются технологические причины, но нередко в него вносятся культурные или политические факторы, чтобы помочь читателю понять, как было принято это решение.

Продолжим работу над ADR, для которой мы уже написали раздел «Статус».

Контекст

Нам необходимо упростить хранение ответов клиентов. В настоящее время данные хранятся в реляционной БД и по мере доработки (например, при внедрении новых или расширенных опросов для клиентов премиум-класса) требования к схеме становятся все более жесткими.

На выбор доступно несколько вариантов, таких как тип данных JSONB в PostgreSQL или хранилища документов MongoDB.



Раздел «Контекст» отвечает на вопрос: «Почему сделан этот выбор?»

Обратите внимание, что мы не описываем решение! Для этого есть другой раздел.



Возьмите в руку карандаш

Продолжите работу над ADR для приложения Two Many Sneakers. Опишите контекст, в котором принято решение использовать очереди для связи торгового сервиса с другими сервисами. (Затем сравните его с нашим описанием в конце главы.)

→ Решение на с. 151

Часто задаваемые Вопросы

В: А как быть с тем, что мы писали раньше на доске? Это часть контекста?

О: Если хотите задокументировать анализ компромиссов, то заведите новый раздел «Альтернативы», перечислите в нем все рассмотренные альтернативы, а затем их плюсы и минусы.

Отдельный раздел для анализа компромиссов позволит избежать загромождения раздела «Контекст».

Создание ADR: описание решения

Наконец мы пришли к окончательному решению. Для начала рассмотрим раздел «Решение», заполненный разработчиками сервиса опроса клиентов:



Решение

Обратите внимание
на авторитетный тон

Для сервиса опросов мы будем использовать хранилище документов.

Отдел маркетинга требует обеспечить быстрый и гибкий способ внесения изменений в опросы.

Переход на хранилище документов увеличит гибкость и скорость, а также упростит внесение изменений за счет оптимизации пользовательского интерфейса.

Раздел «Решение» описывает, почему принято решение: помните Второй закон?

ADR со статусом «Запрос на обсуждение» (RFC) или «Предложено» описывает непринятое (пока) решение. Тем не менее раздел «Решение» начинается с **четкого описания принятого решения**. Лучше всего использовать авторитетный тон, применяя недвусмысленные фразы, такие как «мы будем использовать» (вместо «мы полагаем» или «мы думаем»).

Кроме того, в этом разделе можно объяснить, почему принято это решение, следуя Второму закону архитектуры программного обеспечения: «Почему» намного важнее, чем «как». В будущем вы и любой другой, кто прочитает ADR, поймете не только решение, но и его **обоснование**.

В разделе «Контекст» объясняются мотивы принятия решения. В разделе «Решение», следующем сразу за ним, объясняется само решение. Изложенный в них материал поможет читателю правильно понять решение.

Вдобавок это отличное место для перечисления подписавшихся под этим решением. Например, «Отдел маркетинга требует...».

Своеобразное прикрытие! :)



Будьте
осторожны!

ADR — это не выражение мнения.

Помните, что ADR — это не место для выражения мнений о положении вещей. В это состояние легко впасть при обосновании решения. Даже объясняя контекст, бывает трудно оставаться объективным.

Работая над ADR, придерживайтесь фактов и сохраняйте нейтральный тон.



Упражнение

Пришло время заполнить раздел «Решение» в ADR для приложения Two Many Sneakers. Вот основные факторы, которые учитывались при принятии решения:

- очереди позволяют отправлять гетерогенные сообщения;
- защищенность — важная архитектурная характеристика для заинтересованных сторон.

Мы оставили немного места, чтобы вы могли записать решение и его обоснование. Вы должны ответить на вопрос «Почему именно очереди?» *Подсказка: сосредоточьтесь на решении и «почему».* Наш вариант вы найдете в конце главы.

Вспомните анализ компромиссов, проводившийся выше в этой главе.

Решение на с. 151

Часто
Задаваемые
Вопросы

В: Мне не совсем понятна разница между контекстом и обоснованием, которое мы приводим в разделе «Решение». Разве это не одно и то же?

О: Рассмотрим простой пример. Допустим, у вашего лучшего друга день рождения, и вы с друзьями решили отпраздновать его в ресторане. Это — **контекст**, обстоятельства, в которых принимается решение.

Прежде чем окончательно определиться, вы можете составить список ресторанов (доступных альтернатив) и постараться учесть, насколько предлагаемая ими кухня соответствует предпочтениям каждого участника праздника. Это похоже на анализ компромиссов.

Вы выбираете ресторан с азиатской кухней — это решение. А выбираете вы его потому, что в меню есть диетические блюда и ваши друзья, соблюдающие особый режим питания, смогут подобрать блюдо на свой вкус и насладиться вечером. Это — обоснование вашего решения.

Создание ADR: оценка последствий

Каждое решение имеет *последствия*. Вчера вы усердно тренировались? Тогда сегодня вы можете чувствовать боль в мышцах (и гордость за себя)!

Важно правильно оценить и задокументировать последствия архитектурных решений: и хорошие, и плохие. Это повышает *прозрачность* и гарантирует, что команда, на которую влияет решение, понимает его. Но самое главное, все могут оценить, перевесят ли положительные последствия решения его минусы.

Последствия могут быть как незначительными, так и огромными. Архитектурные решения могут повлиять на все: команду, инфраструктуру, бюджет и даже на саму реализацию ADR. Вот неполный список вопросов, которые вы должны задать себе.

- Как это решение влияет на исполнителей? Например, меняет ли оно алгоритмы? Усложняет или облегчает тестирование? Как узнать, что реализация решения завершена?
- Вводит ли это решение какую-то инфраструктуру или выводит ее? Что это влечет за собой?
- Затрагивает ли решение такие сквозные проблемы, как защищенность или наблюдаемость? Если да, то как это скажется на всей организации?
- Как решение повлияет на сроки и бюджет? Увеличит ли расходы или даст экономию? Усложнит ли реализацию или, наоборот, упростит?
- Вводит ли решение какие-то ограничения? (Например, используя очереди мы не сможем контролировать порядок сообщений.) Если да, опишите их.

Сотрудничество с другими — отличный способ убедиться в тщательности вашей оценки. Как бы хорошо ни оценивались последствия, вы наверняка упустите что-то. Сторонний взгляд поможет вам заметить больше потенциальных последствий. Вот пример раздела «Последствия»:

Обозначьте последствия принятого решения, которые повлияют на команду.

Последствия

Для всех опросов будет использоваться единое представление, поэтому при изменении, добавлении или удалении общего вопроса придется вносить изменения в несколько документов.

На время миграции данных из реляционной базы данных в хранилище документов придется отключить функции опроса, что приведет к простоям.

Отметьте последствия, которые могут повлиять на клиентов или пользователей.



Время и деньги очень важны, поэтому продумайте все до мелочей!

Решение может упростить и удешевить реализацию, и на это стоит обратить особое внимание.

Возьмите в руку карандаш

Помогите команде Two Many Sneakers заполнить раздел «Последствия» в их ADR. Вот несколько моментов, о которых нужно подумать.

- Очередь — это новый элемент инфраструктуры.
- Сами очереди, вероятно, должны иметь высокую доступность.
- Как очереди влияют на связанность компонентов?

← Здесь нет правильных или неправильных ответов. Наше решение вы найдете в конце главы.

→ Решение на с. 152



Мозговой Штурм

Подумайте об архитектурном решении, принятом в вашем текущем или в прошлом проекте. Это может быть выбор языка программирования, структуры приложения или даже базы данных. Попробуйте назвать хотя бы два предполагаемых и два непредвиденных последствия этого решения.

Создание ADR: управление выполнением

Вы когда-нибудь принимали решение что-то начать делать с начала года, но не выполнили это до конца февраля? Может быть, вы записались в спортзал, заплатили, но так и не посетили его? Мы тоже. Решение является хорошим, только когда вы действуете в соответствии с ним и не отступаете от него в будущем.

Вы и ваша команда потратили время на анализ компромиссов и составление ADR. Но как теперь обеспечить воплощение решения в жизнь?

Эта роль возлагается на раздел «Управление», очень важный для любой ADR. В нем описывается, как обеспечить неуклонное выполнение решения сейчас и в будущем, например, с помощью ручных методов, таких как парное программирование и ревью кода, или автоматизированных, таких как специализированные фреймворки тестирования.



Один из авторов написал книгу «Эволюционная архитектура», в которой рассказывает, как использовать «фитнес-функции» для управления архитектурой. Обязательно прочитайте ее. (Когда закончите эту, конечно!)

Эти два раздела не входят в стандартный шаблон ADR, но мы считаем их очень важными.

Если слово «управление» навеивает мысли о нормативных требованиях, то вы ошиблись.



Создание ADR: примечания

Раздел «Примечания» содержит сведения о самой ADR. Вот список сведений, которые мы вносим в наши записи:

- первоначальный автор;
- дата одобрения;
- кем одобрено;
- дата отмены
- дата последнего изменения;
- кем изменено;
- последнее изменение.

Этот раздел пригодится, даже если инструмент, используемый для хранения ADR, автоматически записывает даты создания и изменения. Эта информация может повторяться, но, будучи частью ADR, легче поддается обнаружению.



Упражнение

Соберем все воедино! Вы написали разные разделы ADR для команды Two Many Sneakers. Теперь вернитесь к прошлым упражнениям и скопируйте свои разделы ADR на эту страницу, чтобы получить законченную ADR. Мы даем названия разделов, а вы должны заполнить их. (Предполагается, что статус будет иметь значение «Принято»). Наш вариант вы найдете в конце главы.

Заголовок

Статус

Контекст

Решение

Последствия

Вот она, полноценная ADR!

↙ Вы быстро растете! :)
Мы гордимся вами.

↖ Мы опустили разделы «Управление»
и «Примечания», поскольку они не явля-
ются частью стандартного шаблона.

→ Решение на с. 153

Часть Задаваемые Вопросы

В: Мне очень нравится шаблон записей ADR. Но где их хранить?

О: Вариантов много. Все зависит от того, что подходит вам и вашей команде и кому еще может быть интересно читать или изменять ADR.

Один из вариантов — хранить ADR в текстовых файлах (например, в файлах Markdown или AsciiDoc) в системе управления версиями, такой как Git. Это позволит иметь историю фиксаций, показывающую все изменения в ADR. Недостаток такого подхода — не все могут знать, как получить доступ к документам в системе управления версиями. Если вы решите хранить ADR именно так, то мы советуем создать отдельный репозиторий. Вы еще скажете нам спасибо за это.

Другой вариант — использовать Вики. Большинство таких систем предлагают визуальные редакторы, удобные для людей. Единственное пожелание — выбранная вами Вики должна отслеживать изменения. Если кто-то случайно отредактирует ADR, важно, чтобы остальные члены команды были в курсе.

Какой бы вариант вы ни выбрали, он должен позволять легко добавлять, изменять и искать ADR. Мы видели, как энтузиазм создания ADR затухал только потому, что никто не мог найти созданные ADR, даже если это было критически важно.

В: Наша команда обожает Markdown. (Текстовые файлы рулят!) Что вы посоветуете насчет выбора имен для них?

О: Напомним, что заголовки ADR начинаются с трехзначного префикса, за которым следует краткое описание, поэтому в качестве имен файлов с ADR мы советуем использовать заголовки, в том числе префиксы. Например, ADR с заголовком «042: Использование очередей для взаимодействий» можно сохранить в файле с именем *042-использование-очередей-для-взаимодействий.md*. Мы предпочитаем использовать только строчные буквы, чтобы избежать путаницы в разных операционных системах, а пробелы заменять дефисами.

Вам придется придумывать хорошие заголовки! А трехзначный префикс поможет расположить файлы в нужном порядке, отсортировав их по имени.

В: Можете ли вы посоветовать какие-либо инструменты, упрощающие создание ADR и управление ими?

О: Конечно! Есть инструменты командной строки, инструменты для конкретного языка, позволяющие писать ADR прямо в исходном коде. Список инструментов вы найдете на сайте <https://adr.github.io/#decision-capturing-tools>.

Большинство инструментов предполагают использование определенного формата ADR — они генерируют файлы Markdown или используют определенную структуру каталогов. Попробуйте поработать с инструментом, чтобы оценить его.

И наконец, проверенный совет: стремитесь к простоте. Начинать писать ADR без сложных инструментов или автоматизации. Наработайте начальные навыки, а потом, когда ваши потребности возрастут, выберите более подходящий инструмент.

В: ADR всегда относятся к одному проекту или могут затрагивать несколько проектов и команд и даже всю организацию?

О: Да, да и да. ADR могут быть настолько узкими или широкими, насколько вы захотите. Есть такие, которые затрагивают один проект и одну команду. Некоторые имеют отношение к нескольким или всем командам. В компании Amazon существует ADR, которую ласково называют «мандатом API Джеффа Безоса». Она отражает решение, принятое основателем компании Джеффом Безосом: взаимодействия между сервисами Amazon возможны только через API. Это коснулось всей организации, что немало важно, учитывая ее размеры.

Большинство ADR, затрагивающих несколько проектов или команд, требуют активного сотрудничества, а зачастую и одобрения некого центрального экспертного совета. Такие ADR обычно касаются сквозных проблем, например выбора механизмов и протоколов взаимодействий между сервисами. Кроме того, ADR, связанные с безопасностью или нормативными требованиями, почти всегда имеют отношение к нескольким командам или всей организации.

Преимущества ADR

Надеемся, нам удалось показать, что запись решений в ADR — это не обязательно долгий и трудный процесс. Нам очень нравится формат, представленный в данной главе, но вы можете изменять и дополнять его.

Действительно ли важно записывать архитектурные решения? Мы уверены, что да! Запись решений дает массу преимуществ — не только вам и вашей команде, но и всей вашей организации. Подведем итоги.

ADR — это записи об устройстве системы и особенностях ее работы, которые каждый может просмотреть в любое время.



ADR помогают командам предотвратить повторное прохождение пути, который другие уже опробовали, и поощряют поиск новых решений.

Почему бы не использовать хранилище документов?

Ага! Вот отмененное решение — видимо, когда-то так и было.

Двусторонняя связь между отмененными и принятыми ADR дает огромное преимущество.



ADR помогут узнать ответ на вопрос «Почему принято такое решение?», даже если рядом не окажется никого из тех, кто его принимал.

ADR — это ключ к накоплению знаний. Команды могут читать ADR друг друга и учиться.



Будьте осторожны!

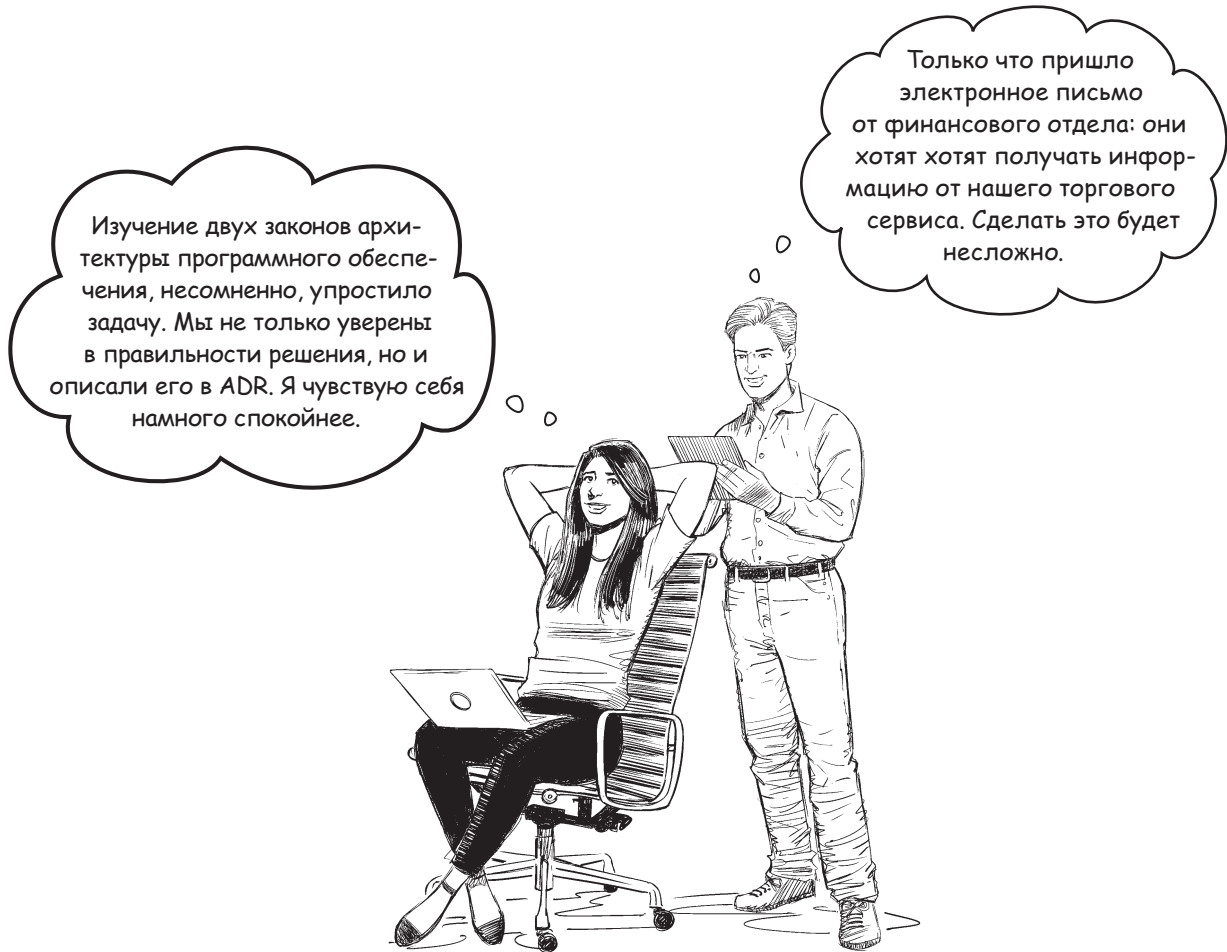
Максимально упрощайте процесс создания ADR.

Не добавляйте новые разделы в шаблон ADR, надеясь сделать его всеобъемлющим. Это благородная цель, но она чревата дополнительной работой. Излишние усложнения сделают процесс документирования трудоемким, и из-за этого некоторые разработчики могут вообще перестать писать ADR.

Сосредоточьтесь на краткости и простоте. Позже вы скажете нам спасибо.

Two Many Sneakers празднует успех

Команда Two Many Sneakers в восторге. Их клиентам понравилось получать уведомления о новых предложениях в режиме реального времени, а улучшенная аналитика дает службе безопасности информацию, помогающую выявлять мошеннические действия с кроссовками.



Знание этих двух законов сослужит вам хорошую службу. Теперь вы знаете, что в архитектуре ПО не существует «лучших практик» — только компромиссы. Именно вам (и вашей команде) решать, какой вариант наиболее приемлем. И не забудьте записать свое решение в ADR!

Дерзайте!

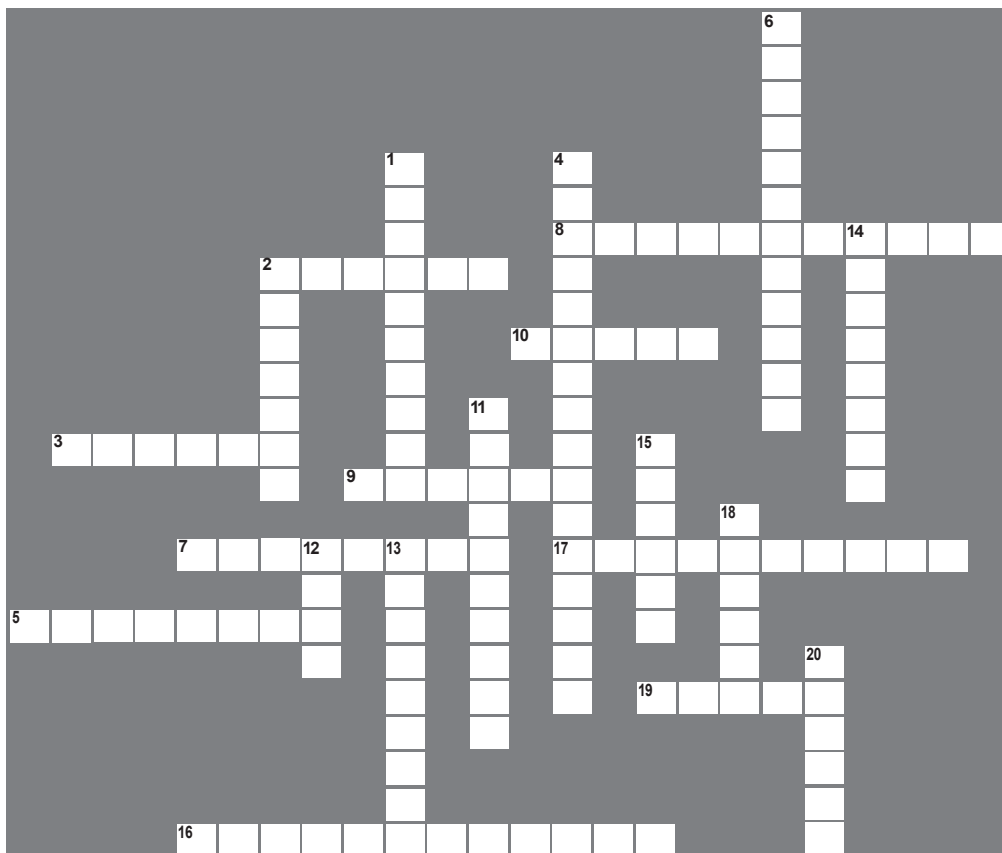
Ключевые моменты

- Архитектура не статична. Она постоянно меняется и развивается.
- Требования и обстоятельства меняются, и вы будете менять свою архитектуру, чтобы достичь новых целей.
- Рассматривая варианты, проведите *анализ компромиссов*. Это поможет выбрать лучший (или наименее худший) из них.
- **Первый закон архитектуры программного обеспечения:** все основано на компромиссах.
- На любой вопрос в архитектуре ПО есть только один ответ: «Зависит от ситуации». Чтобы выявить решения, подходящие в конкретной ситуации, определите основные *приоритеты и цели*. Каковы требования? Какие аспекты особенно важны для бизнеса и клиентов? Вы спешите выйти на рынок или надеетесь добиться стабильного роста?
- Результат анализа компромиссов — *архитектурное решение*: одно из четырех измерений, необходимых для описания любой архитектуры.
- Принятие *архитектурного решения* предполагает оценку плюсов и минусов каждого варианта в свете культурных, технических, коммерческих и клиентских потребностей и выбор варианта, лучше всего соответствующего им.
- Принятие архитектурного решения — это не только совершение выбора, но и понимание причин, *почему* выбирается именно этот вариант.
- **Второй закон архитектуры программного обеспечения:** «почему» намного важнее, чем «как».
- Фиксируйте свои решения в форме *записей об архитектурных решениях* (architectural decision records, ADR). Они состоят из семи разделов: «Заголовок», «Статус», «Контекст», «Решение», «Последствия», «Управление» и «Примечания».
- Со временем ваши записи ADR превратятся в *журнал архитектурных решений*, который послужит хранилищем истории вашего проекта.
- *Заголовок* ADR должен содержать трехзначный цифровой префикс и краткое описание решения, состоящее в основном из существительных.
- Записи ADR можно присвоить один из множества *статусов*, в зависимости от ее вида и места в процессе принятия решения.
- Когда все, кто участвует в принятии решения, утвердят ADR, она получает статус «Принято».
- Если новое решение отменяет прежнее, то вы должны написать новую ADR. Прежняя получает статус «Отменено», а новая — «Принято».
- В разделе «*Контекст*» объясняется, почему принято такое решение.
- В разделе «*Решение*» описывается принятое решение и дается ответ на вопрос «почему».
- В разделе «*Последствия*» описываются ожидаемые последствия решения, как хорошие, так и плохие. Это помогает убедиться, что хорошее перевешивает плохое.
- В разделе «*Управление*» перечисляются мероприятия, обеспечивающие правильное и неукоснительное выполнение решения.
- В последний раздел — «*Примечания*» — в основном записываются сведения о самой ADR: автор, время создания, утверждения и последнего изменения.
- ADR — важный инструмент, помогающий соблюсти Второй закон архитектуры ПО, поскольку наряду со «что» они фиксируют «почему».
- Записи ADR необходимы для накопления знаний и помогают командам учиться друг у друга.



Кроссворд «Два закона»

Проверим, насколько вы усвоили два закона архитектуры программного обеспечения. Разгадайте этот кроссворд.



По горизонтали

2. Гетерогенные (синоним).
3. _____ об архитектурных решениях (с семью разделами).
5. Мобильное приложение Two Many Sneakers взаимодействует с торговым _____.
7. Раздел ADR, описывающий причины принятия решения.
8. Высокая или низкая взаимозависимость.
9. Намного важнее, чем «как», согласно Второму закону.
10. Темы работают по принципу «запустил-и-_____».
16. Лучший тон для написания ADR.
17. Темы допускают независимый _____.
19. Плюсы и минусы можно записать на _____.

По вертикали

1. Другой способ сказать «непоследовательно».
2. Записывайте каждое принимаемое архитектурное _____.

4. Важная архитектурная характеристика для быстро растущего бизнеса.
6. Архитектурная характеристика, особенно важная для финансовых транзакций.
11. Чрезмерное увлечение новым инструментом может вызвать синдром _____ предмета.
12. К механизмам обмена сообщениями относятся очереди и _____.
13. Каким товаром занимается Two Many Sneakers?
14. Новая ADR может _____ старую.
15. Архитекторы несут ответственность за принятие _____ архитектурных решений.
18. ADR может иметь _____ «Принято».
20. «В архитектуре ПО все основано на компромиссах» — это _____ закон архитектуры ПО.

→ Решение на с. 154



Упражнение Решение

На с. 117

Какие из следующих архитектурных характеристик важны для этой конкретной задачи? Подсказка: здесь нет правильных ответов, поскольку многого мы пока не знаем, а в чем-то не уверены. Выбирайте по своему усмотрению. Вот наши мысли:

Бизнес процветает, и можно обратить внимание на это.

расширяемость

Информация нужна многим сервисам. Эта характеристика важна!

модульность

обновляемость

слабая связанность

производительность

защищенность



Возьмите в руку карандаш Решение

На с. 121

Потратьте несколько минут и сравните результаты анализа. Вы заметили, что каждый вариант поддерживает одни характеристики в ущерб другим? Теперь мы перечислим несколько требований, а вы должны определить вариант, который лучше их поддерживает. Вот наши ответы:

Требования

«Защищенность важна для нас».

«Разные сервисы должны получать разную информацию».

«В будущем мы расширим круг сервисов».

Очередь / Темы

Очередь / Темы

Очередь / Темы

Возьмите в руку карандаш

Решение

Теперь мы предлагаем вам самим проанализировать возможные варианты. Для связи торгового сервиса с потребителями мы выбрали обмен сообщениями — асинхронный протокол. Однако выбор между асинхронной и синхронной формами связи сопряжен со своими компромиссами! Мы даем вам две доски, по одной для каждой формы связи, и список «возможностей». Подумайте, как каждая архитектурная характеристика будет работать в обоих контекстах, будет ли для нее плюсом или минусом (или ни тем, ни другим) синхронная и асинхронная связь? Запишите результаты в соответствующие колонки. Подсказка: не все из приведенных характеристик применимы к данному решению. Вот наши ответы:

отзывчивость

обработка ошибок

транзакции

координация

согласованность

расширяемость

развертываемость

отказоустойчивость

Синхронная связь

Плюсы

согласованность

обработка ошибок

транзакции

Минусы

расширяемость

отказоустойчивость

отзывчивость

Асинхронная связь

Плюсы

отзывчивость

расширяемость

отказоустойчивость

Минусы

согласованность

обработка ошибок

транзакции



Упражнение Решение

На с. 131

В следующих упражнениях вы поможете команде Two Many Sneakers написать ADR. Разработчики решили использовать асинхронный обмен сообщениями, связав торговый сервис с другими сервисами с помощью очередей. Предположим, что команда пишет уже **12-ю** запись. Какой заголовок вы бы дали этой ADR? Не забудьте про номер! Вот наше решение:

012: Использование очередей для асинхронных взаимодействий между сервисами



Упражнение Решение

На с. 134

Выше вы придумали заголовок для записи с решением об использовании очередей. Допустим, что она была принята. Запишите выбранный заголовок в поле ниже и присвойте статус своей ADR:

Заголовок: 012: Использование очередей для асинхронных взаимодействий между сервисами

Статус: Принято

Три месяца спустя:

Требования изменились, и последний анализ компромиссов показал, что темы подходят лучше. Все согласились с этим, и теперь вам нужно заменить прежнюю ADR новой, с номером 21. Запишите заголовок и новый статус старой ADR:

Заголовок: 012: Использование очередей для асинхронных взаимодействий между сервисами

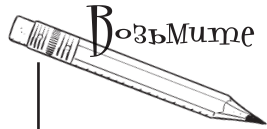
Статус: Отменено решением 021

А здесь запишите заголовок и статус новой ADR:

Заголовок: 021: Использование тем для асинхронных взаимодействий между сервисами

Статус: Принято, отменяет решение 012

На с. 136



Возьмите в руку карандаш Решение

Продолжите работу над ADR для приложения Two Many Sneakers. Опишите контекст, в котором принято решение использовать очереди для связи торгового сервиса с другими сервисами. Вот наше описание:

Торговый сервис должен информировать сервисы уведомлений и анализа о появлении в продаже новых товаров и обо всех транзакциях. Это можно сделать с помощью синхронного обмена сообщениями (используя REST) или асинхронного (используя очереди или темы).



Упражнение Решение

На с. 138

Пришло время заполнить раздел «Решение» в ADR для приложения Two Many Sneakers. Вот основные факторы, которые учитывались при принятии решения:

- очереди позволяют отправлять гетерогенные сообщения;
- защищенность — важная архитектурная характеристика для заинтересованных сторон.

Мы оставили немного места, чтобы вы могли записать решение и его обоснование. Вы должны ответить на вопрос «Почему именно очереди?» *Подсказка: сосредоточьтесь на решении и «почему».* Здесь нет правильных или неправильных ответов. Вот к чему пришли мы:

Мы будем использовать очереди для асинхронного обмена сообщениями между сервисами.

Использование очередей делает систему более расширяемой, поскольку разные очереди могут доставлять сообщения разного типа. Торговый сервис знает обо всех подписчиках, поэтому его придется изменить, чтобы добавить нового потребителя, что повышает безопасность системы.



Возьмите в руку карандаш Решение

Помогите команде Two Many Sneakers заполнить раздел «Последствия» в их ADR. Вот несколько моментов, о которых нужно подумать.

- Очередь — это новый элемент инфраструктуры.
- Сами очереди, вероятно, должны иметь высокую доступность.
- Как очереди влияют на связанность компонентов?

Очереди предполагают более тесную связь между сервисами.

Нам потребуется создать инфраструктуру очередей и использовать кластеризацию, чтобы обеспечить высокую доступность.

Если потребуется уведомлять дополнительные сервисы (помимо уже известных нам), то придется внести изменения в торговый сервис.

Вот наше мнение.



Упражнение Решение

На с. 142

Соберем все воедино! Вы написали разные части ADR для команды Two Many Sneakers. Теперь вернитесь к прошлым упражнениям и скопируйте свои разделы ADR на эту страницу, чтобы получить законченную ADR. Мы даем названия разделов, а вы должны заполнить их. (Предполагается, что статус будет иметь значение «Принято»). Вот наш вариант:

Заголовок

O12: Использование очередей для асинхронных взаимодействий между сервисами

Статус

Принято

Контекст

Торговый сервис должен информировать сервисы уведомлений и анализа о появлении в продаже новых товаров и обо всех транзакциях. Это можно сделать с помощью синхронного обмена сообщениями (используя REST) или асинхронного (используя очереди или темы).

Решение

Мы будем использовать очереди для асинхронного обмена сообщениями между сервисами.

Использование очередей делает систему более расширяемой, поскольку разные очереди могут доставлять сообщения разного типа. Торговый сервис знает обо всех подписчиках, поэтому его придется изменить, чтобы добавить нового потребителя, что повышает безопасность системы.

Последствия

Очереди предполагают более тесную связь между сервисами.

Нам потребуется создать инфраструктуру очередей, и использовать кластеризацию, чтобы достичь высокой доступности.

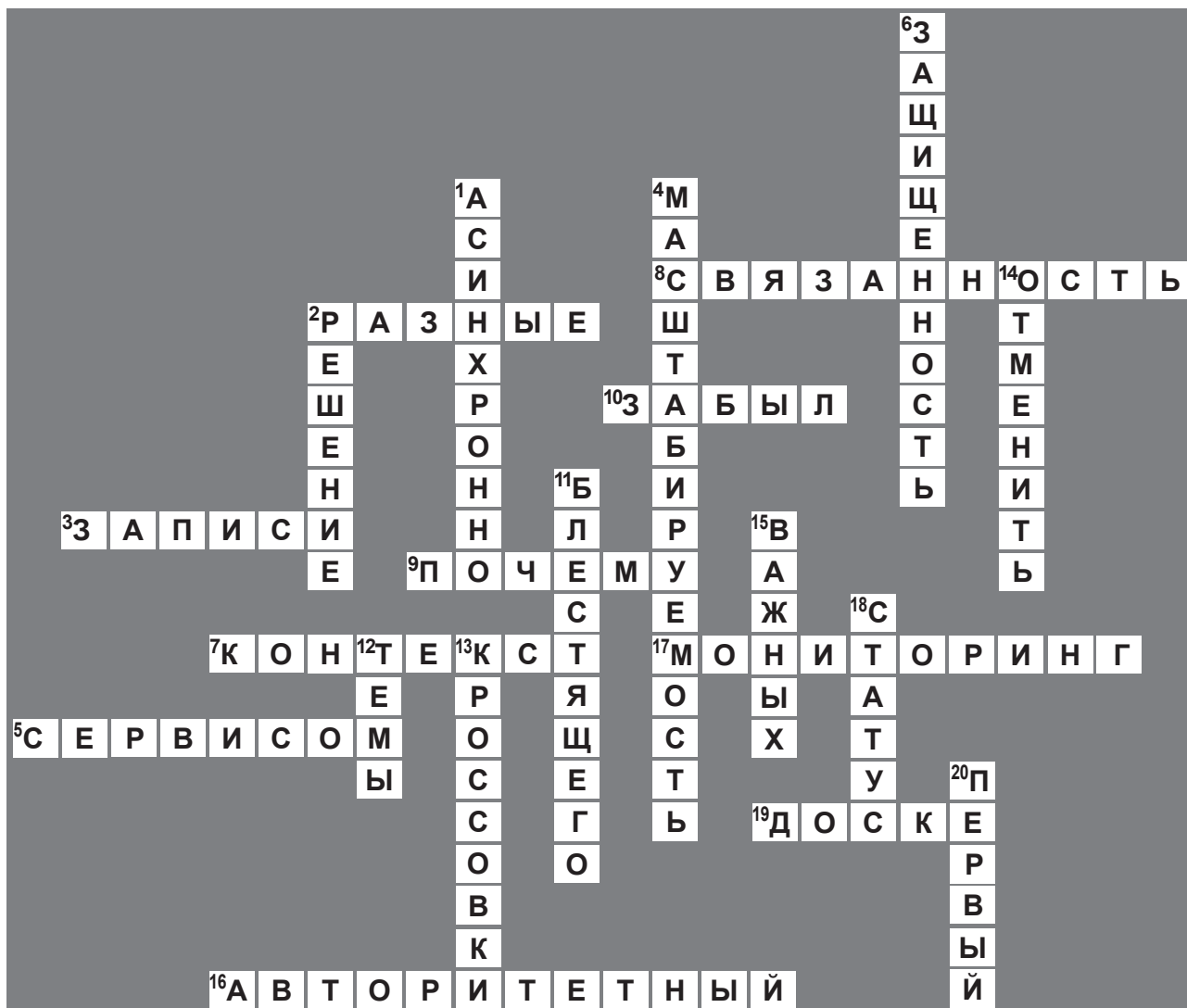
Если потребуется уведомлять дополнительные сервисы (помимо уже известных нам), то придется внести изменения в торговый сервис.

Вот она, полноценная ADR!



Кроссворд «Два закона». Решение

Ha c. 147



Строительные блоки

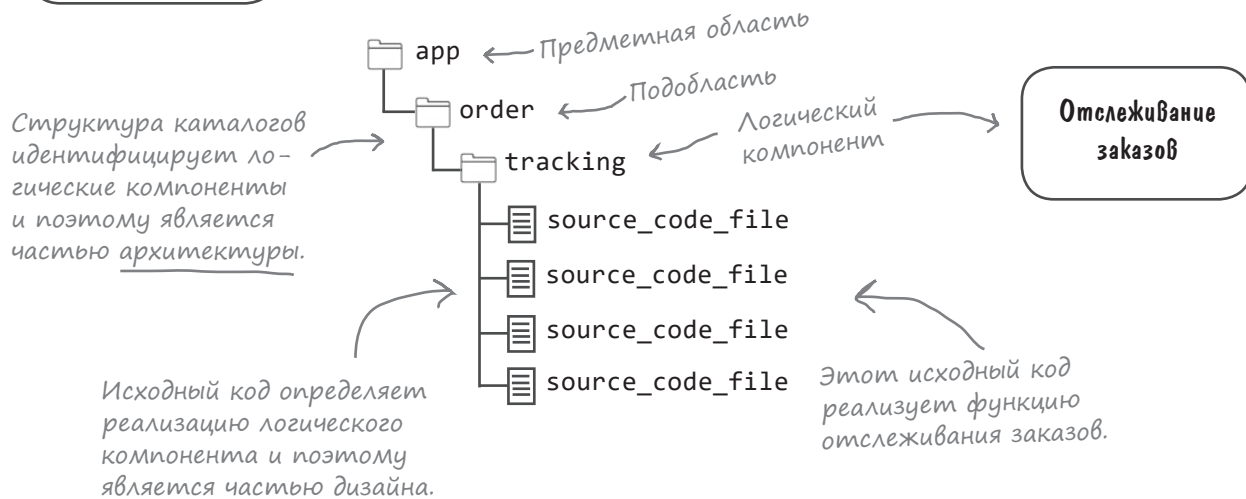
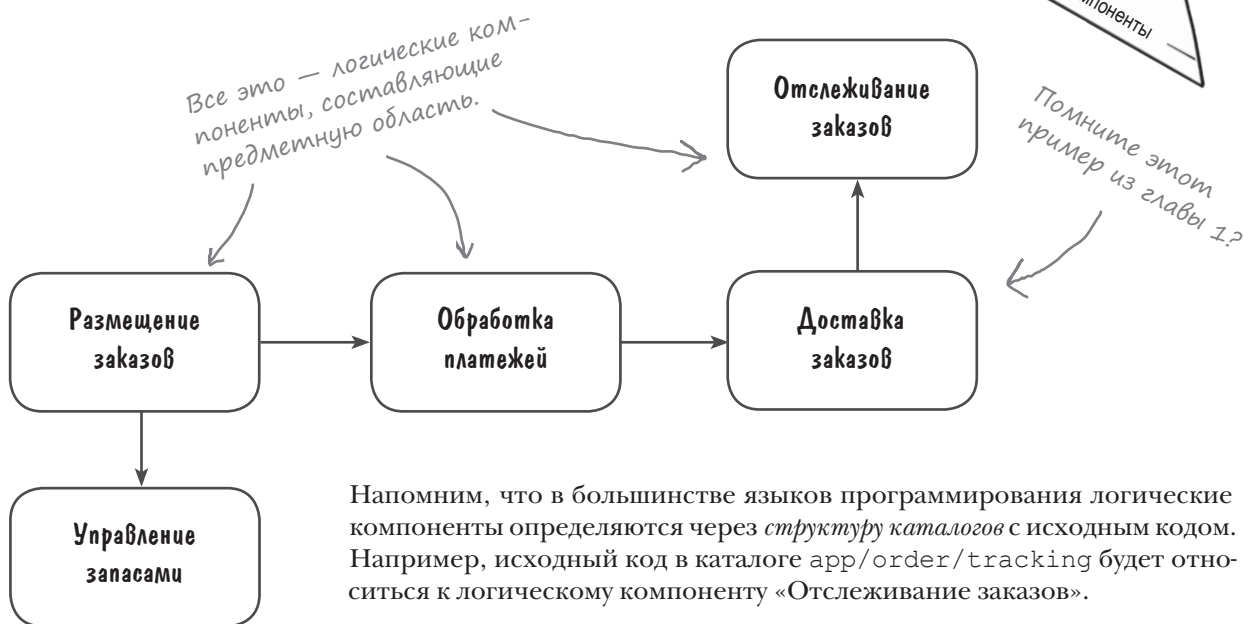
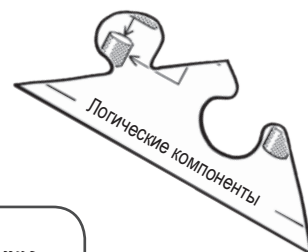


Готовы приступить к созданию архитектуры? Это не так просто, как кажется, и если сделать что-то неправильно, то программная система может обрушиться, как плохо спроектированный небоскреб или мост.

В данной главе показаны несколько приемов определения и создания *логических компонентов* — функциональных строительных блоков системы, которые описывают, как ее части соединяются друг с другом. Эти приемы помогут вам создать надежную архитектуру — фундамент эффективной программной системы.

Еще раз о логических компонентах

Логические компоненты — одно из измерений архитектуры ПО. Они представляют собой функциональные строительные блоки, образующие так называемую *предметную область*. Вы познакомились с ними в главе 1, а в этой мы поговорим более подробно о том, что они собой представляют и как их создавать.

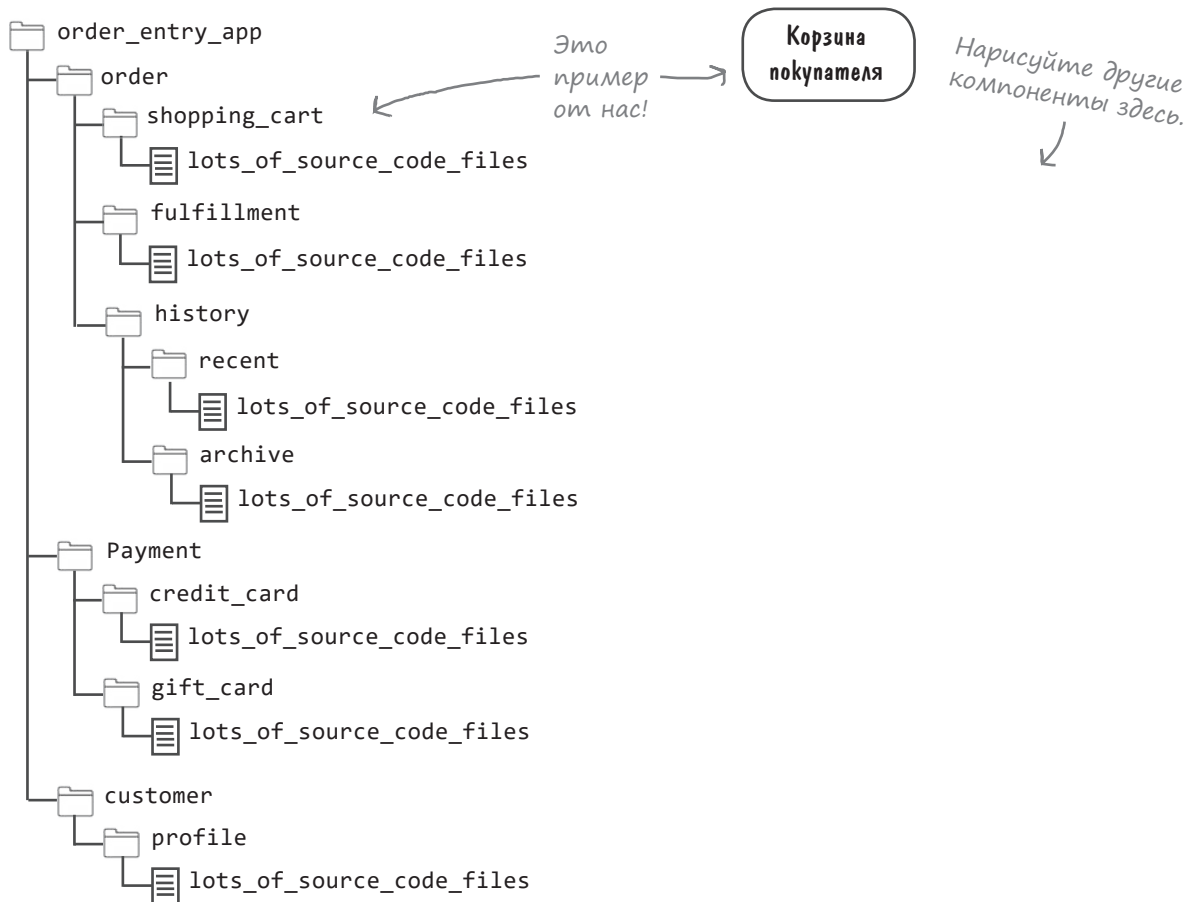




Упражнение

Дайте имя этому компоненту

Вас приняли на работу в должности архитектора и включили в существующий проект по созданию системы обработки заявок на устранение неполадок. Вы решили выяснить, из каких логических компонентов состоит система, но ваша команда ничего не знает о них. Чтобы определить логическую архитектуру, вам нужно взглянуть на структуру каталогов. Сколько отдельных логических компонентов вы сможете выделить из приведенной кодовой базы?



Решение на с. 188

Заныскаем Adventurous Auctions

Хотите отправиться на сафари в Танзанию? Понаблюдать за дикой природой на Галапагосских островах? Добраться до базового лагеря на Эвересте? Бюро приключений Adventurous Auctions поможет вам!

Возможно, вы видели нашу рекламу или посещали наши аукционы, проводимые по всей стране. Подобные путешествия трудно найти, и на их бронирование могут уйти годы, но вы можете купить их на нашем аукционе со значительной скидкой.

Мы хотим, чтобы как можно больше людей смогли съездить в эти замечательные путешествия, поэтому продаем свои туры не только на живых, но и на онлайн-аукционах.

Вот тут-то вы и пригодитесь: нам нужна новая система онлайн-аукционов по продаже наших увлекательных туров.



Вперед,
к приключениям!

Это Кейт. Она
хочет съездить
в увлекательное
путешествие.



Что должна делать новая система:

- Масштабироваться с учетом спроса, чтобы в каждом аукционе могли участвовать сотни и даже тысячи людей.
- Принимать обычные и онлайн-ставки.
- Позволять онлайн-участникам регистрироваться в Adventurous Auctions и предоставлять данные кредитных карт для оплаты в случае выигрыша.
- Позволять пользователям подключаться к видеотрансляциям аукционов и видеть все ставки, сделанные очно или онлайн.
- Позволять пользователям делать ставки онлайн.
- Определять участника, который первым предложил запрошенную цену (это называется «победа в торгах»). Если онлайн-участник делает ставку одновременно с участником, который присутствует на аукционе очно, то победителя определяет аукционист.
- Когда аукционист объявляет победителем онлайн-участника, система снимает деньги с кредитной карты этого участника, уведомляет его о победе и переходит к следующему туру на аукционе.

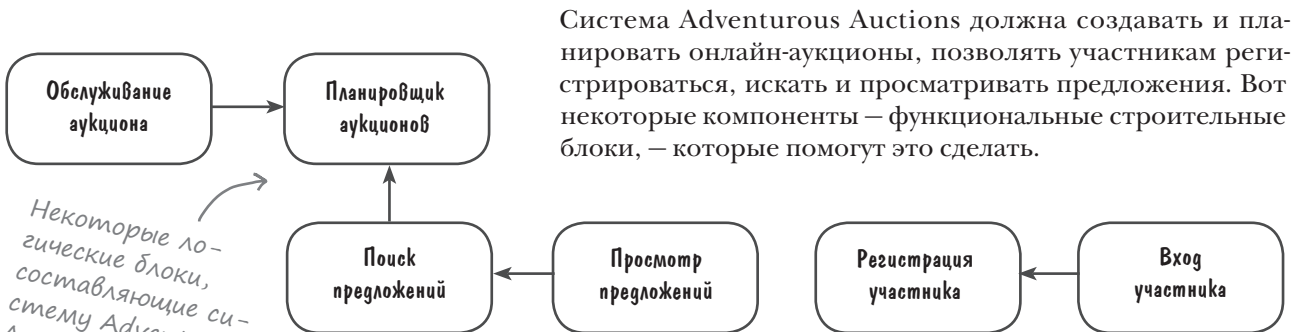
Будьте внимательны, далее мы покажем, как создать
логическую архитектуру для этой системы.

Логическая и физическая архитектура

Логическая архитектура определяет логические строительные блоки системы и взаимодействие между ними (их *связанность*). **Физическая архитектура** определяет архитектурный стиль, сервисы, протоколы, базы данных, пользовательские интерфейсы, шлюзы API и т. д.

Логическая архитектура системы не зависит от физической — то есть логической архитектуре безразличны базы данных, сервисы, протоколы и т. д. Рассмотрим пример того, что мы понимаем под логической архитектурой.

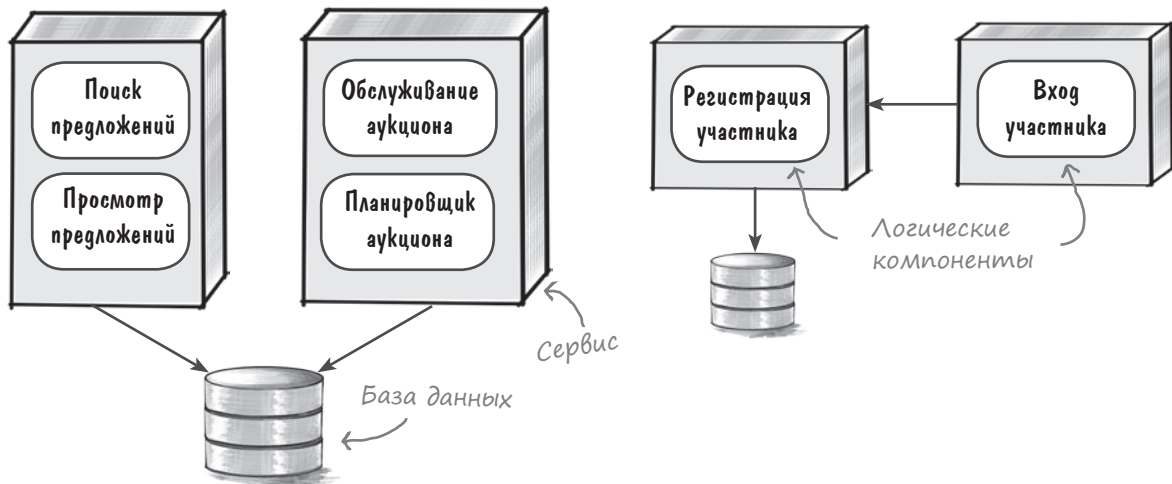
В этой главе мы будем много говорить о связях между компонентами.



Некоторые логические блоки, составляющие систему Adventurous Auctions.

Физическая архитектура определяет сервисы и базы данных.

Как видите, логическая архитектура не содержит компонентов физической архитектуры, упомянутых выше. Это другой взгляд на систему. Сравните схему, приведенную выше, со схемой физической архитектуры, показанной ниже. Обратите внимание, как физическая архитектура связывает сервисы с компонентами логической архитектуры и определяет сервисы и базы данных для системы.



Что чему соответствует?

Мы определили все компоненты логической и физической архитектуры, но они вдруг перепутались. Помогите нам разобраться, что чему соответствует. Имейте в виду: некоторые компоненты могут ничему не соответствовать — ни логической, ни физической архитектуре.

Логическая
архитектура

Пример
от нас

Физическая
архитектура

Определяет язык программирования для реализации каждого компонента.

Связывает логические компоненты с сервисами.

Определяет логические компоненты и связи между ними.

Определяет, сколько баз данных в системе и какие сервисы к ним обращаются.

Определяет способы связи между сервисами и протоколы (например, REST).

Определяет файлы с исходным кодом реализации компонентов.

Определяет компоненты и их связи с пользовательским интерфейсом.

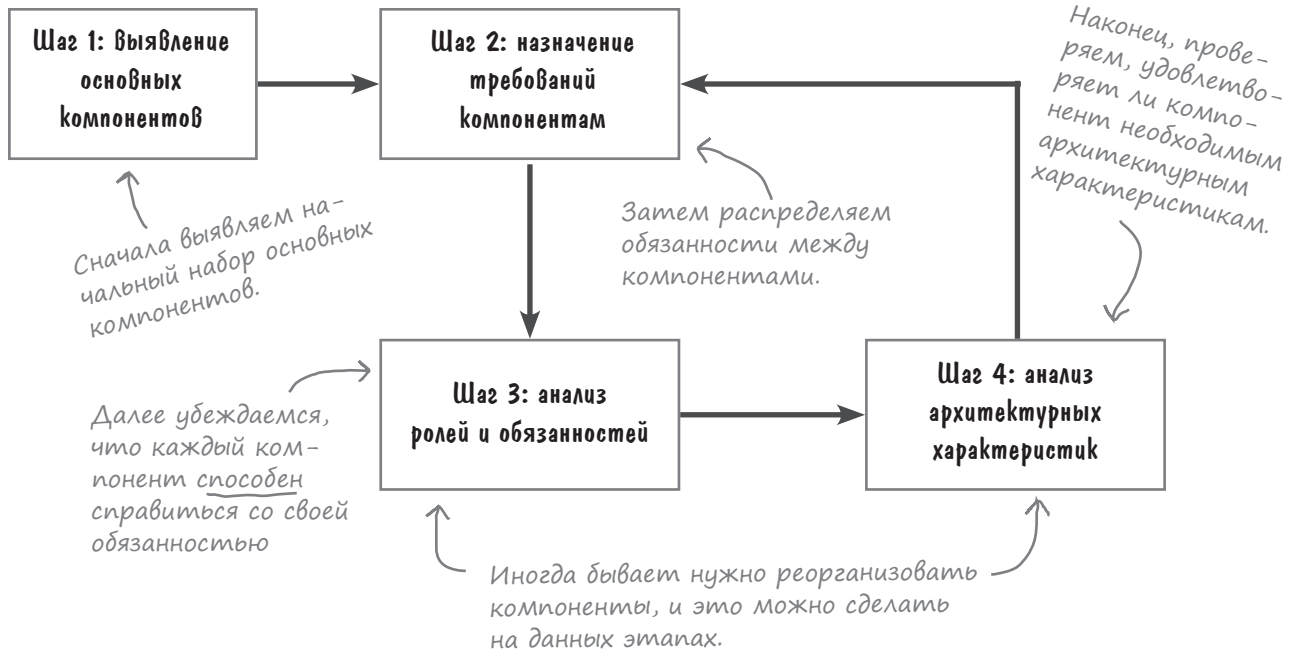
Определяет шлюзы API и балансировщики нагрузки.

Посмотрите наш ответ в конце этой главы, прежде чем продолжать чтение.

→ Решение на с. 189

Создание логической архитектуры

Определить логические компоненты не так просто, как кажется. Чтобы помочь вам, мы создали эту блок-схему. Не волнуйтесь — мы подробно опишем все эти шаги на следующих страницах.

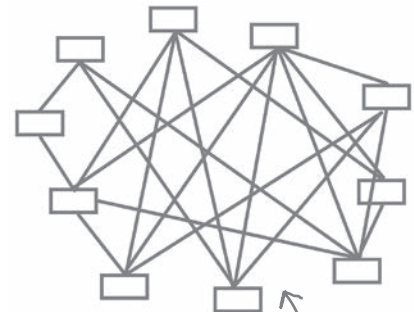


Этот процесс продолжается, пока система существует.

На этой блок-схеме показаны шаги создания *новых* приложений (и систем) и обслуживания *существующих*.

Вы когда-нибудь задумывались, почему нередко хорошо спроектированная система мгновенно превращается в клубок неразрешимых проблем? Такое случается, когда команды не уделяют достаточного внимания логической архитектуре.

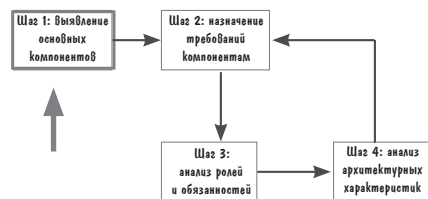
Внося изменения или добавляя новую функцию в систему, вы должны пройти все эти этапы и убедиться, что логические компоненты правильно организованы и выполняют то, для чего предназначены.



Эти четыре шага могут предотвратить превращение вашей архитектуры в большой комок грязи.

Шаг 1: Выявление основных компонентов

Первый шаг в создании логической архитектуры — выявление **основных компонентов**. Часто это просто игра в угадку, и вы, скорее всего, еще не раз реорганизуете первоначально выявленные компоненты. Поэтому не тратьте много времени на размышления о размерах компонентов — мы еще вернемся к этому. Для начала просто посмотрим, что мы имеем в виду под «игрой в угадку».



Эта схема показывает, где вы находитесь.

Это Фрэнк, директор компании Adventurous Auctions.



Процедура аукциона проста. Участники подключаются к текущему аукциону, просматривают предложения и делают ставку на интересующий их тур.

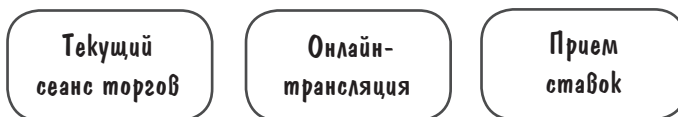
Опираясь на это простое описание, мы можем создать три логических компонента, по одному на каждую из основных задач.

Не забывайте, что это компоненты, а не сервисы.

Этот компонент представляет текущий аукцион и позволяет участникам присоединиться к нему.

Этот компонент отвечает за онлайн-трансляцию торгов.

Этот компонент отвечает за прием ставок.



Эти компоненты еще ничего не делают. Мы пока только **выявили** начальный набор компонентов, но не назначили им никаких требований. Сейчас они — как пустые банки, просто представляют наши предположения, основанные на функциях, которые должна поддерживать система. Вот почему мы называем их **начальным набором основных компонентов**.



Эта банка существует, но она пуста, как и только что выявленный компонент.

Текущий сеанс торгов

Создание логической архитектуры — «игра в угадайку»? Это самое нелепое, что я когда-либо слышала. Разве нет способа избавиться от необходимости угадывать логические компоненты?



Конечно есть! Существует ряд подходов, которые могут помочь избавиться от необходимости угадывать.

Мы пока мало знаем о системе и ее требованиях, и первоначальный набор компонентов, скорее всего, будет меняться по мере накопления знаний. Вот почему мы говорим, что на данном этапе это игра в угадайку, и это совершенно нормально!

Мы покажем вам два наиболее распространенных подхода: на основе *рабочих процессов* и на основе *субъектов и действий*.

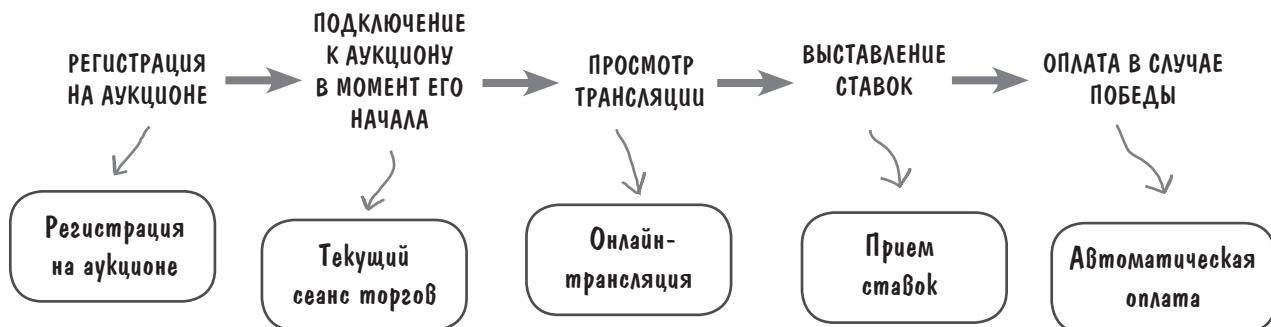
Есть и другие подходы, которые на первый взгляд выглядят неплохо, но могут завести в тупик. Мы поговорим о них после обсуждения хороших подходов.

Подход на основе рабочих процессов

Размышления об основных *рабочих процессах*, выполняемых пользователем в системе, помогают эффективно выявить первоначальный набор основных компонентов. Не стремитесь описать каждый шаг, начните с основных этапов, а затем переходите к описанию деталей.

Теперь применим этот подход и выявим некоторые основные компоненты архитектуры Adventurous Auctions.

Вы можете моделировать разные рабочие процессы, чтобы выявить основные компоненты.



Часть задаваемые вопросы

В: Вы определили «онлайн-трансляцию» как логический компонент, а если мы решим использовать стороннюю библиотеку или сервис для трансляции аукциона?

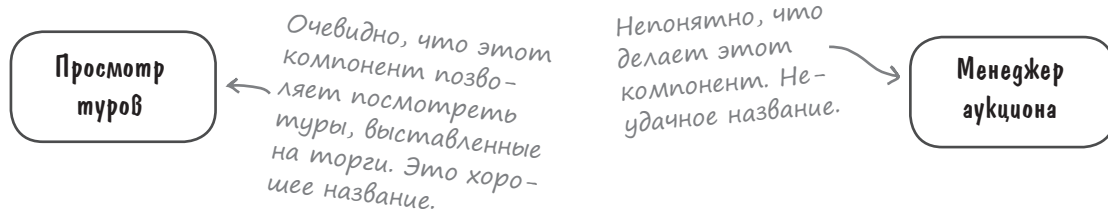
О: Хороший вопрос! Функция, даже реализованная не вами, все равно является частью логической архитектуры.

В: Всегда ли каждому шагу рабочего процесса соответствует свой логический компонент?

О: Не всегда. В рабочем процессе может быть несколько шагов, которые указывают на один логический компонент, особенно если они тесно связаны.

Названия имеют значение.

Уделите особое внимание выбору названий основных компонентов. Хорошее название должно *кратко описывать*, что делает этот компонент.



Возьмите в руку карандаш



Вашей компании нужна новая система распределения рабочих по строительным площадкам, и ваша задача как архитектора ПО — определить ее основные компоненты, используя подход на основе рабочих процессов. Напомним, что одному шагу рабочего процесса может соответствовать несколько компонентов, а одному компоненту — несколько шагов.

Шаг 1: составить список рабочих, указав их квалификацию и местонахождение.

Шаг 2: создать новый строительный проект и указать место площадки.

Шаг 3: составить график начала и окончания строительного проекта.

Шаг 4: в начале назначить рабочих с учетом их профессий и местонахождения.

Шаг 5: по завершении освободить рабочих для перевода на другое место.

Нарисуйте здесь свои логические компоненты. Постарайтесь дать им хорошие описательные названия.



→ Решение на с. 190

Подход на основе субъектов и действий

Подход на основе субъектов и действий полезен, когда в системе имеется несколько участников (пользователей). Сначала определяются участники, затем основные действия, которые участники могут выполнять, и потом каждое действие связывается с новым или существующим компонентом.

Помните Кейт? Она хочет съездить в путешествие и будет участвовать в торгах.



Начнем с Кейт. Вот основные ее действия как участника торгов:

Найти предложения

Поиск предложений

Посмотреть трансляцию

Онлайн-трансляция

Сделать ставку

Компонент «Прием ставок» реализует множество действий и отправляет ставки в компонент отслеживания участников.

Это Сэм. Он аукционист.



Вот некоторые действия Сэма как аукциониста:

Принять ставку от онлайн-участника

Принять ставку от участника, присутствующего на аукционе очно

Начать торги

Отметить лот как проданный

Прием ставок

Компонент «Текущий сеанс торгов» передает информацию об участниках другим компонентам.

Текущий сеанс торгов

Автоматическая оплата

Субъект «система» представляет действия, выполняемые автоматически:

Плата за участие в торгах

Отслеживание действий участников торгов

Отслеживание участников



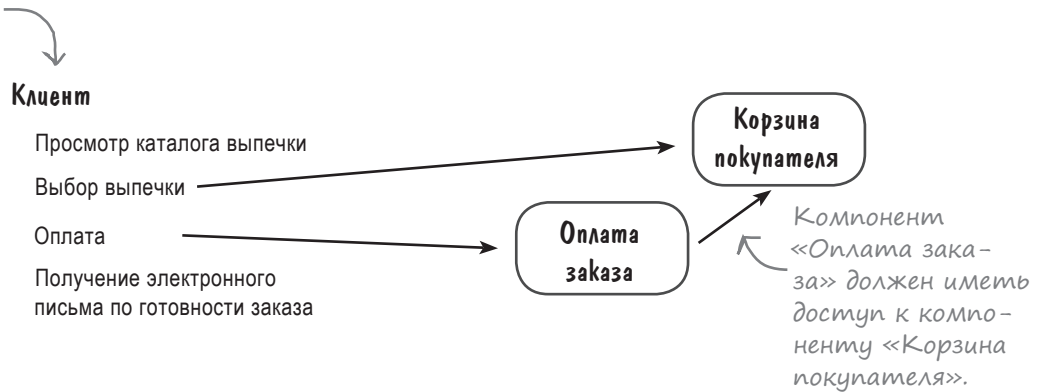


Упражнение

Вы владеете пекарней и готовы расширить свой бизнес, но вам нужна новая система, которая позволит клиентам просматривать, заказывать и оплачивать выпечку онлайн. Заказы передаются координатору пекарни, который закупает ингредиенты и составляет график выполнения заказов. Утром пекарь получает график и сообщает системе о готовности каждого изделия. После этого система отправляет клиентам электронные письма, сообщая, что их заказы готовы к выдаче.

Используя подход на основе субъектов и действий, определите, какие действия может предпринять каждый субъект. Затем нарисуйте как можно больше логических компонентов, соответствующих действиям.

Мы начали
выполнять
это упраж-
нение и до-
бавили пару
компонентов.



→ Решение на с. 191

Ловушка для сущностей



Итак, если я определю набор действий, связанных со ставками, то их все следует объединить в один компонент с названием «Управление ставками»?

Ловушка для сущностей

Мы называем этот подход *ловушкой для сущностей*, поскольку в нее легко попасть, определяя начальный набор основных логических компонентов.

Во-первых, название «Управление ставками» слишком расплывчатое. Сможете ли вы по нему сказать, что делает компонент? И мы не можем. Такое название мало что говорит о его роли и обязанностях.

Во-вторых, у компонента слишком много обязанностей. Компоненты, попавшие в ловушку сущностей, становятся удобной свалкой для функциональности. В результате они оказываются слишком большими и сложными для поддержки, масштабирования и обеспечения отказоустойчивости.

Этот компонент трудно масштабировать, поскольку он делает слишком много.

Управление ставками

- Прием ставок
- Показ всех выигравших ставок
- Отслеживание заявок
- Определение победителя торгов
- Уведомление аукциониста об онлайн-ставке
- Генерирование отчетов о торгах

Компонент делает все это.

Совет от профи



При именовании логических компонентов старайтесь избегать слов а-ля «управление» или «координация» — они являются верным признаком ловушки для сущностей.

Возьмите в руку карандаш

Какие еще слова, помимо «управление», по вашему мнению, могли бы говорить о попадании в ловушку сущностей?

Пример от нас → **супервизор**

Решение на с. 192



Упражнение

Выберите наилучший подход к определению начального набора основных компонентов в следующих сценариях. В некоторых случаях можно использовать несколько подходов.

В системе только один тип пользователей	☐ Рабочий процесс	☐ Субъект/действие
Система имеет четко определенные сущности	☐ Рабочий процесс	☐ Субъект/действие
Функциональные требования минимальны	☐ Рабочий процесс	☐ Субъект/действие
Пользователь выполняет много сложных действий	☐ Рабочий процесс	☐ Субъект/действие
В системе много типов пользователей	☐ Рабочий процесс	☐ Субъект/действие

—————→ Решение на с. 192

Часть
Задаваемые
Вопросы

В: Подход на основе субъектов и действий напоминает штурм событий. Это одно и то же?

О: Мы рады, что вы увидели сходство! Штурм событий — это один из подходов *предметно-ориентированного проектирования* (domain-driven design, DDD). Он заключается в анализе предметной области и выявлении предметных событий. Конечная цель обоих подходов — определение действий, выполняемых в системе, но штурм событий намного точнее идентифицирует элементы. Можно сказать, что подход «субъект/действие», как и штурм событий, выявляет *предметные события* и *субъекты*, но не выявляет другие элементы, такие как *команда*, *агрегат* и *представление*. Больше о штурме событий можно узнать по адресу https://en.wikipedia.org/wiki/Event_storming.

В: Можно ли сочетать подходы на основе рабочих процессов и субъектов/действий?

О: Да, их можно сочетать, и в целом это хорошая идея. Начав с субъектов и действий, можно определить действия, а затем исследовать рабочие процессы, чтобы расположить действия в том порядке, в каком они будут происходить.

В: Вы предлагаете никогда не использовать слова «управление», «координация» и т. д. в названиях компонентов?

О: Не совсем так. В ловушке для сущностей нет жестких правил. Иногда сложно придумать название для чего-то, что решает общую задачу. Например, компонент, управляющий справочными данными в приложении — парами «имя — значение», такими как коды стран, коды магазинов, коды цветов и т. д., вполне можно назвать «Управление справочными данными». Однако такие названия, как «Управление заказами» или «Обработчик ответов», слишком общие и не описывают того, чем фактически занимаются эти компоненты.

В: Сколько действий для каждого субъекта нужно определить в подходе на основе субъектов/действий?

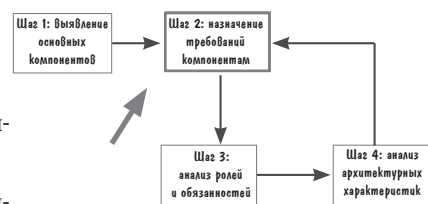
О: Сложный вопрос. Цель подхода — выявить вероятные логические компоненты и их зону ответственности. Обычно рассматриваются только *основные* действия, выполняемые субъектом, без погружения в детали.

Шаг 2: назначение требований компонентам

Следующий шаг после выявления начального набора основных логических компонентов — *назначение требований*.

На этом шаге рассматриваются пользовательские истории и выясняется, какой компонент должен отвечать за каждую из них. Напомним, что каждый компонент представлен структурой каталогов; таким образом, исходный код в каждом каталоге реализует соответствующее требование.

Вернемся к начальному набору компонентов, который мы определили по описанию рабочего процесса, представленному директором компании Adventurous Auctions, и возложим на них некоторые обязанности.



 **Отобразить текущий тур, выставленный на торги**
Компонент «Текущий сеанс торгов» отвечает за управление текущим аукционом, поэтому имеет смысл назначить это требование ему.

 **Отметить тур как проданный и перейти к следующему**
Это требование тоже связано с управлением ходом торгов, поэтому назначим его компоненту «Текущий сеанс торгов».

 **Посылать онлайн-ставки аукционисту, чтобы он мог объявлять их**
Компонент «Прием ставок» получает ставки от участников торгов, поэтому имеет смысл обязать его пересылать ставки аукционисту.

 **Записывать видеопоток в реальном времени на случай, если ставка будет оспорена**
Компонент «Онлайн-трансляция» уже имеет видеопоток, поэтому имеет смысл записать его.

 **Позволять участникам просматривать информацию о текущих турах**
Это требование не подходит ни для одного из компонентов, выявленных ранее, поэтому мы создали новый.

Текущий сеанс торгов

Представляет сеанс торгов, проходящий в данный момент.

Прием ставок

Отвечает за прием и хранение ставок от участников торгов.

Онлайн-трансляция

Отвечает за показ видеотрансляции аукциона.

Просмотр туров

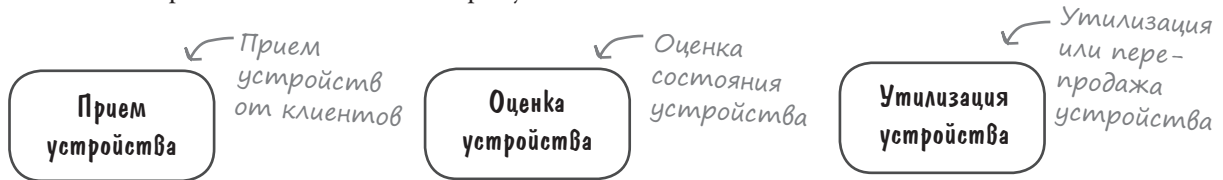
Новый компонент, добавленный для удовлетворения последнего требования.

Создание новых компонентов называется развитием архитектуры.



Возьмите в руку карандаш

В вашей компании, Going Green Corporation, решили создать систему для поддержки новой программы утилизации электроники, в рамках которой клиенты смогут сдавать старые устройства (например, сотовые телефоны) и получать деньги. Мы уже определили несколько основных компонентов. Ваша задача — закрепить за существующими компонентами перечисленные ниже функции или создать новые компоненты. Кроме того, вы должны придумать названия для всех новых компонентов.



Поиск ближайшего пункта утилизации

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Сбор и хранение информации о клиентах (имя, адрес и т. д.).

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Размещение объявления на стороннем сайте для перепродажи

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Выплата вознаграждения клиенту

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Фиксация факта приема устройства для оценки

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Определение стоимости (если имеется) утилизируемого устройства

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Определение ежемесячной прибыли и убытков от утилизации и перепродажи устройств

☐ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

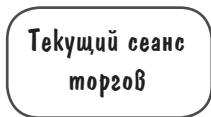
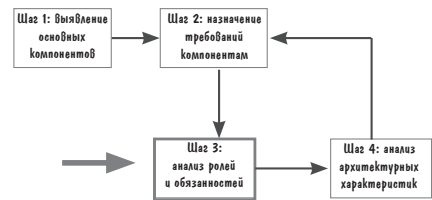
В конце концов, Going Green Corp. хочет получать прибыль.

Решение на с. 193

Шаг 3: анализ ролей и обязанностей

По мере назначения логическим компонентам *функций* (другими словами, пользовательских историй или требований) роли и обязанности этих компонентов начнут расти. Цель шага 3 — убедиться, что компонент, которому назначается функция, *действительно должен* отвечать за нее и на него не взваливается слишком много обязанностей.

Возьмем для примера компонент «Текущий сеанс торгов», на который возлагаются следующие обязанности:



1. Открывать и закрывать торги.
2. Показывать тур, выставленный на торги.
3. Отметить проданный тур и перейти к следующему.

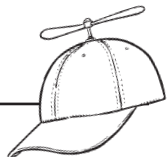
Это хорошо спроектированный компонент, на который возложено достаточно много обязанностей. Однако в ходе разработки команда добавила ему несколько новых требований:

4. Отображать детали каждого тура, выставленного на продажу.
5. Отслеживать присутствующих участников торгов.
6. Отслеживать победителей торгов.
7. Уведомлять участника торгов о выигрыше тура.

Эй, компонент, не мог бы ты еще помыть мою машину?

Если у вас слишком много работы, то, скорее всего, вы перепоручите ее другим. Сделайте то же самое с компонентами: переложите часть ответственности на кой-то другой компонент.

Дополнительные функции возлагают на компонент слишком много обязанностей. Это распространенная ситуация, поэтому не удивляйтесь, столкнувшись с ней. Не паникуйте. Этот шаг и нужен как раз для того, чтобы можно было оценить ситуацию. Посмотрим, как ее исправить, переложив часть обязанностей на другие компоненты.



Серьезное программирование

Вы когда-нибудь создавали класс с именем *Utility*? Что он делал? Скорее всего, содержал множество разрозненных функций, которые трудно было поместить куда-то еще. То же может произойти с логическими компонентами. Старайтесь избегать компонентов, содержащих много не связанных между собой функций.

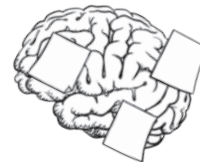
Стремитесь к связности

Анализируя описание роли и обязанности компонента, проверьте, взаимосвязаны ли функции. Степень взаимосвязанности операций, выполняемых компонентом, называют *связностью*. Она помогает убедиться, что компонент имеет правильно очерченный круг обязанностей.

При анализе ролей и обязанностей компонентов часто выявляются *посторонние* функции или слишком широкий круг обязанностей. В таких случаях можно переложить часть ответственности на другие компоненты.

Теперь ваш черед исправить компонент «Текущий сеанс торгов».

Запомните
это!



Все функции компонента должны быть взаимосвязаны. Но не расстраивайтесь, если это не так. Просто разделите компонент на части, и вас сочтут очень умным.



Упражнение

Попробуйте переложить часть обязанностей с компонента «Текущий сеанс торгов» на новые компоненты, ответственные за дополнительные функции, а за ним оставьте только первые три требования.

Это пример
от нас

Текущий сеанс
торгов

Нарисуйте здесь
новые компоненты.

1. Открывать и закрывать торги.
2. Показывать тур, выставленный на торги.
3. Отметить проданный тур и перейти к следующему.

4. Отображать детали каждого тура, выставленного на продажу.
5. Отслеживать присутствующих участников торгов.
6. Отслеживать победителей торгов.

7. Уведомлять участника торгов о выигрыше тура.

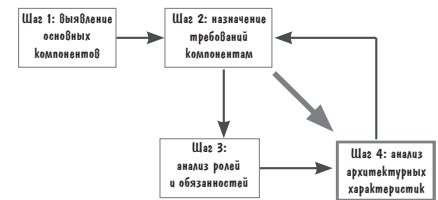
Подсказка: при создании новых компонентов некоторые требования можно объединить.

Решение на с. 194

Шаг 4: анализ архитектурных характеристик

Последний шаг в выявлении основных компонентов — проверка их соответствия *ключевым архитектурным характеристикам*. Обычно это предполагает разделение компонента на части для улучшения масштабируемости, эластичности или доступности, но иногда может потребоваться объединить компоненты, если они тесно связаны.

Еще раз взглянем на наш пример. Выше мы определили компонент «Прием ставок», отвечающий за *прием и сохранение всех ставок в базе данных и пересылку самой высокой ставки аукционисту*. Общий порядок работы этого компонента выглядит так:



Эта архитектура выглядит неплохо, но мы должны убедиться, что компонент «Прием ставок» поддерживает ключевые архитектурные характеристики (которые важны для успешной работы приложения).

Все они важны для успешной работы приложения Adventurous Auctions.

Мы знаем, что система должна одновременно обслуживать тысячи участников, — это *масштабируемость*. Вдобавок нам известно, что система должна запускаться и работать во время торгов, — это *доступность*. Наконец, система должна принять и передать ставку аукционисту как можно быстрее — это *производительность*.

Теперь ваш черед проанализировать компонент «Прием ставок» на предмет соответствия этим ключевым архитектурным характеристикам.

СТАНЬТЕ архитектором



Вообразите, что вы архитектор, и проверьте, соответствует ли компонент «Прием ставок» ключевым архитектурным характеристикам. Наше решение представлено на следующей странице.

Ключевые архитектурные характеристики для Adventurous Auctions:

Масштабируемость: Система должна одновременно обслуживать тысячи участников.

Доступность: Система должна работать во время проведения торгов.

Производительность: Система должна принять и передать ставку аукционисту как можно быстрее.

Подсказки (что нужно учесть):

- Что делать, если база данных выйдет из строя?
- Справится ли база данных с сохранением большого количества ставок, поступивших в единицу времени?
- Будет ли база данных работать достаточно быстро, чтобы своевременно информировать аукциониста?
- Перечислите действия, которые компонент «Прием ставок» должен выполнить после получения ставки.

Запишите здесь, как бы вы изменили компонент «Прием ставок», чтобы он соответствовал ключевым архитектурным характеристикам и соображениям, указанным выше.



Компонент «Прием ставок»

Рассмотрим текущие обязанности компонента «Прием ставок»:

1. Принимать ставки от участников торгов, которые присутствуют онлайн и очно (через аукциониста).
2. Определять самую высокую онлайн-ставку.
3. Записывать все ставки в базу данных.
4. Сообщать аукционисту самую высокую ставку.

Желательно, чтобы компонент «Прием ставок» сохранял ставки в базе данных. Но пропускная способность базы и соединения с ней ограничены, что существенно влияет на *масштабируемость*. Это ограничение также влияет на *производительность*, добавляя время ожидания, пока ставки будут записаны в базе, а также на *доступность*, если база вдруг выйдет из строя.

Возложив последнее требование на *новый* компонент «Регистрация ставок», можно повысить масштабируемость, производительность и доступность компонента «Прием ставок». Это позволит системе быстрее обрабатывать и передавать ставки аукционисту. Компонент «Прием ставок» сможет передавать ставки компоненту «Регистрация ставок», и ему не придется ждать завершения записи в базе данных.

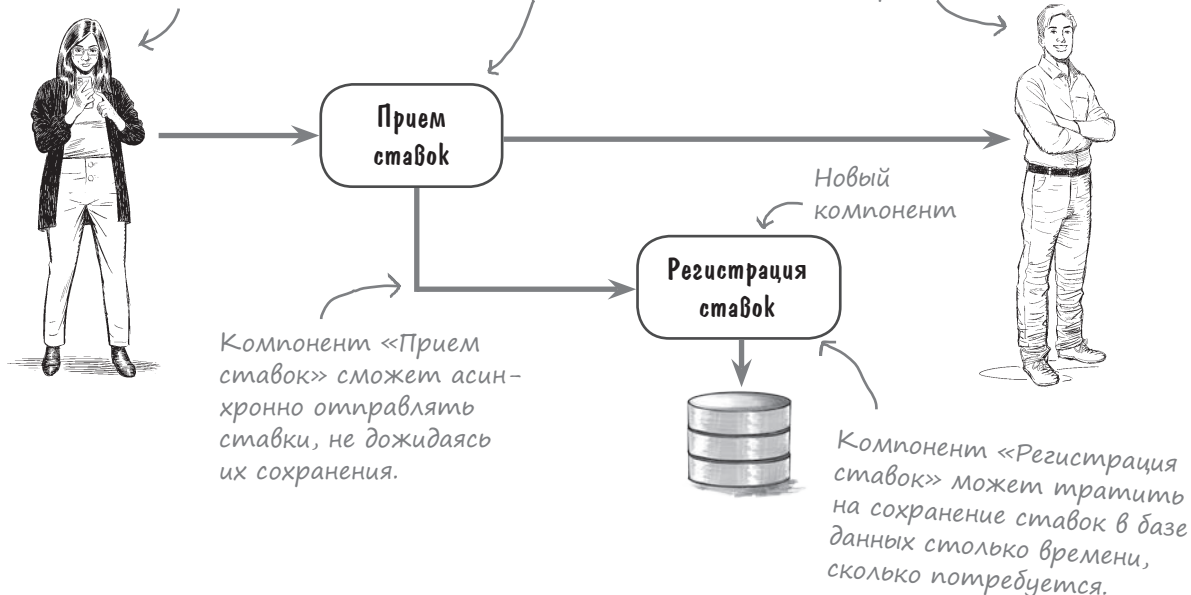
Вот что мы подразумеваем под анализом характеристик.

На данном этапе можно делить и объединять компоненты, чтобы достичь необходимых архитектурных характеристик.

Теперь у Кейт больше шансов выиграть тур, поскольку процесс торгов ускорился.

Компонент «Прием ставок» сможет обслуживать больше запросов, избавившись от необходимости сохранять их.

Сэму, аукционисту, больше не нужно ждать, когда онлайн-ставки сохранятся в базе данных.



Связывание компонентов



Наверное, пора подумать о взаимодействии логических компонентов и их зависимости друг от друга?

Да, сейчас самое время для этого.

Выявляя начальный набор основных компонентов, важно определить, как они взаимодействуют. Степень зависимости компонентов друг от друга называется *связанностью компонентов*. Чем больше связей между компонентами, тем выше связанность системы и тем сложнее ее поддерживать.



Помните эту схему? Ее называют «большой комок грязи», поскольку количество связей и зависимостей между компонентами так велико, что схема начинает напоминать запутанный клубок (или тарелку со спагетти).

Вот почему важно обращать внимание на взаимодействие компонентов и их зависимость друг от друга.

Создавая логические компоненты, учитывайте два типа связей: *афферентные* (входящие) и *эфферентные* (исходящие). Ничего страшного, если раньше вы не слышали этих формальных терминов. Мы объясним их на следующих страницах.

Афферентные связи

Дети зависят от своих родителей, получая от них еду, кров, заботу, опеку, помощь и даже карманные деньги на сладости и развлечения. На формальном языке такая связь с детьми и даже с домашними питомцами называется *афферентной*, поскольку все они в чем-то зависят от родителей.

Афферентная связь — это зависимость других компонентов от некоего целевого компонента (в данном случае от мамы). Иногда ее называют *входящей* связью. В большинстве инструментов анализа кода она обозначается просто как *CA* (coupling afferent — афферентная связь).



Предоставляет возможность зарегистрироваться для участия в аукционе

Управляет всей информацией об участнике торгов

Регистрация на аукционе

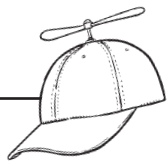
Профиль участника

Автоматическая оплата

Списывает средства с кредитной карты победителя.

Чтобы увидеть афферентную связь в действии, взгляните на схему слева: там показаны три логических компонента в архитектуре Adventurous Auctions.

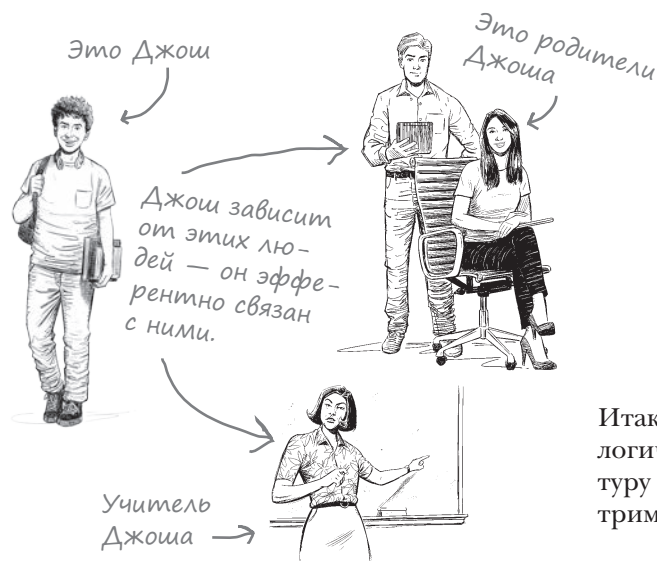
Компоненты «Регистрация на аукционе» и «Автоматическая оплата» зависят от компонента «Профиль участника», получая от него информацию об участнике. В этом примере компонент «Профиль участника» имеет уровень афферентной связанности 2, поскольку от него зависят два компонента.



Серьезное программирование

Знаете ли вы, что странное слово «афферентный» означает «несущий к»? Оно происходит от латинских слов *ad* (означает «к» или «в сторону») и *ferre* («нести»). В медицине афферентными называют, например, нервы, передающие импульсы в мозг (афферентные нервы).

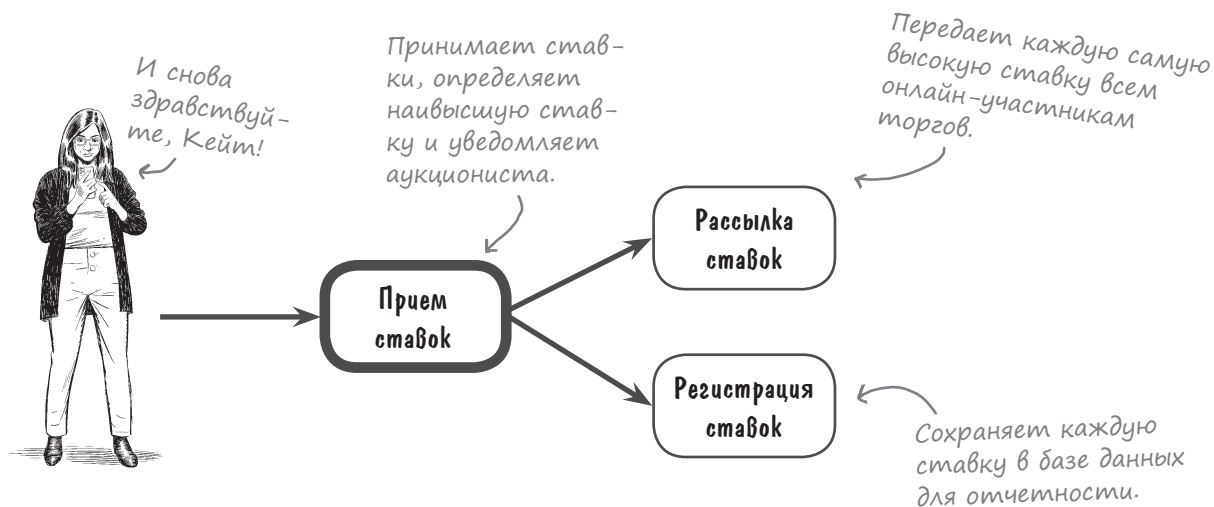
Эфферентная связь



Теперь посмотрим с позиции ребенка. В детстве все мы зависели не только от своих родителей, но и от учителей, друзей, одноклассников и т. д. Зависимость от других называется *эфферентной связью*.

Эфферентная связь прямо противоположна афферентной и измеряется количеством компонентов, от которых зависит целевой компонент. Ее еще называют *исходящей связью*. В инструментах анализа кода она обозначается как *CE* (coupling efferent — эфферентная связь).

Итак, как выглядит эфферентная связь с точки зрения логических компонентов? Еще раз взглянем на архитектуру Adventurous Auctions, однако на этот раз рассмотрим процесс приема ставки от Кейт.



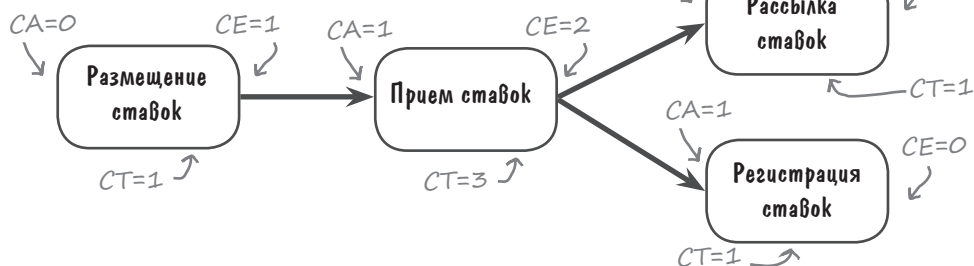
Компонент «Прием ставок» зависит от компонентов «Рассылка ставок» и «Регистрация ставок», поэтому *эфферентно связан* с ними. Он имеет уровень эфферентной связанности 2 (зависит от двух других компонентов).

Оценка связанности

Оценить степень связанности конкретного компонента можно по трем значениям: уровню *афферентной связанности* (CA), уровню *эфферентной связанности* (CE) и уровню *суммарной связанности* (СТ). Они покажут, какие компоненты имеют самую высокую и самую низкую связанность, а также общий уровень связанности системы.

$$CA + CE = CT$$

С помощью этой информации можно наблюдать, как меняется связанность по мере изменения системы.

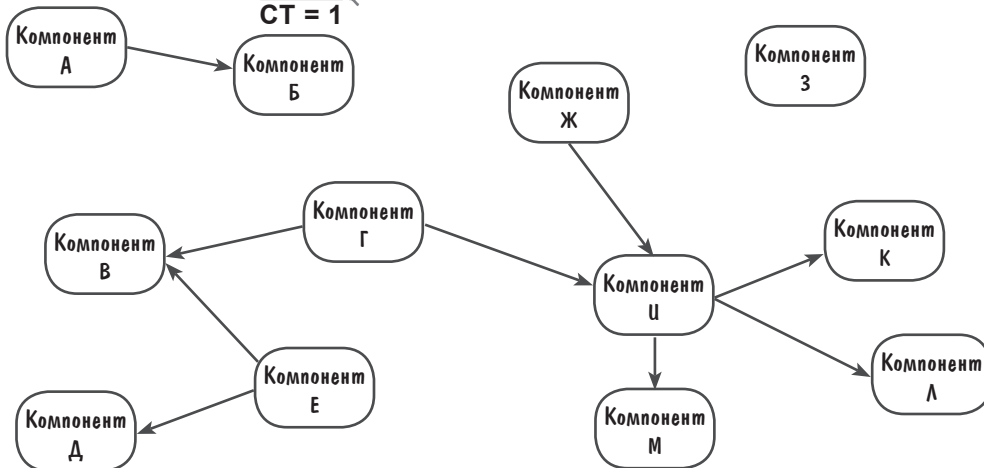


Возьмите в руку карандаш

Для каждого компонента, указанного ниже, определите уровни афферентной (CA), эфферентной (CE) и суммарной (СТ) связанности. Кроме того, оцените общий уровень связанности для всей логической архитектуры. Как вы думаете, уровень СТ для этой архитектуры высокий или низкий?

CA = 1
CE = 0
CT = 1

Пример от нас



Общий уровень связанности системы: _____

Решение на с. 195



Теперь я поняла, что такое связанность и как ее измерить. Но как уменьшить связанность компонентов, чтобы получить слабосвязанную систему?

Хороший вопрос. Разработчиков побуждают создавать слабосвязанные системы, но не рассказывают, *как* это сделать. Мы научим вас этому, представив технику под названием «Закон Деметры».

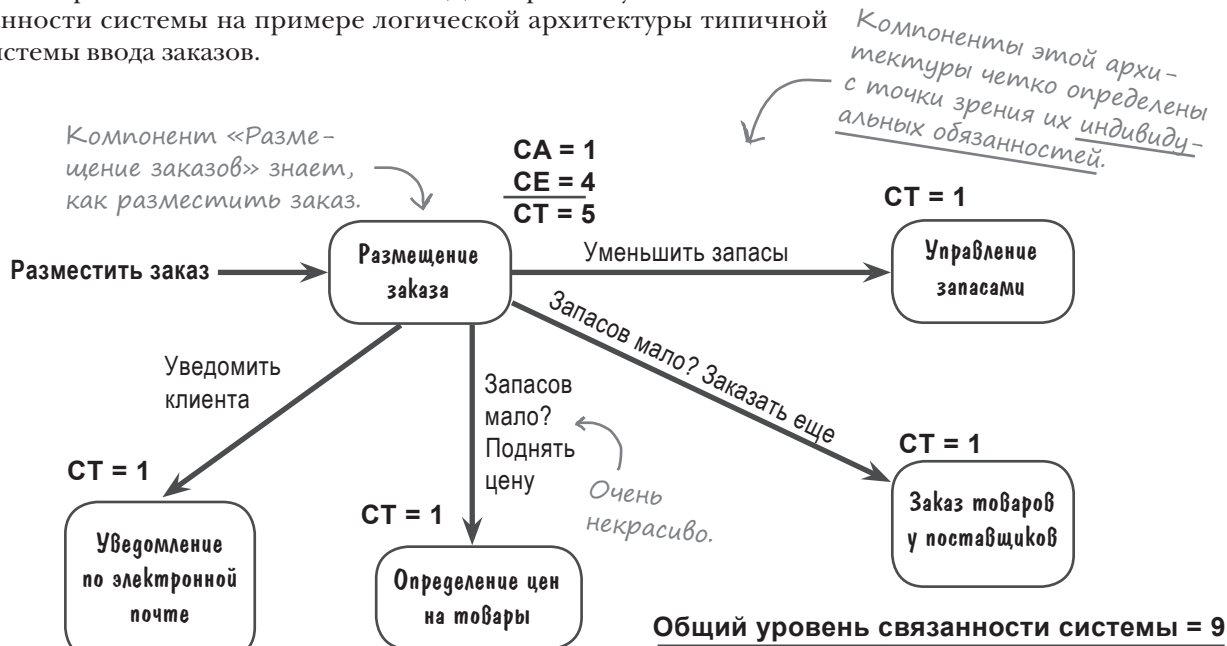
Этот закон, также известный как «Принцип наименьшего знания», назван в честь греческой богини земледелия Деметры. Она помогала земледельцам, но не знала, что они делают с урожаем. Поэтому Деметра была *слабо связана* с миром смертных.

То же относится к логическим компонентам. Чем больше компонент знает о других компонентах и о происходящем в системе, тем он сильнее связан с ними. Уменьшая его *знания* о других компонентах, мы снижаем уровень его связанности.

На следующих страницах мы расскажем о законе Деметры и о том, как с его помощью уменьшать связанность системы.

Сильновязанная система

Посмотрим, как использовать закон Деметры для уменьшения связанности системы на примере логической архитектуры типичной системы ввода заказов.



Мозговой штурм

Какие недостатки вы видите в логической архитектуре, приведенной выше? Мы указали некоторые (но не все). Отметьте те, которые, по вашему мнению, могут быть проблемой, и напишите другие возможные недостатки, которые видите в этой архитектуре.

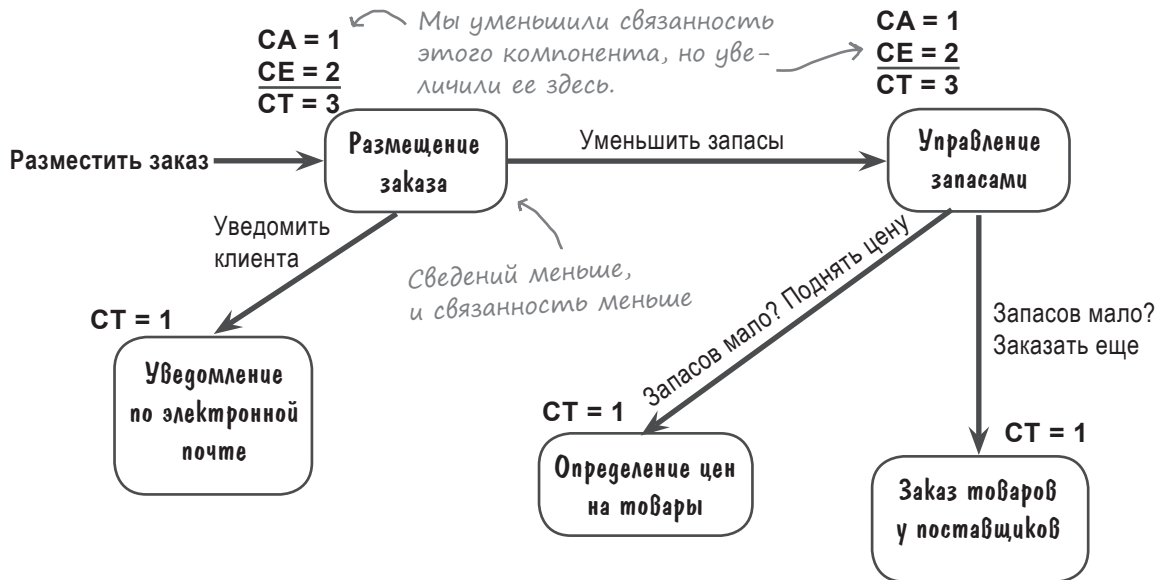
- ☐ Клиент может не получить отправленное письмо.
- ☐ У поставщика может не оказаться нужного товара на складе.
- ☐ Компонент «Размещение заказа» слишком много знает о шагах, связанных с размещением заказа.
- ☐ _____
- ☐ _____
- ☐ _____

Применение закона Деметры

Общий уровень связанности системы нам мало интересен. Нас беспокоит связанность компонента «Размещение заказа» (СТ = 5), несбалансированность его связанности и слишком большой объем сведений о процессе размещения заказа.

Применим закон Деметры для решения этих проблем и перенесем знания о малом количестве запасов в компонент «Управление запасами».

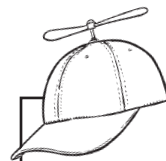
Компонент «Размещение заказа» отвечает за все.



Переместив сведения о действиях в случае малого количества запасов в компонент «Управление запасами», мы уменьшили объем сведений в компоненте «Размещение заказа» и его связанность. Но увеличили объем сведений в компоненте «Управление запасами» и его связанность. В этом и заключается закон Деметры: меньше сведений — меньше связанность, больше сведений — больше связанность.

Связанность зависит от объема знаний в компонентах об остальной части системы.

Общий уровень связанности системы = 9



Серьезное программирование

Вы заметили, что, несмотря на уменьшение связанности компонента «Размещение заказа», общий уровень связанности системы остался прежним? Объясняется это просто: мы не удалили сведения из системы, а лишь перенесли их в компонент «Управление запасами».

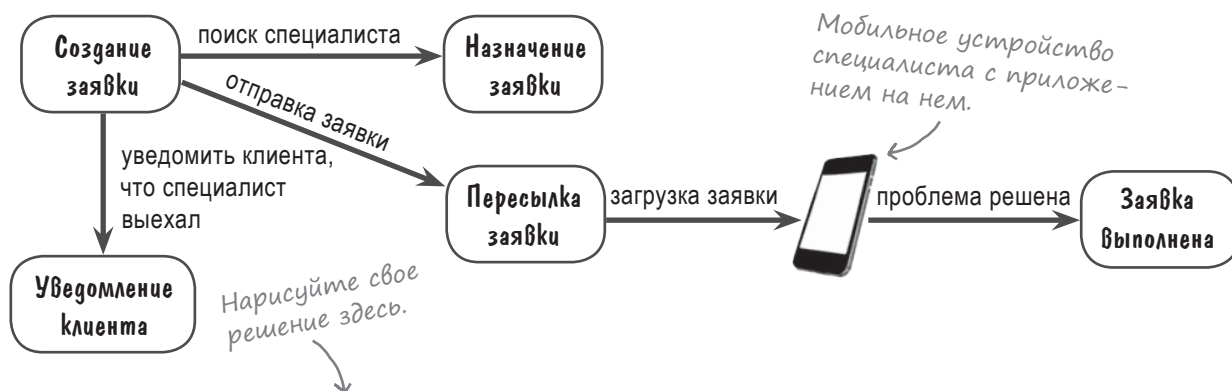


Тест-драйв

Теперь проведите тест-драйв закона Деметры и попробуйте уменьшить связанность логической архитектуры, представленной ниже. С помощью этой системы клиенты, купившие поддержку электронного устройства, могут сообщить о неисправности и вызвать специалиста по ее устранению. Система работает так:

- ❶ Клиент оставляет заявку.
- ❷ Заявка назначается свободному профильному специалисту.
- ❸ Заявка загружается в приложение на мобильном устройстве специалиста («Пересылка заявки»).
- ❹ Клиент получает уведомление о выезде специалиста.
- ❺ После устранения проблемы специалист отмечает заявку как выполненную.

Можно ли уменьшить связанность архитектуры с этим набором компонентов?



Решение на с. 196

Балансирование

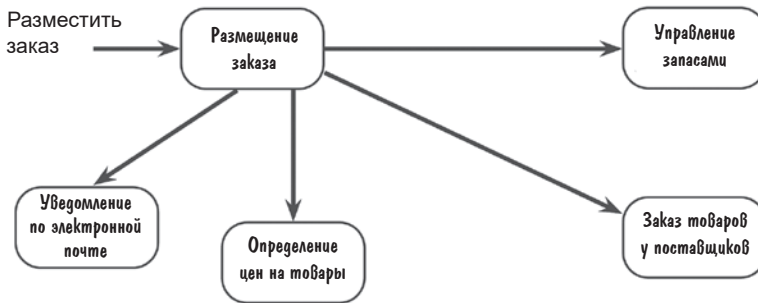
Помните *Первый закон архитектуры программного обеспечения*? Напомним его (поскольку это важно):

В архитектуре все основано на компромиссах.

Слабая связанность — не исключение. Давайте сравним две только что рассмотренные архитектуры и проанализируем их компромиссы.



Сильно связанная



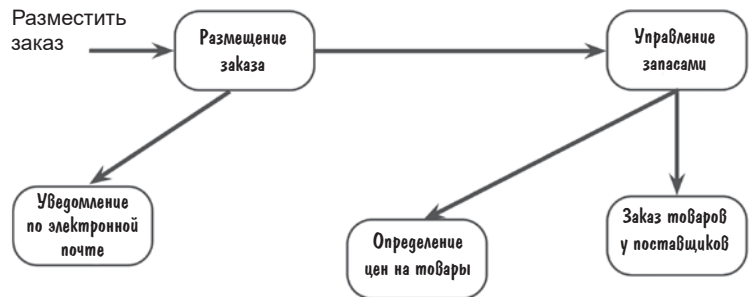
Чтобы понять, какие действия выполняются в *сильно* связанной архитектуре при размещении заказа, достаточно взглянуть на компонент «Размещение заказа».

Однако этот компонент зависит от четырех других. Его работа может нарушиться, если любой из них изменится.

Процесс централизован, но изменения сопряжены с риском.

Таковы компромиссы сильной связанности.

Слабо связанная



Процесс распределен, зато изменения менее рискованны.

Таковы компромиссы слабой связанности.

Запомните это хорошее правило.

Два компонента считаются связанными, если изменение одного может повлечь необходимость изменения другого.

Несколько заключительных слов о компонентах

Поздравляем! Теперь, научившись выявлять логические компоненты и зависимости между ними, вы сделали первый шаг к созданию архитектуры программного обеспечения. Это была длинная глава, но очень важная. Представление о системе как о наборе логических компонентов помогает лучше понять ее общую структуру и особенности ее работы.

В следующей части путешествия по архитектуре программного обеспечения мы сосредоточимся на технических деталях, таких как архитектурные стили, сервисы, базы данных и протоколы связи. Но прежде просмотрите следующие тезисы и убедитесь, что полностью понимаете все, что касается логических компонентов.

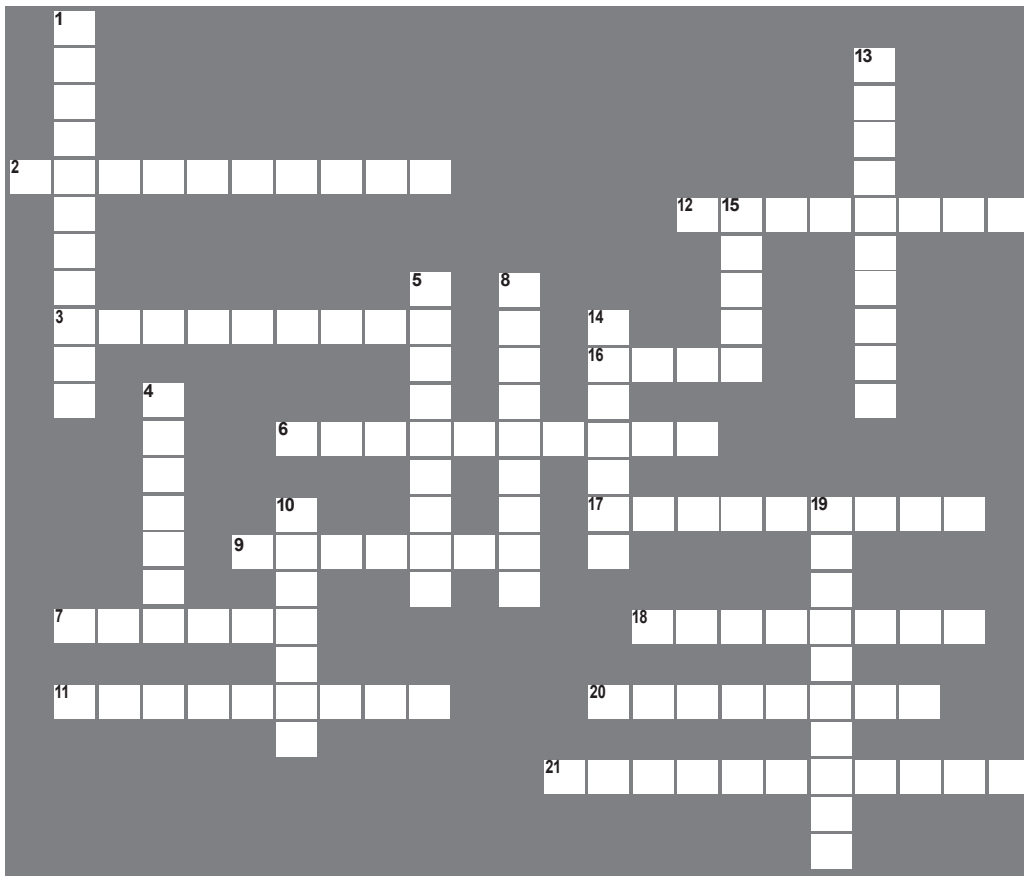
Ключевые моменты

- *Логические компоненты* — это функциональные строительные блоки системы.
- Логический компонент представлен *структурой каталогов*, куда помещается исходный код.
- Выбирайте описательные названия для компонентов, четко определяющие их назначение.
- Создание логической архитектуры подразумевает четыре последовательных шага: выявление основных компонентов, назначение им требований, анализ ролей и обязанностей компонентов, а также анализ архитектурных характеристик.
- Для выявления логических компонентов можно использовать *подход на основе рабочих процессов*, назначая компонентам этапы пути клиента.
- Еще один способ выявления логических компонентов — использовать подход на основе *субъектов и действий*, определяя субъектов в системе и назначая их действия компонентам.
- В подходе «*повушка для сущностей*» компоненты моделируются как основные сущности в системе. Избегайте этого подхода, поскольку в результате получаются слишком неоднозначные и большие компоненты с широким кругом обязанностей.
- Назначая требования компоненту, оцените его роль и обязанности и убедитесь, что именно он должен выполнять эту функцию.
- *Связывание* происходит, когда компоненты начинают зависеть друг от друга.
- *Афферентная (входящая) связь* — это зависимость других компонентов от некоторого целевого компонента.
- *Эфферентная (исходящая) связь* — это зависимость целевого компонента от других компонентов.
- Компоненты, слишком много знающие о происходящем в системе, увеличивают ее связанность.
- *Закон Деметры* гласит, что сервисы или компоненты должны как можно меньше знать о других сервисах или компонентах. Этот закон помогает создавать слабо связанные системы.
- Чем меньше связанность, тем меньше зависимостей между компонентами, но при этом *сведения* о рабочих процессах оказываются распределены по всей системе, что усложняет управление ими.
- Общая связанность (СТ) логической архитектуры определяется как сумма уровней афферентной и эфферентной связанности отдельных компонентов (CA + CE).



Кроссворд по логическим компонентам

Теперь у вас есть возможность немного развлечься и проверить свои знания. Разгадайте этот кроссворд.



По горизонтали

2. Диаграмма архитектуры может отображать логическую или _____ архитектуру.
3. Хорошее место для выявления компонентов — структура _____ кодовой базы.
6. Одним из компонентов системы может быть онлайн-_____.
7. API _____ присутствует в физической архитектуре, но не в логической.
9. Принцип наименьшего знания называют законом _____.
11. Избегайте ловушки для _____.
12. Выявление логических компонентов — часто игра в _____.
16. Логический компонент имеет _____ и обязанности.
17. Для выявления компонентов можно использовать подход на основе _____ и действий.
18. Adventurous Auctions позволяет _____ туры.
20. Дайте каждому компоненту описательное _____.
21. Связь может быть эфферентной или _____.

По вертикали

1. Афферентная и эфферентная связи — это формы _____ связи.
4. Каждый компонент выполняет _____.
5. _____ компонента определяет, насколько взаимосвязаны его операции.
8. На раннем этапе выявляется _____ набор основных компонентов.
10. Физическая архитектура связывает _____ с компонентами.
13. Шаг 2 — _____ требований логическим компонентам.
14. Путь пользователя в системе называется «рабочий _____».
15. Не собирайте большой комок _____.
19. Логические _____ — функциональные строительные блоки системы.

→ Решение на с. 197

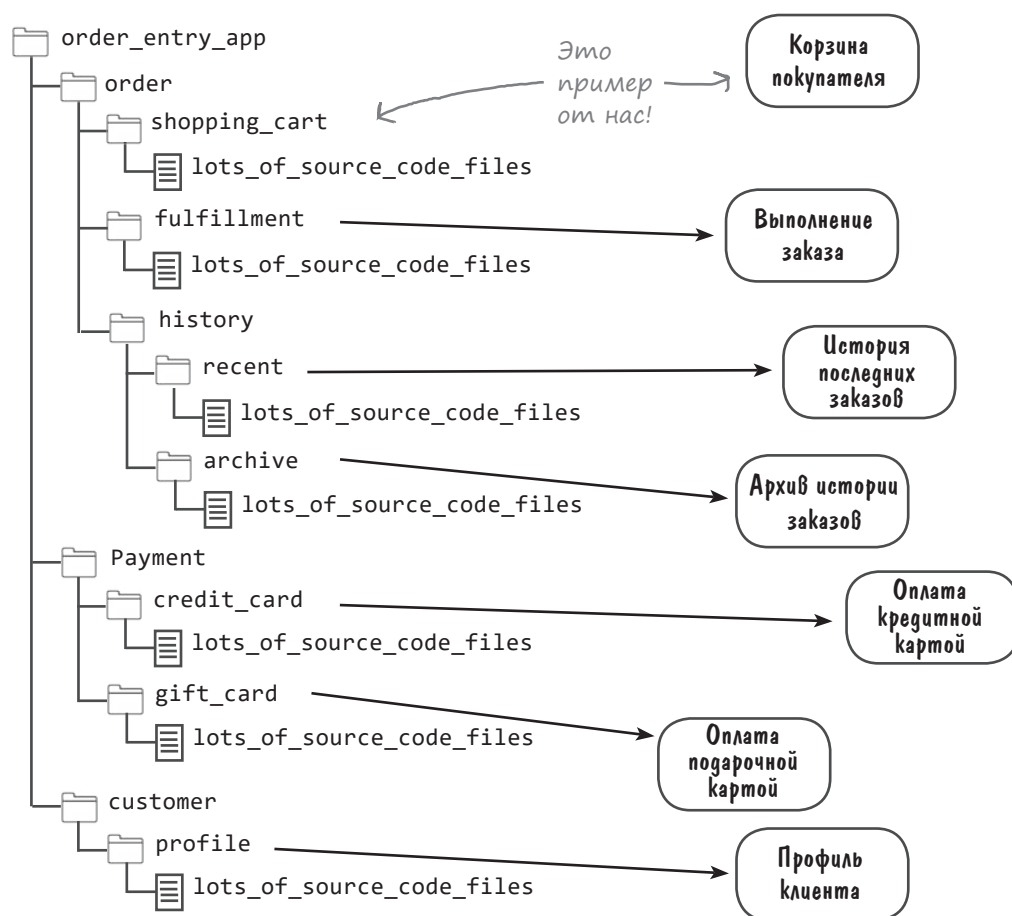


Упражнение Решение

На с. 157

Дайте имя этому компоненту

Вас приняли на работу в должности архитектора и включили в существующий проект по созданию системы обработки заявок на устранение неполадок. Вы решили выяснить, из каких логических компонентов состоит система, но ваша команда ничего не знает о них. Чтобы определить логическую архитектуру, вам нужно взглянуть на структуру каталогов. Сколько отдельных логических компонентов вы сможете выделить из приведенной кодовой базы? Вот наши ответы.



Что чему соответствует? Решение

Мы определили все компоненты логической и физической архитектуры, но они вдруг перепутались. Помогите нам разобраться, что чему соответствует. Имейте в виду: некоторые компоненты могут ничему не соответствовать — ни логической, ни физической архитектуре.



Возьмите в руку карандаш

Решение



Вашей компании нужна новая система распределения рабочих по строительным площадкам, и ваша задача как архитектора ПО — определить ее основные компоненты, используя подход на основе рабочих процессов. Напомним, что одному шагу рабочего процесса может соответствовать несколько компонентов, а одному компоненту — несколько шагов.

Шаг 1: составить список рабочих, указав их квалификацию и местонахождение.

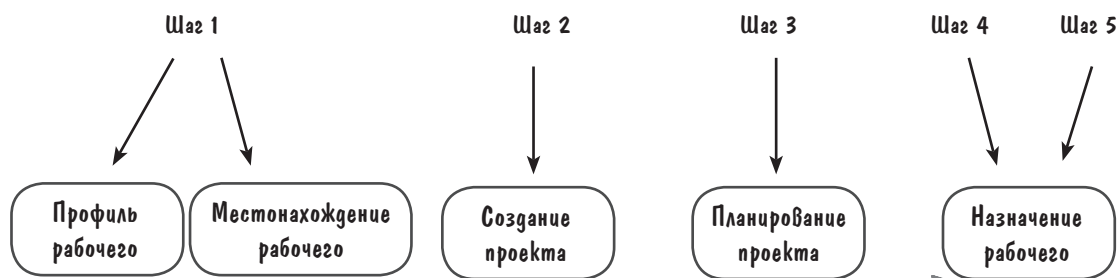
Шаг 2: создать новый строительный проект и указать место площадки.

Шаг 3: составить график начала и окончания строительного проекта.

Шаг 4: в начале назначить рабочих с учетом их профессий и местонахождения.

Шаг 5: по завершении освободить рабочих для перевода на другое место.

Нарисуйте здесь свои логические компоненты. Постарайтесь дать им хорошие описательные названия.



Имеет смысл создать два компонента, поскольку профиль относительно статичен, а местоположение часто меняется.

Компонент «Назначение рабочего» может назначать рабочих (шаг 4) и отменять их назначение (шаг 5).

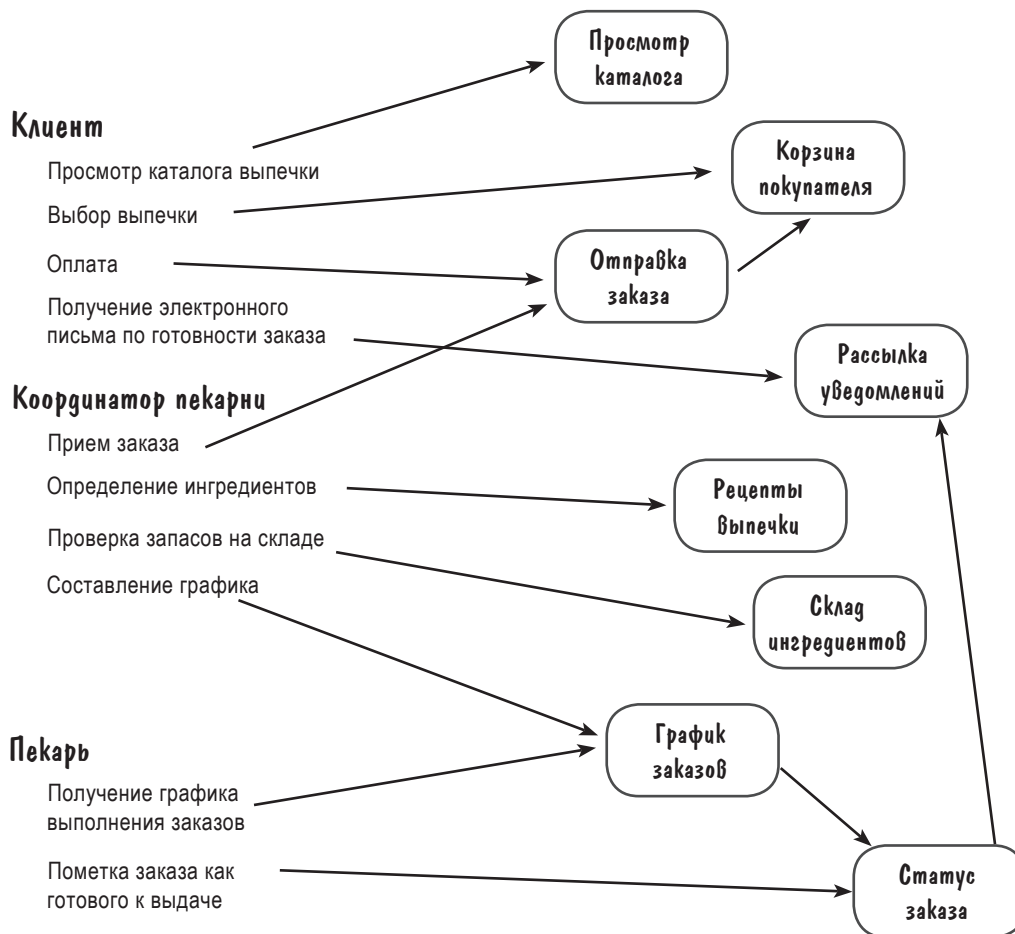


Упражнение Решение

На с. 167

Вы владеете пекарней и готовы расширить свой бизнес, но вам нужна новая система, которая позволит клиентам просматривать, заказывать и оплачивать выпечку онлайн. Заказы передаются координатору пекарни, который закупает ингредиенты и составляет график выполнения заказов. Утром пекарь получает график и сообщает системе о готовности каждого изделия. После этого система отправляет клиентам электронные письма, сообщая, что их заказы готовы к выдаче.

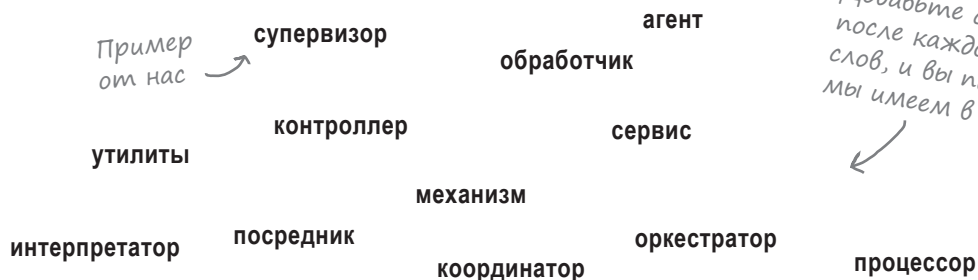
Используя подход на основе субъектов и действий, определите, какие действия может предпринять каждый субъект. Затем нарисуйте как можно больше логических компонентов, соответствующих действиям.



Возьмите в руку карандаш

Решение

Какие еще слова, помимо «управление», по вашему мнению, могли бы говорить о попадании в ловушку сущностей?



Упражнение

Решение

Выберите наилучший подход к определению начального набора основных компонентов в следующих сценариях. В некоторых случаях можно использовать несколько подходов.

В системе только один тип пользователей	☒ Рабочий процесс	☐ Субъект/действие
Система имеет четко определенные сущности	☒ Рабочий процесс	☒ Субъект/действие
Функциональные требования минимальны	☒ Рабочий процесс	☒ Субъект/действие
Пользователь выполняет много сложных действий	☒ Рабочий процесс	☐ Субъект/действие
В системе много типов пользователей	☐ Рабочий процесс	☒ Субъект/действие

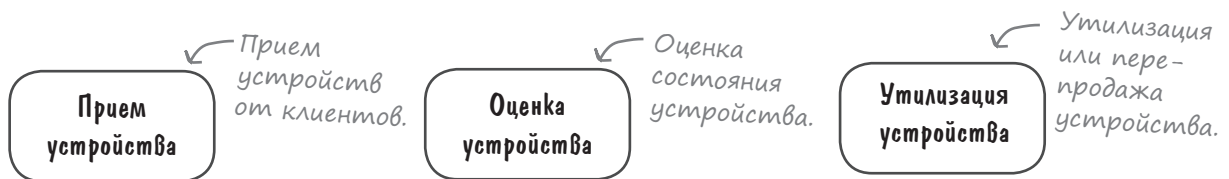
Ни один из подходов не требует большого количества функций для создания начального набора логических компонентов.

Опасайтесь ловушки для сущностей, даже если в системе есть хорошо определенные сущности.

Возьмите в руку карандаш

Решение

В вашей компании, Going Green Corporation, решили создать систему для поддержки новой программы утилизации электроники, в рамках которой клиенты смогут сдавать старые устройства (например, сотовые телефоны) и получать деньги. Мы уже определили несколько основных компонентов. Ваша задача — закрепить за существующими компонентами перечисленные ниже функции или создать новые компоненты. Кроме того, вы должны придумать названия для всех новых компонентов.



Поиск ближайшего пункта утилизации

☐ Прием ☐ Оценка ☒ Утилизация ☐ Другое: _____

Сбор и хранение информации о клиентах (имя, адрес и т. д.)

☐ Прием ☐ Оценка ☐ Утилизация ☒ Другое: Профиль клиента

Размещение объявления на стороннем сайте для перепродажи

☐ Прием ☐ Оценка ☒ Утилизация ☐ Другое: _____

Выплата вознаграждения клиенту

☐ Прием ☐ Оценка ☐ Утилизация ☒ Другое: Оплата

Фиксация факта приема устройства для оценки

☒ Прием ☐ Оценка ☐ Утилизация ☐ Другое: _____

Определение стоимости (если имеется) утилизируемого устройства

☐ Прием ☒ Оценка ☐ Утилизация ☐ Другое: _____

Определение ежемесячной прибыли и убытков от утилизации и перепродажи устройств

☐ Прием ☐ Оценка ☐ Утилизация ☒ Другое: Финансовая отчетность

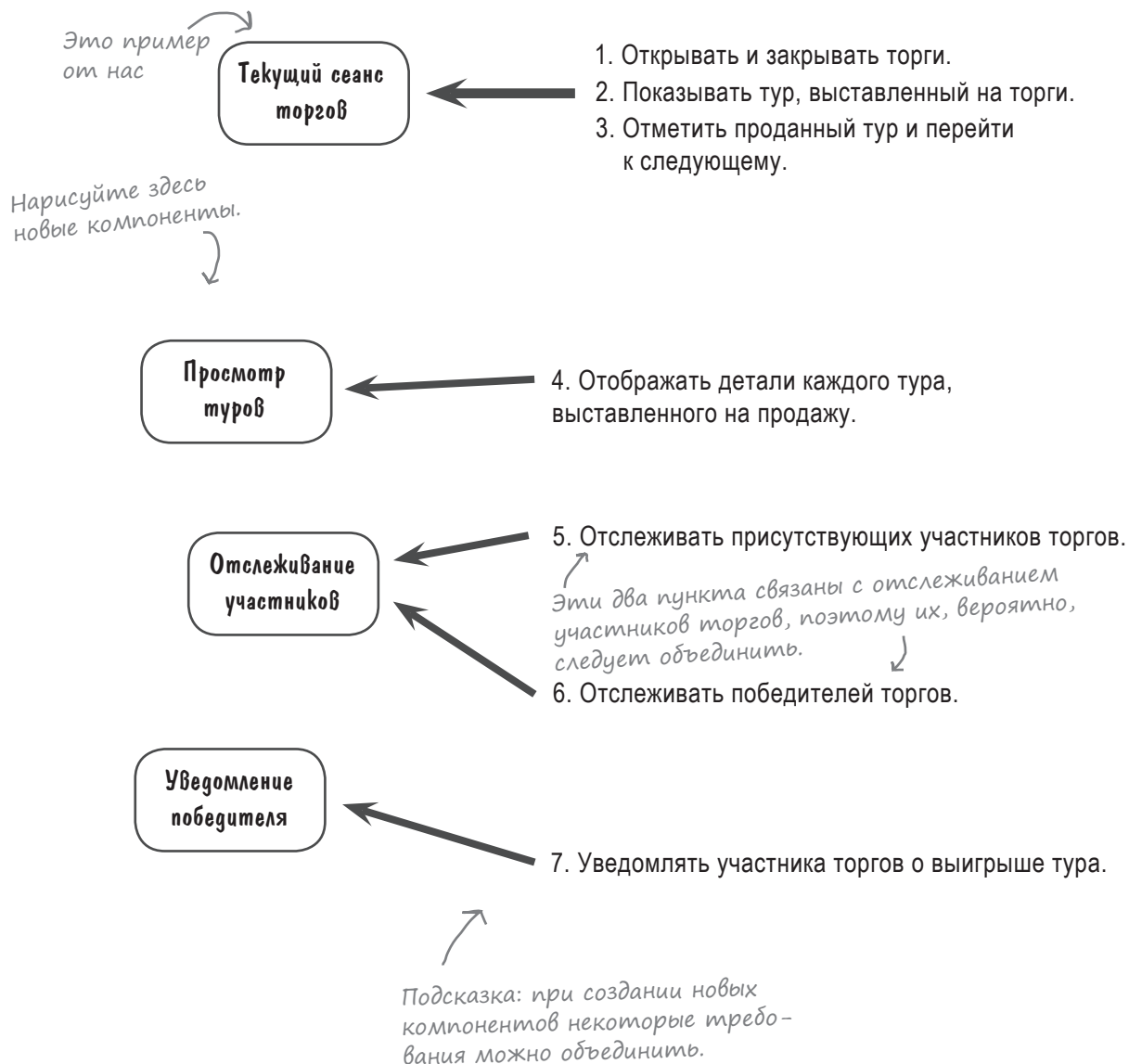
В конце концов, Going Green Corp. хочет получать прибыль.

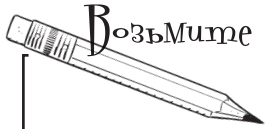


Упражнение Решение

На с. 173

Попробуйте переложить часть обязанностей с компонента «Текущий сеанс торгов» на новые компоненты, ответственные за дополнительные функции, а за ним оставьте только первые три требования.





Возьмите в руку карандаш

Решение

Для каждого компонента, указанного ниже, определите уровни афферентной (СА), эфферентной (СЕ) и суммарной (СТ) связанности. Кроме того, оцените общий уровень связанности для всей логической архитектуры. Как вы думаете, уровень СТ для этой архитектуры высокий или низкий?

СА = 0
СЕ = 1
СТ = 1

Компонент А

СА = 1 *Пример от нас*
СЕ = 0
СТ = 1

Компонент Б

СА = 0
СЕ = 1
СТ = 1

Компонент Ж

СА = 0
СЕ = 0
СТ = 0

Компонент З

СА = 2
СЕ = 0
СТ = 2

Компонент В

Компонент Г

СА = 0
СЕ = 2
СТ = 2

СА = 2
СЕ = 3
СТ = 5

Компонент И

СА = 1
СЕ = 0
СТ = 1

Компонент К

СА = 1
СЕ = 0
СТ = 1

Компонент Д

Компонент Е

СА = 0
СЕ = 2
СТ = 2

Компонент М

СА = 1
СЕ = 0
СТ = 1

СА = 1
СЕ = 0
СТ = 1

Компонент Л

Общий уровень связанности системы: **18. Это высокий уровень.**



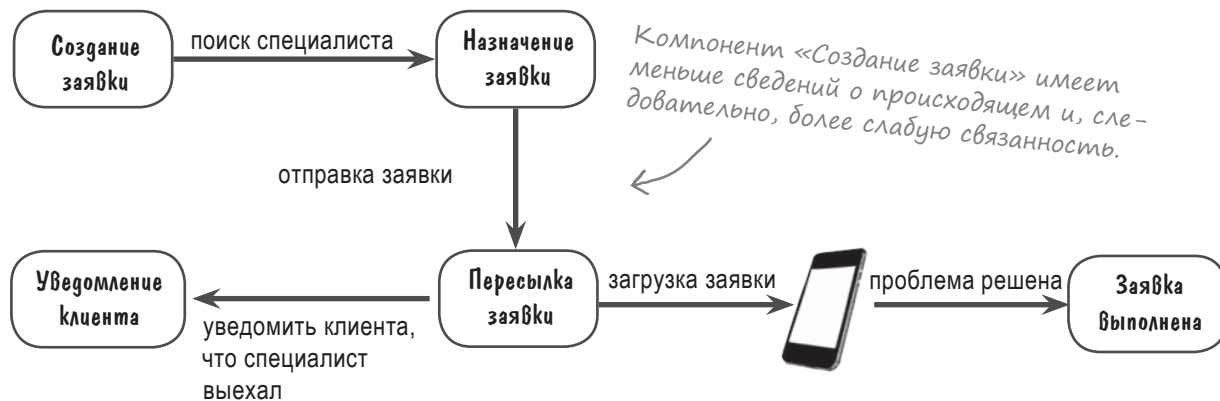
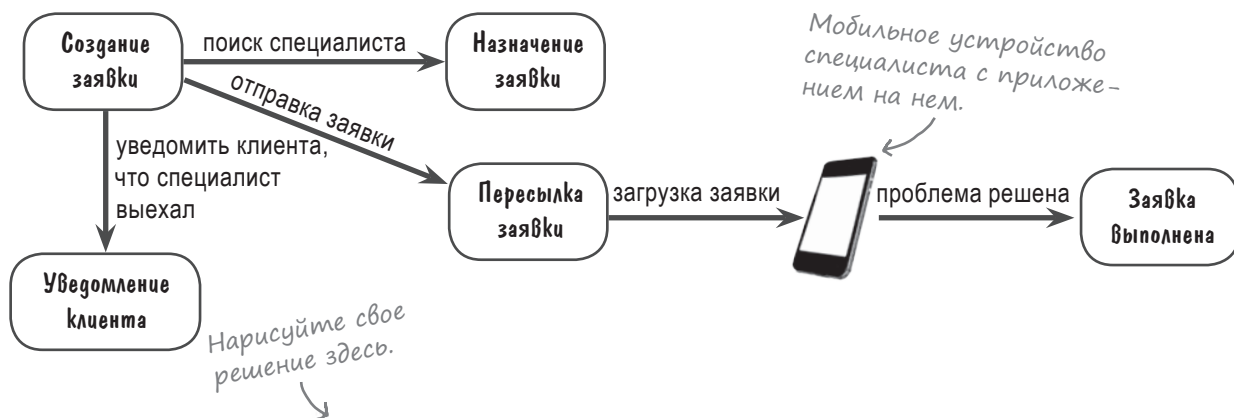
Тест-драйв. Решение

На с. 184

Теперь проведите тест-драйв закона Деметры и попробуйте уменьшить связанность логической архитектуры, представленной ниже. С помощью этой системы клиенты, купившие поддержку электронного устройства, могут сообщить о неисправности и вызвать специалиста по ее устранению. Система работает так:

- ❶ Клиент оставляет заявку.
- ❷ Заявка назначается свободному профильному специалисту.
- ❸ Заявка загружается в приложение на мобильном устройстве специалиста («Пересылка заявки»).
- ❹ Клиент получает уведомление о выезде специалиста.
- ❺ После устранения проблемы специалист отмечает заявку как выполненную.

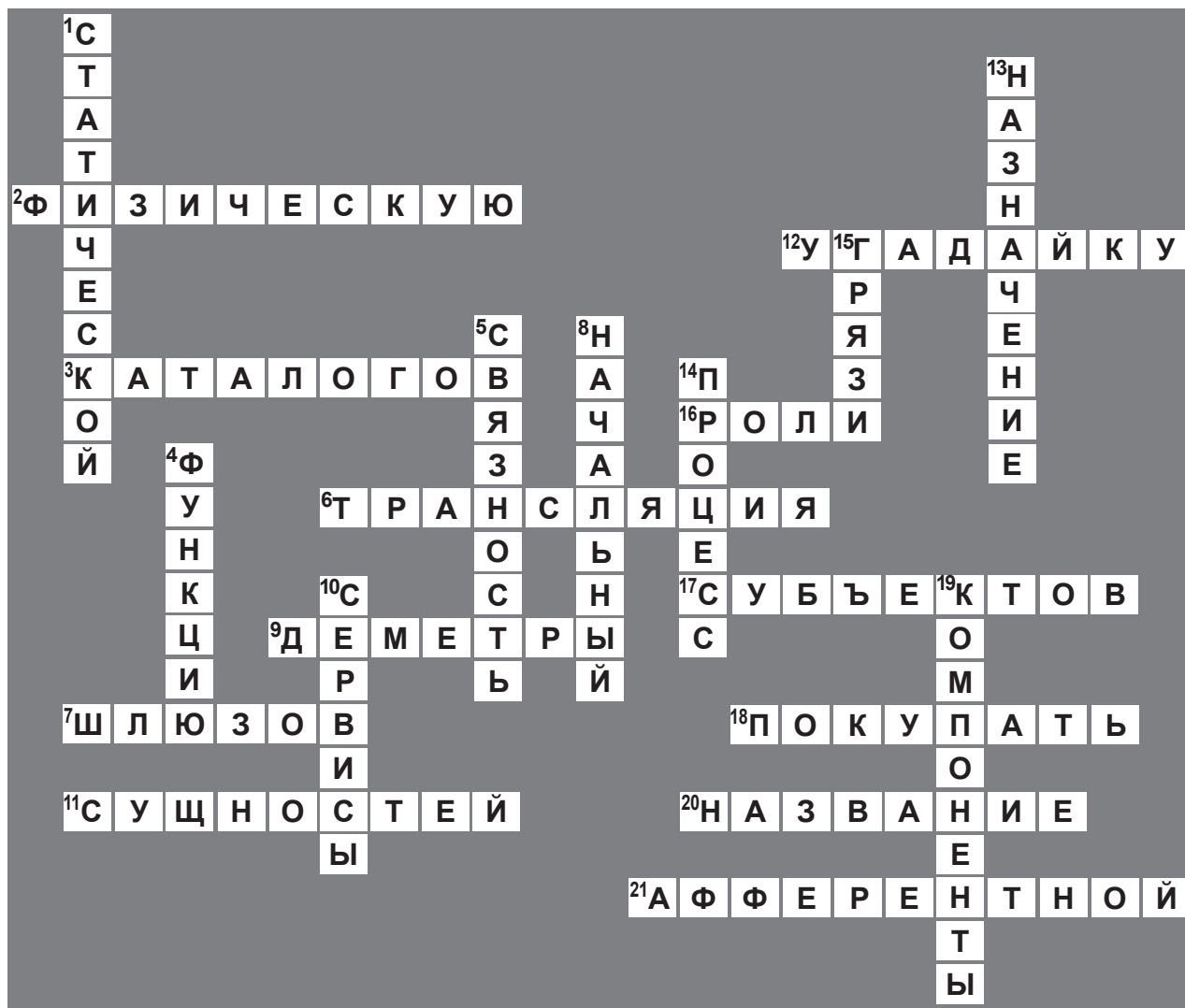
Можно ли уменьшить связанность архитектуры с этим набором компонентов?





Кроссворд по логическим компонентам. Решение

На с. 187



5

Архитектурные стили

Классификация и философия

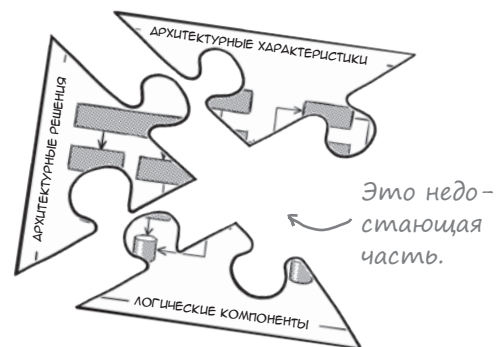
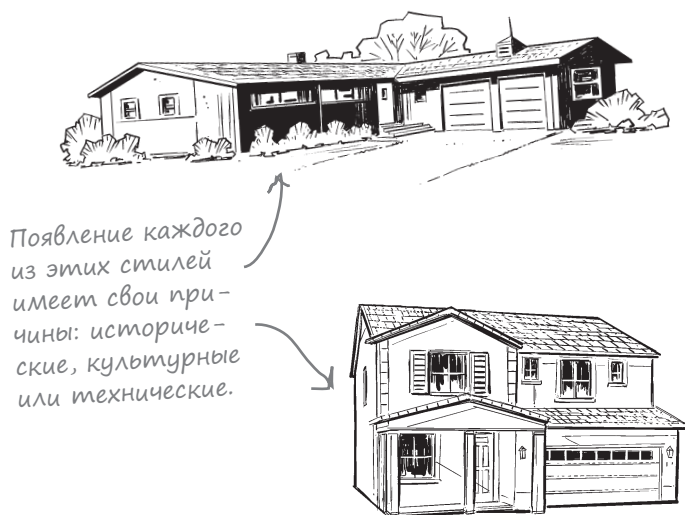


Существует множество разных архитектурных стилей. Каждый из них имеет свои причины создания и принципы использования. Понимание философии стиля поможет вам решить, подходит ли он для вашей области. В этой главе мы коротко поговорим о различных типах архитектурных стилей (рассматриваемых далее в книге), что поможет вам ориентироваться в них, а также в других стилях, с которыми вы рано или поздно столкнетесь в своей профессиональной деятельности.

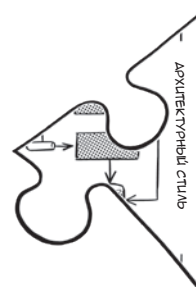
Существует множество архитектурных стилей

Вы уже немало узнали об архитектуре ПО, но есть еще кое-что, о чем мы пока не говорили: *архитектурные стили*. Именно ими мы сейчас займемся — фактически им посвящена вся *остальная* часть книги!

Прежде чем начать, взгляните вокруг, затем посмотрите передачу или фильм, действие которого происходит в другой части света. Сколько стилей домов вы увидели? Их буквально сотни. Все они зависят от местности, природных условий и личных предпочтений владельцев. И каждый день появляются новые.



А вот и недостающая часть!



Это верно и для архитектурных стилей программного обеспечения. Их настолько много, что даже в такой большой книге, как эта, мы не сможем описать все.

В текущей главе вы получите пищу для размышлений об архитектуре и архитектурных стилях в целом. А в последующих главах познакомитесь с несколькими конкретными стилями и их философией, опираясь на знания, обретенные в этой главе. Знакомство с несколькими важнейшими стилями поможет понять другие, с которыми вы столкнетесь в своей профессиональной карьере.

Приступим.

Мир архитектурных стилей

Если прежде вы занимались разработкой программного обеспечения, то наверняка слышали о различных архитектурных стилях, таких как монолиты и микросервисы. Упорядочить наши рассуждения помогут два варианта классификации. Первый вариант — классификация по способу организации кода: по техническим или предметным соображениям. Второй вариант — классификация по способу развертывания системы: в форме единого целого или в виде набора модулей.

Как рассказывалось в главе 2, предметная область — это «причина создания программы».

Мы опишем эти варианты классификации на следующих страницах. А пока просто познакомьтесь с ними.

		Организация кода	
		Техническая	Предметная
Модель развертывания	Монолитная	Многоуровневая Микроядерная	Модульный монолит
	Распределенная	Событийная	Микросервисы

Каждому из этих стилей посвящена отдельная глава.

Как видите, существует несколько классов архитектурных стилей. Конечно, это не полный список. Есть особые архитектуры (например, проблемно-зависимые), решающие определенные задачи, но мы не можем привести описания всех видов в одной книге.

В этой книге мы не будем рассматривать проблемно-зависимые архитектуры. Отметим лишь, что они не имеют широкого распространения.

Каждая категория имеет свои архитектурные характеристики. Например, монолитные архитектуры проще понять, а модульные лучше масштабируются. Рассмотрим каждую категорию более пристально.

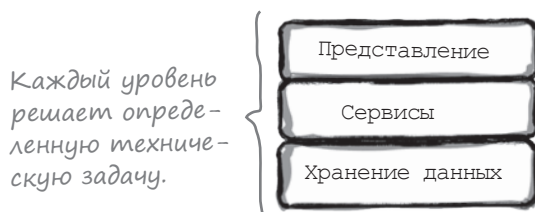
Организация кода: техническая и предметная

Вспомните свой последний ужин в шикарном ресторане. На входе вас встретил *распорядитель* и проводил к столику. *Официант* предложил напитки и меню и рекомендовал некоторые блюда. *Шеф-повар* приготовил еду. А по окончании трапезы *официант* убрал со стола.

Обязанности работников ресторана распределены по **техническому признаку**. Шеф-повар не обязан встречать вас у входа, а официант — готовить еду.

Теперь вспомните ваше последнее приложение. В нем был уровень контроллеров? А сервисы? А уровень хранения данных? Если да, то поздравляем: вы уже работали над архитектурой, разделенной по техническому признаку.

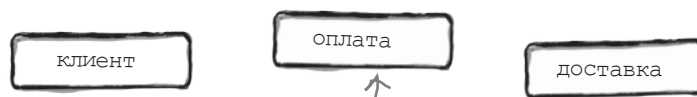
В **архитектуре с технической организацией** код разделен в зависимости от технических задач — в ней может присутствовать уровень представления, уровень сервисов и т. д. Здесь действует принцип деления по задачам, которые большинство представляет себе как горизонтальные уровни.



Еще одна аналогия — гамбургер: две половинки булочки, приправы, овощи и котлета — каждый слой играет свою роль.

А теперь представьте фуд-корт. В нем множество ресторанов, и каждый специализируется на определенных блюдах: пицца, салаты, жаркое, бургеры. Другими словами, у каждого ресторана своя **предметная область**.

В **архитектурах с предметной организацией** структура системы соответствует предметной области. Код (и системы) делится не по ролям, а в соответствии с решаемой задачей.



В архитектурах с предметным разделением код делится согласно предметным областям.

В каждом из этих ресторанов могут быть свои официанты и повара. Но в целом заведения специализируются на определенных блюдах.

Все это мы подробно рассмотрим в последующих главах.

Часть Задаваемые Вопросы

В: Где в архитектуре с предметной организацией располагаются уровни представлений и сервисов?

О: В такой архитектуре главную роль играют предметные области, а техническая реализация остается просто реализацией. Логические компоненты, составляющие архитектуру, организованы по *предметным областям*, а не по техническим задачам.

Компоненты архитектуры с технической организацией могут размещаться в таких пространствах имен, как `app.presentation.customer` и `app.services.customer`. Отметьте, как предметная область *customer* (клиент) отображается в технические разделы. Компоненты архитектуры с предметной организацией будут размещаться в таких пространствах имен, как `app.customer.presentation` и `app.customer.services`.

В: Предметная организация выглядит лучше и логичнее. Так зачем использовать техническую?

О: Мы стараемся не использовать такие оценочные суждения, как «лучше» и «логичнее», обсуждая архитектурные стили. Выбор стиля, как вы знаете, зависит от множества факторов, в том числе таких, как область применения и требуемые архитектурные характеристики.

Техническая организация отлично подходит для специализированных команд, например, если у вас есть команды специалистов по пользовательскому интерфейсу, серверной части и базам данных, тогда как предметная организация лучше согласуется с системой, предназначенной для решений фактической задачи.



Упражнение

Повара в ресторане быстрого питания могут приготовить практически все, что есть в меню, от картошки фри до сэндвичей, а также смешать смузи и разложить десерты. Часто они подают блюда, принимают оплату и даже убирают со столов после ухода клиентов. По вашему мнению, как организована работа такого повара: по техническим или предметным признакам? Почему? Запишите свои мысли ниже.

→ Решение на с. 214

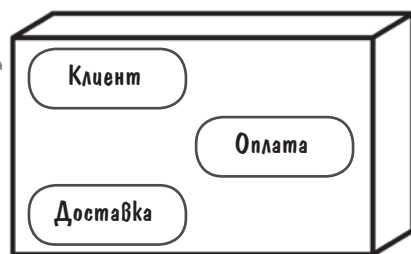
Модель развертывания: монолитная и распределенная

Теперь поиграем в игру: мы говорим слово, а вы отвечаете первое, что приходит на ум. Готовы? *Монолит*.

Не знаем, как у вас, но у нас сразу всплывает образ чего-то *большого*: скалы или ледника. На самом деле именно так организованы монолитные архитектуры.

В **монолитной архитектуре** все логические компоненты, составляющие приложение, развертываются как *одно целое*, и такие приложения выполняются в одном процессе.

Это монолитное приложение, содержащее все логические компоненты в одной единице развертывания.

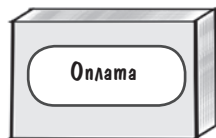


В экосистеме Java такое приложение развертывается как единственный WAR- или JAR-файл, а в мире .NET — как единственный выполняемый файл.

Напомним, что архитектурные характеристики влияют на структуру приложения.

В **распределенной архитектуре**, напротив, логические компоненты помещаются в множество отдельных модулей, которые выполняются в собственных процессах и взаимодействуют друг с другом по сети.

В распределенных архитектурах развертывается множество небольших модулей.



Это важные категории архитектур, поэтому рассмотрим их плюсы и минусы.



Упражнение

Взгляните на свой смартфон. Он позволяет снимать фото и видео, просматривать веб-страницы, писать сообщения в социальных сетях, отслеживать физическую активность и ориентироваться по GPS. А еще в нем есть телефон! Как видите, смартфон — это монолитная система. Мы предлагаем вам записать плюсы и минусы такой системы. Думайте в терминах таких архитектурных характеристик, как доступность, обновляемость, стоимость и юзабилити.



Не так давно для всех перечисленных выше функций мы использовали отдельные устройства. Телефоны были просто *телефонами*, хотя и позволяли обмениваться текстовыми сообщениями. Для просмотра веб-страниц и общения в социальных сетях использовались компьютеры. Для тренировок мы покупали фитнес-трекеры. Для навигации применяли устройства GPS. Все эти «сервисы» разворачивались по отдельности.

Теперь точно так же запишите плюсы и минусы такой системы. И снова подумайте об архитектурных характеристиках.



→ Решение на с. 215

Монолитные модели развертывания: плюсы

В главе 2 вы узнали, что архитектурные характеристики влияют на *структурные аспекты* проекта. Некоторые характеристики монолиты поддерживают лучше, чем распределенные системы, и зная плюсы монолитов, вы сможете решить, когда их использовать.

Монолитные системы, выполняющиеся в одном процессе, проще разрабатывать, по крайней мере на начальном этапе. А благодаря развертыванию как единого целого их легче отлаживать.

Перечислим плюсы и минусы обеих моделей развертывания, начав с монолитов. Вот их плюсы:



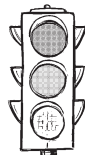
простота

Обычно монолитные приложения имеют единую кодовую базу, что упрощает их разработку.



стоимость

Создание и эксплуатация монолитов обходится дешевле, поскольку они зачастую проще и требуют меньше инфраструктуры.



осуществимость

Спешите выйти на рынок? Монолиты просты и относительно дешевы, что позволяет экспериментировать и быстрее выпускать системы.



надежность

Монолит самодостаточен. Он не использует или почти не использует сетевое взаимодействие, что способствует более высокой надежности.



простота отладки

Отладка ошибки не составит труда, поскольку весь код сосредоточен в одном месте.

Это лишь некоторые из плюсов монолитов.

Вспомните этот момент, когда мы будем обсуждать «минусы» на следующей странице.

Теперь перейдем к минусам...

Монолитные модели: минусы

Некоторые плюсы монолитов могут превратиться в минусы по мере развития приложения. Многие эксплуатационные характеристики, обсуждавшиеся в главе 2, такие как масштабируемость и надежность, ухудшаются по мере усложнения монолитного приложения.



масштабируемость

В монолитах сложно масштабировать одну часть приложения независимо от других. Они подчиняются принципу «все или ничего».



надежность

Монолиты развертываются как единое целое, поэтому любая ошибка, ухудшающая его работу, затронет весь монолит.

И снова надежность!



развиваемость

Вносить изменения в монолитные приложения становится все сложнее. Кроме того, все приложение сосредоточено в единой кодовой базе, поэтому вы не сможете использовать разные технологические стеки для разных областей.

Опять же, это далеко не все минусы, на которые мы решили обратить внимание.



развертываемость

Для внедрения любых изменений требуется повторное развертывание всего приложения, что чревато большими рисками.

Далее мы рассмотрим плюсы и минусы распределенных архитектур.



Мозговой штурм

Поразмышляйте о своей организации. Есть ли в ней особые требования к соответствию нормативным актам или безопасности? Как монолитная архитектура может помочь или помешать достижению нужных архитектурных характеристик? Перечислите все плюсы и минусы:

Плюсы

Минусы

Подсказка: подумайте о таких вещах, как контролируемость, отчетность, «право на забвение» и т. д.

Распределенные модели развертывания: плюсы

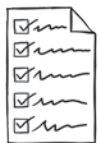
В распределенных архитектурах логические компоненты развертываются по отдельности. Это упрощает их масштабирование независимо друг от друга. А поскольку логические компоненты физически разделены, то распределенные архитектуры способствуют *слабой связанности*.

Итак, какие архитектурные характеристики хорошо поддерживаются распределенными архитектурами? Вот некоторые из них:



масштабируемость

Логические компоненты развертываются по отдельности. Нужно масштабировать один из них? Нет проблем!



тестируемость

В каждом развертывании участвует только часть логических компонентов. Это упрощает тестирование, даже если сложность приложения возрастает.

Распределенные архитектуры проще тестировать, чем монолиты.



отказоустойчивость

Даже если одна часть системы выйдет из строя, остальные продолжат функционировать.



модульность

Распределенные архитектуры способствуют модульности, поскольку требуют, чтобы логические компоненты были слабо связаны между собой.



развертываемость

Распределенные архитектуры имеют модульную организацию. Они начали развиваться после появления современных инженерных принципов, таких как непрерывная интеграция, непрерывное развертывание и автоматизированное тестирование.

Модульность и хорошая тестируемость снижают риски при развертывании изменений.

Как видите, распределенные архитектуры лишены многих недостатков монолитных архитектур. Но верно ли обратное? Давайте выясним.

Распределенные модели: минусы

Не может быть плюсов без минусов. Компромиссы, верно? Все дело в компромиссах.



производительность

Распределенные архитектуры состоят из множества небольших сервисов, взаимодействующих друг с другом по сети. Это может повлиять на производительность, и хотя есть способы улучшить ситуацию, вам следует иметь это в виду.



стоимость

Развертывание нескольких модулей означает рост количества сервисов. Они должны взаимодействовать друг с другом, поэтому возникает необходимость настройки и обслуживания сетевой инфраструктуры.



простота

Распределенные системы сложны. Все — от понимания принципов работы до отладки ошибок — существенно усложняется.

Мы не можем не подчеркнуть, что распределенные архитектуры могут быть очень сложными!

Отладка распределенных систем требует детального изучения журналов и их агрегирования. Это тоже увеличивает стоимость.



простота отладки

Ошибки могут обнаружиться в любом сервисе, участвующем в обработке запроса. Логические компоненты развертываются по отдельности, поэтому поиск ошибок может оказаться очень сложным занятием.

Распределенные архитектуры упрощают одно и усложняют другое.



Будьте осторожны!

Сложность распределенных вычислений легко недооценить!

Распределенные архитектуры зависят от сети, и многие недооценивают возникающие из-за этого сложности. Ознакомьтесь со списком *The Fallacies of Distributed Computing* («Заблуждения о распределенных вычислениях»), составленным Л. Питером Дойчем (L. Peter Deutsch) и другими сотрудниками Sun Microsystems, чтобы знать, чего остерегаться.

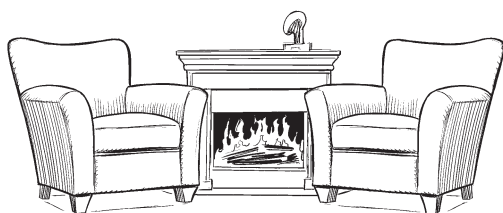


Мозговой шторм

Повторим упражнение, которое вы выполняли выше, но теперь возьмем распределенную архитектуру. Есть ли в вашей организации особые требования к соответствию нормативным актам или безопасности? Как монолитная архитектура может помочь или помешать достижению нужных архитектурных характеристик? Перечислите все плюсы и минусы:

Плюсы

Минусы



Беседы у камина

Актуальная тема: **монолитная и распределенная** архитектуры отвечают на вопрос, **какая из них более актуальна.**

Монолитная архитектура

Хорошо, что есть я. Ты слишком все усложняешь.

Меня не только проще, но и быстрее разрабатывать. Не могу представить, чтобы кто-то, применив тебя, смог создать минимально жизнеспособный продукт!

Зато я намного дешевле. Ты же понимаешь, что большинство компаний не хотят тратить деньги впустую? Не могу представить, чтобы кто-то использовал тебя для создания пробного варианта.

Распределенная архитектура

Ты заблуждаешься. Да, тебя «проще» разработать, но ты сильно отстала от прогресса. Бизнес должен развиваться быстро, и тебе не справиться с этой задачей.

И я могу быть простой и гарантировать успешное завершение проекта. А если продукт станет очень востребованным, то ты поможешь развивать его или будешь только мешать? Я же могу обеспечить успешную работу системы даже при больших масштабах.

Монолитная архитектура

Это хорошо, что ты упрощаешь тестирование, поскольку твои трассировки стека бесполезны! У тебя все наперекосяк, и крайне сложно понять, где и почему на самом деле возникла ошибка.

У меня же, когда случается ошибка, выводится четкая и ясная трассировка.

Я выполняюсь в единственном процессе и не создаю лишний сетевой трафик. А твои сервисы постоянно общаются друг с другом, ты *слишком много* болтаешь.

Из-за этого тебе нужна надежная сеть, без нее ты ничто! Если сеть отключится, то останется лишь молиться.

А со мной не нужна разветвленная сетевая инфраструктура. Ты же знаешь, как дорого ее обслуживать?!

Хочешь сказать, что я устарела? Что ж, в следующий раз, когда понадобится быстро выйти на рынок, не звони мне — вот тогда мы посмотрим, насколько ты на самом деле хороша.

Распределенная архитектура

А еще компании любят *делать* деньги. Достигнув определенного уровня, ты превращаешься в бездонную бочку. Я же — сама гибкость и помогаю компаниям масштабироваться по мере их развития.

К тому же я упрощаю тестирование, а ты лишь наращиваешь технический долг.

Ха! Не справившись с ошибкой, ты просто падаешь. Я же даю высокую надежность. Нужно масштабировать сервис? Масштабируйте! А *тебя* масштабировать очень сложно.

Это плата за масштабируемость бизнеса. Команды могут *начать* с тебя, но решив продолжать развиваться, они обратятся ко мне, а ты останешься в зеркале заднего вида.

Взаимно, подруга. И ты не звони *мне*, когда выяснится, что продукт вашей команды стал очень популярным, но его архитектура не справляется с наплывом желающих воспользоваться им.

Вот и все!

Вы познакомились со способами классификации архитектурных стилей и теперь, когда наступит время, сможете сделать правильный выбор. Но помните, что каждый класс архитектурных стилей имеет свои плюсы и минусы.

Со следующей главы мы начнем детальное рассмотрение отдельных стилей.

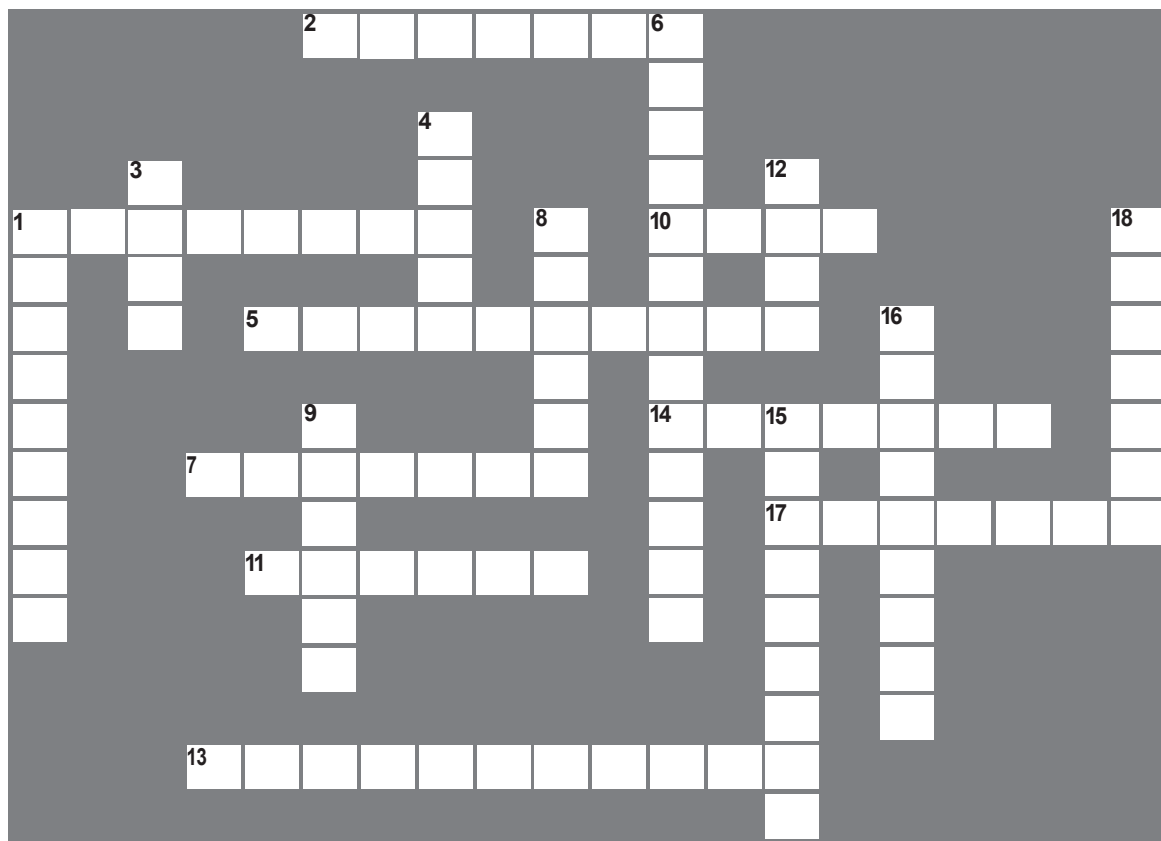
Ключевые Моменты

- Архитектурных стилей очень много — их невозможно сосчитать.
- Существует множество вариантов классификации архитектурных стилей. Один из них — по *организации кода*, которая возможна по техническим или предметным признакам.
- В архитектурных стилях с *технической организацией* код группируется по техническому признаку. Например, в системе может иметься уровень представления и уровень сервисов.
- В стилях с *предметной организацией* код группируется по предметным областям.
- Еще один вариант классификации — по *модели развертывания*. В *монолитных* архитектурных стилях все логические компоненты приложения развертываются как единое целое. В *распределенных* системах логические компоненты развертываются по отдельности.
- Монолитные архитектуры более просты в отладке, а их создание обходится дешевле (на начальном этапе). Поэтому их предпочитают использовать, когда нужно быстро вывести продукт на рынок.
- Масштабирование монолитных приложений по мере их развития становится сложной задачей. Здесь действует принцип «все или ничего»: либо масштабируется все приложение, либо ничего.
- Монолитные приложения обычно менее надежны — ошибка может прервать работу всего приложения.
- Распределенные архитектуры отличаются высокой масштабируемостью. Их логические компоненты развертываются по отдельности, что позволяет масштабировать различные части приложения независимо друг от друга.
- Распределенные архитектуры имеют модульную организацию, а значит, их легче тестировать.
- Разработка, поддержка и отладка распределенных архитектур обходятся значительно дороже.
- Распределенным архитектурам нужна надежная сеть, чтобы различные сервисы могли взаимодействовать друг с другом. Это создает дополнительные сложности.



Кроссворд по архитектурным стилям

Теперь, разобравшись с архитектурными стилями, разгадайте этот кроссворд.



По горизонтали

1. Монолиты хороши для создания _____ варианта.
2. Минимально жизнеспособный _____.
5. После любых изменений монолит нужно снова _____.
7. Модели распределенного развертывания намного _____.
10. Организация архитектуры отражается в пространствах _____.
11. В монолитной архитектуре _____ единиц развертывания.
13. Системы можно классифицировать по _____ кода.
14. Система, развертываемая как одно целое.
17. Монолитные архитектуры часто _____ выходят на рынок.

По вертикали

1. Уровни разделены по _____.
3. Л. Питер _____ помог составить список заблуждений.
4. Архитектуры часто имеют несколько _____.
6. _____ системы позволяет быть более уверенным в ней, независимо от физической архитектуры.
8. Распределенные системы обычно _____ монолитных.
9. Распределенная и монолитная — это _____ развертывания.
12. Сервисы в распределенной архитектуре используют это для связи.
15. Распределенные системы состоят из множества _____ единиц развертывания.
16. Каждый архитектурный стиль имеет свою _____.
18. Монолитные системы проще в _____.

—————→ Решение на с. 216



Упражнение Решение

На с. 203

Повара в ресторане быстрого питания могут приготовить практически все, что есть в меню, от картошки фри до сэндвичей, а также смешать смузи и разложить десерты. Часто они подают блюда, принимают оплату и даже убирают со столов после ухода клиентов. По вашему мнению, как организована работа такого повара: по техническим или предметным признакам? Почему? Запишите свои мысли ниже.

Поскольку повар сам делает все, что нужно, чтобы накормить клиентов, — от сервировки стола до приготовления блюд и уборки со столов, — то владеет всей «предметной областью» приготовления еды. Поэтому его работа имеет предметное разделение.



Упражнение Решение

На с. 205

Взгляните на свой смартфон. Он позволяет снимать фото и видео, просматривать веб-страницы, писать сообщения в социальных сетях, отслеживать физическую активность и ориентироваться по GPS. А еще в нем есть телефон! Как видите, смартфон — это монолитная система. Мы предлагаем вам записать плюсы и минусы такой системы. Думайте в терминах таких архитектурных характеристик, как доступность, обновляемость, стоимость и юзабилити.

Монолитная

Плюсы

Удобство: нужно носить только одно устройство.

Простота обновления: не нужно заниматься обновлением нескольких устройств.

Простота использования: если я спешу или отправляюсь в поездку, то достаточно взять с собой только одно устройство.

Минусы

Доступность: если телефон сломается, то все его функции станут недоступны.

Стоимость: замена смартфона может оказаться дорогим удовольствием.

Переносимость: я могу использовать только приложения, работающие в операционной системе моего телефона.

Не так давно для всех перечисленных выше функций мы использовали отдельные устройства. Телефоны были просто *телефонами*, хотя и позволяли обмениваться текстовыми сообщениями. Для просмотра веб-страниц и общения в социальных сетях использовались компьютеры. Для тренировок мы покупали фитнес-трекеры. Для навигации применяли устройства GPS. Все эти «сервисы» развертывались по отдельности.

Теперь точно так же запишите плюсы и минусы такой системы. И снова подумайте об архитектурных характеристиках.

Распределенная

Плюсы

Слабая связанность: если камера телефона сломается, я все равно смогу звонить или следить за своим состоянием на тренировке.

Модульность: каждое устройство решает только одну задачу, поэтому его проще тестировать.

Развиваемость: я могу купить зеркальную камеру, если захочу делать профессиональные снимки.

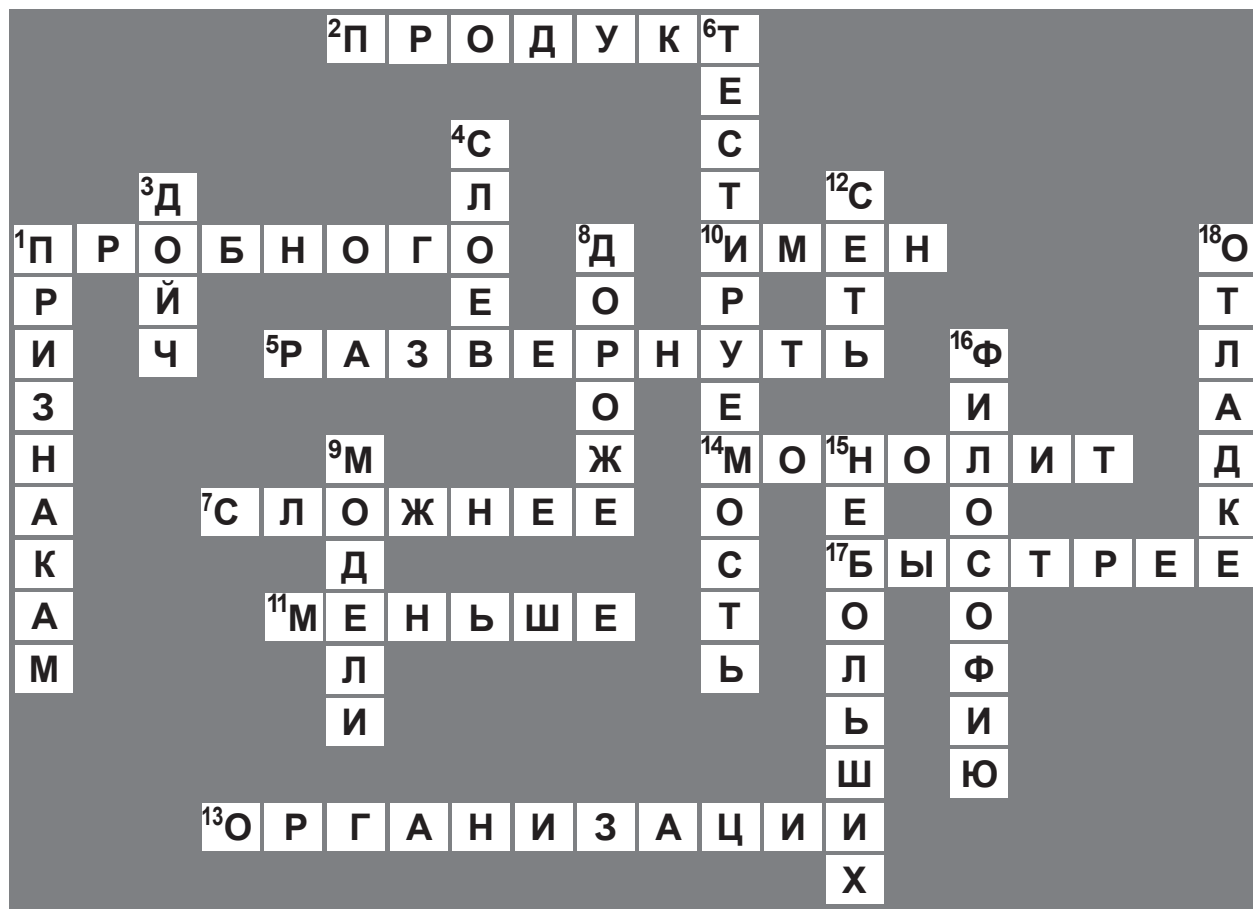
Минусы

Простота обновления: все придется обновлять отдельно.

Сложность: нужно управлять большим количеством устройств (иметь несколько аккумуляторов, зарядных устройств и т. д.).

Надежность: подключение к сети может быть ненадежным; устройства могут терять соединения или иметь нестабильную связь.

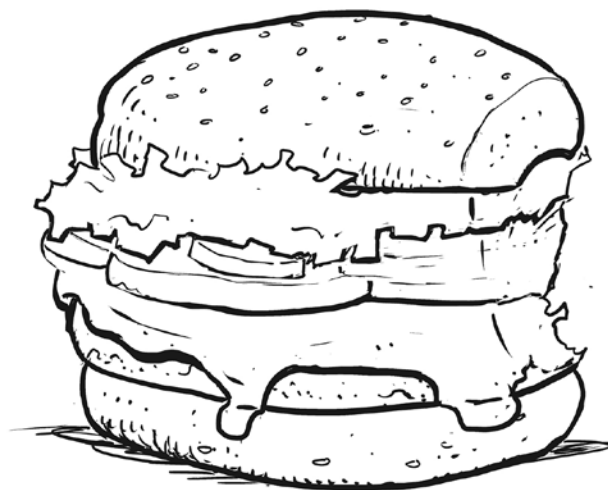
На с. 213



6

Многoурoвневaя архитектура

Разделение обязанностей



Как поступить, если задача проста и время поджимает? Стоит ли беспокоиться об архитектуре? Все зависит от того, как долго должно просуществовать то, что вы создали. Если решение одноразовое, то отбросьте осторожность. Если нет, то выберите самую простую архитектуру, которая обеспечивает некий уровень организации и позволяет получить выгоду, при этом существенно не влияя на скорость разработки. Например, это может быть *многоуровневая архитектура*, поскольку она проста и легко реализуется с помощью известных паттернов проектирования.

Naan & Pop: сбор требований

Сангита любит простые блюда, поэтому открыла семейный ресторан в индийском стиле под названием Naan & Pop, специализирующийся на гамбургерах и газированных напитках.

Ресторану нужен сайт, где клиенты могли бы размещать заказы. Naan & Pop — стартап с небольшим бюджетом, поэтому должен быть простым и его нужно создать быстро.

У Сангиты есть некоторые особые требования.



Время выхода на рынок

Ресторан уже открыт, и чем быстрее сайт появится в интернете, тем быстрее мы начнем зарабатывать деньги. Сайт должен быть простым.

Это Сангита, ресторатор и начинающий менеджер проектов.



И



Разделение обязанностей

В компании есть специалисты по пользовательскому интерфейсу и базам данных. Поэтому мы могли бы разделить систему на отдельные части.



Простая, но расширяемая

Это первый опыт Сангиты как архитектора, но она хотела бы продолжать развивать присутствие компании в интернете и постепенно расширять возможности системы.

Сангита имеет опыт разработки ПО и понимает, что многие из целей требуют разделения обязанностей. Она передала требования команде разработчиков, которую наняла для реализации проекта. Вы часть этой команды, поэтому будьте внимательны.

Разговор в кабинете



Алекс. Менеджер проекта прислала требования для веб-приложения Naan & Pop. Они довольно простые, и я думаю, что мы сможем их удовлетворить, взяв готовый фреймворк или библиотеку.

Мара. Это удовлетворит требование к простоте. Но Сангита говорит и о расширяемости, а готовые фреймворки, как правило, жесткие.

Сэм. Что она имеет в виду под расширяемостью?

Мара. В случае успеха, например, можно реализовать несколько видов пользовательских интерфейсов или создать точки интеграции для сервисов доставки.

Алекс. Да, готовые фреймворки могут не справиться с разделением обязанностей, нужным для такой расширяемости.

Сэм. Но у нас нет времени на сложную архитектуру!

Алекс. Задача кажется невыполнимой — как построить правильную архитектуру со специфическими требованиями с такими ограничениями по времени?

Сэм. Мы с другими членами команды уже определили архитектурные характеристики (возможности приложения) и дизайн предметной области (его поведение). Осталось лишь выбрать подходящую архитектуру.

Мара. Мы имеем серьезные компромиссы и противоречивые цели. Нужен простой архитектурный стиль, позволяющий разделить обязанности по техническим областям, таким как пользовательский интерфейс, хранение данных, бизнес-логика и т. д. Тогда добавление нового пользовательского интерфейса повлияет только на один уровень.

Алекс. «Разделение обязанностей», — я только что прочитал эту фразу в книге «Head First. Паттерны проектирования»! Я читал о паттерне «Модель — Представление — Контроллер».

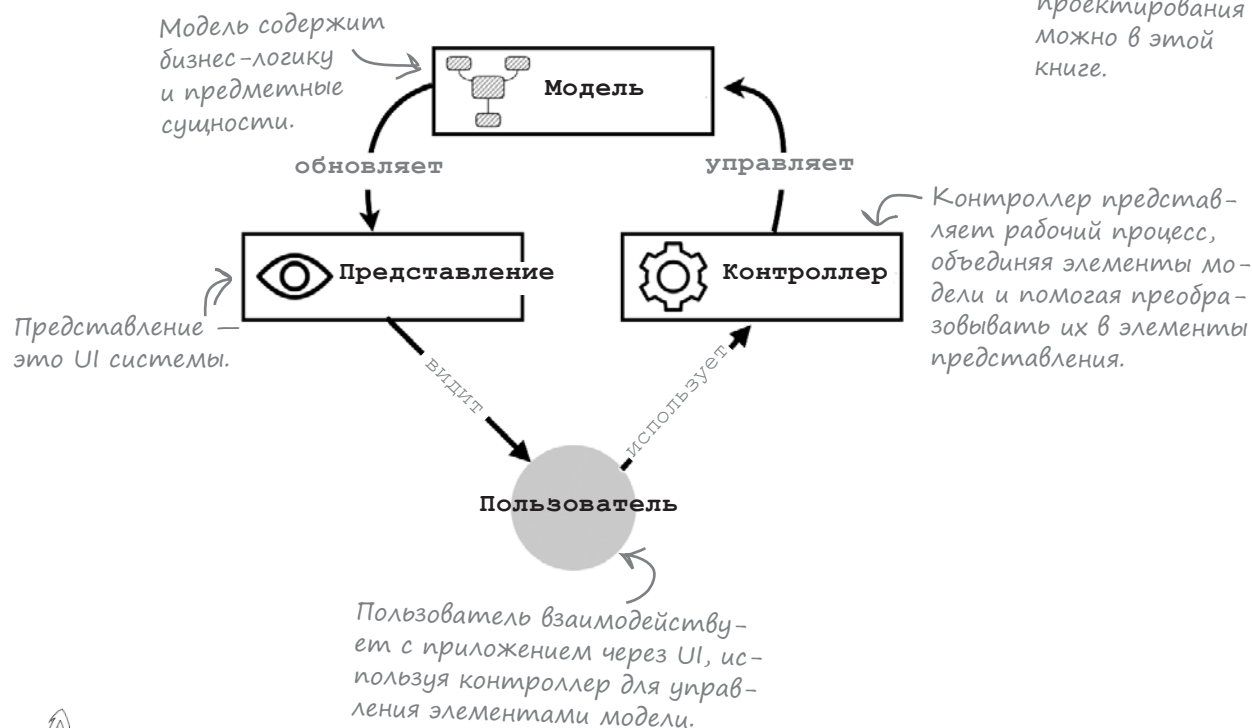
Сэм. Да, но это паттерн проектирования. Как перенести его в архитектуру?

Мара. Многие паттерны проектирования воплощаются в архитектуры, поскольку их цели совпадают. Но, в отличие от паттернов, архитектура должна учитывать ограничения реального мира. Давайте откроем книгу и посмотрим, можно ли воплотить паттерн «Модель — Представление — Контроллер» в архитектуру.

Назад к паттернам проектирования

Для иллюстрации идеи паттернов проектирования в книге «Head First. Паттерны проектирования» используется паттерн «**Модель – Представление – Контроллер**» (Model – View – Controller, MVC), разделяющий обязанности по их назначению.

В MVC **модель** представляет бизнес-логику и сущности приложения; **представление** — пользовательский интерфейс (user interface, UI); а **контроллер** реализует рабочий процесс, объединяя элементы модели и обеспечивая функциональность приложения, как показано ниже:



Узнать больше о паттернах проектирования можно в этой книге.



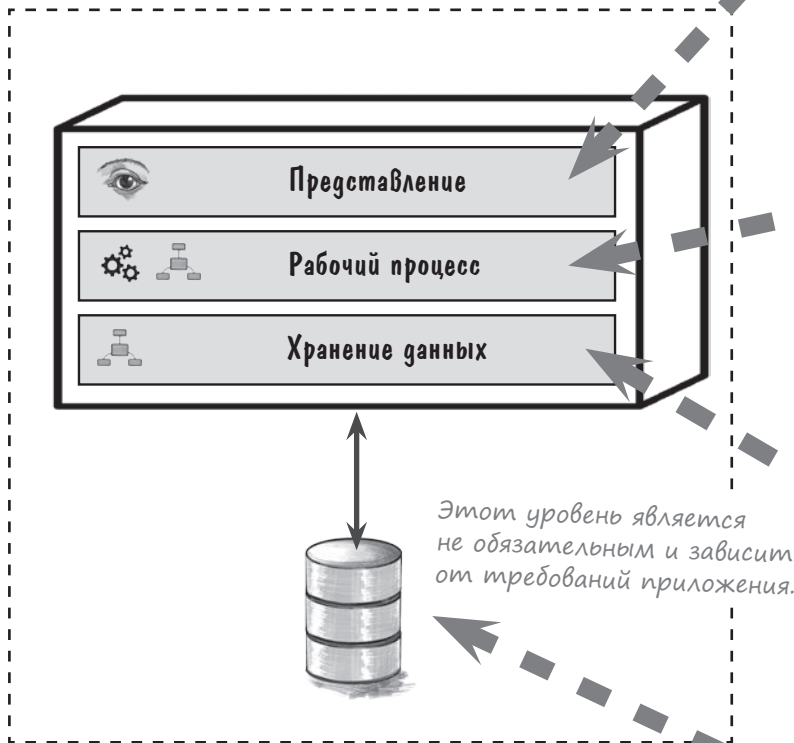
Мозговой штурм

Паттерн проектирования MVC разделяет *логические* обязанности, но в архитектуре ПО приходится учитывать особенности и *физических* систем, таких как браузеры и базы данных. Как бы вы разделили обязанности MVC в рамках ограничений программной архитектуры, сохранив общую цель разделения обязанностей и задач?

Многоуровневый паттерн MVC

Паттерны проектирования — это логические решения задач, но архитектура должна учитывать ограничения баз данных, пользовательских интерфейсов и другие детали реализации.

Типичная многоуровневая архитектура



Пунктирная рамка — граница монолита.
 Монолит — это единый модуль развертывания.

Многоуровневый монолит

Монолитная модель развертывания, о которой рассказывалось в главе 5, часто сочетается с многоуровневыми архитектурами. Над кодом и базой данных обычно работают разные команды, но в монолитной архитектуре любые изменения выпускаются *вместе*.

Представление

V в MVC означает View (представление) и отвечает за взаимодействие с системой. В многоуровневой архитектуре элементы пользовательского интерфейса находятся на уровне *представления*.

Рабочий процесс

Уровень *рабочего процесса* содержит основной код приложения. Бизнес-логика, проверки и другие операции находятся на этом уровне.

Хранение данных

Многие команды используют специальный уровень *хранения данных* для отображения иерархий программных объектов в сущности реляционных баз данных.

База данных

«Модель» в MVC обычно связана с базой данных или другим механизмом хранения данных.

Не все приложения используют базу данных и могут хранить информацию в других местах: в файловой системе, облаке и т. д.



Интервью

Звезда многоуровневой архитектуры: Уровень

Head First. Добро пожаловать в нашу студию. Я знаю, что у вас плотный график, поэтому спасибо, что нашли время.

Уровень. Не за что. Как вы верно заметили, я играю важную роль. В честь меня даже называли архитектуру!

Head First. Давайте поговорим об этом. Зачем строить всю архитектуру с опорой на вас?

Уровень. Хороший вопрос. Я делаю архитектуру красивой и понятной, в ней каждый уровень имеет определенные обязанности.

Head First. Значит, эта архитектура только для аккуратистов?

Уровень. Нет-нет! Связанные функции, размещенные на отдельных уровнях, легче искать и изменять. Например, чтобы добавить еще одну базу данных, достаточно изменить уровень хранения данных.

Head First. Значит, такая организация упрощает поиск и изменение информации? Это хорошая причина для создания архитектуры.

Уровень. Унификация — хорошая, но не единственная причина, почему архитектура основывается на мне.

Head First. Что вы имеете в виду?

Уровень. Не хочу хвастаться, но мы, уровни, очень гибкие — нас можно использовать для самых разных целей!

Head First. Да, вы часто размещаете в себе пользовательские интерфейсы и бизнес-логику.

Уровень. Это типичные задачи. Но вообще из нас можно формировать самые разные интерфейсы. Например, уровень сервисов может предоставлять интерфейс другим приложениям для взаимодействия с ним.

Head First. Приведете несколько хороших примеров вашего использования на практике.

Уровень. Конечно! Однажды мы работали над программой лояльности для отеля. За каждую покупку пользователю начислялись баллы в зависимости от его статуса участника программы, стажа участия и других условий. Команда успешно использовала уровень бонусов, разместив все расчеты в одном месте.

Head First. Очень интересно! Прокомментируйте, пожалуйста недавний спор о ваших прохладных отношениях с предметно-ориентированным проектированием (DDD)?

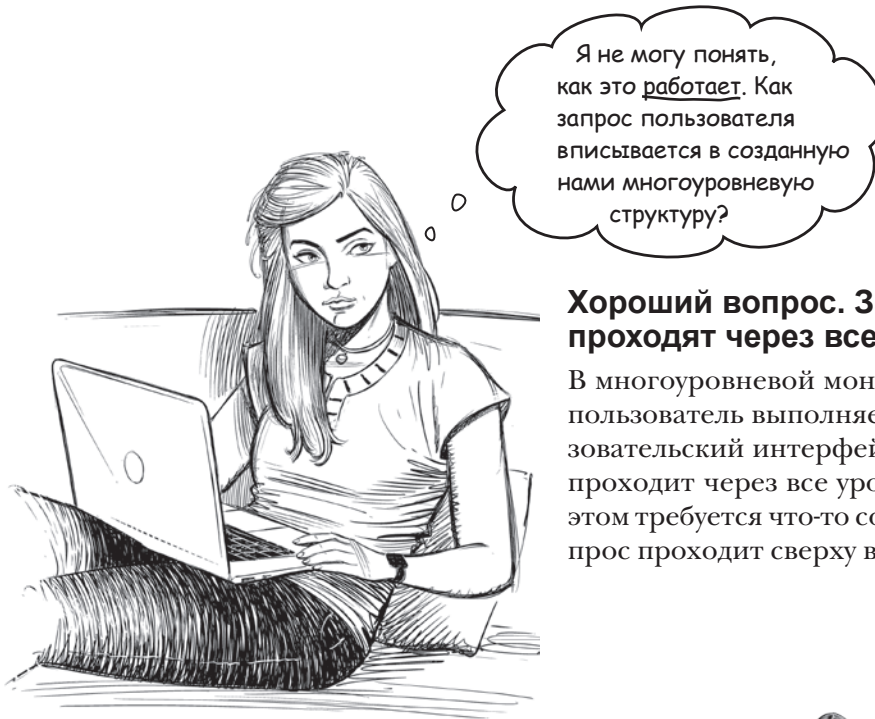
Уровень. Какой спор? Слухи о том, что мы не ладим, высосаны из пальца. Как вы знаете, я специализируюсь на *техническом* разделении, а мой друг DDD — на *предметном*. Я с удовольствием размещу предметные области в своей архитектуре, но их, скорее всего, придется разделить на уровни.

Head First. Правда ли, что вы старше других архитектурных стилей?

Уровень. Идея уровней в архитектуре возникла раньше других концепций. И это неудивительно. Когда архитекторы начинают думать об организации решения, уровни напрашиваются сами собой.

Head First. И в заключение: не могли бы вы рассказать нам о своих теплых отношениях с монолитом? Кажется, вас часто можно встретить вместе.

Уровень. Без комментариев.



Хороший вопрос. Запросы и ответы проходят через все уровни.

В многоуровневой монолитной архитектуре, когда пользователь выполняет некоторое действие, пользовательский интерфейс инициирует запрос, и тот проходит через все уровни архитектуры. Если при этом требуется что-то сохранить в базе данных, то запрос проходит сверху вниз и обратно.

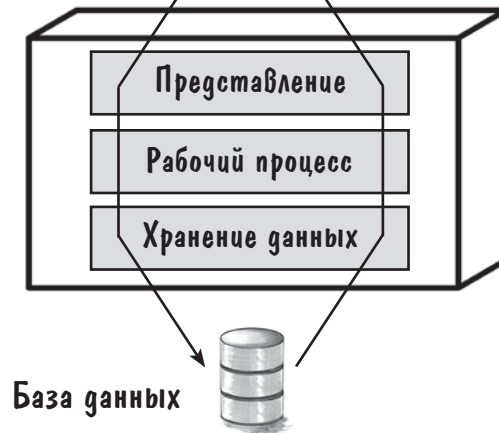
Пользователь посылает приложению запрос...

Запрос



Ответ

...и в какой-то момент получает ответ



Организация уровней

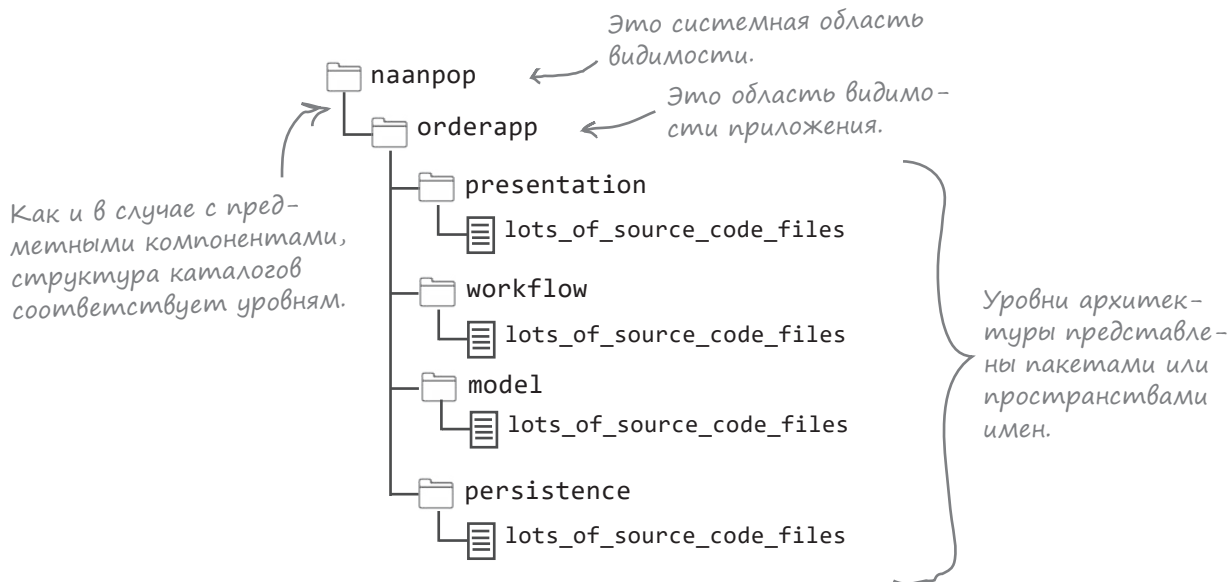
Для сайта Naan & Pop наша команда создаст логические компоненты, соответствующие задаче. Но как их реализовать?

Уровни в архитектурах этого типа создаются с помощью пакетов или пространств имен, как и предметные компоненты. Однако в целях разделения задач структура пакетов уровней обычно отражает их место в разделении:

```
com.naanpop.orderapp.presentation
com.naanpop.orderapp.workflow
com.naanpop.orderapp.model
com.naanpop.orderapp.persistence
```

← Имена уровней будут выглядеть как пакеты в Java, пространства имен в .NET или любые другие структурные единицы, используемые в вашем языке.

Как и логические компоненты, архитектурные уровни используют реализацию компонентов базовой платформы, часто связанную с лежащей в ее основе файловой системой:



Преобразование уровней в код

Когда команда создаст пакеты компонентов (или пространства имен), вам нужно будет помочь им реализовать архитектуру. Вот пример псевдокода на Python, иллюстрирующий преобразование уровней в код.

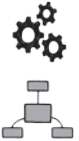


Уровень пользовательского интерфейса (UI), он же **уровень представления**, — самый верхний. Он отвечает за взаимодействие с пользователем и служит той же цели, что и *представление* в MVC.

```
def UI_layer(request):
    data = request.get_data()
    return business_logic_layer(data)
```

Получить запрос от пользователя.

Передать запрос уровню рабочего процесса



Уровень рабочего процесса (или **уровень бизнес-правил**) отвечает за обработку запросов, полученных от уровня пользовательского интерфейса, и возврат ответа.

```
def business_logic_layer(data):
    processed_data = process_data(data)
    return data_access_layer(processed_data)
```

Обработать данные из уровня пользовательского интерфейса.

Передать обработанные данные на уровень доступа к данным.



Уровень хранения данных (или **уровень доступа к данным**) отвечает за доступ к данным из БД и возврат их уровню рабочего процесса.

```
def data_access_layer(data):
    retrieved_data = retrieve_data(data)
    return retrieved_data
```

Доступ к данным в базе.

Вернуть полученные данные на уровень рабочего процесса.

часть Задаваемые Вопросы

В: В главе 5 говорилось, что все архитектурные стили имеют свою категорию и философию. Как классифицируется многоуровневая архитектура?

О: Мы рады, что вы задумались об этом. Как уже говорилось в главе 5, понимание категорий позволяет многое узнать о характеристиках того или иного архитектурного стиля.

Многоуровневая архитектура основывается на *техническом разделении* и обычно развертывается как *монолит*. (Мы говорим «обычно», поскольку есть вариации этой модели, которые мы обсудим чуть позже.)



Постойте-ка! Мы провели анализ логических компонентов и определили поведение приложения. Где эти компоненты находятся на уровнях нашей архитектуры?

Это важный момент — в данной архитектуре предметное поведение *проходит* через все уровни.

Предметная область, как вы помните, представлена логическими компонентами, решающими бизнес-задачи. Но уровни в этой архитектуре определяют *техническое разделение* — пользовательский интерфейс, бизнес-логику и т. д.

При отображении в многоуровневую архитектуру предметная область часто пересекает границы уровней.

Часто Задаваемые Вопросы

В: Почему выбрано именно такое деление — представление, рабочий процесс и хранение данных?

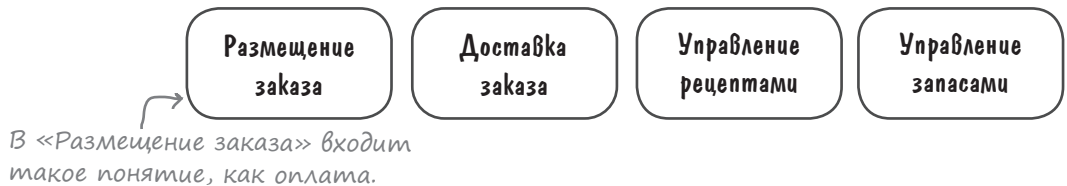
О: Это типичное деление, но не единственное. Оно, хотя бы частично, встречается во многих приложениях, например, пользовательский интерфейс часто отделяется от бизнес-логики, которая, в свою очередь, отделяется от базы данных.

В: Можно ли сказать, что основой для многоуровневой архитектуры послужил паттерн проектирования MVC?

О: Скорее наоборот. Это многоуровневые архитектуры, появившиеся, когда люди начали создавать ПО из отдельных частей, могли стать основой для паттерна проектирования. Паттерны часто создаются на основе наблюдений за распространенными явлениями, а многоуровневая архитектура существует уже довольно давно.

Предметные области, компоненты и уровни

Для простой системы обработки заказов в ресторане Naan & Pop можно предложить следующие компоненты:



Но есть проблема. Эти компоненты основаны на предметной логике поведения, тогда как многоуровневая архитектура базируется на *техническом* делении. Нам нужно разделить логические компоненты (которые содержат рабочие процессы и сущности) на компоненты, соответствующие многоуровневой архитектуре:



Разделив логические компоненты на рабочие процессы и сущности, мы можем наложить компоненты на уровни в архитектуре:

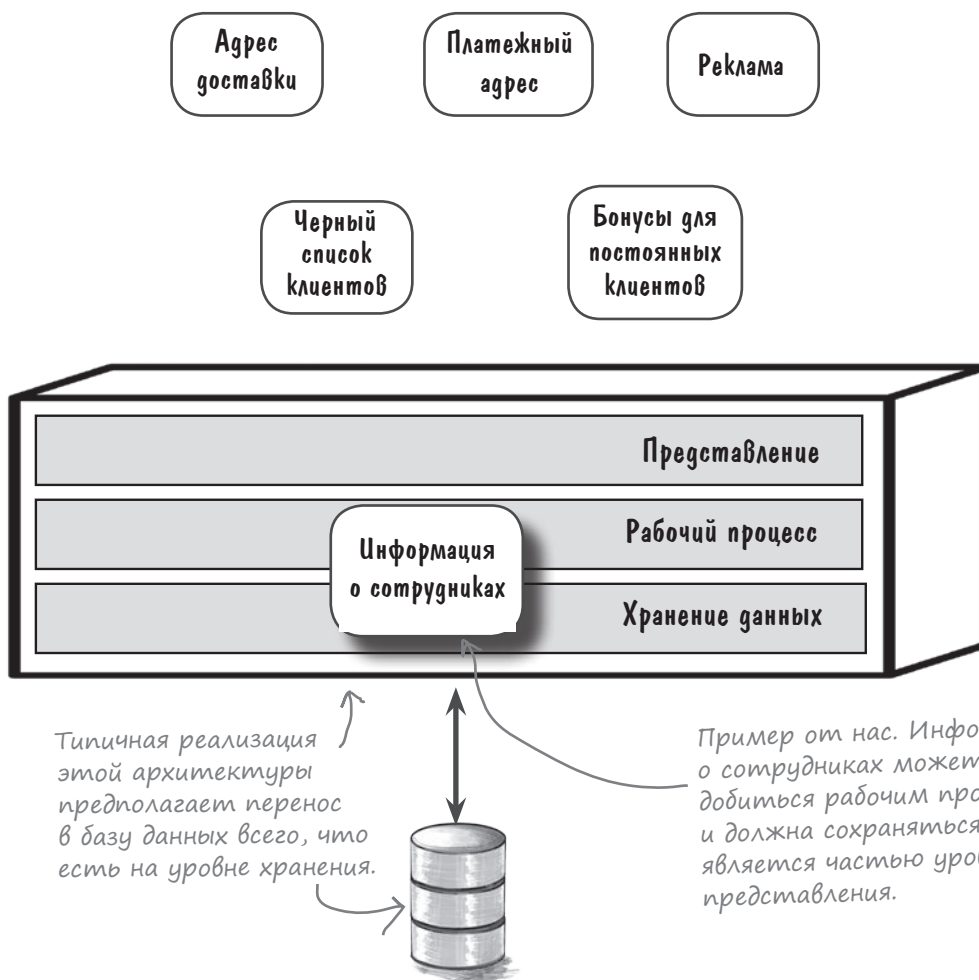


Возьмите в руку карандаш

Архитекторы Naan & Por разработали многоуровневую архитектуру. Но сегодня они отправились на семинар по хлебопечению, чтобы узнать больше о предметной области, а вам предстоит разобраться, какие компоненты куда относятся. Определите, на каком уровне(ях) должен находиться каждый компонент, и разместите их соответствующим образом.

Компоненты

Подсказка: некоторым компонентам могут соответствовать несколько уровней.



Типичная реализация этой архитектуры предполагает перенос в базу данных всего, что есть на уровне хранения.

Пример от нас. Информация о сотрудниках может понадобиться рабочим процессам и должна сохраняться, но не является частью уровня представления.

Решение на с. 242



Часть Задаваемые Вопросы

В: Зачем выявлять логические компоненты, если их все равно нужно разбивать на части, чтобы вписать в эту архитектуру?

О: Логические компоненты представляют решаемую задачу. Их отображение в архитектуру означает применение ограничений (и компромиссов) реального мира. В следующей главе мы покажем более прямой способ отображения предметной области в архитектуру, но и у него есть недостатки.

В: Почему многоуровневая архитектура так популярна?

О: Во-первых, она проста, в ней мало движущихся частей. Во-вторых, она тесно связана с паттерном MVC, поэтому ее легко понять. В-третьих, она настолько распространена, что команды могут быстро создавать простые проекты на ее основе. В-четвертых, многие компании делят сотрудников по уровню квалификации, что упрощает создание таких архитектур.

Достоинства многоуровневой архитектуры

Мы составили список ключевых достоинств многоуровневой архитектуры, которые могут подтолкнуть нас к выбору именно этого архитектурного стиля.



Специализация

Использование многоуровневой архитектуры позволяет организациям создавать команды специалистов и распределять их между разными проектами.

Благодаря возможности специализации эта архитектура популярна в организациях, где необходимо распределять специалистов между несколькими проектами.



Простота повторного использования

Деление по техническим признакам расширяет возможности повторного использования кода. Так, если весь код, отвечающий за хранение данных, находится на одном уровне, то его проще искать, обновлять и повторно использовать.

Пригодность компонентов внутри уровня к повторному использованию — одно из ключевых достоинств этой архитектуры.



Совпадение с физической структурой

В многоуровневой архитектуре логические компоненты разделены согласно физической структуре. Например, команды часто реализуют разные уровни с помощью разных технологических стеков (например, JavaScript, Java и MySQL).

Реальный мир редко позволяет архитекторам использовать то, что они хотят; чаще он ограничивает их тем, что имеется.



Концептуальный двойник MVC

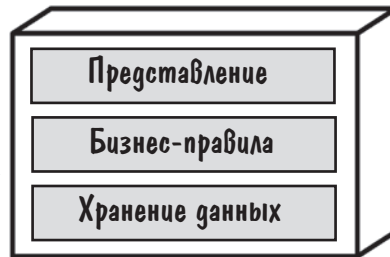
Простота реализации — важное достоинство многих архитектур. Разработчикам проще создавать архитектуры, соответствующие знакомым паттернам проектирования, таким как MVC.

Простота реализации — ценное свойство!

Уровни и реальный мир: физические архитектуры

Многоуровневый монолит описывает логическую архитектуру, но архитекторы могут реализовать ее в виде разных физических архитектур.

Пример: корпоративное приложение, работающее на настольных компьютерах и использующее общую базу данных.



Двухуровневая

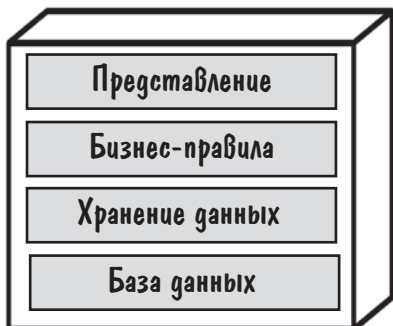
В двухуровневой архитектуре уровни представления, бизнес-правил и хранения помещаются в один модуль развертывания, взаимодействующий с базой данных через локальную сеть. Такая физическая архитектура характерна для настольных и клиент-серверных приложений.

Пример: крупномасштабное веб-приложение с богатым пользовательским интерфейсом, например сервис потокового видео.



Трёхуровневая

В трехуровневой архитектуре используются отдельные физические уровни как в веб-приложении, где роль среднего уровня играет сервер приложений, а уровень представления написан с помощью другого технологического стека. Например, команда разработчиков может написать бизнес-правила и систему хранения данных на Java, пользовательский интерфейс — на HTML и JavaScript, а для хранения данных использовать реляционную базу данных.



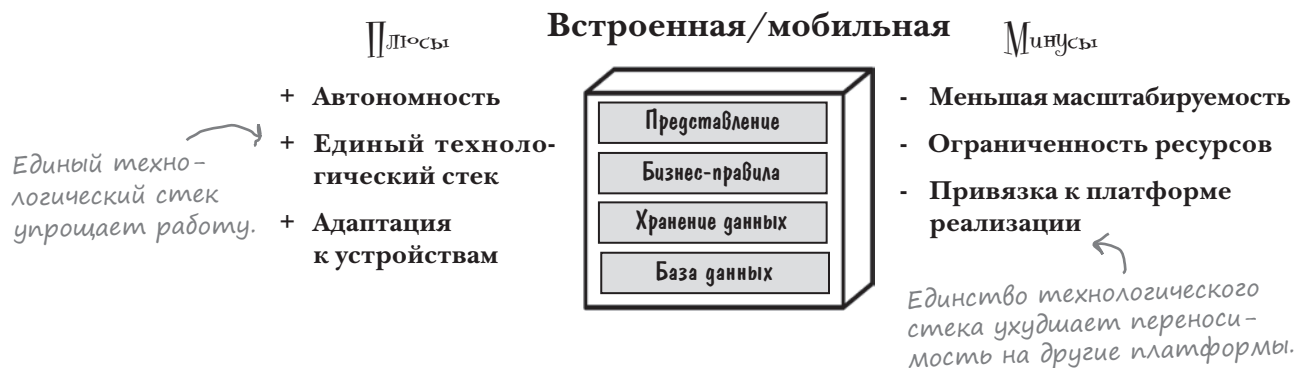
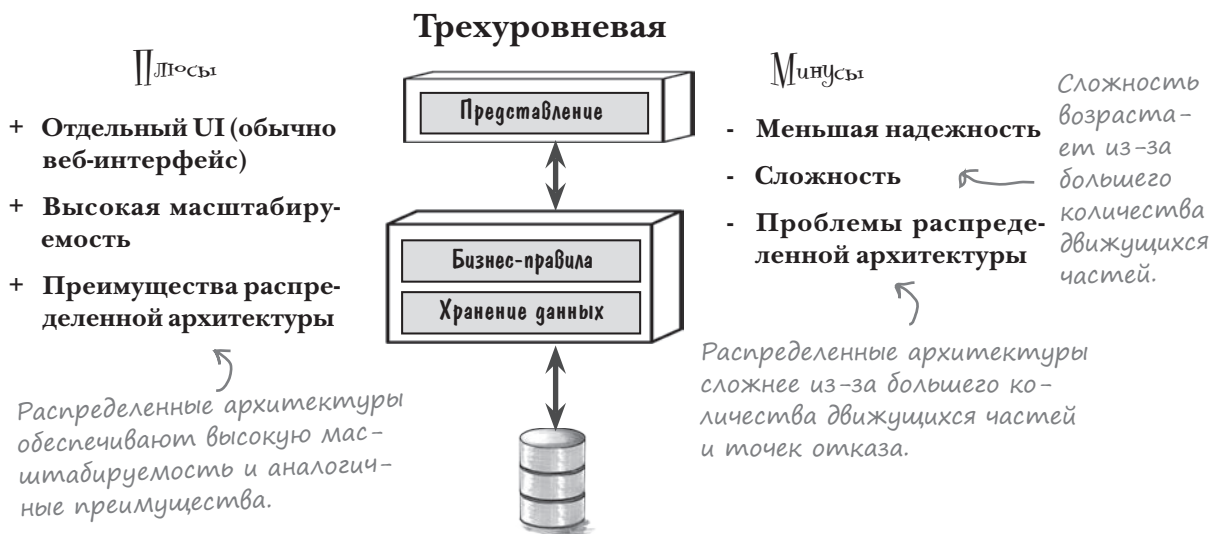
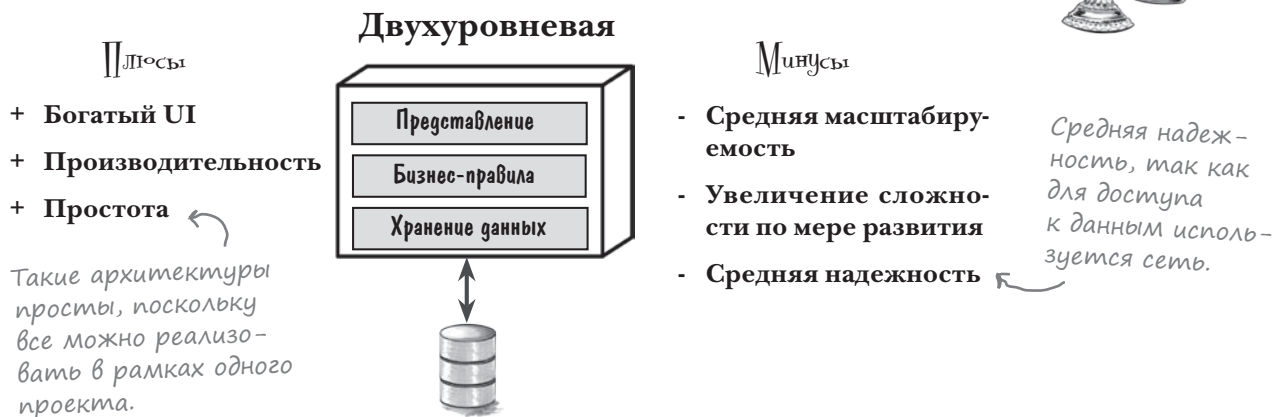
Встраиваемая/мобильная

Часто из-за физических ограничений все логические уровни оказываются в одном модуле развертывания. Такая физическая архитектура характерна для встраиваемых систем и мобильных приложений, где сетевое подключение может быть непостоянным или вообще отсутствовать.

Примеры: мобильная игра или ПО для автомата по продаже газировки

Компромиссы физической архитектуры

Какую физическую архитектуру выбрать? Как и все в программной архитектуре, они имеют свои компромиссы.





Упражнение

Компромиссы компромиссами, но архитектура ПО всегда основывается на реальной системе, и архитекторам Naan & Por нужно оценить физическую архитектуру и сделать выбор. Помогите им определить, с какими компромиссами столкнется приложение Naan & Por в каждом случае.

Двухуровневая

Плюсы

Простота

Мы поможем
вам начать.

Минусы

Масштабируемость

Трёхуровневая

Плюсы

Минусы

Встраиваемая/мобильная

Плюсы

Минусы

Решение на с. 243

Разговор в кабинете

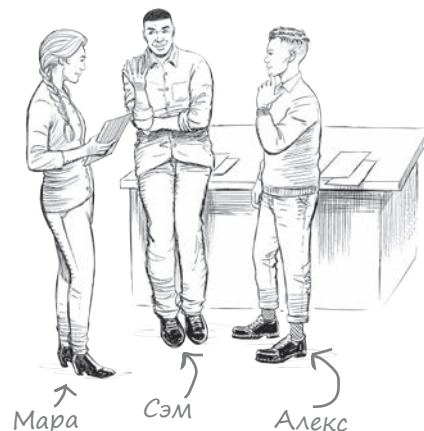
Алекс. Naan & Pop — типичное приложение, в котором будут использоваться только стандартные уровни? Может, надо добавить другие?

Сэм. Зачем добавлять уровни в архитектуру?

Мара. Каждый уровень в многоуровневой архитектуре наделен своими обязанностями, поэтому при необходимости мы добавляем новые уровни.

Сэм. И что это за уровни?

Алекс. Обычно добавляют *уровень сервисов* для интеграции бизнес-задач или *интеграционный уровень* для связи с другими системами. Каждый запрос проходит через все уровни, поэтому каждый уровень может что-то сделать с запросом.



Не скупитесь на уровни — их может быть столько, сколько нужно для обработки каждого запроса.

Мара. Верно. Архитекторы могут добавлять любые уровни для поддержки нового поведения. Например, мы можем добавить интеграционный уровень для связи со сторонними сервисами доставки. Давайте я нарисую на доске и покажу, что имею в виду...



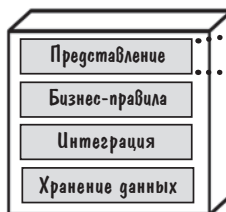
Интеграционные уровни позволяют взаимодействовать со сторонними сервисами доставки.

Алекс. Добавление уровня интеграции для связи с сервисами доставки упростит нашу работу, верно? Мы сможем поместить весь код интеграции в одно место и облегчить его поиск и обновление.

Мара. Это верно и для уровня UI. Кстати, согласно одному из требований мы должны реализовать дополнительный UI для мобильных устройств.

Сэм. Значит, мы сможем добавить мобильный UI, изменив всего один уровень?

Мара. Это одна из лучших особенностей многоуровневой архитектуры!



При добавлении нового UI другие уровни (и база данных) не должны меняться.



Уровень представления можно менять для поддержки новых типов UI.

Предостережение об изменениях в предметной области

Одно из главных преимуществ многоуровневой архитектуры — возможность объединять схожие технические компоненты. Например, в приложении Naan & Pop выделение UI в отдельный набор компонентов позволит команде добавлять новые типы UI, не затрагивая другие уровни.

Теперь поговорим об изменениях в предметной области. Придется ли менять *все уровни* в Naan & Pop, если, например, помимо сэндвичей понадобится добавить в меню еще и пиццу?

Возможность изменять все по отдельности — ценная особенность многоуровневой архитектуры, но она доступна не всегда. Главный ее недостаток в том, что *предметная область распределяется по всем уровням*. Например, логический компонент «Размещение заказа» в архитектуре Naan & Pop охватывает пользовательский интерфейс (представление), бизнес-логику (рабочий процесс) и схему данных (хранение данных).

Это означает, что *технические* компоненты легко изменить и расширить, но изменения в *предметной области* могут повлиять на все уровни.

Мы упоминали об этом в начале главы.

Многоуровневые архитектуры упрощают изменения в технической области, но затрудняют в предметной.

И как быть? В начале книги мы показали, как определить архитектурные характеристики, соответствующие приложению, и сделали это не просто так. Если ожидаются *постоянные и существенные изменения в предметной области*, то рассмотрите другие архитектурные стили.

Теперь кратко перечислим плюсы и минусы многоуровневой архитектуры.

Не волнуйтесь! В следующей главе вы познакомитесь с архитектурным стилем, который лучше приспособлен к изменениям в предметной области.

Часто задаваемые вопросы

В: Это довольно грозное предостережение. Может, лучше вообще отказаться от многоуровневой архитектуры?

О: Вспомните Первый закон архитектуры — все основано на компромиссах. Другие архитектуры могут облегчить реализацию изменений в предметной области, но тоже имеют недостатки. Сопоставьте плюсы и минусы каждого архитектурного стиля с вашими требованиями, а затем выбирайте. Нет единственно верного выбора, есть только тот, который лучше подходит для конкретного набора обстоятельств.

Сверхвозможности многоуровневой архитектуры

Многоуровневые архитектуры уже много лет демонстрируют свои мощные возможности — это один из старейших архитектурных стилей.

БАЦ!

Осуществимость

Если время и бюджет имеют первостепенное значение, то простота этой архитектуры будет весьма кстати.

Если ваша компания работает за счет инвестиций, то осуществимость особенно важна.



Бах!

Техническое разделение

Создание компонентов на основе технических возможностей упрощает их повторное использование. Например, если нескольким командам нужна одна и та же функциональность, то они могут реализовать ее один раз, а затем использовать совместно.

Обработка данных

Системы, выполняющие много операций с данными, могут выиграть от использования многоуровневой архитектуры, поскольку она изолирует обработку данных в единой оптимизированной базе данных.

Чем реже система обращается к данным по сети, тем выше ее эффективность.

БУМ!

Производительность

Хорошо спроектированные многоуровневые монолиты могут показывать высокую производительность — отсутствие дробления на сервисы и обработка данных в одном месте (монолитная база данных) исключают затраты на сетевые вызовы, способные ухудшить производительность.

Скорость разработки

Простота и единый модуль развертывания позволяют командам быстро создавать небольшие системы.

Компактность и эффективность

Небольшие размеры систем позволяют не обращать внимания на их слабые стороны, о которых пойдет речь на следующей странице.



Слабые стороны многоуровневой архитектуры

Данная архитектура очень популярна, поэтому есть риск использовать ее не к месту и даже во вред. Многие команды выбирают эту архитектуру из-за ее простоты, долгой истории и широкого распространения, не задумываясь о том, действительно ли она — самый подходящий вариант.



Масштабируемость

Если у вас только одно ведро и вы продолжаете лить в него воду, то в конце концов оно переполнится. То же можно сказать о монолитах: в какой-то момент неизбежно встанет проблема ограниченности ресурсов (памяти или чего-то еще).

Эластичность

Одному процессу сложнее справиться с внезапным наплывом пользователей.



Развертываемость

По мере увеличения размеров монолитной системы развертывать ее становится сложнее, особенно когда она продолжает развиваться.

Тестируемость

Высокая связанность и обширная кодовая база со временем значительно усложняют тестирование.

Большой комок грязи

Вся эта архитектура пронизана связями между компонентами, поэтому без тщательного управления может превратиться в большой комок грязи.



Звездные рейтинги многоуровневой архитектуры

Архитекторы Naan & Por решили использовать оценочную таблицу из книги «Фундаментальный подход к программной архитектуре»*, написанной двумя из авторов этой книги. Одна звезда означает, что архитектурная характеристика поддерживается плохо, а пять — что она поддерживается очень хорошо.

В точности как
в рецензиях к фильмам.

Тестирование —
сложный процесс,
но команда так
долго работает
с многоуровневыми
архитектурами,
что накопила не-
малый опыт.

Многоуровневые
архитектуры
устроены очень
просто.

Монолиты вообще
неэластичны и плохо
масштабируются,
а многоуровневые —
тем более.

Продуманные мно-
гоуровневые архи-
тектуры имеют
высокую произво-
дительность.

Простота архи-
тектуры делает
ее доступной.

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★
Тестируемость	★ ★
Развертываемость	★
Простота реализации	★ ★ ★ ★ ★
Развиваемость	★
Производительность	★ ★ ★
Масштабируемость	★
Эластичность	★
Отказоустойчивость	★
Общая стоимость	\$

*Ричардс М., Форд Н. Фундаментальный подход к программной архитектуре: паттерны, свойства, проверенные методы. — СПб.: Питер, 2024.



Упражнение

Для каких из следующих систем подойдет архитектура многоуровневого монолита и почему? Подсказка: учитывайте сверхвозможности и слабости архитектуры и характер системы.

Онлайн-аукцион, где пользователи делают ставки

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

→ Решение на с. 244

В заключение

Поздравляем! Команда Naan & Pop рассмотрела несколько вариантов и, учитывая приоритеты бизнеса, выбрала многоуровневую архитектуру. Это решение окупилось с лихвой, позволив бизнесу расти без проблем.

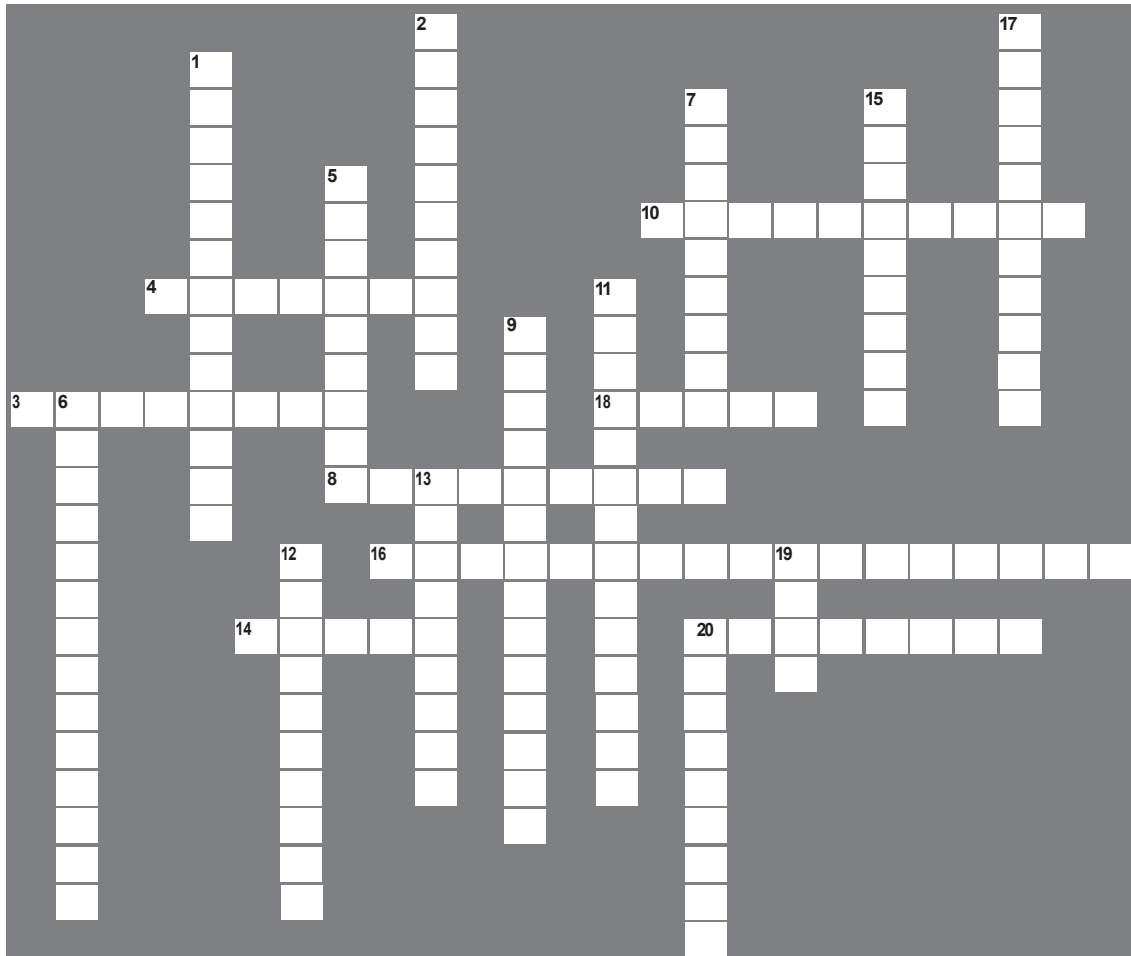
Ключевые моменты

- Многоуровневая архитектура — *монолитна*: вся система (код и база данных) развертывается как одно целое.
- Уровни выделяются по техническим признакам. К типичным уровням в этой архитектуре относятся: *представление* (пользовательский интерфейс), *бизнес-правила* (рабочий процесс и прикладная логика) и *хранение данных* (поддержка базы данных для систем, нуждающихся в долгосрочном хранении данных).
- Многоуровневая архитектура легко осуществима, проста, понятна и позволяет быстро создавать несложные системы.
- Многоуровневая архитектура прекрасно разделяет технические задачи, позволяя легко добавлять новые возможности, такие как пользовательский интерфейс или база данных.
- Многоуровневая архитектура обладает некоторыми особенностями паттерна MVC, но отображается на физические уровни и подвержена ограничениям реального мира.
- Пользовательские запросы проходят через UI и все остальные уровни, и только после этого ответ возвращается пользователю.
- Каждый запрос в этой архитектуре проходит через все уровни.
- Постоянно расширяя возможности многоуровневой архитектуры, можно столкнуться с нехваткой ресурсов (например, памяти).
- Многоуровневая архитектура хорошо поддерживает специализацию разработчиков (дизайнеры пользовательского интерфейса, программисты, администраторы баз данных и т. д.).
- Логические компоненты представляют предметную область, а уровни — технические задачи, что требует их согласования.
- Многоуровневые архитектуры могут воплощаться в нескольких физических архитектурах, в том числе в *двухуровневой* (клиент/сервер), *трехуровневой* (веб) и *встраиваемой/мобильной*.
- Технические возможности, представленные уровнями, легко расширяются; многоуровневая архитектура способствует этому.
- Изменение предметной области требует координации на всех уровнях, что вызывает дополнительные сложности.



Кроссворд о Многоуровневой архитектуре

Готовы обрести новые знания, разгадав этот кроссворд?



По горизонтали

3. Уровень, отображающий объектные модели в реляционные модели баз данных.
4. Многоуровневые архитектуры похожи на _____.
8. Пространства имен и пакеты повторяют _____ каталогов.
10. Логические и _____ компоненты часто разделяются одинаково.
14. Сетевой доступ к данным делает архитектуру _____ надежной.
16. Уровень интеграции позволяет _____ с другими системами.
18. Сильная связанность влечет образование большого компонента _____.
20. Запрос пользователя и ответ _____ через все уровни.

По вертикали

1. Паттерн проектирования «Модель — _____ — Контроллер».
2. Логические _____ располагаются на уровнях.
5. Пользовательский _____ — часть уровня представления.
6. Уровень, применяющий бизнес-правила (два слова слитно).
7. Многоуровневая архитектура для смартфонов.
9. Предметно-ориентированное _____.
11. Тип архитектуры, рассматриваемый в этой главе.
12. _____ область охватывает все уровни.
13. Паттерн MVC и многоуровневая архитектура _____ обязанности.
15. _____ могут ограничиваться одним уровнем.
17. Тип БД, часто используемый для хранения данных.
19. Многоуровневая архитектура может иметь два и более _____.
20. Многоуровневая архитектура облегчает _____ использование.

→ Решение на с. 245

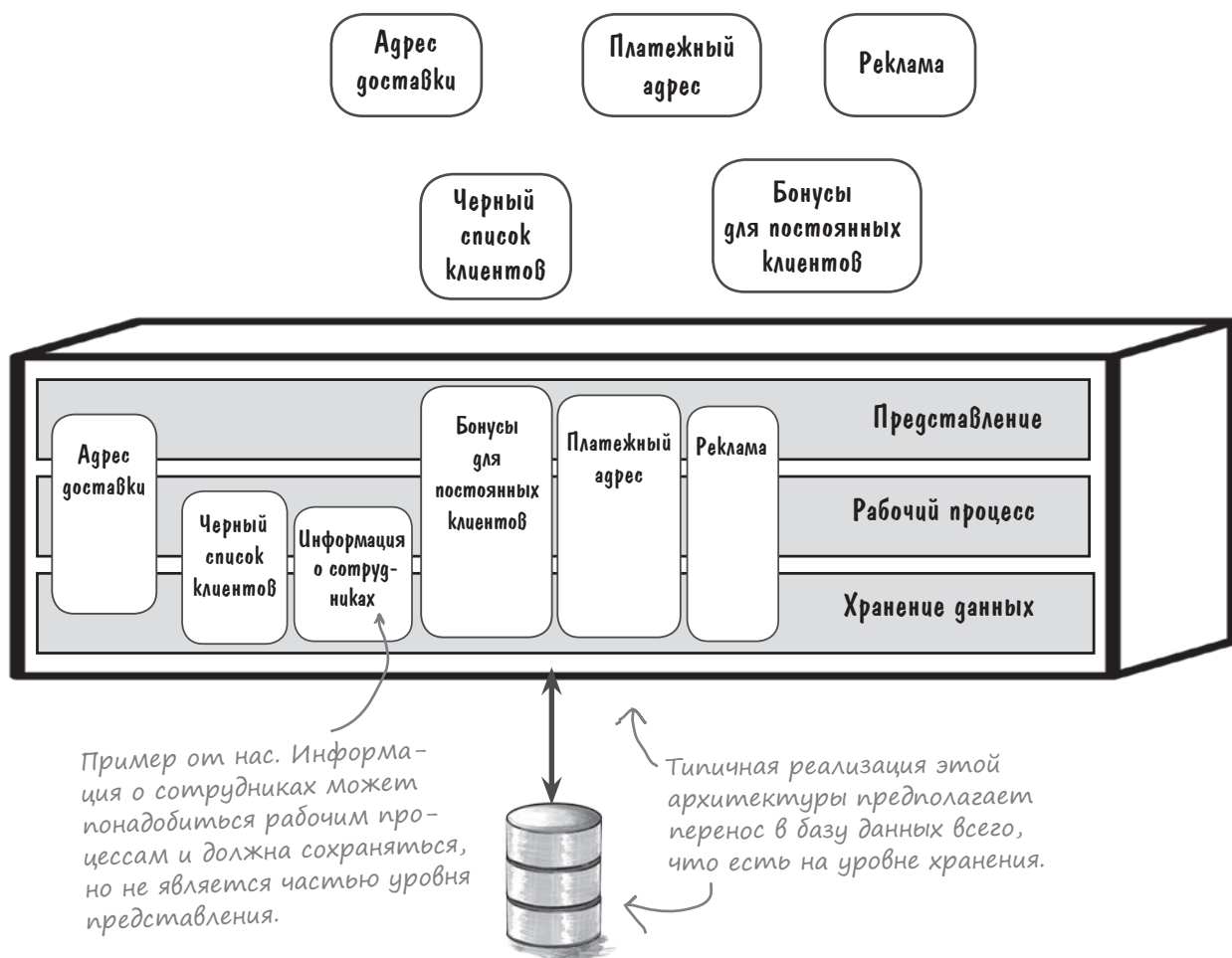


Возьмите в руку карандаш Решение

Архитекторы Naan & Por разработали многоуровневую архитектуру. Но сегодня они отправились на семинар по хлебопечению, чтобы узнать больше о предметной области, а вам предстоит разобраться, какие компоненты куда относятся. Определите, на каком уровне(ях) должен находиться каждый компонент, и разместите их соответствующим образом.

Компоненты

Подсказка: некоторым компонентам могут соответствовать несколько уровней.





Упражнение Решение

На с. 233

Компромиссы компромиссами, но архитектура ПО всегда основывается на реальной системе, и архитекторам Naan & Por нужно оценить физическую архитектуру и сделать выбор. Помогите им определить, с какими компромиссами столкнется приложение Naan & Por в каждом случае.

Двухуровневая

Плюсы

Простота сокращает время выхода на рынок.

Минусы

Чем меньше уровней, тем хуже масштабируемость.

Простота

Производительность

Обычно чем меньше уровней, тем выше производительность.

Масштабируемость

Разделение поддерживает одну из целей.

Плюсы

Архитектурное разделение

Масштабируемость

Улучшенная масштабируемость обеспечит поддержку большего количества пользователей.

Трёхуровневая

Чем больше уровней, тем больше частей в архитектуре и тем сложнее зависимости и взаимодействия...

Минусы

Сложность

замедлению выхода на рынок.

...приводит к...

Встраиваемая/мобильная

Плюсы

Чистый монолит имеет меньше частей.

Автономность

Минусы

Расширить встраиваемую систему сложно, что противоречит одной из целей.

Ухудшенная расширяемость

Трудно создать автономный монолит, обладающий высокой масштабируемостью и/или производительностью.

Масштабируемость

Производительность



Упражнение Решение

На с. 239

Для каких из следующих систем подойдет архитектура многоуровневого монолита и почему?

Подсказка: учитывайте сверхспособности и слабости архитектуры и характер системы.

Онлайн-аукцион, где пользователи делают ставки

Почему? Онлайн-аукцион требует большей масштабируемости и производительности, чем могут дать многоуровневые архитектуры.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? Эта система требует высокой пропускной способности и доступности, что сложно для многоуровневых монолитов.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? Многоуровневые архитектуры разделяют задачи по техническим возможностям, упрощая некоторые изменения.

- ☐ Хорошо подходит
☒ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? У маленькой пекарни простая задача и не-большой масштаб, хорошо подходящие для воплощения в виде простой архитектуры.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

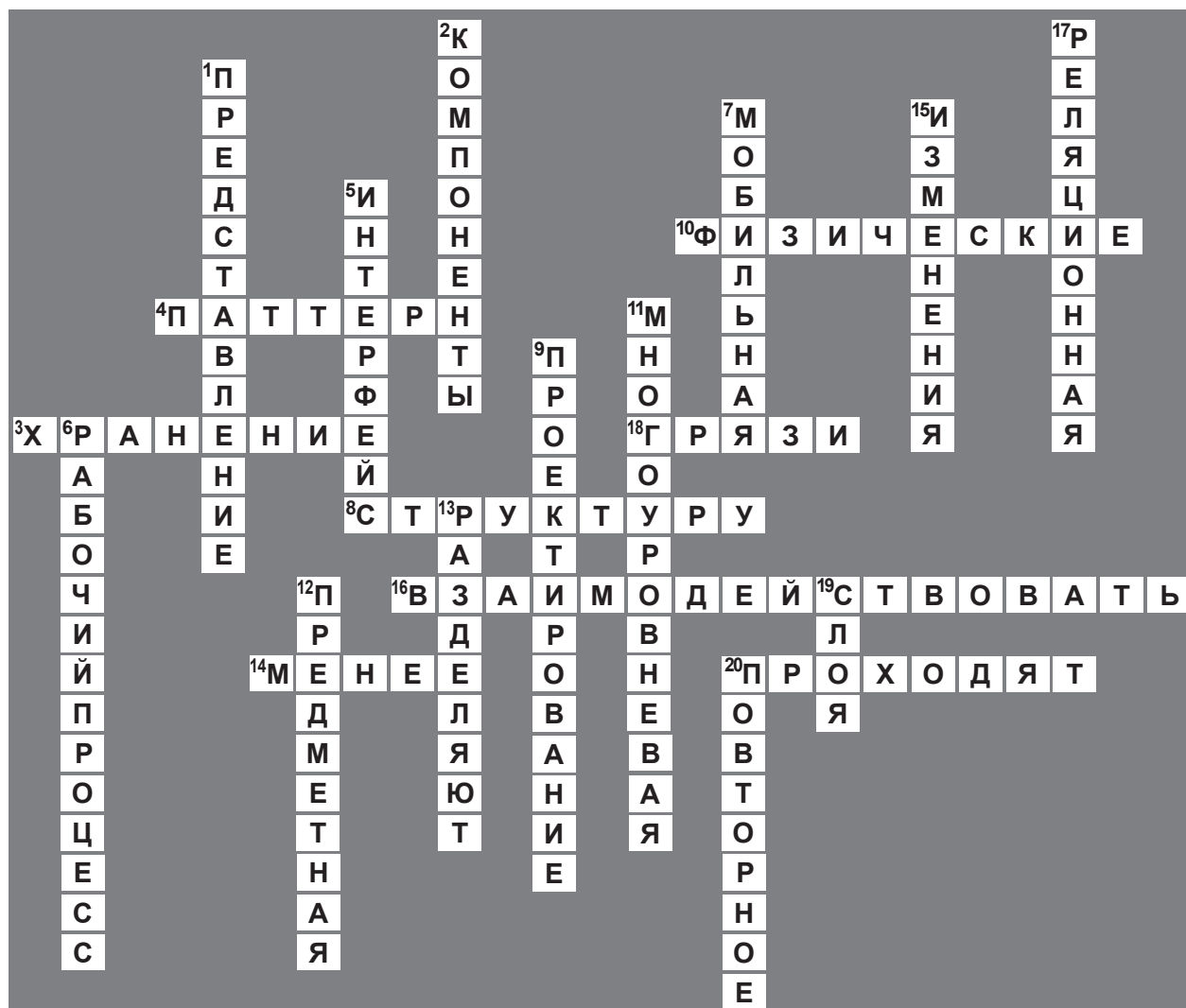
Почему? Система заявок должна поддерживать разные архитектурные характеристики для клиентов и технических специалистов. Эти характеристики сложно реализовать в монолитной архитектуре.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит



Кроссворд о Многоуровневой архитектуре. Решение

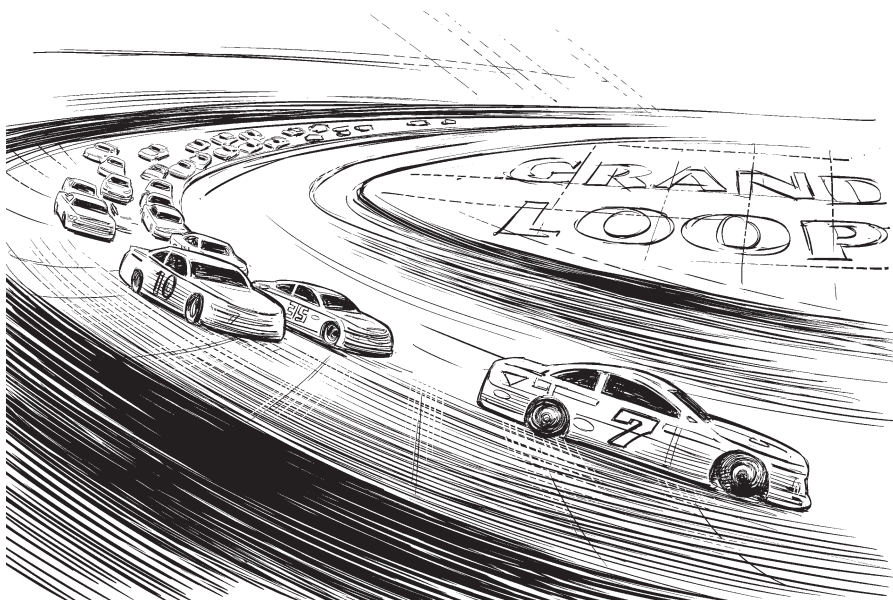
На с. 241



1

Модульные Монолиты

Управляются предметной областью



Создать монолит можно несколькими способами. Вы уже познакомились с многоуровневой архитектурой, увязывающей компоненты *технически*. Можно долго развивать многоуровневый монолит, но когда возникает необходимость координировать взаимодействие разных команд, то может потребоваться нечто большее, возможно, даже другой архитектурный стиль.

В этой главе описывается архитектурный стиль *модульный монолит*, в котором приложения разделены по *бизнес-задачам*, а не по техническим признакам. Вы узнаете, что это значит и на что следует обратить внимание. Кроме того, мы поговорим обо всех компромиссах этого стиля.



Наконец, после многочисленных просьб мы решили добавить в наше меню пиццу! Но прежде я хочу поговорить с командой разработчиков...

Помните Сангиту, основательницу Naan & Pop и начинающего менеджера проектов из главы 6?

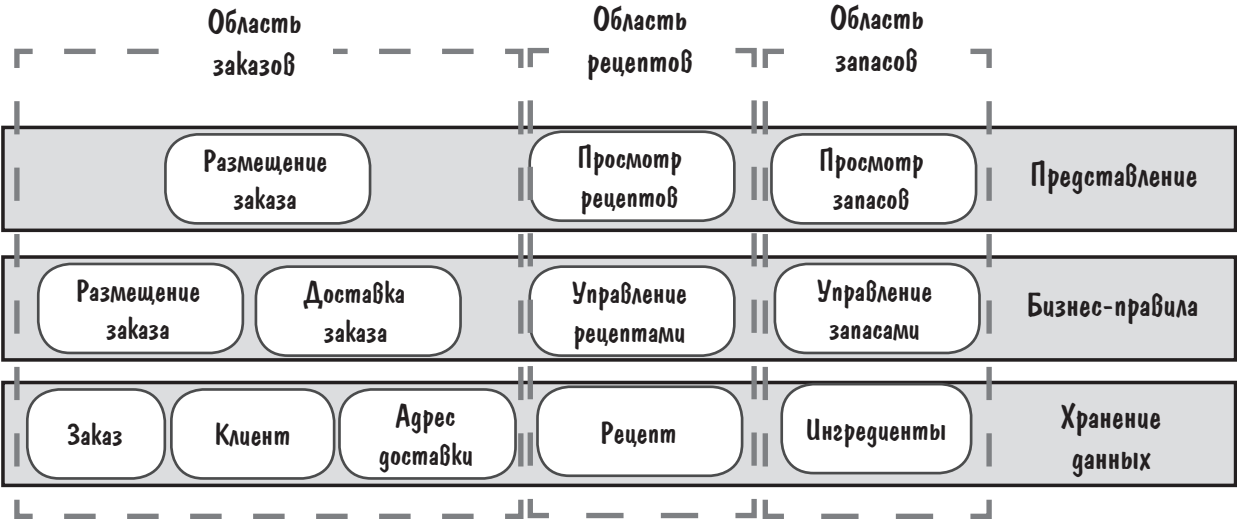
Напомним, что в магазине сэндвичей Naan & Pop работает небольшая команда разработчиков, и ее требования мало изменились после создания многоуровневого приложения в главе 6. Но жесткая конкуренция и скорость выхода на рынок остаются проблемой. Система должна оставаться простой.

Перечитайте главу 6, если вам нужно освежить память.



Упражнение

Вернемся к многоуровневой архитектуре, созданной для Naan & Pop в главе 6. Ниже показаны ее уровни и логические компоненты. Добавление нового пункта в меню потребует изменить многие из них. Возьмите маркер и нарисуйте треугольник (▲) на компонентах, которые затронет это новое требование.



Запишите здесь свои мысли.

Решение на с. 272

Разговор в кабинете



Алекс. Пф-ф, *легко!* У нас уже есть работающая и расширяемая система. Приступим к делу.

Мара. Не спеши. Это наша первая встреча с подобными изменениями, но, я думаю, не последняя.

Сэм. И что? У нас есть опыт создания расширяемой системы, так зачем осторожничать?

Мара. Давайте поразмыслим: чтобы добавить в меню пиццу, нам придется не только добавить новые рецепты и ингредиенты, но и позволить посетителям заказывать пиццу онлайн. Итак, что мы должны изменить?

Сэм. Много чего! По меньшей мере все, что связано с заказами и рецептами. Но я не вижу причин для беспокойства.

Алекс. Кажется, я вижу. Мы построили многоуровневую архитектуру, над каждым уровнем которой работают специалисты. Подобное изменение потребует координации действий между *всеми* ними.

Мара. Верно! Многоуровневая архитектура *распределяет* предметную область по всем уровням, поэтому реализация изменений в предметной области может оказаться сложной задачей.

Алекс. Ты хочешь сказать, что выбор многоуровневой архитектуры был ошибкой?

Мара. Многоуровневую архитектуру было просто создать, и это позволило нам быстро сделать приложение. Но теперь нужно подумать о *совершенствовании* архитектуры и о поддержке модульности, чтобы в будущем было проще вносить подобные изменения.

Алекс. Итак, с чего начнем?

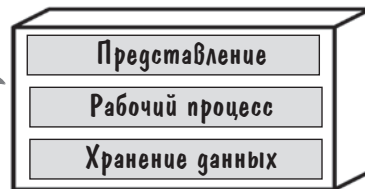
Мара. Со знакомства с модульным монолитом! Теперь мы будем делить систему не по техническим, а по *предметным признакам*, используя модули — отсюда и слово «модульный». По ходу объяснения я покажу, как это выглядит.

Сэм. Я в нетерпении. Давайте приступим!

Модульный монолит

Модульный монолит, как и многоуровневая архитектура, разворачивается как одно целое, обычно с собственной базой данных.

Многоуровневая архитектура следует монолитной модели разворачивания...



...и основывается на монолитной базе данных.

Как рассказывалось в главе 5, архитектурные стили можно разделить по способу деления кода на части и по модели разворачивания.

На этом сходство заканчивается. Приложение с архитектурой модульного монолита делится не по технической, а по предметной *функциональности*. Любой бизнес связан с конкретной предметной сферой: банковской, образовательной или торговой. Интернет-магазины обычно имеют несколько *подобластей*, таких как размещение заказа, оплата и управление запасами. Все вместе они образуют область «интернет-магазин». Вы организуете свое приложение в соответствии с этими подобластями, размещая их в модулях.

Обратите внимание: термины «область» и «подобласть» используются как взаимозаменяемые. Оба обозначают бизнес-задачи.

Модульный монолит — это тоже монолитная модель разворачивания.



Приложение организуется не по техническим, а по предметным задачам.

Модульные монолиты тоже используют монолитную базу данных.



Что такое *модуль*? В общем случае это единица организации кода. В некоторых языках модули могут называться пакетами или пространствами имен.

Деление кода на модули влияет на распределение задач между модулями и на порядок их взаимодействия. Далее мы подробно расскажем о множестве нюансов, а пока хотели бы научить вас отличать многоуровневую архитектуру от модульного монолита.

Часто
Задаваемые
Вопросы

В: Объясните подробнее, что вы имеете в виду под словом «модуль»?

О: Модуль — это независимая единица в структуре программного обеспечения, реализующая одну часть функциональности. Уровни в многоуровневой архитектуре тоже являются модулями, просто эти модули разделены по техническим задачам.

В модульном монолите, напротив, каждый модуль представляет часть предметной области — *подобласть* — и содержит все необходимые для нее бизнес-функции.

Что чему соответствует?

Часто бывает непросто определить взаимосвязи между компонентами и модулями. В этой игре «Что чему соответствует?» вы должны связать каждый компонент с наиболее подходящим ему модулем. Некоторые компоненты могут быть не связаны ни с каким модулем.

Корзина покупателя

Форма ввода кредитной карты

Адрес электронной почты

Процесс выполнения заказа

История заказов

Заказ

Оплата

Клиент

→ Решение на с. 273

Изменения в предметной области болезни

В Naan & Pop решили добавить новую категорию блюд (пиццу). Какие элементы архитектуры нужно для этого изменить? Что ж, посмотрим.

▲ Изменившиеся компоненты

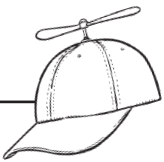


Добавление новой категории затрагивает по меньшей мере область заказов. Меню должно позволять заказывать пиццу и выбирать начинку. Для этого может потребоваться изменить или добавить бизнес-правила для поддержки персонализации и, возможно, доставки (пицца должна доставляться горячей), а также способы хранения данных о заказах (чтобы сохранять настройки).

Как видите, добавление новых пунктов меню может затронуть множество компонентов в разных уровнях, поскольку, как отмечалось в предыдущей главе, в архитектуре с техническим разделением предметная область распределяется по нескольким уровням. Технические изменения, такие как смена технологии представления или базы данных, не вызывают больших проблем. Но если изменение затрагивает предметную область, то вам придется собрать людей из нескольких команд, чтобы понять, как его реализовать.

Именно в такой ситуации могут помочь модульные монолиты.

Едва ли кто-то когда-то просыпался с мыслью: «А не добавить ли мне в свой календарь еще одну встречу?»



Серьезное программирование

Нередко команды начинают с многоуровневой архитектуры, а затем, достигнув определенной стадии, преобразуют ее в модульный монолит.

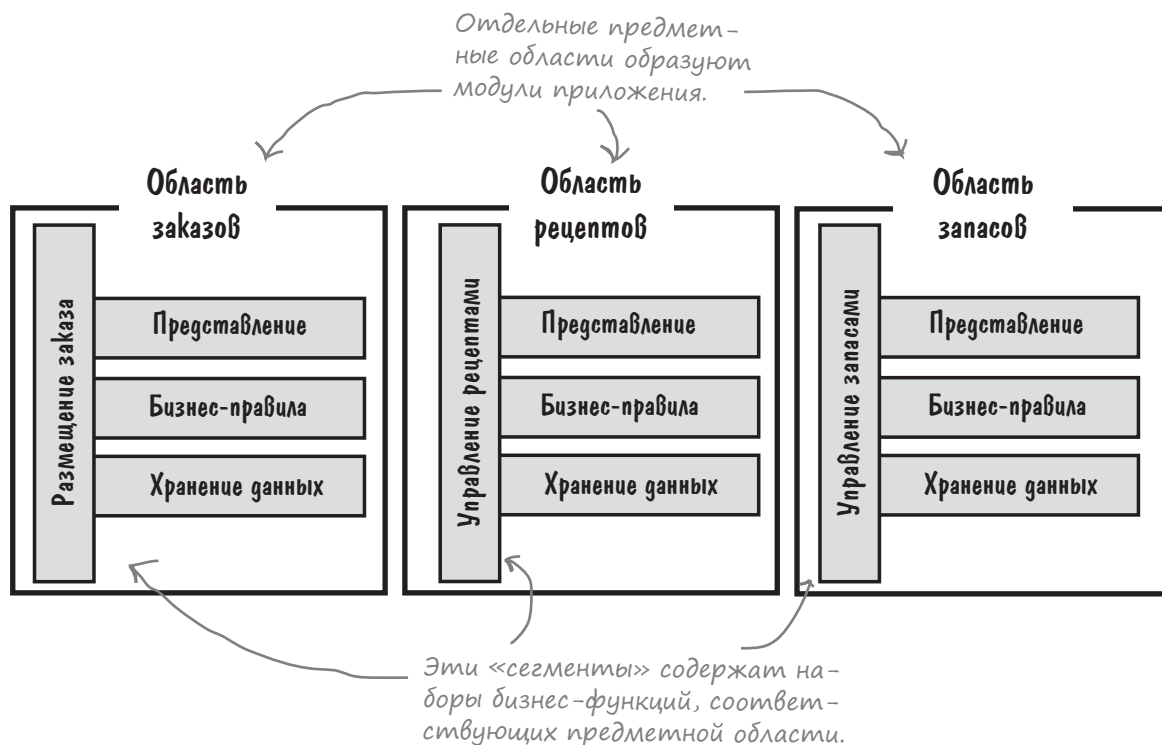
Что такое модульные монолиты

Когда вы едите бургер (обычный или вегетарианский), то сперва откусываете от верхней части булочки, затем кусочек сыра, а потом кусочек котлеты? Или делаете один укус, откусывая сразу все слои гамбургера? Последнее, верно?

Именно так можно представить модульный монолит. Приложение организуется не в виде горизонтальных уровней, решающих технические задачи, а в виде вертикальных сегментов. Каждый из них соответствует определенной предметной области и заключен в модуль, а тот, в свою очередь, содержит набор бизнес-функций. Например, модуль «Размещение заказа» будет содержать функции размещения, выполнения и доставка заказа.

Каждый слой в бургере служит своей цели, поэтому вкуснее пробовать все слои вместе.

Вкусно!



Что это значит? Изменения в предметной области, затрагивающие несколько уровней, требуют координации действий разных команд. Вам нужно убедиться, что изменения не привели к нарушению работы чего-либо.

Модульные монолиты создают **кросс-функциональные** команды, специализирующиеся на *предметных областях*, а не на представлении или хранении данных. Благодаря этому координация предметных изменений упрощается, поскольку за реализацию каждого из них отвечает одна команда.

На словах все просто! Но на деле не всегда легко создать кросс-функциональные команды, работающие с разными технологическими стеками.



Постойте-ка! Я все еще вижу уровни в каждом модуле. Чем эта архитектура отличается от многоуровневой?

А вы наблюдательны! Вы совершенно правы, каждый модуль по-прежнему состоит из уровней, но это не обязательно. Важно, что приложение организовано по предметным областям.

Но система все равно должна обрабатывать запросы и возвращать ответы. Поэтому вам понадобятся точка входа (уровень представления), некая бизнес-логика (уровень рабочего процесса) и возможность сохранять данные (уровень хранения). Даже в модульной архитектуре имеет смысл разделить эти обязанности по аналогии с многоуровневой архитектурой.

Однако, посмотрев на архитектуру более широко, можно увидеть, что приложение разбито на предметные области. Организация каждого модуля в виде множества технических уровней становится деталью реализации, а не архитектурной особенностью. Другими словами, *деление архитектуры не зависит от внутреннего устройства модулей*. Модульные монолиты разбиты на предметные области.

Теперь посмотрим, как это выглядит в коде.

Покажи мне код!

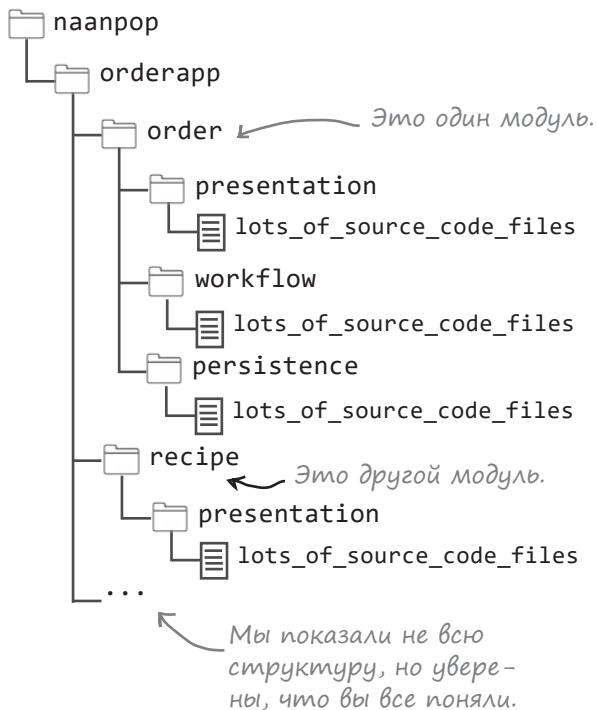
Вы, наверное, поняли, что модульные монолиты решают проблему иначе, чем многоуровневая архитектура, организуя приложения по предметным областям, а не по техническим задачам. Но как это отражается на коде?

Сначала поговорим о самих модулях. Они представляют части предметной области. Вот как выглядят пространства имен в Naan & Pop:

```
com.naanpop.orderapp.order
com.naanpop.orderapp.recipe
com.naanpop.orderapp.inventory
```

Вернитесь на с. 224 в предыдущей главе и сравните их с пространствами имен в многоуровневой архитектуре.

Не забывайте, что это все еще монолит, развертываемый как единое целое. Обычно такой код находится в одной кодовой базе и организован в нескольких пространствах имен. Каждое пространство представляет отдельный модуль, например:





Упражнение

Успешная разработка модульных монолитов требует хороших знаний предметной области. Это позволит разбить ее на отдельные модули. Один из способов сделать это (а их много) — прислушаться к мнению бизнес-экспертов.

Представьте, что вы работаете над приложением учета расходов для малого и среднего бизнеса. Бизнес-требования выглядят так:

- есть пользователи и аудиторы: пользователи добавляют расходы, а аудиторы проверяют отчеты на предмет соответствия правилам;
- когда пользователь добавляет расход, он записывается в базу данных от имени этого пользователя;
- приложение создает журнал аудита, проверив который аудиторы смогут убедиться, что все в порядке.

Определите подобласти, составляющие это приложение. *Подсказка: не все, что скажут бизнес-пользователи, можно преобразовать в модуль.*

→ II а с. 274

Разговор в кабинете (продолжение)



Алекс. Я понял. Разбивая приложение на модули, мы делаем код более модульным.

Сэм. Теоретически это решение выглядит прекрасно, но смогут ли все эти модули работать независимо друг от друга? Разве модулю заказов не нужно знать, что есть у нас на складе? А если вдруг закончатся грибы?

Мара. Вы оба правы. Разбив приложение по бизнес-задачам, мы делаем его более модульным. Тем не менее разным модулям может потребоваться взаимодействовать друг с другом.

Алекс. Но это же одна кодовая база! Я могу просто из модуля размещения заказов вызвать API модуля управления запасами, верно?

Сэм. Хм-м... Я уже вижу, как образуется большой комок грязи! Еще чуть-чуть, и каждый модуль будет взаимодействовать со всеми остальными, и тогда прощай наша модульность.

Мара. Ты прав. Если просто начать вызывать модули друг из друга, то вскоре границы между ними сотрутся и приложение превратится в большой комок грязи.

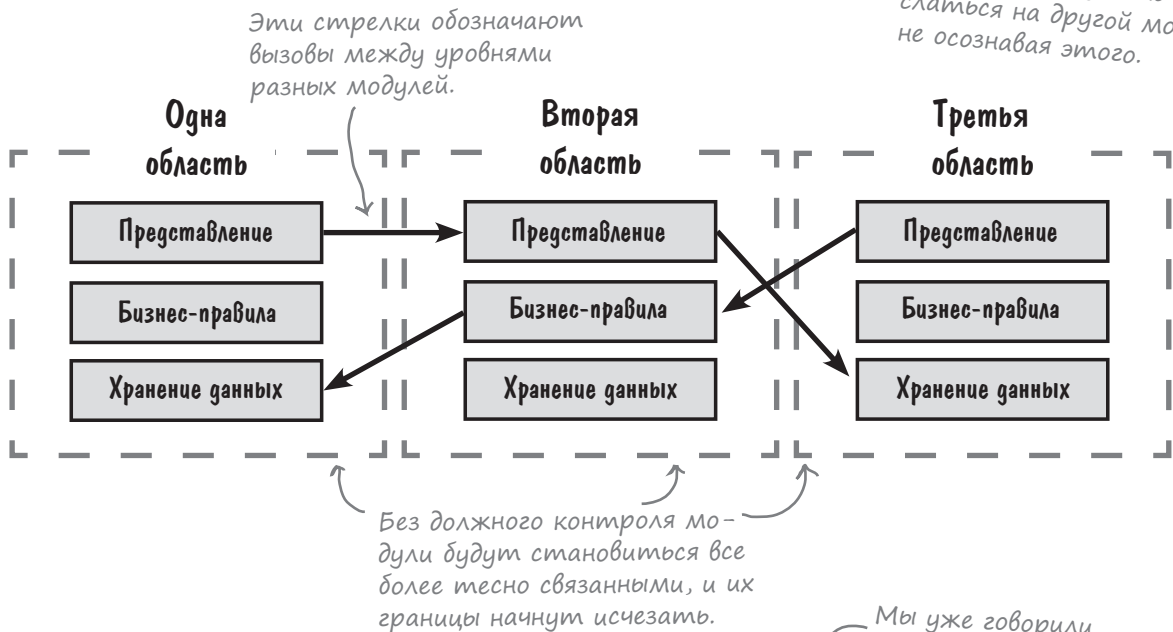
Алекс. Как же сохранить модульность и при этом дать модулям возможность взаимодействовать друг с другом?

Мара. Я сейчас покажу.

Как сохранить модульность модулей

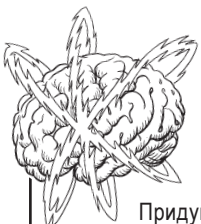
Модульные монолиты обычно хранятся в единой кодовой базе. Из-за этого человек, работающий над одним модулем, может нечаянно попасть в другой и тем самым связать их.

Функция автоимпорта в вашей IDE в этом случае вам не поможет: так как вы можете случайно сослаться на другой модуль, не осознавая этого.



Мы уже говорили об этом в главе 5.

Идея модульного монолита состоит в **разделении по предметным областям в рамках монолитной модели развертывания**. Ваша цель — создать слабо связанные модули, чтобы изменения в одном не влияли на другие. Но как избежать образования большого комка грязи? Об этом мы расскажем далее.



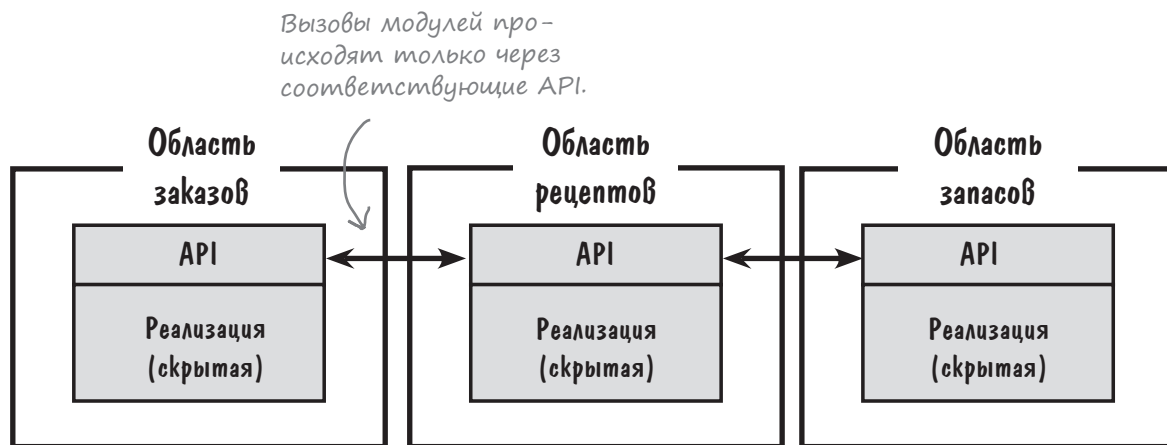
Мозговой штурм

Придумайте какие-нибудь механизмы, которые помогут гарантировать, что одна часть приложения не получит доступ к другой. Например, возможно, в вашем языке программирования есть поддержка разделения модулей во время компиляции? Запишите ниже все идеи, которые придут вам в голову. Некоторые из наших идей вы можете увидеть в конце главы.

→ Решение на с. 274

Как сохранить модульность модулей (продолжение)

С точки зрения организации кода *каждый модуль можно рассматривать как отдельный сервис*, даже при том, что все вместе они образуют единый монолитный комплекс. Каждый «сервис» предоставляет другим модулям открытый API, но скрывает свою внутреннюю реализацию.



Пока модули взаимодействуют *только* через открытые API, можно безопасно изменять их внутренние реализации благодаря слабой связанности.

Теоретически эта идея выглядит привлекательно. Но как на практике обеспечить сохранность границ модулей, чтобы спокойно спать по ночам? Рассмотрим несколько вариантов.

Часто задаваемые вопросы

В: Разделение приложения на модули выглядит хлопотным делом. Это в самом деле лучший вариант?

О: Мы уже говорили и повторим еще раз: нам не нравится слово «лучше», так как все определяется компромиссами. До сих пор мы подчеркивали (и продолжим делать это) преимущества модульных монолитов, но у них есть и недостатки.

Модульные монолиты требуют больших размышлений и дисциплины, а нередко и дополнительных инструментов (как будет показано ниже).

Но на другой чаше весов — более модульная архитектура, позволяющая кросс-функциональным командам работать независимо и, следовательно, быстрее решать поставленные задачи.

Как сохранить модульность модулей (завершение!)

Сохранить модульность модулей не так просто, как кажется, но не теряйте надежды. У вас есть технические средства и ваша творческая смекалка!



Некоторые языки имеют встроенную поддержку модулей. В Java изолированные модули можно создавать с помощью системы модулей — Java Platform Module System (JPMS). В .NET есть пространства имен и ключевое слово `internal`.

Gradle, инструмент сборки для Java, поддерживает подпроекты.

Другой подход — поместить каждый модуль в отдельную папку в репозитории. Такие **подпроекты** (или, как их называют многие системы сборки, **модульные проекты**) обеспечивают изоляцию просто потому, что являются разными проектами. Можно даже подумать о создании разных репозиториях для модулей, а затем сшивать приложение во время сборки.



Вы все так же разворачиваете монолит, поэтому вам, вероятно, придется объединить модули с помощью средств сборки. Но монолитная модель разворачивания не обязательно означает монолитную кодовую базу!



Сохранить модульность с ростом проекта могут помочь инструменты управления архитектурой, такие как ArchUnit для Java и ArchUnitNET для .NET.

Эти инструменты можно использовать по одному или сразу несколько вместе.

Для TypeScript и JavaScript есть ts-arch!

Часть Задаваемые Вопросы

В: Похоже, эти методы можно использовать при работе с многоуровневыми архитектурами. Верно?

О: Мы очень рады, что вы это поняли! Конечно, их можно использовать независимо от способа деления проекта — по техническим или предметным задачам. Применение инструментов (таких, как ArchUnit) и возможностей языка (таких, как JPMS) для сохранения границ модулей — отличная идея независимо от архитектурного стиля.

Вы подсказали несколько отличных идей об организации кода в модули по бизнес-задам. Но если продолжать использовать одну базу данных, не останутся ли модули связанными на ее уровне? Не пора ли подумать о модульной организации базы данных?



Это было бы логическим завершением внедрения модульной организации, верно? Деления кода на модули может быть недостаточно. Если данные по-прежнему переплетены между собой, то это означает, что вы просто перенесли комки грязи в базу данных!

Прежде чем двигаться дальше, заметим, что многие разработчики не привыкли мыслить в рамках бизнес-задач или разбивать код на отдельные модули. Деление базы данных на модули — отличная идея, но ее реализация может оказаться слишком сложным делом. Поэтому развивайте свою архитектуру постепенно, по мере необходимости, а не пытайтесь сделать все правильно с первого раза.

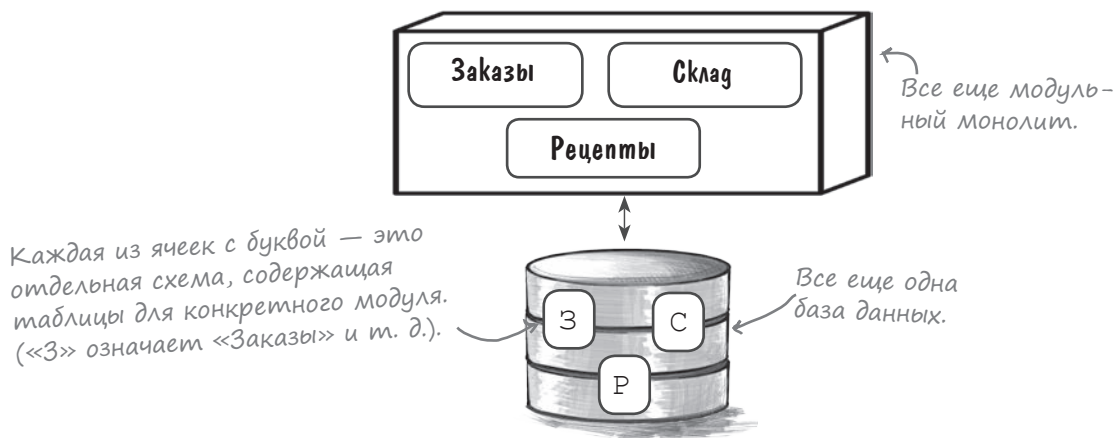
Примените на практике уроки, полученные в этой книге. Архитектурные характеристики побуждают выбрать модульный монолит? Тогда сначала разделите на модули код, а когда команда освоит модульное мышление, посмотрите, можно ли применить этот же подход к базе данных.

Далее посмотрим, как может выглядеть база данных с модульной организацией.

Модульность вплоть до базы данных

Модульный монолит все еще разворачивается вместе с монолитной базой данных как одно целое. Наличие единой базы данных *сильно* упрощает работу: не нужно беспокоиться о транзакциях или согласованности, и многим разработчикам удобнее работать только с одной базой данных. Однако, намереваясь поддерживать модульность на всех уровнях, вы должны подумать о модульной организации данных.

Правило простое: каждый модуль должен обращаться только к своим таблицам. Вот как это можно сделать для Naan & Pop:



Для каждого модуля в приложении нужно определить схему и набор таблиц. Все данные, относящиеся к некоему модулю, будут храниться только в таблицах этого модуля.

Как видите, модульность можно распространить на базу данных, разместив данные, принадлежащие разным модулям, в разных таблицах (и даже в разных схемах).



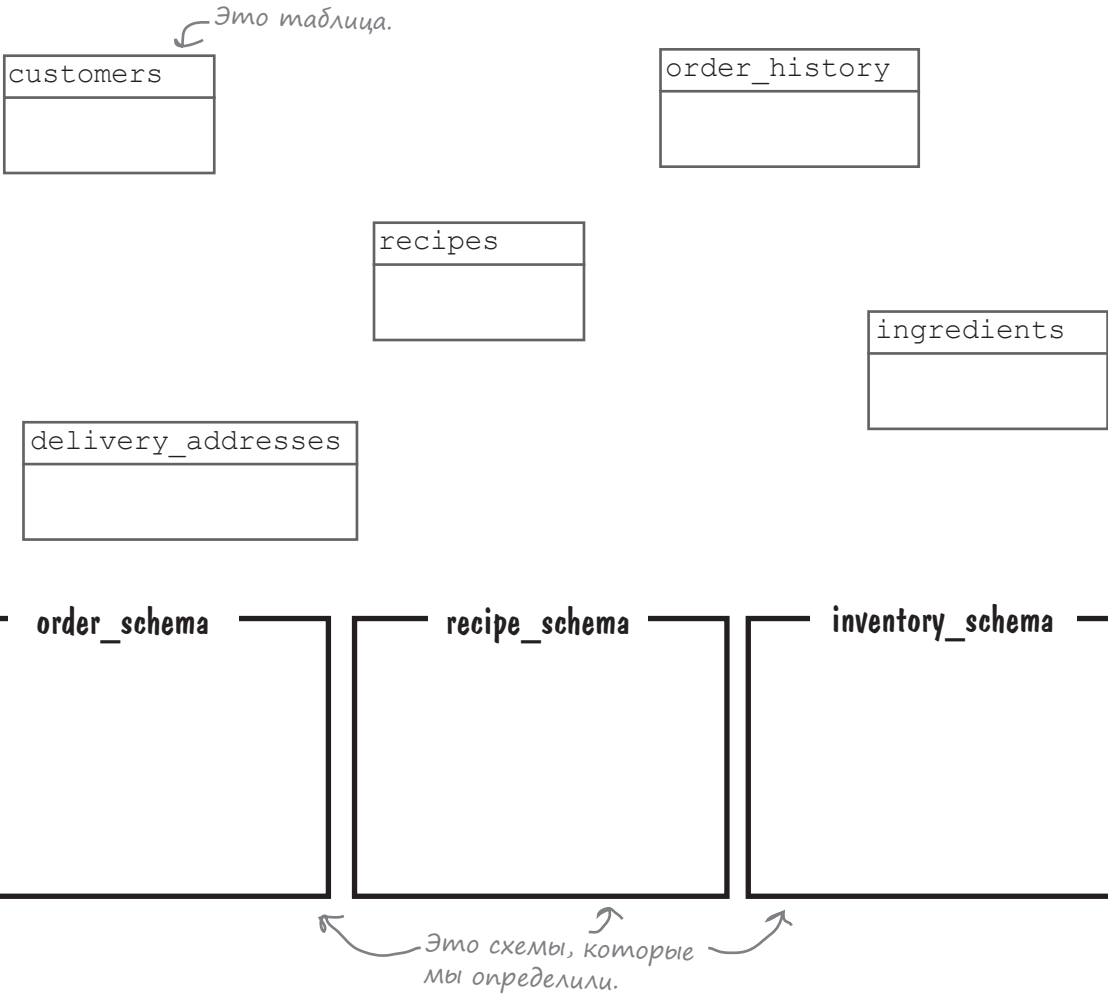
Мозговой
штурм

Подумайте о последствиях модульной организации вашей базы данных. Какие плюсы и минусы вы можете назвать?



Возьмите в руку карандаш

Посмотрите на имена таблиц ниже и попробуйте определить, к каким схемам, которые мы определили для базы данных Naan & Pop, они относятся. Нарисуйте стрелки между таблицами и схемами.



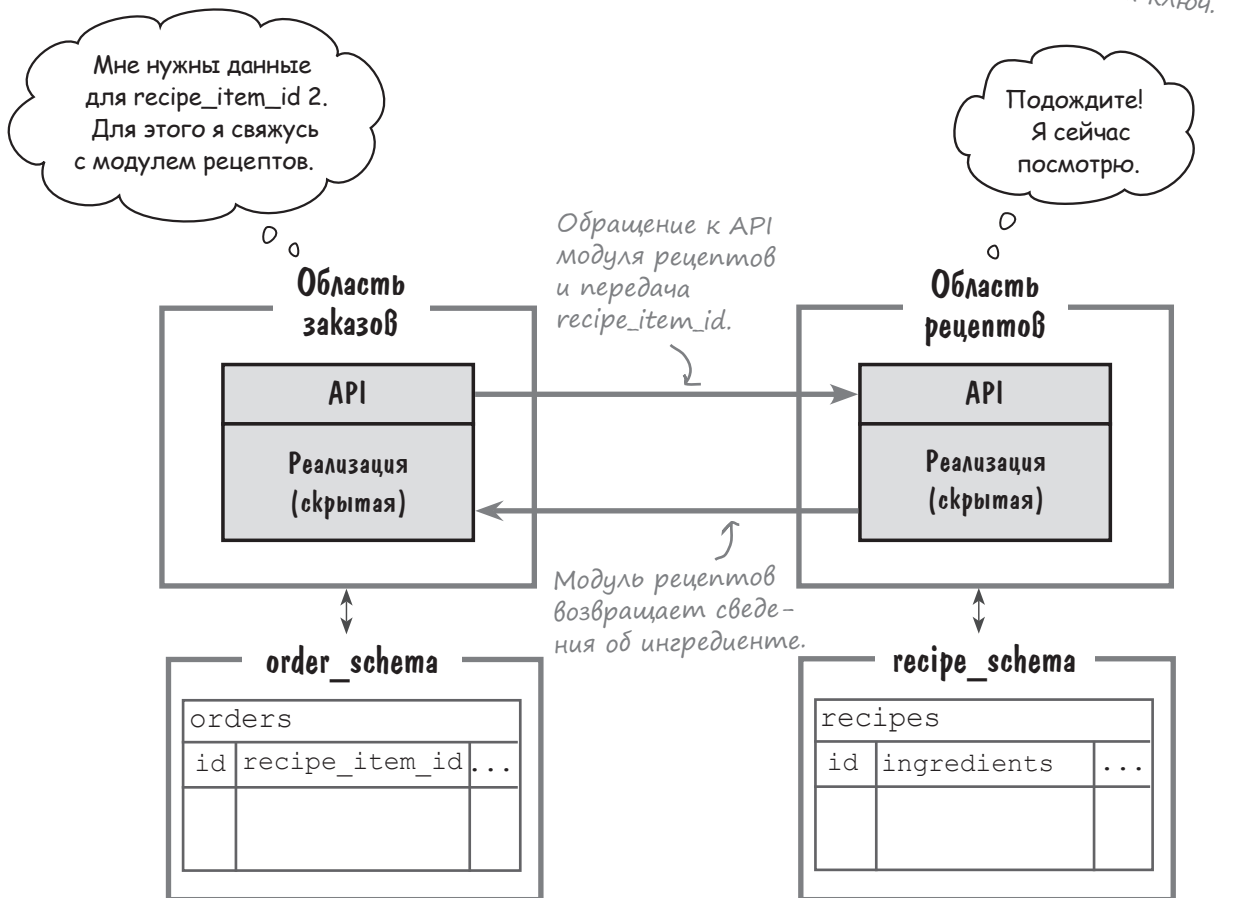
→ Решение на с. 275

Остерегайтесь соединений

Использование разных таблиц и разных схем позволяет отделить друг от друга данные разных модулей, но при этом легко ошибиться и случайно создать SQL-соединение таблиц, принадлежащих разным модулям, тем самым вновь связав их!

Однако допускается хранить идентификаторы ID записей из таблиц одного модуля в таблицах другого. Например, модуль заказов может хранить ID «ингредиентов» рецептов в своих таблицах в `order_schema`, и если ему понадобится информация об ингредиенте, то он обратится к API модуля рецептов и передаст ему соответствующий ID.

Прочтите еще раз! Это не ссылка на внешний ключ.



Вот и все. Теперь, как и в предыдущей главе, рассмотрим сильные и слабые стороны модульных монолитов, а затем приведем оценочную таблицу.

Сверхвозможности модульного монолита

Вот несколько веских причин использовать модульные монолиты.

Бах!

Разделение предметных областей

Архитекторы могут проектировать компоненты, ориентируясь на предметные области, и создавать команды, специализирующиеся на этих областях (а не на технических задачах). Разделение предметных областей — важная сверхспособность.



Предметная специализация

Модульные монолиты способствуют созданию кросс-функциональных команд, специализирующихся на предметных областях, а не на технических задачах, как в многоуровневых архитектурах.

Производительность

Производительность обычно очень высокая, что свойственно многим монолитным архитектурам. Модули взаимодействуют напрямую, и все операции производятся в одном месте.

Бум!

Удобство сопровождения

Модульные монолиты отделяют бизнес-задачи друг от друга, и каждая команда специализируется на своей области. Это упрощает сопровождение кода, когда изменения не затрагивают другие области.

Тестируемость

Изменения, ограниченные одним модулем, проще тестировать. А поскольку члены каждой команды хорошо разбираются в своей области, то могут создать целый набор тестов, в том числе интеграционные, дымовые и сквозные тесты.

Еще!

Слабые стороны модульного монолита

Как мы уже знаем, все зависит от компромиссов. Вот несколько причин, почему не стоит использовать модульные монолиты.



Сложность повторного использования

Модульная организация затрудняет повторное использование логики. Например, чтобы использовать общую функциональность, вам придется определить ее как зависимость, что увеличит связанность модулей.



(Все еще) единый набор архитектурных характеристик

Несмотря на модульную организацию монолита, вы все равно получаете единый набор архитектурных характеристик для всего приложения, даже если потребности бизнес-задач отличаются.



Модульность бывает хрупкой

Границы модулей легко нарушить. Чтобы код не превратился в большой комок грязи, требуются усилия, а чтобы сохранить базу данных модульной, нужны еще большие усилия.

Особенно трудно избежать образования SQL-соединений.



Эксплуатационные характеристики

Модульный монолит ориентирован на бизнес-задачи, но остается монолитом. Поэтому для него трудно достижимы такие эксплуатационные характеристики, как эластичность и отказоустойчивость.

Звездные рейтинги модульного монолита

Мы создали оценочную таблицу для модульных монолитов по аналогии с многоуровневой архитектурой в предыдущей главе. Одна звезда означает, что архитектурная характеристика поддерживается плохо, а пять — что она поддерживается очень хорошо.

В точности как в рецензиях к фильмам.

Лучше, чем в многоуровневой архитектуре.

Монолиты вообще имеют высокую производительность, особенно хорошо спроектированные.

В целом монолиты дороже многоуровневой архитектуры, поскольку требуют более тщательного проектирования и обслуживания.

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★ ★ ★
Тестируемость	★ ★ ★
Развертываемость	★ ★ ★
Простота реализации	★ ★ ★ ★
Развиваемость	★ ★ ★
Производительность	★ ★ ★
Масштабируемость	★
Эластичность	★
Отказоустойчивость	★
Общая стоимость	\$ \$

Как видите, наши оценки модульных монолитов мало отличаются от оценок многоуровневых архитектур. Однако с точки зрения удобства сопровождения, тестируемости и развертываемости модульный монолит намного лучше многоуровневой архитектуры, поскольку изменения в конкретном модуле затрагивают только этот модуль и могут быть протестированы изолированно, что снижает риски, связанные с развертыванием. Модульные монолиты обходятся несколько дороже многоуровневых архитектур, требуя особого внимания. Они также требуют более высокой дисциплины и инструментов, позволяющих управлять границами модулей.



Упражнение

Для каких из следующих систем подходит или не подходит архитектура модульного монолита и почему?
Подсказка: учитывайте сверхвозможности и слабости архитектуры и предметную область.

Онлайн-аукцион, где пользователи делают ставки

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

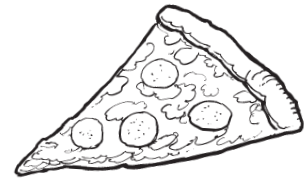
Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

→ Решение на с. 276

Naan & Pop доставляет пиццу!

Команда разработчиков наконец освоила модульные монолиты! Внедрив модульную организацию кода и базы данных, они готовы к любым изменениям в меню Naan & Pop. Ходят слухи, что владельцы планируют добавить полноценное средиземноморское меню. Ждем с нетерпением и желаем им удачи!



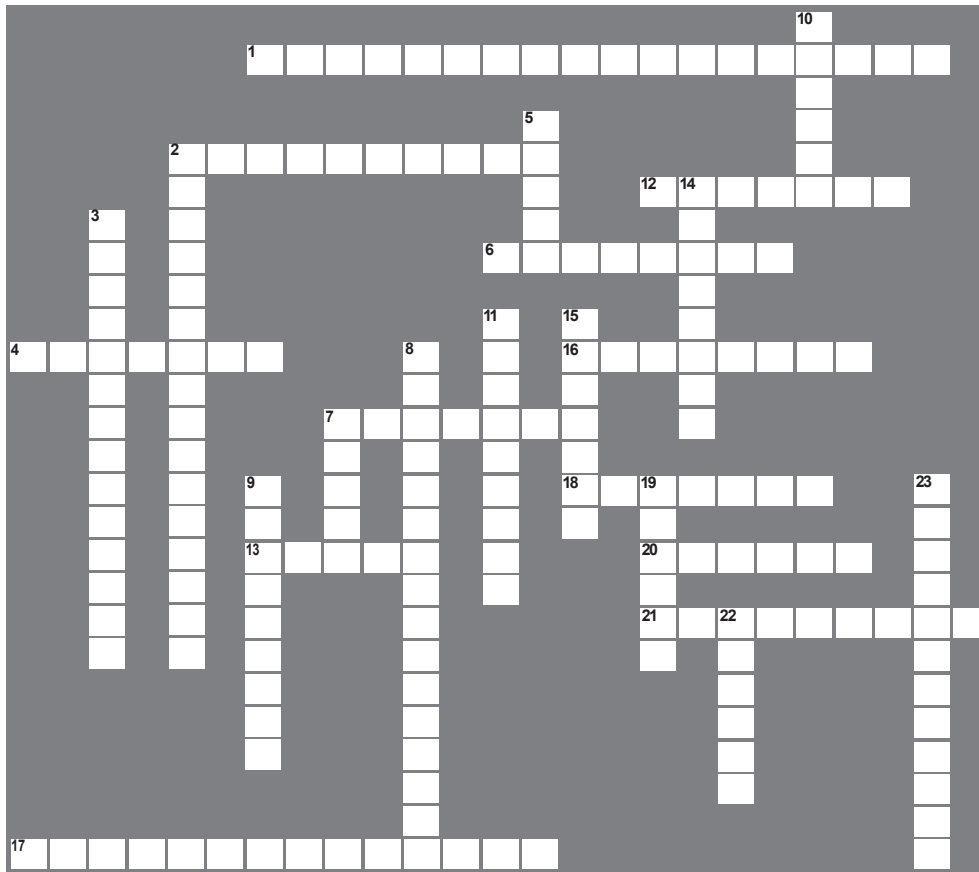
Ключевые моменты

- *Модульный монолит* — это монолитная архитектура, разделенная на предметные области, соответствующие бизнес-задачам.
- Каждой *предметной области* соответствует один *модуль*. Каждый модуль может поддерживать несколько вариантов использования.
- Каждый модуль можно разделить на уровни по техническим задачам, чтобы разделить организацию функциональности.
- Избегайте использовать прямое взаимодействие между модулями, иначе есть риск стереть границы между ними.
- Каждый модуль должен иметь открытый API для взаимодействия с другими модулями, но внутренняя реализация модуля должна оставаться закрытой от остального мира.
- Отказ от прямого взаимодействия модулей позволяет вносить изменения в их реализацию, не заботясь о других модулях.
- Поддержание разделения монолита на модули требует времени и усилий.
- Управлять модульным монолитом можно с помощью разных приемов. Например, некоторые языки имеют встроенную поддержку модулей.
- Еще один подход — физическое разделение кода на отдельные подпроекты или даже разные репозитории. Обычно этот подход предполагает использование инструментов для объединения модулей при сборке монолита.
- Сторонние инструменты тоже могут помочь в управлении этой архитектурой.
- Для сохранения границ отдельных модулей можно использовать несколько приемов и методов.
- Модульную организацию можно применить и к базе, обеспечив хранение данных модулей отдельно друг от друга.
- Следите, чтобы случайно не связать модули в операциях вставки или извлечения данных (например, применяя SQL-оператор `join` к таблицам, принадлежащим разным модулям).



Кроссворд по Модульным Монолитам

Модульные монолиты делятся на модули по бизнес-задачам. Проверьте свои знания об этом архитектурном стиле, разгадав кроссворд.



По горизонтали

1. Характеристика монолитных архитектур с высокой оценкой.
2. Большие предметные области могут делиться на ____.
4. Монолиты обычно имеют одну большую ____ базу.
6. Модули используют идентификаторы данных в ____.
7. ____ модулей в Java.
12. Важно поддерживать ____ между модулями.
13. Java — пример ____ со встроенной поддержкой модулей.
16. Модули взаимодействуют через ____ API.
17. Стиль монолитной архитектуры, первым описанный в этой книге.
18. Все, что является ____ реализации, не относится к архитектуре.
20. Каждая область в этом стиле представляет ____-задачу.
21. ____ в одном модуле не сказываются на других модулях.

По вертикали

2. ____ интерфейс рецептов — часть области рецептов.
3. Модульные монолиты хороши, когда нужно ____ работу многих команд.
5. Последнее дополнение к меню Naan and Pop.
7. Модульные БД могут представлять сущности через ____.
8. Сложные архитектурные характеристики для монолитов.
9. «Спагетти-код» — это сильно ____ код.
10. Монолитные и распределенные — две ____ развёртывания.
11. Каждый модуль имеет открытый прикладной ____.
14. Naan and Pop — это семейный ____.
15. Модульные монолиты разрабатывают кросс-функциональные ____.
19. Схемы баз данных строятся на основе связанных ____.
22. В архитектуре с разделением на предметные области каждой области соответствует свой ____.
23. Технические задачи подобны горизонтальным «срезам» монолита, а бизнес-задачи — ____.

→ Решение на с. 271



Упражнение Решение

На с. 249

Вернемся к многоуровневой архитектуре, созданной для Naan & Pop в главе 6. Ниже показаны ее уровни и логические компоненты. Добавление нового пункта в меню потребует изменить многие из них. Возьмите маркер и нарисуйте треугольник (▲) на компонентах, которые затронет это новое требование.



Вот некоторые объяснения нашего хода мыслей:

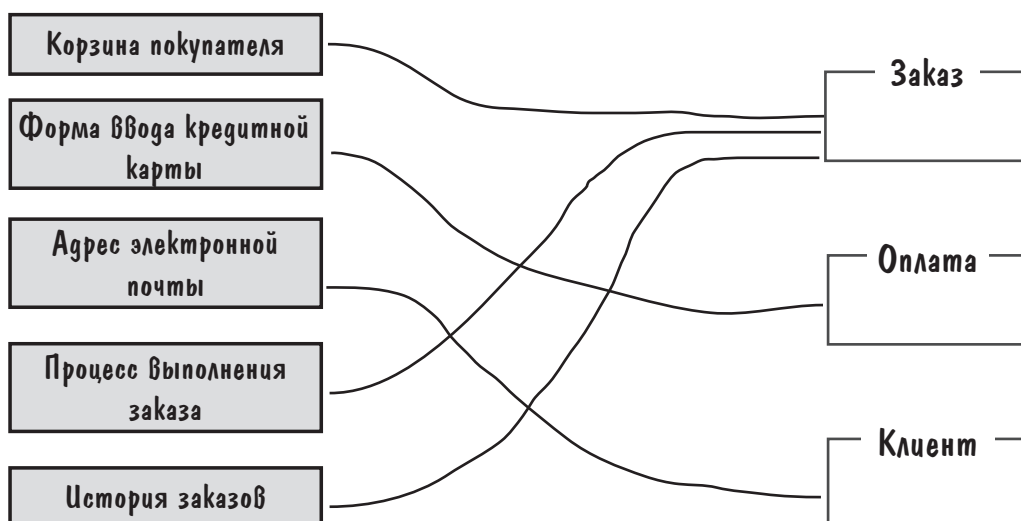
После добавления в меню пиццы может понадобиться разрешить пользователям указывать желаемые ими начинки. Это повлияет на уровень представления, и потребуется ввести новые правила. Могут возникнуть ограничения по времени доставки (все хотят получить пиццу горячей). Кроме того, возможность выбора начинки может повлиять на способ хранения заказов на пиццу (в отличие от других заказов).

Однако области рецептов и запасов не будут затронуты. Рецепт — он и есть рецепт, список ингредиентов с последовательностью выполняемых шагов. И для области запасов управление новыми ингредиентами ничем не отличается от управления любыми другими.

Что чему соответствует? Решение

На с. 252

Часто бывает непросто определить взаимосвязи между компонентами и модулями. В этой игре «Что чему соответствует?» вы должны связать каждый компонент с наиболее подходящим ему модулем. Некоторые компоненты могут быть не связаны ни с каким модулем.





Упражнение Решение

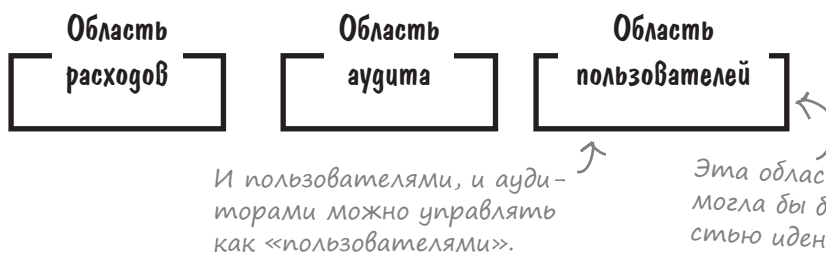
На с. 257

Успешная разработка модульных монолитов требует хороших знаний предметной области. Это позволит разбить ее на отдельные модули. Один из способов сделать это (а их много) — прислушаться к мнению бизнес-экспертов.

Представьте, что вы работаете над приложением учета расходов для малого и среднего бизнеса. Бизнес-требования выглядят так:

- есть пользователи и аудиторы: пользователи добавляют расходы, а аудиторы проверяют отчеты на предмет соответствия правилам;
- когда пользователь добавляет расход, он записывается в базу данных от имени этого пользователя;
- приложение создает журнал аудита, проверив который аудиторы смогут убедиться, что все в порядке.

Определите подобласти, составляющие это приложение. Подсказка: не все, что скажут бизнес-пользователи, можно преобразовать в модуль.

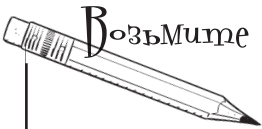


Мозговой шторм

На с. 259

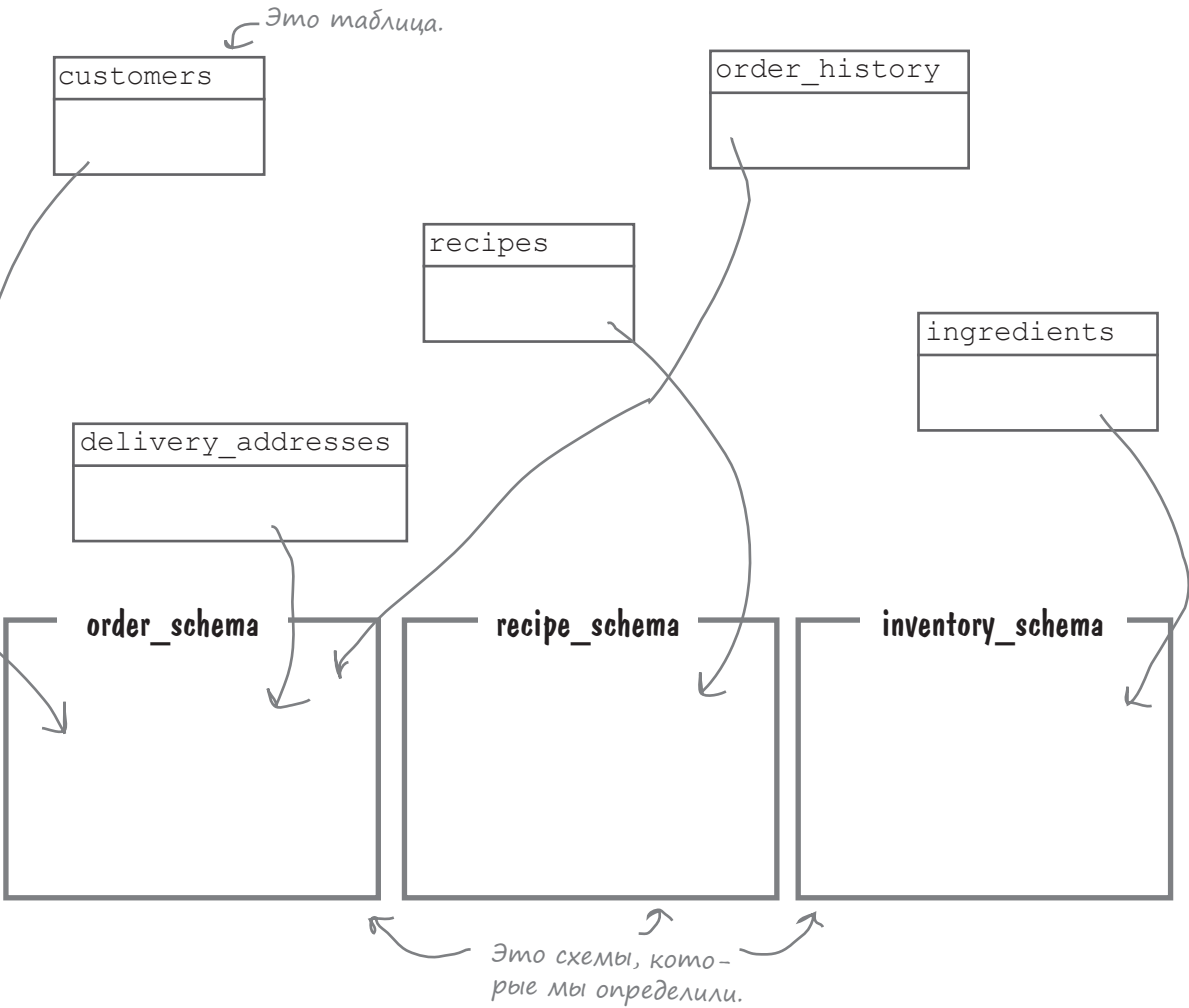
Придумайте какие-нибудь механизмы, которые гарантируют, что одна часть приложения не получит доступ к другой. Например, возможно, в вашем языке программирования есть поддержка разделения модулей во время компиляции? Запишите ниже все идеи, которые придут вам в голову.

- Поддержка в <вставьте сюда название своего языка программирования>.
- Структура репозитория.
- Возможности инструментов сборки.
- Сторонние библиотеки и фреймворки.



Возьмите в руку карандаш Решение

Посмотрите на имена таблиц ниже и попробуйте определить, к каким схемам, которые мы определили для базы данных Naan & Pop, они относятся. Нарисуйте стрелки между таблицами и схемами.





Упражнение Решение

На с. 269

Для каких из следующих систем подойдет архитектура модульного монолита и почему? Подсказка: учитывайте сверхспособности и слабости архитектуры и предметную область.

Онлайн-аукцион, где пользователи делают ставки

Почему? Этой системе нужны возможность масштабирования и высокая эластичность. Монолиты не подходят для таких систем.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? Финансовые системы имеют множество предметных областей, а масштабируемость и эластичность здесь не требуются.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? Модульность хорошо подходит для систем, в которые часто вносятся изменения, хотя многое зависит от характера изменений.

- ☐ Хорошо подходит
☒ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? Мы же отдали предпочтение другой архитектуре! Разве не так?

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

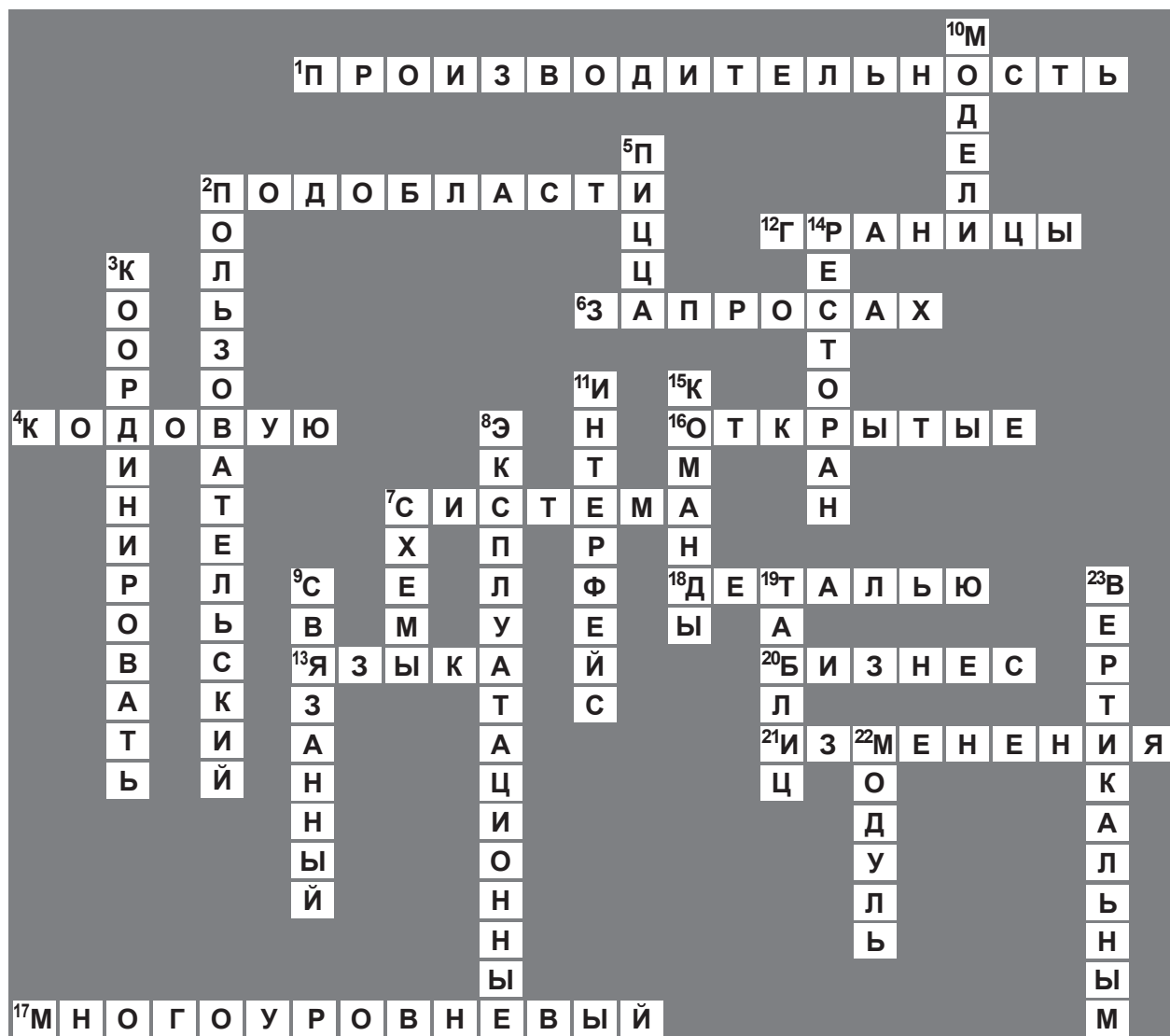
Почему? Слишком много движущихся частей, и еще этой системе понадобятся эластичность и масштабируемость.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

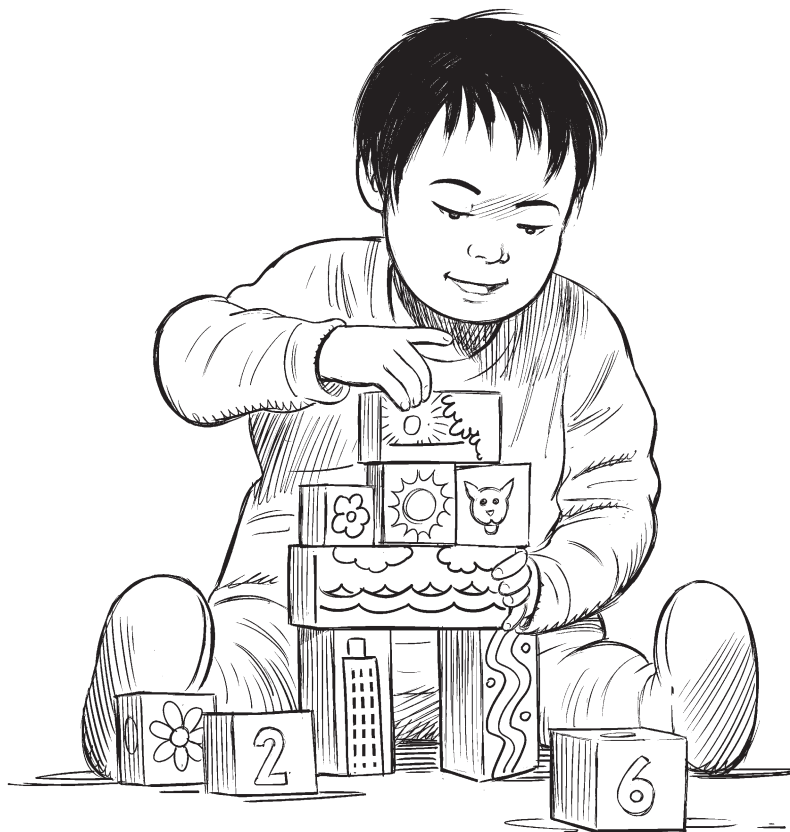


Кроссворд по модульным монолитам. Решение

На с. 271



Индивидуальная подгонка



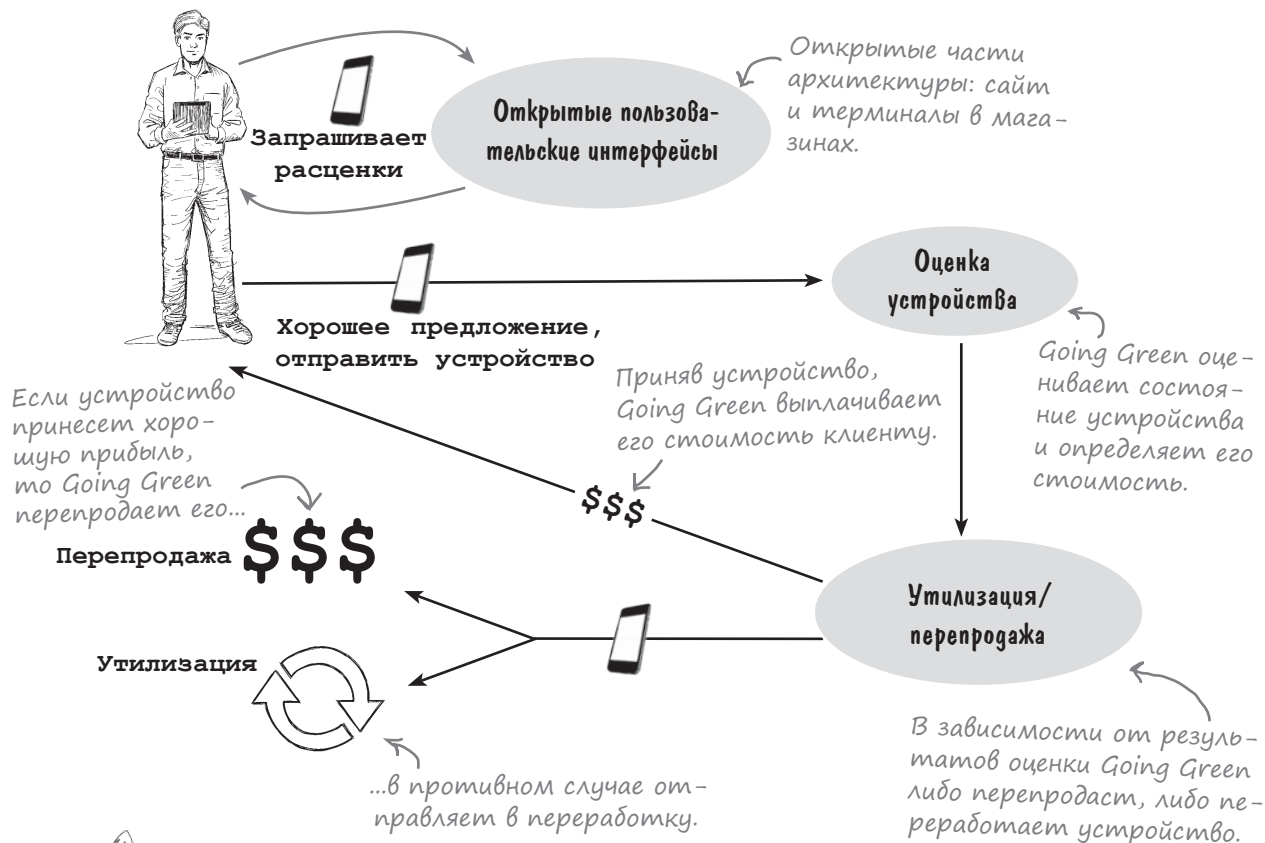
Новые возможности можно создавать по одной. Есть цели, которые особенно хорошо достигаются с помощью определенных архитектурных стилей. Микроядерная архитектура — чемпион по индивидуальной подгонке. Она хорошо подходит для огромного количества приложений. Поняв идею этого стиля, вы начнете видеть его повсюду!

В данной главе вы познакомитесь с архитектурой, позволяющей пользователям делать все по своему усмотрению.

Преимущества Going Green

Что у многих валяется без дела? Старая электроника! Going Green — это стартап, который планирует заработать на рынке перепродажи и переработки старых сотовых телефонов, плееров и другой мелкой электроники.

Проанализировав необходимые характеристики, архитекторы разработали систему из трех частей, нуждающихся в разных возможностях. Команде нужна ваша помощь. Готовы? Вот как выглядит система на данный момент:



Мозговой
Штурм

Получив представление о бизнес-модели Going Green и задачах, решаемых ее системой, какую архитектуру вы бы посоветовали этой компании — монолитную или распределенную? Почему?

СТАНЬТЕ архитектором



Архитекторы не могут не анализировать. Попробуйте и вы определить три важные архитектурные характеристики для каждого из трех сервисов в новой архитектуре *Going Green*.

Открытые
пользовательские
интерфейсы

Архитектурные характеристики

Масштабируемость

Перечислите архитектурные характеристики, важные для каждого сервиса.

Архитектурные характеристики

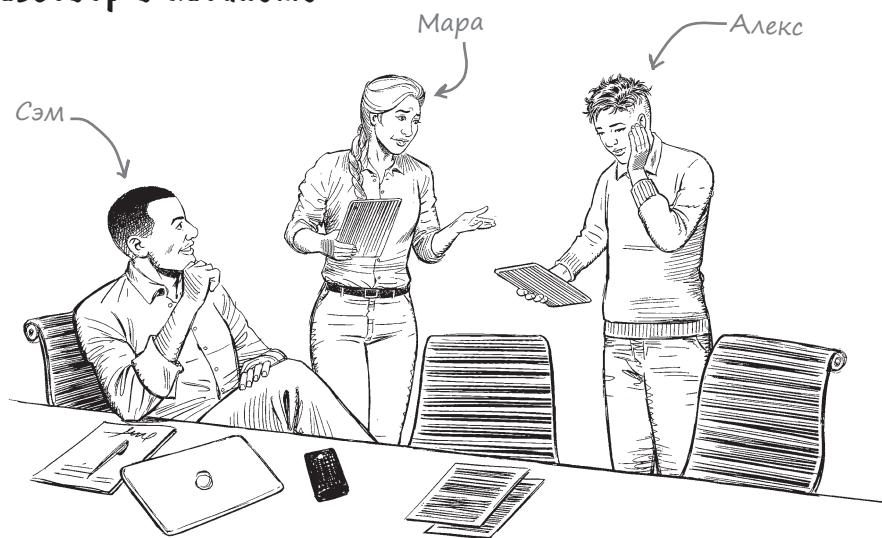
Оценка
устройства

Архитектурные характеристики

Утилизация/
перепродажа

→ Решение на с. 304

Разговор в кабинете



Мара. Нам нужно разделить работу над архитектурой приложения Going Green. Сэм, вы с Алексом займетесь сервисом оценки устройств.

Алекс. Это тот, который определяет ценность присланных устройств? Похоже, нам досталась одна из самых интересных частей приложения.

Сэм. Как часто нам нужно будет добавлять в сервис новые конфигурации устройств?

Мара. Как минимум несколько раз в месяц или даже в неделю. Это очень важно: чем быстрее мы сможем обновлять сервис оценки устройств, тем больше заработает компания.

Сэм. Чем обусловлена такая прямая зависимость?

Мара. Going Green получает прибыль от перепродажи электроники. Обычно чем новее устройства, тем лучше их состояние и выше стоимость, и чем чаще мы сможем добавлять новые конфигурации, тем больше заработает компания.

Алекс. Ого! Поддержка таких частых изменений — сложная задача.

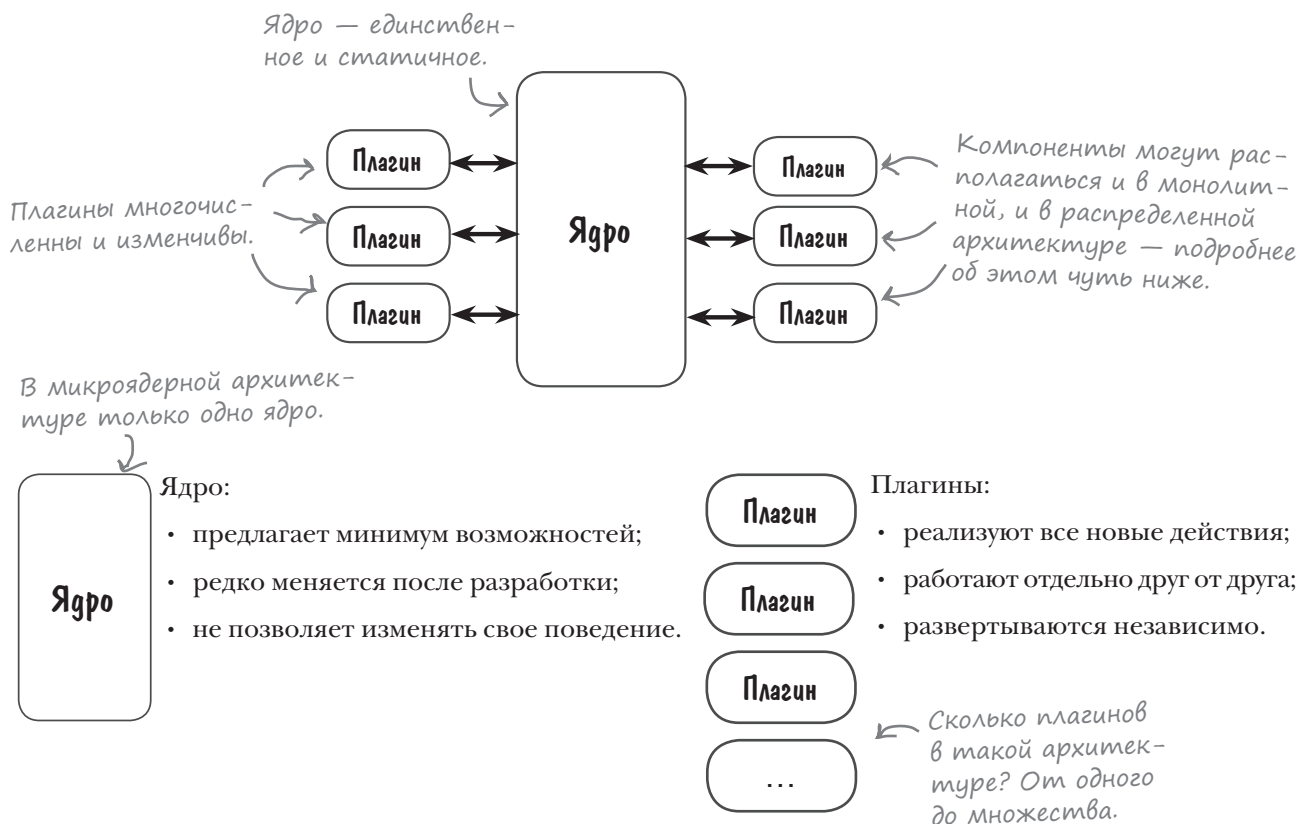
Мара. Это точно! И не забудьте: вы должны гарантировать, что добавление поддержки новых устройств не повлияет на поддержку существующих.

Сэм и Алекс. Ну конечно!

Мара. Я думаю, нам стоит взглянуть на микроядерную архитектуру. Она позволяет легко создавать и добавлять новые возможности в виде плагинов.

Две части микроядерных архитектур

Свое название *микроядерная* архитектура заимствовала из области разработки операционных систем. **Ядро** операционной системы очень маленькое и предлагает только самые основные возможности. Микроядерная архитектура состоит из двух основных частей: *ядра* и множества *плагинов*.

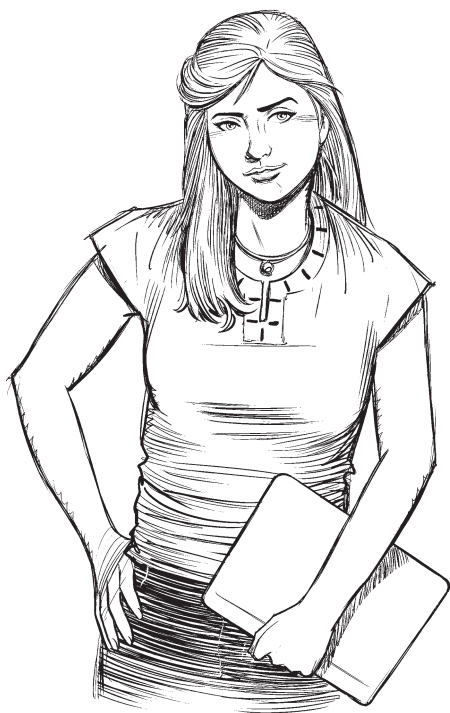


Часть задаваемые вопросы

В: Разделение в микроядерной архитектуре осуществляется по техническим или предметным задачам?

О: Вспомните, что говорилось в главе 5! Большинство микроядерных архитектур разделены на части по техническим задачам. Ядро можно разделить на уровни: например, представления, бизнес-логики и т. д. Однако это один из тех странных архитектурных стилей, которые также можно разделить по предметным задачам, — все зависит от подхода к разработке плагинов.

В отношении модели разворачивания микроядерные архитектуры обычно монолитны. Однако, как вы увидите далее, иногда распределенная модель выглядит более естественной.



Постойте-ка. Практически все программы, которые я использую, так или иначе поддерживают плагины. Вы хотите сказать, что все они реализуют микроядерную архитектуру?

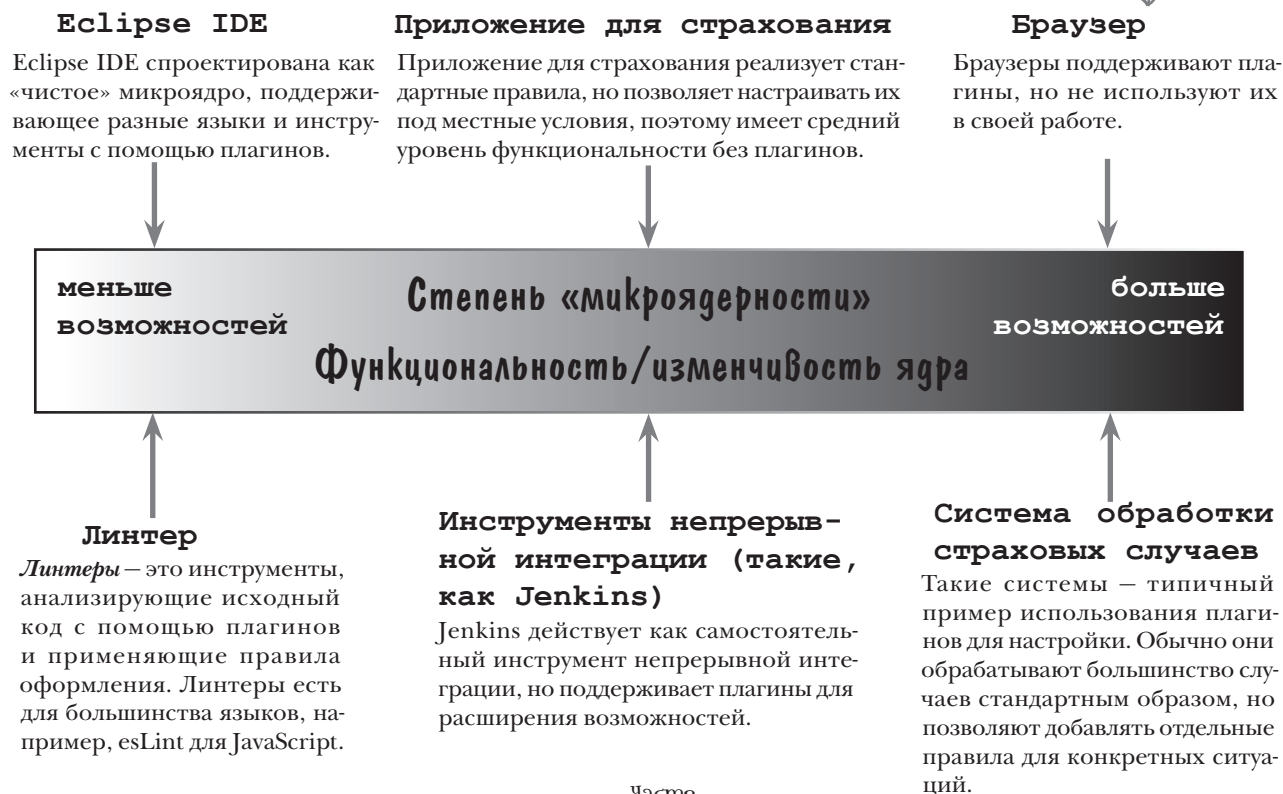
Не совсем. В любой микроядерной архитектуре есть плагины, но они используются не только в ней.

К сожалению, нет четкой границы между микроядерными системами и системами с плагинами. Главный признак «микроядерности» архитектуры (назовем это так) — широта возможностей, предлагаемых ядром без плагинов, и его *изменчивость* (как часто его нужно менять).

Спектр «микроядерности»

Многие программы поддерживают плагины: IDE, браузеры, инструменты сборки и т. д. Но простая поддержка плагинов не делает систему микроядерной — все дело в ядре. В микроядерной архитектуре оно мало полезно без плагинов.

Браузер прекрасно можно использовать и без плагинов, так что это явно не микроядерная архитектура.



Часть Задаваемые Вопросы

В: Все системы, поддерживающие плагины, являются микроядерными?

О: Вовсе нет. Плагины поддерживаются многими системами, и степень их микроядерности зависит от изменчивости и функциональности ядра (хотя мало кто оценивает их по этому показателю).

В: Этот архитектурный стиль полезен только для инструментов разработки ПО?

О: Эту архитектуру с поддержкой программной настройки реализуют не только инструменты, но и многие бизнес-приложения. Она подходит для любой области, где требуется настройка и каждое изменение изолировано.

В: Микроядерная и микросервисная архитектуры — это одно и то же?

О: Нет, сходство в названиях — лишь совпадение. Название «микроядерная» взято из мира операционных систем, а «микросервисная» обусловлено использованием небольших и самостоятельных единиц развертывания в распределенной архитектуре.



Упражнение

«Микроядерность» проявляется повсюду. Попробуйте поместить инструменты и категории приложений, перечисленные ниже, в правильное место на спектре, определив, насколько они «микроядерные». Подсказка: степень «микроядерности» каждой системы зависит от ее полезности без плагинов.

Наш пример. Мы определили Eclipse как «микроядерный» продукт.

Не знакомы с паттерном BFF? Загляните на сайт <https://samnewman.io/patterns/architectural/bff/>.

Eclipse IDE

Операционная система для мобильных устройств

IDE с поддержкой плагинов (не Eclipse)

Паттерн проектирования Backend for Frontend (BFF)

Сервис по оценке устройств для компании Going Green.

Браузер

меньше возможностей

Функциональность/изменчивость ядра

больше возможностей

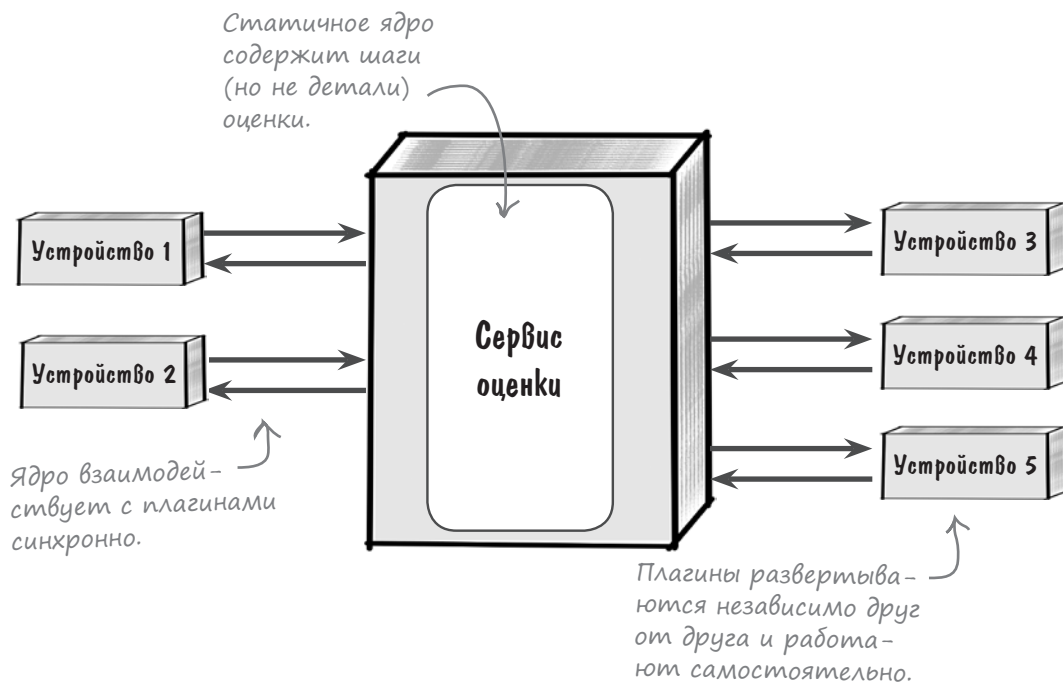
Степень «микроядерности»

Решение на с. 305

Ядро сервиса оценки устройств

Вы и ваша команда решили реализовать микроядерную архитектуру в новом сервисе оценки устройств.

Базовая система реализует основные критерии оценки, такие как возраст, состояние и номер модели устройства. А для оценки каждого конкретного типа устройств она использует плагин, реализующий правила определения стоимости устройства в случае перепродажи.



Для плагинов вы решили использовать распределенную физическую архитектуру, чтобы улучшить масштабируемость. Вдобавок это решение даст Going Green возможность писать плагины на других языках.

Наконец, вы выбрали синхронные взаимодействия, поскольку сервис должен быть отзывчивым и необходимости в сложных асинхронных взаимодействиях нет.

Рассмотрим эти решения более подробно.



Я понимаю, что обычно плагины реализуются как компоненты в монолитной архитектуре. Но не усложняется ли развертывание при добавлении в него плагинов? Было бы проще, если бы они развертывались отдельно. Можно ли создавать распределенные плагины?

Да, можно! Архитектура может определять способ развертывания плагинов.

Одни микроядерные архитектуры добавляют в единый монолит не только ядро системы, но и плагины; другие реализуют распределенную модель развертывания, как рассказывалось в главе 5.

Микроядерные архитектуры занимают промежуточное положение между монолитными и распределенными архитектурами. Архитекторы могут реализовать любую из моделей развертывания.

Часть
**Задаваемые
Вопросы**

В: Должно ли быть монолитом ядро микроядерной архитектуры?

О: Не обязательно. Когда это имеет смысл (например, в приложениях для ПК), ядро можно реализовать как единую систему; в других случаях можно распространять части ядра и плагины по отдельности.

В: Как реализовать плагины?

О: Плагины можно реализовать, используя интерфейсы, но практически все платформы и технологические стеки предлагают вспомогательные библиотеки и фреймворки.

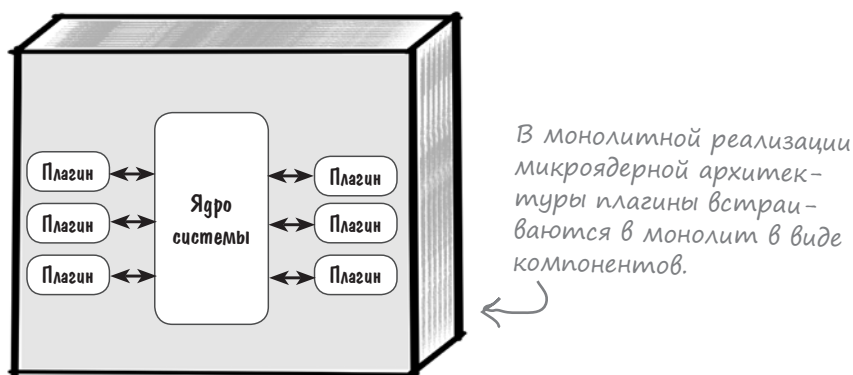
В: Можно ли реализовать плагины, используя технологический стек, отличный от стека ядра?

О: Да, и это одно из преимуществ распределенных плагинов. Вы можете использовать любую платформу, поддерживающую вызовы через сетевое соединение.

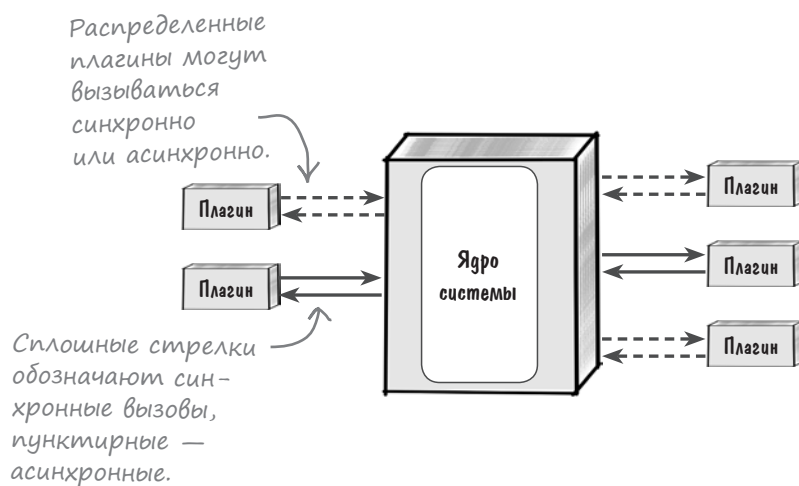
Инкапсулированные и распределенные плагины

Ядро в микроядерной архитектуре — это то, к чему подключаются плагины. Подключение, как правило, реализуется через *интерфейс*. Плагин реализует интерфейс, через который ядро поддерживает компонент.

При использовании монолитной модели каждый плагин реализуется как компонент, который подключается к ядру через интерфейс.



В других моделях плагины подключаются как распределенные компоненты: через конечные веб-точки, очереди событий и т. д. Кроме того, такие плагины могут вызываться синхронно или асинхронно. (Подробнее об этом рассказывается в главе 11.)





Упражнение

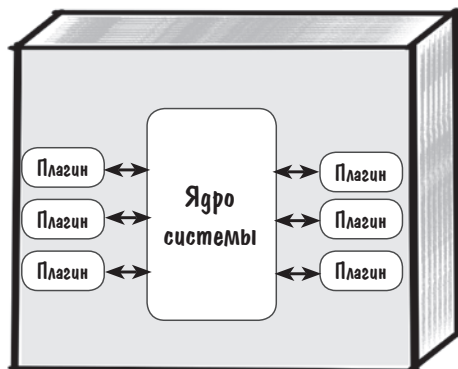
Команда разработки сервиса оценки в Going Green должна выбрать физическую архитектуру – **монолитную** (ядро и плагины развертываются вместе) или **распределенную** (плагины развертываются отдельно от ядра). Помогите им проанализировать компромиссы: перечислите плюсы и минусы каждого варианта. *Подсказка: подумайте, как каждый вариант повлияет на архитектурные характеристики.*

Монолиты обычно имеют большую производительность из-за меньшего количества сетевых вызовов.

Плюсы

Производительность

Монолитная



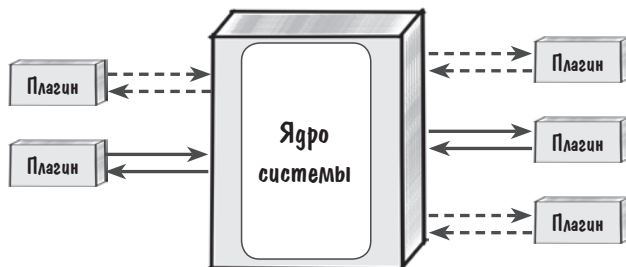
Монолиты загружают плагины только при запуске, что усложняет развертывание.

Минусы

Одна единица развертывания

Плюсы

Распределенная

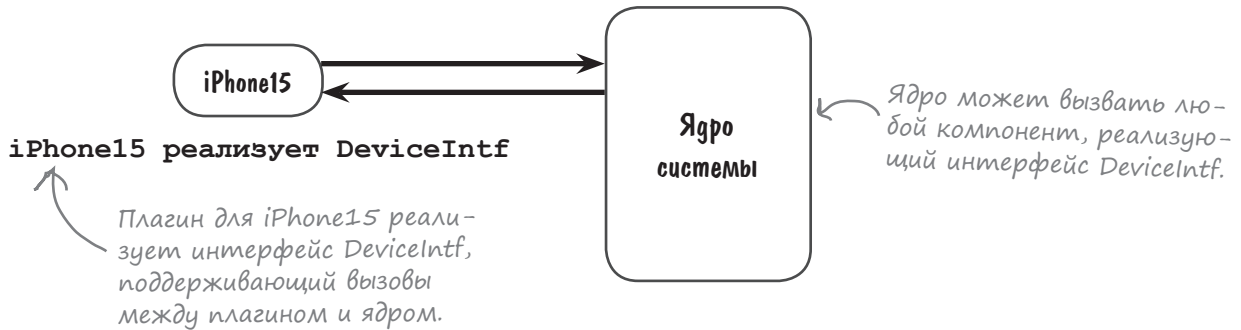


Минусы

→ Решение на с. 306

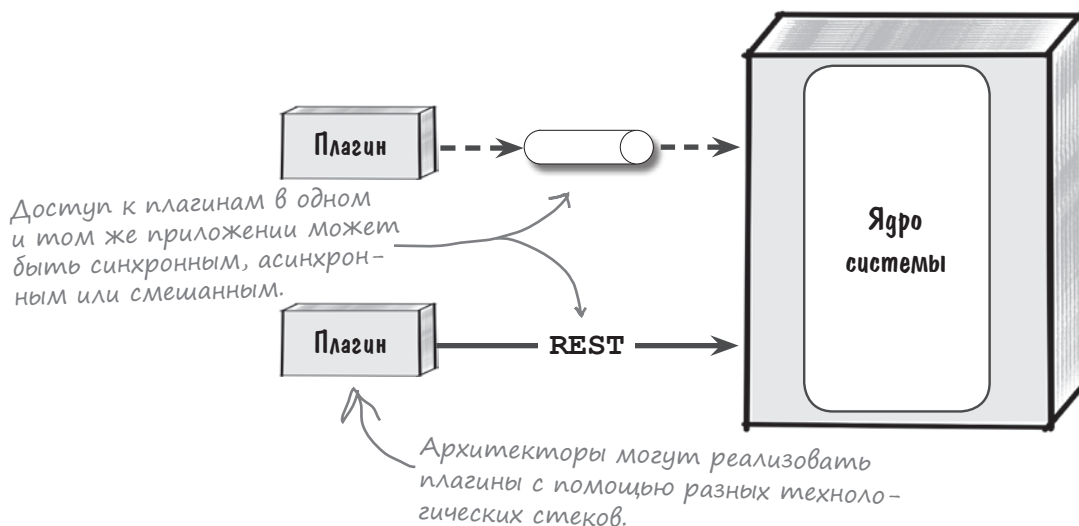
Взаимодействие с плагинами

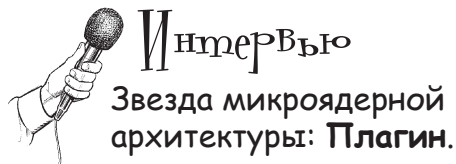
Плагины должны поддерживать взаимодействие с ядром системы, чтобы оно могло вызвать метод плагина через интерфейс и использовать результаты. Взаимодействие можно реализовать несколькими способами, зависящими от такого фактора, как физическая архитектура.



Как происходит фактический вызов, зависит от физической архитектуры. В монолитных архитектурах плагины реализуются в том же технологическом стеке, что и ядро, и развертываются как встроенные компоненты (в виде JAR-файлов для Java, DLL-библиотек для .NET или GEM-библиотек для Ruby).

В распределенных архитектурах ядро может вызывать плагины синхронно или асинхронно. Кроме того, разработчики не ограничены платформой ядра и могут писать плагины на других языках.





Head First. Минуточку... я подключу микрофон. Раз-два-три, проверка... Итак, встречайте: Плагин!

Плагин. Как уместно! Меня *повсюду* используют для работы. Я рад быть здесь и помочь прояснить некоторые противоречия.

Head First. Все говорят, что вы — звезда микроядерной архитектуры, однако в прессе ядру уделяется больше внимания. Справедливо ли это?

Плагин. Может, это и так, но что толку от ядра без меня? Может, оно и больше, но без меня *ни на что не способно*.

Head First. Вы много где встречаетесь. А как насчет вашей роли в архитектурах, не связанных с микроядрами? Многие системы, не являющиеся микроядерными, поддерживают плагины...

Плагин. Я готов работать где угодно, но предпочитаю микроядра. В других архитектурах плагины — это приправа: мы не нужны и лишь добавляем оттенки вкуса. В микроядерной архитектуре, напротив, я — главное блюдо! Вся архитектура основана на мне. Мне льстит такой уровень важности.

Head First. Поговорим о ваших чертах, постоянно упоминаемых в анализе компромиссов. Верно ли, что ваша распределенная версия имеет проблемы с производительностью?

Плагин. Не будем всех причислять под одну гребенку! Использование сетевых вызовов для связи действительно снижает производительность. Но распределенные плагины лучше масштабируются и их можно писать на разных языках. Для кого-то это большой плюс. А еще разные физические архитектуры плагинов имеют разные компромиссы.

Head First. Справедливое замечание. Но давайте поговорим о работе с чересчур изменчивыми ядрами.

Плагин. Изменчивое ядро в микроядерной архитектуре — это плохо. Изменение поведения — *моя* ответственность! Чем чаще меняется ядро, тем выше вероятность, что оно помешает моей работе. Я стремлюсь к профессионализму, поэтому предпочитаю работать со стабильными ядрами.

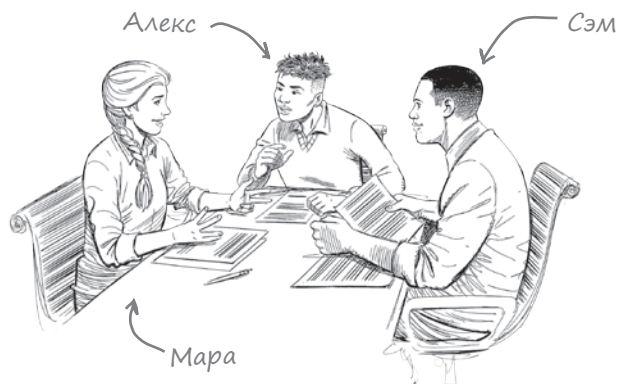
Head First. Ваша способность приспосабливаться широко известна, а есть ли у вас другие функции? Что еще вам под силу?

Плагин. Хороший вопрос! Я не люблю, когда меня загоняют в рамки моей самой известной роли. Я легко справляюсь с индивидуальной подгонкой без длинных операторов `switch`, используя элегантный архитектурный стиль. Плагины можно применять для разных целей — везде, где нужна хорошая изоляция. Например, я постоянно участвую в А/В-тестировании. Архитекторы сохраняют старое поведение в `PluginA`, добавляют новое в `PluginB` и решают, какое из них вызвать. Кроме того, важную роль я играю в средствах интеграции, инструментах разработчика и много где еще.

Head First. И в заключение, может быть, вы хотите что-то добавить?

Плагин. Конечно! Ищите меня в ближайшей к вам архитектуре либо как часть монолита, либо как конечную точку, к которой обращается микроядро.

Разговор в кабинете



Мара. Всем привет! Зашла к вам, чтобы взглянуть на анализ компромиссов для сервиса оценки. Какую архитектуру вы выбрали? Монолитную или распределенную?

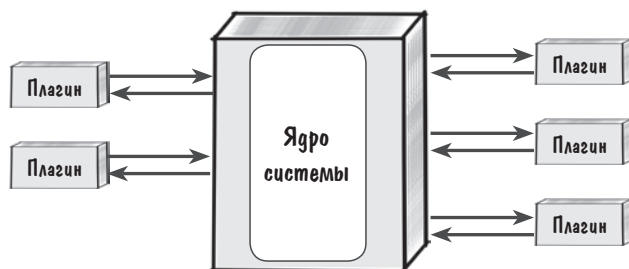
Сэм. Мы работаем над этим. Я исследовал компромиссы распределенной версии, и мне она нравится больше. Вот наши выводы:

Плюсы

Доступность

Развертываемость

Масштабируемость



Минусы

Производительность

Сложность

Мара. Объясните, как вы пришли к этим выводам.

Алекс. В распределенной версии не нужно перезапускать ядро системы, чтобы добавить новые устройства. Это обеспечивает лучшую *доступность* по сравнению с монолитной версией, требующей перезапускать систему для загрузки новых плагинов. Кроме того, один плагин развернуть проще, чем весь сервис. Плагины запускаются в отдельных процессах, поэтому систему можно сделать более *масштабируемой*. Но это скажется на ее *производительности*, ведь сетевые вызовы требуют больше времени, чем вызовы внутри процесса.

Мара. Это действительно хороший компромисс. Бизнесмены согласны с тем, что масштабируемость очень важна для новой системы.

Часто
Задаваемые
Вопросы

В: Где в микроядерной архитектуре находится пользовательский интерфейс?

О: Зависит от обстоятельств! В монолитной системе UI обычно добавляется в ядро. Но если проектируется распределенный микроядерный сервис, то UI обычно реализуют другие части системы.

В: Может ли пользовательский интерфейс использовать микроядро?

О: Конечно! Многие паттерны проектирования UI (например, BFF, упоминавшийся выше) используют микроядро для взаимодействия с конечными точками UI, такими как iOS, Android и браузеры.

В: Разве микроядро — это не паттерн проектирования «Декоратор»?

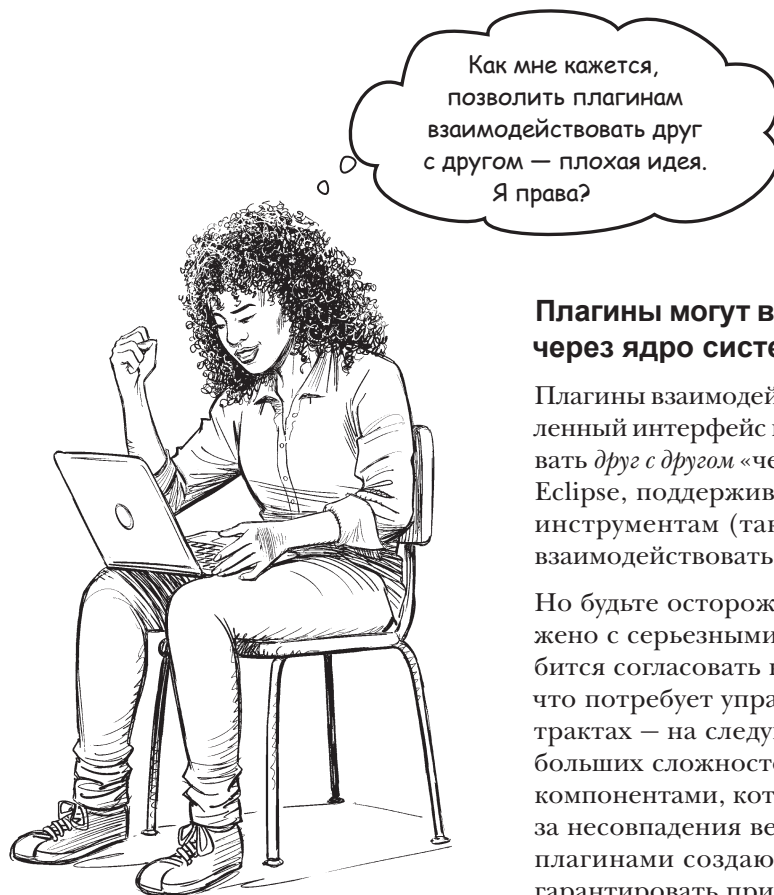
О: Верно подмечено! Их цели во многом совпадают, но микроядерная архитектура — это лишь один из способов реализации паттерна «Декоратор». В отличие от паттернов проектирования, архитектура требует более глубокого анализа ограничений и возможностей. Например, паттерны не учитывают такие возможности, как масштабируемость.

В: Является ли микроядерная архитектура единственным способом реализовать индивидуальную подгонку?

О: Вовсе нет. Это один из многих способов. Микроядерная архитектура хороша, когда структура системы — ее основные возможности, которые редко меняются, — требует точной подгонки с помощью плагинов. Для систем без такой структуры лучше использовать другие архитектурные стили.

В: Как организована основная система? Это просто один большой логический компонент?

О: Выбирая микроядерную архитектуру, мы проектируем ядро, исходя из желаемой организации логических компонентов в системе. Например, если нужно разделить функции внутри ядра, то его можно реализовать в виде набора уровней, как в многоуровневой архитектуре. С другой стороны, можно, следуя принципам DDD, спроектировать ядро вокруг ограниченного контекста как в модульном монолите. Микроядерный стиль часто используется в гибридных архитектурах, где настраиваемость является основной характеристикой.



Плагины могут взаимодействовать друг с другом через ядро системы. Но нужно ли это?

Плагины взаимодействуют с ядром системы через установленный интерфейс и, соответственно, могут коммуницировать *друг с другом* «через» ядро. Например, именно так IDE Eclipse, поддерживающая множество языков, позволяет инструментам (таким, как компиляторы и отладчики) взаимодействовать.

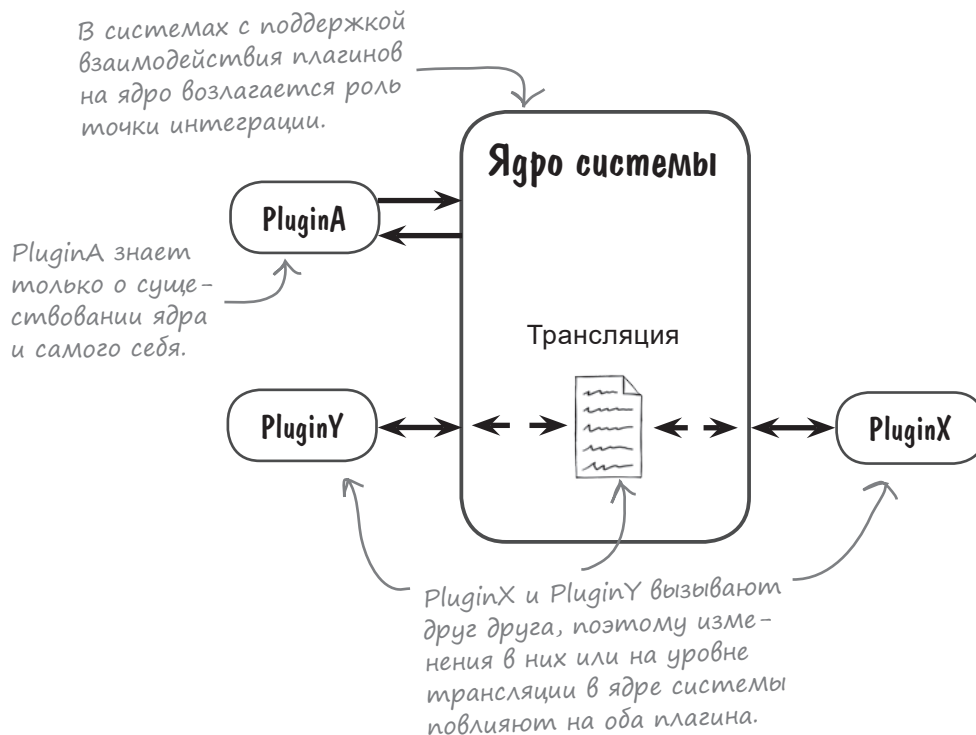
Но будьте осторожны! Взаимодействие плагинов сопряжено с серьезными проблемами. Во-первых, вам понадобится согласовать контракты между ядром и плагинами, что потребует управления версиями. (Подробнее о контрактах — на следующей странице.) Например, одна из больших сложностей в Eclipse — это зависимости между компонентами, которые могут вызвать головную боль из-за несовпадения версий. Во-вторых, зависимости между плагинами создают проблемы с доступностью, требуя гарантировать присутствие всех необходимых плагинов во время выполнения.

Чтобы осознать проблемы, связанные с взаимодействием плагинов, мы должны рассмотреть два способа взаимодействия ядра с плагинами.

Контракты плагинов

Обычно взаимодействие плагинов организуется через **контракты** (другое слово для обозначения *интерфейса*). Коммуникация происходит *исключительно между плагинами и ядром*. Чтобы сделать возможным взаимодействие плагинов, ядро должно взять на себя роль посредника.

В примере ниже PluginA ничего не знает о других плагинах и взаимодействует только с ядром. Однако PluginX должен взаимодействовать с PluginY, и эта коммуникация протекает через ядро системы.

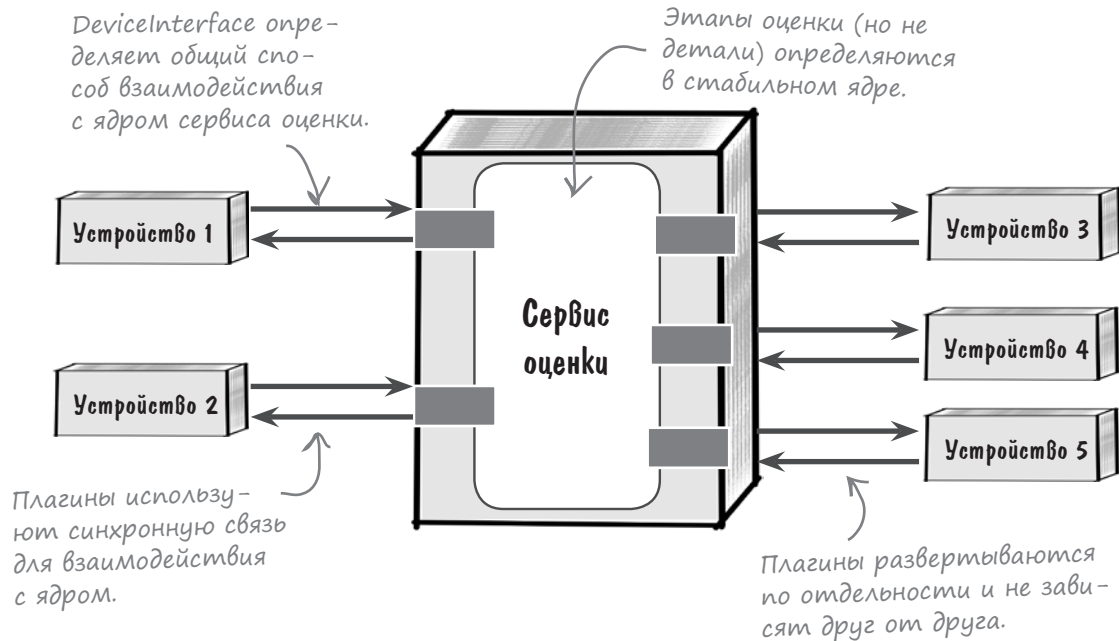


Мозговой штурм

Вы уже знаете, что должны позаботиться о трансляции через ядро системы, а теперь подумайте вот о чем: что случится, если при обновлении PluginY изменится контракт между PluginX и PluginY? Можно ли организовать взаимодействие так, чтобы не менять PluginX при изменении PluginY?

Going Green Включается в работу

Поразмыслив над организацией взаимодействий плагина с ядром, команда решила определить интерфейс *DeviceInterface*, который должен реализовать каждый плагин. Теперь они смогут добавлять новые устройства, реализуя этот интерфейс и процедуру их оценки.



Ваш сервис оценки устройств получился очень удачным. Отличная работа!

В заключение кратко перечислим сильные и слабые стороны этого архитектурного стиля.

Сверхвозможности микроядерной архитектуры

Микроядерная архитектура — распространенный архитектурный стиль. Его часто можно встретить в гибридных архитектурах, требующих индивидуальной подгонки.

БАЦ!

Нужно настраивать отдельные части приложения? Микроядерная архитектура вам поможет!

Настройка поведения

Микроядерная архитектура прекрасно справляется с подгонкой под индивидуальные требования.

Бах!

Развиваемость

Эта характеристика означает возможность постоянно дорабатывать систему, отказываясь от старых моделей поведения и реализуя новые. В отличие от адаптируемости, развиваемость предполагает сохранность старого поведения только в течение короткого времени. Плагины дают отличный способ реализовать эту характеристику.

Разделение

Настройка может осуществляться на уровне кода или архитектуры. В подходе на уровне кода разработчикам придется потрудиться, чтобы все сделать правильно. В системе, построенной на плагилах, разделение функций между ядром и плагинами более четкое, что упрощает реализацию.

Микроядро — отличный пример архитектурного подхода к настройке.



Адаптируемость

Адаптируемость как архитектурная характеристика подразумевает возможность сохранять существующие функции и добавлять новые. Микроядерная архитектура хорошо поддерживает эту характеристику, вы можете сохранять старые плагины и добавлять новые.

Простота — ценное свойство.

Простая структура

Микроядерная архитектура состоит из двух основных движущихся частей — ядра и плагинов. Это упрощает ее понимание и реализацию.

БУМ!

Слабые стороны микроядерной архитектуры

Они проявляются в основном при неправильном ее использовании: например, когда ядро меняется слишком часто или когда плагины должны взаимодействовать.



Изменчивость

Ядро после реализации должно меняться очень редко. Часто меняющееся ядро — верный признак того, что выбрано не лучшее архитектурное решение или плагины организованы неправильно.

Одна из распространенных ошибок в микроядерных архитектурах — нестабильное ядро из-за частых изменений в предметной области.

Общие зависимости плагинов

Часто возникает соблазн использовать в плагинах одни и те же зависимости (например, библиотеки), но это может повлечь проблемы со связанностью и развертыванием.

Чтобы микроядро не превратилось в большой комок грязи, следите за связанностью через общие зависимости.



Взаимодействие плагинов

Оно связано с множеством сложных компромиссов. Системы, использующие такой подход (например, Eclipse), отличаются очень высокой сложностью.

Производительность

При использовании физической распределенной архитектуры для плагинов можно заметить снижение производительности. Это зависит от применяемых протоколов и объема передаваемой информации.



Звездные рейтинги микроядерной архитектуры

Мы создали оценочную таблицу для микроядерной архитектуры по аналогии с многоуровневой архитектурой и модульными монолитами, чтобы показать, насколько хорошо она поддерживает каждую из перечисленных архитектурных характеристик. Одна звезда означает, что архитектурная характеристика поддерживается плохо, а пять — что она поддерживается очень хорошо.

В точности как в рецензиях к фильмам.

Тестирование упрощается благодаря тому, что ядро и плагины можно тестировать отдельно.

Встроенные плагины дают хорошую производительность.

Простота снижает общую стоимость.

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★ ★ ★
Тестируемость	★ ★ ★
Развертываемость	★ ★ ★
Простота реализации	★ ★ ★ ★
Развиваемость	★ ★ ★
Производительность	★ ★ ★
Масштабируемость	★
Эластичность	★
Отказоустойчивость	★
Общая стоимость	\$

Микроядерная архитектура имеет простую и понятную структуру.

Распределенные плагины дают хорошую масштабируемость.



Упражнение

Для каких из следующих систем подходит или не подходит микроядерная архитектура и почему? Подсказка: учитывайте сверхвозможности и слабости архитектуры и предметную область.

Онлайн-аукцион, где пользователи делают ставки

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

→ Решение на с. 307

В заключение

Благодаря вашим усилиям в Going Green стали оценивать устройства быстро и точно, а рентабельность значительно выросла. Этот случай показывает, насколько полезными могут быть микроядерные архитектуры. В задачах, требующих настраиваемой и стабильной системы, микроядерную архитектуру трудно превзойти.

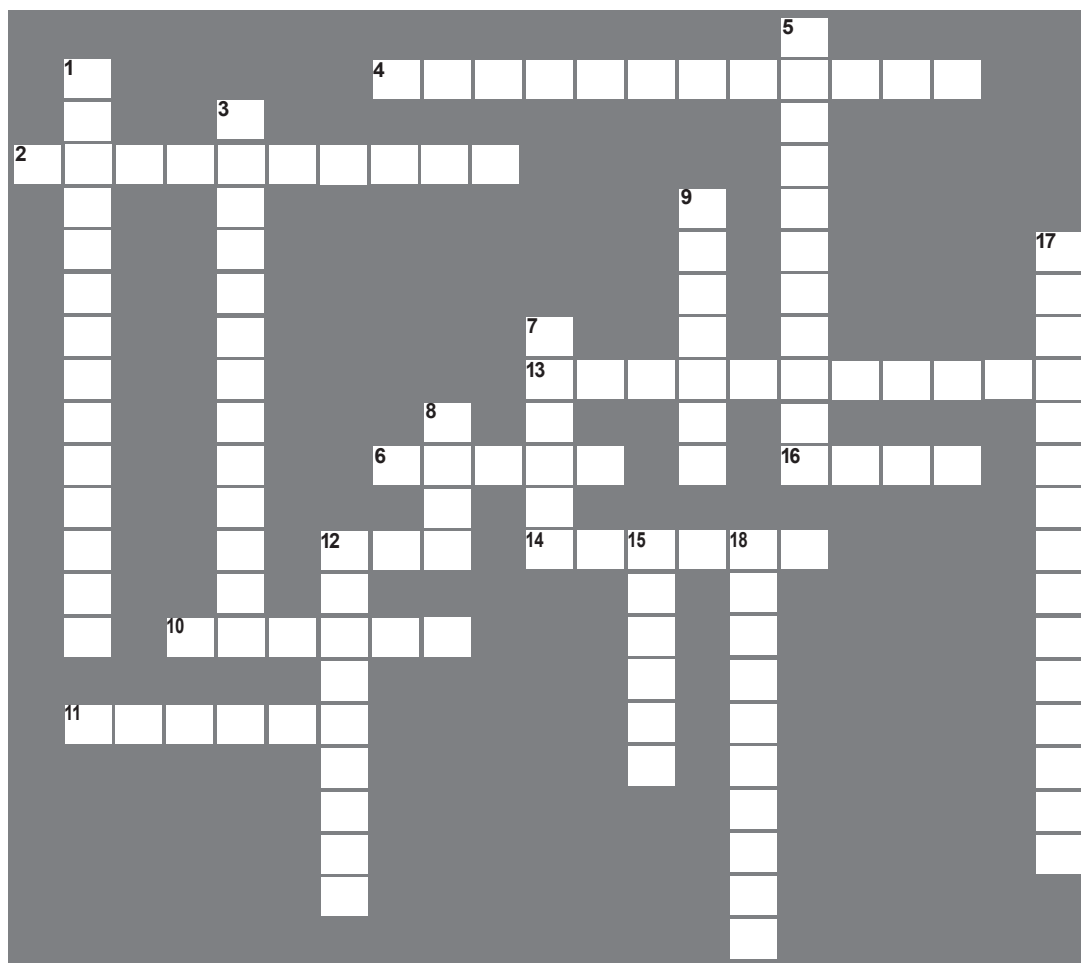
Ключевые моменты

- Микроядерная архитектура дает структурированный способ работы с настройками с помощью плагинов.
- Микроядерная архитектура состоит из двух основных частей: ядра и одного или нескольких плагинов.
- Ядро системы содержит минимальную функциональность и редко изменяется.
- Плагины служат для настройки и/или расширения поведения системы.
- Как правило, плагины взаимодействуют только с ядром, но не друг с другом.
- Чтобы плагины могли взаимодействовать друг с другом, ядро должно взять на себя роль посредника и решать проблемы, связанные с версиями и зависимостями, и, по сути, стать уровнем интеграции.
- Микроядерные архитектуры могут быть монолитными или распределенными, реализованными в виде сервисов.
- В монолитной архитектуре ядро и плагины должны быть написаны на одном языке.
- Плагины могут вызываться синхронно (например, в стиле REST) или асинхронно (с помощью потоков в монолитной архитектуре или обмена сообщениями в распределенной). Независимо от способа вызова плагины в распределенной архитектуре можно реализовать в различных технологических стеках.
- Производительность монолитных плагинов обычно выше, поскольку вызовы выполняются в рамках одного процесса.
- Микроядерные монолиты имеют типичные ограничения, присущие всем монолитам, такие как ограниченные масштабируемость и эластичность.
- Распределенные микроядерные архитектуры имеют улучшенную масштабируемость, используя несколько процессов и поддерживая масштабируемые взаимодействия (события).
- Микроядерные архитектуры хорошо подходят для систем, решающих часто меняющиеся отдельные задачи.
- Если ядро системы с микроядерной архитектурой часто меняется, то это означает, что архитекторы выбрали неправильный архитектурный стиль или неверно разделили работу.
- Микроядерный стиль имеет широкое применение, например, в IDE, обработке текста, инструментах сборки и развертывания, интеграции, приложениях для страхования и утилизации электроники.



КроссВорд по Микроядерной архитектуре

Готовы проверить свои знания о микроядерной архитектуре?
Разгадайте этот кроссворд!



По горизонтали

2. Постоянно меняющийся.
4. Микроядерная архитектура легко _____ путем расширения ее возможностей.
6. Для взаимодействующих плагинов ядро играет роль _____ интеграции.
10. Плагины можно писать в разных технологических _____.
11. Инструмент, применяющий к коду правила оформления архитектуры.
12. Количество типов основных компонентов в микроядерной архитектуре.
13. Между взаимодействующими плагинами возникает _____.
14. Плагины выполняют _____ при взаимодействии с ядром.
16. То, к чему подключаются плагины.

По вертикали

1. Плагины обычно _____ независимо друг от друга.
3. Монолитная архитектура _____ и плагины, и ядро.
5. Микроядерная архитектура хорошо подходит для систем с часто _____ правилами и поведением.
7. Микроядерные архитектуры часто поддерживают несколько _____ программирования.
8. IDE — инструмент для написания _____.
9. Используя их, можно добавлять новые возможности.
12. Шаблон проектирования, часто применяемый для реализации микроядерной архитектуры.
15. Момент, когда многие монолиты загружают плагины.
17. Eclipse — пример IDE, где плагины _____ друг с другом.
18. _____ плагины дают хорошую производительность.

→ Решение на с. 308

СТАНЬТЕ архитектором. Решение

На с. 281



Архитекторы не могут не анализировать. Попробуйте и вы определить три важные архитектурные характеристики для каждого из трех сервисов в новой архитектуре Going Green.

Открытые
пользовательские
интерфейсы

Архитектурные характеристики

Масштабируемость

Доступность

Архитектурные характеристики

Компании Going Green требуется быстро внедрять поддержку новых устройств, не затрагивая существующие.

Гибкость (развертываемость, тестируемость, ...)

Удобство сопровождения

Оценка
устройства

Утилизация/
перепродажа

Архитектурные характеристики

Защищенность

Целостность данных

Контролируемость



Упражнение Решение

На с. 286

«Микроядерность» проявляется повсюду. Попробуйте поместить инструменты и категории приложений, перечисленные ниже, в правильное место на спектре, определив, насколько они «микроядерные». Подсказка: степень «микроядерности» каждой системы зависит от ее полезности без плагинов.

На этом конце основная функциональность реализуется плагинами.

На этом конце основная функциональность реализуется без плагинов.





Упражнение Решение

На с. 290

Команда разработки сервиса оценки в Going Green должна выбрать физическую архитектуру — **монолитную** (ядро и плагины развертываются вместе) или **распределенную** (плагины развертываются отдельно от ядра). Помогите им проанализировать компромиссы: перечислите плюсы и минусы каждого варианта. *Подсказка: подумайте, как каждый вариант повлияет на архитектурные характеристики.*

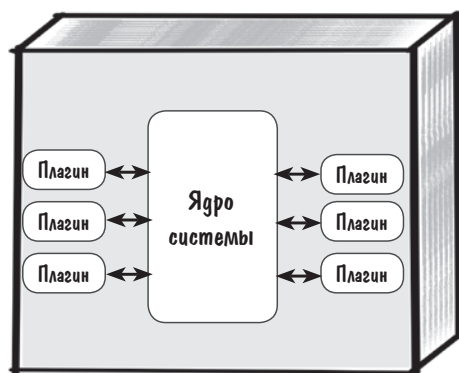
Плюсы

Производительность

Простота

Одноплатформенные монолиты проще распределенных архитектур

Монолитная



Минусы

Одна единица развертывания

Масштабируемость

Монолитную архитектуру сложнее масштабировать.

Плюсы

Масштабируемость

Выбор стиля взаимодействия (синхронный/асинхронный)

Распределенные архитектуры позволяют выбирать стиль взаимодействия (и имеют больше компромиссов).

Распределенная



Минусы

Производительность

Сложность

Как вы узнали в главе 5, распределенные архитектуры более сложные.



Упражнение Решение

На с. 301

Для каких из следующих систем подойдет микроядерная архитектура и почему? *Подсказка: учитывайте сверхвозможности и слабости архитектуры и предметную область.*

Онлайн-аукцион, где пользователи делают ставки

Почему? Этой системе нужна масштабируемость, но она редко изменяется, поэтому микро-ядерная архитектура для нее не подходит.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? Правила переводов в разных странах могут различаться. Микроядро — одно из решений этой проблемы. (Однако с микроядерным монолитом могут возникнуть проблемы с масштабируемостью).

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? Сумея сосредоточить изменения в плагирах и избежать нестабильности ядра, компания сможет выпускать новые функции по отдельности.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? В отсутствие веских причин для подгонки и вследствие простоты микроядерная архитектура не подходит для этой системы.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

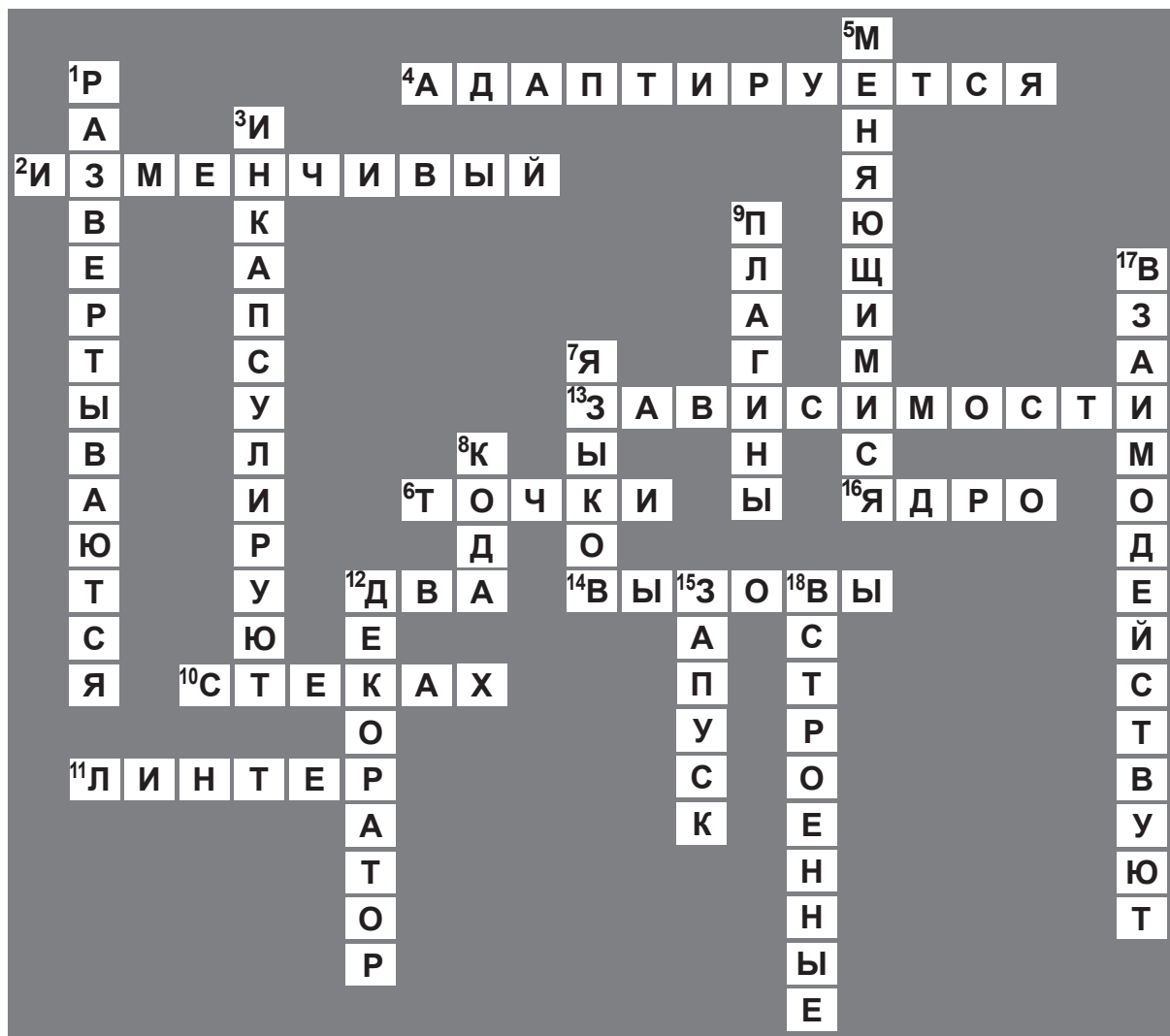
Почему? Если клиенты, устройства или другие части требуют частой настройки, то микроядерная архитектура поможет их реализовать.

- ☐ Хорошо подходит
☒ Может использоваться
☐ Не подходит



Кроссворд по микроядерной архитектуре. Решение

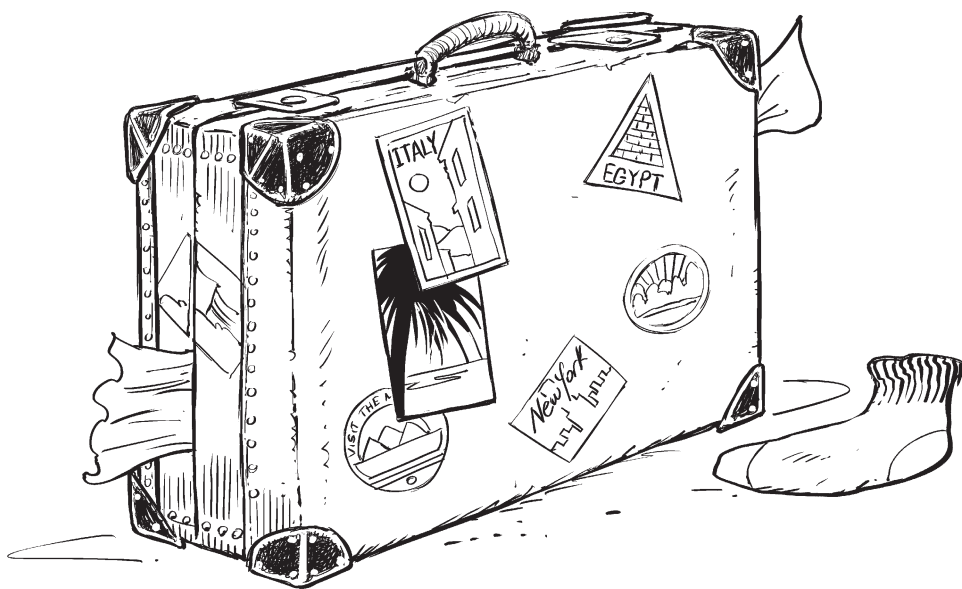
На с. 303



9

Сделай сам

Приложение для путешественников TripEZ



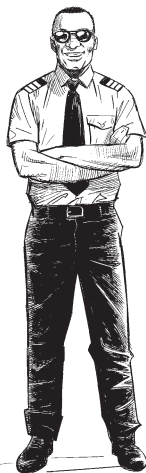
Готовы продолжить знакомство с архитектурой ПО? В этой главе вы будете играть роль архитектора. Вам предстоит определять архитектурные характеристики, создавать архитектуру, принимать решения и выбирать между многоуровневой, модульной и микроядерной архитектурами. Упражнения в этой главе помогут вам лучше понять, что делает архитектор программного обеспечения, и покажут, как много вы узнали. Ваша задача — создать архитектуру для стартапа, разрабатывающего сайт для путешественников.

Облегчаем возможность путешествий

Вас только что приняли на работу в роли архитектора ПО в новый интересный стартап под названием TripEZ, цель которого — облегчить путешествия, особенно для тех, кто много путешествует. Приложение имеет вид панели управления поездками, отображающей все позиции бронирования, сгруппированные по поездкам.

Список требований к TripEZ

- ☐ Система должна постоянно проверять наличие у клиента писем, связанных с поездкой.
- ☐ Система должна взаимодействовать с системами партнеров (туристических агентств, авиакомпаний, отелей и компаний по прокату автомобилей) и обновлять информацию о поездке, в том числе задержки, отмены, уточнения и изменение номеров выходов на посадку. Обновления должны появляться в приложении в течение пяти минут.
- ☐ Пользователи могут вручную добавлять, обновлять или удалять бронирование.
- ☐ Пользователи могут группировать элементы в панели управления по поездкам. По завершении поездки элементы автоматически удаляются с панели.
- ☐ Пользователи могут делиться информацией о поездках в стандартных социальных сетях и пересылать ее конкретным людям.
- ☐ Система должна иметь UI с широчайшим функционалом, доступный на всех платформах.
- ☐ В конце года система должна предоставлять отчеты с широким спектром показателей о поездках за этот год.
- ☐ На основе поездок система должна собирать аналитические данные, такие как предпочитаемые направления, авиакомпании и отели, а также частота отмен и изменений.
- ☐ Желательно выпустить TripEZ через полгода — к дате открытия важной выставки.



**Будьте внимательны,
это очень важно.**

*Знакомьтесь — это Трэвис,
бывший пилот, а ныне консуль-
тант TripEZ. У него есть еще
несколько важных требований.*

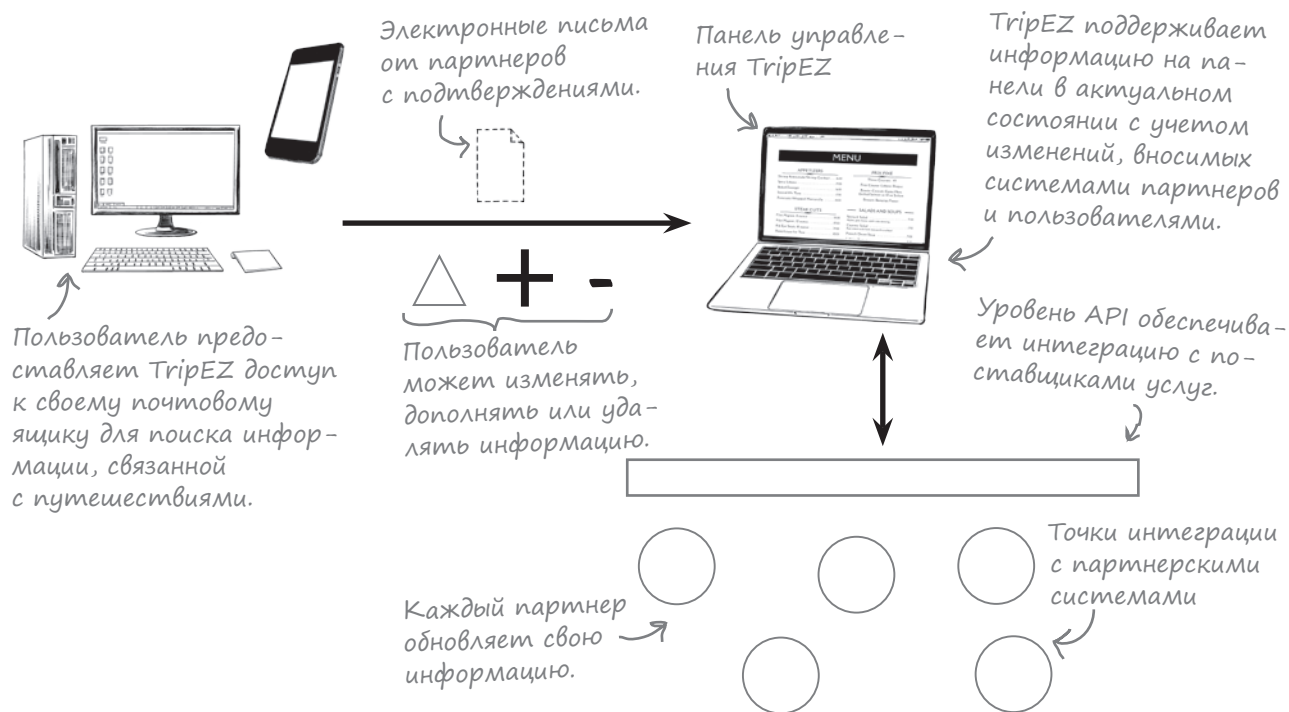
«TripEZ должна интегрироваться с системами, используемыми в туристической индустрии, в том числе на международном уровне.

Она должна интегрироваться с туристическим агентством, выбранным пользователем (если такой выбор сделан), чтобы быстро решать проблемы.

Наконец, система должна быть постоянно доступна. Время незапланированного простоя не должно превышать пяти минут в месяц».

Путь пользователя через TripEZ's

Теперь разберемся с требованиями и рассмотрим процесс работы путешественника с системой TripEZ.

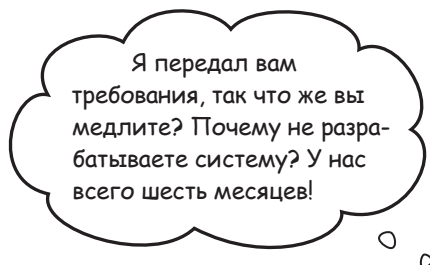


Возьмите в руку карандаш

Учитывая требования к TripEZ, перечислите задачи, которые вам придется решить при создании архитектуры.

Решение на с. 326

Планирование архитектуры



Трэвису не терпится увидеть хоть какой-то прогресс, но не позволяйте ему помешать вам создать надежную и хорошо продуманную архитектуру.

Сначала нужно создать архитектуру.

Как вы уже знаете, архитектура — важная часть любой программной системы. Без нее система едва ли сможет достичь своих целей.

Прежде чем приступить к разработке кода, нужно создать архитектуру. А для этого мы должны вернуться к главе 1, в которой узнали о четырех измерениях архитектуры программного обеспечения.

Не волнуйтесь, мы создадим систему. Но сначала нужно понять, что мы создаем.

План действий архитектора

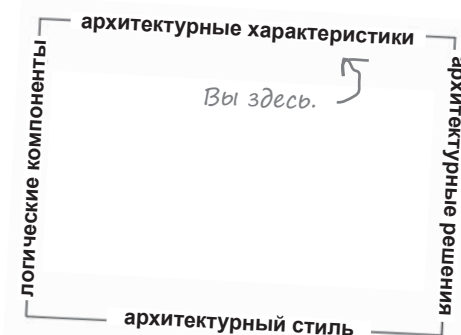
Создадим архитектуру TripEZ, выполнив шаги, описанные в предыдущих главах, и преобразовав требования в архитектуру.



Данная схема послужит вам планом решения задачи, поэтому привыкайте к ней. Вы пройдете каждый из указанных в ней шагов, выполняя задания на следующих страницах.

Желаем успехов! TripEZ рассчитывает на вас.

Шаг 1: определение архитектурных характеристик



«TripEZ должна интегрироваться с системами, используемыми в туристической индустрии, в том числе на международном уровне.

Она должна интегрироваться с туристическим агентством, выбранным пользователем (если такой выбор сделан), чтобы быстро решать проблемы.

Наконец, система должна быть постоянно доступна. Время незапланированного простоя не должно превышать пяти минут в месяц.»

Мы скопировали требования, чтобы вам было проще определить основные архитектурные характеристики.

Список требований TripEZt

- ☐ Система должна постоянно проверять наличие у клиента писем, связанных с поездкой.
- ☐ Система должна взаимодействовать с системами партнеров (туристических агентств, авиакомпаний, отелей и компаний по прокату автомобилей) и обновлять информацию о поездке, в том числе задержки, отмены, уточнения и изменение номеров выходов на посадку. Обновления должны появляться в приложении в течение пяти минут.
- ☐ Пользователи могут вручную добавлять, обновлять или удалять бронирование.
- ☐ Пользователи могут группировать элементы в панели управления по поездкам. По завершении поездки элементы автоматически удаляются с панели.
- ☐ Пользователи могут делиться информацией о поездках в стандартных социальных сетях и пересылать ее конкретным людям.
- ☐ Система должна иметь UI с широчайшим функционалом, доступный на всех платформах.
- ☐ В конце года система должна предоставлять отчеты с широким спектром показателей о поездках за этот год.
- ☐ На основе поездок система должна собирать аналитические данные, такие как предпочитаемые направления, авиакомпании и отели, а также частота отмен и изменений.
- ☐ Желательно выпустить TripEZ через полгода — к дате открытия важной выставки.



Упражнение

В главе 2 мы показали этот прием ограничения количества архитектурных характеристик. Вернитесь на с. 102, чтобы вспомнить, как его использовать.

Топ-3	Движущие характеристики	Неявные характеристики
-------	-------------------------	------------------------

☐

осуществимость (цена/время)

☐

защищенность

☐

удобство сопровождения

☐

наблюдаемость

☐

↖ Это неявные характеристики. Перенесите их в колонку «Движущие характеристики», если считаете их важными для успеха.

☐

☐

↖ Отметьте три самые важные (в любом порядке).

Вернитесь к главе 2, чтобы освежить в памяти определения этих общих архитектурных характеристик.

Возможные кандидаты в архитектурные характеристики

производительность

целостность данных

развертываемость

отзывчивость

согласованность данных

тестируемость

доступность

адаптируемость

конфигурируемость

отказоустойчивость

расширяемость

настраиваемость

масштабируемость

интегрируемость

восстанавливаемость

эластичность

параллелизм

контролируемость

→ Решение на с. 325

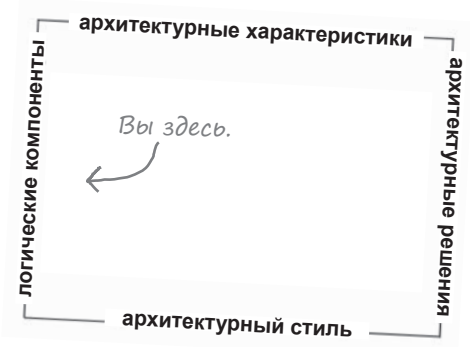
Шаг 2: определение логических компонентов

Отлично! Выявив важные архитектурные характеристики TripEZ, можно переходить к определению логических компонентов.

Опираясь на требования и рабочий процесс, описанные выше, примените подход, представленный в главе 4, и выявите субъекты и их действия. А затем (на следующей странице) определите как можно больше логических компонентов.

Помочь вам выполнить данное упражнение может эта дополнительная информация.

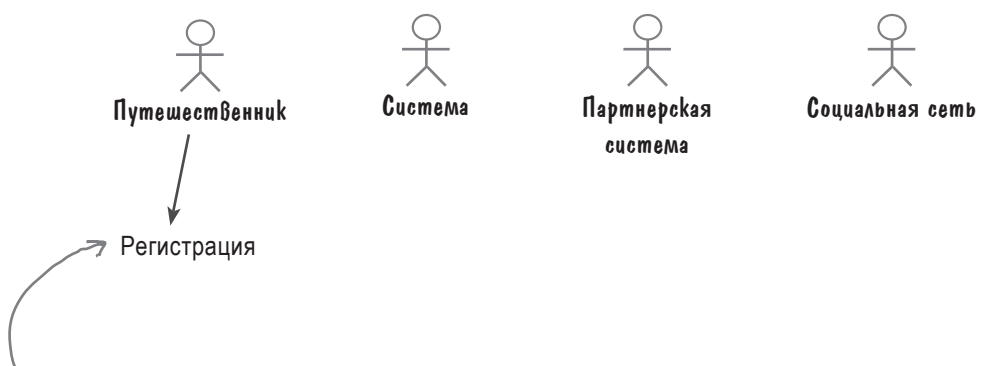
- Регистрируясь в TripEZ, пользователи предоставляют учетные данные, чтобы туристические сервисы могли своевременно сообщать о задержках, отменах и т. д.
- Если точка интеграции партнерского сервиса не предоставляет обновлений в течение пятиминутного окна, то система должна сама запросить его.
- Обновления, особенно в мобильном приложении, должны содержать минимум данных, чтобы учесть потенциально неустойчивую сотовую связь.
- TripEZ не может нести ответственность за доступность точки интеграции; в случае неудачи вызова система должна сообщить об ошибке, не замалчивая ее (чтобы пользователь не решил, что обновление прошло успешно).



Это позволит системе работать с сервисами, которые не соответствуют установленным пороговым значениям.

Возьмите в руку карандаш

Используя подход «субъекты/действия», определите действия для каждого из перечисленных субъектов.



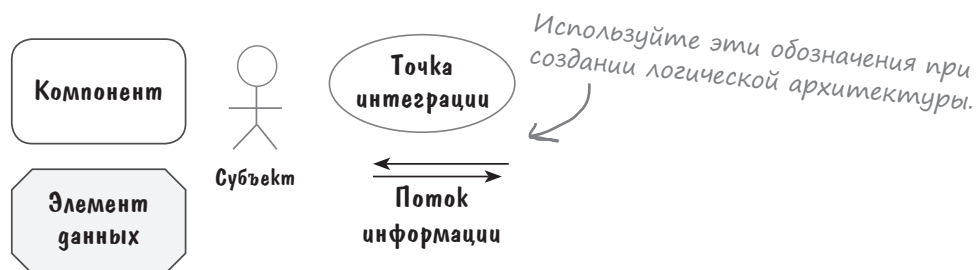
Пример от нас, но путешественник может выполнять и другие действия.

Решение на с. 326



Упражнение

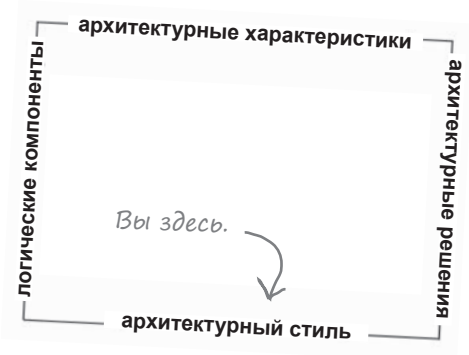
Нарисуйте логические компоненты и связи между ними.



Решение на с. 327

Шаг 3: Выбор архитектурного стиля

Используя знания о многоуровневой, модульно-монолитной и микроядерной архитектурах, проанализируйте плюсы и минусы этих архитектур применительно к системе TripEZ. Кроме того, вам нужно вспомнить требования, логические архитектуры и звездные рейтинги этих стилей (мы добавили их ниже). По результатам анализа выберите наиболее подходящий архитектурный стиль.



Многоуровневая

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★
Тестируемость	★ ★
Развертываемость	★
Простота реализации	★ ★ ★ ★ ★
Развиваемость	★
Производительность	★ ★ ★
Масштабируемость	★
Эластичность	★
Отказоустойчивость	★
Общая стоимость	\$

Модульно-монолитная

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★ ★ ★
Тестируемость	★ ★ ★
Развертываемость	★ ★ ★
Простота реализации	★ ★ ★ ★
Развиваемость	★ ★ ★
Производительность	★ ★ ★
Масштабируемость	★
Эластичность	★
Отказоустойчивость	★
Общая стоимость	\$ \$

Микроядерная

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★ ★ ★
Тестируемость	★ ★ ★
Развертываемость	★ ★ ★
Простота реализации	★ ★ ★ ★
Развиваемость	★ ★ ★
Производительность	★ ★ ★
Масштабируемость	★
Эластичность	★
Отказоустойчивость	★
Общая стоимость	\$

Чем больше звезд, тем лучше поддерживается характеристика.

Извините за мелкий шрифт, но вы всегда можете вернуться к соответствующим главам и посмотреть рейтинги там.

Выбрать архитектурный стиль для TripEZ вам помогут эти соображения.

- Вернитесь к схеме логической архитектуры и попробуйте выделить отдельные технические или бизнес-задачи. Если у вас это получится, то хорошим выбором может стать многоуровневая или модульно-монолитная архитектура.
- Подумайте о точках интеграции. Если их много и каждая имеет свою логику, то более подходящей может оказаться микроядерная архитектура.



Упражнение

Опишите плюсы и минусы каждого архитектурного стиля. Это поможет вам сделать выбор, наиболее подходящий для TripEZ.

Многоуровневая монолитная архитектура

Плюсы

Минусы

Модульно-монолитная архитектура

Плюсы

Минусы

Микроядерная архитектура

Плюсы

Минусы

Укажите здесь ваш выбор:

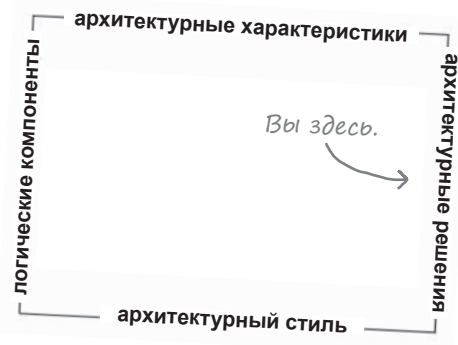


Решение на с. 328

Шаг 4: документирование решения

Поздравляем вас с выбором архитектурного стиля для TripEZ. Теперь попробуйте *объяснить* этот выбор и задокументируйте свое архитектурное решение.

Как вы узнали в главе 3, *записи об архитектурных решениях* (architectural decision record, ADR) — эффективный способ документирования. Запишите ADR на следующей странице, предположив, что это ваше 11-е архитектурное решение.



Вернитесь к главе 3, чтобы вспомнить, что такое ADR.



Что указывать в разделе «Последствия», если архитектурное решение не имеет никаких последствий?

Каждое архитектурное решение имеет последствия.

Это может быть стоимость или необходимость жертвовать производительностью ради защищенности. В любом случае *каждое* архитектурное решение имеет последствия.

Вспомните проведенный анализ компромиссов. Любой компромисс подразумевает последствия — то, от чего вы готовы отказаться (или с чем согласиться), чтобы получить нечто лучшее. Раздел «Последствия» отлично подходит для документирования компромиссов и последствий решения.

Не найдя никаких последствий в своем архитектурном решении, продолжайте искать, поскольку они есть всегда.



Упражнение

Запись об архитектурном решении

Заголовок:

Статус: Предложено

← Мы начали
за вас.

Контекст:

Решение:

Последствия:

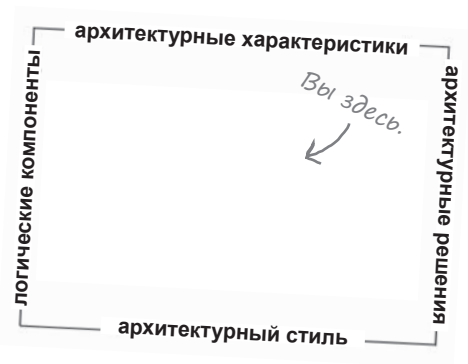
← Какие последствия несет ваше решение?
На какие компромиссы вы готовы пойти?

→ Решение на с. 329–331

Шаг 5: составление схемы архитектуры

Пришло время объединить все четыре измерения и показать свое видение архитектуры TripEZ. В этом последнем упражнении вы должны нарисовать схему архитектуры, используя обозначения, перечисленные ниже.

Мы намеренно не отводим много места для схемы. Архитекторы часто рисуют очень подробные архитектурные схемы, однако мы просим вас набросать лишь *краткое физическое представление* о пользовательских интерфейсах, базах данных и компонентах, составляющих архитектуру, и о связях между ними.



Обозначения для схемы физической архитектуры

Пользовательский интерфейс



Для представления *пользовательского интерфейса* нарисуйте экран компьютера и укажите, какие категории *пользователей* взаимодействуют с ним. Если для настольных компьютеров и мобильных устройств используются разные интерфейсы, то нарисуйте два экрана.

Компонент

Для обозначения *компонентов* используйте овалы. Они должны соответствовать логическим компонентам, выявленным в предыдущем упражнении. Дайте компонентам осмысленные и описательные названия.

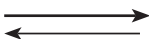
Уровень

Для *уровней* (если они нужны) используйте прямоугольники. Дайте каждому уровню осмысленное название.

База данных



Для представления каждой *физической базы данных* в вашем решении нарисуйте цилиндр.



Стрелки обозначают связь с пользовательским интерфейсом и базой данных.

Не стесняйтесь снабжать свою схему комментариями и пояснениями.



Упражнение

Нарисуйте схему физической архитектуры TripEZ, используя обозначения с предыдущей страницы.

Решение на с. 332–334

Нет правильных (или неправильных) ответов!

Поздравляем! Вы только что создали архитектуру!

Далее мы покажем вам «решения» упражнений. Мы использовали кавычки, поскольку представленные решения — лишь одни из *множества* возможных. Ведь в архитектуре ПО нет правильных или неправильных ответов: все дело в анализе компромиссов и умении обосновать свои решения.

Сравните свои ответы с нашими. Возможно, вы заметите, что сделали что-то не так, как мы, и наш вариант лучше, или, наоборот, утвердитесь в правильности своего выбора. Мы покажем наши варианты многоуровневой, модульно-монолитной и микроядерной архитектуры для TripEZ, так как все они являются жизнеспособными.

Разработка архитектуры ПО — *непрерывный* процесс обучения. Каждая новая задача подразумевает свой набор условий, ограничений и проблем. Не существует универсальной архитектуры, все зависит от вас, архитектора, — именно вы должны придумать архитектуру, наиболее подходящую для вашей ситуации.

Ключевые Моменты

- Анализируя бизнес-требования, всегда собирайте дополнительную информацию, которую могут дать заинтересованные стороны или спонсор проекта.
- Не существует «контрольного списка» для создания архитектуры, но есть четыре измерения архитектуры ПО (представленные в главе 1), которые послужат хорошим ориентиром.
- Определение движущих архитектурных характеристик требует анализа бизнес-требований и технических ограничений.
- Неявные архитектурные характеристики становятся *движущими*, если важны для успешной работы системы.
- Каждая движущая характеристика должна быть связана с каким-то требованием или потребностью.
- Определяя логические компоненты и создавая логическую архитектуру, старайтесь не добавлять физические детали, такие как сервисы, базы данных, очереди и пользовательские интерфейсы, — они относятся к *физической архитектуре*.
- При выборе архитектурного стиля учитывайте его свойства, особенности предметной области и выявленные вами основные архитектурные характеристики.
- *Гибридные архитектуры* (сочетающие два или более архитектурных стиля) встречаются очень часто. Если вы используете одну из них, то убедитесь, что она соответствует вашим архитектурным характеристикам.
- *Записи об архитектурных решениях* (ADR) — отличный способ задокументировать выбор. В них указываются причины решений и результаты анализа компромиссов.
- Составляя схему физической архитектуры, не забудьте добавить в нее все компоненты, которые вы определили в логической архитектуре.
- Помните, что в архитектуре программного обеспечения нет правильных или неправильных ответов. Если вы можете обосновать свои архитектурные решения, то находитесь на правильном пути.



Упражнение Решение

На с. 315

В главе 2 мы показали этот прием ограничения количества архитектурных характеристик. Вернитесь на с. 102, чтобы вспомнить, как его использовать.

Топ-3	Движущие характеристики	Неявные характеристики
☐	<i>У нас всего полгода</i> осуществимость	<u>осуществимость (цена/время)</u>
☒	производительность	<u>защищенность</u>
☒	масштабируемость	<u>удобство сопровождения</u>
☐	эластичность	<u>наблюдаемость</u>
☒	расширяемость/простота развития	<i>← TripEZ должна интегрироваться с множеством сервисов электронной почты и сайтов социальных сетей.</i>
☐	доступность	
☐		

Возможные кандидаты в архитектурные характеристики

производительность	целостность данных	развертываемость
отзывчивость	согласованность данных	тестируемость
доступность	адаптируемость	конфигурируемость
отказоустойчивость	расширяемость	настраиваемость
масштабируемость	интегрируемость	восстанавливаемость
эластичность	параллелизм	контролируемость

Возьмите в руку карандаш

Решение

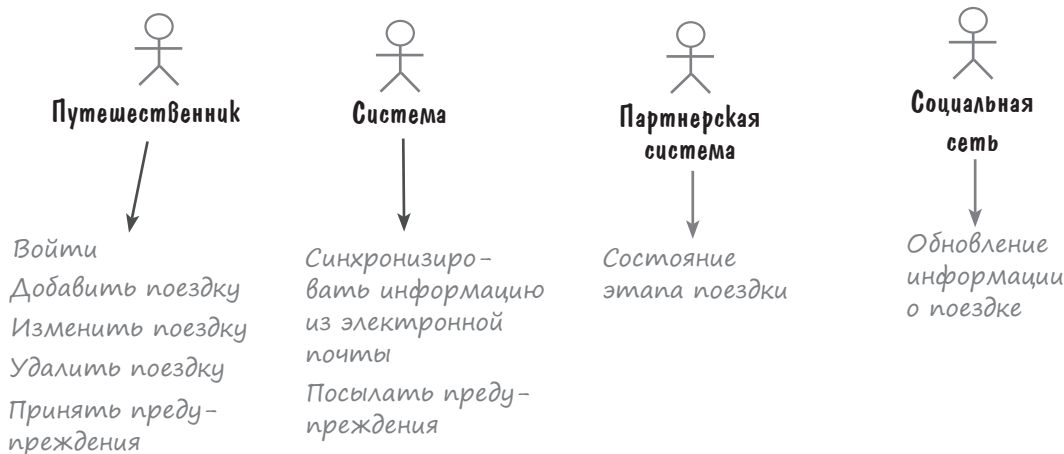
Учитывая требования к TripEZ, перечислите задачи, которые вам придется решить при создании архитектуры.

- Своевременная доставка уведомлений.
- Поддержка большого количества пользователей.
- Выбор способа управления различными точками интеграции.
- Интеграция с учетными записями в социальных сетях.

Возьмите в руку карандаш

Решение

Используя подход «субъекты/действия», определите действия для каждого из перечисленных субъектов.

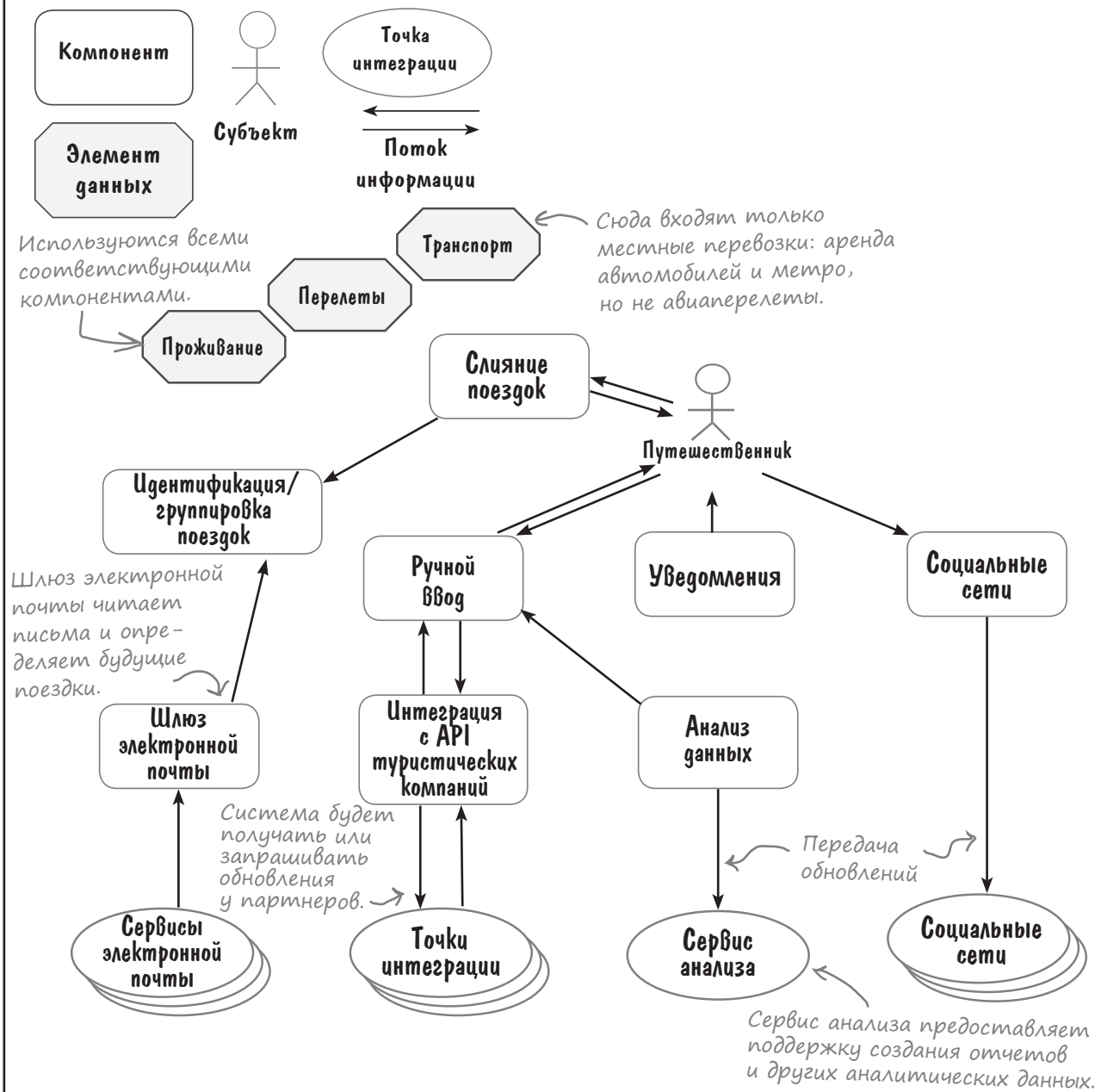




Упражнение Решение

На с. 317

Нарисуйте логические компоненты и связи между ними.





Опишите плюсы и минусы каждого архитектурного стиля. Это поможет вам выбрать наиболее подходящий для TripEZ.

Многоуровневая монолитная архитектура

Плюсы

Многоуровневая монолитная архитектура довольно проста и потому легко осуществима.

Разделение по техническим задачам облегчает интеграцию со сторонними сервисами.

Монолитные архитектуры отличаются высокой производительностью, что является одним из приоритетов TripEZ.

Минусы

Изменения в предметной области могут быть затруднены, поскольку она распределена по нескольким уровням.

По мере развития системы TripEZ придется постоянно следить за ней, чтобы обеспечить ее масштабирование.

Модульно-монолитная архитектура

Плюсы

Модульно-монолитная архитектура довольно проста и потому легко осуществима.

Разделение по бизнес-задачам подходит лучше, поскольку избавляет от необходимости делить предметную область по техническим задачам.

Монолитные архитектуры отличаются высокой производительностью, что является одним из приоритетов TripEZ.

Минусы

Добавление интеграции с новыми партнерами может потребовать внесения изменений в большое количество бизнес-задач.

По мере развития системы TripEZ придется постоянно следить за ней, чтобы обеспечить ее масштабирование.

Микроядерная архитектура

Плюсы

Монолитная микроядерная архитектура довольно проста и потому легко осуществима.

Микроядерная архитектура хорошо подходит для этой работы, учитывая потребность TripEZ в гибкой интеграции.

Монолитные микроядерные архитектуры отличаются высокой производительностью, что является одним из приоритетов TripEZ.

Минусы

Любые изменения, затрагивающие ядро, должны тщательно обдумываться.

По мере развития системы TripEZ придется постоянно следить за ней, чтобы обеспечить ее масштабирование.

Укажите здесь ваш выбор:

Подойдет любая из этих архитектур, просто у них разные компромиссы.

Мы покажем вам ADR и архитектурные схемы для всех трех вариантов!



Упражнение Решение

На с. 321

Запись об архитектурном решении

Заголовок: *О11: Выбор архитектурного стиля «многоуровневый монолит» для системы TripEZ*

Статус: *Предложено*

Контекст:

TripEZ — быстро развивающийся стартап, поэтому для осуществимости нужна простая архитектура. Кроме того, система должна быть легко расширяемой, чтобы интегрироваться с множеством сторонних решений.

Решение:

Решено использовать многоуровневую монолитную архитектуру. Разные части TripEZ не требуют уникальных архитектурных характеристик, поэтому многоуровневого монолита будет достаточно. Основное ограничение — масштабируемость.

Кроме того, разделение системы по техническим задачам облегчает ее расширение.

Последствия:

Выбор монолитной архитектуры в будущем может вызвать проблемы с масштабируемостью.

В многоуровневой архитектуре сложнее вносить изменения в предметные области, поскольку они затрагивают несколько уровней.

Благодаря техническому разделению вносить изменения в технические задачи (например, добавлять поддержку новых пользовательских интерфейсов) будет проще.



Запись об архитектурном решении

Заголовок: *011: Выбор архитектурного стиля «модульный монолит» для системы TripEZ*

Статус: *Предложено*

Контекст:

TripEZ — быстроразвивающийся стартап, которому нужна архитектура, обеспечивающая максимально простой переход на распределенную архитектуру и достаточно простая для реализации в сжатые сроки.

Решение:

Решено использовать модульный монолит. Выбор обусловлен хорошей согласованностью процесса разработки с разделением этой архитектуры по бизнес-задачам.

Совпадение границ ограниченных контекстов и компонентов поможет разработчикам понять организацию системы. Кроме того, система сможет расти вместе с предметной областью.

В нашей организации используется методика предметно-ориентированного проектирования, и выбранный нами архитектурный стиль хорошо согласуется с ней.

Последствия:

Выбор монолитной архитектуры может в будущем вызвать проблемы с масштабируемостью.

В этой архитектуре сложнее вносить изменения в технические задачи (например, пользовательские интерфейсы), поскольку UI охватывает почти все ограниченные контексты.



Упражнение Решение

На с. 321

Запись об архитектурном решении

Заголовок: O11: Выбор микроядерной архитектуры для системы TripEZ

Статус: Предложено

Контекст:

Обратите внимание, что контекст немного другой, хотя это все та же система. Это различие отражает разные приоритеты команды.



По своей сути TripEZ — это интеграционная архитектура, управляющая схожей инфраструктурой, которая поступает от разных партнеров. Она позволяет изолировать и настраивать каждую точку интеграции с помощью плагинов.

Решение:

Решено использовать микроядерную архитектуру.

Время выхода на рынок и расширяемость важны для компании, поэтому микроядерная архитектура с плагинами для интеграции с партнерами позволит разработчикам лучше понять организацию системы и реализовать ее.

Мы решили, что простота монолитной системы перевешивает преимущества распределенных плагинов (но усложняет разработку).

Последствия:

Выбор монолитной архитектуры может вызвать проблемы с масштабируемостью. В будущем мы можем подумать о переходе на распределенные плагины, но сейчас это кажется нам излишним.

Ядро можно разделить так, что пользовательский интерфейс будет обслуживаться другим микроядром с разными плагинами для разных типов интерфейса.

Следует избегать добавления в ядро часто меняющихся требований. Оно должно оставаться максимально стабильным.

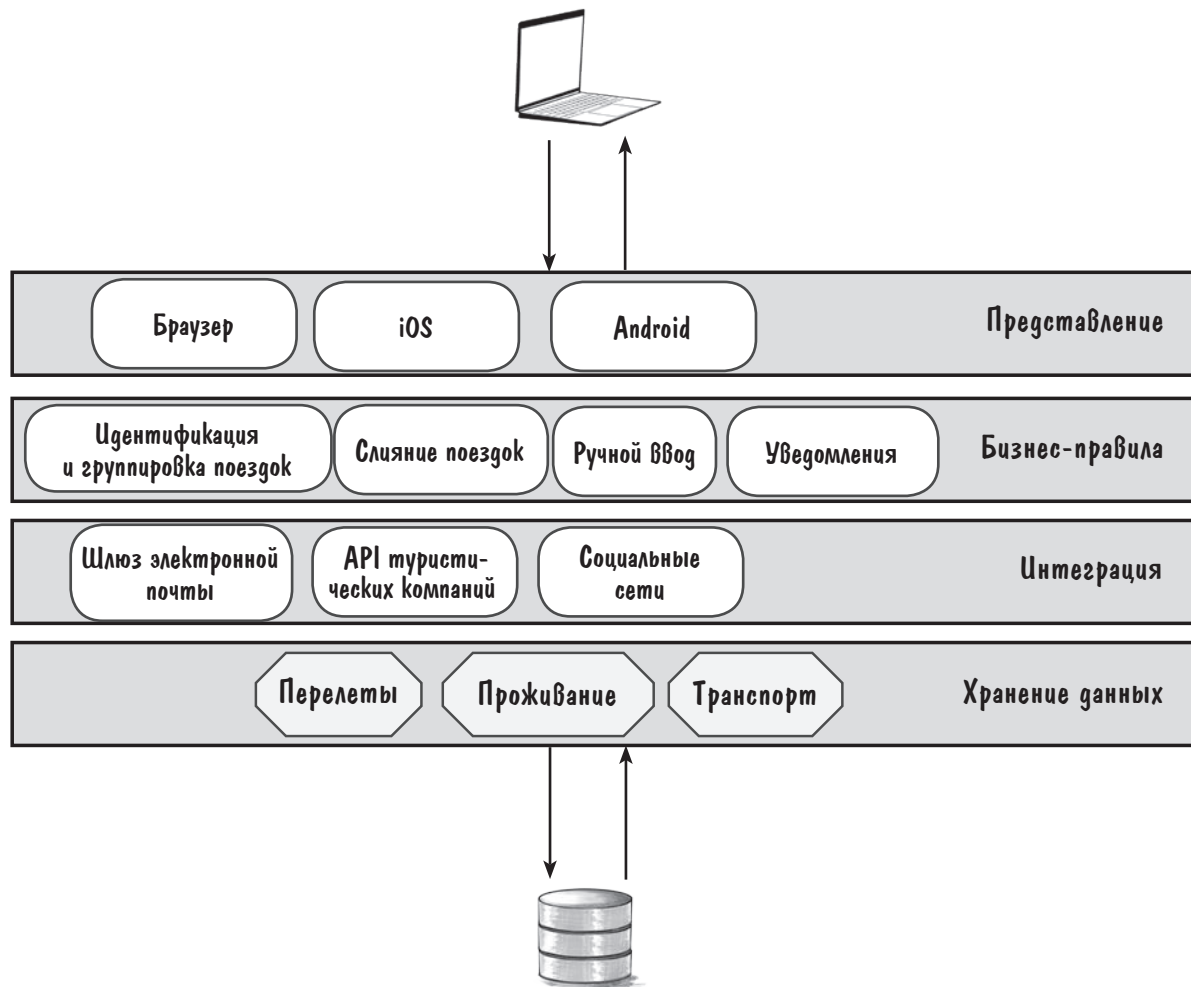


Упражнение Решение

На с. 323

Нарисуйте схему физической архитектуры TripEZ, используя обозначения на с. 322.

Схема многоуровневой монолитной архитектуры





Упражнение Решение

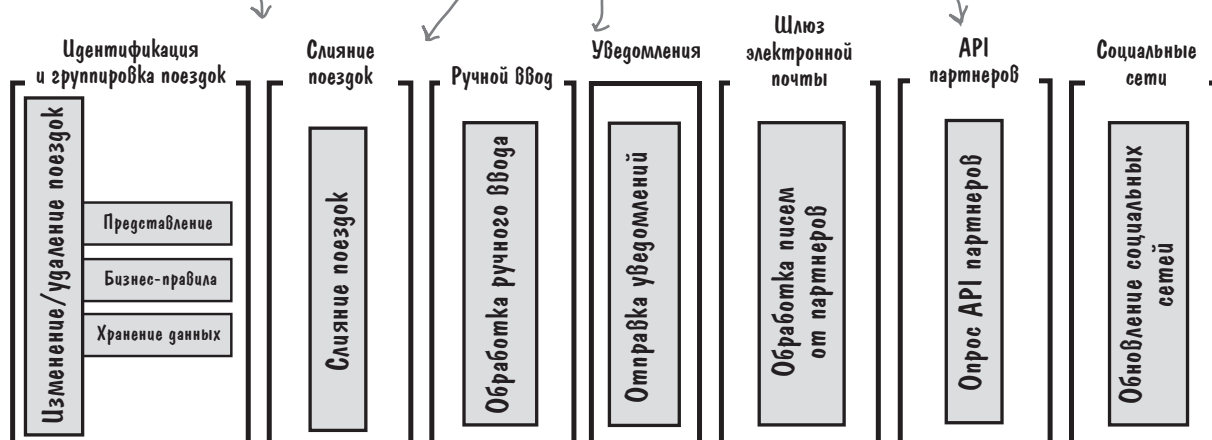
На с. 323

Нарисуйте схему физической архитектуры TripEZ, используя обозначения на с. 322.

Схема модульно-монолитной архитектуры

Помните: это монолит с одной базой данных.

Все эти области могут иметь те же уровни, что и область «Идентификация и группировка поездок». Мы просто не показываем их.



Уровень хранения в области «Ручной ввод» будет управлять всеми данными, такими как рейсы, проживание и т. д.



Для каждой предметной области можно создать отдельную схему в базе данных.



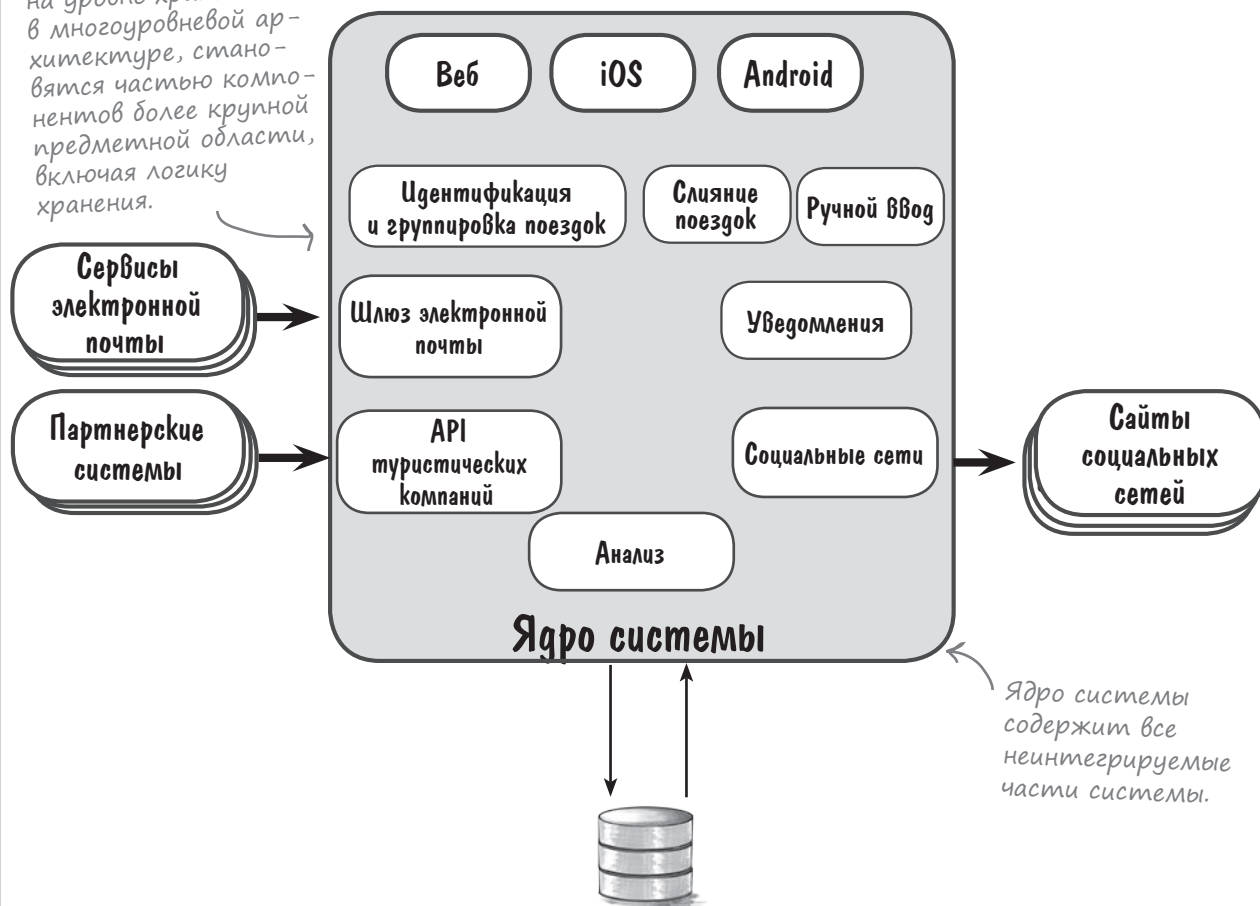
Упражнение Решение

На с. 323

Нарисуйте схему физической архитектуры TripEZ, используя обозначения на с. 322.

Схема микроядерной архитектуры

Данные, указанные на уровне хранения в многоуровневой архитектуре, становятся частью компонентов более крупной предметной области, включая логику хранения.



10

Микросервисная архитектура

Шаг за шагом



Как упростить изменение архитектуры? Потребности бизнеса меняются быстро, и архитектуры ПО должны соответствовать им. В этой главе вы узнаете, как создать гибкую архитектуру, которая может меняться вслед за изменениями требований бизнеса, масштабироваться по мере его роста и сохранять работоспособность даже после системных сбоев. Заинтригованы? В данной главе вы познакомитесь с *микросервисами* — архитектурным стилем, решающим все эти и многие другие проблемы.

Хорошо ли вы себя чувствуете?

StayHealthy, Inc. — компания, которая производит медицинские системы мониторинга состояния пациентов, помогающие следить за частотой пульса, уровнем кислорода, температурой тела, уровнем сахара в крови и многими другими показателями. Еще они определяют, спит пациент или нет, и если что-то идет не так, то сразу уведомляют врача или медсестру.

Последние достижения в медицине привели к появлению новых потребностей в области мониторинга. Поэтому в StayHealthy было решено использовать современные технологии и заменить текущее ПО новой системой под названием MonitorMe. А на роль архитектора для этого проекта они выбрали вас.

Ниже перечислены требования к новой системе. Вам нужно определить, какая архитектура подойдет лучше всего.

Будем надеяться, что вы не заболеете и не окажетесь здесь.



Список требований к StayHealthy MonitorMe

- ☐ Система читает показатели с датчиков и отправляет результаты на единый экран мониторинга.
- ☐ MonitorMe должна анализировать показатели жизнедеятельности каждого пациента и оповещать медицинского работника, если обнаружит изменения, превышающие установленный порог.
- ☐ Система должна записывать изменения всех жизненно важных показателей за последние пять минут. Медицинский работник должен иметь возможность просмотреть эту пятиминутную историю.
- ☐ Медицинские работники выбирают способ оповещения. Сами оповещения могут отправляться на мобильный телефон медсестры или врача или на центральный пост дежурной медсестры.
- ☐ MonitorMe читает данные с восьми различных датчиков. Очень важно, чтобы в случае выхода из строя любого из них остальные по-прежнему контролировались и их показания записывались.
- ☐ Система MonitorMe отслеживает такие показатели, как частота пульса, артериальное давление, уровень кислорода и сахара в крови, частота дыхания, температура тела и состояние сна (спит или бодрствует), а также записывает электрокардиограмму (ЭКГ).
- ☐ Система может измерять показатели множества пациентов (до 500) в пределах одной больницы, то есть в каждом физическом отделении больницы действует своя копия системы (включая данные).



Основываясь на предметной области и требованиях, перечисленных на предыдущей странице, отметьте **пять** характеристик, которые вы считаете **особенно важными** для архитектуры MonitorMe, и укажите, почему они важны.

- ☐ **Тестируемость (простота и полнота тестирования)**

Причина: _____

- ☐ **Отзывчивость (время подготовки ответа на запрос медицинского работника)**

Причина: _____

- ☐ **Развертываемость (сложность выпуска и развертывания изменений)**

Причина: _____

- ☐ **Абстракция (уровень изоляции частей системы друг от друга)**

Причина: _____

- ☐ **Масштабируемость (способность обслуживать больше пользователей или пациентов)**

Причина: _____

- ☐ **Отказоустойчивость (способность продолжать работу при отказе отдельных частей)**

Причина: _____

- ☐ **Целостность данных (согласованность и корректность данных и невозможность их потери)**

Причина: _____

- ☐ **Рабочий процесс (способность системы обрабатывать сложные рабочие процессы)**

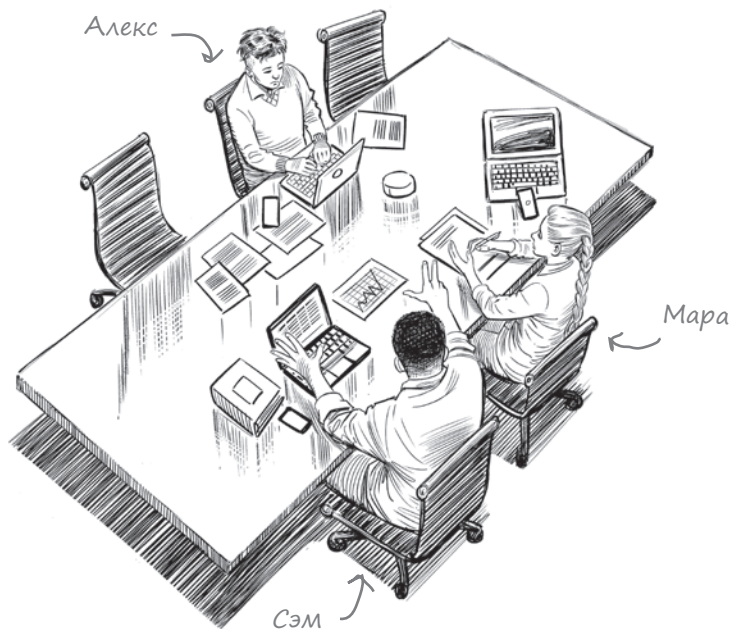
Причина: _____

- ☐ **Параллелизм (способность обрабатывать сразу несколько запросов или операций)**

Причина: _____

—————→ Решение на с. 370

Разговор в кабинете



Подобные предположения — распространенная ловушка. Мы расскажем, как в нее не попасть, подробно объяснив идеи архитектурного стиля микросервисов и его компромиссы.

Сэм. Думаю, все согласны, что для новой системы MonitorMe лучше всего подойдут микросервисы.

Мара. Почему ты так решил?

Сэм. Но это же очевидно! Нам нужны независимые функции мониторинга, высокая отказоустойчивость и возможность всестороннего тестирования. Микросервисы позволяют получить все это и даже больше.

Алекс. А можно подробнее? Я не знаком с микросервисами и понятия не имею, как они поддерживают все то, о чем ты говоришь.

Мара. Я согласна с Алексом. Я понимаю, что нам нужна распределенная архитектура, и микросервисы вполне соответствуют этой цели. Но давайте рассмотрим их более внимательно, прежде чем делать какие-либо выводы.

Сэм. Понимаю, то, что кажется очевидным мне, может быть не очевидным для вас. Важно, чтобы мы все понимали, что такое микросервисы, и тогда *вместе* мы сможем решить, подходят ли они для MonitorMe. Итак, начнем.

Что такое микросервисы

За этим названием скрывается очень многое. Мы расскажем вам, что такое микросервис и чем он отличается от других типов сервисов.



Сервис, как правило, — это отдельно развертываемая единица ПО, решающая какую-либо бизнес-задачу. Например, один сервис мониторинга всех важных показателей в MonitorMe может выполнять несколько функций, включая мониторинг пульса, артериального давления, температуры и т. д. Он совершает довольно много действий, но мы все равно называем его *сервисом*.

↑ Этот крупный сервис следит за всеми важными показателями состояния пациента.

Это довольно маленький сервис, выполняющий только одну функцию, — назовем его «микро»-сервисом.

Приставка *микро-* в слове «микросервис» означает не физический размер, а количество выполняемых функций. Например, сервис мониторинга частоты пульса выполняет только одну функцию: следит за сердечным ритмом. В этом и заключается идея микросервисов. Более крупный сервис мониторинга всех важных показателей, напротив, выполняет множество функций.



Микросервис — это отдельно развертываемая единица программного обеспечения, решающая одну задачу.



Упражнение

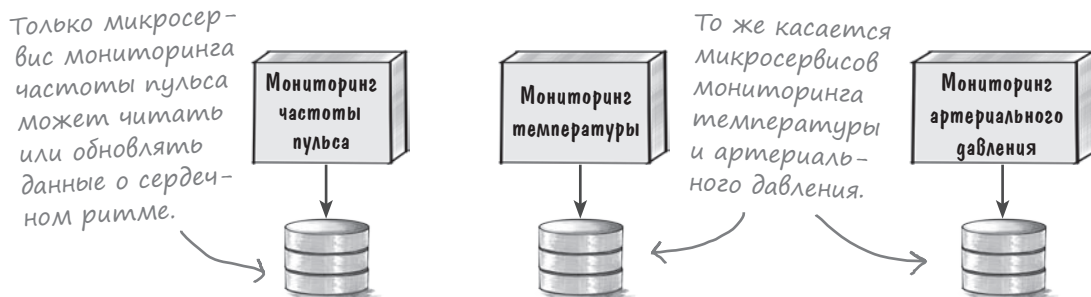
Мы затрудняемся дать определение слова «одноцелевой». Помогите нам, отметив все функции, которые вы считаете одноцелевыми и, следовательно, микросервисами.

- ☐ Добавление фильма в личный список «что посмотреть»
- ☐ Оплата заказа с помощью кредитной карты
- ☐ Составление прогноза продаж и отчета с финансовыми показателями
- ☐ Подача заявки на кредит для приобретения нового автомобиля, о котором вы мечтали
- ☐ Определение стоимости доставки онлайн-заказа

→ Решение на с. 311

Эти данные только мои

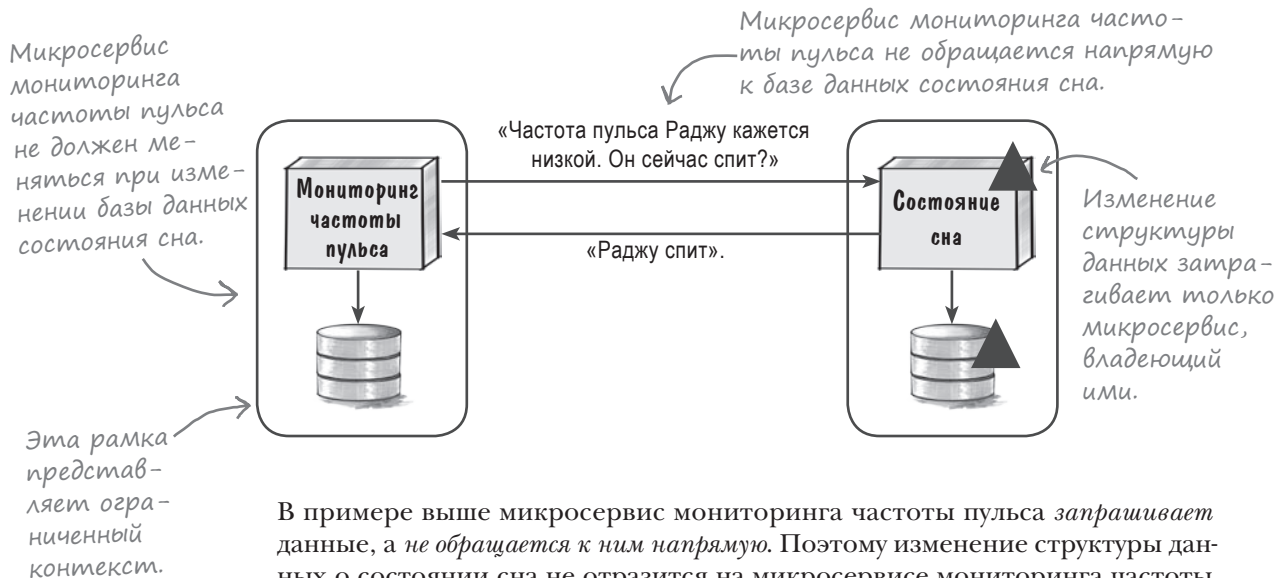
Еще одна особенность микросервисов — они владеют своими данными. Другими словами, каждый микросервис — единственный, кто может напрямую обращаться к своим данным.



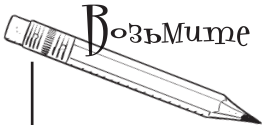
Почему? Основная причина — *управление изменениями*. Представьте 50 микросервисов, работающих с одними и теми же данными. Если один изменит структуру своих данных, к которым обращаются *остальные 49* микросервисов, то *одновременно* придется изменить *все* сервисы. (Как вам такая перспектива?)

Объединение микросервиса с его данными называют **ограниченным контекстом**. Такие контексты помогают управлять изменениями и связями. Если другим микросервисам потребуются данные, которые им не принадлежат, то они должны обратиться к сервису-владельцу.

Ограниченный контекст содержит и микросервис, и все его данные.



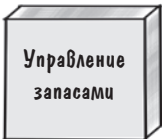
В примере выше микросервис мониторинга частоты пульса запрашивает данные, а не обращается к ним напрямую. Поэтому изменение структуры данных о состоянии сна не отразится на микросервисе мониторинга частоты пульса. В этом и заключается идея ограниченных контекстов.



Возьмите в руку карандаш

Мы создали следующие сервисы для сайта электронной коммерции и хотим определить права собственности на данные и ограниченные контексты. Свяжите приведенные ниже таблицы базы данных с микросервисами, которые должны владеть этими данными.

Это микросервис



Это таблица базы данных



Адрес клиента

Хранит адреса клиентов для выставления счетов и доставки

Пример от нас



Информация о товарах

Хранит информацию о товарах



Стоимость доставки

Используется для расчета стоимости доставки



Список желаний

Список товаров, которые клиент может захотеть приобрести позже



Кошелек клиента

Хранит информацию о кредитных картах и платежах



Запасы

Хранит количество товаров на складе в данный момент



Местонахождение товара

Хранит информацию о местонахождении товаров на складе



Имя клиента

Хранит идентификатор, имя, электронный адрес и номер телефона клиента



Декларации о доставке

Хранит все данные о доставке заказа

Решение на с. 372

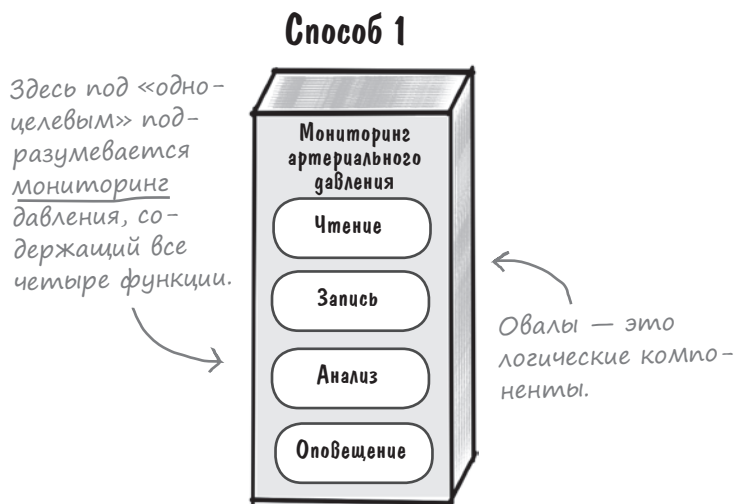
Насколько малы эти «микро»

Определить, насколько большим или маленьким должен быть микросервис, непросто. Вы уже знаете, что *микросервис — это сервис с единственной целью, развертываемый отдельно*. Но как определить размер этой единственной цели?

Для примера рассмотрим функцию измерения артериального давления. Мониторинг важных показателей подразумевает получение данных от медицинских датчиков, запись этих данных, анализ измерений и оповещение медперсонала, если что-то пошло не так. Мы можем смоделировать эту функцию одним из трех способов.

- Способ 1: создать единый микросервис мониторинга, выполняющий все функции мониторинга и оповещения.
- Способ 2: создать два микросервиса — один для сбора и записи данных, а второй для анализа этих данных и оповещения медперсонала.
- Способ 3: создать четыре микросервиса, каждый из которых будет выполнять определенную функцию мониторинга артериального давления.

Здесь под «одноцелевым» подразумевается анализ данных о давлении и оповещение персонала, если что-то не так.



Здесь под «одноцелевым» подразумевается только запись артериального давления.



Расслабьтесь

Сейчас мы не выбираем, а только рассматриваем разные способы создания микросервисов. Как сделать правильный выбор в этой ситуации, вы узнаете чуть позже.

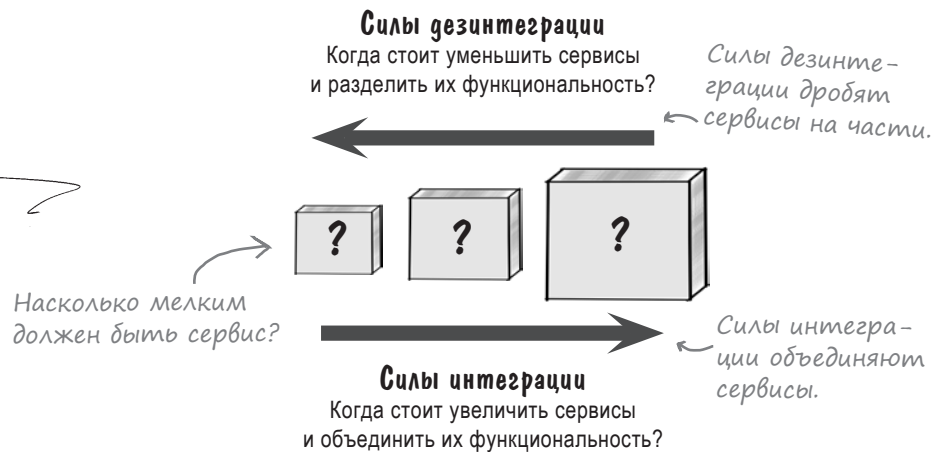


И как узнать, какой из способов использовать для мониторинга артериального давления? Это еще одна из «игр-угадаек», как с логическими компонентами?

Нет, детализация — это не игра-угадка.

Детализация — объем задач, решаемых микросервисом, — важный фактор при определении микросервисов. Слишком детальные микросервисы чаще взаимодействуют друг с другом для решения своих задач, что увеличивает связанность и ухудшает производительность и надежность. Обычно это называют антипаттерном *избыточной детализации*, когда сервисы *настолько* малы, что начинают напоминать песок на пляже. Однако *слишком крупные* микросервисы сложнее и дороже поддерживать, тестировать и масштабировать (что сводит на нет саму идею микросервисов).

Как же определить наиболее подходящий уровень детализации? Применить силы дезинтеграции и интеграции. Силы *дезинтеграции* требуют уменьшить размеры сервисов (поручить им меньше работы), а силы *интеграции* — увеличить (поручить больше работы). Посмотрим, как работают эти силы.

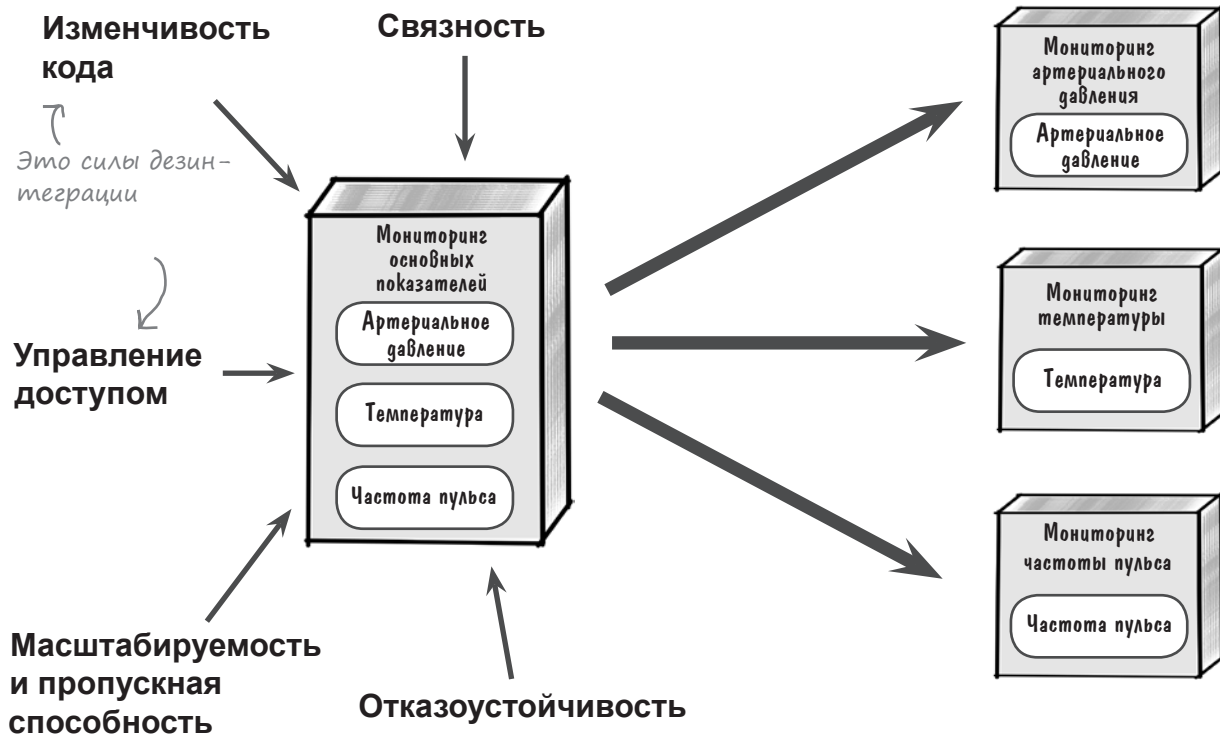


Разукрупнение

Силы дезинтеграции помогают понять, стоит ли разукрупнить сервис. Чтобы показать, как эти силы влияют на решение о разукрупнении, рассмотрим функцию мониторинга основных показателей в MonitorMe.

Основными показателями считаются артериальное давление, температура и частота пульса. Мы можем поместить их в один микросервис, как взаимосвязанные, или создать для каждого отдельный микросервис.

Силы дезинтеграции дробят сервисы на части.



Проанализируем каждую из этих сил, чтобы понять, как они помогают понять, стоит ли разукрупнять сервис мониторинга.

Почему желательно уменьшать микросервисы



Связность

Одноцелевой микросервис отличается высокой *связностью* функций, то есть все его действия тесно взаимосвязаны. Если микросервис содержит слабо связанные функции, то, возможно, его стоит разделить на части.



Отказоустойчивость и доступность

Некоторые функции в микросервисе часто сталкиваются с фатальными ошибками? В больших сервисах ошибка *в одной части* делает недоступными *все* его функции. Однако если такая функциональность находится в отдельном микросервисе, то ошибки в ней не повлияют на другие функции.



Управление доступом

В большом сервисе труднее управлять доступом к закрытой информации. Например, микросервис профиля пациента с функциями доступа к историям болезни может непреднамеренно позволить неавторизованному персоналу получить доступ к этой информации.

Перенос таких функций (например, доступа к историям болезни) в отдельные микросервисы *изолирует* их и упростит управление доступом к закрытой информации.

Некоторые используют также название «декомпозиция на основе изменчивости».



Изменчивость кода

Одна часть микросервиса меняется чаще других? Постоянные изменения в одной части большого микросервиса требуют тестировать *весь* микросервис, в том числе части, которые не менялись, а это дополнительная работа.

Перенос часто меняющихся функций в отдельный микросервис поможет изолировать эти изменения и значительно упростить сопровождение и тестирование.



Масштабируемость и пропускная способность

Разным частям микросервиса нужна разная масштабируемость? Если да, то разукрупнение сервиса позволит точнее управлять масштабируемостью его частей.

Например, предположим, что функция мониторинга частоты пульса читает показания каждую секунду, а функция мониторинга температуры — раз в 5 минут. Поместив эти функции в отдельные сервисы, можно позволить им работать с разной пропускной способностью.

Маленькие микросервисы запускаются быстрее, что позволяет быстрее предоставить пользователю необходимую функциональность.

Укрупнение

Силы интеграции работают в обратном направлении и помогают решить, когда следует укрупнять сервисы. Для иллюстрации случаев, когда может понадобиться объединять отдельные микросервисы в один большой сервис, мы используем тот же микросервис мониторинга основных показателей.



Проанализируем каждую из этих сил, чтобы понять, как они помогают понять, стоит ли укрупнять сервис мониторинга.

Почему желательно укрупнять микросервисы



Транзакции в базе данных

Когда запрос охватывает несколько микросервисов, невозможно откатить или зафиксировать сразу все изменения в базе данных. Каждый микросервис вносит изменения в своей транзакции и должен фиксировать или откатывать ее отдельно.

Если *согласованность и целостность данных* важнее любых сил дезинтеграции, то лучше объединить функции в один микросервис, чтобы операции выполнялись в рамках одной транзакции.



Рабочий процесс и хореография

Если в обработку одного бизнес-запроса вовлекается несколько микросервисов, это говорит об их *связанности*. Сильная связанность микросервисов может иметь множество негативных последствий.

Например, задержки в сети, подсистеме защиты и базе данных влияют на *производительность*. Потребность масштабировать каждый микросервис в цепочке вызовов в зависимости от масштабирования других (что трудно скоординировать) влияет на *общую масштабируемость*. Отказоустойчивость каждого микросервиса в цепочке влияет на *общую отказоустойчивость*.

Если рабочий процесс требует координации действий множества микросервисов, то подумайте об укрупнении.

Зависимости в данных — одна из основных сил интеграции.



Зависимости в данных

Разделяя микросервис на части, также приходится распределять его *данные*. Но когда данные *сильно взаимосвязаны*, сложно разделить их и сформировать новые ограниченные контексты.

Примером взаимосвязанных данных может служить ссылка из одной таблицы на ключ в другой (*ограничение внешнего ключа*). Еще пример — когда сущность (например, *информация о клиенте*) распределена по нескольким таблицам.

Если данные сильно взаимосвязаны и функции должны обмениваться ими, то лучше объединить их в микросервис.

Слишком большое количество связей между микросервисами может сделать архитектуру похожей на тарелку спагетти.



Все дело в балансе

Выбрать уровень детализации микросервисов сложно. Нужно взвесить компромиссы, связанные с каждой из сил интеграции и дезинтеграции, и оценить их важность. А для этого обычно необходимо сотрудничать с владельцем продукта или заинтересованными сторонами, особенно если компромиссы существенные.

Вы угадали — эти две силы имеют свои компромиссы, поэтому нужно найти правильный баланс между ними.

Силы дезинтеграции

Когда стоит уменьшить сервисы и разделить их функциональность?



Силы интеграции

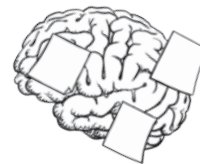
Когда стоит укрупнить сервисы и объединить их функциональность?

Разукрупнение микросервисов позволит улучшить масштабируемость, что очень важно.

Укрупнение микросервисов улучшит целостность данных, что очень важно.

Верно подмечено! Следующий шаг — выяснить, что важнее: масштабируемость или целостность данных. Как говорится, за двумя зайцами погонишься — ни одного не поймаешь.

**Запомните
это!**



Насколько маленькими должны быть микросервисы? Воспользуйтесь нашим советом. Сначала сделайте их крупными, а затем начинайте делить!





Упражнение

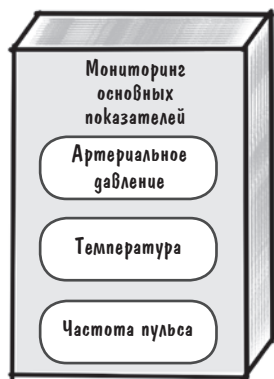
Теперь ваша очередь оценить силы интеграции и дезинтеграции и решить, как реализовать мониторинг основных показателей (артериального давления, температуры и частоты пульса) — в виде одного или трех отдельных сервисов. Вот некоторая дополнительная информация:

- * пульс и артериальное давление — два наиболее важных показателя. Они должны продолжать работать вне зависимости от работоспособности мониторинга температуры;
- * все функции мониторинга вызывают общую функцию оповещения медперсонала, если что-то идет не так;
- * функция мониторинга пульса читает показания датчиков раз в секунду, а функции мониторинга температуры и артериального давления — только раз в 5 минут;
- * значения каждого показателя записываются и хранятся отдельно, в виде простых пар «имя — значение» в формате JSON в общей базе данных. Например, показания частоты пульса хранятся в таком виде:

BPM означает *beats per minute* (ударов в минуту).

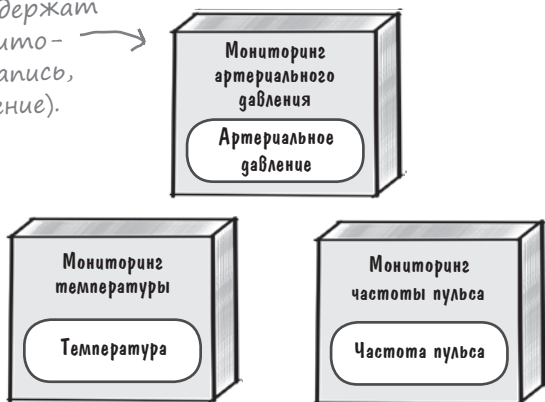
```
{"patient_id": "123",  
  "timestamp": "10452955668",  
  "bpm": "64"}
```

Вариант 1: один микросервис



Оба варианта содержат все функции мониторинга (чтение, запись, анализ и оповещение).

Вариант 2: отдельные микросервисы



Какой из двух вариантов вы бы выбрали и почему? _____

→ Решение на с. 373



Если микросервисы служат для
разделения бизнес-функций
на отдельные сервисы, то как быть
с общими функциями, такими как
протоколирование, авторизация
и обработка дат?

В микросервисах все еще можно использовать общий код.

Повторное использование кода — важная сторона разработки программного обеспечения. Без этого многие функции будут повторяться повсюду в вашей системе. Такие функции, как протоколирование, потоковая передача данных, авторизация пользователей и базовые утилиты, например, для преобразования дат, используются в большинстве систем.

В монолитных системах вы пишете общую функциональность один раз и используете ее везде, поскольку система компилируется как единое целое. Но в *распределенных* архитектурах все намного сложнее, поскольку каждый микросервис — это отдельно развертываемая единица программного обеспечения.

Куда же лучше всего поместить общие функции, используемые микросервисами? Обычно в *общую библиотеку* или в *общий сервис*. Далее мы обсудим компромиссы этих двух вариантов.

Общая функциональность

Все микросервисы в MonitorMe используют общую функцию оповещения медперсонала о том, что с пациентом что-то не так. Код выглядит так:

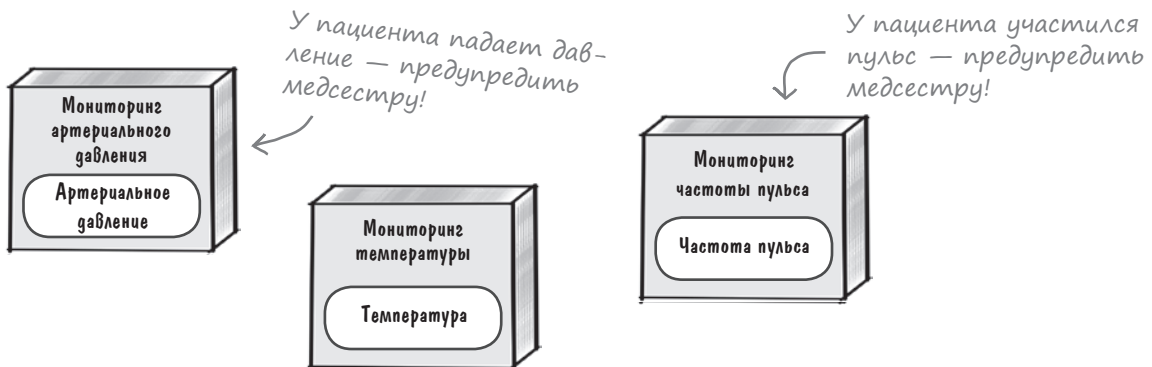
```
package monitorme.common;
public class AlertNurse {
    public static void sendAlert(AlertType type, String data) {
        ...
    }
}
```

Температура → (указывает на AlertType type)

Это код на Java. → (указывает на package monitorme.common;)

«41 градус Цельсия» → (указывает на String data)

Допустим, мы решили создать три отдельных микросервиса для мониторинга давления, температуры и пульса. Каждому из них нужна общая функция оповещения.



Где должен храниться исходный код общей функциональности оповещения? Мы можем скопировать его в каждый микросервис, но это приведет к проблемам, если кто-то изменит этот код, исправив ошибку или добавив новую возможность. Поэтому остается поместить код в общий сервис или в общую библиотеку. Рассмотрим оба варианта.

«Копирование» означает, что каждый микросервис имеет свою копию исходного кода. Этот подход можно использовать, только когда исходный код практически не изменяется.

Возьмите в руку карандаш

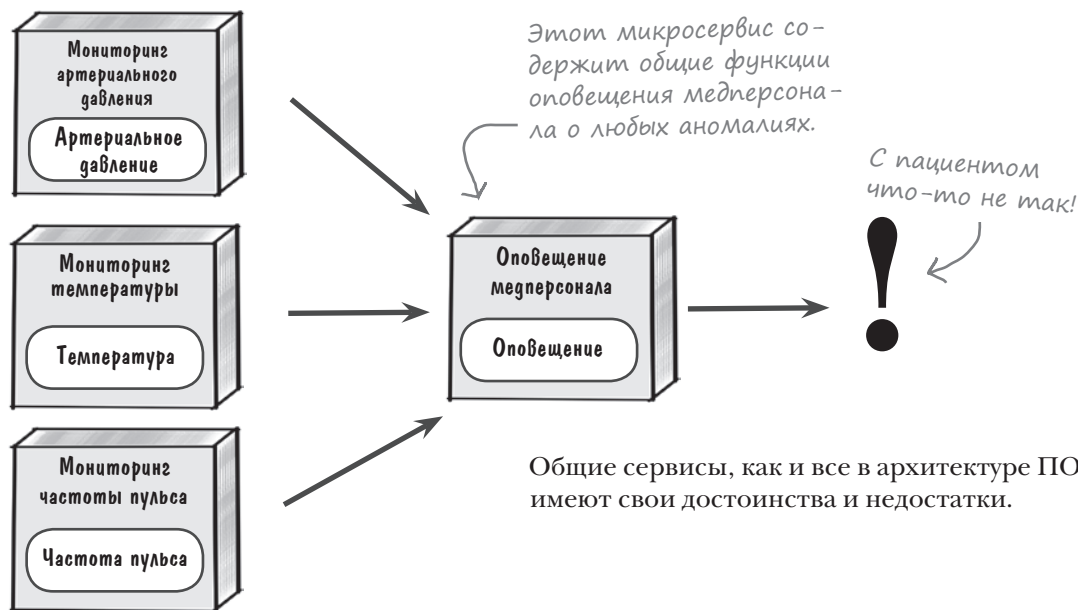
Перечислите другие функции, кроме оповещения, которые могут понадобиться сервисам MonitorMe.

Решение на с. 314

Совместное использование кода через общий сервис

Общий сервис — это отдельный микросервис с общими функциями, которые другие микросервисы могут вызывать удаленно.

Если поместить функции оповещения в отдельный общий сервис, то его сможет вызвать любой микросервис мониторинга, если обнаружит, что с пациентом что-то не так.



Общие сервисы, как и все в архитектуре ПО, имеют свои достоинства и недостатки.

Плюсы

Изменение кода в общем сервисе не требует изменения других микросервисов.

Общий сервис может быть написан на любом языке и на любой платформе, что очень удобно, когда микросервисы реализованы на нескольких языках.

Это еще одна из проблем связанности сервисов.



Минусы

Изменять общий сервис рискованно, так как это может повлиять на вызывающие его микросервисы.

Сетевые задержки при вызове удаленного общего сервиса могут замедлить работу.

Если общий сервис недоступен, то другие микросервисы, которым нужны его функции, не смогут работать.

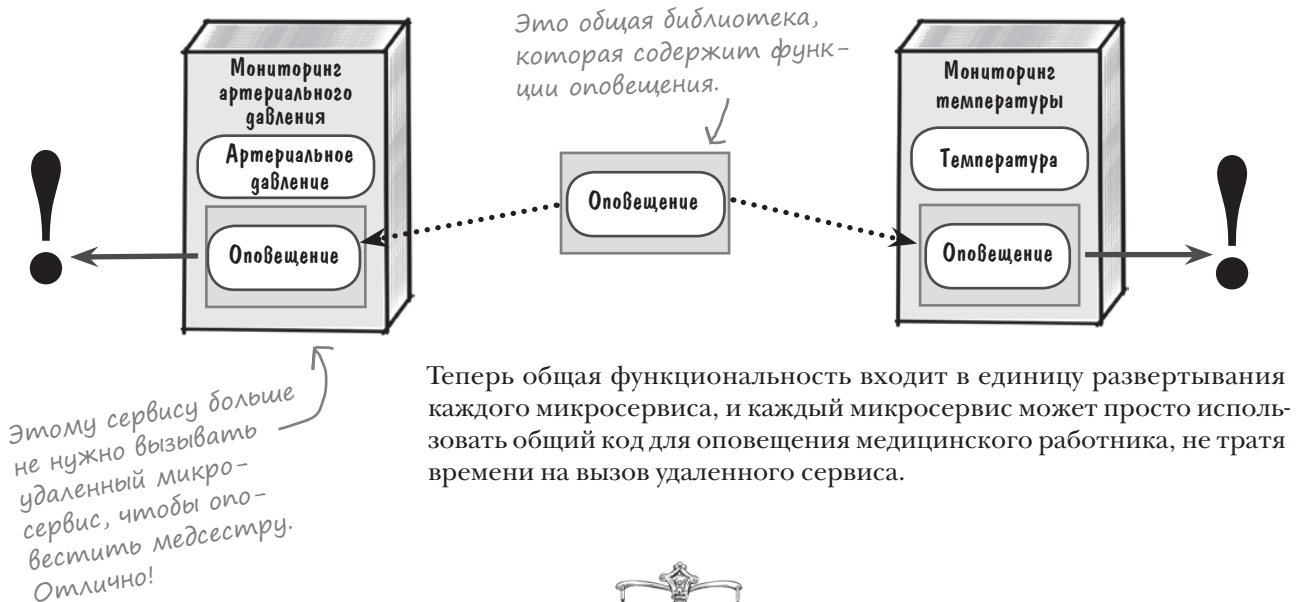
Общий сервис нужно масштабировать вместе с вызывающими его микросервисами.

Совместное использование кода через общую библиотеку

Более распространенный подход — поместить общие функции в *общую библиотеку*, независимый артефакт (например, JAR-файл в Java или DLL в C#), который подключается к каждому микросервису во время компиляции. Благодаря этому после развертывания *каждый* микросервис будет иметь доступ ко *всем* общим функциям.

Большинство платформ и языков программирования определяют свои форматы файлов с общими библиотеками.

Посмотрим, как могла бы выглядеть общая функциональность оповещения при использовании общей библиотеки.



Плюсы



Минусы

Производительность, доступность и масштабируемость лучше, поскольку общая функциональность добавляется непосредственно в микросервис во время компиляции.

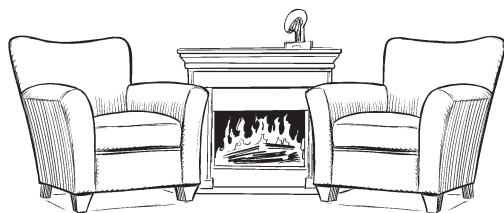
Изменение кода в общей библиотеке менее рискованно, так как при работе с библиотеками можно различать их версии для обратной совместимости.

Поскольку есть возможность различать версии библиотек, вам не придется делать это сразу.

Если микросервисы написаны на разных языках или используют разные платформы, то вам понадобятся несколько библиотек.

Управление зависимостями может стать сложной задачей, особенно при большом количестве микросервисов (что типично для этого архитектурного стиля).

При изменении общих функций придется вновь протестировать и развернуть микросервисы, которые их используют.



Беседы у камина

Актуальная тема: Общий сервис и Общая библиотека выясняют, кто из них круче.

Общий сервис

Привет представителям «старой школы»! Ты еще здесь?

Сомневаюсь, подруга. В мире распределенной архитектуры я — король. *Все вокруг* основано на сервисах, в том числе и я, сервис с общей функциональностью. Нужно повторно авторизовать пользователя или предупредить медсестру? Вызовите меня. Что может быть проще?

Согласен! Но если мне нужно измениться, я просто меняюсь, и никого это не затрагивает. Когда меняешься ты, то все сервисы, подключающие тебя, нужно заново собрать и развернуть. Это так утомительно!

Но я хотя бы не конформист. Признай, что тебя придется копировать для каждого языка программирования, используемого в системе. Я — сама независимость. Меня можно написать на *любом* языке или платформе, поскольку у меня нет проблемы связанности, характерной для тебя.

Неважно. Пока, конформистка, пойду найду кого-нибудь, с кем можно поболтать и кто ценит меня.

Общая библиотека

«Старая школа?» Дружище, я не только не уйду, но и переживу тебя.

Верно. А когда тебя нет рядом, что тогда? Видишь ли, в отличие от тебя, я всегда *на месте*, рядом с каждым микросервисом.

Я тебя умоляю! Я могу иметь несколько версий. Это делает изменения во мне более безопасными. Ты же, напротив, полон риска: можешь сломать всю систему после развертывания! Опасность — твое второе имя!

Проблема связанности? Реально? Моя связанность с сервисами означает, что я быстрее, доступнее, масштабируемое и надежнее, чем ты.

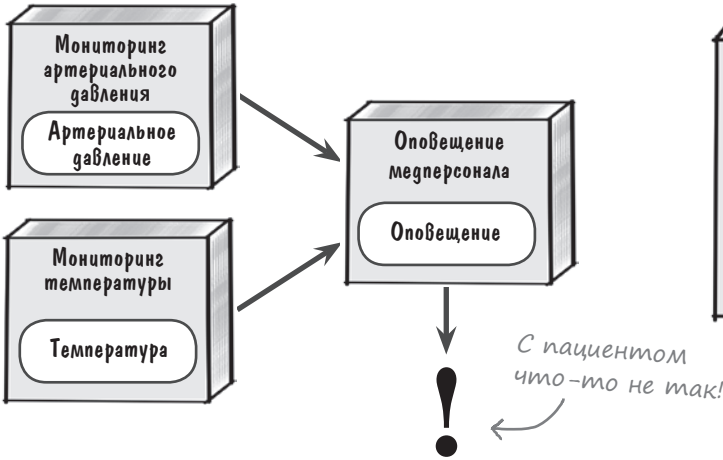
И вот он стал недоступен... Понимаете, о чем я?



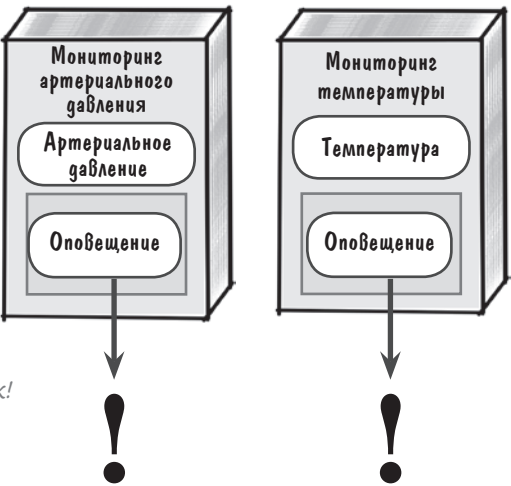
Возьмите в руку карандаш

Теперь, после знакомства с двумя вариантами использования общей функциональности в микросервисах, вы должны решить, куда поместить функции оповещения: в общую библиотеку или общий сервис. Не забудьте учесть внешние факторы (например, требования предметной области) помимо плюсов и минусов и обоснуйте свой выбор.

Вариант 1: общий сервис



Вариант 2: общая библиотека



Изложите здесь свое решение и его обоснование.

- ☐ Вариант 1: общий сервис
- ☐ Вариант 2: общая библиотека

Обоснование:

→ Решение на с. 375

Предположим,
мне нужно получить информа-
цию из нескольких сервисов.
Это возможно в архитектуре
микросервисов?



Да, конечно! Это называется *управлением рабочим процессом*.

Рабочий процесс — программное решение, при котором для обработки одного бизнес-запроса, поступающего из пользовательского интерфейса, требуется вызвать несколько микросервисов. Есть два приема управления рабочим процессом в микросервисах: *оркестрация* и *хореография*. Далее мы покажем, как они работают, и обсудим их преимущества и недостатки.

Часто Задаваемые Вопросы

В: Если я выполняю запрос, для ответа на который требуется участие нескольких микросервисов, то зачем разбивать эти микросервисы на части? Не лучше ли создать один сервис и вообще обойтись без рабочих процессов?

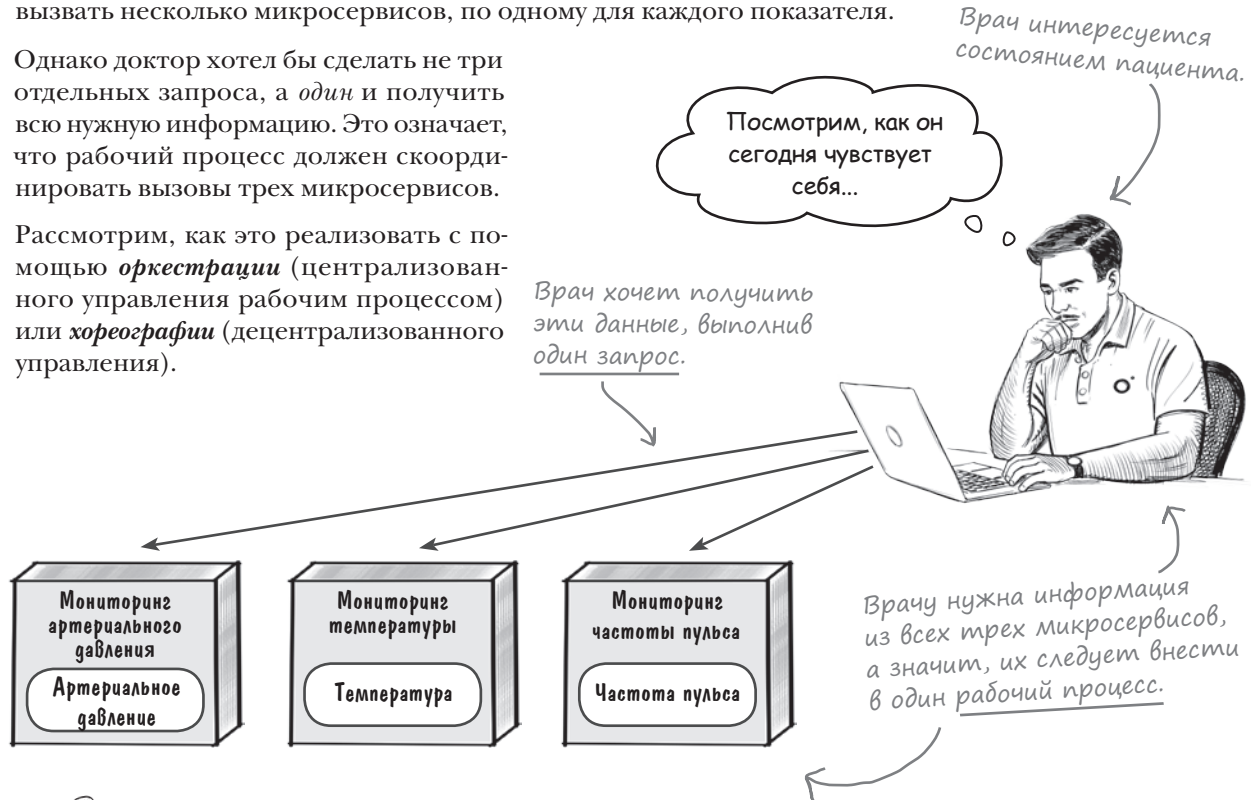
О: Хороший вопрос. Помните о силах дезинтеграции, обсуждавшихся выше? Некоторые из факторов: масштабируемость, отказоустойчивость, изменчивость кода и управление доступом — могут быть достаточно важными, чтобы оправдать разделение микросервисов, даже если это означает, что позже вам придется преодолевать сложности, чтобы связать эти сервисы в рабочий процесс.

Управление рабочими процессами

Рабочий процесс необходим, когда в обработке бизнес-запроса участвуют несколько микросервисов. Например, чтобы ответить на запрос врача «Как сегодня чувствует себя пациент?», система должна собрать информацию о температуре, пульсе и артериальном давлении пациента, то есть вызвать несколько микросервисов, по одному для каждого показателя.

Однако доктор хотел бы сделать не три отдельных запроса, а *один* и получить всю нужную информацию. Это означает, что рабочий процесс должен скоординировать вызовы трех микросервисов.

Рассмотрим, как это реализовать с помощью **оркестрации** (централизованного управления рабочим процессом) или **хореографии** (децентрализованного управления).



Упражнение

Придумайте рабочие процессы, содержащие несколько микросервисов, которые могут иметь место в системе мониторинга MonitorMe. Перечислите их ниже.

Получить основные показатели (температуру, артериальное давление и частоту пульса).

Пример от нас.

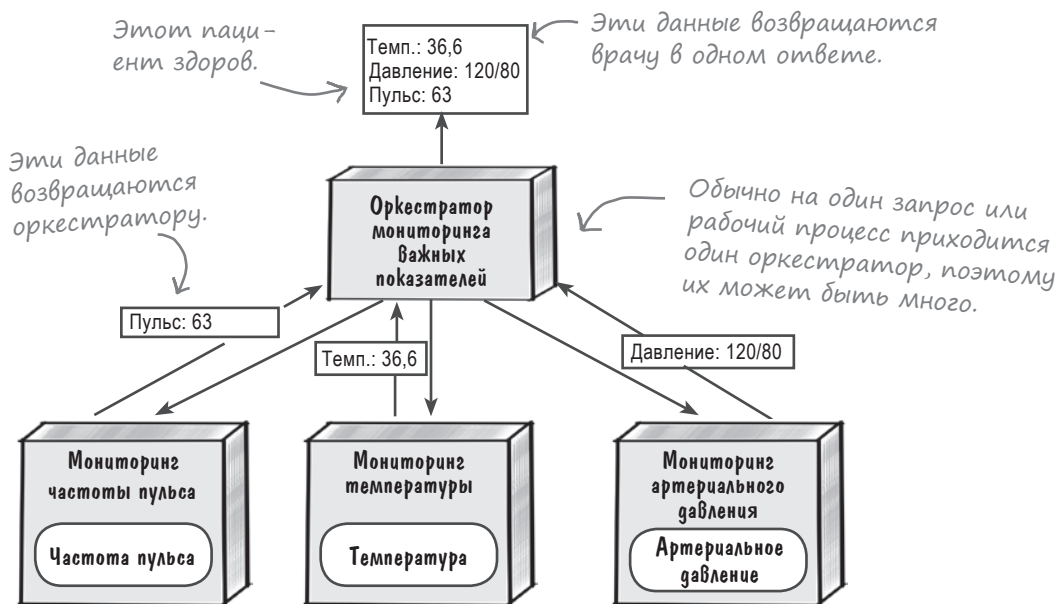
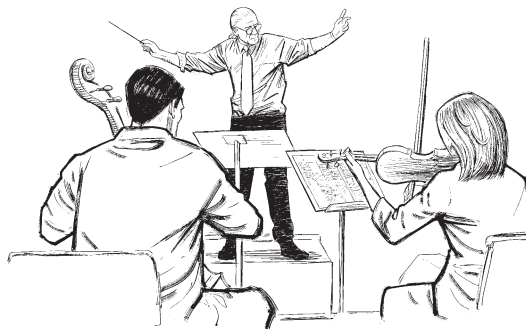
→ Решение на с. 376

Оркестрация: дирижирование микросервисами

Кто руководит музыкантами на концерте симфонической музыки? Конечно, дирижер! Это отличная аналогия, помогающая представить оркестрацию микросервисов.

Оркестрация — это координация действий микросервисов, участвующих в рабочем процессе. Централизованный микросервис — **оркестратор** — подобен дирижеру, руководящему музыкантами симфонического оркестра.

Оркестратор — это отдельный микросервис, отвечающий за вызов нужных микросервисов. Вдобавок он обрабатывает ошибки и возвращает данные вызывающей стороне (обычно пользовательскому интерфейсу).



Объявление о вакансии: оркестратор микросервисов

Требуется оркестратор микросервисов для управления мониторингом основных показателей пациентов. Главные обязанности:

- вызывать нужные микросервисы в правильном порядке;
- всегда знать текущее состояние рабочего процесса и понимать, что произойдет дальше;
- объединять данные, полученные из микросервисов;
- обрабатывать ошибки при сбоях в работе любого из микросервисов.

Работы много.
Надеюсь, труд
будет хорошо
оплачиваться.

Как все в архитектуре ПО, оркестрация имеет свои компромиссы. Будем позитивны и начнем с плюсов.

Плюсы...



Как навигатор GPS, который всегда знает, где вы находитесь.

Централизованные процессы

Обработка запросов централизована и очевидна. Чтобы понять рабочий процесс, достаточно заглянуть в оркестратор.

Состояние процесса

Оркестратор всегда знает, на каком этапе рабочего процесса находится обработка запроса, и в случае сбоя может перезапустить запрос с того места, на котором процесс остановился.

Обработка ошибок

Микросервисам не нужно беспокоиться о том, что делать в случае ошибки, — отслеживанием и обработкой ошибок занимается оркестратор.

Изменения в рабочем процессе

Рабочий процесс легко изменить, поскольку все изменения вносятся в одном месте.

Производительность

Оркестрация требует дополнительного времени на взаимодействие между оркестратором и микросервисами, а также на сохранение состояния рабочего процесса в базе данных после выполнения каждого этапа.

Масштабируемость

Оркестратор может стать узким местом при увеличении частоты запросов, поскольку каждый запрос должен пройти через него, прежде чем попасть в микросервисы.

Тесная связанность

Необходимость постоянных взаимодействий между оркестратором и микросервисами приводит к их тесной связанности.

Доступность

Когда дирижер покидает сцену, концерт заканчивается. Точно так же, если оркестратор окажется недоступен, запрос не будет обработан. Эта проблема единой точки отказа обычно решается путем запуска нескольких экземпляров оркестратора.

Минусы...

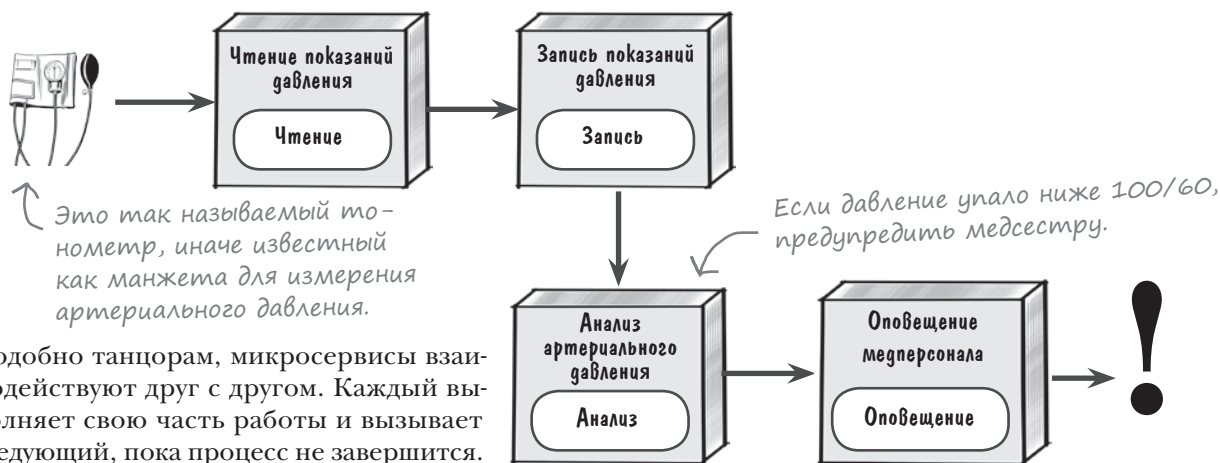


Это все равно что иметь второго дирижера за кулисами на случай, если первый дирижер заболеет.

Хореография: станцуем

Оркестрами дирижируют, а в танцах *ставят хореографию*. Редко можно увидеть дирижера, руководящего группой танцоров. Обычно они сначала разучивают свои партии и затем пробуют станцевать вместе. Это отличная аналогия хореографии микросервисов.

Представьте, что мониторинг артериального давления в системе MonitorMe разделен на четыре отдельных микросервиса, взаимодействующих в целях выполнения рабочего процесса.

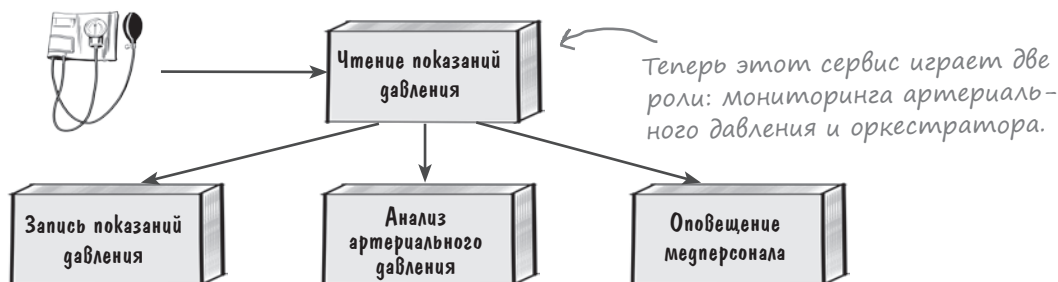


Подобно танцорам, микросервисы взаимодействуют друг с другом. Каждый выполняет свою часть работы и вызывает следующий, пока процесс не завершится.



Не руководите танцорами!

Используя прием хореографии, старайтесь не превратить один из микросервисов в оркестратор. Эта ловушка известна как паттерн «Контроллер входа». Он хорош для оркестрации, а не хореографии.



У хореографии тоже есть свои компромиссы

Плюсы...



Отзывчивость

Отсутствие необходимости постоянно взаимодействовать с центральным оркестратором повышает отзывчивость и производительность.

Слабая связанность

Микросервисы не зависят от центрального сервиса оркестрации, управляющего ими, поэтому система получается менее связанной.

Масштабируемость

Каждый микросервис может масштабироваться независимо от других микросервисов в рабочем процессе.

Хореография обычно дает скорость и масштабируемость. Но не забывайте о компромиссах.

Обработка ошибок

Каждый микросервис сам обрабатывает свои ошибки. Это может повлечь слишком большое количество взаимодействий между сервисами.

Восстанавливаемость

Когда пользователь повторяет неудачный или незавершенный запрос, системе сложно понять, с какого места его перезапустить, поскольку ни один сервис не отвечает за передачу запроса конкретному микросервису, участвующему в рабочем процессе, — каждый отправляет запрос только следующему микросервису в цепочке.

Управление состоянием

При использовании хореографии сложно определить состояние рабочего процесса из-за отсутствия центрального управляющего сервиса. Обычно один из микросервисов (как правило, первый в цепочке вызовов) назначается владельцем состояния, а другие сообщают ему свое состояние.

Будьте осторожны: потребность в управлении состоянием может привести к паттерну «Контроллер входа», описанному на предыдущей странице.

Минусы...





Упражнение

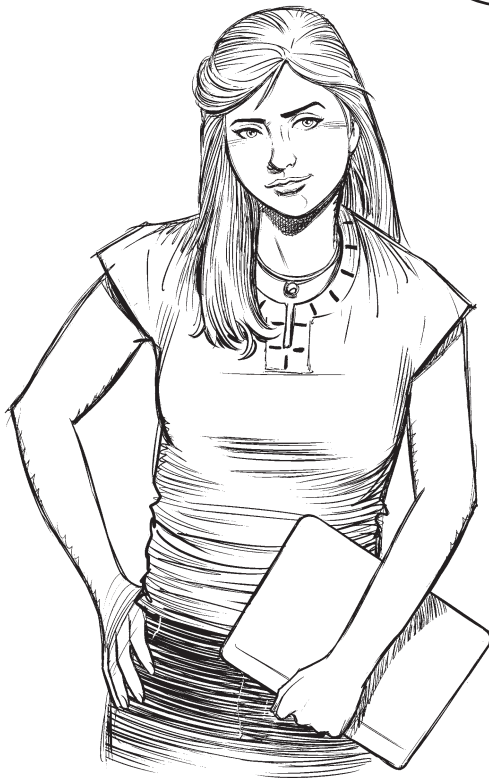
Компания StayHealthy, Inc. гарантирует поддержку MonitorMe и устройств мониторинга. Если возникает проблема с программным обеспечением или оборудованием, то сотрудник медицинского учреждения сможет выйти в интернет и оставить заявку на устранение неполадки. Выездной специалист получит заявку на мобильный телефон, приедет в учреждение, устранит проблему и отметит заявку как выполненную.

Как бы вы организовали управление микросервисами, участвующими в обработке заявок на устранение неполадок, — с помощью оркестрации или с помощью хореографии? Укажите причины вашего выбора.



Решение на с. 311

Архитектурный стиль микросервисов кажется мне чрезмерно сложным. Приходится принимать так много непростых решений. Микросервисы действительно нужны?



Да, архитектурный стиль микросервисов сложный, но очень эффективный.

Микросервисы помогают решать сложные бизнес-задачи. Этот архитектурный стиль не только поддерживает такие характеристики, как масштабируемость, надежность, доступность и отказоустойчивость, но и позволяет быстро реагировать на изменения как в бизнесе, так и в технологиях (так называемая *гибкость*).

При этом архитектурный стиль микросервисов подойдет не для каждой системы. Далее мы расскажем о некоторых сильных и слабых сторонах микросервисов.

Сверхвозможности микросервисной архитектуры

Каждый архитектурный стиль имеет свои сверхвозможности. Вот несколько причин использовать микросервисы.

БАЦ!

Удобство сопровождения

Каждый микросервис преследует одну цель и развертывается отдельно, поэтому вы легко найдете код, если потребуется изменить конкретную функцию.

Тестируемость

Объем тестирования микросервиса меньше, чем в случае с монолитным приложением или системой со множеством сервисов. Это облегчает полное тестирование микросервисов.

Бах!

Развертываемость

Микросервисы развертываются отдельно друг от друга, поэтому выпуск микросервиса несет меньше рисков, чем выпуск большой монолитной системы. Кроме того, микросервисы можно развертывать чаще — иногда даже каждый день.

Масштабируемость

Микросервисы масштабируются на уровне *функций*, а не *систем*. Это позволяет масштабировать только функции, необходимые для удовлетворения растущего спроса, экономить ресурсы и снижать затраты.



Развиваемость

Микросервисные архитектуры легко расширяются: вы создаете новый сервис, тестируете его и развертываете рядом с существующими.

Отказоустойчивость

Если какой-то микросервис выйдет из строя, это не приведет к сбою всей системы. Пользователи смогут продолжать пользоваться другими функциями.

БУМ!

Слабые стороны микросервисной архитектуры

Есть ли причины *отказаться* от микросервисной архитектуры? Конечно да! Как криптонит уменьшает силы супергероя, так и некоторые архитектурные и бизнес-характеристики уменьшают аргументы в пользу микросервисов. Остерегайтесь их!



Сложность

Микросервисы — один из самых сложных архитектурных стилей. Он предполагает множество сложных решений: о детализации, транзакциях, рабочих процессах, контрактах, общем коде, протоколах связи, организации команд и стратегиях развертывания, и это лишь некоторые из них.

Производительность

Чем больше микросервисы взаимодействуют друг с другом, тем хуже их производительность. На производительность могут влиять задержки в сети, дополнительные проверки для безопасности и обращения к базе данных.



Сложные рабочие процессы

Всегда, когда нужно вызвать несколько микросервисов для обработки одного бизнес-запроса, возникают *рабочие процессы*. Если функции системы слишком тесно взаимосвязаны, то разбиение ее на отдельно развертываемые сервисы приведет лишь к увеличению комка распределенной грязи. Ужасно!

Монолитные базы данных

Микросервисы должны владеть своими данными, образуя ограниченные контексты. Если данные нельзя разделить на части, то не используйте этот архитектурный стиль.

Ограниченный контекст содержит микросервис и все его данные.



Деление команд по техническим задачам

Ваша организация состоит из разрозненных команд разработчиков пользовательского интерфейса, серверной части и баз данных? Если да, то микросервисы вам не подойдут (согласно закону Конвея). Микросервисная архитектура требует, чтобы команды были кросс-функциональными: каждая должна владеть своей группой микросервисов, начиная с пользовательского интерфейса и заканчивая базой данных.

Звездные рейтинги микросервисной архитектуры

Ниже приводится оценочная таблица, позволяющая понять, в чем микросервисная архитектура хороша, а в чем не очень. Одна звезда означает, что архитектурная характеристика поддерживается плохо, а пять — что она поддерживается очень хорошо.

В точности как в рецензиях к фильмам.

Эти характеристики увеличивают гибкость — способность быстро реагировать на изменения.

Микросервисы поддерживают масштабирование на уровне функций.

Общие затраты высоки из-за лицензионных платежей, разделения данных, оптимизации конвейеров развертывания и реорганизации команд.

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★ ★ ★ ★ ★
Тестируемость	★ ★ ★ ★ ★
Развертываемость	★ ★ ★ ★ ★
Простота реализации	★
Развиваемость	★ ★ ★ ★ ★
Производительность	★ ★
Масштабируемость	★ ★ ★ ★ ★
Эластичность	★ ★ ★ ★
Отказоустойчивость	★ ★ ★ ★ ★
Общая стоимость	\$ \$ \$ \$ \$

Микросервисы СЛОЖНЫ в реализации.

Слишком большое количество взаимодействий замедляет обработку запросов.

Как видите, микросервисы хороши с точки зрения маневренности (удобство сопровождения, тестируемость и развертываемость) и эксплуатации. Но по части производительности, простоты реализации и стоимости их использование вызывает вопросы.

Возможно, вас удивил низкий рейтинг производительности. Это связано с задержками трех видов, возникающими при взаимодействиях сервисов: *задержки в сети* (время, необходимое для передачи запроса и ответа), *задержки безопасности* (например, время, необходимое для повторной авторизации пользователя) и *задержки доступа к данным* (поскольку каждый микросервис обращается к своим данным в БД).



Упражнение

Для каких из следующих систем подходит или не подходит микросервисная архитектура и почему?
Подсказка: учитывайте достоинства и недостатки микросервисов, а также особенности предметной области.

Онлайн-аукцион, где пользователи делают ставки

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Система заявок на устранение неисправностей в электронике для купивших платную поддержку

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

→ Решение на с. 378

В заключение

Поздравляем! Система MonitorMe запущена, и это большой успех. Все это благодаря вашему пониманию, для каких типов систем подходят микросервисы.

Вы узнали, что микросервисы могут быть сложны в реализации, но при этом имеют сверхвозможности, помогающие решать сложные бизнес-задачи (как, например, система MonitorMe). В заключение главы рассмотрим некоторые ключевые аспекты микросервисов.

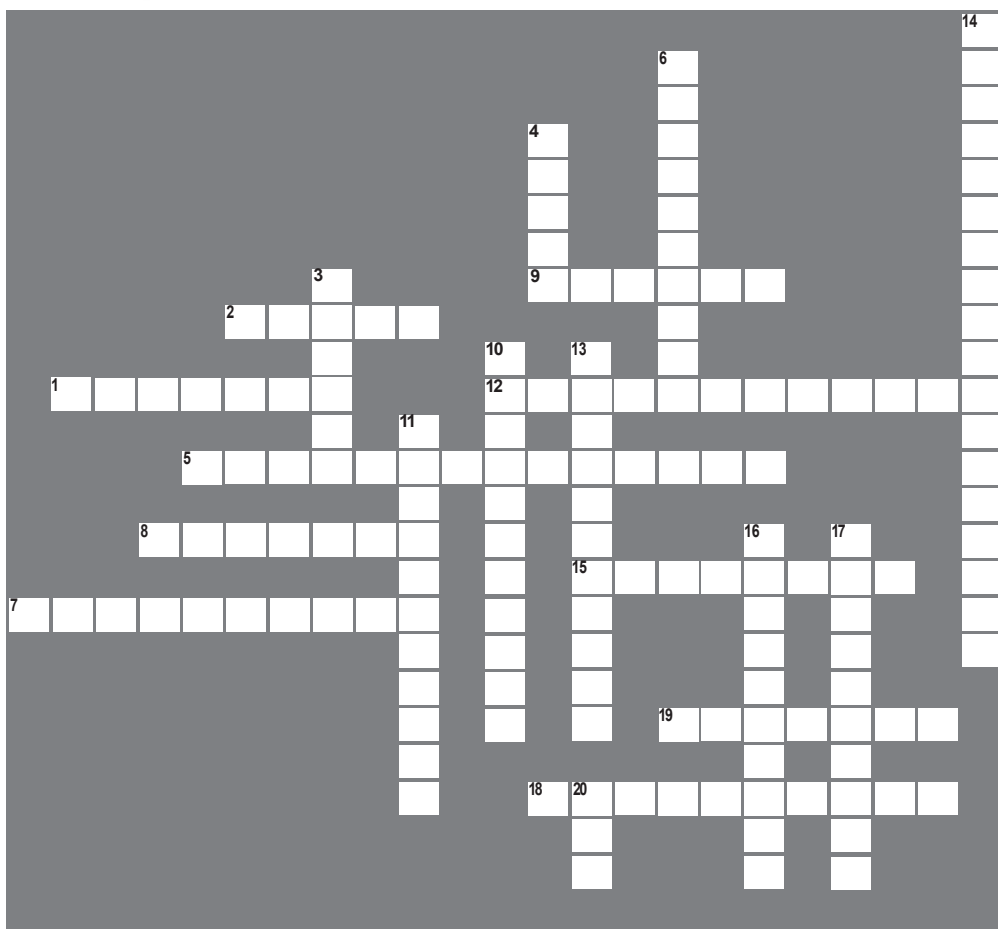
Ключевые моменты

- *Микросервис* — это одноцелевая, отдельно развертываемая единица ПО, хорошо справляющаяся с одной задачей.
- Микросервис образует *ограниченный контекст*: только он владеет своими данными и может обратиться к ним. Чтобы получить данные другого микросервиса, он должен запросить их.
- *Детализация* микросервиса — это мера не физического размера, а масштаба его функций.
- Силы, стремящиеся уменьшить микросервисы, называются *силами дезинтеграции*.
- Силы, стремящиеся увеличить микросервисы, называются *силами интеграции*.
- Сбалансируйте силы интеграции и дезинтеграции, чтобы подобрать подходящий уровень детализации микросервисов.
- На первых порах микросервисы можно сделать крупными, а затем, по мере изучения, уменьшать.
- Есть два приема организации общего кода в микросервисах: *общие сервисы* и *общие библиотеки*.
- *Общий сервис* содержит общие функции. Он развертывается отдельно, и другие микросервисы вызывают его удаленно. Общие сервисы более гибкие и хорошо подходят для разнородных сред, но плохо влияют на масштабируемость, отказоустойчивость и производительность.
- *Общая библиотека* — это независимый артефакт (например, JAR или DLL-файл), подключаемый к микросервису во время компиляции. Общие библиотеки позволяют улучшить масштабируемость, производительность и отказоустойчивость, но усложняют управление зависимостями и изменениями.
- *Рабочий процесс* возникает, когда для обработки одного бизнес-запроса требуется участие нескольких микросервисов.
- Рабочие процессы, управляемые методом *оркестрации*, требуют центрального микросервиса-оркестратора, действующего подобно дирижеру симфонического оркестра.
- В рабочих процессах, управляемых методом *хореографии*, сервисы взаимодействуют друг с другом подобно танцорам, выступающим вместе.
- Сильные стороны микросервисов: масштабируемость, отказоустойчивость, развиваемость и общая гибкость (удобство сопровождения, тестируемость и развертываемость).
- Слабые стороны микросервисов: производительность, сложность, стоимость, монолитные базы данных и высокая семантическая связанность.
- Микросервисы должны быть максимально независимыми; слишком тесная связанность снижает преимущества этого архитектурного стиля.



Кроссворд по микросервисам

Готовы проверить свои знания? Разгадайте этот кроссворд!



По горизонтали

1. Каждый микросервис _____ своими данными.
2. Остерегайтесь шаблона Контроллер _____.
5. Тип архитектуры микросервисов.
7. Сила _____ стремится увеличить размер микросервиса.
8. Каждый микросервис — это отдельно развертываемая _____ ПО.
9. Обработка _____ возлагается на оркестратор.
12. Контекст, регулирующий доступ к данным.
15. Микросервисы упрощают _____ системы.
18. Тип компонента внутри микросервиса.
19. Когда для обработки запроса нужно участие нескольких микросервисов — это рабочий _____.

По вертикали

3. Хорошая защита ограничивает _____ к данным.
4. _____ сервис действительно хорошо делает что-то одно.
6. Плохой шаблон проектирования — это _____ шаблон.
10. Децентрализованное управление рабочим процессом.
11. Уменьшая сервис, вы увеличиваете его _____.
13. Централизованное управление рабочим процессом.
14. _____ не дает проблеме разрушить всю систему.
16. Ресурс кода, который часто используется совместно.
17. Тип ограниченного контекста.
20. Микросервисы — _____ целевые.

→ Решение на с. 379



Возьмите в руку карандаш

Решение

Основываясь на предметной области и требованиях, перечисленных на с. 336, отметьте **пять** характеристик, которые вы считаете **особенно важными** для архитектуры MonitorMe, и укажите, почему они важны.

☒ **Тестируемость (простота и полнота тестирования)**

Причина: Это критически важная медицинская система, и в ней нет места ошибкам (полнота).

☒ **Отзывчивость (время подготовки ответа на запрос медицинского работника)**

Причина: От оперативности оповещения медицинского работника часто зависит жизнь пациента.

☐ **Развертываемость (сложность выпуска и развертывания изменений)**

Причина: Некритично — отсутствуют требования, диктующие необходимость частого изменения.

☐ **Абстракция (уровень изоляции частей системы друг от друга)**

Причина: Некритично — отсутствуют требования к абстракции.

☐ **Масштабируемость (способность обслуживать больше пользователей или пациентов)**

Причина: Некритично — система рассчитана на одну больницу (с ограниченным количеством коек).

☒ **Отказоустойчивость (способность продолжать работу при отказе отдельных частей)**

Причина: Сбой в мониторинге одного показателя не должен мешать мониторингу других показателей.

☒ **Целостность данных (согласованность и корректность данных и невозможность их потери)**

Причина: Данные о состоянии пациента должны быть максимально точными.

☐ **Рабочий процесс (способность системы обрабатывать сложные рабочие процессы)**

Причина: Некритично — фиксация показателей по отдельности не требует сложных процессов.

☒ **Параллелизм (способность обрабатывать сразу несколько запросов или операций)**

Причина: Система должна отслеживать множество разных показателей одновременно.



Упражнение Решение

На с. 339

Мы затрудняемся дать определение слова «одноцелевой». Помогите нам, отметив все функции, которые вы считаете одноцелевыми и, следовательно, микросервисами.

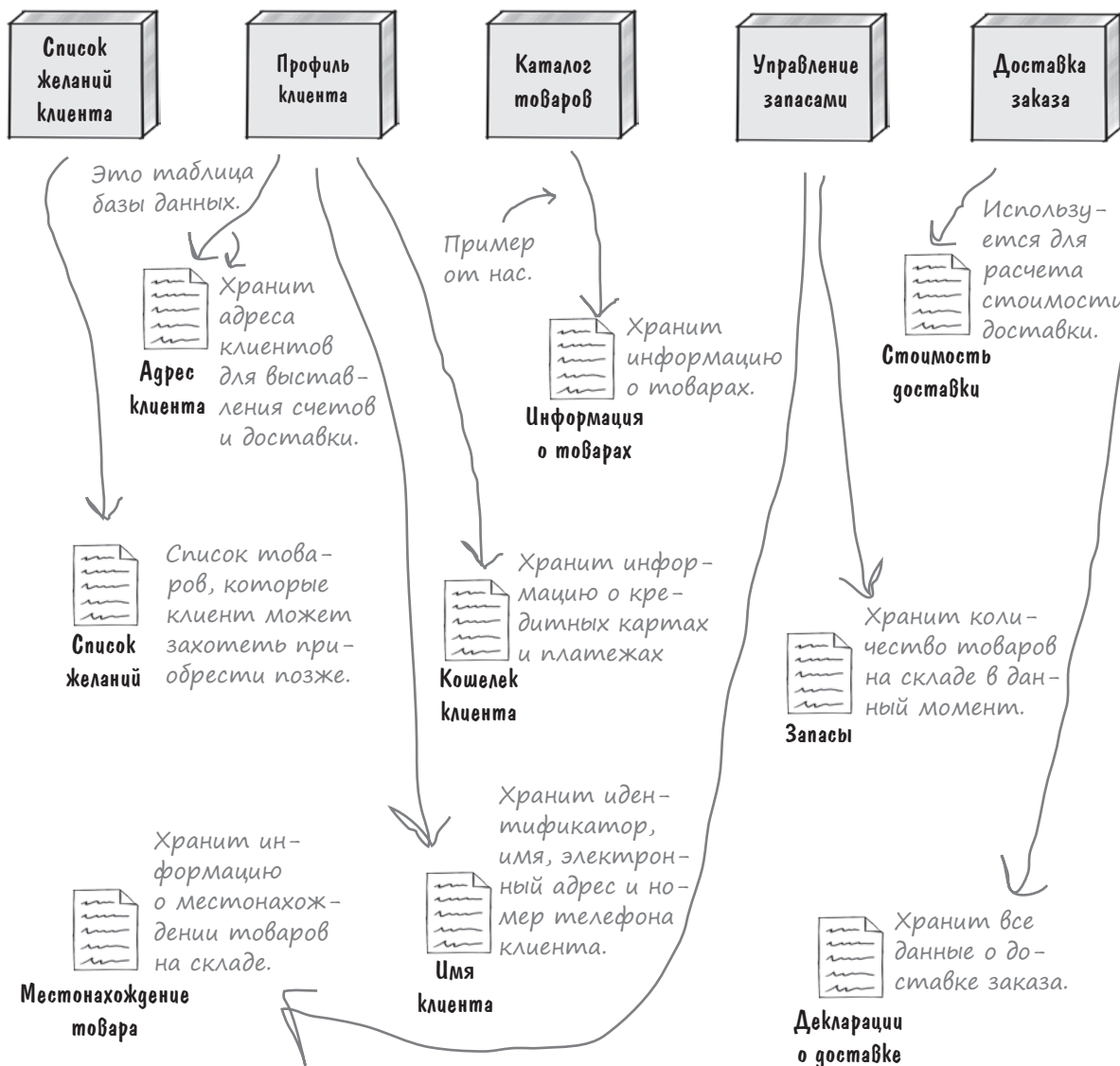
- ☒ Добавление фильма в личный список «что посмотреть»
- ☒ Оплата заказа с помощью кредитной карты
- ☐ Составление прогноза продаж и отчета с финансовыми показателями
- ☐ Подача заявки на кредит для приобретения нового автомобиля, о котором вы мечтали
- ☒ Определение стоимости доставки онлайн-заказа

*Это сложные процессы
с множеством разных
функций.*

Возьмите в руку карандаш

Решение

Мы создали следующие сервисы для сайта электронной коммерции и хотим определить права собственности на данные и ограниченные контексты. Свяжите приведенные ниже таблицы базы данных с микросервисами, которые должны владеть этими данными.





Упражнение Решение

На с. 349

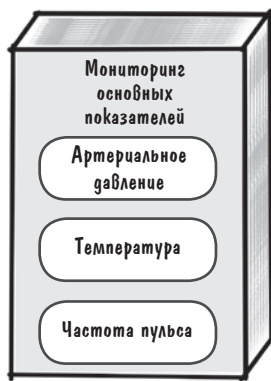
Теперь ваша очередь оценить силы интеграции и дезинтеграции и решить, как реализовать мониторинг основных показателей (артериального давления, температуры и частоты пульса) — в виде одного или трех отдельных сервисов. Вот некоторая дополнительная информация:

- * пульс и артериальное давление — два наиболее важных показателя. Они должны продолжать работать в независимости от работоспособности мониторинга температуры;
- * все функции мониторинга вызывают общую функцию оповещения медперсонала, если что-то идет не так;
- * функция мониторинга пульса читает показания датчиков раз в секунду, а функции мониторинга температуры и артериального давления — только раз в 5 минут;
- * значения каждого показателя записываются и хранятся отдельно, в виде простых пар «имя — значение» в формате JSON в общей базе данных. Например, показания частоты пульса хранятся в таком виде:

BPM означает *beats per minute* (ударов в минуту).

```
{ "patient_id": "123",  
  "timestamp": "10452955668",  
  "bpm": "64" }
```

Вариант 1: один микросервис



Оба варианта содержат все функции мониторинга (чтение, запись, анализ и оповещение).

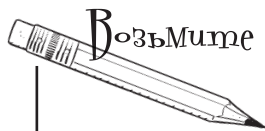
Вот наше мнение.

Вариант 2: отдельные микросервисы



Какой из двух вариантов вы бы выбрали и почему? Вариант 2: отдельные микросервисы.

Отдельные сервисы обеспечивают лучшую отказоустойчивость — отказ одного сервиса не повлияет на другие. Кроме того, данные хранятся отдельно, способствуя физическому разграничению контекстов в отдельных базах данных, что позволяет улучшить отказоустойчивость. Наконец, сервисы можно масштабировать в зависимости от частоты чтения данных (раз в секунду для пульса, раз в 5 минут для других).



Возьмите в руку карандаш Решение

Перечислите другие функции, кроме оповещения, которые могут понадобиться сервисам MonitorMe.

Наблюдаемость — измерение времени отклика и безотказной работы, ошибок и других показателей.

Журналирование — фиксация сообщений об ошибках и событиях в работе сервиса.

Аудит — фиксация того, когда отправлено оповещение и кто из медработников его получил.

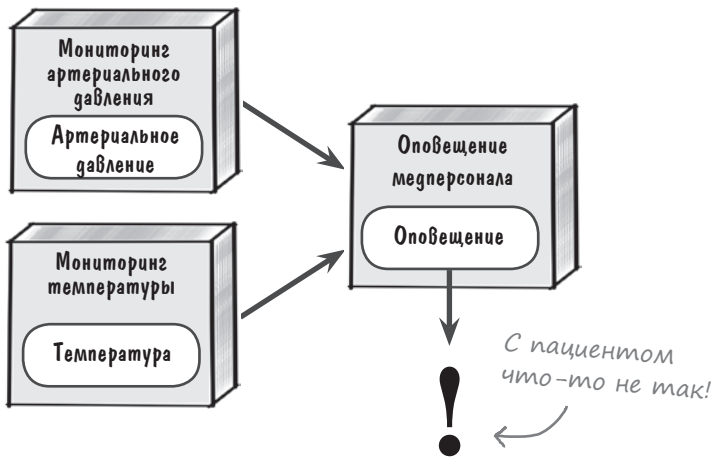
Защищенность — разрешение доступа к сервису мониторинга только авторизованным работникам.

Возьмите в руку карандаш

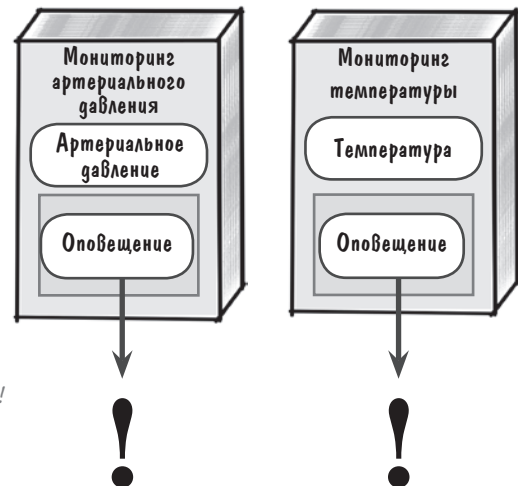
Решение

Теперь, после знакомства с двумя вариантами использования общей функциональности в микросервисах, вы должны решить, куда поместить функции оповещения: в общую библиотеку или общий сервис. Не забудьте учесть внешние факторы (например, требования предметной области) помимо плюсов и минусов и обоснуйте свой выбор.

Вариант 1: общий сервис



Вариант 2: общая библиотека



Наше решение и обоснование.

- ☐ Вариант 1: общий сервис
- ☒ Вариант 2: общая библиотека

Обоснование: Мы выбрали вариант 2 (общая библиотека), потому что он обеспечивает:

- улучшенную производительность (медработник будет оповещен быстрее);
- более высокую надежность и отказоустойчивость (если общий сервис оповещения выйдет из строя, то система не сможет оповестить медработника);
- улучшенный параллелизм (если одновременно возникнет нескольких проблем).



Упражнение Решение

Придумайте рабочие процессы, содержащие несколько микросервисов, которые могут иметь место в системе мониторинга MonitorMe. Перечислите их ниже.

Получить информацию о состоянии пациента (температуру, артериальное давление и частоту пульса).

Выяснить, какие показатели отслеживаются у данного пациента (для этого требуется запросить несколько сервисов мониторинга).

Зарегистрировать нового пациента для мониторинга: ввести большой объем информации, такой как имя, пол, возраст, анамнез и т. д., и для всего этого можно создать отдельные микросервисы.



Упражнение Решение

На с. 362

Компания StayHealthy, Inc. гарантирует поддержку MonitorMe и устройств мониторинга. Если возникает проблема с программным обеспечением или оборудованием, то сотрудник медицинского учреждения сможет выйти в интернет и оставить заявку на устранение неполадки. Выездной специалист получит заявку на мобильный телефон, приедет в учреждение, устранит проблему и отметит заявку как выполненную.

Как бы вы организовали управление микросервисами, участвующими в обработке заявок на устранение неполадок, — с помощью оркестрации или хореографии? Укажите причины вашего выбора.



Обоснование: Хореография — более простое решение с меньшим объемом взаимодействия сервисов. К тому же сам рабочий процесс несложен, поэтому использование оркестрации кажется излишеством.



Упражнение Решение

На с. 367

Для каких из следующих систем подходит или не подходит микросервисная архитектура и почему? Подсказка: учитывайте достоинства и недостатки микросервисов, а также особенности предметной области.

Онлайн-аукцион, где пользователи делают ставки

Почему? Требования к масштабируемости, параллелизму и независимости функций высоки.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? Сверхвозможности микросервисов не нужны в таких сложных системах.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? Благодаря гибкости и развиваемости микросервисы могут подойти, но нужно больше информации.

- ☐ Хорошо подходит
☒ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? Высокая стоимость и сложность микросервисов неприемлемы для небольшой пекарни.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

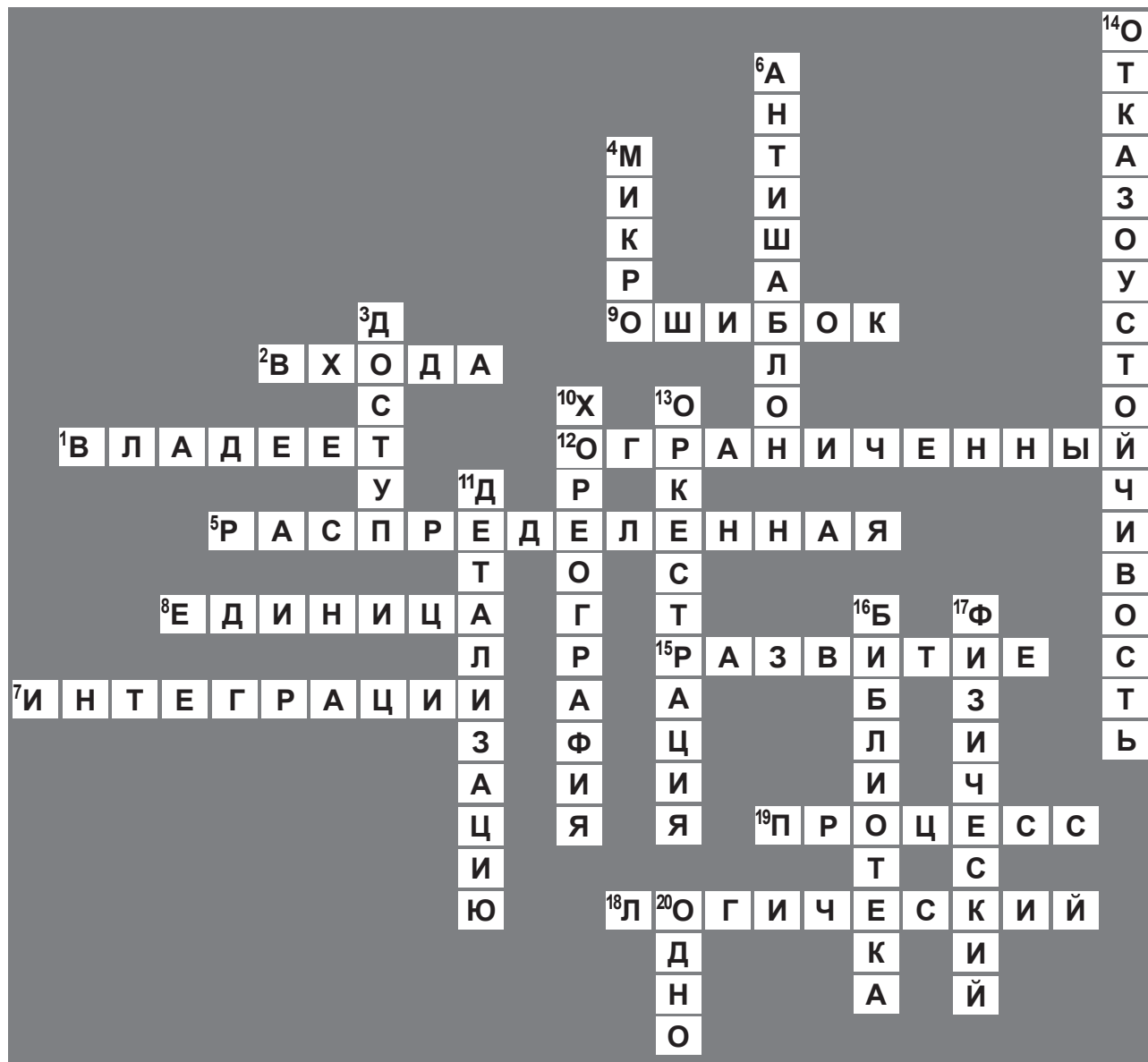
Система заявок на устранение неисправностей в электронике для купивших платную поддержку

Почему? Независимые функции, хорошая масштабируемость и эластичность, простые рабочие процессы.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Кроссворд по микросервисам. Решение

На с. 369



Асинхронные приключения



Может ли архитектура делать много дел одновременно? Чтобы развиваться, бизнес должен иметь возможность обслуживать все больше пользователей, без задержек и сбоев. В этой главе вы узнаете, как создавать высокопроизводительные системы на основе *событийно-управляемой архитектуры* — популярной разновидности распределенных архитектур, очень быстрых, легко масштабируемых и расширяемых, но при этом довольно сложных. Вы познакомитесь с такими новыми понятиями, как события, сообщения и асинхронная связь. Это поможет вам создать архитектуру, способную решать множество задач одновременно.

Слишком медленно

Представьте, что вы зашли в любимую закусочную заказать сэндвич с сыром на гриле, хрустящую картошку фри и молочный коктейль. Ничего сложного, правда? Но что, если человеку, принимающему заказ, придется все делать одному, без посторонней помощи? Потребуется больше времени, и к тому же закусочная не сможет обслуживать много клиентов. Визуализируем этот рабочий процесс:



1. Принять заказ у следующего клиента (1 минута).
2. Принять оплату (1 минута).
3. Приготовить сэндвич с сыром на гриле (4 минуты).
4. Приготовить картошку фри (5 минут).
5. Приготовить молочный коктейль (4 минуты).
6. Разложить блюда на тарелки и подать клиенту.



На каждый заказ уходит 15 минут, поэтому в обеденный перерыв закусочная сможет обслужить только четыре клиентов.

Этот клиент недоволен холодным обедом.



Такое положение вещей неприемлемо! Можем ли мы ускорить процесс, чтобы обслужить больше клиентов?



МозГовой
Штурм

Представьте, что вы — клиент этой закусочной. Что бы вы предложили, чтобы ускорить процесс?

Запишите свои мысли здесь.

Ускоряемся

Если закусочная наймет еще трех работников (по одному для приготовления сэндвичей, картошки фри и молочных коктейлей) и они будут готовить блюда одновременно, то выполнение заказов займет вдвое меньше времени. Это удвоит количество клиентов, которых закусочная сможет обслужить.



1. Принять заказ у следующего клиента (1 минута).
2. Принять оплату (1 минута).

Теперь на выполнение заказа уходит 7 минут, а значит, закусочная сможет обслужить больше клиентов.

3. Приготовить сэндвич с сыром на гриле (4 минуты).
4. Приготовить картошку фри (5 минут).
5. Приготовить молочный коктейль (4 минуты).

Блюда готовятся одновременно, но из-за картошки фри на один заказ уходит 5 минут.

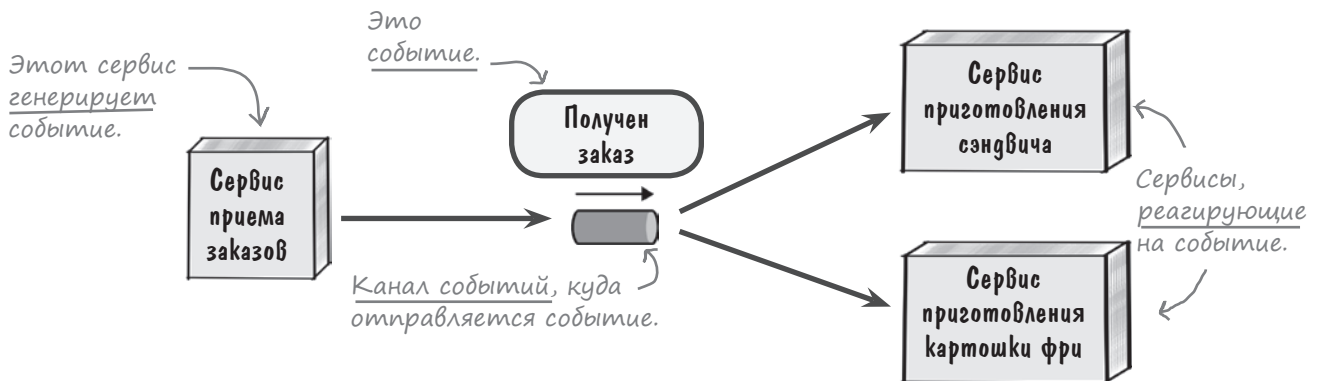
6. Разложить блюда на тарелки и подать клиенту.

Одновременное приготовление блюд сокращает время ожидания (назовем это *отзывчивостью*), благодаря чему официант, принимающий заказы, сможет обслужить больше клиентов (назовем это *масштабируемостью*).

Это фундаментальная идея *событийно-управляемой архитектуры* (Event-Driven Architecture, EDA) — одновременная обработка данных по *событиям* (то, что только что произошло) отдельными сервисами, каждый из которых решает свою задачу. В EDA сервисы взаимодействуют асинхронно через *канал событий*, не дожидаясь ответов от других сервисов.

Примечание: в дальнейшем событийно-управляемую архитектуру мы будем называть EDA.

Не волнуйтесь: мы объясним все эти термины.



Конечно, сразу все не понять, поэтому не будем спешить. Но прежде познакомимся с немецким интернет-магазином Der Nile, который испытывает трудности с расширением бизнеса и нуждается в вашей помощи.

Der Nile развивается быстрее, чем прежде

Der Nile — крупнейший в Германии интернет-магазин, продающий все: от подгузников до средств от облысения.

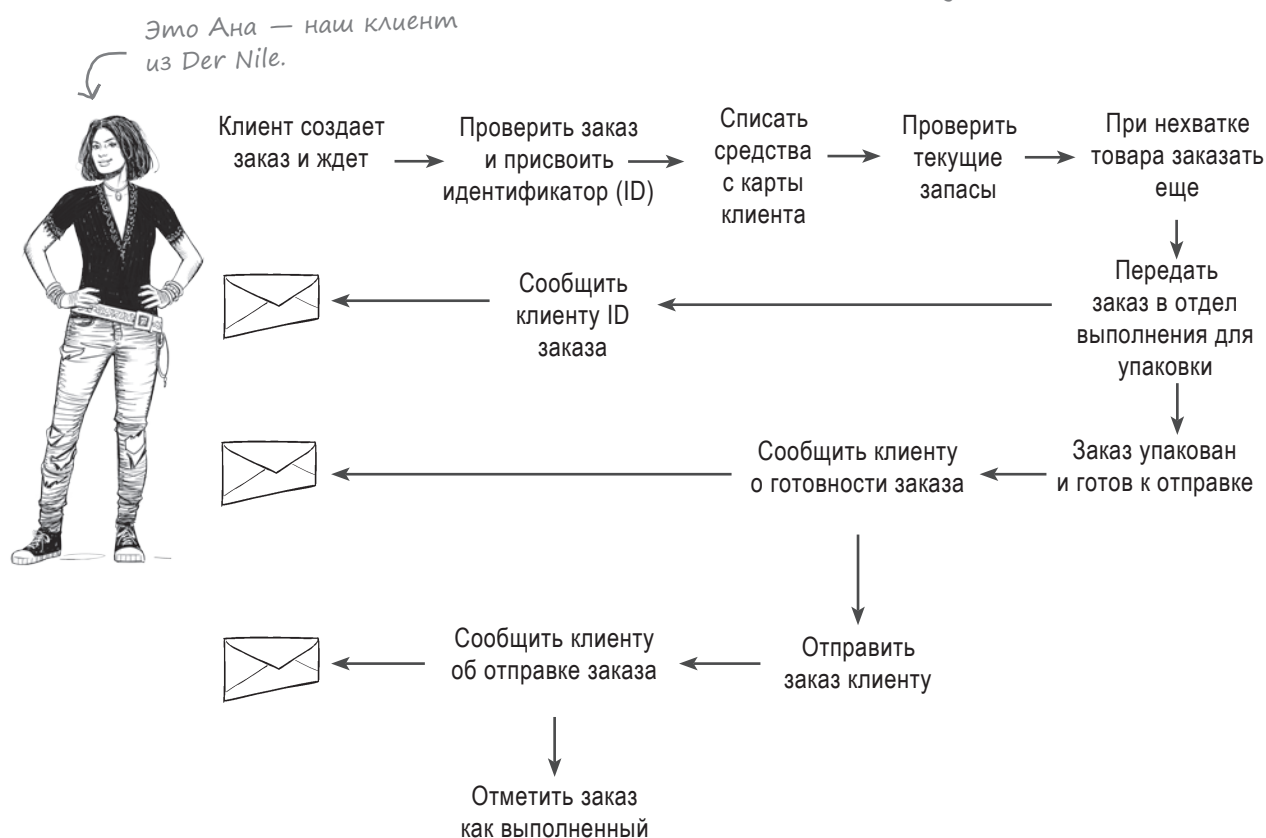
Онлайн-покупки пользуются такой популярностью, что компания работает на пределе своих возможностей. Клиенты требуют быстрого выполнения заказов, но текущая система Der Nile не справляется с их количеством. Обработка заказов занимает слишком много времени, а сроки выполнения увеличиваются.

В Der Nile хотели бы создать новую систему, способную справиться с этим ростом популярности, и им нужна ваша помощь.

Вот примерная схема работы системы онлайн-заказов, действующей в настоящее время:



В Der Nile хотят сделать онлайн-шопинг увлекательным.





Упражнение

Как бы вы, будучи архитектором, изменили рабочий процесс (показанный на предыдущей странице), чтобы ускорить его? Нарисуйте свой вариант ниже. *Подсказка: какие операции можно выполнять одновременно?*

Начните с этого.



Клиент создает
заказ и ждет



Решение на с. 423

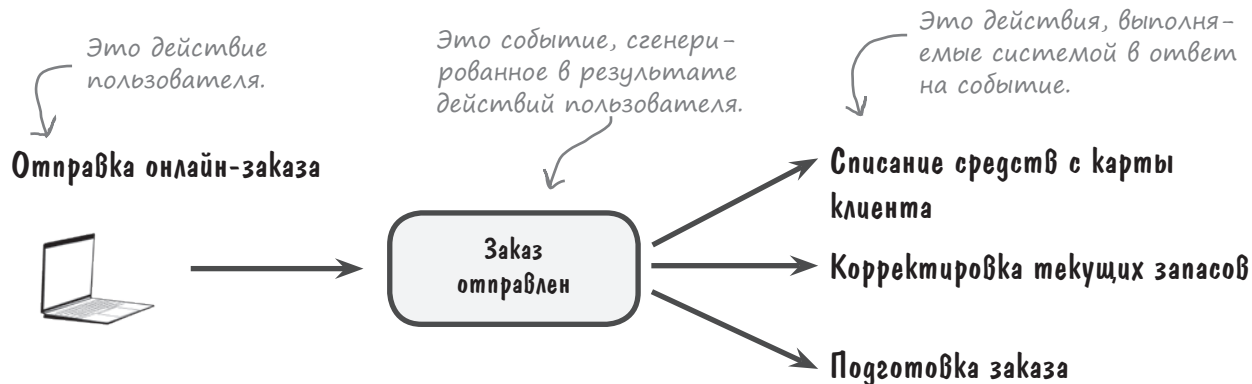
Что такое событие

Когда премьер-министра Великобритании Гарольда Макмиллана спросили, что в его работе было самым сложным, он ответил фразой, ставшей крылатой: «События, мой мальчик, события». Возможно, для Гарольда Макмиллана они и были самыми сложными, но нам они могут сильно помочь.

Событие — это то, что происходит, особенно если это что-то важное. Чемпионат мира по футболу, музыкальные концерты, повышение на работе, свадьбы и дни рождения — все это важные события.

Публикация этой книги стала важным событием для авторов.

В программных системах определенные действия пользователя вызывают события, такие как размещение ставки на аукционе, подача страхового заявления или совершение покупки.



События позволяют сервису сообщить остальной системе, что произошло нечто важное. В EDA события — это средство передачи информации другим сервисам.

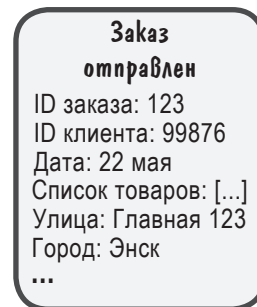
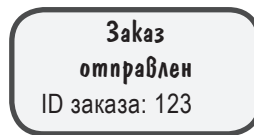
Напоминаем, что EDA расшифровывается как event-driven architecture — событийно-управляемая архитектура.

Часто событие содержит данные: *все* детали заказа или только ключевую информацию (например, ID заказа). В последнем случае сервисы, обрабатывающие событие, должны обратиться к базе данных, чтобы получить дополнительные сведения.

Данные, содержащиеся в событии, называются полезной нагрузкой.

Это событие содержит только ID заказа.

Это событие содержит всю информацию о заказе.





Постойте-ка, в главе 3 вы рассказывали о передаче информации с помощью **сообщений**. Теперь то же самое говорите о **событиях**. Получается, что события и сообщения — это одно и то же?

Этот стиль часто называют «запустил и забыл».

Подробнее об этом чуть ниже.

Нет, события — это не сообщения. И те и другие доставляют информацию в разные части системы, но имеют несколько важных различий.

Сервисы используют **события**, чтобы **сообщить** другим сервисам о некоем только что выполненном действии. Например, сервис может сообщить: *клиент только что оформил заказ*. Сервис, отправляющий событие, **не ждет ответа** и часто даже не знает, какие другие сервисы принимают это событие и какие действия выполняют в ответ.

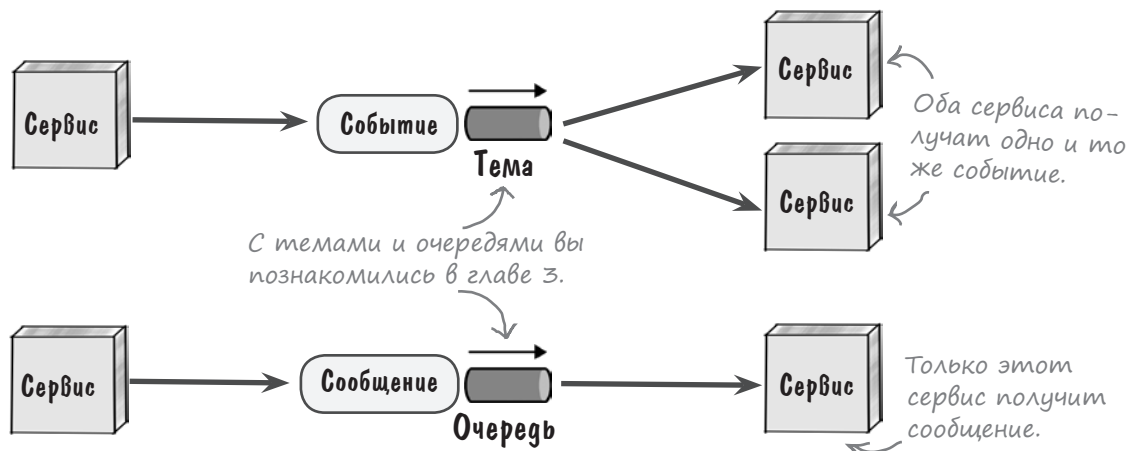
Сообщение, напротив, — это **команда**, например, *принять оплату этого заказа*, или **запрос**, например, *передай варианты доставки для этого клиента*. Каждое сообщение адресовано одному конкретному сервису, поэтому другие ничего не знают о нем. Иногда сервисы приостанавливаются и ждут ответа (если сообщение — это запрос на получение информации). В других случаях сервис может просто отдать команду и проверить, выполнил ли ее подчиненный сервис.

На следующей странице вы найдете еще несколько различий между событием и сообщением.

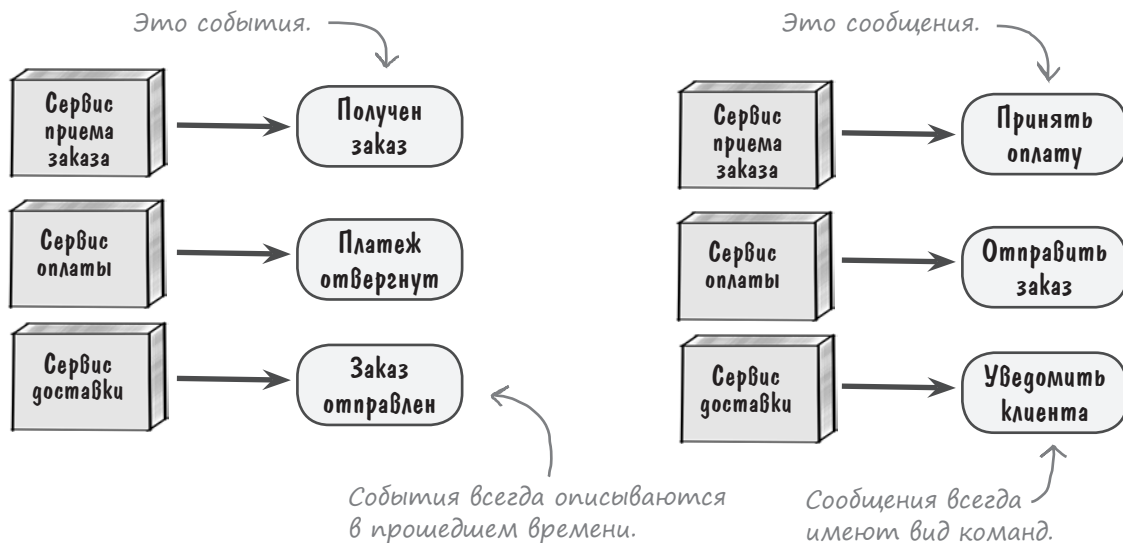
События и сообщения

Вот два действительно важных различия.

1. События рассылаются с помощью тем, а сообщения отправляются конкретным сервисам с помощью очередей.



2. События всегда сообщают о чем-то, что уже произошло, тогда как сообщения запрашивают действия, которые необходимо выполнить.





Пришло время проверить ваши знания о событиях и сообщениях. Для каждой из следующих цитат укажите, чем она является: событием или сообщением, и объясните почему.

«Борт Adventurous Air 12, поверните влево, курс 230 градусов».

☐ Событие ☐ Сообщение

Обоснование: _____

«Другие новости: в район только что пришел зимний штормовой фронт».

☐ Событие ☐ Сообщение

Обоснование: _____

«Откройте страницу 42 в ваших рабочих тетраях».

☐ Событие ☐ Сообщение

*Будьте осторожны —
здесь не все так просто!*

Обоснование: _____

«Всем привет! Прошу прощения за опоздание».

☐ Событие ☐ Сообщение

Обоснование: _____

«О нет! Я только что опоздал на поезд!»

☐ Событие ☐ Сообщение

Обоснование: _____

«Простите, можете уделить мне немного времени?»

☐ Событие ☐ Сообщение

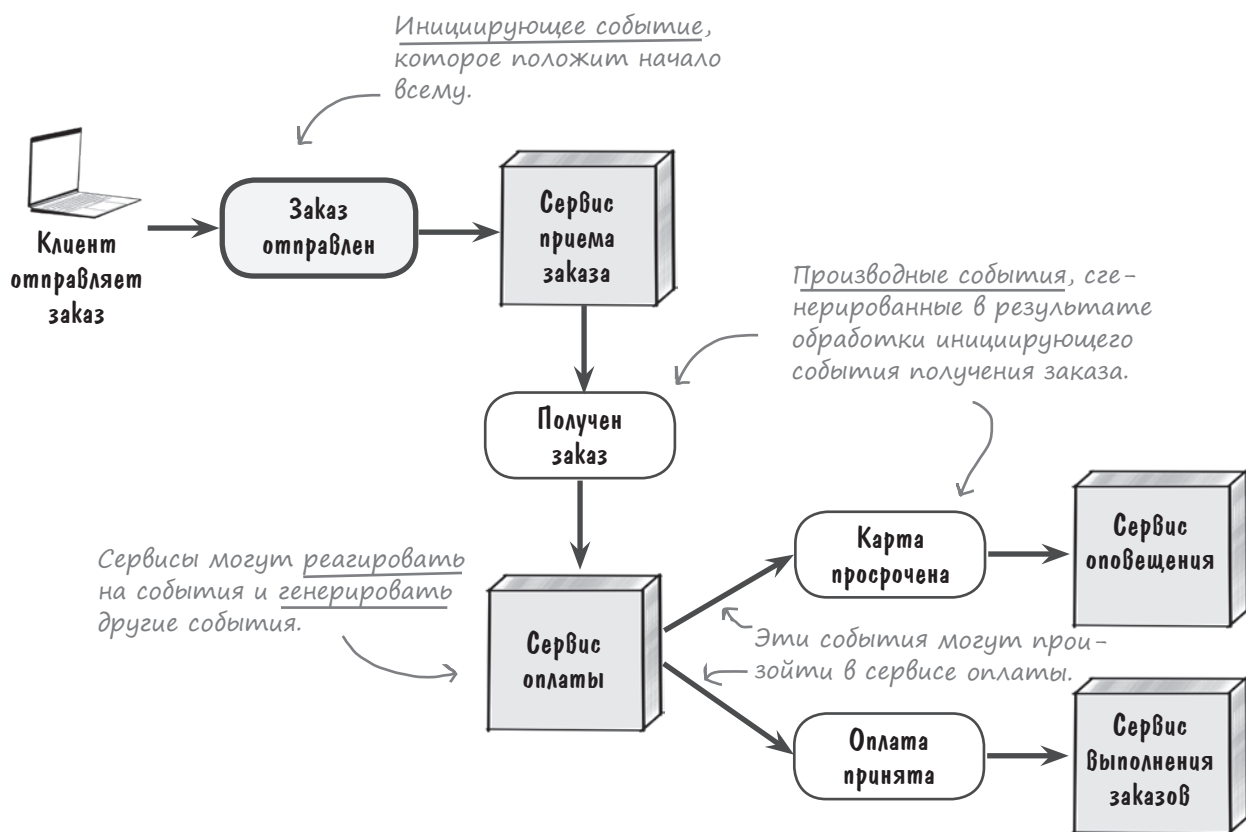
Обоснование: _____

→ Решение на с. 424

Иницирующие и производные события

События, исходящие от клиента или конечного пользователя, называются *иницирующими*. Это особый тип событий, запускающих бизнес-процесс.

Откликаясь на иницирующее событие, сервис может послать остальной системе свое событие с информацией о собственных действиях. Такие события называются *производными*, поскольку являются внутренними событиями, генерируемыми в ответ на иницирующее.



Заметили, что сервис оплаты генерирует два разных производных события? Это типично для EDA. Любое действие сервиса может привести к созданию производного события.



Упражнение

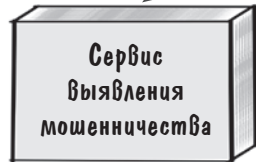
На основе инициирующего события «Оплата картой» и последующей обработки определите производные события для каждого сервиса. Придумайте как можно больше событий.

Клиент совершает
покупку по карте

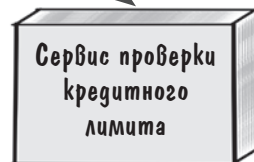
Это инициирующее
событие.



Оплата
картой



Проверка
на мошенничество



Проверка
кредитного лимита

Укажите возможные
производные события
для каждого сервиса.

Придумайте сколько
сможете. Возможно,
не все они понадобятся.

Это производные
события.

Four empty rounded rectangular boxes for listing derived events for the 'Проверка на мошенничество' service.

Four empty rounded rectangular boxes for listing derived events for the 'Проверка кредитного лимита' service.

Решение на с. 425

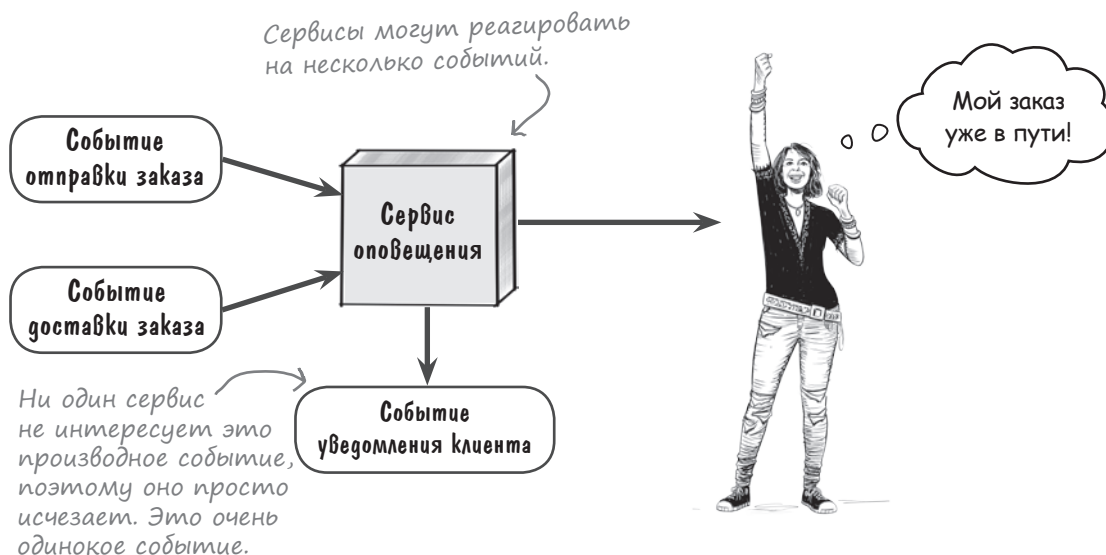
Меня кто-нибудь слышит?

Публикуя что-то в социальных сетях, вы часто получаете ответную реакцию: кто-то ставит лайк или оставляет комментарий. Но случалось ли вам не получать никакой ответной реакции? Возможно, вы спрашивали себя: *А видел ли кто-то мое сообщение? Кому-то это интересно?*

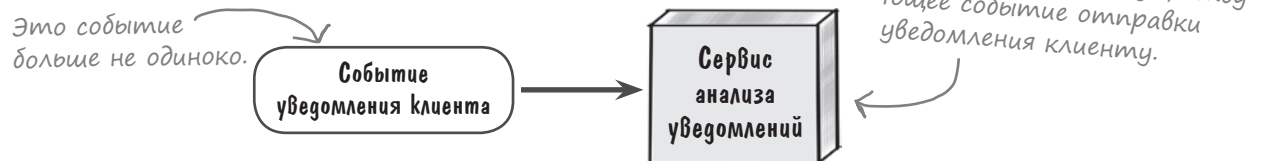
В EDA любое действие сервиса должно вызывать производное событие. Однако есть вероятность, что некоторые события никого не интересуют. Так зачем публиковать их? Затем, чтобы обеспечить **расширяемость архитектуры** — возможность добавления новой функциональности.

Допустим, клиенты системы Der Nile получают уведомления об отправке заказа и о том, что он доставлен, которые посылает сервис оповещения. Эти действия, в свою очередь, вызывают производное событие «Клиент уведомлен».

Расширяемость — одна из архитектурных характеристик, о которых вы узнали в главе 2.



Теперь предположим, что в Der Nile захотели узнать, в какое время суток система отправляет больше всего писем с уведомлениями. Производное событие «Клиент уведомлен» уже публикуется, поэтому вы можете просто создать новый сервис анализа уведомлений и настроить его на прием этих событий. *Ничего другого в системе изменять не придется.*



Асинхронная связь

Вы говорили, что эта архитектура очень быстрая. Как она может быть быстрее микросервисов?



Событийно-управляемая архитектура работает быстро, поскольку в ней используется преимущественно асинхронная связь.

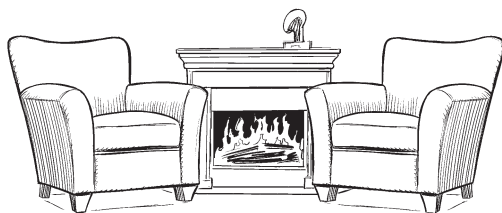
Вероятно, вы уже знакомы с такими стилями взаимодействия, как REST или HTTP, особенно если вам приходилось вызывать API или другой сервис. Это формы *синхронной связи*. При синхронном общении сервис отправляет информацию и ждет ответа от принимающей стороны, прежде чем сделать что-то еще (даже если это просто подтверждение получения информации). В результате работа систем замедляется и их масштабируемость ухудшается. Это как позвонить другу — вы должны дождаться, пока ваш телефон установит соединение с телефоном друга и тот поднимет трубку, и только потом вы сможете поговорить.

Асинхронная связь позволяет сервисам отправлять информацию и не ждать ответа или подтверждения от других сервисов. Благодаря этому системы получают слабо связанными и очень быстрыми. Это одна из уникальных особенностей событийной архитектуры. Представьте, что отправили другу СМС: вы можете заниматься другими делами, пока ждете его ответа.

Приходится ждать ответа — это очень медленный процесс.



Поскольку не нужно ждать, можно заняться другими делами.



Беседы у камина

Актуальная тема: Синхронный и Асинхронный стили взаимодействия — что лучше?

Асинхронный стиль

Что же, самое время тебе появиться.

Да, я заметил. Ты медлительный, как стадо улиток, ползущих по арахисовому маслу.

Ждать? Ты хочешь, чтобы я *ждал*? В этом твоя проблема: ты всегда ждешь ответов. А вот я никогда не жду. Это делает меня быстрым — *очень быстрым*.

В отличие от тебя, я доверяю другим. Тебе же постоянно нужно проверять *все и вся*.

Никак. Я не могу отвечать за всех. Я позволяю другим самим решать их проблемы.

О, прости, ты что-то сказал? Я прослушал.

Простите, я опять прослушал. Что ты сейчас сказал?

Синхронный стиль

Простите, что опоздал. Я не могу работать в многозадачном режиме, поэтому все мои действия занимают *так много* времени.

Подождите всего минуту!

Возможно, это так, но твоя проблема в том, что ты никогда не знаешь, что происходит. Ты просишь других сделать что-то, но не знаешь, было ли это сделано на самом деле.

Ты прав. А скажи, как ты справляешься с ошибками, возникающими в процессе работы?

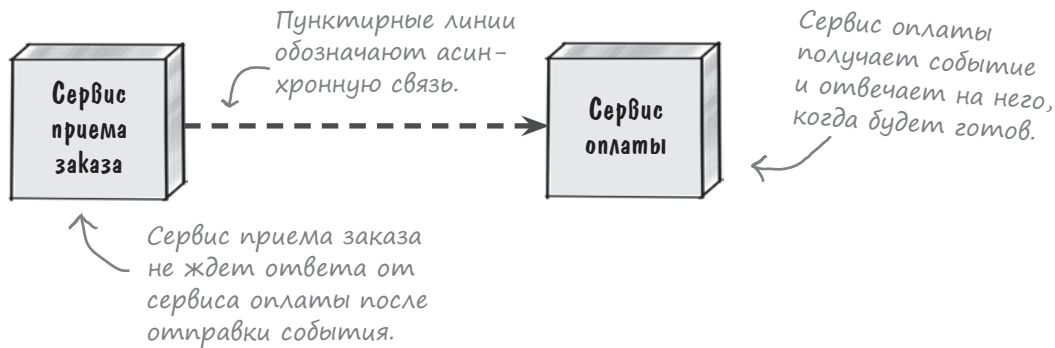
А если тебе нужна информация от кого-то еще, чтобы закончить свою работу? Что ты делаешь в таком случае?

Именно об этом я и говорю! Ты никогда не слушаешь других. Может, я и медлительный, но хотя бы обращаю внимание на происходящее и общаюсь с другими.

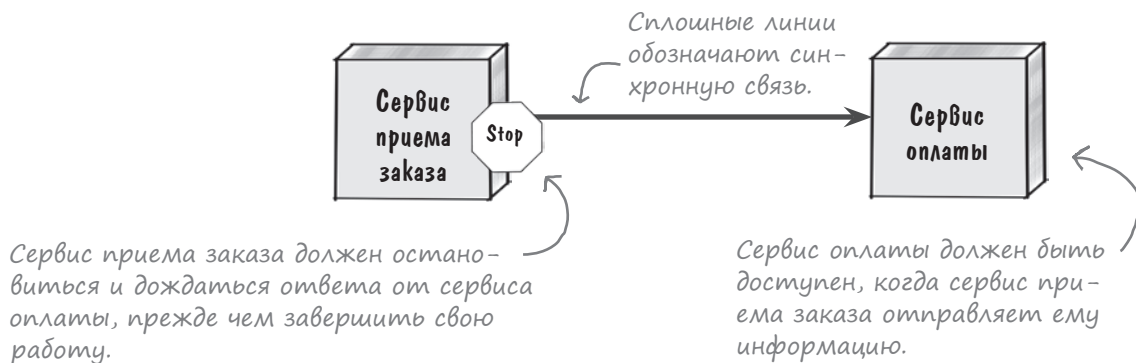
Эх-х...

Запустил и забыл

Асинхронная связь — одна из основ событийно-управляемой архитектуры. Когда сервис передает информацию другим сервисам, он не ждет ответа и не заботится о доступности этих сервисов. Этот стиль известен как **«запустил и забыл»**: событие отправляется (это часть «запустил»), и сервис переходит к другим действиям (это часть «забыл»). Для обозначения асинхронных взаимодействий архитекторы обычно используют пунктирные линии.



Синхронная связь, напротив, требует, чтобы сервис-отправитель дождался ответа от сервиса-получателя, прежде чем продолжить работу. Поэтому вызываемый сервис *должен быть* доступен, иначе произойдет ошибка. Синхронные взаимодействия обычно обозначаются сплошной линией.



Событийно-управляемая архитектура использует асинхронную связь для отправки и получения событий.



Возьмите в руку карандаш

Какую связь, синхронную или асинхронную, вы бы использовали для каждой из этих задач?

Сообщить варианты доставки заказа

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Принять оплату заказа и сообщить, прошел ли платеж

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Выполнить заказ, собрав товары и упаковав их в коробку

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Сообщить текущий статус заказа

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Запасы товара сокращаются — заказать еще

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Сообщить клиенту, что заказ отправлен и находится в пути

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Обновить фотографию в профиле клиента

☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

Разместить отзыв покупателя на странице продукта

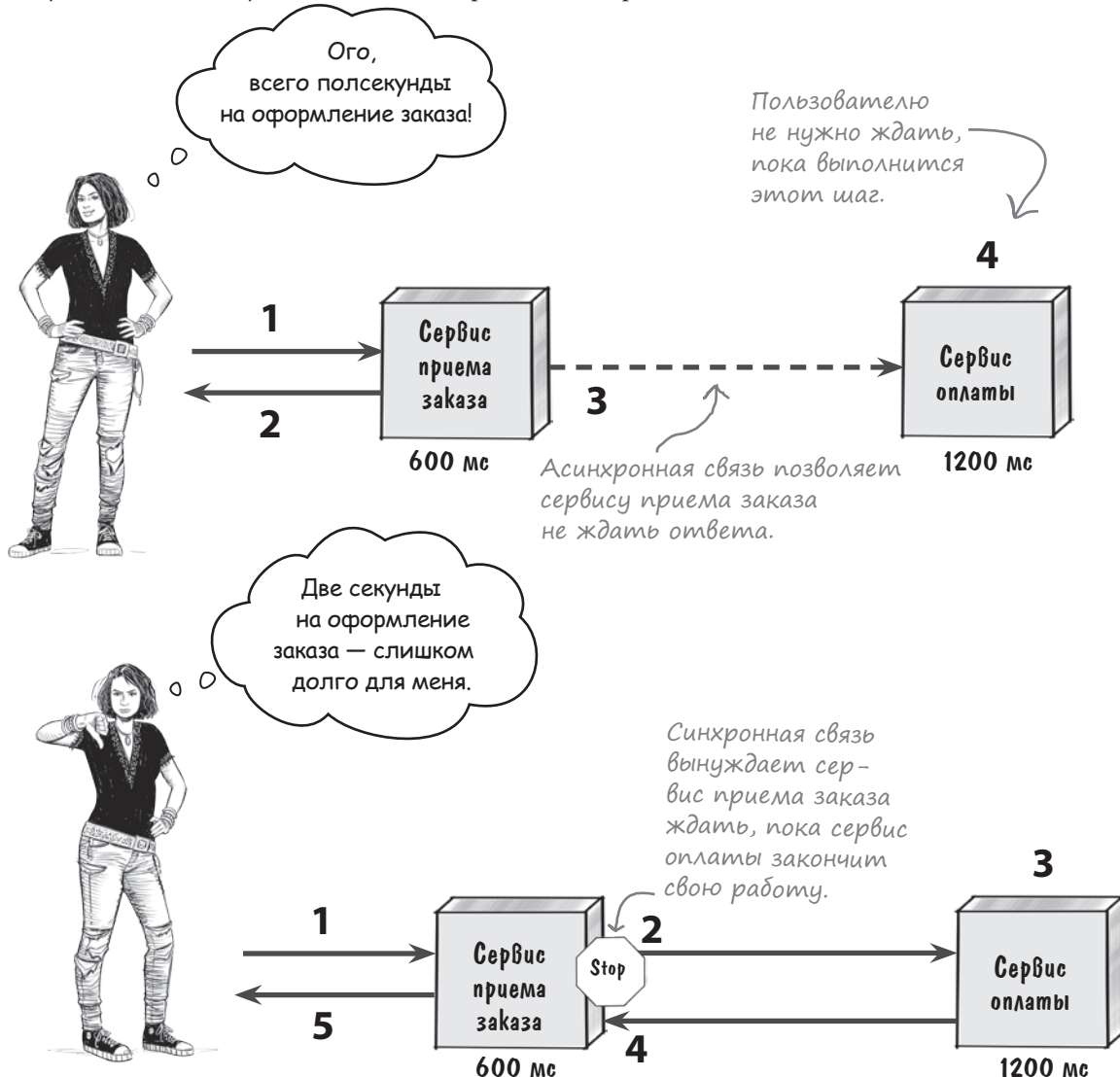
☐ Асинхронная ☐ Синхронная ☐ Подойдет любая

→ Решение на с. 426

Асинхронность раги победы

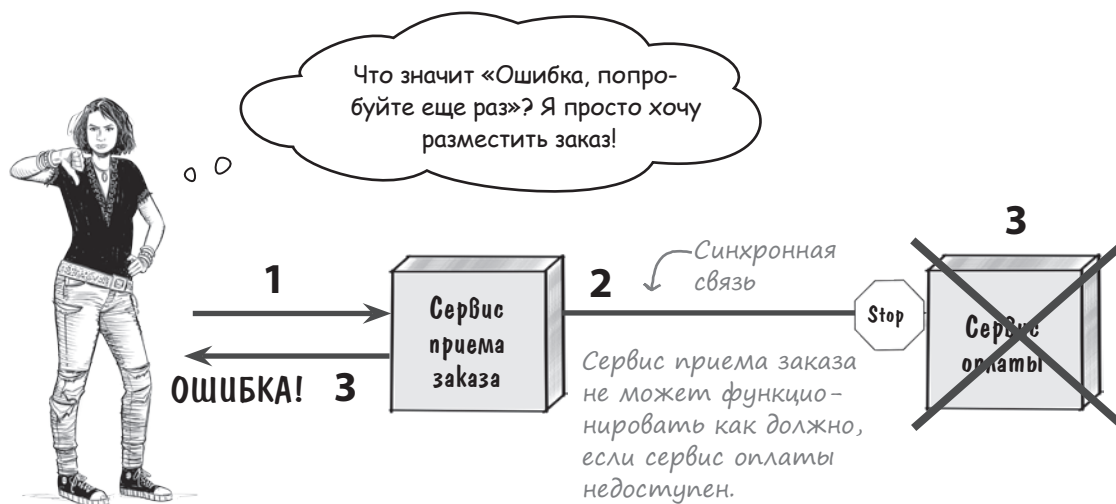
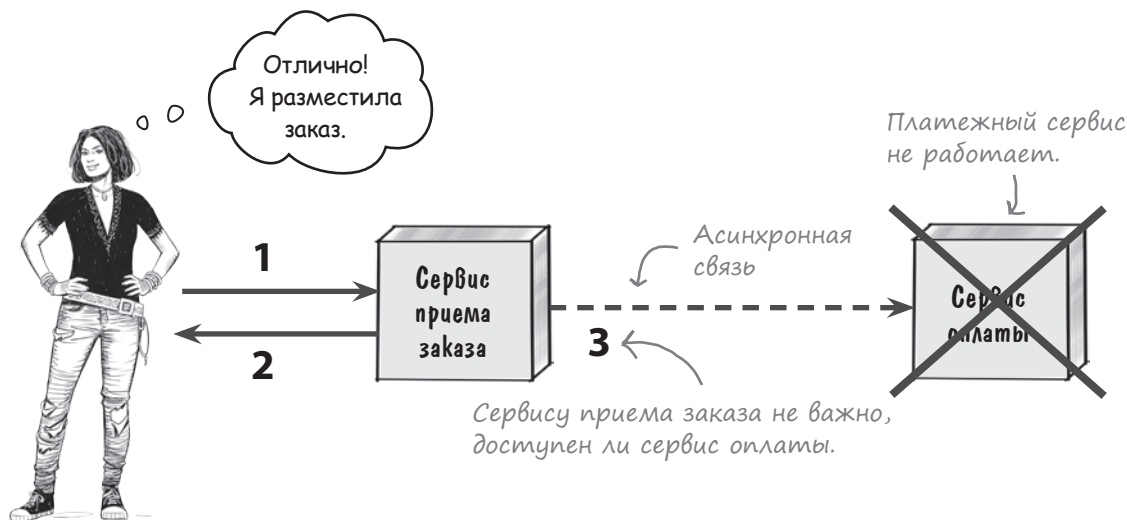
Асинхронная связь имеет много преимуществ. Первое — *отзывчивость*. Обработка асинхронных запросов требует меньше времени.

Предположим, клиент размещает заказ в Der Nile. Сервису приема заказа требуется 600 миллисекунд, или чуть больше половины секунды, чтобы проверить и разместить заказ, и 1200 миллисекунд, чтобы сервис оплаты принял платеж. При использовании асинхронной связи клиент будет ждать ответа 600 миллисекунд. При синхронной связи — 1800 миллисекунд (чуть меньше 2 секунд). Это большая разница во времени отклика.



Асинхронность ради победы (продолжение)

Еще одно преимущество асинхронной связи — *доступность*. Посмотрим, что случится, если сервис оплаты перестанет отвечать на запросы.

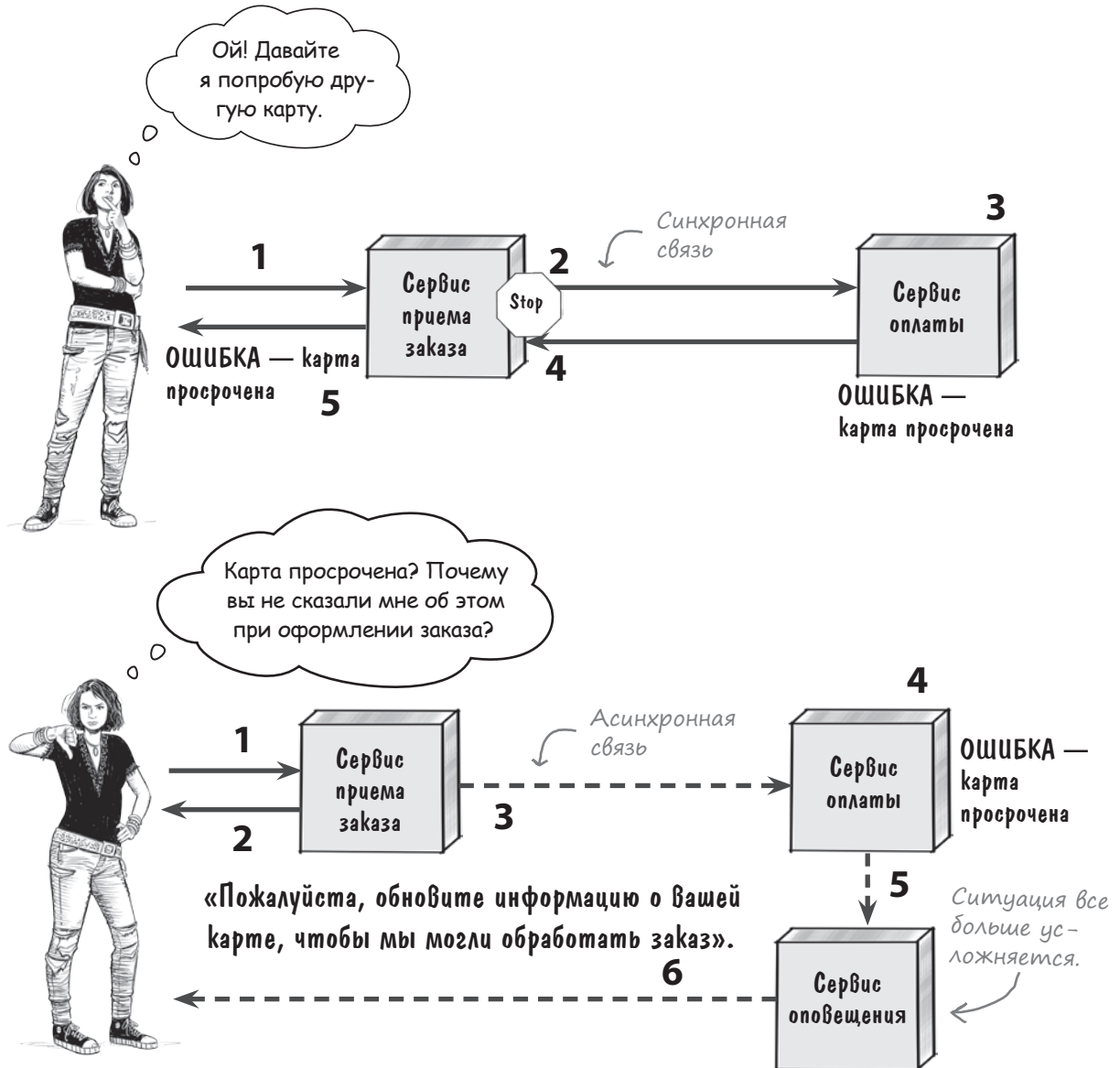


Мы убедили вас, что асинхронная связь — хороша? А теперь рассмотрим ее недостатки.

Вспомните Первый закон: все в архитектуре ПО основано на компромиссах.

Синхронность ради победы

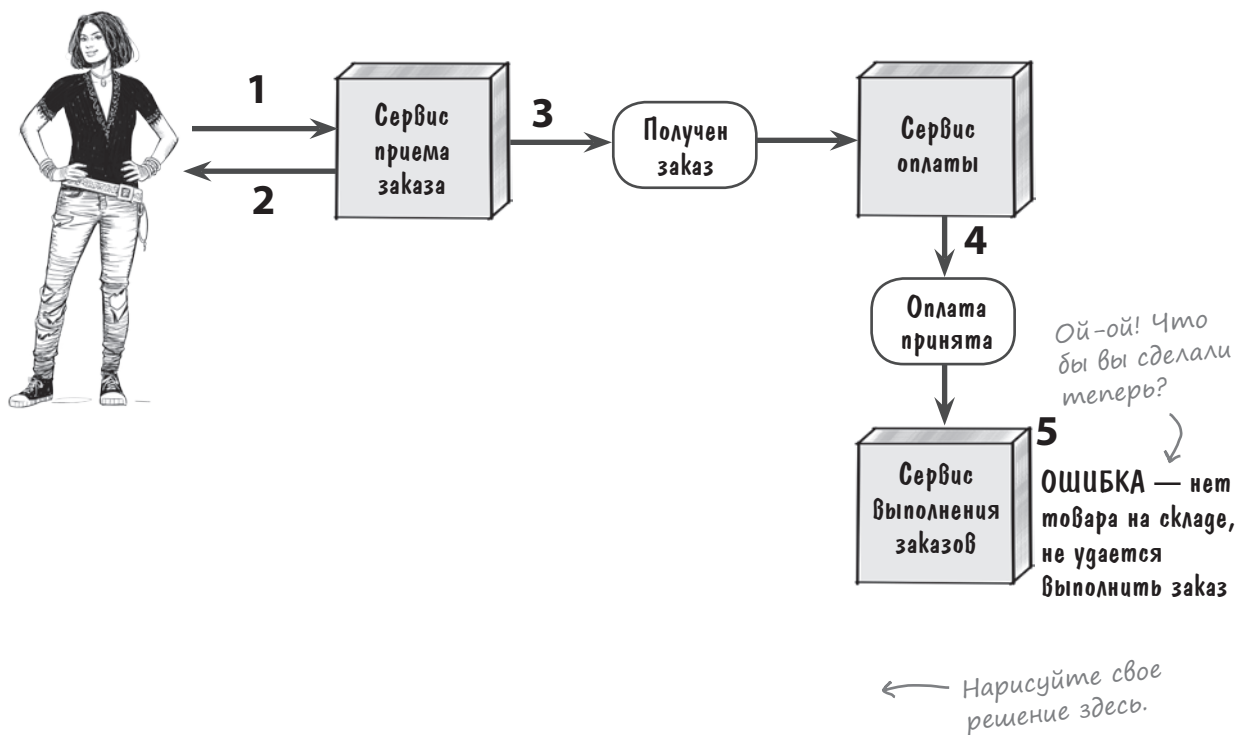
Главный недостаток асинхронности — *обработка ошибок*. При синхронных взаимодействиях клиент сразу узнает о проблеме с оплатой, сможет исправить ошибку и повторно отправить заказ. При асинхронных взаимодействиях клиент думает, что все в порядке, поскольку система не сообщила об обратном, но сервис приема заказа не сможет обработать заказ, пока клиент не решит проблему с оплатой. Это усложняет обработку ошибок.



Возьмите в руку карандаш

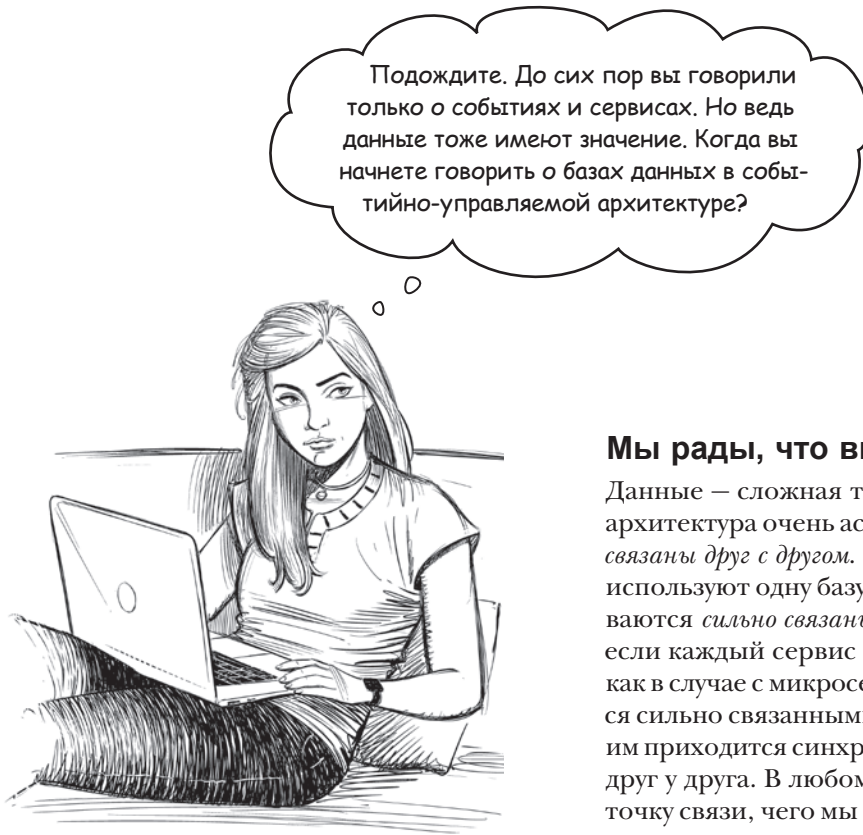
Сейчас у вас есть шанс проверить свои знания о событийно-управляемой архитектуре, событиях и асинхронной обработке. Сервис запасов компании Der Nile периодически рассинхронизируется, как показано ниже. Когда это происходит, выполнение заказа откладывается. Der Nile никогда не заставляет платить за товары, которые недоступны, и клиент может выбрать, ждать или отменить заказ.

Какие дополнительные сервисы и события вы как архитектор Der Nile добавили бы для решения проблемы?



Решение на с. 427

Топологии баз данных



Подождите. До сих пор вы говорили только о событиях и сервисах. Но ведь данные тоже имеют значение. Когда вы начнете говорить о базах данных в событийно-управляемой архитектуре?

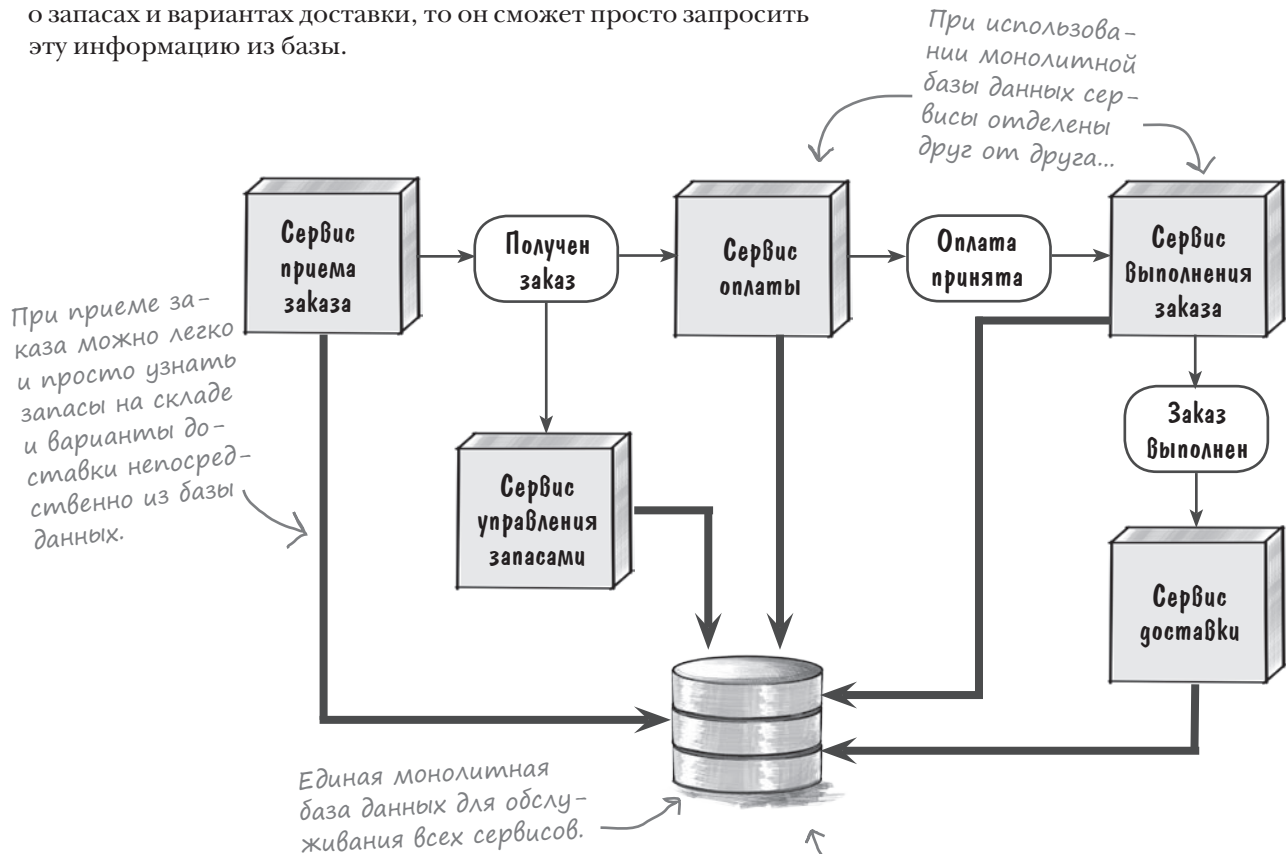
Мы рады, что вы подняли эту тему.

Данные — сложная тема в EDA, поскольку эта архитектура очень асинхронна и сервисы *слабо связаны друг с другом*. Однако если все сервисы используют одну базу данных, то в итоге оказываются *сильно связаны с ней*. С другой стороны, если каждый сервис владеет своими данными, как в случае с микросервисами, то они становятся сильно связанными друг с другом, поскольку им приходится синхронно запрашивать данные друг у друга. В любом случае данные образуют точку связи, чего мы стараемся избегать в EDA.

Далее мы покажем разные способы организации баз данных в EDA: *монолитные базы данных*, *базы данных с разделением на предметные области* и паттерн «*база данных на сервис*». Мы расскажем об их преимуществах, чтобы впоследствии вы могли решить, какой из них лучше подходит для вашей ситуации.

Монолитная база данных

В **монолитной** топологии все сервисы используют одну базу данных. Главное преимущество заключается в том, что когда сервисам нужны не принадлежащие им данные, они могут обратиться непосредственно к базе. То есть им не нужно выполнять синхронные вызовы других сервисов ради получения данных. Например, если сервису приема заказа понадобятся данные о запасах и вариантах доставки, то он сможет просто запросить эту информацию из базы.



За такое разделение приходится платить сложностью управления изменениями в базе данных. Все сервисы используют одну базу, поэтому при изменении структуры одной таблицы будет сложно определить, на какие сервисы они повлияют. Тестирование и выпуск превращаются в сложный процесс, и это приводит к появлению хрупких систем, которые сложно поддерживать.

Более того, общая база данных становится единой точкой отказа и плохо масштабируется по мере роста системы.

Оценка монолитной топологии базы данных

Вот оценочный лист для данной топологии.

Монолитные базы данных не обязательно хороши или плохи — все зависит от того, что для вас важно.

Оценочный лист монолитной топологии базы данных

☒

Связанность сервисов:

Низкая

Высокая

ХОРОШО

Сервисы слабо связаны благодаря отсутствию синхронных взаимодействий, выполняемых для получения данных.

☒

Производительность:

Низкая

Высокая

ХОРОШО

Данные извлекаются быстро, поскольку сервисы обращаются к БД напрямую, а не через синхронные вызовы других сервисов.

☒

Простота реализации:

Низкая

Высокая

ХОРОШО

Монолитная топология БД довольно проста. Соединения и сложные запросы выполняются в базе, а не в сервисах.

☒

Простота изменения:

Низкая

Высокая

ПЛОХО

Изменение структуры таблицы может повлиять на множество сервисов, что затрудняет тестирование систем.

☒

Отказоустойчивость:

Низкая

Высокая

ПЛОХО

Если монолитная БД выйдет из строя, то «упадет» и вся система, что ухудшает общую отказоустойчивость и доступность.

☒

Масштабируемость:

Низкая

Высокая

ПЛОХО

Обеспечивать масштабируемость и эластичность сложнее, поскольку БД необходимо масштабировать вместе с сервисами, а они могут исчерпать возможности соединения с БД.

Базы данных с разделением на предметные области

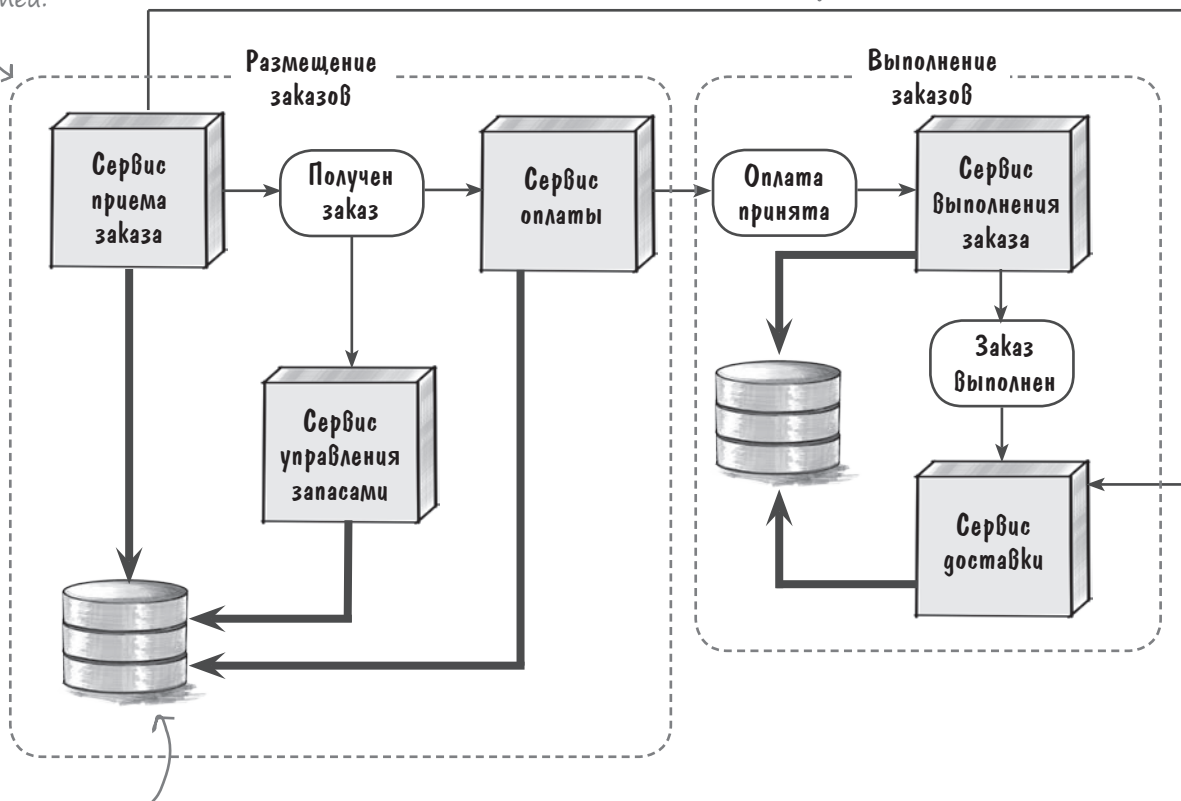
При *разделении на предметные области* каждая *область* в системе получает свою базу данных, то есть все сервисы, принадлежащие к определенной области, вместе используют свою базу. Например, сервисы приема заказов, оплаты и управления запасами являются частью области «Размещение заказов», поэтому все они используют одну и ту же физическую базу данных.

Однако каждая область образует свой ограниченный контекст, поэтому сервисы из одной области не могут напрямую обращаться к базе другой области. А это значит, что для получения данных им придется выполнять синхронные вызовы, из-за чего возрастает связанность сервисов.

Об ограниченных контекстах рассказывается в главе 10.

Сервис приема заказа должен синхронно вызывать сервис доставки, чтобы получить варианты доставки.

Пунктиром обозначены границы предметных областей.



Все сервисы в области размещения заказов могут напрямую обращаться к базе данных своей области.

Эта топология — хорошо сбалансированное промежуточное решение в рамках событийной архитектуры.

Оценка топологии с разделением на предметные области

Вот оценочный лист для данной топологии.

Топология базы данных с разделением на предметные области обеспечивает хороший баланс.

Оценочный лист топологии базы данных с разделением на предметные области

☒ Связанность сервисов: Низкая Высокая

ПЛОХО

Сервисы *внутри* области слабо связаны благодаря общим данным, но им приходится синхронно вызывать другие сервисы, чтобы получить данные *из другой* области.

☒ Производительность: Низкая Высокая

ХОРОШО

Данные *внутри* области извлекаются быстро, так как сервисы обращаются к БД напрямую. Но из *других* областей данные извлекаются медленнее из-за использования синхронной связи.

☒ Простота реализации: Низкая Высокая

ХОРОШО

Так как данные часто имеют естественное предметное деление, эта топология не очень сложна (но и не так проста, как монолитная).

☒ Простота изменения: Низкая Высокая

ПЛОХО

Изменение структуры таблиц БД влияет только на сервисы предметной области, что ограничивает круг влияния изменений.

☒ Отказоустойчивость: Низкая Высокая

ПЛОХО

Сбои в работе БД влияют только на сервисы внутри области, что делает эту топологию более отказоустойчивой, чем монолитная.

☒ Масштабируемость: Низкая Высокая

ПЛОХО

Хотя это все еще минус, но масштабируемость этой топологии немного лучше, чем монолитной, — БД должны масштабироваться только на уровне области, а не всей системы.

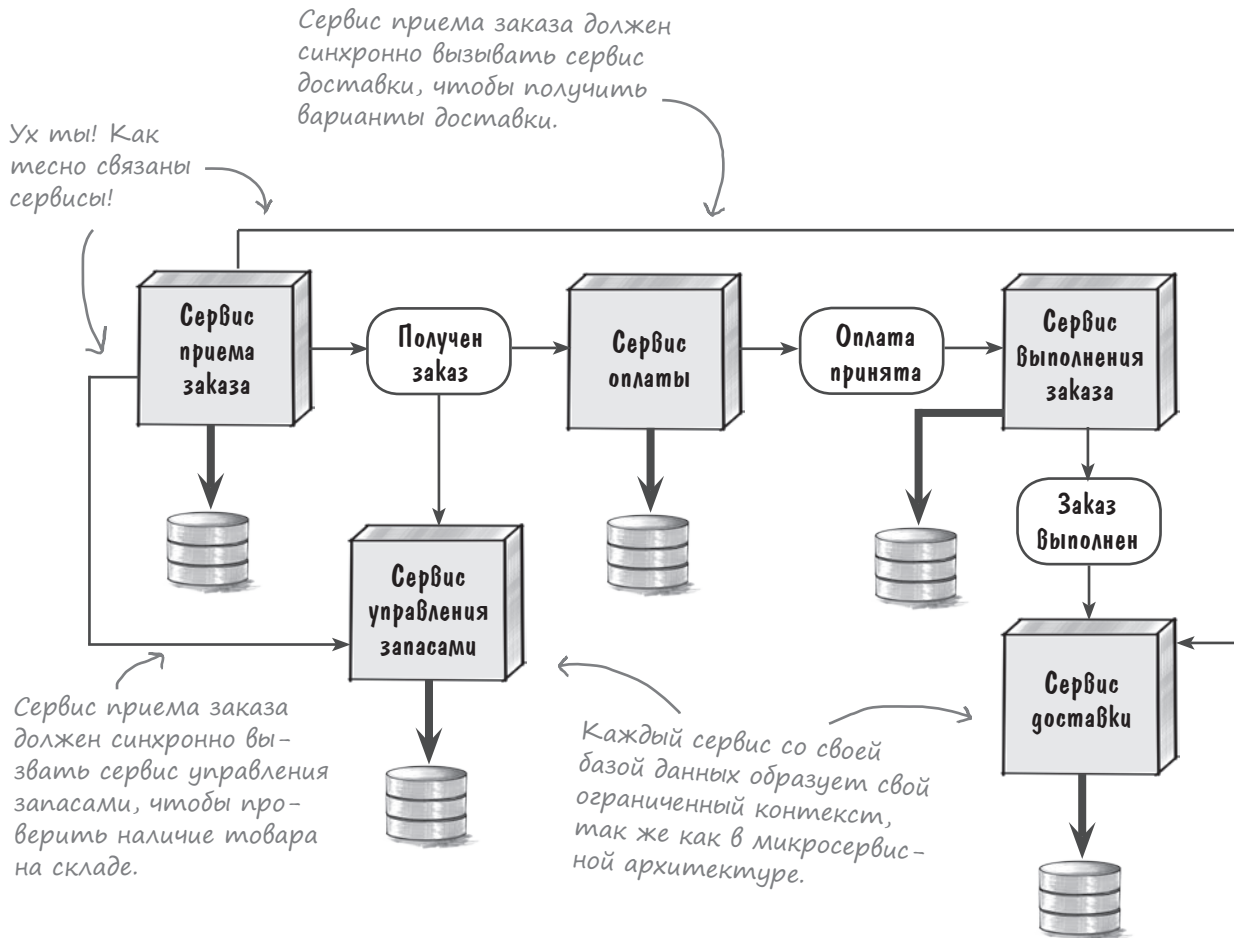
Отдельная база данных для каждого сервиса

Помните обсуждение паттерна «База данных на сервис» в главе 10? Так вот, он подходит не только для микросервисов, но и для EDA.

Название «База данных на сервис» точно отражает идею паттерна: каждый сервис получает свою базу, образуя еще более узкий ограниченный контекст, чем в топологии с предметными областями. Внести изменения в такие базы не составляет никакого труда, поскольку изменения затрагивают только сервис, владеющий данными (записывающий их в базу). Кроме того, повышаются отказоустойчивость и масштабируемость. Чем плохо?

К сожалению, многим. Для получения дополнительных данных сервисам приходится *обращаться* к их владельцам, используя синхронные вызовы. Это приводит к увеличению связанности сервисов и росту их взаимодействия, что влечет снижение производительности.

Мы надеемся, у вас достаточно денег! Это может быть дорого.



Оценка топологии с отдельными БД для каждого сервиса

Вот оценочный лист для этой топологии.

Эта топология по своим сильным и слабым сторонам — полная противоположность монолитной топологии.

Оценочный лист топологии с отдельными базами данных для каждого сервиса

☒	Связанность сервисов:	Низкая	Высокая
ПЛОХО		Сервисы имеют прямой доступ только к своим данным, а за чужими должны обращаться к другим сервисам, что создает тесные связи в слабо связанной архитектуре. Это плохо.	
☒	Производительность:	Низкая	Высокая
ПЛОХО		Другие сервисы для получения данных используют синхронные вызовы, поэтому производительность ниже, чем в других топологиях.	
☒	Простота реализации:	Низкая	Высокая
ПЛОХО		Разбить данные на ограниченные контексты очень трудно из-за связей между таблицами и артефактами (такими, как внешние ключи, триггеры, представления и хранимые процедуры).	
☒	Простота изменения:	Низкая	Высокая
ХОРОШО		Изменение структуры таблиц затрагивает только один сервис в пределах ограниченного контекста, что упрощает внесение изменений в БД по сравнению с другими топологиями.	
☒	Отказоустойчивость:	Низкая	Высокая
ХОРОШО		Сбои в базе данных влияют только на владеющий ею сервис, что делает эту топологию самой отказоустойчивой из трех.	
☒	Масштабируемость:	Низкая	Высокая
ХОРОШО		Базы данных масштабируются на уровне сервисов, что делает эту топологию высокомасштабируемой и эластичной.	



Упражнение

Для каждой из этих бизнес-потребностей выберите топологии, наиболее подходящие на ваш взгляд. Можно выбрать несколько.

Мы ожидаем, что с системой одновременно будут работать от 20 до 300 000 клиентов.

☐ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис

Система должна работать максимально быстро.

☐ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис

Это система медицинского мониторинга, и при выходе из строя одной ее части остальные должны продолжать работать.

☐ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис

Ожидается, что в будущем структура базы данных сильно изменится.

☐ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис

Нам нужно как можно скорее запустить новую систему.

☐ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис

Наша модель данных велика и сложна, она пронизана множеством взаимосвязей.

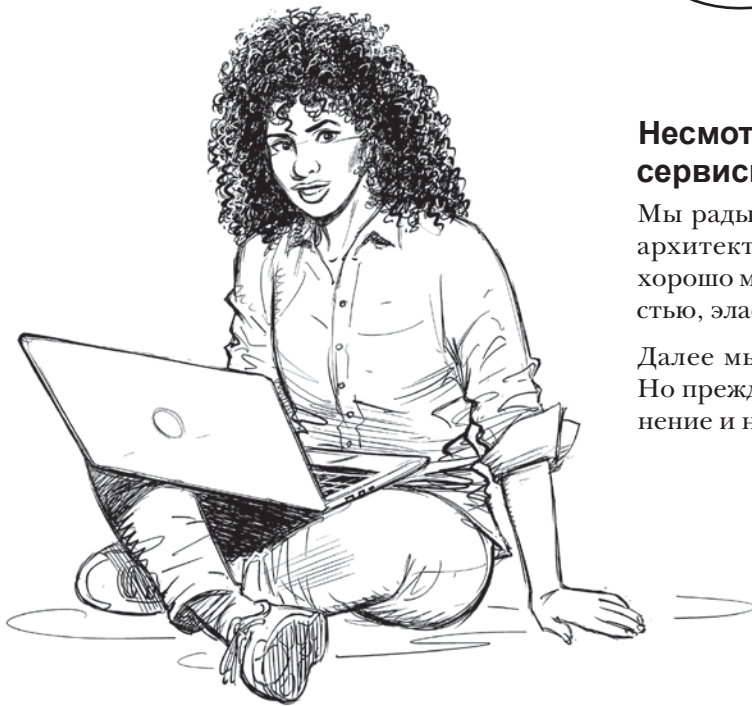
☐ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис

→ Решение на с. 428

Событийно-управляемая архитектура, использующая топологию «база данных на сервис», очень похожа на микросервисы. В чем же разница?



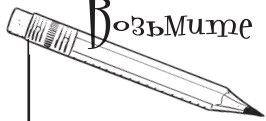
Несмотря на видимое сходство, микросервисы и EDA сильно различаются.

Мы рады, что вы заметили сходство между двумя архитектурами. Обе являются распределенными, хорошо масштабируются и обладают высокой гибкостью, эластичностью и отказоустойчивостью.

Далее мы покажем несколько важных различий. Но прежде попробуйте выполнить короткое упражнение и найти их самостоятельно.

Если вы не справитесь с упражнением, то ничего страшного — продолжайте читать!

Возьмите в руку карандаш



Перечислите все различия между событийно-управляемой архитектурой и микросервисами, которые сможете назвать.

Решение на с. 429

Сравнение событийно-управляемой и микросервисной архитектур

Далее мы познакомимся с шестью основными различиями между событийно-управляемой и микросервисной архитектурами в порядке возрастания важности. Готовы? Поехали!

Настало время
обратного отсчета
для Head First!

Номер 6: производительность

На шестом месте — *производительность*.

В книге «Фундаментальный подход к программной архитектуре» мы, ее авторы, составили оценочные таблицы для каждой архитектуры. За производительность мы дали микросервисам только две звезды, а EDA — пять. Почему?

EDA сочетает асинхронную обработку с возможностью одновременно выполнять несколько действий, позволяя создавать очень быстрые системы. Микросервисы, в силу ограниченности контекста и детализации, часто нуждаются в *синхронных* взаимодействиях, создающих большие задержки, которые замедляют работу системы.

Архитектурная характеристика	Рейтинг в звездах
Производительность	★★★★★

Событийно-управляемая архитектура

Архитектурная характеристика	Рейтинг в звездах
Производительность	★★

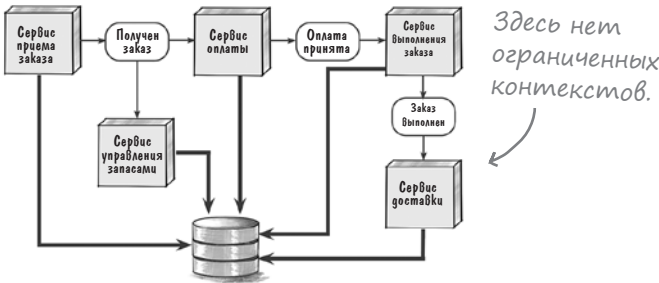
Микросервисная архитектура

Номер 5: ограниченные контексты

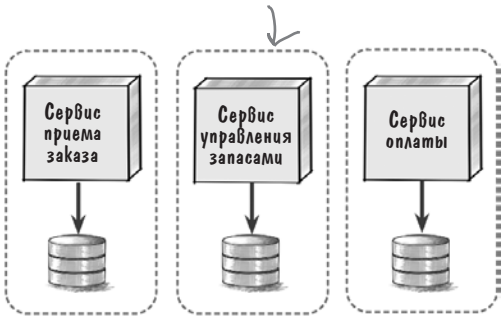
На пятое место мы поместили *ограниченные контексты*. Без них микросервисы не будут работать.

Однако в EDA ограниченные контексты играют роль лишь приятного дополнения и не являются основополагающими (или даже необходимыми). Совместное использование данных довольно типично для EDA, поэтому она не ограничивает владение данными так строго, как микросервисы.

Все эти сервисы и их данные
находятся в ограниченных
контекстах.



Событийно-управляемая архитектура



Микросервисная архитектура

Номер 4: детализация данных

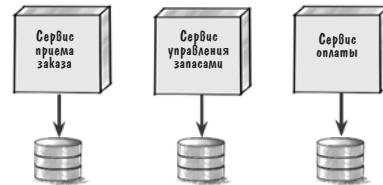
Еще одно ограничение, присутствующее в микросервисах и отсутствующее в EDA, — *детализация данных*.

Микросервисная архитектура требует, чтобы каждый сервис владел своими данными. Поэтому данные приходится делить на небольшие базы, или *схемы* (совокупности таблиц), которыми владеют сервисы (и куда они записывают данные). Но в EDA можно выбирать между монолитной базой данных, БД с разделением на предметные области или паттерном «База данных на сервис».

Вспомнить причины этого ограничения вы сможете, вернувшись к главе 10.



Событийно-управляемая архитектура



Микросервисная архитектура

Номер 3: детализация сервисов

Номер 3 в нашем топ-6 различий — *детализация сервисов*. Как отмечалось в главе 10, микросервис — это *сервис, решающий одну задачу*. Поэтому микросервисы часто получаются узкоспециализированными.

В EDA нет такого ограничения. Сервисы в событийной архитектуре (формально называемые *обработчиками событий*), могут иметь любой размер: быть и крупными, и мелкими.

Все сервисы должны быть одноцелевыми — отсюда и приставка «микро».

Сервисы могут иметь любой размер.



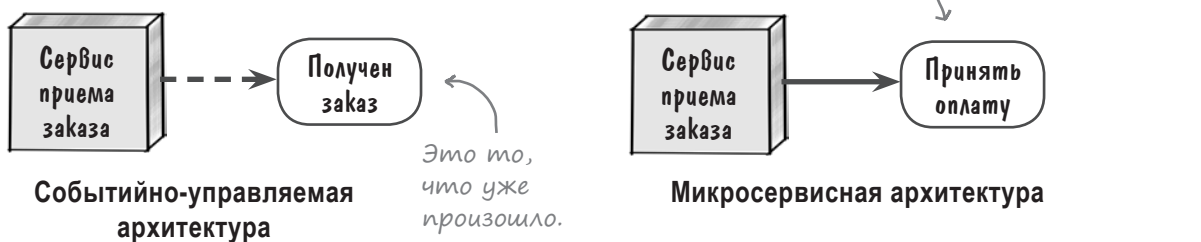
Событийно-управляемая архитектура



Микросервисная архитектура

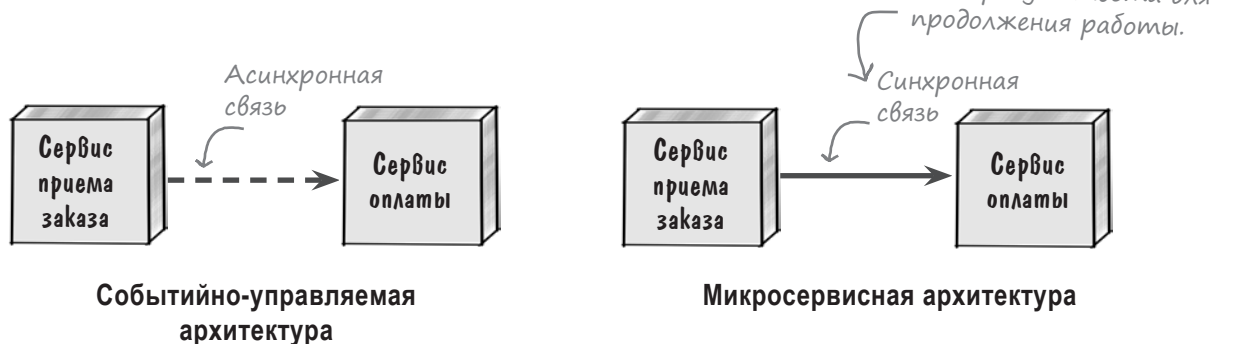
Номер 2: обработка событий в сравнении с обработкой запросов

Мы почти у цели. На втором месте еще одно фундаментальное различие: *обработка событий и обработка запросов*. Событийная архитектура базируется на *обработке событий* — реагировании на происходящее и запуске других событий. Микросервисная архитектура построена на *обработке запросов* — реагировании на что-то, что должно произойти (например, команда или запрос), и обработке этого запроса.



Номер 1: стиль взаимодействия

И наконец, первое место занимает самое важное различие между EDA и микросервисами: *стиль взаимодействия*. В EDA сервисы обычно взаимодействуют асинхронно, тогда как микросервисы часто используют синхронную связь и REST. Сервисы в EDA тоже могут задействовать синхронные вызовы, например, для получения данных, к которым у них нет прямого доступа, а микросервисы могут использовать асинхронную связь, когда команды не требуют ответа. Но это скорее исключение, чем правило.



Что чему соответствует?

Мы попытались упорядочить эти факты, но запутались. Помогите нам разобраться, какие утверждения относятся к EDA, а какие к микросервисам. Будьте внимательны, некоторые утверждения относятся к обеим архитектурам.

Событийно-
управляемая
архитектура

Пример
от нас

Микросервисная
архитектура

Мне нужен ограниченный контекст.

Я позволяю использовать паттерн «База данных на сервис».

Я создаю масштабируемые системы.

Я использую в основном синхронную связь.

Я создаю высокопроизводительные системы.

Я экономлю деньги, поскольку стою недорого.

Я использую в основном асинхронную связь.

Мои сервисы общаются с помощью событий.

Я могу похвастаться отказоустойчивостью.

Я могу использовать монолитную БД.

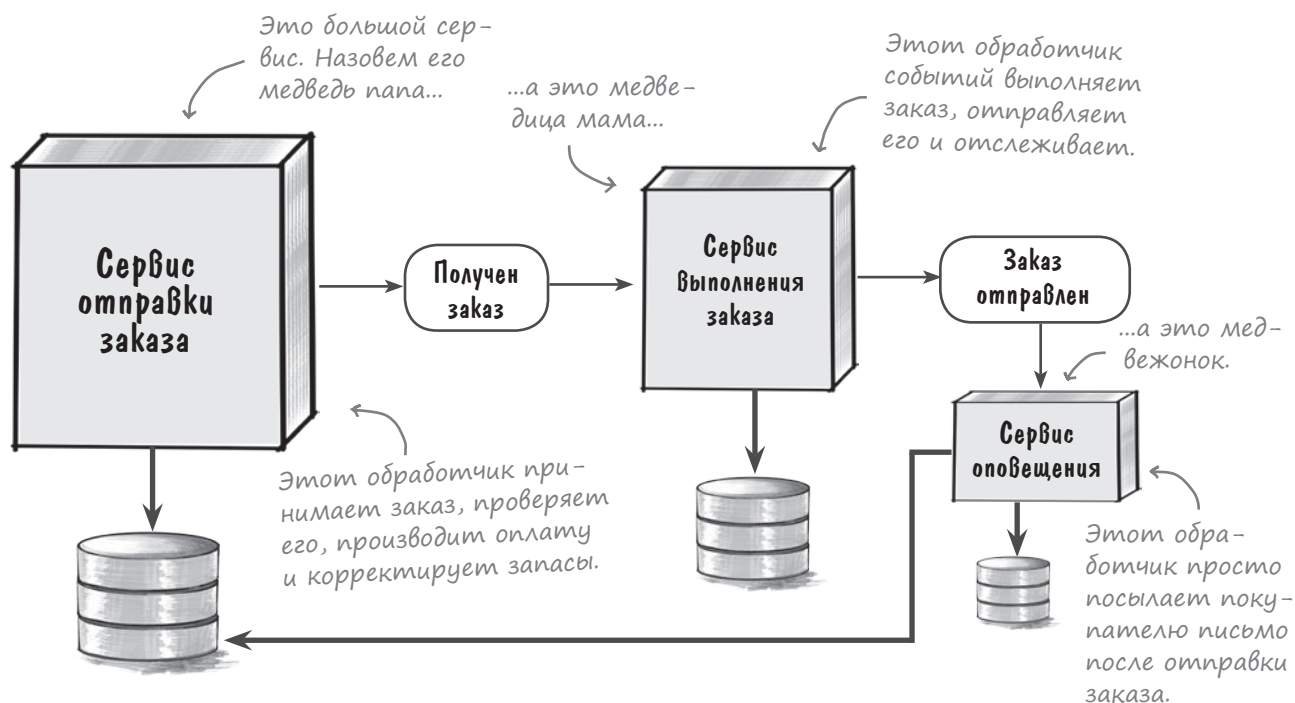
Решение на с. 430

Гибриды: событийно-управляемые микросервисы

Несмотря на все различия между EDA и микросервисами, нет причин, мешающих их объединению. На их основе можно создать *гибридную* архитектуру, называемую *событийно-управляемыми микросервисами*.

Возможно, вы заметили, что паттерн «База данных на сервис» очень похож на микросервисы. Однако простое использование этого паттерна не приводит к созданию архитектуры событийно-управляемых микросервисов. Чтобы было понятнее, взгляните на эту EDA.

Гибридная архитектура сочетает несколько архитектурных стилей.



Возьмите в руку карандаш

Архитектура выше — правильно сформированная EDA, но она не является правильно сформированной микросервисной архитектурой. В ней недостает двух фундаментальных принципов, которые превратили бы ее в событийно-управляемые микросервисы. Попробуйте указать эти принципы.

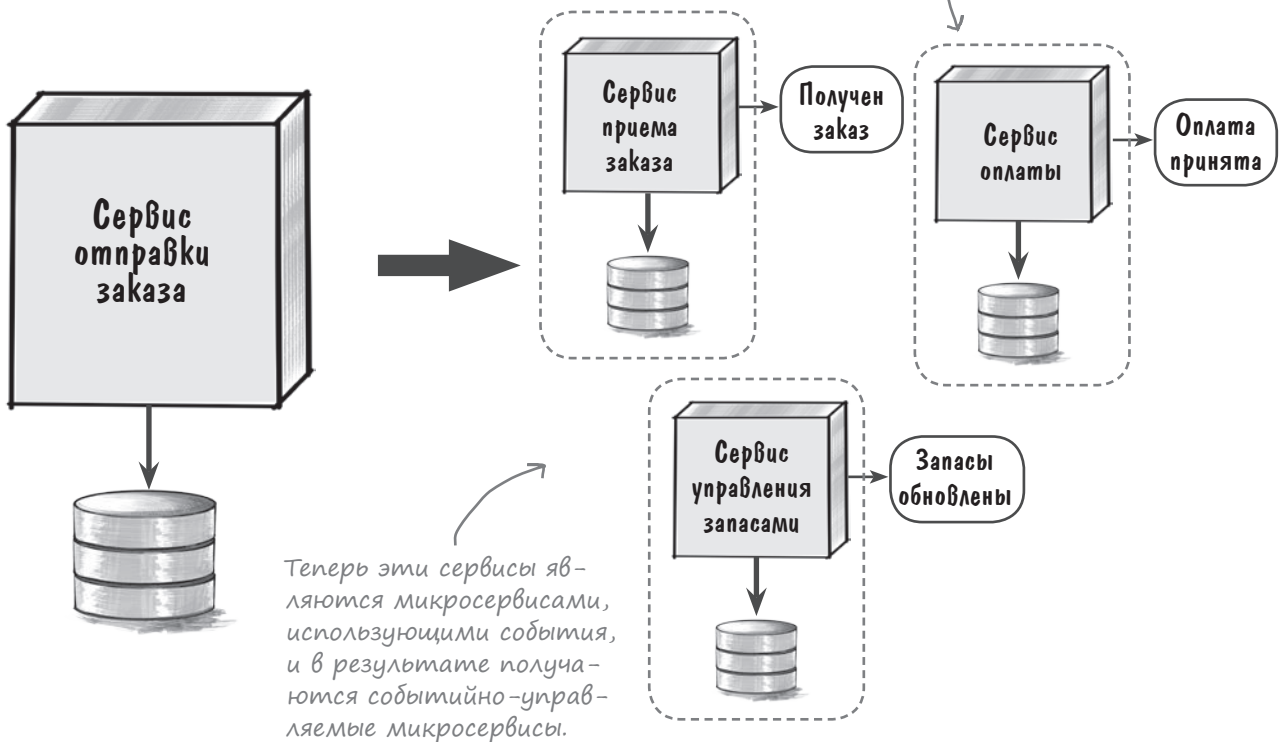
1. _____

2. _____

→ Решение на с. 431

Чтобы превратить эту архитектуру в событийно-управляемые микросервисы, нужно применить два очень важных принципа микросервисов: сервисы должны быть одноцелевыми и образовывать ограниченные контексты.

Сервис отправки заказа должен принять и проверить заказ, получить оплату и скорректировать запасы. Это явно не одноцелевой сервис. Такое допустимо в EDA, но не в микросервисах. То же можно сказать о сервисе выполнения заказов. Чтобы превратить эту архитектуру в событийно-управляемые микросервисы, нужно разделить эти сервисы на отдельные одноцелевые микросервисы, каждый из которых будет запускать свои события.



На предыдущей странице можно заметить, что сервис оповещения напрямую обращается к базе данных сервиса отправки заказа. В микросервисах это недопустимо из-за ограниченных контекстов. В новой гибридной архитектуре, чтобы получить данные о заказе, сервис оповещения должен вызвать сервис отправки заказа (теперь это сервис приема заказа, появившийся после разделения). Жесткие ограниченные контексты запрещают прямой доступ к данным всем, кроме владельца.

Сверхвозможности событийно-управляемой архитектуры

Пришло время перечислить сверхвозможности EDA.

Удобство сопровождения

Сервисы в EDA слабо связаны, что делает их более независимыми и удобными в сопровождении.

Бах!



Производительность

EDA очень быстрая, использует в основном асинхронные взаимодействия и может работать в многозадачном режиме.

Бум!

Развиваемость

Сервисы EDA всегда генерируют производные события, обработку которых легко расширить. Это очень упрощает развитие EDA.

Едац!

Масштабируемость

Событийно-управляемые архитектуры обладают высокой масштабируемостью благодаря асинхронной обработке и слабой связанности сервисов. Каждый сервис может масштабироваться независимо от других, а каналы событий служат клапанами сброса давления при возникновении узких мест.

Отказоустойчивость

Сервисы в EDA связаны слабо, поэтому выход нарушению работы одного из них не приведет к остановке других сервисов.

Слабые стороны событийно-управляемой архитектуры

Как криптонит уменьшает силу супергероя, так и эти характеристики уменьшают привлекательность EDA. Будьте внимательны при работе с ними!



Сложность

EDA — очень сложная архитектура из-за использования асинхронных взаимодействий и параллельной обработки событий, а также из-за разнообразия топологий баз данных и их компромиссов.

Тестируемость

Асинхронную и параллельную обработку очень сложно тестировать, что делает тестируемость слабым местом EDA.



Синхронные вызовы

Если вам нужно, чтобы сервисы и процессы выполняли множество синхронных вызовов, то EDA не для вас.

Мы знаем, что базы данных нужны, но хотим сказать, что они могут связывать системы, которые в ином случае были бы совершенно независимыми.

Базы данных

Независимо от выбранной топологии базы данных, сильно связанные сервисы останутся связанными — либо через базу данных, либо напрямую друг с другом. Здесь не так много хороших компромиссов.



Звездные рейтинги событийно-управляемой архитектуры

Следующая оценочная таблица позволит понять, в чем EDA хороша, а в чем не очень. Одна звезда означает, что архитектурная характеристика поддерживается плохо, а пять — что она поддерживается очень хорошо.

В точности как в рецензиях к фильмам.

Найти код, который нужно изменить, легко, но тестирование и развертывание — трудная и рискованная задача.

Меньшая связанность улучшает масштабируемость и эластичность.

Асинхронность и слабая связанность способствуют повышенной отказоустойчивости.

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★ ★ ★
Тестируемость	★ ★
Развертываемость	★ ★ ★
Простота реализации	★
Развиваемость	★ ★ ★ ★ ★
Производительность	★ ★ ★ ★ ★
Масштабируемость	★ ★ ★ ★ ★
Эластичность	★ ★ ★ ★
Отказоустойчивость	★ ★ ★ ★ ★
Общая стоимость	\$ \$ \$

Обработка ошибок и асинхронная связь делают EDA сложной.

Наконец, найден архитектурный стиль с отличной производительностью!

EDA обладает отличной производительностью, масштабируемостью, эластичностью, развиваемостью и отказоустойчивостью, но сложна в реализации и тестировании. Асинхронные взаимодействия тяжело тестировать, и не менее трудно проверить, влияет ли изменение или событие в одном сервисе на другие.



Упражнение

Для каких из следующих систем подходит или не подходит событийно-управляемая архитектура и почему? Подсказка: учитывайте достоинства и недостатки EDA, а также особенности предметной области.

Онлайн-аукцион, где пользователи делают ставки

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? _____

- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Сайт социальной сети, где пользователи могут оставлять комментарии и отвечать на них

Почему? _____

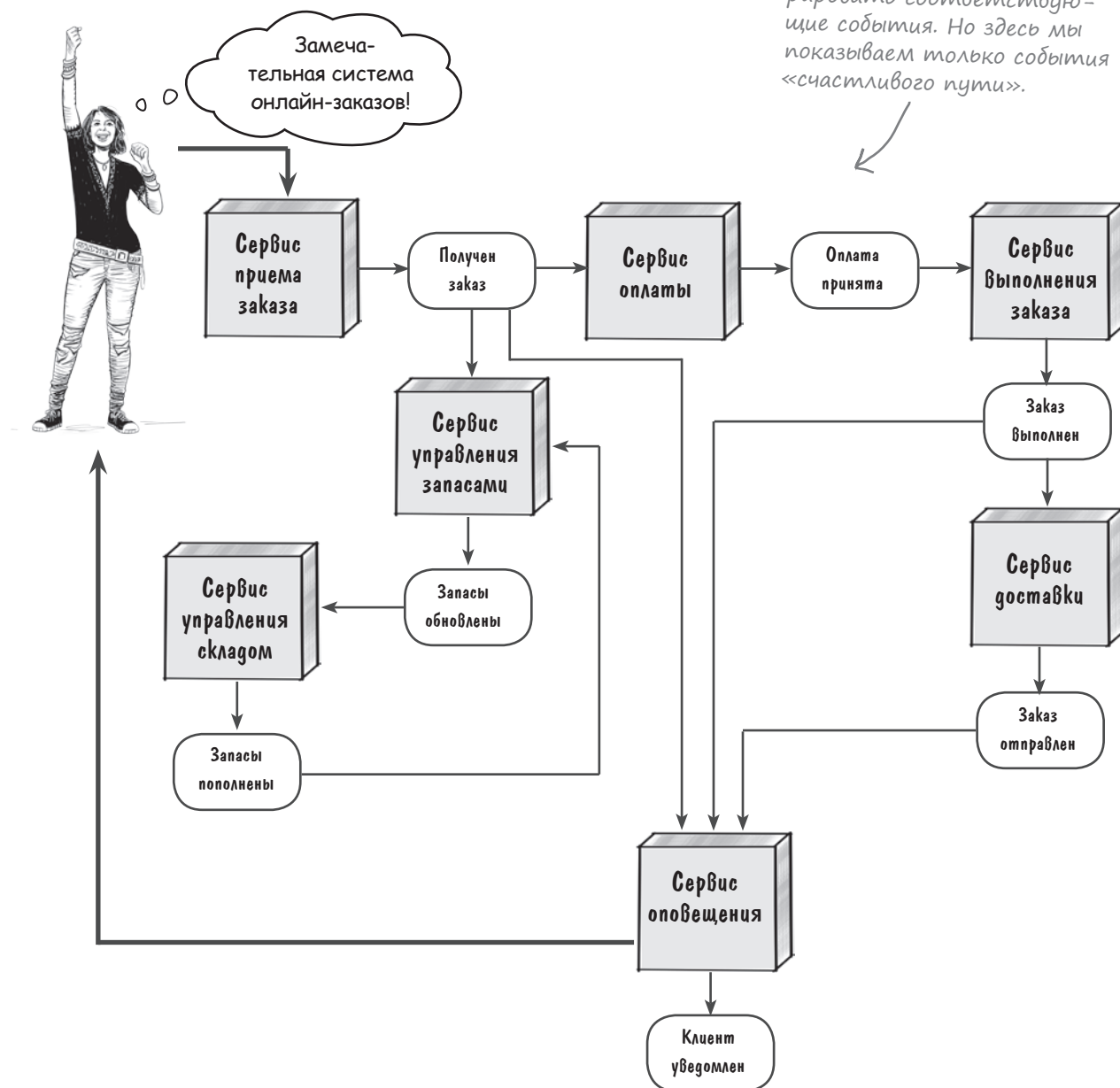
- ☐ Хорошо подходит
☐ Может использоваться
☐ Не подходит

→ Решение на с. 432

Собираем все воедино

Теперь рассмотрим полную картину системы онлайн-заказов для Der Nile с событийно-управляемой архитектурой. На выбор доступно несколько топологий баз данных, поэтому мы сосредоточимся на основных частях системы: обработчиках событий (сервисах) и событиях.

В этой архитектуре может возникнуть множество ошибок, в ответ на которые система будет генерировать соответствующие события. Но здесь мы показываем только события «счастливого пути».



В заключение

Поздравляем! Ваш кропотливый труд и знания EDA помогли компании Der Nile удовлетворить увеличивающийся спрос и получить возможность роста. В заключение вспомним некоторые ключевые особенности событийной архитектуры.

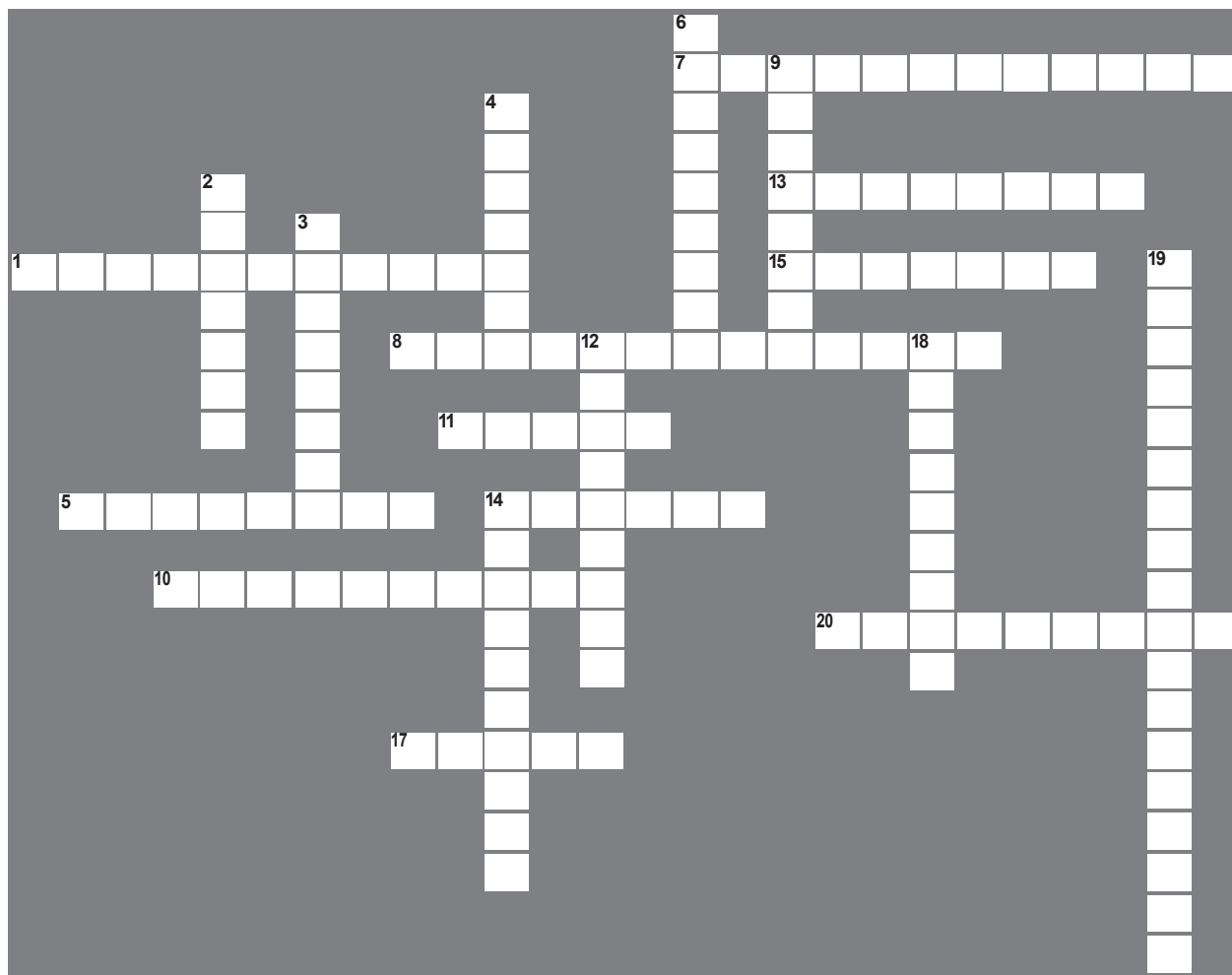
Ключевые Моменты

- *Событие* — это нечто, произошедшее в системе; основной способ взаимодействия сервисов в EDA.
- События и сообщения не одно и то же: *события* передают информацию о свершившихся действиях, а *сообщения* — это команды или запросы, адресованные одному сервису.
- *Иницилирующее событие* исходит от клиента или конечного пользователя и запускает процесс.
- *Производное событие* генерируется сервисом в ответ на иницилирующее событие.
- Любое действие, выполняемое сервисом, должно запускать производное событие, чтобы обеспечить *расширяемость архитектуры*.
- EDA работает быстро благодаря *асинхронной связи* — сервисы не ждут ответа или подтверждения от других сервисов, когда отправляют им информацию.
- Иногда асинхронные взаимодействия называют «запустил и забыл».
- Для обозначения асинхронных взаимодействий на диаграммах обычно используются пунктирные линии, а синхронных — сплошные.
- Событийно-управляемая архитектура может использовать несколько разных топологий баз данных.
 - При использовании *монолитной* топологии все сервисы работают с одной базой данных.
 - При использовании топологии с *разделением на предметные области* каждая область получает свою базу данных, с которой работают все сервисы из этой области.
 - При использовании паттерна «База данных на сервис» каждый сервис получает свою базу данных в ограниченном контексте.
- Событийная архитектура и микросервисы — это совершенно разные архитектурные стили.
 - EDA использует *асинхронные* взаимодействия между сервисами, тогда как микросервисы обычно задействуют *синхронные* вызовы, такие как REST.
 - EDA основана на *обработке событий* — того, что уже произошло, а архитектура микросервисов — на *обработке запросов* — команд или запросов, указывающих, что должно произойти.
 - Микросервисы — небольшие и одноцелевые сервисы, а в EDA сервисы могут иметь любой размер.
 - Микросервисы требуют, чтобы каждый сервис владел своими данными, тогда как в EDA сервисы могут вместе использовать данные.
- Микросервисы и EDA можно объединить и создать гибридную архитектуру событийно-управляемых микросервисов.
- EDA сложна в реализации, поскольку используются асинхронные взаимодействия, параллельная обработка событий и разные топологии баз данных.
- Асинхронную и параллельную обработку сложно тестировать, что делает тестируемость слабым местом EDA.
- Производные события позволяют добавлять новые функции, что делает EDA простой для развития.
- EDA хорошо масштабируется благодаря асинхронной обработке и слабой связанности сервисов.



Кроссворд по событийно-управляемой архитектуре

Готовы немного развлечься и проверить свои знания о событиях, асинхронных взаимодействиях, обработчиках событий и многозадачности? Разгадайте этот кроссворд.



По горизонтали

1. Событие, порождаемое другими событиями.
5. Стиль взаимодействий « ____ и забыл».
7. Характеристика системы, быстро откликающейся на действия пользователей.
8. ____ архитектуры позволяет добавлять новые функции.
10. Базы данных с разделением на ____ области.
11. Асинхронная связь позволяет не ____ ответа.
13. События используются для ____ новостей о произошедшем.
14. Часть архитектуры, выполняющая некоторую функцию.
15. Важное происшествие.
17. События доставляются через ____ событий.
20. Сервисы могут ____ события, чтобы реагировать на них.

По вертикали

2. Определенные условия могут ____ событие.
3. Объединение сервиса с базой данных дает ограниченный ____.
4. Последовательность событий образует рабочий ____.
6. ____ отправляются конкретным сервисам с помощью очередей.
9. Сообщение может быть командой или ____.
12. Приложение может вызвать прикладной программный ____.
14. Тип взаимодействий, требующий дождаться ответа.
16. Событие, запускающее бизнес-процесс.
18. Способ организации баз данных в архитектуре.
19. Архитектурная характеристика, связанная со скоростью.

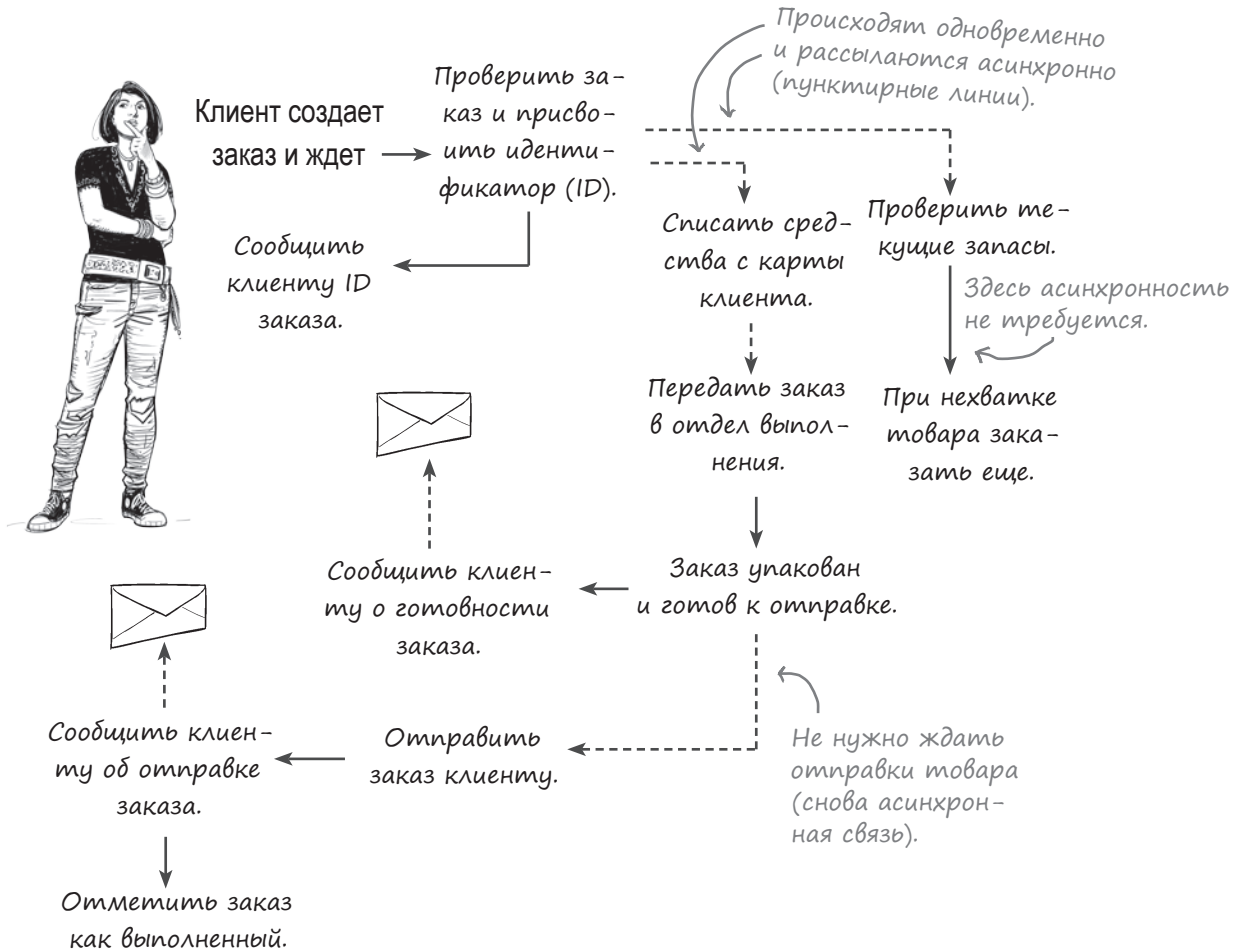
—————> Решение на с. 433



Упражнение Решение

На с. 385

Как бы вы, будучи архитектором, изменили рабочий процесс (показанный на с. 384), чтобы ускорить его? Нарисуйте свой вариант ниже. Подсказка: какие операции можно выполнять одновременно?





Возьмите в руку карандаш
Решение

На с. 389

Пришло время проверить ваши знания о событиях и сообщениях. Для каждой из следующих цитат укажите, чем она является: событием или сообщением, и объясните почему.

«Борт Adventurous Air 12, поверните влево, курс 230 градусов».

☐ Событие ☒ Сообщение

Обоснование: Это команда. Она отправлена одному самолету и сообщает,
что должно произойти.

«Другие новости: в район только что пришел зимний штормовой фронт».

☒ Событие ☐ Сообщение

Обоснование: Это информация для множества людей о только что произо-
шедшем событии.

«Откройте страницу 42 в ваших рабочих тетрадах».

☐ Событие ☒ Сообщение

Будьте осторожны —
здесь не все так просто!

Обоснование: Даже при том что данная информация передается многим
студентам, это команда, сообщающая, что должно произойти.

«Всем привет! Прошу прощения за опоздание».

☒ Событие ☐ Сообщение

Обоснование: Это информация о произошедшем для многих людей. Никакой
реакции не ожидается.

«О нет! Я только что опоздал на поезд!»

☒ Событие ☐ Сообщение

Обоснование: Даже если никто не слушает, это информация о произошедшем.
Она не направлена никому конкретному.

«Простите, можете уделить мне немного времени?»

☐ Событие ☒ Сообщение

Обоснование: Это просьба, обращенная к одному человеку, о том, что должно
произойти.



Упражнение Решение

На с. 391

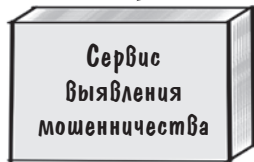
На основе инициирующего события «Оплата картой» и последующей обработки определите производные события для каждого сервиса. Придумайте как можно больше событий.

Клиент совершает
покупку по карте

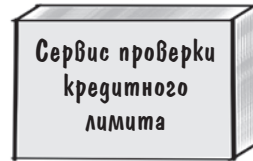
Это инициирующее
событие.



Оплата
картой

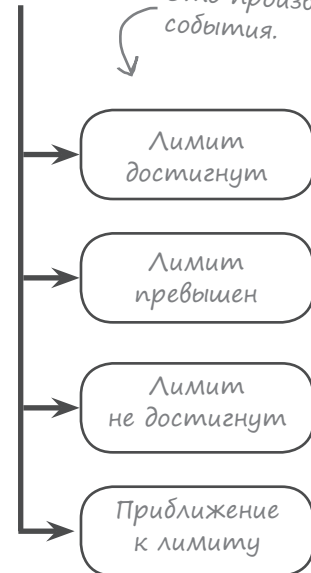
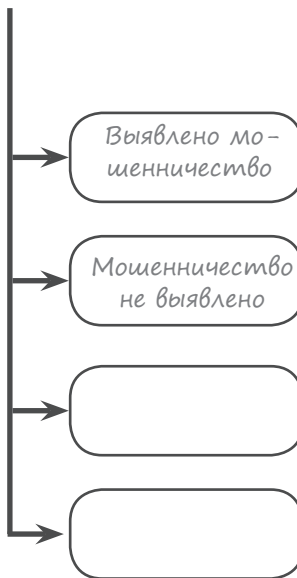


Сервис
выявления
мошенничества
Проверка
на мошенничество



Сервис проверки
кредитного
лимита
Проверка
кредитного лимита

Это производные
события.



Сообщить клиенту, что
его кредитный лимит
близок к исчерпанию.



Решение

Какую связь, синхронную или асинхронную, вы бы использовали для каждой из этих задач?

Сообщить варианты доставки заказа

☐ Асинхронная ☒ Синхронная ☐ Подойдет любая

Принять оплату заказа и сообщить, прошел ли платеж

☐ Асинхронная ☒ Синхронная ☐ Подойдет любая

Выполнить заказ, собрав товары и упаковав их в коробку

☒ Асинхронная ☐ Синхронная ☐ Подойдет любая

Сообщить текущий статус заказа

☐ Асинхронная ☒ Синхронная ☐ Подойдет любая

Запасы товара сокращаются — заказать еще

☒ Асинхронная ☐ Синхронная ☐ Подойдет любая

Сообщить клиенту, что заказ отправлен и находится в пути

☒ Асинхронная ☐ Синхронная ☐ Подойдет любая

Обновить фотографию в профиле клиента

☐ Асинхронная ☐ Синхронная ☒ Подойдет любая

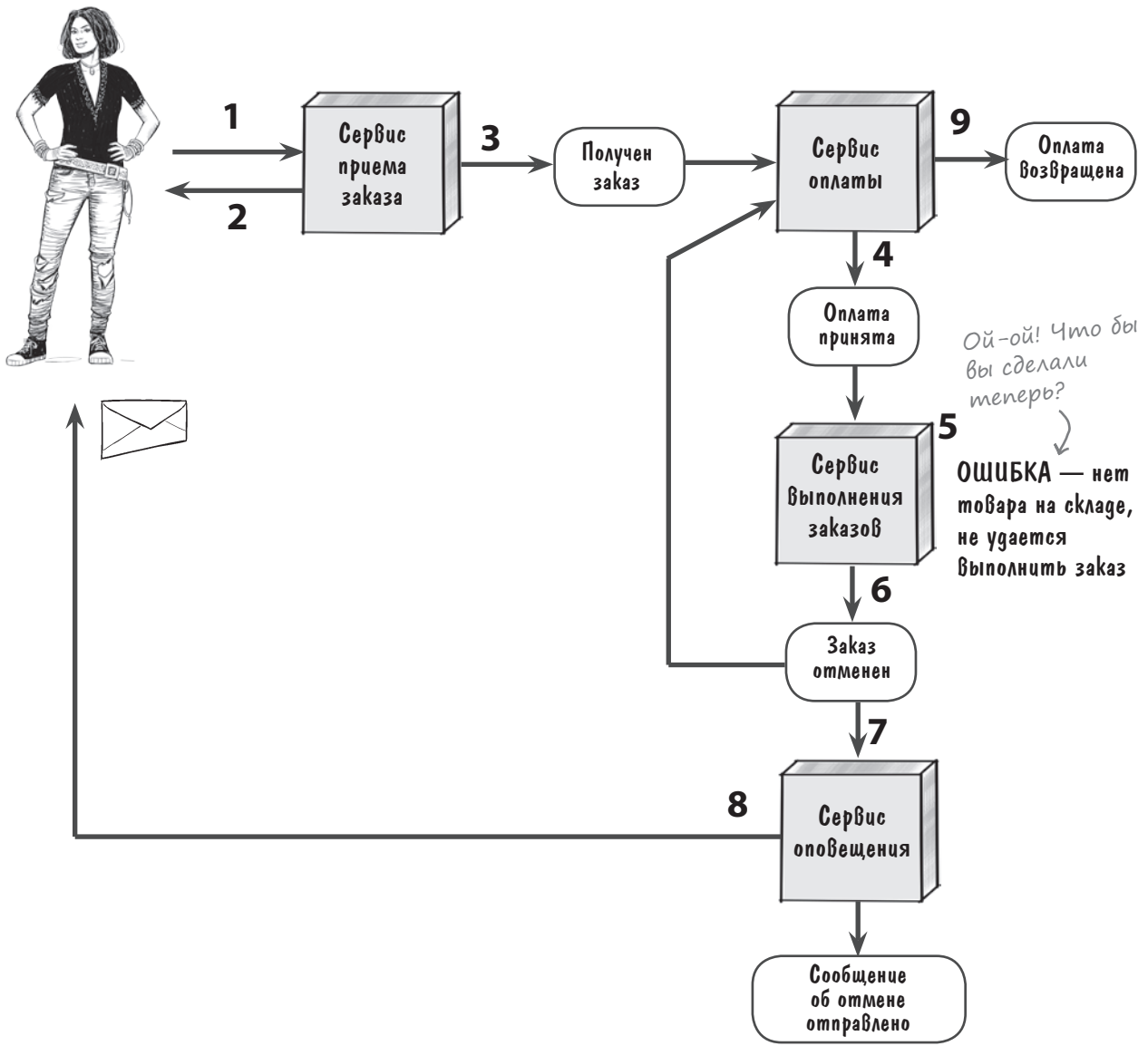
Разместить отзыв покупателя на странице продукта

☐ Асинхронная ☐ Синхронная ☒ Подойдет любая

Возьмите в руку карандаш Решение

Сейчас у вас есть шанс проверить свои знания о событийно-управляемой архитектуре, событиях и асинхронной обработке. Сервис запасов компании Der Nile периодически рассинхронизируется, как показано ниже. Когда это происходит, выполнение заказа откладывается. Der Nile никогда не заставляет платить за товары, которые недоступны, и клиент может выбрать, ждать или отменить заказ.

Какие дополнительные сервисы и события вы как архитектор Der Nile добавили бы для решения проблемы?





Упражнение Решение

На с. 408

Для каждой из этих бизнес-потребностей выберите топологии, наиболее подходящие, на ваш взгляд. Можно выбрать несколько.

Мы ожидаем, что с системой одновременно будут работать от 20 до 300 000 клиентов.

☐ Монолитная

☒ С разделением на предметные области

☒ База данных на сервис

Система должна работать максимально быстро.

☒ Монолитная

☒ С разделением на предметные области

☐ База данных на сервис

Это система медицинского мониторинга, и при выходе из строя одной ее части остальные должны продолжать работать.

☐ Монолитная

☐ С разделением на предметные области

☒ База данных на сервис

Ожидается, что в будущем структура базы данных сильно изменится.

☐ Монолитная

☒ С разделением на предметные области

☒ База данных на сервис

Нам нужно как можно скорее запустить новую систему.

☒ Монолитная

☒ С разделением на предметные области

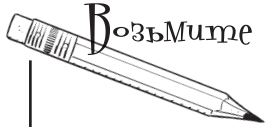
☐ База данных на сервис

Наша модель данных велика и сложна, она пронизана множеством взаимосвязей.

☒ Монолитная

☐ С разделением на предметные области

☐ База данных на сервис



Возьмите в руку карандаш

Решение

Перечислите все различия между событийно-управляемой архитектурой и микросервисами, которые сможете назвать.

Производительность

Ограниченные контексты

Детализация данных

Детализация сервисов

Различия между обработкой событий и запросов

Различия между синхронной и асинхронной обработкой

Вот что
придумали мы.

Что чему соответствует? Решение

Мы попытались упорядочить эти факты, но запутались. Помогите нам разобраться, какие утверждения относятся к EDA, а какие к микросервисам. Будьте внимательны, некоторые утверждения относятся к обеим архитектурам.





Возьмите в руку карандаш

Решение

Архитектура выше — правильно сформированная EDA, но она не является правильно сформированной микросервисной архитектурой. В ней недостает двух фундаментальных принципов, которые превратили бы ее в событийно-управляемые микросервисы. Попробуйте указать эти принципы.

1. Одноцелевые сервисы

2. Ограниченный контекст (владение данными)



Упражнение Решение

На с. 419

Для каких из следующих систем подходит или не подходит событийно-управляемая архитектура и почему? Подсказка: учитывайте достоинства и недостатки EDA, а также особенности предметной области.

Онлайн-аукцион, где пользователи делают ставки

Почему? Предметная область соответствует EDA, и для этой системы важны масштабируемость, эластичность и отзывчивость.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит

Финансовая система быстрых международных электронных переводов

Почему? Для решения этой задачи не нужны сверхвозможности EDA.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Компания, расширяющая бизнес и ожидающая постоянных изменений в своей системе

Почему? EDA может стать одним из вариантов, так как она упрощает внесение изменений.

- ☐ Хорошо подходит
☒ Может использоваться
☐ Не подходит

Небольшая пекарня, принимающая заказы онлайн

Почему? EDA слишком сложная и дорогая для маленькой пекарни.

- ☐ Хорошо подходит
☐ Может использоваться
☒ Не подходит

Сайт социальной сети, где пользователи могут оставлять комментарии и отвечать на них

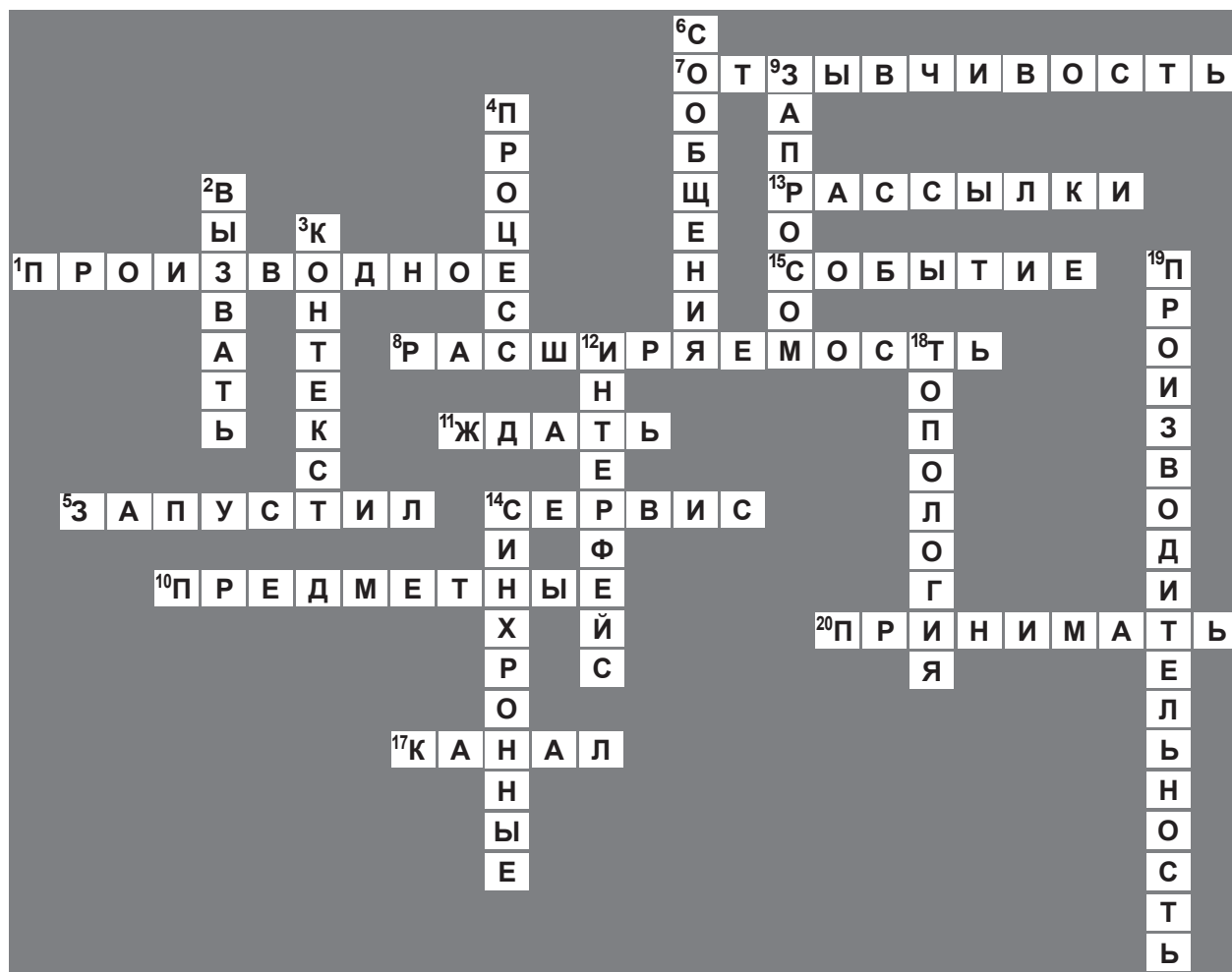
Почему? Здесь пригодится возможность асинхронной рассылки, присущая EDA.

- ☒ Хорошо подходит
☐ Может использоваться
☐ Не подходит



Кроссворда о событийной архитектуре. Решение

На с. 422



Проверка знаний



Готовы проверить свои навыки создания распределенной архитектуры? В этой главе вам предстоит побывать в роли архитектора программного обеспечения. Вы будете определять архитектурные характеристики, принимать решения, создавать логическую архитектуру и выбирать, какую архитектуру использовать: микросервисную или событийно-управляемую. Упражнения в этой главе помогут вам лучше понять, что делает архитектор ПО, и покажут, как много вы узнали. Вам предстоит создать архитектуру для системы тестирования учащихся Make the Grade. Удачи! Мы надеемся, что вы получите пятерку!

Добро пожаловать в Make the Grade

Поздравляем! Вас только что наняли в управление образования Датавилла для создания новой системы тестирования учащихся. Все учащиеся в каждом классе сдают один и тот же тест, по результатам которого определяется успеваемость ученика, эффективность учителя и школы.

Список требований к Make the Grade

- ☐ Учащиеся сдают веб-тест в классе под наблюдением учителя. Время тестирования ограничено (2 часа), поэтому система должна выводить вопросы как можно быстрее.
- ☐ На экран выводится вопрос и несколько вариантов ответов. Получив ответ, система фиксирует его и выводит следующий вопрос. Учащиеся могут пропускать вопросы, но не могут возвращаться к предыдущим, — разрешено двигаться только вперед.
- ☐ Каждый ответ автоматически оценивается как правильный или неправильный, и результат сохраняется в центральной базе данных, допускающей не более 300 подключений.
- ☐ Одновременно тест могут сдавать от 20 до 200 000 учащихся.
- ☐ Администратор системы отвечает за составление расписания тестирования, а также за хранение тестов, ключей ответов и списков учащихся (используются для организации входа учащихся в систему).
- ☐ Рита, глава управления образования Датавилла, использует результаты тестирования для составления отчетов об успеваемости учеников, эффективности учителей и школ.
- ☐ Преподаватели используют систему, чтобы узнать дату и время тестирования.

Рита — глава управления образования Датавилла.



Будьте внимательны, потому что это очень важно.

У Риты есть и другие важные требования к системе.

«**Очень важно**, чтобы ответы не терялись ни в коем случае, даже если система выйдет из строя».

«Эта система нужна нам через шесть месяцев, к **началу следующего семестра**».

«Вы должны сделать все, чтобы учащиеся **не смогли** взломать систему и украсть ключи с ответами на тесты».

«Тестирование проводится не каждый день. В одни дни тест сдают всего 20 учеников, в другие — до 200 000 учеников одновременно. Иногда тестирование проводится в течение дня».

Процесс тестирования учащихся

Теперь разберемся с требованиями и рассмотрим процесс работы с системой Make the Grade.



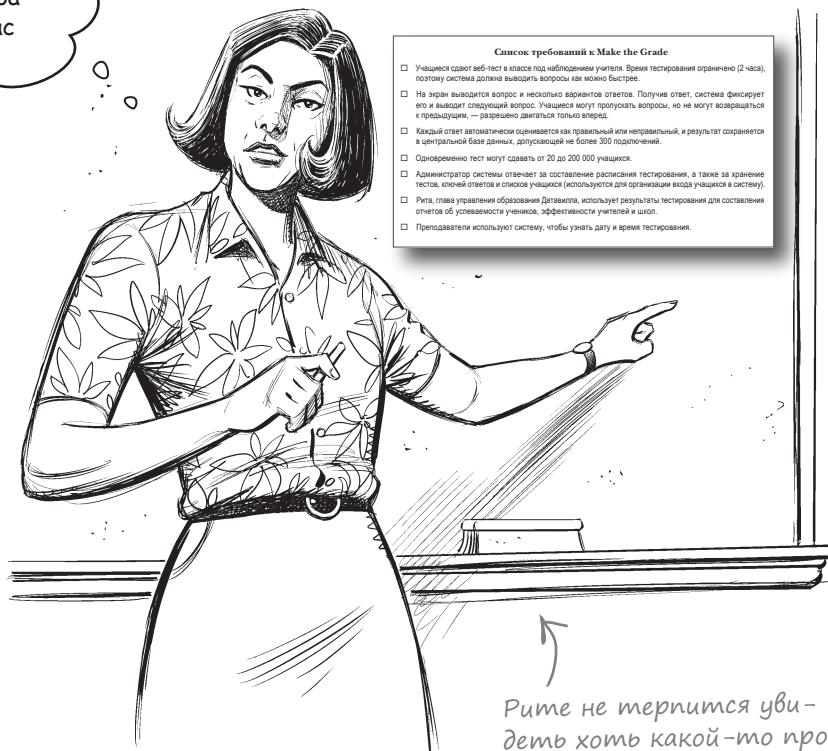
 Возьмите в руку карандаш

Учитывая требования к Make the Grade, перечислите проблемы, которые вам придется решить при создании архитектуры.

Решение на с. 452

Планирование архитектуры

Я передала вам требования, так что же вы медлите? Почему не разрабатываете систему? У нас всего шесть месяцев!



Список требований к Make the Grade

- ☐ Учащиеся сдают веб-тест в классе под наблюдением учителя. Время тестирования ограничено (2 часа), поэтому система должна выводить вопросы как можно быстрее.
- ☐ На экран выводится вопрос и несколько вариантов ответов. Получив ответ, система фиксирует его и выводит следующий вопрос. Учащиеся могут пропускать вопросы, но не могут возвращаться к предыдущим — разрешено двигаться только вперед.
- ☐ Каждый ответ автоматически оценивается как правильный или неправильный, и результат сохраняется в центральной базе данных, допускающей не более 300 подключений.
- ☐ Одновременно тест могут сдавать от 20 до 200 000 учащихся.
- ☐ Администратор системы отвечает за составление расписания тестирования, а также за хранение тестов, ключей ответов и списков учащихся (используются для организации входа учащихся в систему).
- ☐ Рита, глава управления образования Делавэра, использует результаты тестирования для составления отчетов об успеваемости учащихся, эффективности учителей и школ.
- ☐ Преподаватели используют систему, чтобы узнать дату и время тестирования.

Рите не терпится увидеть хоть какой-то прогресс, но не позволяйте ей помешать вам создать надежную и хорошо продуманную архитектуру.

Сначала нужно создать архитектуру.

Как вы уже знаете, архитектура — важная часть любой программной системы. Без нее система едва ли сможет достичь своих целей.

Прежде чем приступить к разработке кода, нужно создать архитектуру. А для этого мы должны вернуться к главе 1, в которой узнали о четырех измерениях архитектуры программного обеспечения.

Не волнуйтесь, мы создадим систему. Но сначала нужно понять, что мы создаем.

План действий архитектора

Создадим архитектуру Make the Grade, выполнив шаги, описанные в предыдущих главах, и преобразовав требования в архитектуру.



Данная схема послужит вам планом решения задачи, поэтому привыкайте к ней. Вы пройдёте каждый из указанных в ней шагов, выполняя задания на следующих страницах.

Желаем успехов! Управление образования Датавилла рассчитывает на вас.

Шаг 1: определение архитектурных характеристик

На этом первом шаге, основываясь на приведенных ниже требованиях, вы определите архитектурные характеристики, важные для системы тестирования Make the Grade. На следующей странице перечислите *до семи* основных характеристик. Затем выберите три *наиболее важные*, по вашему мнению.

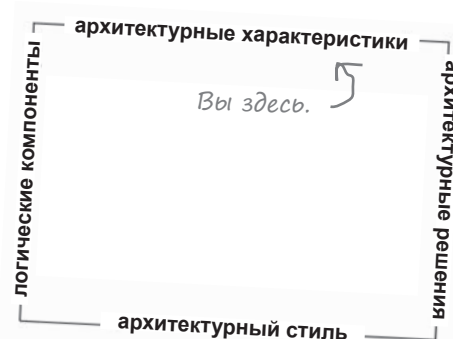
Не забывайте о *неявных характеристиках*, которые подразумеваются практически в каждой архитектуре ПО (например, *защищенность*). Увидев *важную*, на ваш взгляд, характеристику, перенесите ее в список движущих.

«**Очень важно**, чтобы ответы не терялись ни в коем случае, даже если система выйдет из строя».

«Эта система нужна нам через шесть месяцев, к **началу следующего семестра**».

«Вы должны сделать все, чтобы учащиеся **не смогли** взломать систему и украсть ключи с ответами на тесты».

«Тестирование проводится не каждый день. В одни дни тест сдают всего 20 учеников, в другие — до 200 000 учеников одновременно. Иногда тестирование проводится в течение дня».



Мы скопировали требования, чтобы вам было проще определить основные архитектурные характеристики.

Список требований к Make the Grade

- ☐ Учащиеся сдают веб-тест в классе под наблюдением учителя. Время тестирования ограничено (2 часа), поэтому система должна выводить вопросы как можно быстрее.
- ☐ На экран выводится вопрос и несколько вариантов ответов. Получив ответ, система фиксирует его и выводит следующий вопрос. Учащиеся могут пропускать вопросы, но не могут возвращаться к предыдущим, — разрешено двигаться только вперед.
- ☐ Каждый ответ автоматически оценивается как правильный или неправильный и результат сохраняется в центральной базе данных, допускающей не более 300 подключений.
- ☐ Одновременно тест могут сдавать от 20 до 200 000 учащихся.
- ☐ Администратор системы отвечает за составление расписания тестирования, а также за хранение тестов, ключей ответов и списков учащихся (используются для организации входа учащихся в систему).
- ☐ Рита, глава управления образования Датавилла, использует результаты тестирования для составления отчетов об успеваемости учеников, эффективности учителей и школ.
- ☐ Преподаватели используют систему, чтобы узнать дату и время тестирования.



Упражнение

В главе 2 мы показали этот прием ограничения количества архитектурных характеристик. Вернитесь на с. 102, чтобы вспомнить, как его использовать.

Топ-3

Движущие характеристики

Неявные характеристики

☐	_____
☐	_____
☐	_____
☐	_____
☐	_____
☐	_____
☐	_____

_____ *осуществимость (цена/время)*

_____ *защищенность*

_____ *удобство сопровождения*

_____ *наблюдаемость*

↖ Это неявные характеристики.
Перенесите их в колонку «Движущие характеристики», если считаете их важными.

Вернитесь к главе 2, чтобы освежить в памяти определения этих общих архитектурных характеристик.

↖ Отметьте три самые важные (в любом порядке).

Возможные кандидаты в архитектурные характеристики

производительность	целостность данных	развертываемость
отзывчивость	согласованность данных	тестируемость
доступность	адаптируемость	конфигурируемость
отказоустойчивость	расширяемость	настраиваемость
масштабируемость	интегрируемость	восстанавливаемость
эластичность	параллелизм	контролируемость

→ Решение на с. 453

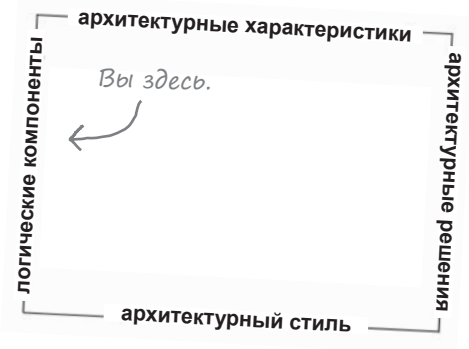
Шаг 2: определение логических компонентов

Отлично! Теперь, определив важные архитектурные характеристики Make the Grade, вспомните, о чем рассказывалось в главе 4, и определите логические компоненты.

Опираясь на требования и рабочий процесс, описанные выше, определите *пользователей и их действия*. А потом, на следующей странице, определите как можно больше *логических компонентов*.

Помочь вам выполнить это упражнение может следующая информация.

- Учащиеся входят в систему, используя свой ID. После входа система проверит дату, ID, тест и преподавателя.
- Рита, глава управления образования Датавилла, будет ждать по меньшей мере один день после окончания тестирования, прежде чем составлять отчеты.
- Классный руководитель выступает в роли ответственного во время тестирования. Он следит, чтобы учащиеся не списывали, и оказывает помощь. Учителя используют систему, чтобы узнать, когда запланирован тест для их класса.
- При создании новых тестов вопросы и ответы отправляются администратору, а тот вводит их в систему. То же касается любых изменений в существующих тестах.



Это дает системе время для записи всех ответов в базу данных.

Возьмите В руку карандаш

Используя подход «субъекты/действия», определите действия для каждого из перечисленных субъектов.

Учащийся

Система

Администратор

Учитель

Глава управления
образования
Датавилла

Входит

Пример
от нас.

Решение на с. 452



Упражнение

Нарисуйте логические компоненты и связи между ними.

Элементы логической архитектуры



Субъект



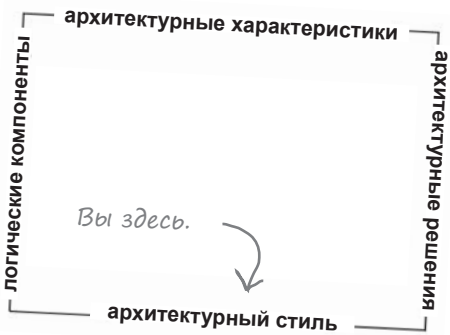
Используйте эти обозначения при создании логической архитектуры.



→ Решение на с. 454

Шаг 3: Выбор архитектурного стиля

Мы знаем, что система будет состоять из отдельных частей, требующих разных архитектурных характеристик, поэтому имеет смысл использовать **распределенную архитектуру**, например микросервисную или событийную. Используя знания об этих двух стилях, проанализируйте их плюсы и минусы применительно к системе Make the Grade. Кроме того, вам нужно будет вспомнить требования, логические архитектуры и звездные рейтинги этих стилей (мы добавили их ниже). По результатам анализа выберите наиболее подходящий архитектурный стиль.



Чем больше звезд, тем лучше под-держивается характери-стика.

Оба стиля сложны. Добро пожаловать в распределен-ные архитек-туры.

Микросервисы	
Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★★★★★
Тестируемость	★★★★★
Развертываемость	★★★★★
Простота реализации	★
Развиваемость	★★★★★
Производительность	★★
Масштабируемость	★★★★★
Эластичность	★★★★★
Отказоустойчивость	★★★★★
Общая стоимость	\$\$\$\$\$

Событийно-управляемая архитектура

Архитектурная характеристика	Рейтинг в звездах
Удобство сопровождения	★★★★★
Тестируемость	★★
Развертываемость	★★★
Простота реализации	★
Развиваемость	★★★★★
Производительность	★★★★★
Масштабируемость	★★★★★
Эластичность	★★★★★
Отказоустойчивость	★★★★★
Общая стоимость	\$\$\$

Эти три ха-рактеристики имеют одина-ковые оценки в обоих стилях.

Вот некоторые соображения, которые помогут вам выбрать архитектурный стиль для Make the Grade.

- Вернитесь к схеме логической архитектуры и посчитайте действия, которые вы определили как **события**. Если событий не так много, то, возможно, событийно-управляемая архитектура — не лучший выбор.
- Подумайте о характере данных. Если они по большей части общие, то выбор микросервисов, скорее всего, будет неправильным.
- Посчитайте **синхронные** и **асинхронные** действия. Если получится много синхронных действий, то событийно-управляемая архитектура может не подойти.



Упражнение

Опишите плюсы и минусы каждого архитектурного стиля. Это поможет вам сделать выбор, наиболее подходящий для Make the Grade.

Микросервисная архитектура

Плюсы

Минусы

Событийно-управляемая архитектура

Плюсы

Минусы

Укажите ваш выбор:

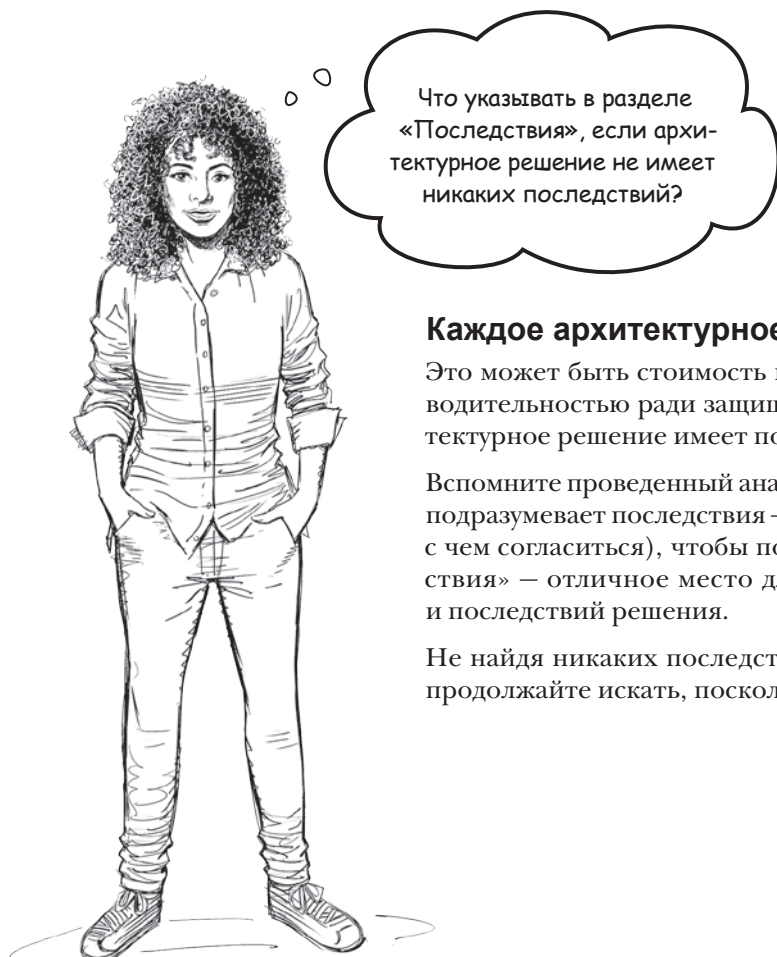
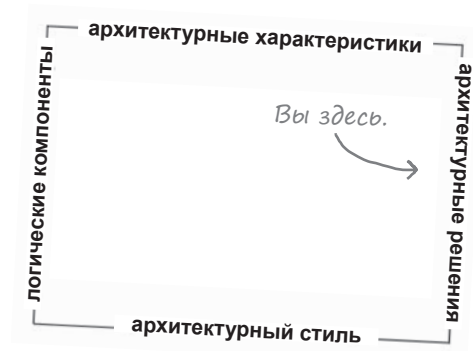
→ Решение на с. 455

Шаг 4: документирование решения

Поздравляем с выбором архитектурного стиля для Make the Grade. Теперь попробуйте *объяснить* этот выбор и задокументируйте свое архитектурное решение.

Как вы узнали в главе 3, *записи об архитектурных решениях* (architectural decision record, ADR) — эффективный способ документирования. Запишите ADR на следующей странице, предположив, что это ваше 11-е архитектурное решение.

↖ Вернитесь к главе 3, чтобы вспомнить, что такое ADR.



Каждое архитектурное решение имеет последствия.

Это может быть стоимость или необходимость жертвовать производительностью ради защищенности. В любом случае *каждое* архитектурное решение имеет последствия.

Вспомните проведенный анализ компромиссов. Любой компромисс подразумевает последствия — то, от чего вы готовы отказаться (или с чем согласиться), чтобы получить нечто лучшее. Раздел «Последствия» — отличное место для документирования компромиссов и последствий решения.

Не найдя никаких последствий в своем архитектурном решении, продолжайте искать, поскольку они есть всегда.



Упражнение

Запись об архитектурном решении

Заголовок:

Статус: Предложено

← Мы начали
за вас.

Контекст:

Решение:

Последствия:

← Какие последствия несет ваше решение?
На какие компромиссы вы готовы пойти?

→ Решение на с. 456

Шаг 5: составление схемы архитектуры

Пришло время объединить все четыре измерения и показать свое видение архитектуры Make the Grade. В этом последнем упражнении вы должны нарисовать схему архитектуры, используя обозначения, перечисленные ниже.

Мы намеренно не даем много места для схемы. Архитекторы часто рисуют очень подробные архитектурные схемы, однако мы просим вас набросать лишь *краткое* физическое представление о пользовательских интерфейсах, базах данных, компонентах и о связях между ними (синхронных и асинхронных).



Обозначения для схемы физической архитектуры

Пользовательский интерфейс



Для представления *пользовательского интерфейса* нарисуйте экран компьютера и укажите категории *пользователей*, взаимодействующих с ним. Например, если у вас есть отдельные интерфейсы для администратора и управления образованием (для создания отчетов), то нарисуйте два экрана. Если интерфейс общий, то нарисуйте один экран, с которым взаимодействуют несколько субъектов.



Сервисы представьте в виде коробок. Обязательно укажите *компоненты*, которые они реализуют. Компоненты должны соответствовать логическим компонентам, выявленным ранее. Кроме того, укажите, какие пользовательские интерфейсы и сервисы взаимодействуют с этим сервисом. Наконец, дайте сервису *осмысленное и описательное название*.

База данных



Для представления каждой *физической базы данных* в вашем решении нарисуйте цилиндр. В подписи укажите, какие типы данных в них хранятся (например, «Ответы студентов»). Укажите, какие сервисы записывают в базу данных, а какие только читают из нее. Для этого *нарисуйте стрелки*, указывающие направления потоков данных. (Запись предполагает чтение.)



Пунктирными линиями обозначьте *асинхронные* (например, с помощью очереди, темы или потока), а сплошными — *синхронные* (блокирующие) взаимодействия между сервисами и пользовательским интерфейсом.



Событие или сообщение

Если в архитектуре используются сообщения или события, то нарисуйте рамку или конверт и укажите, какие данные передаются (например, «Ответ студента») или какое событие запускается (например, «Ответ отправлен»).

Не скупитесь на подписи к своей схеме, чтобы что-то прояснить или описать.





Упражнение

Нарисуйте схему физической архитектуры Make the Grade, используя обозначения с предыдущей страницы.

→ Решение на с. 457–458

Нет правильных (или неправильных) ответов!

Поздравляем! Вы только что создали архитектуру!

Далее мы покажем вам «решения» упражнений. Мы использовали кавычки, поскольку представленные решения — лишь одни из множества возможных. Ведь в архитектуре ПО нет правильных или неправильных ответов: все дело в анализе компромиссов и умении обосновать свои решения.

Сравните свои ответы с нашими. Возможно, вы заметите, что сделали что-то не так, как мы, и наш вариант лучше, или, наоборот, утвердитесь в правильности своего выбора. Мы покажем вам наши варианты микросервисной и событийной архитектуры для Make the Grade, поскольку оба они являются жизнеспособными.

Разработка архитектуры ПО — это *всегда* непрерывный процесс обучения. Каждая новая задача подразумевает свой набор условий, ограничений и проблем. Не существует универсальной архитектуры, все зависит от вас, архитектора, — именно вы должны придумать архитектуру, наиболее подходящую для вашей ситуации.

Ключевые Моменты

- Анализируя бизнес-требования, всегда собирайте дополнительную информацию, которую могут дать заинтересованные стороны или спонсор проекта.
- Не существует «контрольного списка» для создания архитектуры, но есть четыре измерения архитектуры ПО (представленные в главе 1), которые послужат хорошим ориентиром.
- Определение движущих архитектурных характеристик требует анализа бизнес-требований и технических ограничений.
- Неявные архитектурные характеристики становятся *движущими*, если важны для успешной работы системы.
- Каждая движущая характеристика должна быть связана с каким-то требованием или потребностью.
- Определяя логические компоненты и создавая логическую архитектуру, старайтесь не добавлять физические детали, такие как сервисы, базы данных, очереди и пользовательские интерфейсы, — они относятся к *физической архитектуре*.
- При выборе архитектурного стиля учитывайте его свойства, особенности предметной области и выявленные вами основные архитектурные характеристики.
- *Гибридные архитектуры* (сочетающие два или более архитектурных стиля) встречаются очень часто. Если вы используете одну из них, то убедитесь, что она соответствует вашим архитектурным характеристикам.
- *Записи об архитектурных решениях* (ADR) — отличный вариант, чтобы задокументировать выбор. В них указываются причины решений и результаты анализа компромиссов.
- Составляя схему физической архитектуры, не забудьте добавить в нее все компоненты, которые вы определили в логической архитектуре.
- Помните, что в архитектуре программного обеспечения нет правильных или неправильных ответов. Если вы можете обосновать свои архитектурные решения, то находитесь на правильном пути.

Вы прочли всю книгу?
Решили все упражнения?
Тогда жмем вам руку!



Вы на пути к архитектурному мышлению!

Мы верим, что вы действительно прочли эту книгу до конца, а не просто пролистали до этого места, и поздравляем вас! Вы хорошо потрудились.

Поздравляем!

Вы дошли до конца.

Впрочем, еще есть приложение.

И сайт...

Так легко вам от нас не отделаться!

(Ладно, признаем: вы просто хотите еще больше узнать об архитектуре ПО, верно?)

Возьмите в руку карандаш

Решение

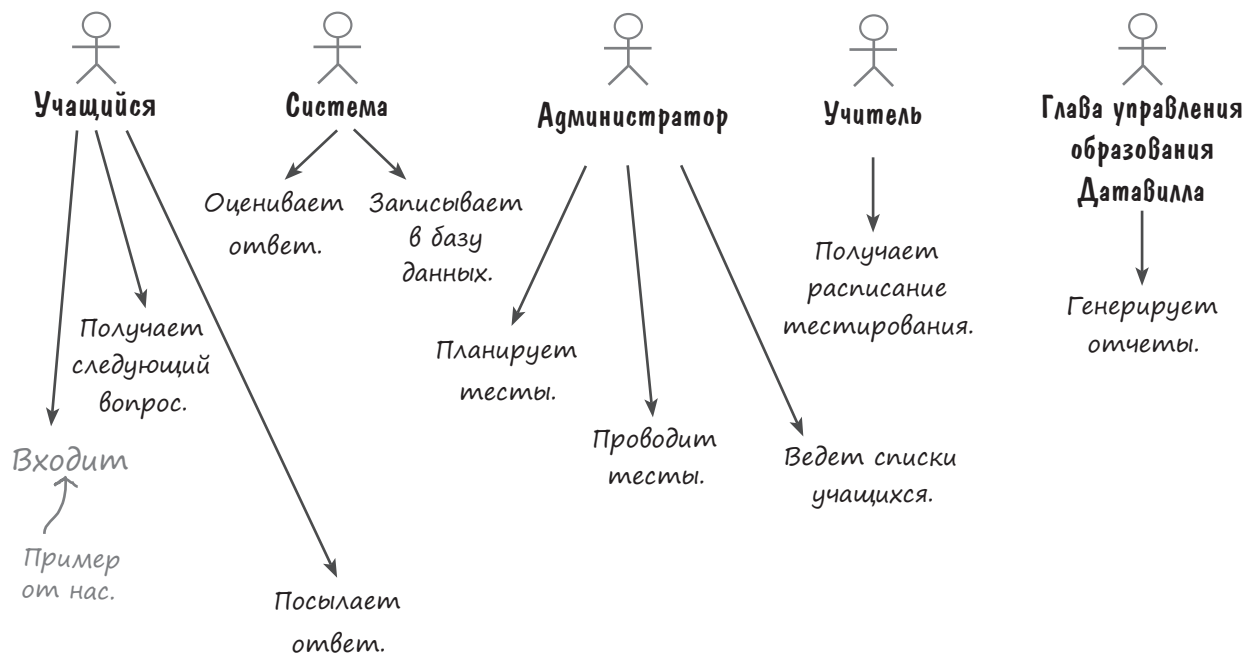
Учитывая требования к Make the Grade, перечислите проблемы, которые вам придется решить при создании архитектуры.

- Сохранение до 200 000 ответов студентов одновременно в базе данных, имея всего 300 соединений.
- Быстрый переход к следующему вопросу и гарантии сохранности ответов учеников.
- Разработка жизнеспособного решения, которое можно реализовать за шесть месяцев.
- Обеспечение эластичности системы, которая экономит ресурсы, когда тестирование не проводится или тест сдают всего несколько учащихся.

Возьмите в руку карандаш

Решение

Используя подход «субъекты/действия», определите действия для каждого из перечисленных субъектов.





Упражнение Решение

На с. 441

В главе 2 мы показали этот прием ограничения количества архитектурных характеристик. Вернитесь на с. 102, чтобы вспомнить, как его использовать.

Топ-3 Движущие характеристики Неявные характеристики

☐	Осуществимость	← У нас всего полгода!	← <u>осуществимость (цена/время)</u>
☐	Защищенность	← Нужно защитить доступ к ответам.	← <u>защищенность</u>
☒	Эластичность	← Может быть от 20 до 200 000 учеников — самое точное определение эластичности.	← <u>удобство сопровождения</u>
☒	Отзывчивость	← Мы должны быстро задать следующий вопрос.	← <u>наблюдаемость</u>
☐	Доступность	← Если система недоступна, то учащиеся не смогут пройти тест.	
☒	Целостность данных	← Нам никак нельзя терять ответы учащихся.	
☐	Восстанавливаемость	← Мы не хотим, чтобы в случае сбоя учащимся пришлось начинать тест заново.	

Возможные кандидаты в архитектурные характеристики

производительность	целостность данных	развертываемость
отзывчивость	согласованность данных	тестируемость
доступность	адаптируемость	конфигурируемость
отказоустойчивость	расширяемость	настраиваемость
масштабируемость	интегрируемость	восстанавливаемость
эластичность	параллелизм	контролируемость

Ha c. 443

Элементы логической архитектуры





Упражнение Решение

На с. 445

Опишите плюсы и минусы каждого архитектурного стиля. Это поможет вам сделать выбор, наиболее подходящий для Make the Grade.

Микросервисная архитектура

Плюсы

Микросервисы обеспечивают важные для Make the Grade эластичность, масштабируемость и отказоустойчивость.

Разделение данных между микросервисами обеспечивает хорошую отказоустойчивость и контроль доступа к ним.

В системе Make the Grade много независимых частей: тестирование, оценивание, сопровождение и отчетность — поэтому хорошо подойдут отдельно развертываемые микросервисы без особых взаимодействий.

Минусы

Минус микросервисов — низкая производительность, но эту проблему можно решить, добавив кэширование и уменьшив количество взаимодействий.

Функции администрирования записывают данные в базу, откуда их читают другие сервисы (вопросы, информация об учащихся, ответы и т. д.). Это означает необходимость обмена данными, для чего микросервисы плохо подходят. Однако данные довольно статичны и их можно использовать совместно, задействуя кэширование.

Событийно-управляемая архитектура

Плюсы

Событийно-управляемая архитектура (EDA) обладает высокой отзывчивостью, эластичностью, масштабируемостью и отказоустойчивостью.

Отправку ответа можно считать событием. В ответ можно предложить следующий вопрос и автоматически оценить ответ. Однако это довольно изолированное событие — фактически единственное в системе.

Минусы

Функции администрирования (информации об учащихся, тестов, отчетов и графиков тестирования) не очень подходят для EDA.

В этой системе мало событий — в основном в нее поступают запросы. Единственное событие, которое мы выявили, — отправка ответа, но у него есть только один слушатель (функция автоматического оценивания).

Это не единственный ответ, а лишь наш выбор, основанный на анализе.

Укажите ваш выбор:

На основе анализа мы выбрали для Make the Grade микросервисы.

Упражнение
Решение

На с. 447

Запись об архитектурном решении

Заголовок: O11: Выбор архитектурного стиля микросервисов для системы Make the Grade

Статус: Предложено

Контекст:

Make the Grade — система тестирования, для которой важны целостность данных, отзывчивость, отказоустойчивость и эластичность. Разные части системы (оценивание, администрирование, отчетность и тестирование) имеют разные требования, поэтому решено использовать распределенную архитектуру. Рассматривались два варианта: микросервисы и событийно-управляемая архитектура.

Решение:

Мы выбрали микросервисную архитектуру.

Микросервисы дают необходимые отказоустойчивость, эластичность и масштабируемость.

Недостаточные производительность и отзывчивость можно компенсировать, уменьшив количество взаимодействий сервисов (кэшированием информации об учащих, вопросов и ответов), а также используя асинхронное оценивание и сохранение ответов.

Целостность данных обеспечивается за счет использования хранимых очередей между компонентами «Прием ответа» и «Оценка ответа», а также режима подтверждения в компоненте «Оценка ответа». Это позволит гарантировать сохранность каждого ответа в очереди до его записи в базу данных.

Функции администрирования тестов будет представлять единый микросервис, объединяющий функции управления информацией о тестах и учащих. Отчеты тоже будет генерировать один микросервис.

Последствия:

Команды, организованные по техническим задачам, придется реорганизовать в кросс-функциональные, которые должны работать параллельно, чтобы закончить систему к сроку.

Для решения проблемы производительности, эластичности и совместного использования данных понадобится механизм кэширования в памяти.

Для поддержки микросервисов понадобится дополнительная инфраструктура: в частности, оркестратор сервисов, такой как Kubernetes, и более эффективный конвейер CI/CD.

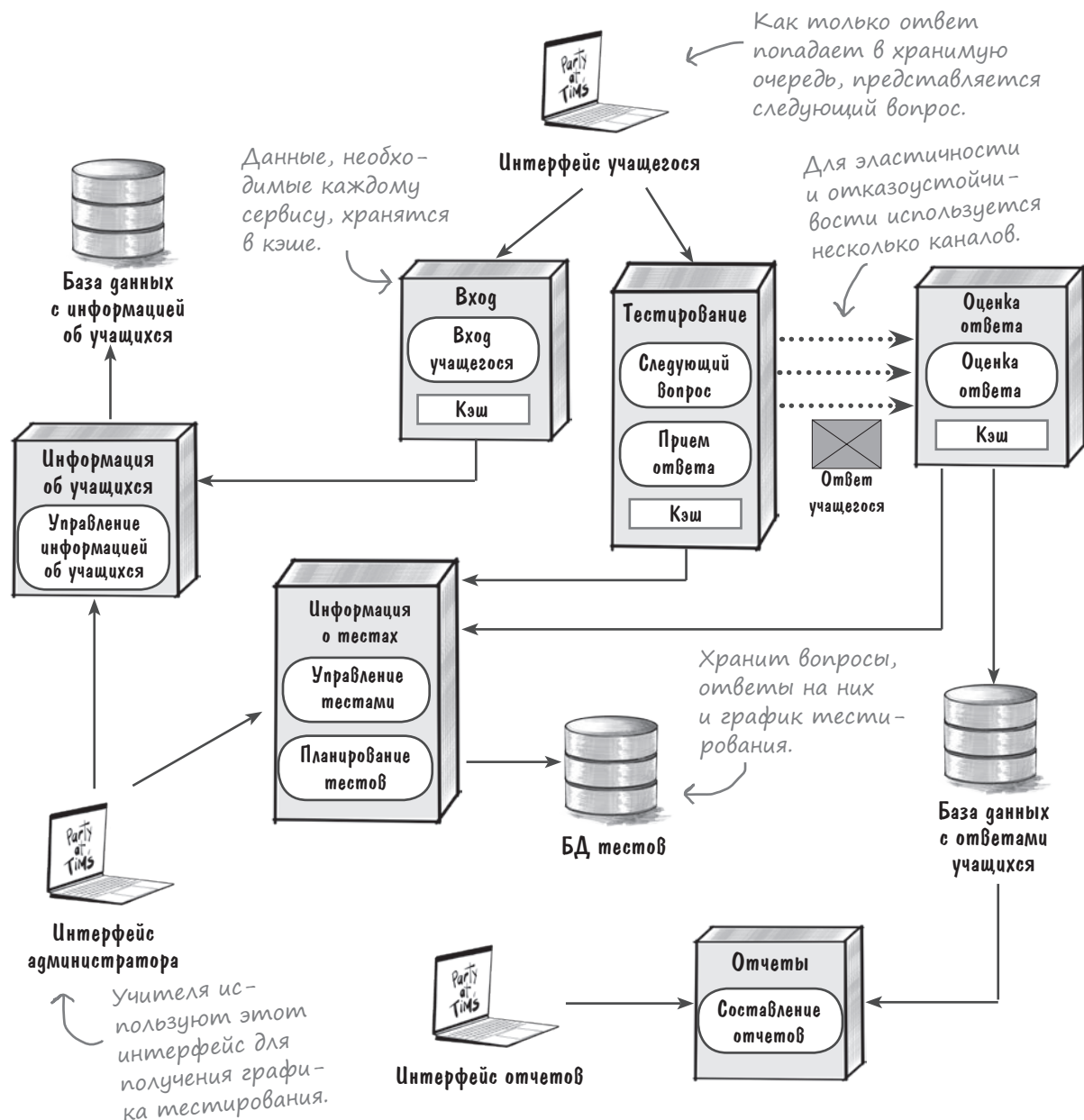


Упражнение Решение

На с. 449

Нарисуйте схему физической архитектуры Make the Grade, используя обозначения со с. 448.

Схема архитектуры микросервисов



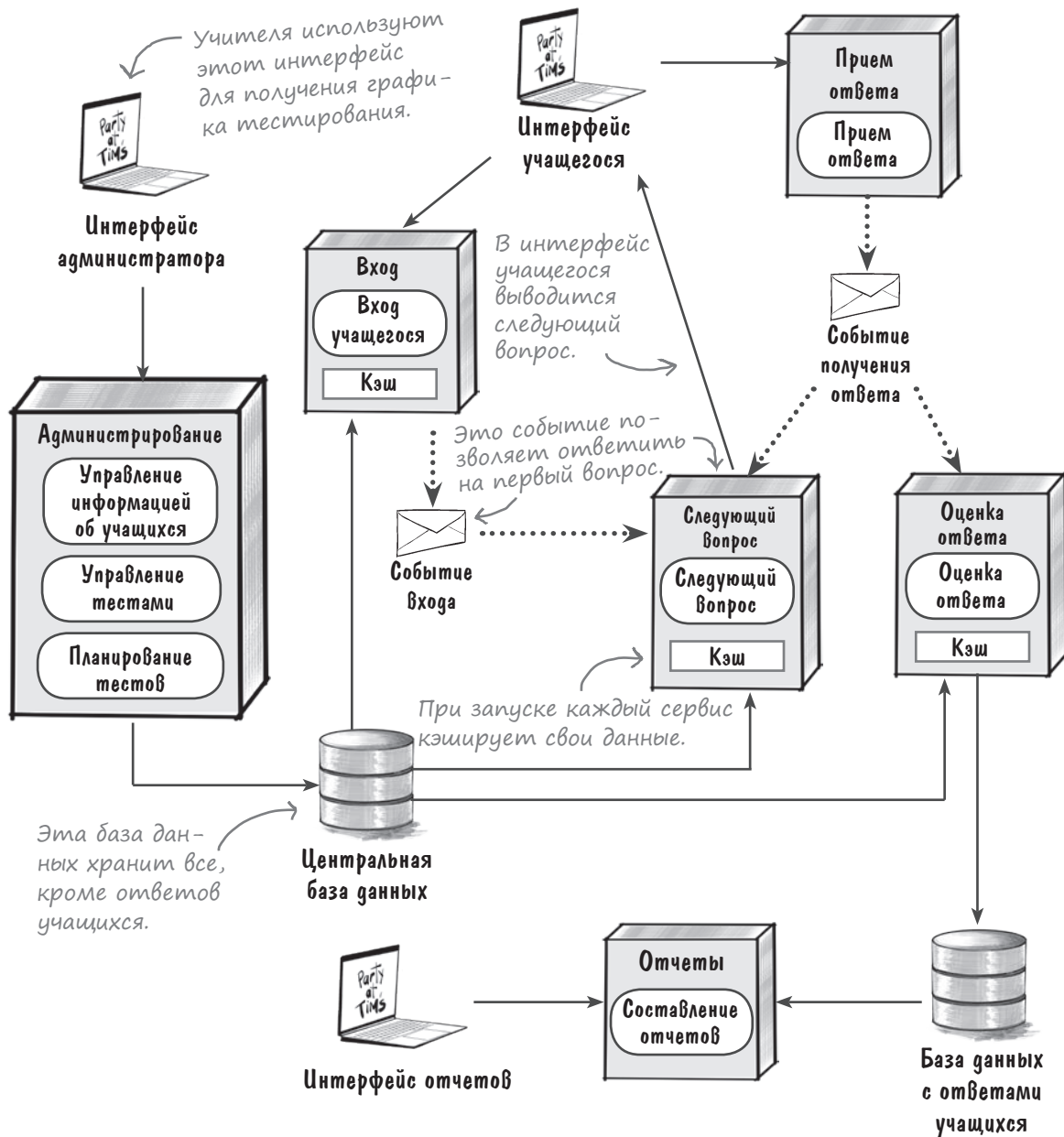


Упражнение Решение

На с. 449

Нарисуйте схему физической архитектуры Make the Grade, используя обозначения со с. 448.

Схема событийно-управляемой архитектуры



Приложение: Что осталось за рамками обсуждения

Шесть главных незатронутых тем



Мы рассказали об архитектуре программного обеспечения далеко не всё. Прочитав эту книгу, вы сделали всего лишь первый шаг к обретению архитектурного мышления, и мы не можем отпустить вас, не обеспечив дополнительными ресурсами. Поэтому в данном приложении мы кратко опишем еще несколько интересных тем, заслуживающих такого же внимания, как и те, которые были представлены в главах.

1. Программирующий архитектор



Конечно! Мы убеждены, что архитекторы ПО должны продолжать писать код. Так вы не только сможете поддерживать свои навыки, но и увидите, как ваши архитектурные решения воплощаются в жизнь.

Однако не всегда удастся найти баланс между программированием и архитектурой. Как вы уже убедились, архитектура — сложная сфера. Она будет занимать большую часть вашего времени (если не все).

Но не волнуйтесь — мы поделимся с вами некоторыми советами и методиками, позволяющими писать программы и при этом быть эффективным архитектором.



Не становитесь источником проблем

Будьте осторожны, не беритесь за код на критическом этапе развития продукта. Оставьте инфраструктурный код и сложные или критически важные части системы своей команде разработчиков. В таком случае, если вам придется отвлечься на решение архитектурных вопросов, вы не задержите свою команду.



Вот еще несколько способов не стать источником проблем.



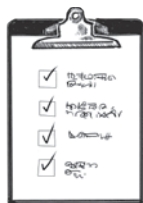
Помогайте ликвидировать технический долг

Практически все команды разработчиков накапливают *технический долг* (изменения, которые откладываются на более поздний срок). Помогите своей команде, ликвидировав часть данного долга. Они это оценят, а если вас отзовут, то это их не задержит.



Участвуйте в ликвидации сбоев

В случае сбоев в работе системы по возможности участвуйте в их ликвидации. Помогите команде разработчиков выявлять первопричины и вносить изменения в код, помогающие восстановить работоспособность системы. Это даст вам возможность увидеть детали реализации вашей архитектуры.



Делайте ревью кода

Никто *не любит* делать ревью кода. Но это хороший способ оставаться вовлеченным в процесс. Он поможет вам убедиться, что исходный код соответствует вашим архитектурным решениям.

А это пара способов постоянно участвовать в процессе.

2. Обязанности архитектора

В этой книге мы много говорили об архитектуре, но мало о *роли* архитектора. В зависимости от компании обязанности этой роли могут быть разными, но есть и такие, которые должен выполнять любой архитектор, независимо от должности.



Принимать архитектурные решения

Архитектор должен определять архитектурные решения и принципы проектирования и использовать их для принятия технологических решений в рамках команды, отдела и/или всего предприятия.

Следить за последними тенденциями

Чтобы оставаться востребованными (и сохранить работу!), разработчики, а тем более архитекторы, должны постоянно следить за техническими и отраслевыми тенденциями. Это поможет вам подготовиться к будущим переменам и принимать правильные решения.

Постоянно анализировать архитектуру

Архитектор должен постоянно анализировать текущую архитектуру и технологическую среду и давать рекомендации по решениям и улучшениям. Постоянный анализ — это способ проверки *жизнеспособности архитектуры* с учетом постоянных изменений в бизнесе и технологиях.

Обеспечивать соответствие архитектуре

Архитекторы должны постоянно проверять, следуют ли команды разработчиков утвержденным архитектурным решениям и принципам проектирования. Это называется *архитектурным управлением*. Какой бы хорошей ни была архитектура, система не будет работать, если разработчики будут отступать от ваших архитектурных решений.

Приобретать и сохранять разнообразные знания и опыт

Архитектор должен быть знаком с разными технологиями, фреймворками, платформами и средами. Он не обязан быть экспертом во всех сферах, но должен иметь представление о существующих возможностях.

Для архитектора важно
знать понемногу о многом.

Подробнее об этом чуть ниже!

Обладать исключительными навыками межличностного общения

Архитектор должен обладать незаурядными лидерскими качествами и навыками общения. Технологи, разработчики и архитекторы часто предпочитают решать технические проблемы, а не проблемы людей. Но от архитектора ожидается, что он будет осуществлять техническое руководство и вести людей за собой. Лидерские качества — залог успеха эффективного архитектора.

Американский ученый-информатик Джеральд Вайнбург однажды сказал: «Неважно, как это выглядит на первый взгляд, но проблема всегда в людях» (<https://oreil.ly/wyDB8>).

Знать предметную область бизнеса

Эффективные архитекторы хорошо знают предметную область бизнеса. Не имея таких знаний, трудно понять задачи, цели и требования настолько хорошо, чтобы спроектировать эффективную архитектуру.

Понимать офисную политику и уметь ориентироваться в ней

Многие будут оспаривать ваши решения. Владельцы продуктов, руководители проектов и представители бизнеса могут посчитать их слишком дорогими или долгими в реализации. Разработчики и архитекторы могут оспаривать ваш подход, считая свой более хорошим. От вас будут требовать аргументированного обоснования предложений. Поэтому, чтобы добиться одобрения, вы должны ориентироваться в офисной политике и владеть базовыми навыками ведения переговоров.

3. Нетехнические навыки архитектора

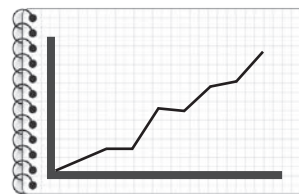
Как отмечалось выше, работа архитектора ПО *как минимум* наполовину состоит в общении с людьми. Навыки общения понадобятся вам, чтобы повести за собой команду разработчиков, завоевать уважение коллег и помочь всем прийти к единому мнению.

Мы называем эти навыки «нетехническими», но обрести их не просто. Для их развития нужны годы практики, проб и ошибок. Вот несколько приемов развития «нетехнических» навыков, которые помогут вам стать эффективным архитектором.



Демонстрируйте, а не обсуждайте

Не спорьте с архитекторами и разработчиками, а просто продемонстрируйте свою идею. Все среды разные, поэтому простой поиск в интернете редко дает правильные ответы. С тем, кто сравнивает варианты в эксплуатационной среде и демонстрирует результаты, спорить практически невозможно.



↑ Повторяйте за нами:
«Демонстрация побед —
дает споры».



Различайте ситуации, когда нужно бороться, а когда лучше отпустить

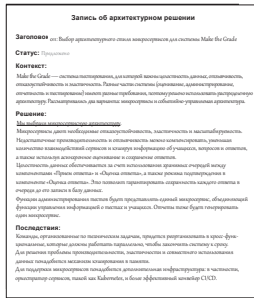
Умение видеть ситуации, когда действительно стоит вступить в бой, — признак хорошего лидера, с которым можно работать. Это умение поможет вам завоевать уважение людей. Бейтесь до конца за то, что имеет решающее значение, но оставьте все как есть, если это не так важно.

← Это умение пригодится
вам и в повседневной жизни.
Обязательно овладейте им.

Сосредоточьтесь на ценностях для бизнеса

Разговаривая с заинтересованными сторонами, описывайте свое решение с точки зрения *ценности для бизнеса*. Бизнесменов не интересует отказоустойчивость или тестируемость — их больше волнует время выхода на рынок, соответствие нормативным требованиям, а также слияния и поглощения. Переведите технические проблемы на язык бизнеса и говорите на их языке.





↑
Это ADR
из главы 12.

Привлекайте разработчиков к принятию архитектурных решений

За годы работы мы усвоили два основных правила.

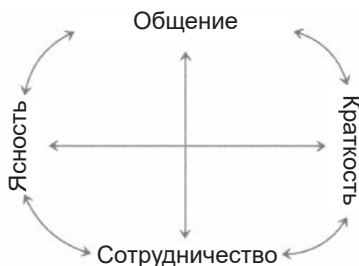
Правило 1. Если разработчики не знают, *почему* принято то или иное решение, то с меньшей вероятностью согласятся с ним.

Правило 2. Если разработчики *не участвуют* в принятии решения, то с меньшей охотой будут его выполнять.

Привлекайте разработчиков к принятию решений. Сотрудничайте с ними, спрашивайте их мнение, привлекайте их к выявлению рисков и используйте статус «Запрос на обсуждение» в ADR. Всегда объясняйте свои решения и убедитесь, что все члены команды понимают вас.

Разделяй и властвуй

В книге «Искусство войны» древнекитайский полководец Сунь Цзы советует: «Если силы врага объединены и сплочены, то разделите их». Эту тактику можно использовать в ситуациях «все или ничего». Так ли важно время отклика 400 миллисекунд или доступность 99,999 % для всех частей системы? Разделив проблему на части, вы сможете определить, чего трудно достичь, и облегчите переговоры.



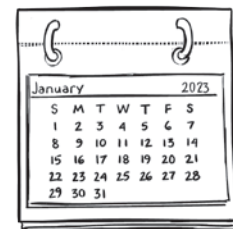
Будьте простыми, ясными и краткими

Ничто так не помогает архитектору завоевать уважение и доверие, как умение кратко и ясно объяснить ситуацию. Это умение будет притягивать людей. Они снова захотят обратиться к вам с просьбой или привлечь вас к работе. Мы называем это умение четырьмя главными навыками архитектора: ясность, краткость, общение и сотрудничество. Хороший архитектор должен владеть всеми ими.

Будьте доступны для своей команды разработчиков

Ничто так не расстраивает разработчиков и заинтересованных лиц, как ваше отсутствие, когда возникает важный вопрос. Не распыляйтесь, *будьте рядом со своей командой*.

Отведите в своем календаре время для работы рано утром или поздно вечером — это отличный способ не сидеть весь день на совещаниях. Используйте это время для сотрудничества с командой и будьте готовы отвечать на их вопросы. Они будут вам благодарны за это, и вы заслуженно будете пользоваться большим уважением.



4. Приемы построения схем

В главе 3 мы представили ADR как лучший способ документирования процесса анализа, ведущий к принятию решения. Другой хороший метод документирования архитектуры — **схемы**. Они иллюстрируют многие важные детали и помогают членам команды увидеть структуру, топологию, связи, зависимости и точки интеграции.

Этой теме можно (и нужно) посвятить целую книгу. Мы же просто дадим несколько советов, которые помогут вам создавать хорошие схемы.

Не усложняйте

Не пытайтесь создавать всеобъемлющие архитектурные схемы, отображающие все нюансы. Иначе ваши схемы станут слишком сложными для понимания.

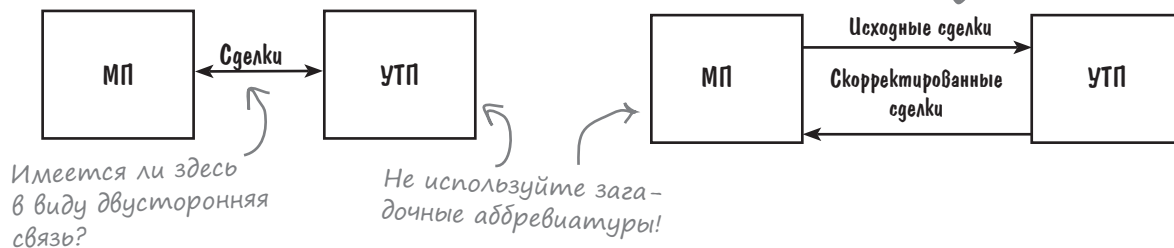
Всегда добавляйте заголовок

Большинство архитектурных схем представляют собой вид или перспективу, и ее название должно явно указывать на это.

Для обозначения связей используйте односторонние стрелки

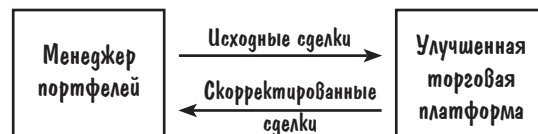
Двусторонние стрелки неоднозначны. Что делает эта стрелка: указывает на двустороннюю связь или просто используется по умолчанию?

Односторонние стрелки устраняют все двусмысленности и делают схему более понятной.

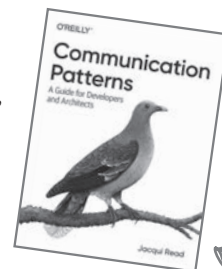


Не используйте аббревиатуры

Аббревиатуры понятны только тем, кто в теме. Пишите внятные названия, чтобы избежать путаницы и не создавать лишней документации.



Используйте говорящие подписи. Разве это не понятнее, чем МП и УТП?



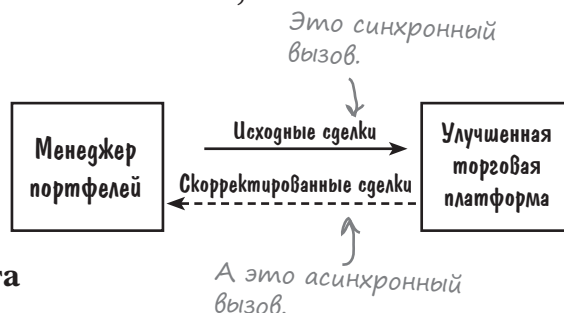
Книга *Communication Patterns* Жаки Риды (Jacqui Read) — отличный ресурс для желающих узнать больше.

Торговая платформа:
Основные информационные потоки

Используйте для схем понятные названия.

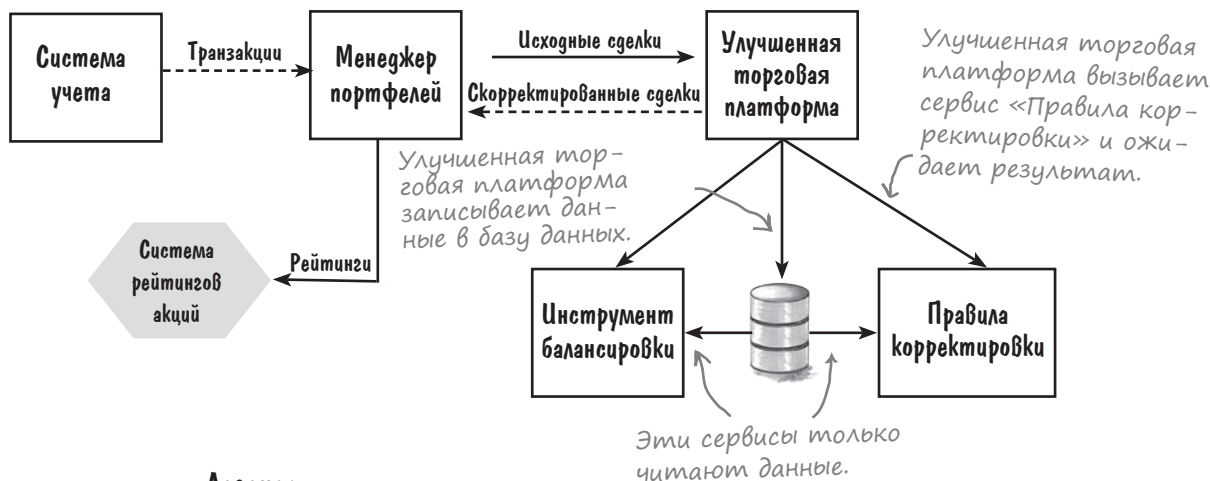
Обозначайте синхронные связи сплошными линиями, а асинхронные — пунктирными

Явно указывайте, является ли обмен данными синхронным (блокирующим) или асинхронным (неблокирующим). Обозначайте сплошными линиями синхронные взаимодействия, а пунктирными — асинхронные.

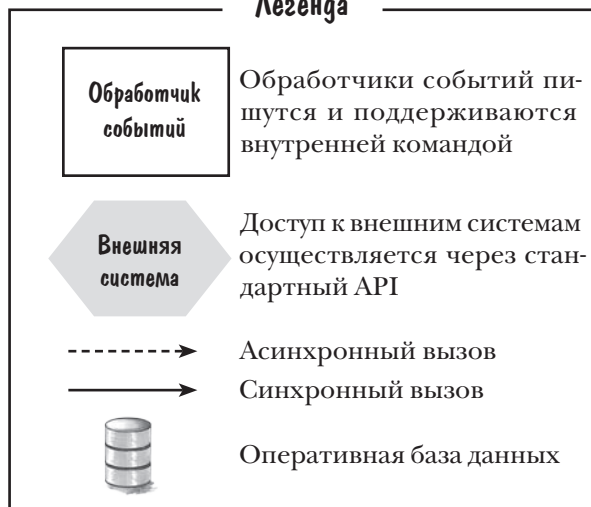


Используйте одинаковые формы и цвета

Не используйте произвольные формы ради экономии места; одинаковые формы и цвета уменьшают визуальный «шум», создаваемый несоответствием.



Легенда



Всегда добавляйте легенду

Единственная, пожалуй, универсальная форма в схемах — цилиндр базы данных. Практически все остальное уникально. Не заставляйте других гадать! Добавьте *легенду*, чтобы сделать схему доступной для более широкой аудитории.

5. Глубина и широта знаний

Когда вы становитесь архитектором ПО, с вашим мозгом происходит невероятная трансформация: *новая роль меняет все, что вы ищете и изучаете*. Взгляните на эту пирамиду, в которой классифицирована вся информация в мире и ваши знания, которыми вы обладали в начале карьеры.

В начале карьеры вы многого не знаете, но это многое вам пригодится.



Знания восторженного младшего разработчика: «Не могу поверить, что мне за это платят!»

Пирамида знаний младшего разработчика

Со временем вы начинаете разбираться в более сложных вещах.



Старший разработчик/технический руководитель

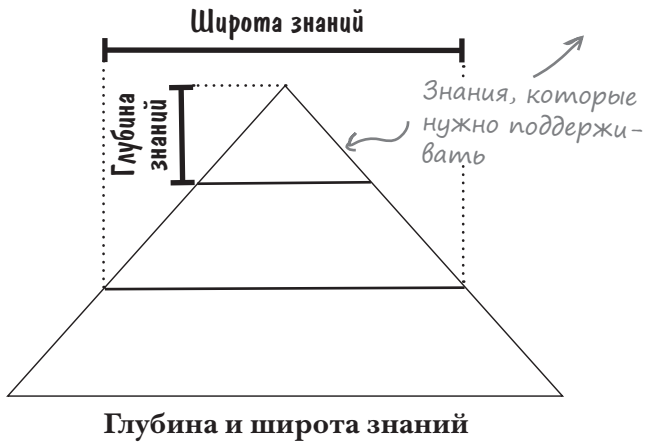
По мере продвижения по карьерной лестнице от младшего разработчика до технического руководителя растет объем ваших знаний. Вы овладеваете новыми языками программирования, фреймворками, инструментами и платформами. Этих знаний уже достаточно, чтобы помогать другим разработчикам.

Со временем, когда вы накопите опыт и станете ведущим специалистом, у вас начинает расти средняя часть пирамиды — «все, о существовании чего вы подозреваете». Чтобы достичь этой точки, сохранив основные навыки (ваши знания), вам потребуется приложить немало усилий.



Младший архитектор

Узнавать о том, что вы чего-то не знаете, довольно тяжело морально. Имейте это в виду.



Глубина знаний охватывает все темы, в которых вы разбираетесь.

Чтобы оставаться экспертом в любой области, необходимо постоянно следить за изменениями. Таким образом, поддержание глубины знаний требует времени.

Широта знаний охватывает области, в которых вы компетентны, и области, о существовании которых вы знаете. Для архитектора знание, что существует пять способов решения задачи, важнее, чем глубокое знание одного способа.

Пирамида знаний начинающего архитектора первое время похожа на пирамиду знаний технического специалиста, но вы должны стремиться расширять свои знания. Например, если у вас есть опыт работы с .NET, попробуйте поработать над проектом на Java или JavaScript с большим количеством пользовательских интерфейсов или сложной организацией данных... Надеемся, вы поняли, о чем речь.



Для архитекторов предпочтительнее широта знаний: осведомленность о вариантах и компромиссах для различных ситуаций и реализаций.



Со временем ваш опыт будет понемногу утрачивать актуальность (что очень болезненно), но это принесет пользу в виде увеличения широты знаний. Помните, что в архитектуре ПО все основано на компромиссах.

6. Архитектурные каты

Мастер боевых искусств и актер Брюс Ли однажды сказал: «Я боюсь не того, кто отработал 10 000 ударов один раз, но того, кто отработал один удар 10 000 раз». Как достичь такого мастерства в архитектуре ПО? Хороший вопрос! Рекомендуем *архитектурные каты*.

Ката — это форма индивидуальных упражнений, используемая во многих боевых искусствах для отработки приемов. Архитектурные каты имитируют процесс проектирования реальной архитектуры.

Архитектурные каты рассчитаны на небольшие группы по три-пять человек. Каждая группа объявляется проектной командой и работает над отдельной катой. (Иногда мы собираем две команды для выполнения каждой каты, чтобы посмотреть на различия.) Модератор следит за временем и помогает выполнить упражнение.

Модератор выдает проектное задание каждой группе. В течение некоторого времени члены команды задают уточняющие вопросы «заказчику» (модератору). Потом обсуждают варианты технологий и набрасывают примерное видение своего решения. А затем представляют решение другим командам и отвечают на вопросы (порой трудные, но справедливые). Выбирать победителя при этом не обязательно.



Ключ к успешному созданию архитектуры программного обеспечения — практика, пусть даже с помощью учебных упражнений.

Мы привлекли ваше внимание? Продолжайте читать, чтобы узнать больше.

Как Выполнять каты

Каты должны быть адаптируемыми упражнениями, поэтому следуйте правилам ниже, *если они приемлемы* для вашей организации, контекста и потребностей. Вопросы, не предусмотренные правилами, относятся к компетенции модератора.

Подготовка

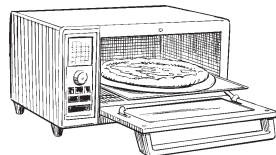
Соберите несколько команд по три-пять человек. (Мы предпочитаем нечетное число, чтобы споры могли решаться большинством голосов.) Желательно, чтобы в одну команду входили люди, не работающие вместе в реальности; в этом упражнении необходимо сотрудничать с другими архитекторами, которых вы еще не знаете.

Выдайте материалы например, бумагу, планшеты или белые доски. Создаваемые артефакты могут быть очень простыми, в зависимости от сложности задания, выделенного времени и ресурсов.

По времени ката может длиться всего 45 минут или несколько недель!



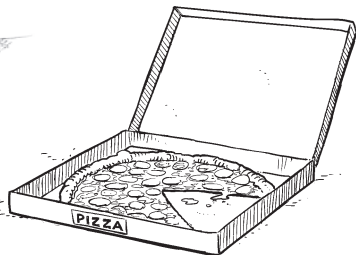
Авторы книги провели несколько кат в реальных компаниях и давали командам по восемь недель на разработку решения.



Обсуждение

Команды в точности следуют процессу, описанному в этой книге: анализируют архитектурные характеристики, определяют логические компоненты, выбирают архитектурный стиль и документируют решения.

Любая технология — это честная игра, но установите разумные ограничения (у вас может не быть неограниченного бюджета или возможности нанимать новых разработчиков). Основное внимание уделяйте архитектуре и анализу компромиссов.



Презентация

Каждая команда представляет свое решение и отвечает на вопросы.

Слушая презентацию другой проектной команды, задавайте вопросы. Но будьте конструктивными. Не заикливайтесь только на плюсах или только на минусах. Старайтесь дать сбалансированную обратную связь.

Я чувствую, что мне не хватает знаний. Есть ли какие-нибудь другие ресурсы, к которым я могу обратиться?

Вы знаете о нашем сайте? Там вы найдете обновления, интересные ссылки, статьи и многое другое!



Не волнуйтесь. Мы не прощаемся.

Поздравляем! Вы дочитали книгу до конца и выполнили все упражнения!

Вы прошли долгий путь, но ваше знакомство с архитектурой ПО только начинается. Мы хотели бы предложить ряд следующих шагов.

Откройте в браузере эту ссылку, чтобы узнать, что будет дальше!

<https://www.headfirstsoftwarearchitecture.com>

А потом познакомьтесь с этими книгами:

Что дальше? Это и многое другое!

Начните с этой книги. Кое-что в ней покажется знакомым, но это достойный преемник нашей книги.



Очень говорящее название. В этой книге вы погрузитесь в действительно сложные аспекты архитектуры ПО.

Раджу Ганди, Марк Ричардс, Нил Форд

Head First. Архитектура ПО

Перевел с английского А. Киселев

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес:
010000, Казахстан, город Астана, район Алматы, проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 04.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 20.02.25. Формат 84х108/16. Бумага офсетная. Усл. п. л. 50,400. Тираж 1000. Заказ 0000.



*Нил Форд, Ребекка Парсонс, Патрик Куа,
Прамод Садаладж*

ЭВОЛЮЦИОННАЯ АРХИТЕКТУРА. АВТОМАТИЗИРОВАННОЕ УПРАВЛЕНИЕ ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ 2-е международное издание

Новые инструменты, фреймворки, методики и парадигмы вновь и вновь меняют экосистему разработки программного обеспечения. Непрерывный прогресс основных практик разработки на протяжении последних пяти лет заставил искать новые пути и подходы к архитектуре, чтобы соответствовать постоянно меняющимся требованиям пользователей. В обновленном издании авторы Нил Форд, Ребекка Парсонс, Патрик Куа и Прамод Садаладж приводят реальные примеры, соответствующие потребностям современной разработки ПО.

Эта книга знаменует собой важную веху, обозначающую нынешний уровень понимания проблемы. По мере того как люди начинают осознавать роль ПО в XXI веке, информация о том, как реагировать на изменения, сохраняя достигнутое, становится важнейшим навыком в области создания программного обеспечения.

Мартин Фаулер

SPRINT
book

Марк Ричардс , Нил Форд

**ФУНДАМЕНТАЛЬНЫЙ ПОДХОД
К ПРОГРАММНОЙ АРХИТЕКТУРЕ:
ПАТТЕРНЫ, СВОЙСТВА,
ПРОВЕРЕННЫЕ МЕТОДЫ**



Архитекторы ПО стабильно входят в десятку самых высокооплачиваемых профессий. Но до сих пор не было реального руководства, которое позволило бы разработчикам стать архитекторами. И вот наконец появилась книга, в которой дается всеобъемлющий обзор разнообразных аспектов архитектуры программного обеспечения. Начинающие и уже состоявшиеся архитекторы найдут в ней паттерны архитектур, определения компонентов, приемы построения эволюционных архитектур и множество других тем.

Марк Ричардс и Нил Форд обладают бесценным практическим опытом, профессионально занимаются этой темой, уделяя особое внимание принципам построения архитектуры, применимым ко всем технологическим стекам. Они предлагают современный взгляд на архитектуру ПО с учетом всех нововведений последнего десятилетия.

SPRINT
book



Кент Бек

ЧИСТЫЙ ДИЗАЙН. ПРАКТИКА ЭМПИРИЧЕСКОГО ПРОЕКТИРОВАНИЯ ПО

Грязный код создает проблемы. Чтобы код было проще читать, приходится проводить его очистку, разбивая на части, с которыми удобно работать. Кент Бек, создатель методологии экстремального программирования и первопроходец в области паттернов проектирования, рассказывает нам, где и когда лучше проводить очистку для улучшения кода с учетом общей структуры системы.

Книга не заставляет читателя проводить очистку сразу и целиком, а позволяет протестировать несколько примеров, которые подходят для поставленной задачи. Вы узнаете, как логически разделить на части большую функцию, содержащую множество строк кода. Познакомитесь с теоретическими понятиями программного дизайна: сцеплением, связностью, дисконтированными денежными потоками и вариативностью.

SPRINT
book

Таня Рейли

КАРЬЕРА РАЗРАБОТЧИКА. СТАФФ — КРУЧЕ, ЧЕМ SENIOR



Кем вы видите себя через пять лет? Стандартный вопрос на собеседовании. А что делать, если вы сеньор-разработчик ПО и хотите дальше продвигаться по карьерной лестнице?

Многие годы считалось, что путь успешного разработчика — переход в «управленцы». Но это не единственный вариант: вы можете стать техническим лидером и без подчиненных. Таких специалистов называют стафф-разработчиками, они ведут большие проекты, определяют техническую стратегию и развивают навыки коллег по команде.

Вы научитесь управлять своим временем, овладеете стратегическим мышлением, будете устанавливать стандарты технической работы. Вы узнаете, как быть лидером, не занимая формальную руководящую должность, как планировать и принимать верные технические решения, как поднять уровень людей, которые работают рядом с вами, и при этом самому постоянно расти как специалист в выбранной предметной области.

SPRINT
book



Максимилиано Контъери

РЕЦЕПТЫ ЧИСТОГО КОДА

Для инженеров-разработчиков и архитекторов, имеющих дело с большим объемом сложного кода, важно эффективно его масштабировать и обслуживать. Автор Максимилиано Контъери выходит за рамки концепции чистого кода, показывая, как искать возможности улучшения и оценивать их влияние на код готового программного продукта. Когда речь идет о надежности и развитии, эти методы обеспечивают преимущества, которые со временем окупаются.

Проверенные рецепты на JavaScript, PHP, Python, Java и других языках программирования помогут масштабировать и поддерживать большие системы. В каждом разделе рассматриваются такие фундаментальные понятия, как читаемость кода, связанность, тестируемость, безопасность и расширяемость, а также запахи кода и соответствующие рецепты их устранения.

SPRINT
book

Джеймс Феникс, Майк Тейлор

ПРОМТ-ИНЖИНИРИНГ ДЛЯ GENAI. ПАТТЕРНЫ НАДЕЖНЫХ ЗАПРОСОВ ДЛЯ КАЧЕСТВЕННЫХ РЕЗУЛЬТАТОВ



Стремительное развитие ИИ меняет нашу жизнь, а поспевать за ним становится все труднее. Каждый, кто профессионально использует ИИ, быстро приходит к пониманию, что качество результатов напрямую зависит от качества входных данных. Поэтому нам сейчас не обойтись без промт-инжиниринга (prompt engineering) — набора методов повышения надежности, эффективности и точности моделей ИИ.

Большие языковые (LLM) и диффузионные модели, такие как ChatGPT и DALL-E, постоянно обучаются на общедоступных текстах и изображениях из Интернета. Так что практически любой человек может использовать модели ИИ для решения задач, которые ранее не поддавались автоматизации.

Эта книга поможет вам осваивать и применять на практике модели генеративного ИИ. Необдуманная интеграция LLM и диффузионных моделей в рабочие процессы часто дает весьма спорные результаты. Джеймс Феникс и Майк Тейлор научат вас использовать промт-инжиниринг, чтобы эффективно внедрить ИИ в любой процесс.

SPRINT
book



Стивен Вендел

ДИЗАЙН И ПОВЕДЕНИЕ ПОЛЬЗОВАТЕЛЕЙ. ПРИМЕНЕНИЕ ПСИХОЛОГИИ И ПОВЕДЕНЧЕСКОЙ ЭКОНОМИКИ В РАЗРАБОТКЕ И UX

Разработчики и дизайнеры надеются, что их продукты станут такими же незаменимыми и востребованными, как многие популярные приложения. Стивен Вендел — глава отдела поведенческих наук в Morningstar — шаг за шагом знакомит с процессом внедрения методов поведенческого подхода в разработку продуктов. Продакт-менеджеры, дизайнеры и аналитики данных узнают простой и эффективный подход к определению целевых пользователей и моделей поведения, созданию продукта и оценке его эффективности. Книга предлагает понимание того, как лучше всего проектировать продукт, что заставляет пользователей совершать определенные действия и как изменить их поведение.

SPRINT
book